

Medii de proiectare și programare

2024-2025

Conținut

- Build automation tool (Gradle)
- Conectarea la baze de date relaționale
- Inversion of Control - Spring
- Aplicații client-server - Șablonul de proiectare Proxy
- Introducere în apelul metodelor la distanță (Remote Procedure Call) - **Remoting/WCF, RMI, Spring Remoting**
- Enterprise Application Integration - Protocol buffers, gRPC, Thrift, ActiveMQ/RabbitMQ
- Object Relational Mapping (Strategii, Hibernate, Entity Framework)
- REST Services
- Dezvoltarea aplicațiilor web folosind frameworkuri (React)
- Web sockets
- Securitate Web - Acces bazat pe roluri (JWT Token)

Bibliografie

- Joseph Albahari, *C# 10.0 in a Nutshell*, Sixth Edition, O'Reilley, 2022.
- Craig Larman, *Applying UML and Design Patterns: An Introduction to OO Analysis and Design and Unified Process*, Berlin, Prentice Hall, 2002.
- Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.
- Hohpe, G., Woolf, B., *Enterprise integration patterns*, Addison-Wesley, 2003.
- ***, Microsoft Developer Network, Microsoft Inc., <http://msdn.microsoft.com/>
- ***, The Java Tutorials, Oracle Java Documentation, Inc. <https://docs.oracle.com/javase/tutorial/>
- Craig Walls, *Spring in Action*, 4th Edition, Ed. O'Reilley, 2015.
- Documentație Spring <http://spring.io/projects>
- Alte tutoriale

Evaluare

- Examen (**NE**) - **60**%
- Teme în timpul laboratorului (**TL**) - **10**%
- Laboratoare (**LB**) - **30** %
- Nota finală **$NF=0.6*NE+0.1*TL+0.3*LB$**
- Condiții pentru promovare:
 - **$NE \geq 4.5$, $LB \geq 4.5$**
 - **$NF \geq 4.5$**

Laborator

- Asignarea și proiectarea aplicației. Studenții trebuie să proiecteze și să dezvolte o aplicație **client-server**.
- Configurarea aplicației folosind *Gradle*, IoC.
- Proiectarea și implementarea persistenței (baze de date relaționale, ORM)
- Proiectarea și implementarea serviciilor (Șablonul *Proxy*).
- Enterprise Application Integration (*Protobuff/ Thrift/ gRPC*)
- Servicii REST
- Aplicații web (web sockets / securitate).

Notare laborator

- Fiecare temă de laborator are un termen de predare.
- Predare completă a temei la termen: **nota 10.**
- Pentru fiecare săptămână întârziere în care nu s-a predat nimic din temă: **penalizare 3 puncte/săpt.**
- Pentru fiecare săptămână întârziere în care s-a predat ceva (este incompletă, erori): **penalizare 1 punct/săpt**, până la predarea completă a temei.
- Tema nepredată: **nota 0.**
- Tema copiată: **nota 0.**



Titlu proiect: *Azi Student, Mâine Antreprenor!*

Acronim proiect: ASMA

Cod proiect: POCU/379/6/21/123894

Beneficiar: Universitatea Babeș-Bolyai din Cluj-Napoca

Proiect cofinanțat din FSE prin Programul Operațional Capital Uman 2014-2020

Curs 1

2024-2025

Curs 1

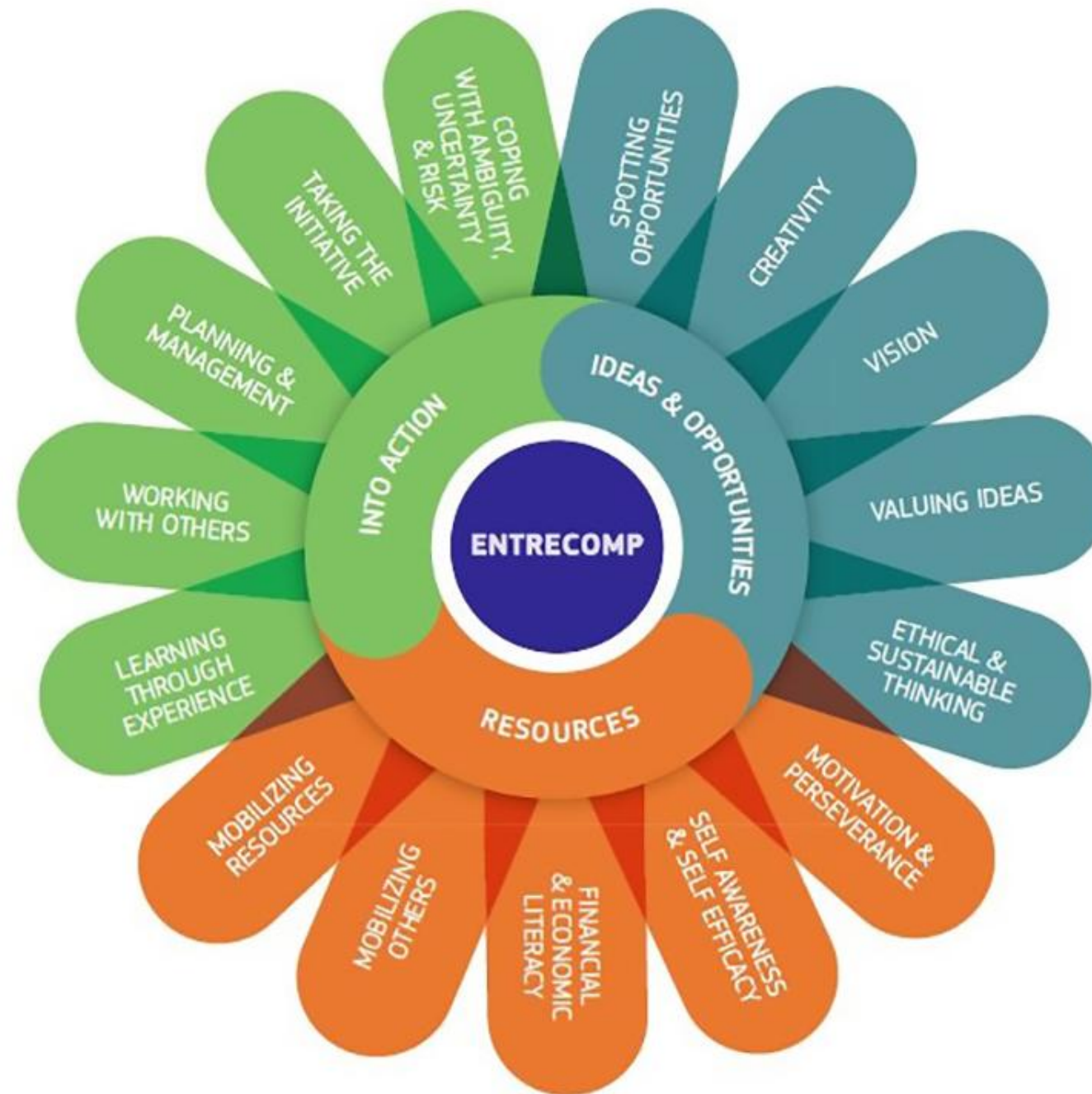
- Competențe antreprenoriale (informatică)
- Instrumente pentru construire automată
 - Gradle

Competențe antreprenoriale



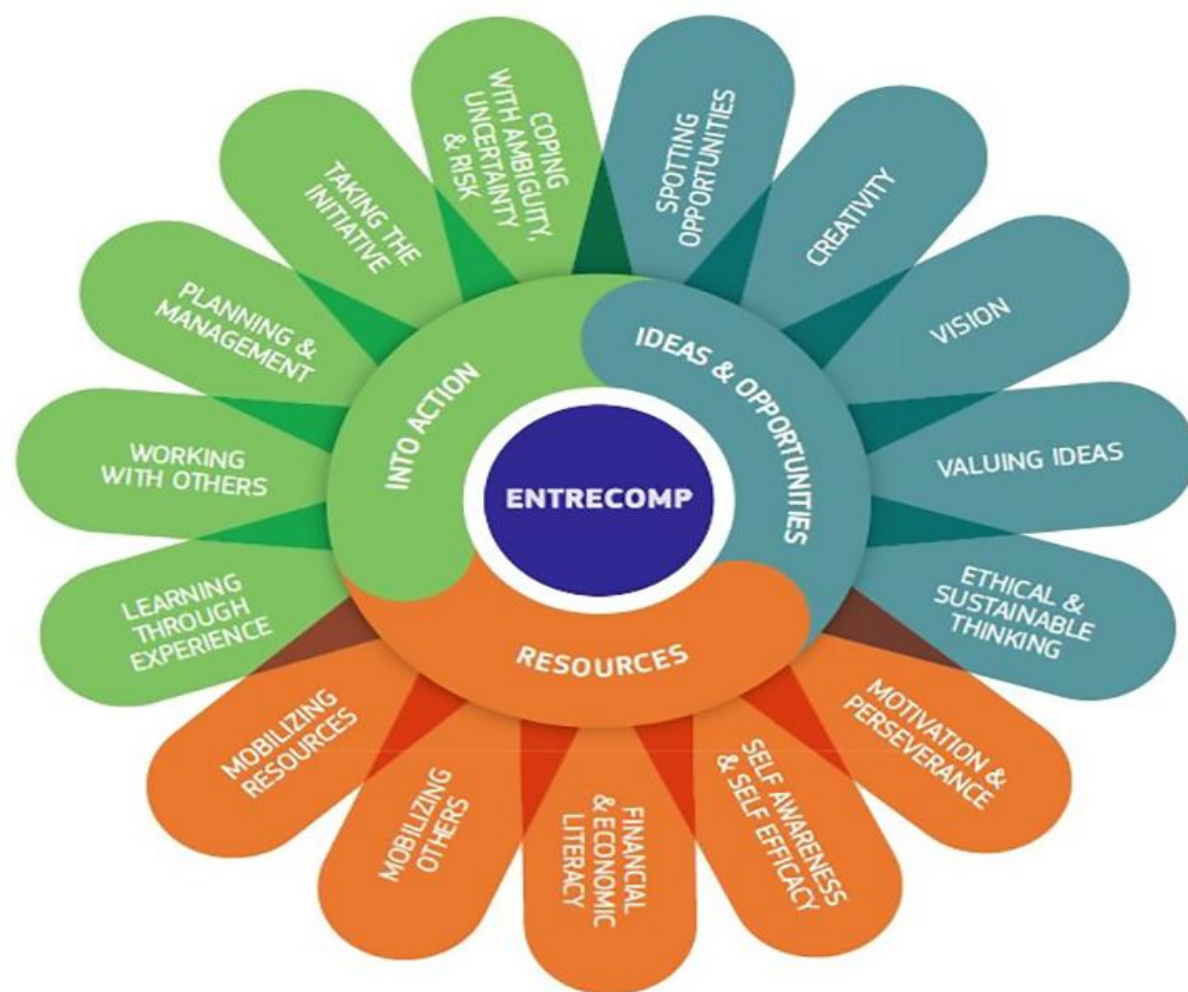
Key competences for lifelong learning, European Commission, Directorate-General for Education, Youth, Sport and Culture, 2019

Competențe antreprenoriale



EntreComp -The European Entrepreneurship Competence Framework, 2018

Informatică - competențe antreprenoriale



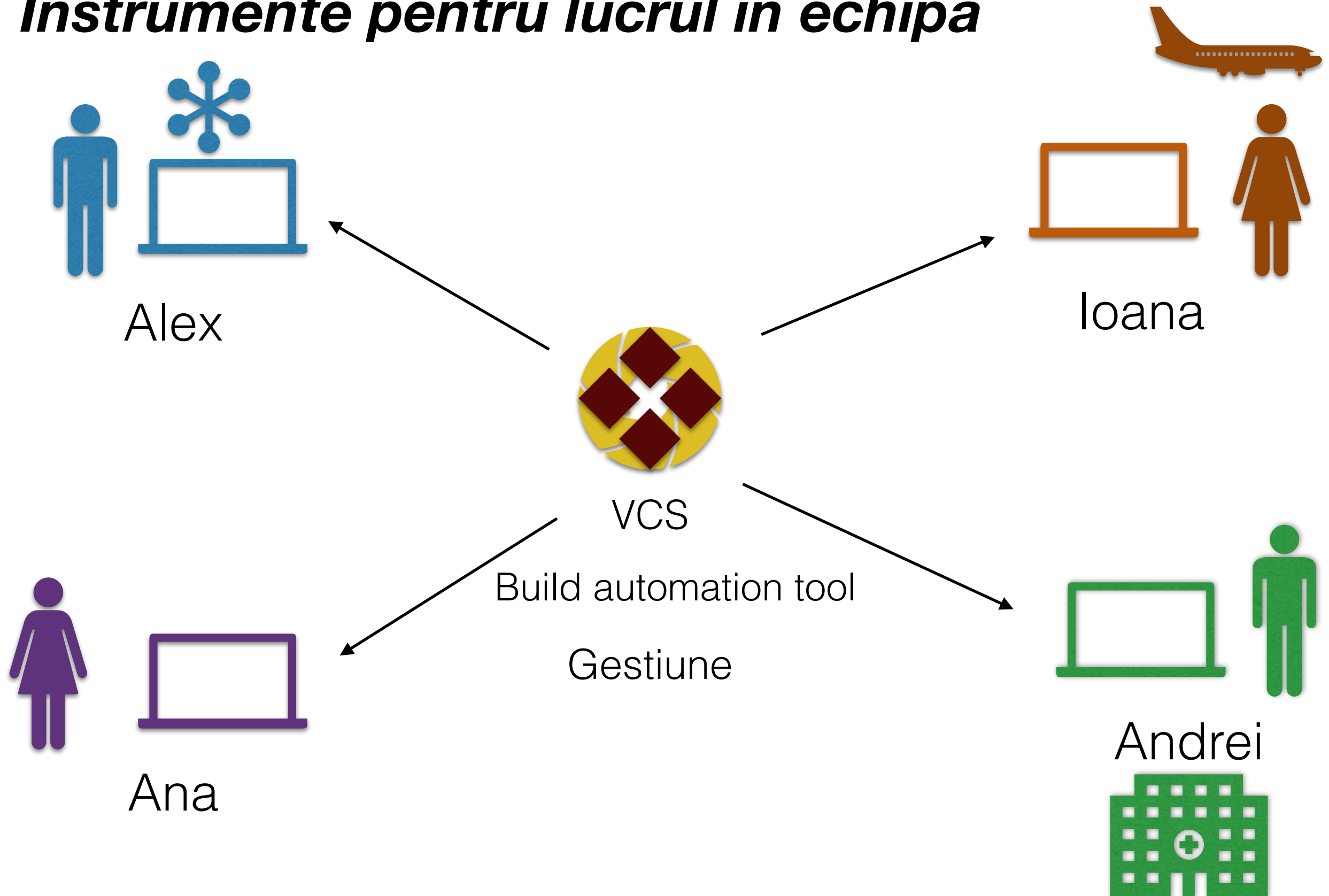
- Lucrul în echipă
- Învățarea prin experiență
- Creativitate
- Identificarea de oportunități

Proiect semestrial facultativ!

Instrumente pentru lucrul în echipă

- Gestiuinea proiectelor soft:
 - Jira, Trello, Atlassian, Zoho, etc.
- Controlul versiunilor și colaborare:
 - GitHub, Git, GitLab, Atlassian Git, BitBucket, etc.
- Construire automată:
 - Gradle, Maven, Jenkins, Travis CI, etc.
- Jurnalizare:
 - Log4j2, SLF4J, Logger.NET, NLog, etc.

Instrumente pentru lucrul în echipă



Instrumente pentru construire automată*

- **Procesul automatizat** corespunzător **construirii** unui sistem soft și a proceselor asociate (compilarea codului sursă, rularea testelor automate, împachetarea codului binar, etc).
 - Makefile
- Categori:
 - a. **Utilitare** pentru construire automată:
 - Generează artefactele corespunzătoare construirii în timpul compilării și/sau link-editării codului.
 - Make, MS build, Ant, **Gradle**, Maven etc.
 - b. **Servere** pentru construire automată:
 - Sisteme soft care execută/rulează utilitarele de construire automată la perioade de timp predefinite sau în momentul apariției anumitor evenimente.
 - TeamCity, Jenkins, CruiseControl, etc.

*Eng. *Build automation tools*

Utilitare pentru construire automată

Avantaje

- Permit **automatizarea** sarcinilor simple și repetabile.
- Permit **atingerea** unui scop/obiectiv prin execuția fiecărei sarcini dintr-un set de sarcini specifice în ordinea corectă.

Îmbunătățesc
calitatea produselor

Accelerează
compilarea și link-
editarea

Elimină sarcinile
redundante

Elimină
dependențele de
angajați 'cheie'

Oferă un **jurnal** al
construcțiilor ce
pot fi folosite când
apar **probleme**

Reduc numărul
construcțiilor
eșuate

Economisesc
timp și bani



Gradle Build Tool

- Gradle Build Tool, <https://gradle.org/>
- Guides <https://gradle.org/guides/#getting-started>
- Alte tutoriale ...

**Permite gestiunea
avansată a
construirii
aplicațiilor**

**Oferă suport pentru
configurarea și
descărcarea
automată a
bibliotecilor și a altor
dependențe**

**Permite
refolosirea
artefactelor
construite folosind
alte utilitare**



Gradle Build Tool

**Oferă suport pentru
depozitarele* Maven și
Ivy pentru refacerea
dependențelor**

**Oferă suport pentru
construiri multi-
proiect și multi-
artefacte**

*eng. *repositories*



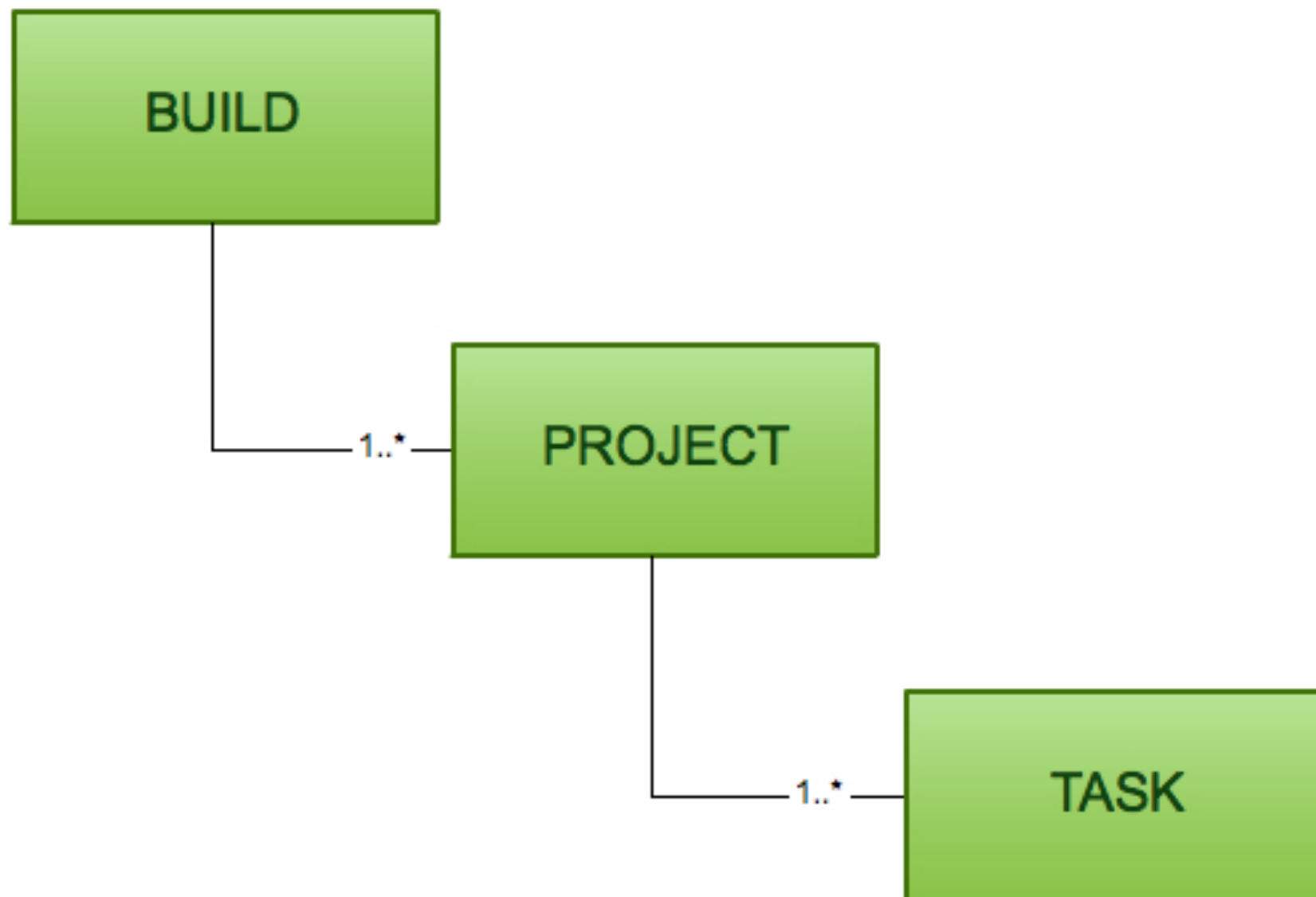
Gradle Build Tool

- Concepte de bază:
 - proiect (eng. *project*)
 - sarcini (eng. *tasks*).
- ❖ Un proiect este:
 - ceva ce se dorește *a se construi* (ex. un sistem soft, un fișier jar),
 - ceva ce se dorește *a se face* (instalarea unui aplicații pentru folosirea de către utilizatori).
- ❖ Un proiect are asociate una sau mai multe *sarcini*.
- ❖ O sarcină este o *unitate de lucru* care este efectuată pentru construirea automată:
 - compilarea unui proiect
 - rularea testelor automate
 - ...



Gradle Build Tool

- Fiecare construire Gradle conține unul sau mai multe proiecte.





Gradle Build Tool

- O construire Gradle (cu Groovy) se configurează folosind fişierele:
 - **build.gradle** (*Gradle build script*) - specifică un proiect şi sarcinile sale asociate.
 - **gradle.properties** (*Gradle properties file*) - este folosit pentru configurarea proprietăţilor construirii.
 - **settings.gradle** (*Gradle Settings file*) - opţional la construirile ce conţin un singur proiect.
 - Dacă construirea este formată din mai multe proiecte Gradle, fişierul este obligatoriu şi descrie proiectele conţinute.
 - Fiecare construire multi-proiect trebuie să aibă un fişier **settings.gradle** în directorul rădăcină al proiectului.



Gradle Build Tool

- **Exemplu fișier build.gradle**

```
plugins{
    id 'java'
}
repositories {
    //pentru rezolvarea dependențelor: Maven, JCenter, Ivy
    mavenCentral()
    //jcenter()
}
dependencies {
    //format GAV Grup:Artefact:Versiune
    compile 'com.google.guava:guava:20.0' //versiuni vechi
    testCompile group: 'junit', name: 'junit', version: '4.+' //versiuni vechi
    //alte dependențe
    //versiuni noi Gradle
    implementation 'com.google.guava:guava:20.0'
    testImplementation group: 'junit', name: 'junit', version: '4.+'
}
```



Gradle Build Tool

- settings.gradle

```
rootProject.name = 'NumeProiect'
```



Gradle Build Tool

Pluginuri

Toate caracteristicile/facilitățile utile unui proiect sunt furnizate de pluginuri.

Pot să **ofere** o
configurație implicită
la sarcinile adăugate

Pot să **adauge noi**
dependențe
proiectului

Pot să **adauge**
sarcini noi la proiect

Pot să **adauge noi**
proprietăți care sunt
folosite pentru a redefini
configurația implicită a
proiectului



Gradle Build Tool

- Un plugin poate fi adăugat unui proiect folosind numele sau tipul acestuia:
- Folosind numele: în fișierul `build.gradle`:

```
apply plugin: 'foo' **
```

- Folosind tipul: în fișierul `build.gradle`:

```
apply plugin: 'com.bar.foo' **
```

******versiuni mai vechi Gradle



Gradle Build Tool

- Blocul plugins (versiuni mai **noi** Gradle):

```
plugins {
```

```
    id 'java'
```

```
    id 'application'
```

```
}
```

- Se poate adăuga și versiunea unui plugin

```
id 'org.openjfx.javafxplugin' version '0.0.7'
```



Gradle Build Tool

- Pluginuri Gradle standard:
 - **java**: adaugă sarcini pentru compilarea, testarea și împachetarea unui proiect Java. Este un plugin de bază pentru alte pluginuri.
 - **groovy**: adaugă suport pentru proiecte Groovy.
 - **cpp**: adaugă sarcini pentru compilarea codului unui proiect C++.
 - **application**: adaugă sarcini pentru rularea și împachetarea unui proiect Java ca și aplicație executabilă.
 - **distribution**: adaugă suport pentru construirea distribuțiilor de tip ZIP și TAR.
 - **signing**: adaugă abilitatea de a semna digital fișierele și artefactelor construite.
 - etc.



Gradle Build Tool

- Structura unui proiect Java:
 - *src/main/java* - directorul corespunzător codului sursă.
 - *src/main/resources* - directorul corespunzător resurselor folosite în proiect (fișiere de proprietăți, imagini, fxml etc.).
 - *src/test/java* - directorul corespunzător testelor automate.
 - *src/test/resources* - directorul corespunzător resurselor necesare testelor automate.
 - *build* - directorul ce conține toate artefactele construite folosind Gradle.
 - *classes* - conține fișierele *.class*.
 - *libs* - conține fișierele *jar*, *war*, *ear* create folosind Gradle.
 - etc.



Gradle Build Tool

- Crearea unui proiect Java

În fișierul `build.gradle`:

```
apply plugin: 'java'
```

```
plugins{  
    id 'java'  
}
```

- Crearea unui proiect Java cu structura implicită***:

```
gradle init --type 'java-library'
```

```
gradle init --type 'java-application'
```

***în directorul rădăcină al proiectului



Gradle Build Tool

- Sarcinile asociate unui proiect Java (plugin java):
 - *assemble* - compilează codul sursă al aplicației și creează fișierul jar. Nu rulează testele automate.
 - *build* - construiește toate artefactele asociate proiectului.
 - *clean* - șterge directorul *build* asociat proiectului.
 - *compileJava* - compilează codul sursă al aplicației.
 - etc.



Gradle Build Tool

- **gradle tasks** - afișează lista completă a sarcinilor ce pot fi executate pentru un proiect și descrierea acestora:
 - *assemble* - Assembles the outputs of this project.
 - *build* - Assembles and tests this project.
 - *buildDependents* - Assembles and tests this project and all projects that depend on it.
 - *buildNeeded* - Assembles and tests this project and all projects it depends on.
 - *classes* - Assembles main classes.
 - *clean* - Deletes the build directory.
 - *jar* - Assembles a jar archive containing the main classes.
 - *testClasses* - Assembles test classes.
 - *check* - Runs all checks.
 - *test* - Runs the unit tests.
 - etc.



Gradle Build Tool

- Împachetarea aplicației (obținerea artefactelor):

- *gradle assemble*

:compileJava

:processResources

:classes

:jar

:assemble



Gradle Build Tool

- Împachetarea aplicației (obținerea artefactelor și rularea testelor automate):

- *gradle build*

:compileJava

:processResources

:classes

:jar

:assemble

:compileTestJava

:processTestResources

:testClasses

:test

:check

:build

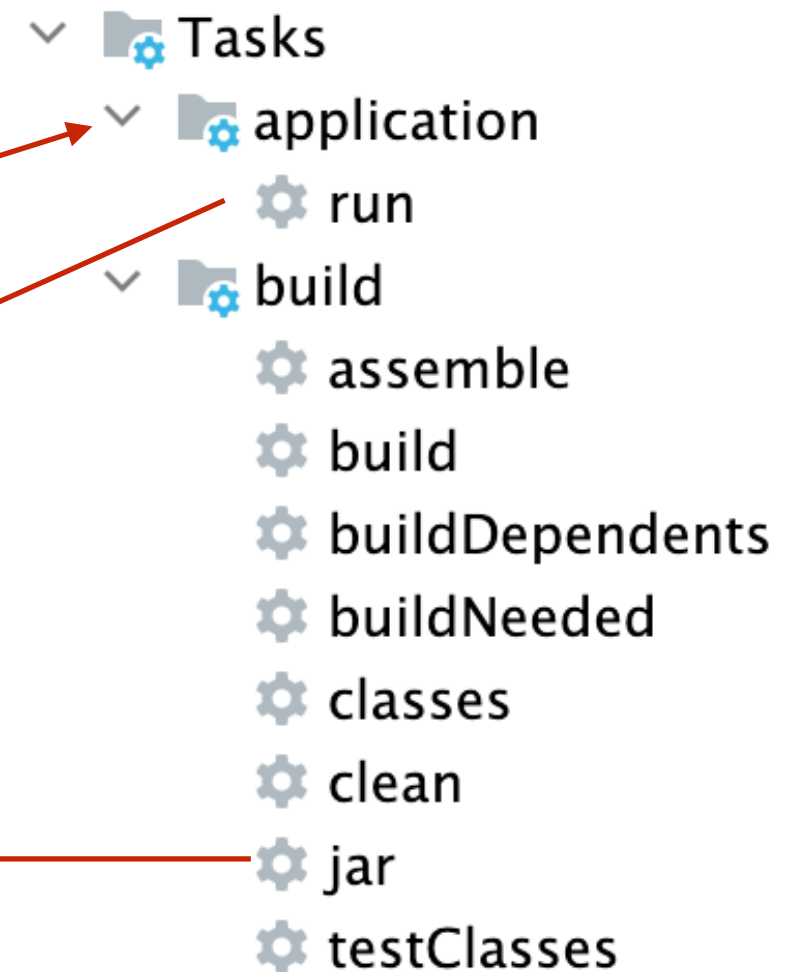


Gradle Build Tool

- Fișiere *jar* executabile:

- build.gradle*

```
plugins{
    id 'java'
    id 'application'
}
application{
    mainClass='NumeClasaCuMain'
}
jar {
    manifest {
        attributes('Main-Class':'NumeClasaCuMain')
    }
    from {
        configurations.runtimeClasspath.collect { it.isDirectory() ? it : zipTree(it) }
    }
    duplicatesStrategy = DuplicatesStrategy.EXCLUDE
}
```



Medii de proiectare și programare

2024-2025

Curs 2

Conținut curs 2

- Gradle (cont. curs 1)
- Jurnalizare
- SQLite, SQLiteStudio, MySQL
- Accesul la baze de date relaționale
 - Java: JDBC
 - C#: ADO.NET - curs 3
- Configurarea (Java properties, C# app.config - curs 3)
- Ierarhia repository - curs 3



Gradle Build Tool

- Structura unui proiect Java:
 - *src/main/java* - directorul corespunzător codului sursă.
 - *src/main/resources* - directorul corespunzător resurselor folosite în proiect (fișiere de proprietăți, imagini, fxml etc.).
 - *src/test/java* - directorul corespunzător testelor automate.
 - *src/test/resources* - directorul corespunzător resurselor necesare testelor automate.
 - *build* - directorul ce conține toate artefactele construite folosind Gradle.
 - *classes* - conține fișierele *.class*.
 - *libs* - conține fișierele *jar*, *war*, *ear* create folosind Gradle.
 - etc.



Gradle Build Tool

- Crearea unui proiect Java

În fișierul `build.gradle`:

```
apply plugin: 'java'
```

```
plugins{  
    id 'java'  
}
```

- Crearea unui proiect Java cu structura implicită***:

```
gradle init --type 'java-library'
```

```
gradle init --type 'java-application'
```

***în directorul rădăcină al proiectului



Gradle Build Tool

- Sarcinile asociate unui proiect Java (plugin java):
 - *assemble* - compilează codul sursă al aplicației și creează fișierul jar. Nu rulează testele automate.
 - *build* - construiește toate artefactele asociate proiectului.
 - *clean* - șterge directorul *build* asociat proiectului.
 - *compileJava* - compilează codul sursă al aplicației.
 - etc.



Gradle Build Tool

- **gradle tasks** - afișează lista completă a sarcinilor ce pot fi executate pentru un proiect și descrierea acestora:
 - *assemble* - Assembles the outputs of this project.
 - *build* - Assembles and tests this project.
 - *buildDependents* - Assembles and tests this project and all projects that depend on it.
 - *buildNeeded* - Assembles and tests this project and all projects it depends on.
 - *classes* - Assembles main classes.
 - *clean* - Deletes the build directory.
 - *jar* - Assembles a jar archive containing the main classes.
 - *testClasses* - Assembles test classes.
 - *check* - Runs all checks.
 - *test* - Runs the unit tests.
 - etc.



Gradle Build Tool

- Împachetarea aplicației (obținerea artefactelor):

- *gradle assemble*

- :compileJava*

- :processResources*

- :classes*

- :jar*

- :assemble*



Gradle Build Tool

- Împachetarea aplicației (obținerea artefactelor și rularea testelor automate):

- *gradle build*

:compileJava

:processResources

:classes

:jar

:assemble

:compileTestJava

:processTestResources

:testClasses

:test

:check

:build

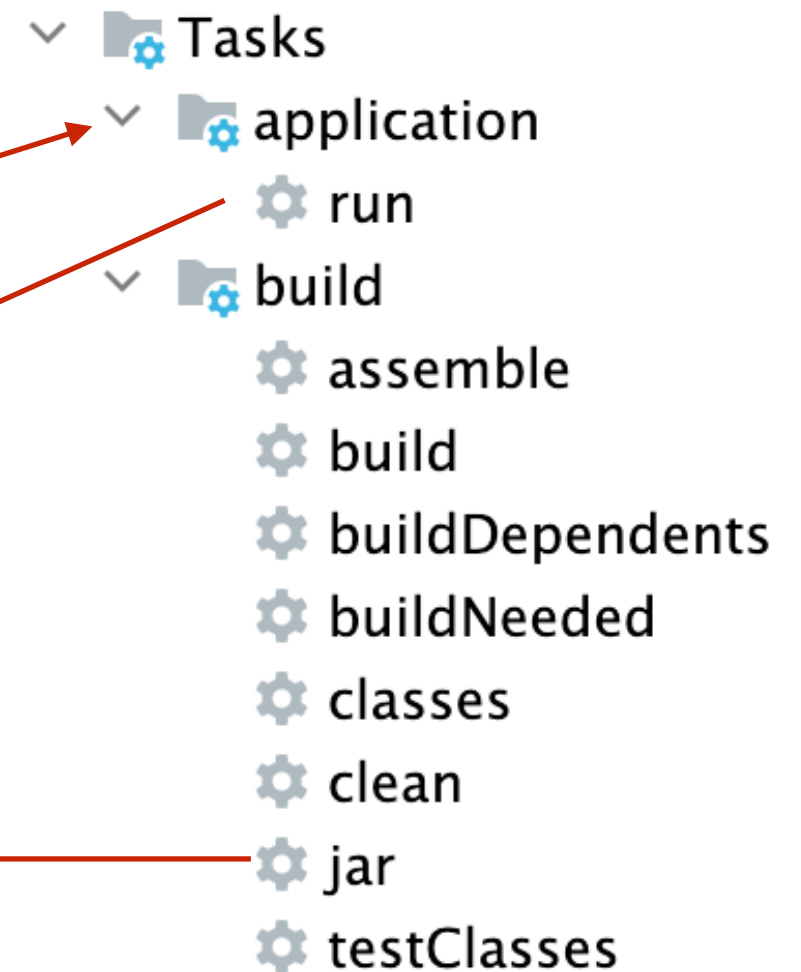


Gradle Build Tool

- Fișiere *jar* executabile:

- build.gradle*

```
plugins{
    id 'java'
    id 'application'
}
application{
    mainClass='NumeClasaCuMain'
}
jar {
    manifest {
        attributes('Main-Class':'NumeClasaCuMain')
    }
    from {
        configurations.runtimeClasspath.collect { it.isDirectory() ? it : zipTree(it) }
    }
    duplicatesStrategy = DuplicatesStrategy.EXCLUDE
}
```





Gradle Build Tool

- Proiecte multiple:
 - Fiecare proiect (subproiect) are aceeași structură - corespunzătoare proiectelor Gradle Java.
 - Fiecare proiect (subproiect) va conține fișierul `build.gradle` propriu, cu configurările specifice.
 - Proiectul rădăcină (root) conține obligatoriu și fișierul `settings.gradle`:
 - `include 'A'`
 - `include 'B'`



Gradle Build Tool

- Dependente între (sub)proiecte: Subproiectul B depinde de subproiectul A:
- `build.gradle` corespunzător subproiectului B:

```
dependencies {  
    implementation project(':A')  
}
```




Gradle Build Tool

- Proiectul A: `build.gradle`

```
plugins{  
    id 'java'  
}  
repositories {  
    mavenCentral()  
}  
dependencies {  
    implementation 'com.google.guava:guava:20.0'  
    testImplementation 'junit:junit:4.11'  
}
```



Gradle Build Tool

- Proiectul B: `build.gradle`

```
plugins{  
    id 'java'  
    id 'application'  
}  
repositories {  
    mavenCentral()  
}  
application{  
    mainClass='StartApp'  
}  
dependencies {  
    testImplementation 'junit:junit:4.11'  
    implementation project(':A')  
}
```



Gradle Build Tool

- Project Root: `build.gradle`

```
allprojects {  
    plugins{  
        id 'java'  
    }  
    repositories {  
        mavenCentral()  
    }  
    dependencies {  
        testImplementation 'junit:junit:4.11'  
    }  
}
```



Gradle Build Tool

- Proiectul A: `build.gradle` modificat

```
dependencies {  
    implementation 'com.google.guava:guava:20.0'  
}
```

- Proiectul B: `build.gradle` modificat

```
plugins{  
    id 'application'  
}  
application{  
    mainClass='StartApp'  
}  
dependencies {  
    implementation project(':A')  
}
```



Gradle Build Tool

- Proiectul Root: `build.gradle`

```
subprojects {  
    //Configurări comune tuturor subproiectelor  
}  
  
project(':A') {  
    //Configurări specifice proiectului A  
}  
  
project(':B') {  
    //Configurări specifice proiectului B  
}
```

Instrumente pentru jurnalizare

- Un ***instrument pentru jurnalizare*** permite programatorilor să înregistreze diferite tipuri de mesaje din codul sursă cu diverse scopuri: depanare, analiza ulterioară, etc.
- Majoritatea instrumentelor definesc diferite nivele pentru mesaje: *debug*, *warning*, *error*, *information*, *sever*, etc.
- Configurarea instrumentelor se face folosind fișiere text de configurare și pot fi oprite sau pornite la rulare.
- Apache Log4j, Logging SDK (inclus JDK), slf4j, etc.



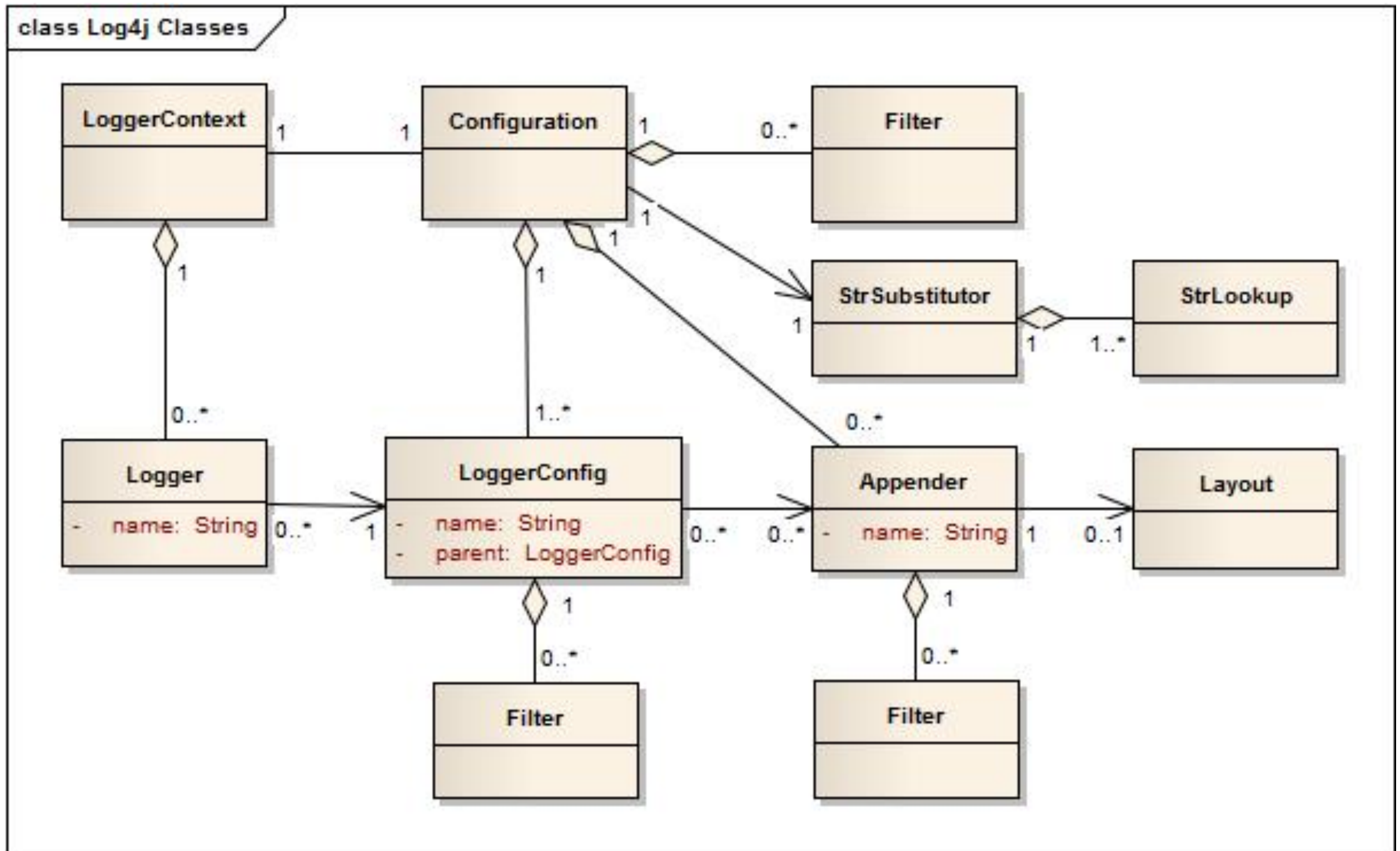
- Proiect open source dezvoltat de Apache Foundation.

<http://logging.apache.org/log4j/2.0/>

- Log4j 2 are 3 componente principale:
 - *loggers*,
 - *appenders* (pentru stocare),
 - *layouts* (pentru formatare).

<https://logging.apache.org/log4j/2.x/index.html>

Arhitectura





- Aplicațiile care folosesc Log4j 2 cer o referință către un obiect de tip *Logger* cu un anumit nume de la *LogManager*.
- *LogManager* va localiza obiectul *LoggerContext* corespunzător numelui și va obține referința către obiectul *Logger* de la el.
- Dacă obiectul de tip *Logger* corespunzător încă nu a fost creat, se va crea unul nou și va fi asociat cu un obiect de tip *LoggerConfig* care fie:
 - are același nume ca și *Logger*,
 - are același nume ca și pachetul părinte,
 - este rădăcina *LoggerConfig*.
- Obiectele de tip *LoggerConfig* sunt create folosind declarațiile din fișierul de configurare.
- Fiecărui *LoggerConfig* îi sunt asociate unul sau mai multe obiecte de tip *Appender*.



- Fișierul de configurare (XML, JSON, proprietăți Java, yaml)
- Dacă nu este configurat, log4j 2 afișează doar mesajele de tip *error* la consolă

Exemplu fișier de configurare în format XML: ***log4j2.xml***

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="TRACE">
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/>
    </Console>
  </Appenders>
  <Loggers>
    <Root level="TRACE">
      <AppenderRef ref="Console"/>
    </Root>
  </Loggers>
</Configuration>
```



- Log level - fiecare mesaj are asociat un anumit nivel
- TRACE, DEBUG, INFO, WARN, ERROR și FATAL

Event Level	LoggerConfig Level						
	TRACE	DEBUG	INFO	WARN	ERROR	FATAL	
ALL	YES	YES	YES	YES	YES	YES	NO
TRACE	YES	NO	NO	NO	NO	NO	NO
DEBUG	YES	YES	NO	NO	NO	NO	NO
INFO	YES	YES	YES	NO	NO	NO	NO
WARN	YES	YES	YES	YES	NO	NO	NO
ERROR	YES	YES	YES	YES	YES	NO	NO
FATAL	YES	YES	YES	YES	YES	YES	NO
OFF	NO	NO	NO	NO	NO	NO	NO



- Numele asociat unui logger: structura ierarhică asemănătoare structurii pachetelor Java.

```
public class LogTest {  
    //a  
    private static final Logger logger = LogManager.getLogger(LogTest.class);  
    //b  
    private static final Logger logger =  
        LogManager.getLogger(LogTest.class.getName());  
    //c  
    private static final Logger logger = LogManager.getLogger();  
}
```



- Clasa **Logger** conține metode ce permit urmărirea fluxului execuției unei aplicații.
 - **entry(...)** - 0 ..4 parametrii
 - **traceEntry(String, ...)** - String și o lista variabilă de parametri
 - **exit(...), traceExit(String, ...)**
 - **throwing (...)** - când se aruncă o excepție
 - **catching(...)** când se prinde o excepție
 - **trace(...)**
 - **error(...)**
 - **log(...)**
 - etc.
- **Exemplu**



- Salvarea mesajelor: fișier, consolă, baze de date, etc.

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<Configuration status="TRACE">
```

```
<Appenders>
```

```
<File name="FisierLog" fileName="logs/app.log">
```

```
<PatternLayout pattern="%d{DATE} [%t] %class{36} %L %M - %msg%xEx%n"/>
```

```
</File>
```

```
</Appenders>
```

```
<Loggers>
```

```
<Root level="TRACE">
```

```
<AppenderRef ref="FisierLog"/>
```

```
</Root>
```

```
</Loggers>
```

```
</Configuration>
```

SGBD

- Sqlite
- SQLiteStudio
- MySQL/MariaDB



- <https://www.sqlite.org/>
- Bază de date relațională
- Nu necesită configurări adiționale
- Nu necesită pornirea unui proces separat
- Toate informațiile sunt păstrate într-un singur fișier
- Formatul fișierului este independent de platformă
- Open source, gratuit.

```
sqlite_dir> sqlite3
```




- <https://sqlitestudio.pl/>
- Sistem de gestiune a unei baze de date Sqlite
- Interfață grafică ușor de folosit
- Independent de platformă
- Gratuit
- Open source



- <https://www.mysql.com/> <https://mariadb.com/>
- Sistem de gestiune a bazelor de date relaționale
- Rapid, scalabil, ușor de folosit
- Sistem de tip client-server/ embedded
- Gratuit
- Open source (MariaDB)

JDBC

- Java Database Connectivity (JDBC) API conține o mulțime de clase ce asigură accesul la date.
- Se pot accesa orice surse de date: baze de date relaționale, foi de calcul (*spreadsheets*) sau fișiere.
- JDBC oferă și o serie de interfețe ce pot fi folosite pentru construirea instrumentelor specializate.
- Pachete:
 - `java.sql` conține clase și interfețe pentru accesarea și procesarea datelor stocate într-o sursă de date (de obicei bază de date relațională).
 - `javax.sql` - adaugă noi funcționalități pentru partea de server.

Stabilirea unei conexiuni

- Conectarea se poate face în două moduri:
 - Clasa **DriverManager**: Conexiunea se creează folosind un URL specific.
 - Necesita încărcarea unui driver specific bazei de date (JDBC<4).
 - Incepând cu JDBC 4.0 nu mai este necesară încărcarea driverului.
 - Interfața **DataSource**: Este recomandată folosirea interfeței pentru aplicații complexe, deoarece permite configurarea sursei de date într-un mod transparent.
- Stabilirea unei conexiuni se realizează astfel:
 - Încărcarea driverului (versiuni JDBC <4.0)

Class.forName(<DriverClassName>) ;

- **Class.forName** creează automat o instanță a driverului și o înregistrează la **DriverManager**.
- Nu este necesară crearea unei instanțe a clasei.
- Crearea conexiunii.

Crearea unei conexiuni

- Folosind clasa **DriverManager** :
 - Colaborează cu interfața Driver pentru gestiunea driverelor disponibile unui client JDBC.
 - Când clientul cere o conexiune și furnizează un URL, clasa DriverManager este responsabilă cu găsirea driverului care recunoaște URL și cu folosirea lui pentru a se conecta la sursa de date.
 - Sintaxa URL-ului corespunzător unei conexiuni este:

`jdbc:subprotocol:<numeBazaDate>[listaProprietati]`

```
Connection conn = DriverManager.getConnection("jdbc:sqlite:users.db");
```

```
String url = "jdbc:mysql:Test";
```

```
String url = "jdbc:mariadb://localhost:3306/Test"
```

```
Connection conn = DriverManager.getConnection(url, <user>, <passwd>);
```

Crearea unei conexiuni

- Folosind interfața **DataSource**:

```
InitialContext ic = new InitialContext()
```

```
//a)
```

```
DataSource ds = ic.lookup("java:comp/env/jdbc/myDB");
```

```
Connection con = ds.getConnection();
```

```
//b)
```

```
DataSource ds =
```

```
    (DataSource)org.apache.derby.jdbc.ClientDataSource()
```

```
ds.setPort(1527);
```

```
ds.setHost("localhost");
```

```
ds.setUser("APP")
```

```
ds.setPassword("APP");
```

```
Connection con = ds.getConnection();
```

Clasa Connection

- Reprezintă o sesiune cu o bază de date specifică.
- Orice instrucțiune SQL este executată și rezultatele sunt transmise folosind contextul unei conexiuni.
- Metode:
 - `close()` , `isClosed():boolean`
 - `createStatement():Statement` //overloaded
 - `prepareCall():CallableStatement` //overloaded
 - `prepareStatement():PreparedStatement` //overloaded
 - `rollback()`
 - `setAutoCommit(boolean)` //tranzactii
 - `getAutoCommit():boolean`
 - `commit()`

Medii de proiectare și programare

2024-2025

Curs 3

Conținut curs 3

- Accesul la baze de date relaționale (Java, configurare)
- Ierarhia repository
- ADO.NET
 - Configurarea(C# app.config)
- Adnotări

Clasa Connection

- Reprezintă o sesiune cu o bază de date specifică.
- Orice instrucțiune SQL este executată și rezultatele sunt transmise folosind contextul unei conexiuni.
- Metode:
 - `close()` , `isClosed():boolean`
 - `createStatement():Statement` //overloaded
 - `prepareCall():CallableStatement` //overloaded
 - `prepareStatement():PreparedStatement` //overloaded
 - `rollback()`
 - `setAutoCommit(boolean)` //tranzactii
 - `getAutoCommit():boolean`
 - `commit()`

Clasa Statement

- Se folosește pentru executarea unei instrucțiuni SQL și pentru transmiterea rezultatului.
- Metode:
 - `execute(sql:String, ...):boolean` //pentru orice instructiune SQL
 - `getResultSet():ResultSet`
 - `getUpdateCount():int`
 - `executeQuery(sql:String, ...):ResultSet` //pentru SELECT
 - `executeUpdate(sql:String, ...):int` //INSERT, UPDATE, DELETE
 - `cancel()`
 - `close()`

Exemplu Statement – Structura bazei de date



	Field Name	Data Type
🔑	ID	AutoNumber
	title	Text
	authors	Text
	isbn	Text
	year	Number
▶		

Statement exemplu

```
//Conectarea la o baza de date SQLite
Connection conn=DriverManager.getConnection("jdbc:sqlite:/Users/teste/database/
books.db");
//select
try(Statement stmt=conn.createStatement()){
    try(ResultSet rs=stmt.executeQuery("select * from books")){
        ...
    }
}catch(SQLException ex){
    System.err.println(ex.getSQLState());
    System.err.println(ex.getErrorCode());
    System.err.println(ex.getMessage());
}

//update
String upString="update books set isbn='tj234' where isbn='tj237'"
try(Statement stmt=conn.createStatement()){
    int res=stmt.executeUpdate(upString);
    if (res==0) {...} //nu s-a modificat nicio inregistrare
}catch(SQLException ex){...}
```

Statement exemplu

//insert

```
String insert="insert into books (title, authors, isbn, year) values  
('Nuvele', 'Mihai Eminescu', '4567567', 2008)";  
try(Statement stmt=conn.createStatement()) {  
    int res=stmt.executeUpdate(insert);  
    if (res==0){//nu s-a inserat  
    }  
} catch (SQLException e) {  
    System.out.println("Insert error "+e);  
}
```

//delete

```
String delString="delete from books where isbn='tj234'"  
try(Statement stmt=conn.createStatement()){  
    int res=stmt.executeUpdate(delString);  
    if (res==0){} //nu s-a sters nicio inregistrare  
}catch(SQLException ex){  
    //...  
}
```

ResultSet

- Conține o tabelă ce reprezintă rezultatul unei instrucțiuni SELECT.
- Un obiect de tip `ResultSet` conține un cursor care indică linia curentă din tabelă.
- La început cursorul este poziționat înaintea primei linii din tabelă.
- Metoda **`next`** mută cursorul pe următoarea linie din tabelă. Rezultatul returnat este **`false`**, dacă nu mai există linii neparcurse în obiectul `ResultSet`.
- Metoda **`next`** se folosește pentru a parcurge toate liniile din tabelă.
- Se pot configura anumite proprietăți (daca tabela poate fi modificata, modul de parcurgere, etc.) .
- Configurarea se face în momentul apelului metodei de tip **`createStatement(...)`** :

```
Statement stmt = con.createStatement(  
    ResultSet.TYPE_SCROLL_INSENSITIVE,  
    ResultSet.CONCUR_UPDATABLE);  
ResultSet rs = stmt.executeQuery("SELECT name, address FROM users");  
    // rs poate fi iterat, nu va fi notificat de modificari facute  
    //de alti utilizatori ai BD, si poate fi actualizat.
```


ResultSet

- Metode:
 - `absolute(row:int)`
 - `relative(n:int)`
 - `afterLast()`, `beforeFirst()`, `first()`, `last()`, `next():boolean`
 - `getRow():int`
 - `getInt(columnIndex|columnLabel):int`
 - `getFloat(...)`, `getString(...)`, `getObject(...)`, etc
 - `updateInt(columnIndex|columnLabel, newValue)`
 - `updateFloat(...)`, `updateString(...)`, etc
 - `updateRow()`
 - `refreshRow()`
 - `rowDeleted()`, `rowInserted()`, `rowUpdated()`

ResultSet

- Implicit un object de tip **ResultSet** este unidirecțional, cu parcurgerea înainte și nu poate fi modificat (actualizat).

```
try(Statement stmt=conn.createStatement()) {  
    try(ResultSet rs=stmt.executeQuery("select * from books")) {  
        while(rs.next()) {  
            System.out.println("Book "+rs.getString("title")+  
                '+rs.getString("author")+ ' '+rs.getInt("year"));  
        }  
    }  
} catch(SQLException ex) {  
  
    // ...  
}
```

Clasa PreparedStatement

- Unui obiect de tip **PreparedStatement** i se transmite instrucțiunea SQL în momentul creării.
- Instrucțiunea SQL este transmisă sistemului de gestiune a bazei de date (SGBD), unde este compilată.
- Când se execută instrucțiunea asociată unui **PreparedStatement**, SGBD execută direct instrucțiunea SQL fără a o reverifica în prealabil.
- Este mai eficientă decât Statement.
- **Poate să aibă parametrii.** Aceștia sunt **marcați folosind ‘?’**.

```
PreparedStatement preStmt = con.prepareStatement(  
    "select * from books WHERE year=?");
```

- Valoarea unui parametru este transmisă folosind metodele de tip **setXYZ**, unde **xyz** reprezinta tipul parametrului.
- **Pozițiile parametrilor încep de la 1.**

```
preStmt.setInt(1, 2008);  
ResultSet rs=preStmt.executeQuery();
```

Tranzacții

- Implicit, fiecare instrucțiune SQL este tratată ca și o tranzacție și este înregistrată/operată imediat după execuție.
- Comportamentul implicit poate fi modificat folosind metoda `setAutoCommit(false)` din clasa `Connection`.
- Metode:
 - `commit`
 - `rollback`
 - `setSavePoint`

Tranzacții - exemplu

```
con.setAutoCommit(false);  
PreparedStatement updateSales = con.prepareStatement(  
    "UPDATE COFFEES SET SALES = ? WHERE COF_NAME LIKE ?");  
updateSales.setInt(1, 50);  
updateSales.setString(2, "Black");  
updateSales.executeUpdate();  
PreparedStatement updateTotal = con.prepareStatement(  
    "UPDATE COFFEES SET TOTAL = TOTAL + ? WHERE COF_NAME  
    LIKE ?");  
updateTotal.setInt(1, 50);  
updateTotal.setString(2, "Black");  
updateTotal.executeUpdate();  
con.commit();  
con.setAutoCommit(true);
```

Proceduri stocate

- O procedură stocată este un grup de instrucțiuni SQL care formează o unitate logica și îndeplinesc o anumită sarcină.
- Ele sunt folosite pentru a îngloba o serie de operațiuni sau interogări ce trebuie executate pe un server de baze de date.
- De exemplu, operațiunile de pe o bază de date angajat (angajarea, concedierea, promovarea, cautarea) ar putea fi codificate ca proceduri stocate executate în funcție de codul cerere.
- Procedurile stocate pot fi compilate și executate cu diferiți parametri și pot avea orice combinație de intrare, ieșire sau intrare/ieșire.

CallableStatement

- Este folosită pentru executarea procedurilor stocate.
- Tehnologia JDBC API furnizează o sintaxă de apelare a procedurilor stocate independentă de SGBD folosit.
- Sintaxa folosită are două variante:
 - conține un parametru de tip rezultat
 - nu conține un parametru de tip rezultat.
- Dacă se folosește prima varianta, parametrul de tip rezultat trebuie să fie înregistrat ca și parametru de tip OUT. Ceilalți parametrii pot fi folosiți pentru intrare, ieșire sau ambele.
- Parametrii sunt referiți secvențial, folosind numere, primul parametru fiind pe poziția 1.

```
{?= call <procedure-name>[ (<arg1>,<arg2>, ...)]}  
{call <procedure-name>[ (<arg1>,<arg2>, ...)]}
```

```
CallableStatement cs = con.prepareCall("{call SHOW_SUPPLIERS}");  
ResultSet rs = cs.executeQuery();
```

Properties

- Clasa **Properties** (pachetul **java.util**) se folosește pentru a păstra perechi cheie-valoare. Perechile pot fi citite sau salvate dintr-un/într-un flux de date (ex. fișier). Cheia și valoarea sunt de tip String, cheia fiind unică.

```
//exemplu.properties  
tasksFile=tasks.txt  
inputDir=input  
outputDir=output
```

```
//Citirea fișierului cu proprietăți  
Properties props=new Properties();  
try {  
    props.load(new FileInputStream("exemplu.properties"));  
} catch (IOException e) {  
    System.out.println("Eroare: "+e);  
}
```


Properties

- Metode:
 - `getProperty(cheie:String):String`
 - `setProperty(c:String, v:String):Object`
 - `list(PrintWriter)`
 - `load(Reader)`
 - `store(w:Writer, comentarii:String)`

```
Properties props=new Properties();
try {
    props.load(new FileInputStream("exemplu.properties"));
} catch (IOException e) {
    System.out.println("Eroare: "+e);
}
String tasksFile=props.getProperty("tasksFile");
if (tasksFile==null) //proprietatea nu a fost gasita in fisier
    System.out.println("fisier incorect");
...
```


System + Properties

- Metode din clasa System:
 - setProperty(Properties)
 - setProperty(c:String, v:String):String
 - getProperty(String):String
 - ...

```
Properties serverProps=new Properties(System.getProperties());
try {
    serverProps.load(new FileReader("exemplu.properties"));
    System.setProperty(serverProps);
    System.getProperties().list(System.out);
} catch (IOException e) {
    System.out.println("Eroare "+e);
}
String tasksFile=System.getProperty("tasksFile");
```

Exemplu configurare BD (System)

```
//Fisierul bd.properties sau bd.config  
jdbc.url=jdbc:mysql://localhost/mpp  
jdbc.user=test  
jdbc.pass=test
```

```
//cod
```

```
Connection getConnection() {  
    String url=System.getProperty("jdbc.url");  
    String user=System.getProperty("jdbc.user");  
    String pass=System.getProperty("jdbc.pass");  
    Connection con=null;  
    try {  
        con= DriverManager.getConnection(url,user,pass);  
    } catch (SQLException e) {  
        System.out.println("Eroare stabilire conexiune "+e);  
    }  
    return con;  
}
```

Exemplu configurare BD

```
//Fisierul bd.properties sau bd.config  
jdbc.url=jdbc:mysql://localhost/mpp  
jdbc.user=test  
jdbc.pass=test
```

```
//cod
```

```
Connection getConnection(Properties props) {  
    String url=props.getProperty("jdbc.url");  
    String user=props.getProperty("jdbc.user");  
    String pass=props.getProperty("jdbc.pass");  
    Connection con=null;  
    try {  
        con= DriverManager.getConnection(url,user,pass);  
    } catch (SQLException e) {  
        System.out.println("Eroare stabilire conexiune "+e);  
    }  
    return con;  
}
```

Dependențe driver JDBC - Gradle

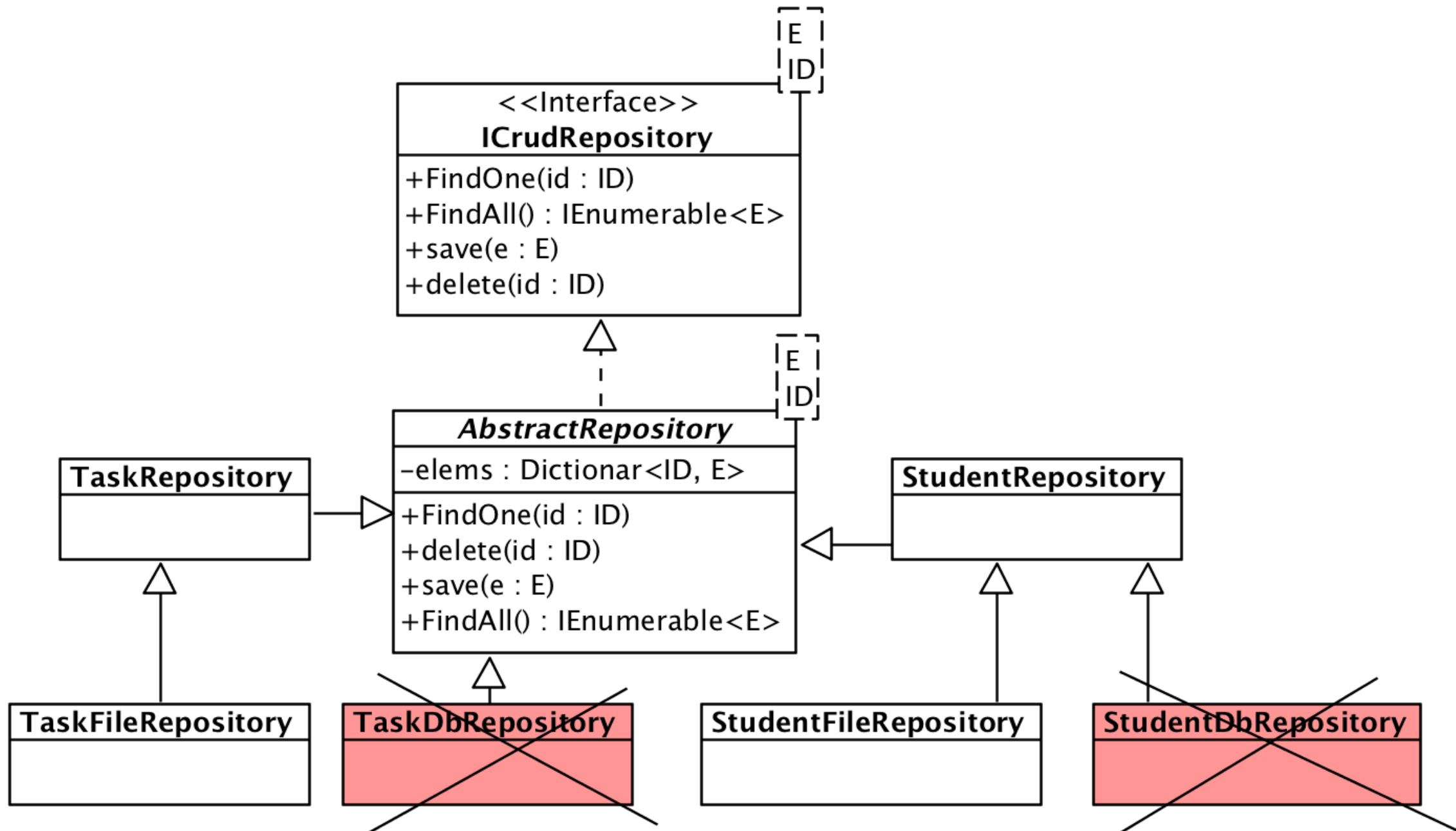
```
//Fisierul build.gradle
dependencies {
    testImplementation group: 'org.junit.jupiter', name: 'junit-jupiter-api', version:
        '5.8.2'
    testImplementation group: 'org.junit.jupiter', name: 'junit-jupiter-engine',
        version: '5.8.2'
    //jurnalizare (cautati pe Maven Repository)
    implementation group: 'org.apache.logging.log4j', name: 'log4j-core', version:
        '2.18.0'
    implementation group: 'org.apache.logging.log4j', name: 'log4j-api', version:
        '2.18.0'
    //drivere conectare la baza de date (cautati pe Maven Repository)
    runtimeOnly group: 'org.xerial', name: 'sqlite-jdbc', version: '3.36.0.3'

    runtimeOnly 'org.mariadb.jdbc:mariadb-java-client:2.1.2'
    runtimeOnly 'mysql:mysql-connector-java:5.1.20'
}

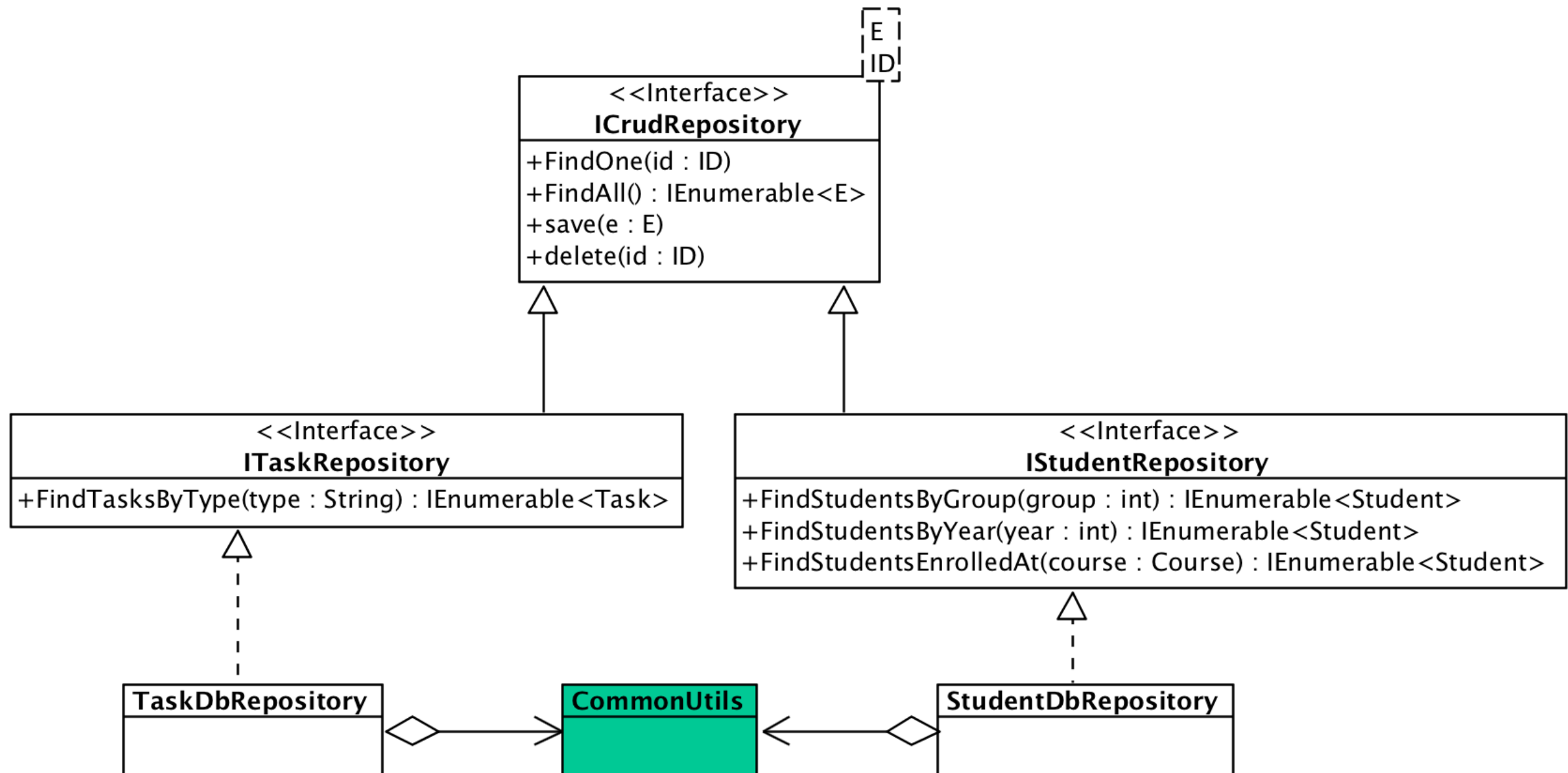
//Maven repository
//https://mvnrepository.com/
```

Exemplu Java

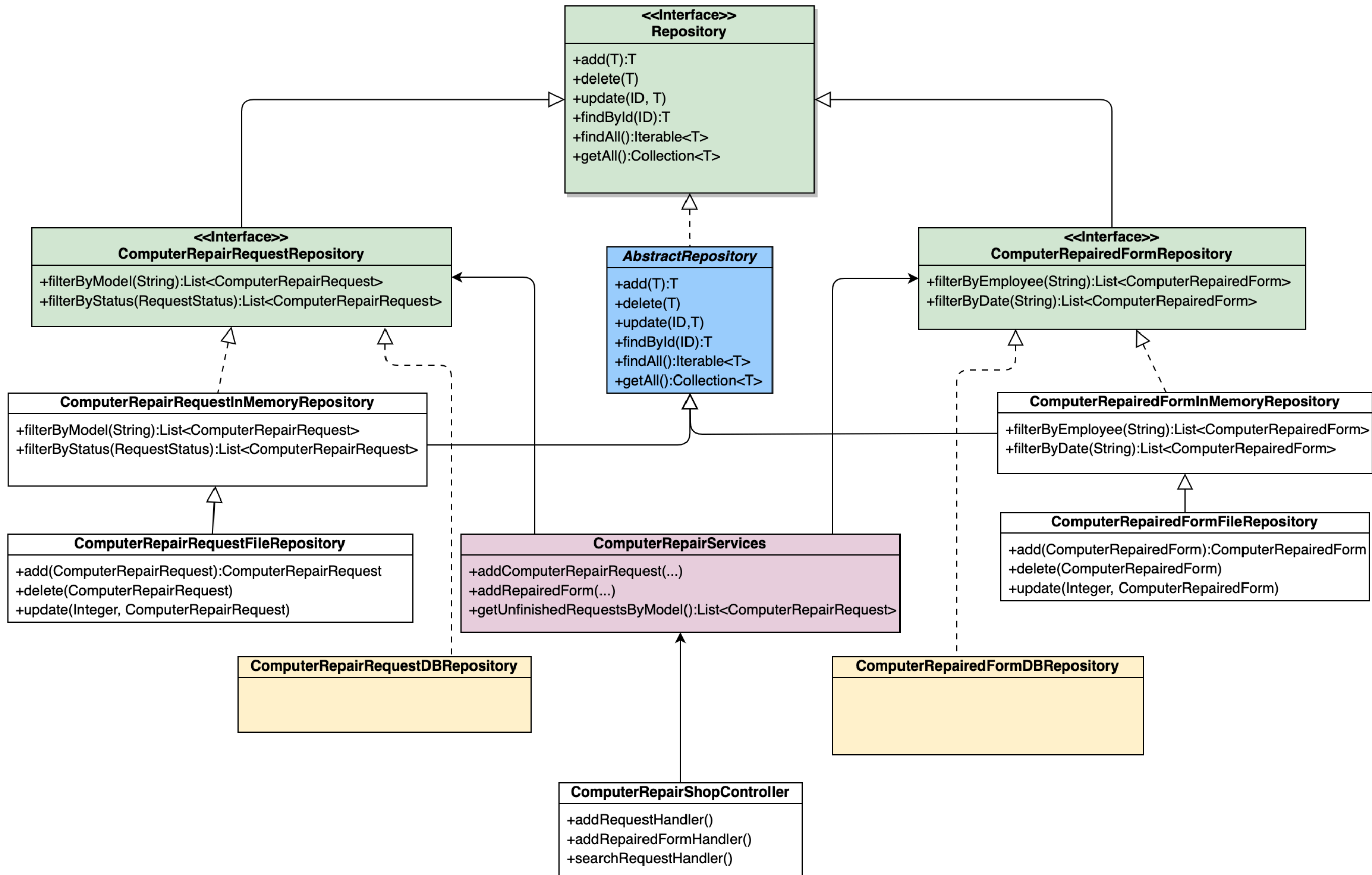
Ierarhie repositories (1)



Ierarhie repositories (2)



Ierarhie repositories (3)



ADO.NET

- ADO.NET este o bibliotecă orientată pe obiecte care permite unei aplicații să interacționeze cu diferite surse de date:
 - baze de date relaționale
 - fișiere text
 - fișiere Excel
 - fișiere XML
- Conține 4 spații de nume pentru interacțiunea cu 4 tipuri de baze de date:
 - SQL Server
 - Oracle
 - Surse ODBC
 - OLEDB.

ADO.NET

Spații de nume

- **System.Data**—Toate clasele generice pentru accesarea datelor.
- **System.Data.Common**—Clase comune sau redefinite de furnizori de date specifici.
- **System.Data.Odbc**—Clasele pentru ODBC
- **System.Data.OleDb**—Clasele pentru OLE DB
- **System.Data.Oracle**—Clasele pentru Oracle
- **System.Data.SqlClient**—Clasele pentru SQL Server
- **System.Data.SqlTypes**—Tipurile de date SQL Server

System.Data

- Conține clasele și interfețele folosite indiferent de sistemul de gestiune a bazelor de date.
- **DataSet**—Clasa pentru lucru offline. Poate conține o mulțime de **DataTables** și relații între acestea.
- **DataTable**—Un container ce conține una sau mai multe coloane. Când este populat va avea una sau mai multe **DataRows** conținând informația.
- **DataRow**—O mulțime de valori corespunzând unei linii dintr-o tabelă dintr-o bază de date relațională, sau unei linii dintr-o foaie de calcul.
- **DataColumn**—Conține definiția unei coloane dintr-o tabelă: numele și tipul.
- **DataRelation**—Reprezintă o relație între două tabele dintr-un **DataSet**. Se folosește pentru a reprezenta relația "cheie străină".
- **Constraint**—Definește constrângeri pentru una sau mai multe **DataColumn** (ex. valori unice).

System.Data.Common

- **DataColumnMapping**—Mapează numele unei coloane dintr-o tabelă din baza de date cu numele unei coloane dintr-un **DataTable**.
- **DataTableMapping**—Mapează numele unei tabele dintr-o bază de date cu un **DataTable** dintr-un **DataSet**.
- **DbCommandBuilder**—Generează automat comenzi pentru a sincroniza modificările dintr-un **DataSet** cu baza de date asociată.

ADO.NET API

- ADO.NET conține clase specifice interacțiunii cu anumite tipuri de baze de date.
- Aceste clase implementează o mulțime de interfețe standard din spațiul de nume System.Data, permițând claselor să fie folosite într-o manieră generică, dacă este necesar.
 - **IDbConnection** folosită pentru conectarea la o baza de date.
 - **IDataAdapter** folosită pentru păstrarea instrucțiunilor *select*, *insert*, *update* și *delete* care sunt apoi folosite pentru popularea unui **DataSet** și pentru actualizarea bazei de date.
 - **IDataReader**: folosit ca și un cititor de date, forward-only.
 - **IDbCommand**: folosit ca și wrapper pentru instrucțiuni SQL sau apeluri de proceduri stocate.
 - **IDbDataParameter**: reprezintă un parametru pentru un obiect de tip Command.
 - **IDbTransaction**: folosit pentru reprezentarea unei tranzacții ca și un obiect.

IDbConnection

- Reprezintă o conexiune deschisă către o sursă de date:
 - `SqlConnection, OleDbConnection, OracleConnection, OdbcConnection`
 - `MySqlConnection, SQLiteConnection (Windows),`
`SQLiteConnection (Mono)`
- Metode:
 - `BeginTransaction`
 - `ChangeDatabase`
 - `Open`
 - `Close`
 - `CreateCommand`
- Proprietăți:
 - `ConnectionString, ConnectionTimeout, Database, State`

SqlConnection

- Conectarea la Sql Server

```
var conn = new SqlConnection(  
    "Data Source=(local);Initial Catalog=Northwind;User Id=test;  
    Password=test");
```

- Conectarea la o bază de date Access folosind OleDb

```
String connectionString="Provider=Microsoft.Jet.OLEDB.4.0;Data Source=books.mdb";  
var conn=new OleDbConnection(connectionString);
```

- Conectarea la MySql:

```
String connectionString = "Database=mpp;Data Source=localhost;User id=test;"  
    + "Password=passtest;";  
var conn= new MySqlConnection(connectionString);
```

- Conectarea la Sqlite (folosind Mono.Sqlite - specific Linux/MacOS):

```
String connectionString = "URI=file:/Users/test/database/tasks.db,Version=3";  
var conn= new SQLiteConnection(connectionString);
```

<https://www.connectionstrings.com/>

SqlCommand

- Reprezintă o instrucțiune SQL executată când există o conexiune către sursa de date.
 - `SqlCommand`, `OleDbCommand`, `OracleCommand`, `ODBCCommand`
 - `MySqlCommand`, `SQLiteCommand (Mono)`, `SQLiteCommand (Windows)`
- Metode:
 - `ExecuteReader`, `ExecuteNonQuery`, `ExecuteScalar`
 - `CreateParameter`
 - `Cancel`
- Proprietăți:
 - `CommandText`, `CommandTimeout`, `CommandType`, `Connection`, `Parameters`, etc.
- `CommandType`:
 - `Text` (o comandă SQL), `StoredProcedure`, `TableDirect` (numele unei tabele, doar pentru furnizori OleDb).

SqlCommand

- Text:

```
String select = "SELECT ContactName FROM Customers";
```

```
SqlCommand cmd = new SqlCommand(select , conn);
```

- Stored Procedure

```
SqlCommand cmd = new SqlCommand("CustOrderHist", conn);
```

```
cmd.CommandType = CommandType.StoredProcedure;
```

- Table Direct

```
OleDbCommand cmd = new OleDbCommand("Categories", conn);
```

```
cmd.CommandType = CommandType.TableDirect;
```

SqlCommand

- ExecuteNonQuery:

```
string source =...;

string sqlCom = "UPDATE Customers SET ContactName = 'Bob' " +
                "WHERE ContactName = 'Bill'";

using(var conn = new OleDbConnection(source)) {
    conn.Open();

    var cmd = new OleDbCommand(sqlCom, conn);

    int rowsReturned = cmd.ExecuteNonQuery();

    Console.WriteLine("{0} rows affected.", rowsReturned);
}
```

SqlCommand

- `ExecuteReader`:

```
string source = ...;

string select = "SELECT ContactName,CompanyName FROM Customers";

using(var conn = new MySqlConnection(source)) {

    conn.Open();

    var cmd = new MySqlCommand(select, conn);

    using(var reader = cmd.ExecuteReader()) {

        while(reader.Read())

        {

            Console.WriteLine("Contact:{0} Company:{1}", reader[0] ,
            reader[1]);

        }

    }

}
```

SqlCommand

- ExecuteScalar:

```
string source = ...;

string select = "SELECT COUNT(*) FROM Customers";

using(var conn = new SqlConnection(source)) {

    conn.Open();

    using(var cmd = new SqlCommand(select, conn)) {

        object o = cmd.ExecuteScalar();

        Console.WriteLine ("Customers: {0}", o);

    }

}
```

IDataReader

- Oferă posibilitatea citirii unui flux sau mai multor fluxuri secvențial (forward-only) obținute prin executarea unei comenzi asupra unei surse de date.
 - `SqlDataReader`, `OleDbDataReader`, `OracleDataReader`, `ODBCDataReader`
 - `MySqlDataReader`, `SQLiteDataReader (Mono)`, `SQLiteDataReader (Windows)`
- O instanță de tip `IDataReader` este obținută apelând metoda `IDbCommand.ExecuteReader`.
- Metode:
 - `Read`
 - `GetBoolean`, `GetByte`, `GetDouble`, `GetFloat`, `GetInt16`, `GetString`, etc.
 - `Close`
- Proprietăți:
 - `Item` (index sau nume), `IsClosed`

IDataReader

```
string source = ...;

string selectCmd = "SELECT name,address FROM persons";

using(var conn = new SqlConnection(source)) {

    conn.Open();

    using(var cmd = conn.CreateCommand()) {

        cmd.CommandText=selectCmd;

        using(var reader = cmd.ExecuteReader()) {

            while(reader.Read())

                Console.WriteLine("{0} {1}", reader["name"] , reader["address"]);

        }

    }

}
```

IDataAdapter

- Reprezintă un set de proprietăți folosite pentru completarea unui DataSet și pentru actualizarea unei surse de date.
 - `SqlDataAdapter`, `OleDbDataAdapter`, `OracleDataAdapter`, `ODBCDataAdapter`
 - `MySqlDataAdapter`, `SqliteDataAdapter(Mono)`, `SQLiteDataAdapter (Windows)`
- Este folosit în asociere cu un DataSet.
- Un `DataSet` este un obiect în memorie care poate păstra mai multe tabele.
- `DataSets` păstrează doar informația, nu interacționează cu sursa de date.
- `IDataAdapter` gestionează conexiunile către sursa de date.
- `IDataAdapter` deschide o conexiune doar când este necesar și o închide imediat ce sarcina și-a încheiat execuția.

IDataAdapter

- Execută următoarele când **populează un DataSet** cu date:
 - Deschide o conexiune la sursa de date
 - Obține și încarcă datele în **DataSet**
 - Închide conexiunea
- Execută următoarele când **actualizează sursa de date** cu modificările din DataSet:
 - Deschide conexiunea
 - Scrie modificările din DataSet în sursa de date.
 - Închide conexiunea
- **Între populare și actualizare conexiunile către sursa de date sunt închise.**
- Metode:
 - **Fill** (adaugă sau actualizează linii în DataSet potrivite cu cele din sursa de date),
 - **Update** (apelează instrucțiunile INSERT, UPDATE, sau DELETE corespunzătoare fiecărei inserări, actualizări sau ștergeri din DataSet)
- Proprietăți: **DeleteCommand**, **InsertCommand**, **SelectCommand**, **UpdateCommand**

IDataAdapter

```
string source =...;

var Connection conn = new MySqlConnection(source);

string select = "SELECT * FROM books";

DataSet data=new DataSet();

var dataAdapter=new MySqlAdapter(select, conn);

dataAdapter.Fill(data, "Books");

DataRowCollection dra=data.Tables["Books"].Rows;

foreach(DataRow in dra)

    Console.WriteLine(dr["isbn"]+dr["author"]+dr["title"]);
```

IDataParameter

- Reprezintă parametrul unui obiect de tip `Command`.
 - `SqlParameter`, `OracleParameter`, `OleDbParameter`, `OdbcParameter`
 - `MySqlParameter`, `SQLiteParameter(Mono)`, `SQLiteParameter (Windows)`
- Membrii
 - `Value`
 - `ParameterName`
 - `DbType`
- `DbType`:
 - `Boolean`, `Date`, `Double`, `Int32`, `String`, etc.

IDataParameter

```
string source = ...;

string select = "SELECT * FROM Customers where city=@City";

using(var conn = new SqlConnection(source)) {

    conn.Open();

    using(var cmd = new SqlCommand(select, conn)) {

        var param = cmd.CreateParameter();

        param.ParameterName = "@City";

        param.Value          ="ABC";

        cmd.Parameters.Add(param);

        using(var reader = cmd.ExecuteReader()) {

            while(reader.Read())

            {

                Console.WriteLine("Contact:{0} Company:{1}", reader["CompanyName"],
                    reader["ContactName"]);

            }

        }

    }

}
```

app.config

- Fișier de configurare pentru aplicații .NET

```
<?xml version="1.0" encoding="utf-8"?>

<configuration>
    <connectionStrings>
        <add name="tasksDB"
            connectionString="URI=file:/Users/xyz/MPP/database/tasks.db,Version=3" />
        <!--
    <add name="tasksDB"
        connectionString="Database=mpp;Data Source=localhost;User id=test;Password=passtest;" />
    -->
    </connectionStrings>
</configuration>
```

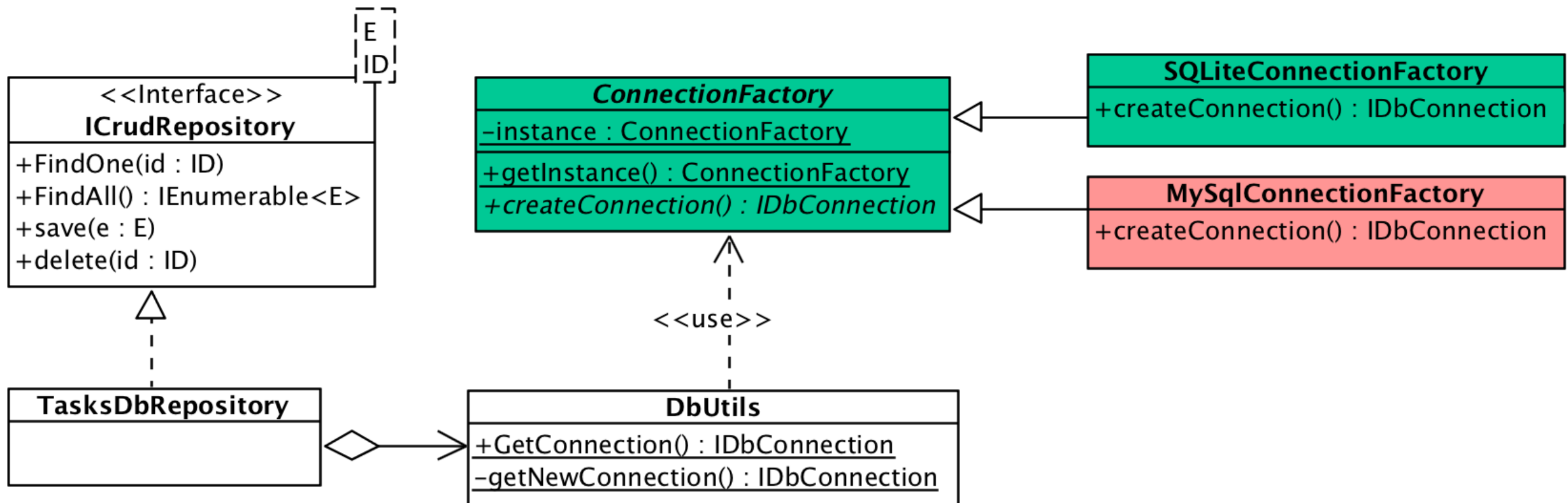
- La compilare fișierul este copiat în directul bin/debug cu numele `NumeApp.exe.config` (unde NumeApp este numele proiectului)

app.config

- Obținerea datelor din app.config
- Clasa `ConfigurationManager` (spațiul de nume `System.Configuration`)

```
static string GetConnectionStringByName(string name){  
    // Presupunem ca nu exista.  
    string returnValue = null;  
  
    // Cauta numele in sectiunea connectionStrings.  
    ConnectionStringSettings settings = ConfigurationManager.ConnectionStrings[name];  
  
    // Daca este gasit, returneaza valoarea asociata la connection string.  
    if (settings != null)  
        returnValue = settings.ConnectionString;  
  
    return returnValue;  
}
```

Arhitectura C#



Exemplu C#

Jurnalizare C#

- Log4net
- Instalare pachete **NuGet**
- Configurare:

<https://stackify.com/log4net-guide-dotnet-logging/>

Medii de proiectare și programare

2024-2025

Curs 4

Conținut curs 4

- Adnotări
- Java Beans
- Introducere in Spring

Adnotări (*Annotations*)

- Începând cu versiunea 1.5
- Adaugă informații unei părți de cod (clasă, metodă, pachet, etc), dar **nu** fac parte din program. Adnotările **nu au nici un efect direct asupra codului pe care îl marchează.**
- Utilizări:
 - *A furniza informații suplimentare compilatorului.* Adnotările pot fi folosite de compilator pentru a detecta erori sau pentru a elimina atenționări.
 - *Procesare automata din timpul compilării sau deploymentului.* Instrumente soft specializate pot folosi adnotările pentru a genera automat cod, fișiere XML, etc.
 - *Procesare în timpul execuției.* Unele adnotări sunt disponibile pentru a fi examinate în timpul execuției codului.

Definirea adnotărilor

```
[declaratii meta-adnotari]
public @interface NumeAdnotare {
    [declaratii elemente]
}
```

Meta-adnotările (pachetul `java.lang.annotation`)(adnotări pentru adnotări) pot fi:

- `@Target(ElementType)` : specifică locul din codul sursă unde poate fi folosită adnotarea.
 - `CONSTRUCTOR`: declararea unui constructor
 - `FIELD`: declararea unui atribut (inclusiv constante enum)
 - `LOCAL_VARIABLE`: declararea unei variabile locale
 - `METHOD`: declararea unei metode
 - `PACKAGE`: declararea unui pachet
 - `PARAMETER`: declararea unui parametru
 - `TYPE`: declararea unei noi clase, interfețe, adnotări sau enum.

Definirea adnotărilor

Meta-adnotările pot fi:

- **@Retention(RetentionPolicy)** : specifică cât timp va fi păstrată adnotarea:
 - **SOURCE**: Adnotările nu sunt salvate la compilare.
 - **CLASS**: Adnotările sunt disponibile în fișierul .class, dar pot fi eliminate de mașina virtuală.
 - **RUNTIME**: Adnotările sunt păstrate de mașina virtuală în timpul execuției și pot fi citite folosind reflecție.
- **@Documented**: adnotarea va fi inclusă în documentația Javadocs.
- **@Inherited**: Permit subclaselor să moștenească adnotările părinților.

Elementele unei adnotări

- Sintaxa:

`Tip numeElement() [default valoare_implicita];`

unde **Tip** poate fi:

- orice tip primitiv (`int`, `float`, `double`, `byte`, etc.)
- `String`
- `Class`
- Enumerări (`enum`)
- Adnotări (`annotation`)
- Tablouri de tipurile menționate mai sus.

Observații:

1. Dacă se folosește **alt tip la declararea unui element**, compilatorul va genera **eroare**.
2. Dacă o adnotare nu conține nici un element, adnotarea se numește de tip *marker*.

Constrângeri valori implicite

- Exista **două constrângeri** pentru valoarea unui element:
 1. *Nici un element nu poate avea o valoare nespecificată* (fie se declară o valoare implicită, fie se atribuie o valoare pentru fiecare element în momentul folosirii adnotării).
 2. *Pentru elementele care nu sunt de tip primitiv, NU se acceptă valoarea **null** (în momentul folosirii sau ca și valoare implicită).*

Adnotări - exemplu

```
import java.lang.annotation.*;

@Target(ElementType.CLASS)
@Retention(RetentionPolicy.RUNTIME)
public @interface ClassPreamble {
    String author();
    String date();
    int currentRevision() default 1;
    String lastModified() default "N/A";
    String lastModifiedBy() default "N/A";
    String[] reviewers();
}
```


Folosirea adnotărilor

Adnotarea apare prima, de obicei pe linie proprie, și poate conține elemente.

Observații:

1. Dacă adnotarea conține un singur element numit **value**, numele acestuia poate fi omis.
2. Dacă adnotarea nu conține nici un element, parantezele pot fi omise.

```
@ClassPreamble (  
    author = "Popescu Vasile",  
    date = "3/17/2008",  
    currentRevision = 4,  
    lastModified = "4/11/2011",  
    lastModifiedBy = "Ionescu Matei"  
    reviewers = {"Vasilescu Ana", "Marinescu Ion", "Pop Ioana"}  
)  
public class A extends B{  
    //...  
}
```


Exemplu adnotări

Declararea:

```
import java.lang.annotation.*;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface UseCase {
    public int id();
    public String description() default "no description";
}
```

Folosirea

```
public class A{
    @UseCase(id=3, description="abcd")
    public void f(){
    }
}
```

Adnotări standard

- JSE conține 3 adnotări standard:
 - **@Override** pentru a indica că o metoda redefinește o metodă din clasa de bază. Dacă numele metodei sau semnatura nu sunt corecte, compilatorul va genera o eroare.

```
class A{  
    @Override  
    public String toString(){...}  
}
```

- **@Deprecated** pentru a genera o atenționare la compilare când se folosește elementul adnotat (clasă, metodă, etc.)
- **@SuppressWarnings** spune compilatorului să nu furnizeze anumite atenționări:

```
unchecked, deprecated  
@SuppressWarnings("unchecked", "deprecated")  
void metodaA() { }
```

Beans

- Orice clasă Java este un **POJO** (eng. *Plain Old Java Object*).
- JavaBeans: este o clasă Java specială. Reguli:
 - **Trebuie să aibă un constructor implicit (public și fără nici un parametru).**
Alte instrumente specializate vor folosi acest constructor pentru a instanția un obiect.
 - **Atributele trebuie să poată fi accesate folosind metode de tip `getXyz`, `setXyz` și `isXyz` (pentru atribute de tip boolean).** Atributele pentru care sunt definite aceste metode se numesc **proprietăți**, numele proprietății fiind **xyz**. Când se modifică sau se dorește valoarea unei proprietăți se apelează una dintre metodele corespunzătoare.
 - **Clasa trebuie să fie serializabilă.** Acest lucru permite instrumentelor specializate să salveze și să refacă starea unui JavaBean.
 - Exemplu: **Componentele GUI**
- Enterprise Java Beans (EJBs): pentru aplicații complexe (tranzacții, securitate, acces la baze de date)

Exemplu Java Beans

```
public class Student implements java.io.Serializable {  
    private String nume;  
    private int grupa;  
    private boolean licentiat;  
    private int note[];  
    public Student() { }  
    public Student(String nume, int grupa, boolean licentiat){...}  
    public String getName() { return nume; }  
    public void setName(String name) { nume = name; }  
    public int getGrupa() {return grupa;}  
    public void setGrupa(int g){grupa=g;}  
    public void setLicentiat(boolean l){licentiat=l;}  
    public boolean isLicentiat(){ return licentiat;}  
    public void setNote(int[] n){ note=n;}  
    public int[] getNote(){return note;}  
}
```

Introducere în Spring - Motivație

- Orice aplicație medie sau complexă este compusă dintr-o mulțime de obiecte care colaborează pentru atingerea unui scop. Aceste obiecte știu despre celelalte obiecte (asocierile) și comunică prin transmiterea de mesaje.
- Abordarea tradițională pentru crearea asocierilor dintre obiecte (prin instanțiere sau căutare) generează cod complicat care este dificil de reutilizat și testat (folosind unit testing).

//varianta 1

```
class ConcursService{  
    private ParticipantRepositoryMock repo;  
    public ConcursService(){  
        repo=new ParticipantRepositoryMock();  
    }  
    //...  
}
```

Introducere în Spring

//varianta 2

```
class ConcursService{
    private ParticipantRepositoryFile repo;
    public ConcursService(){
        repo=new ParticipantRepositoryFile("Participanti.txt");
    }
}
```

//varianta 2a

```
public ConcursService(){
    repo=new ParticipantRepositoryFile("Participanti2.txt",
                                       new ParticipantValidator());
}
```

//varianta 3

```
class ConcursService{
    private ParticipantRepositoryJdbc repo;
    public ConcursService(){
        Properties props=...
        repo=new ParticipantRepositoryJdbc(props);
    }
}
```

Introducere în Spring

- Spring este un framework open-source, creat inițial de Rod Johnson și descris în cartea sa, *Expert One-on-One: J2EE Design and Development* (2002).
- Frameworkul Spring a fost creat pentru a facilita dezvoltarea aplicațiilor complexe și foarte mari.
- În *Spring se pot folosi obiecte simple Java (POJO)*, pentru a crea aplicații care anterior erau posibile doar folosind EJB.
- Un **bean Spring** este orice clasă Java (nu respectă regulile Java Beans).
- *Spring promovează cuplarea slabă prin “injectarea” asocierilor și folosirea interfețelor.*
- Spring folosește principiul **Inversion of Control (IoC)** pentru “injectarea” asocierilor/dependențelor.

Introducere în Spring

```
public interface ParticipantRepository{...}

class ConcursService{
    private ParticipantRepository repo;
    public ConcursService(ParticipantRepository r){
        repo=r;
    }
    ...
}

//sau

class ConcursService{
    private ParticipantRepository repo;
    public ConcursService(){... }
    public void setParticipantRepo(ParticipantRepository r){repo=r;}
}

public class ParticipantRepositoryFile implements
    ParticipantRepository{...}

public class ParticipantRepositoryJdbc implements
    ParticipantRepository{...}
```

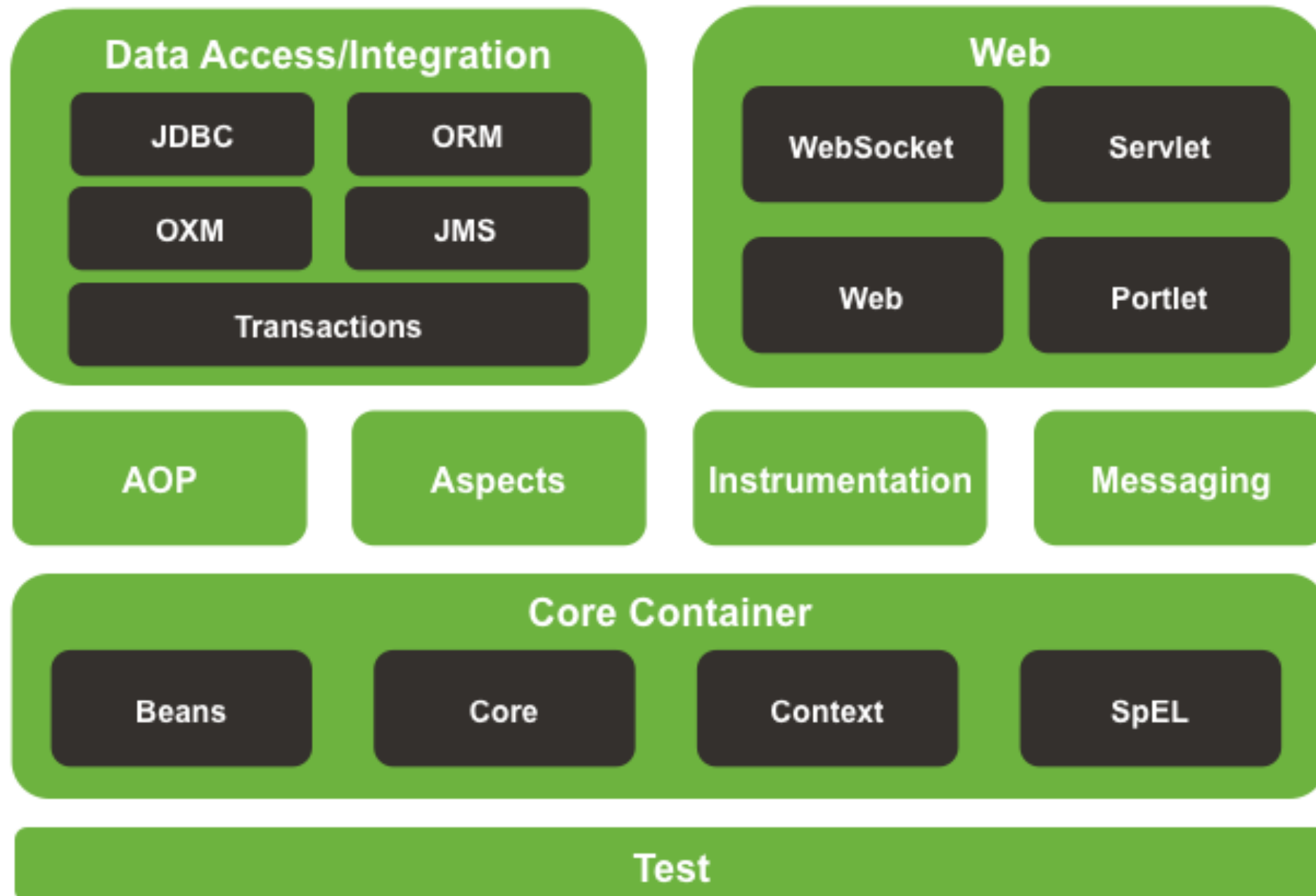

IoC, Dependency Injection

- Principiul *Inversion of Control* este cunoscut și ca *dependency injection* (DI).
- DI este procesul prin care obiectele își definesc asocierile (dependențele) fie prin parametrii constructorilor, fie prin argumentele unei metode de tip factory sau prin proprietăți de tip set, care trebuie apelate imediat după crearea obiectului.
- Un container “injectează” aceste dependențe când creează obiectul. Acest proces este invers celui tradițional, în care obiectul este responsabil de instanțierea sau localizarea dependențelor sale.
- În Spring, obiectele care formează un sistem (aplicație) soft sunt gestionate de containerul bazat pe IoC și sunt numite **bean**-uri.
- Un **bean Spring** este un **obiect Java obișnuit** care este **instanțiat, asamblat și gestionat de containerul Spring IoC**.
- Bean-urile și asocierile dintre ele sunt descrise în **datele de configurare** folosite de container.
- Două variante de a descrie bean-urile: folosind *fișiere de configurare în format XML* sau *cod Java* (fișiere de configurare sau autowire).

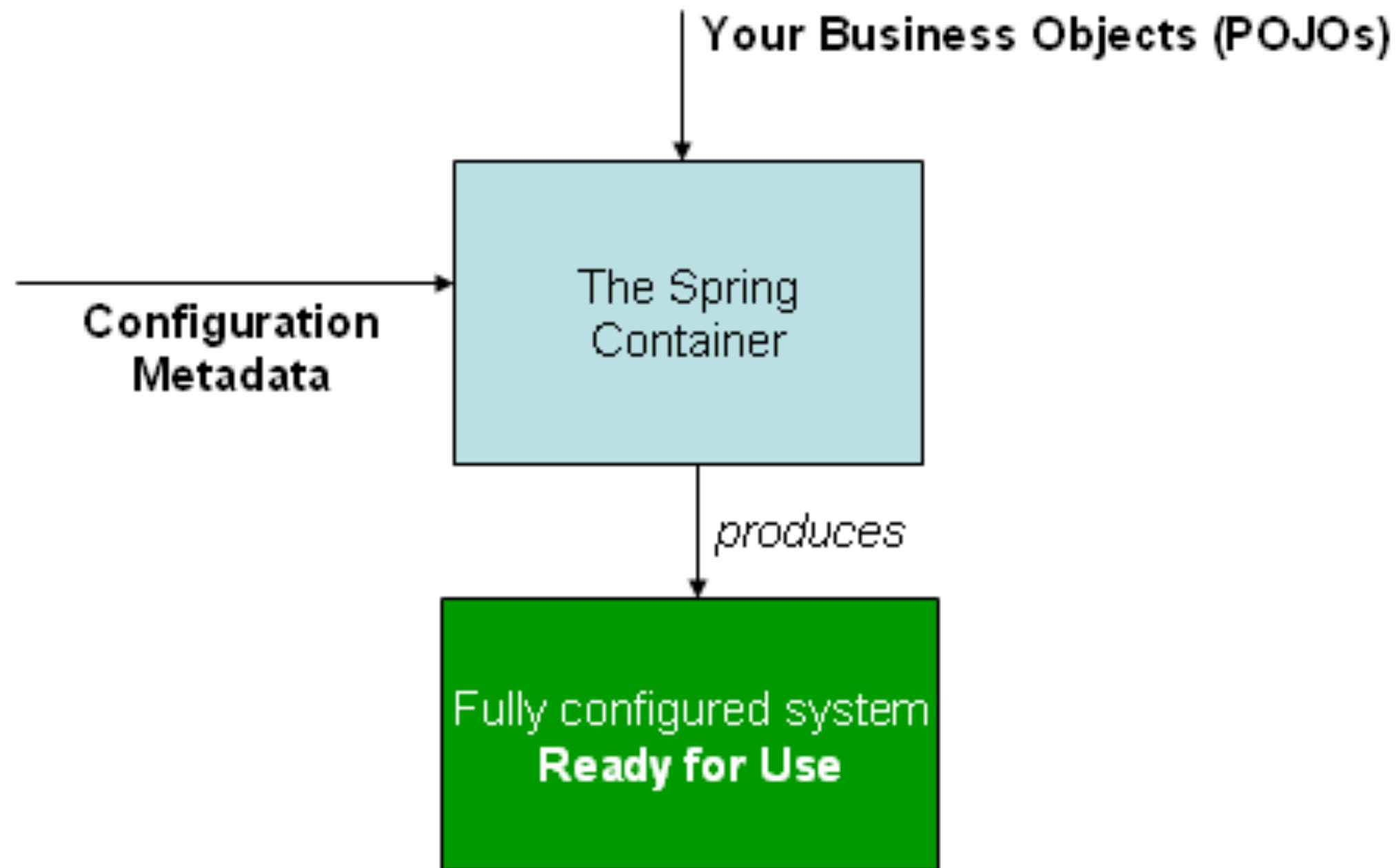
Arhitectura framework-ului Spring



Spring Framework Runtime



Containerul Spring



Crearea containerului Spring

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class StartApp{
    public static void main(String[] args){
        ApplicationContext factory = new
            ClassPathXmlApplicationContext("classpath:spring-concurs.xml");
        //obtinerea referintei catre un bean din container
        ConcursService concurs= factory.getBean(ConcursService.class);
    }
}
```

Fișierul de configurare XML

- Când se declară bean-urile folosind fișiere XML, elementul rădăcină a fișierului de configurare este `<beans>`.
- Un șablon simplu pentru fișierul de configurare este:

```
<?xml version="1.0"encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/schema/beans"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xsi:schemaLocation="http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<!-- Declararea bean-urilor-->
```

```
</beans>
```

- În interiorul elementului `<beans>`, sunt descrise toate configurările specifice containerului Spring (dacă există) și toate declarațiile bean-urilor.

Declararea unui bean simplu

```
package pizzax.validation;
import pizzax.model.Pizza;
public class DefaultPizzaValidator implements Validator<Pizza> {
    public void validate(Pizza pizza) {
        //...
    }
}
```

```
//spring-pizza.xml
<beans ...>
    <bean id="pizzaValidator"
        class="pizzax.validation.DefaultPizzaValidator"/>
</beans>
```

- Elementul **<bean>** este elementul de bază dintr-un fișier de configurare XML. El spune containerului Spring să creeze un obiect.
- Atributul **id** specifică numele prin care obiectul va fi referit în container.
- Când containerul Spring va încărca bean-urile, el va instanția bean-ul **"pizzaValidator"** folosind constructorul implicit.

DI - Constructori

```
package pizzax.repository.file;
import pizzax.repository;
public class PizzaRepositoryFile implements PizzaRepository {
    private String numefis;
    public PizzaRepositoryFile(String numefis){
        ...
    }
    //implementarea metodelor
    ...
}
//spring-pizza.xml
...
<bean id="pizzaRepository"
    class="pizzax.repository.file.PizzaRepositoryFile">
    <constructor-arg value="Pizza.txt" />
</bean>
```


DI - Constructori (2)

```
public class PizzaRepositoryFile implements PizzaRepository {
    private String numefis;
    private Validator<Pizza> valid;
    public PizzaRepositoryFile(String numefis, Validator<Pizza> valid) { ... }
    ...
}
//spring-pizza.xml
...
<bean id="pizzaValidator" class="pizzax.validation.DefaultPizzaValidator"/>
<bean id="pizzaRepository"
    class="pizzax.repository.file.PizzaRepositoryFile">
    <constructor-arg value="Pizza.txt" />
    <constructor-arg ref="pizzaValidator" />
</bean>
```

```
Validator<Pizza> pizzaValidator=new DefaultPizzaValidator();
PizzaRepository pizzaRepository=new PizzaRepositoryFile("Pizza.txt",
    pizzaValidator);
```


DI - Constructori (3)

```
public class Probus {
    private String denumire="";
    private double pret=0;
    public Probus(String denumire, double pret) {
        this.pret = pret;
        this.denumire = denumire;
    }
    //...
}

//spring-exemplu.xml
<bean id="mere" class="Probus">
    <constructor-arg index="0" value="Mere" />
    <constructor-arg index="1" value="3.14"/>
</bean>

<!--sau -->
<bean id="mere" class="Probus">
    <constructor-arg type="java.lang.String" value="Mere" />
    <constructor-arg type="double" value="3.14"/>
</bean>
```

DI - Metode factory

```
public class A {  
    private static A instance;  
    private A(){...};  
    public static A getInstance() { ...}  
    ...  
}
```

```
//spring-exemplu.xml
```

```
...
```

```
<bean id="instanta" class="A" factory-method="getInstance" / >
```

```
//echivalent cu
```

```
A objA=A.getInstance();
```

Scopul

Implicit toate bean-urile sunt *singleton* (se creează o singură instanță, indiferent de câte ori un bean este folosit la configurare, sau folosind metoda `getBean()` din clasa `ApplicationContext`).

Pentru a schimba scopul implicit, se folosește atributul “**scope**” al tag-ului `<bean>`

```
<bean id="bilet" class="xyz.Bilet" scope="prototype"/>
```

- Valorile posibile pentru atributul “**scope**” sunt:
 - **singleton**: o singură instanță pentru un container Spring.
 - **prototype**: pentru fiecare utilizare se creează un nou bean.
 - **request, session, global-session**: se utilizează pentru aplicații Web.

```
<bean id="beanA" class="test.B" scope="prototype"/>
```

```
<bean id="beanB" class="test.B"/>
```

```
<bean id="beanC" class="test.C">
```

```
    <constructor-arg ref="beanA"/> <!--se creeaza o noua instanta -->
```

```
</bean>
```

```
<bean id="beanD" class="test.D">
```

```
    <constructor-arg ref="beanA"/> <!--se creeaza o noua instanta -->
```

```
</bean>
```

DI folosind proprietăți

```
package pizzax.repository.file;
import pizzax.repository;
public class PizzaRepositoryFile implements PizzaRepository {
    private String numefis;
    public PizzaRepositoryFile() { ... }
    public void setNumeFisier(String numefis){...}
    //implementarea metodelor
    ...
}
```

```
//spring-pizza.xml
...
<bean id="pizzaRepository"
      class="pizzax.repository.file.PizzaRepositoryFile">
    <property name="numeFisier" value="Pizza.txt"/>
</bean>
```

DI folosind proprietăți

```
package pizzax.repository.file;
import pizzax.repository;
public class PizzaRepositoryMock implements PizzaRepository {
    private Validator<Pizza> valid;
    public PizzaRepositoryMock() { ... }
    public void setValidator(Validator<Pizza> v) {valid=v;}
    //implementarea metodelor
    ...
}
```

```
//spring-pizza.xml
```

```
...
<bean id="pizzaRepository"
      class="pizzax.repository.file.PizzaRepositoryMock">
    <property name="validator" ref="pizzaValidator"/>
</bean>
```

DI Constructor + proprietăți

```
package pizzax.repository.file;
import pizzax.repository;
public class PizzaRepositoryFile implements PizzaRepository {
    private Validator<Pizza> valid;
    private String numefis;
    public PizzaRepositoryFile(String numefis) { ... }
    public void setValidator(Validator<Pizza> v){valid=v;}
    //implementarea metodelor
    ...
}
```

```
//spring-pizza.xml
```

```
...
<bean id="pizzaRepository"
    class="pizzax.repository.file.PizzaRepositoryFile">
    <constructor-arg value="Pizza.txt"/>
    <property name="validator" ref="pizzaValidator"/>
</bean>
```

Bean-uri inner

```
package pizzax.repository.file;
import pizzax.repository;
public class PizzaRepositoryMock implements PizzaRepository {
    private Validator<Pizza> valid;
    public PizzaRepositoryMock() { ... }
    public void setValidator(Validator<Pizza> v) {valid=v;}
    //implementarea metodelor
    ...
}

//spring-pizza.xml
...
<bean id="pizzaRepository"
    class="pizzax.repository.file.PizzaRepositoryMock">
    <property name="validator">
        <bean class="pizzax.validation.DefaultPizzaValidator"/>
    </property>
</bean>
```


Bean-uri inner

Observații:

- Bean-urile inner **nu** necesită specificarea atributului **id**. Se poate declara o valoare pentru **id**, dar nu este folosită de container.
- **Aceste bean-uri nu pot fi refolosite**. Sunt folosite pentru “injectare” o singură dată și nu pot fi referite de alte bean-uri.

Proprietăți de tip colecție

- Există situații când o proprietate/parametru-constructor este de tip ***container*** (*colecție, mulțime, dicționar, tablou, etc...*).
- Pentru a inițializa acest tip de proprietăți Spring a definit 4 elemente de configurare:
 - **<list>**: o listă de valori ce poate conține duplicate
 - **<set>**: o lista de valori ce nu conține duplicate
 - **<map>**: o mulțime de perechi cheie-valoare (dicționar)
 - **<props>**: o mulțime de perechi cheie-valoare, unde atât cheia cât și valoarea sunt de tip **String** (clasa **java.util.Properties**)

Proprietăți de tip colecție

- Liste, mulțimi, tablouri:

```
class Produs{
    private String denumire;
    private double pret;
    public Produs(){...}
    public void setDenumire(String d){...}
    public void setPret(double d){...}

    //metode get si set
}
```

```
class Depozit{
    //...
    public void setProduse(java.util.List<Produs> lp){...}
    //sau
    public void setProduse(java.util.Collection<Produs> lp){...}
    //sau
    public void setProduse(Produs[] lp){...}
}
```

Proprietăți de tip colecție

- Liste, tablouri:

```
//spring-exemplu.xml
```

```
<bean id="mere" class="Probus">
    <property name="denumire" value="Mere"/>
    <property name="pret" value="2.3"/>
</bean>
<bean id="pere" class="Probus"> ...</bean>
<bean id="prune" class="Probus"> ...</bean>
<bean id="depozit" class="Depozit">
    <property name="produse">
        <list>
            <ref bean="mere"/>
            <ref bean="pere"/>
            <ref bean="prune"/>
        </list>
    </property>
</bean>
```

Proprietăți de tip colecție

- Mulțimi:

```
//spring-exemplu.xml
```

```
<bean id="mere" class="Produs">
    <property name="denumire" value="Mere"/>
    <property name="pret" value="2.3"/>
</bean>
<bean id="pere" class="Produs"> ...</bean>
<bean id="prune" class="Produs"> ...</bean>
<bean id="depozit" class="Depozit">
    <property name="produse">
        <set>
            <ref bean="mere"/>
            <ref bean="pere"/>
            <ref bean="prune"/>
            <ref bean="prune"/>
        </set>
    </property>
</bean>
```

Proprietăți de tip colecție

- Dicționare:

```
class Depozit{  
    //...  
    public void setProduse(java.util.Map<String, Produs> lp){...}  
  
}
```

```
//spring-exemplu.xml  
<bean id="mere" class="Produs">...</bean>  
<bean id="pere" class="Produs"> ...</bean>  
<bean id="prune" class="Produs"> ...</bean>  
<bean id="depozit" class="Depozit">  
    <property name="produse">  
        <map>  
            <entry key="pMere" value-ref="mere"/>  
            <entry key="pPere" value-ref="pere"/>  
            <entry key="pPrune" value-ref="prune"/>  
        </map>  
    </property>  
</bean>
```

Proprietăți de tip colecție

- Dicționare: elementul `<entry>` are următoarele attribute:
 - **key**: specifică *cheia* ca și *string*;
 - **key-ref**: specifică *cheia* ca și *referință la alt bean din container*;
 - **value**: specifică *valoarea* ca și *string*;
 - **value-ref**: specifică *valoarea* ca și *referință la un alt bean* din container.
- Proprietăți: elementele **props** și **prop**

```
class Depozit{  
    public void setProprietati(Properties p){...}  
}
```

```
<bean id="depozit" class="Depozit">  
<property name="proprietati">  
    <props>  
        <prop key="prop1"> A1 </prop>  
        <prop key="prop2"> B C2 </prop>  
    </props>  
</property>  
</bean>
```

Bean-uri tip container

- Există situații când trebuie creat un **bean de tip container** (colecție, mulțime, dicționar, tablou, etc...).
- Pentru a crea un bean de tip container:

- `<util:list>`, `<util:set>`, `<util:map>`, `<util:props>`
`<beans xmlns="http://www.springframework.org/schema/beans"`
`xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"`
`xmlns:p="http://www.springframework.org/schema/p"`
`xmlns:util="http://www.springframework.org/schema/util"`
`xsi:schemaLocation="http://www.springframework.org/schema/beans`
`http://www.springframework.org/schema/beans/spring-beans.xsd`
`http://www.springframework.org/schema/util`
`http://www.springframework.org/schema/util/spring-util.xsd"> ... </beans>`

```
<util:properties id="jdbcProps">  
    <prop key="tasks.jdbc.url">jdbc:sqlite:database.db</prop>  
</util:properties>
```

sau

```
<util:properties id="jdbcProps" location="classpath:bd.config"/>
```


Valori null

- E posibil ca valoarea unei proprietăți să fie setată la null. Pentru aceasta se folosește elementul `<null/>`

```
<property name="numeProprietate"> <null/></property>
```

- **SpEL (Spring Expression Language)**
 - a fost introdus începând cu versiunea 3.0
 - permite calcularea/ determinarea valorilor unor proprietăți în timpul execuției:

```
<property name="numarAleator" value="#{T(java.lang.Math).random()}" />
```

```
<property name="song" value="#{songSelector.selectSong().toUpperCase()}" />
```

```
<property name="fullName"
```

```
value="#{person.firstName + ' ' + person.lastName}" />
```


DI constructor vs. DI proprietăți

Recomandări:

- **DI constructor** - pentru **dependențe obligatorii**
- **DI proprietăți** - pentru **dependențe opționale**
- Situații speciale:
 - constructori cu prea mulți parametri
- **Verificarea dependențelor opționale că sunt nenule!**

Configurare folosind Gradle

- Fișierul `build.gradle`

```
dependencies {
```

```
    implementation 'org.springframework:spring-context:6.1.5'
```

```
    runtimeOnly group: 'org.xerial', name: 'sqlite-jdbc', version: '3.16.1'
```

```
    testImplementation group: 'junit', name: 'junit', version: '4.11'
```

```
}
```

Configurare Spring folosind XML

- Exemplu
 - TaskSpringXMLConfig

Configurare folosind JavaConfig

- Specificarea bean-urilor care trebuie create se face într-o altă clasă, adnotată cu `@Configuration`:

`@Configuration`

```
public class PizzeriaConfig {  
    //...  
}
```

- Adnotarea marchează clasa ca și o clasă de configurare
- Containerul Spring consideră că această clasă conține detalii despre bean-urile care trebuie create și cum trebuie create.

Configurare folosind JavaConfig

- Declararea unui bean se face cu adnotarea `@Bean`:

```
import pizzax.validation.DefaultPizzaValidator;

@Configuration
public class PizzeriaConfig {
    @Bean
    public Validator<Pizza> validator() {
        return new DefaultPizzaValidator();
    }
}
```

- Id-ul implicit al bean-ului este numele metodei (ex. `validator`)
- Setarea unui id explicit se face folosind elementul “`name`”.

```
@Bean(name="pizzaVal")
public Validator<Pizza> validator() {
    return new DefaultPizzaValidator();
}
```

Configurare folosind JavaConfig

- Injectarea dependențelor:

`@Configuration`

```
public class PizzeriaConfig {  
    @Bean  
    public Validator<Pizza> validator() {  
        return new DefaultPizzaValidator();  
    }  
    //varianta a - apelul metodei care creeaza bean-ul  
    @Bean  
    public PizzaRepositoryMock pizzaRepo() {  
        return new PizzaRepositoryMock(validator());  
    }  
  
    @Bean  
    public PizzaRepositoryFile pizzaFileRepo() {  
        return new PizzaRepositoryFile(validator(), "Pizza.txt");  
    }  
}
```

- **Un singur bean de tip `Validator<Pizza>` este creat!**

Configurare folosind JavaConfig

- Injectarea dependențelor:

`@Configuration`

```
public class PizzeriaConfig {  
    @Bean  
    public Validator<Pizza> validator(){  
        return new DefaultPizzaValidator();  
    }  
  
    //varianta b - transmiterea bean-ului ca si parametru  
    @Bean  
    public PizzaRepositoryMock pizzaRepo(Validator<Pizza> val){  
        return new PizzaRepositoryMock(val);  
    }  
  
    @Bean  
    public PizzaRepositoryFile pizzaFileRepo(Validator<Pizza> val){  
        return new PizzaRepositoryFile(val, "Pizza.txt");  
    }  
}
```


Configurare folosind JavaConfig

- În interiorul unei metode adnotate cu @Bean se poate folosi orice cod Java necesar creării bean-ului:

@Configuration

```
public class PizzeriaConfig {
```

```
    @Bean
```

```
    public PizzaRepositoryFile pizzaFileRepo(Validator<Pizza> val) {
```

```
        PizzaRepositoryFile repo=new new PizzaRepositoryFile("Pizza.txt");
```

```
        repo.setValidator(val);
```

```
        return repo;
```

```
    }
```

```
    @Bean
```

```
    public PizzaRepositoryJdbc pizzaJdbcRepo() {
```

```
        Properties jdbcProps=new Properties();
```

```
        try {
```

```
            jdbcProps.load(new FileReader("bd.config"));
```

```
        } catch (IOException e) {
```

```
            System.out.println("No properties were set. Cannot find bd.config "+e);
```

```
        }
```

```
        return new PizzaRepositoryJdbc(jdbcProps);
```

```
    }
```

```
}
```


Crearea containerului Spring JavaConfig

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.AnnotationConfigApplicationContext;

public class StartApp{
    public static void main(String[] args){
        ApplicationContext context=new
            AnnotationConfigApplicationContext(PizzeriaConfig.class);
        //obtinerea referintei catre un bean din container
        PizzaService repo= factory.getBean(PizzaService.class);
    }
}
```

Configurare automată folosind Java

- Configurarea automată (implicită) este o metodă prin care **containerul Spring descoperă automat bean**-urile care trebuie create, dependențele dintre acestea și încearcă să creeze aceste bean-uri.
- **@ComponentScan** specifică containerului opțiunea de configurare automată

```
package pizzeria.config;  
  
@Configuration  
@ComponentScan  
public class PizzeriaAutowireConfig {  
  
}
```

- Implicit, containerul va încerca să descopere bean-urile începând cu pachetul clasei de configurare.

```
@ComponentScan("pizzeria")  
@ComponentScan(basePackages={"pizzeria","cofetarie"})  
@ComponentScan(basePackageClasses={C.class, D.class})
```

Configurare automată folosind Java

- Declararea bean-urilor: **@Component**

@Component

```
public class DefaultPizzaValidator implements Validator<Pizza> {  
    public void validate(Pizza pizza) {  
        //...  
    }  
}
```

- Implicit, **id-ul** bean-ului este numele clasei cu prima literă transformată în literă mică.
- Bean cu id explicit:

@Component("pizzaVal")

```
public class DefaultPizzaValidator implements Validator<Pizza> {  
    public void validate(Pizza pizza) {  
        //...  
    }  
}
```

Configurare automată folosind Java

- Marcarea dependențelor: **@Autowired**
- Constructori, atribute, metode (set, etc.)

@Component

```
public class PizzaInMemoryRepository implements PizzaRepository {  
    private Validator<Pizza> valid;  
    @Autowired  
    public PizzaInMemoryRepository(Validator<Pizza> valid) { ... }  
    ...  
}
```

@Component

```
public class PizzaInMemoryRepository implements PizzaRepository {  
    private Validator<Pizza> valid;  
    public PizzaInMemoryRepository() { ... }  
    @Autowired  
    public void setValidator(Validator<Pizza> val) {...}  
}
```

Configurare automată folosind Java

- Marcarea dependențelor: @Autowired

@Component

```
public class PizzaInMemoryRepository implements PizzaRepository {  
    private Validator<Pizza> validator;  
    public PizzaInMemoryRepository() { ... }  
    @Autowired(required=false)  
    public void setValidator(Validator<Pizza> val) {...}  
}
```

- Dacă nu există nici un bean care să satisfacă dependență, proprietatea va rămâne neinițializată.

```
public void save(Pizza p) {  
    if (validator!=null) {  
        //...  
    }  
}
```

Configurare automată folosind Java

- `@Component`, `@Autowired`: adnotări specifice frameworkului Spring
- Dependența codului de frameworkul Spring
- `@Named`, `@Inject` : adnotări din specificația *Java Dependency Injection*
- Pachetul `javax.inject`

```
import javax.inject.Inject;  
import javax.inject.Named;
```

`@Named`

```
public class PizzaRepositoryMock implements PizzaRepository {  
    private Validator<Pizza> valid;  
    @Inject  
    public PizzaRepositoryMock(Validator<Pizza> val){ ... }  
    //...  
}
```

- În majoritatea cazurilor sunt interschimbabile.

Configurare automată folosind Java

- **@Scope**: specificarea scopului (*implicit singleton*, prototype, request, session)
- @Component, @Bean

@Component

@Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)

```
public class ABean { ... }
```

@Bean

@Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)

```
public ABean abean() {  
    return new ABean();  
}
```


Crearea containerului Spring Java Autowire

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.AnnotationConfigApplicationContext;

public class StartApp{
    public static void main(String[] args){
        ApplicationContext context=new
        AnnotationConfigApplicationContext(PizzerieAutowireConfig.class);
        //obtinerea referintei catre un bean din container
        PizzaService repo= factory.getBean(PizzaService.class);
    }
}
```


Configurare Spring folosind JavaConfig

- Exemplu
 - TaskSpringJavaConfig

Configurare automată folosind Java

- Ambiguități.

`@Autowired`

```
public void setDessert(Dessert dessert) {  
    this.dessert = dessert;  
}
```

`@Component`

```
public class Cake implements Dessert { ... }
```

`@Component`

```
public class Cookies implements Dessert { ... }
```

`@Component`

```
public class IceCream implements Dessert { ... }
```

- **Care bean satisface dependența?**

`NoUniqueBeanDefinitionException: nested exception is`

`org.springframework.beans.factory.NoUniqueBeanDefinitionException:`

`No qualifying bean of type [com.desserteater.Dessert] is defined:`

`expected single matching bean but found 3: cake,cookies,iceCream`

Configurare automată folosind Java

- Ambiguități - Soluția 1 - `@Primary`.

`@Autowired`

```
public void setDessert(Dessert dessert) {  
    this.dessert = dessert;  
}
```

`@Component`

`@Primary`

```
public class Cake implements Dessert { ... }
```

`@Component`

```
public class Cookies implements Dessert { ... }
```

`@Component`

```
public class IceCream implements Dessert { ... }
```

- Soluția 2 - `@Qualifier`
- Soluția 3 - Anotare proprie

Configurare Spring folosind Autowire

- Exemplu
 - TaskSpringAutowire

Configurare XML vs. JavaConfig vs Autowire

XML	JavaConfig	Autowire
• Nu necesită modificarea codului sursă • Nu necesită recompilarea când apar modificări • Necesită învățarea unui nou limbaj (XML) • Se poate folosi când nu avem acces la tot codul sursă al aplicației • Nu se pot verifica tipurile bean-urilor și dependențele la compilare • Nu apar ambiguități	• Necesită recompilare când apar modificări • Nu necesită învățarea unui nou limbaj • Se verifică static tipurile • Se poate folosi când nu avem acces la tot codul sursă al aplicației	• Necesită recompilare când apar modificări • Nu necesită învățarea unui nou limbaj • Se verifică static tipurile • NU se poate folosi când nu avem acces la tot codul sursă al aplicației • Dependența codului sursă de Spring

Referințe Spring

- Documentația frameworkului Spring
<http://docs.spring.io/spring-framework/docs/current/spring-framework-reference/html/index.html>
- Craig Walls, *Spring in Action*, Fourth Edition, Ed. Manning, 2015 (sau versiuni mai noi: fifth edition, *sixth* edition)
- Alte tutoriale ...

Spring .NET

- <https://springframework.net/>
- Versiunea 3.0.0 (2021)
- IoC, DI

Medii de proiectare și programare

2024-2025

Curs 5

Conținut curs 5

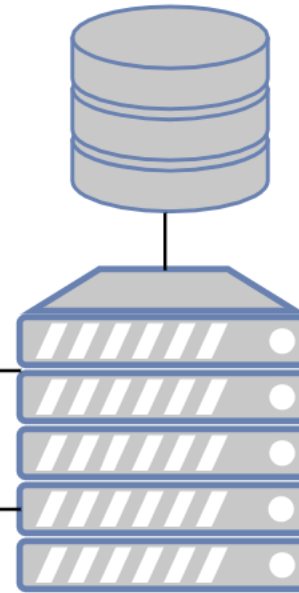
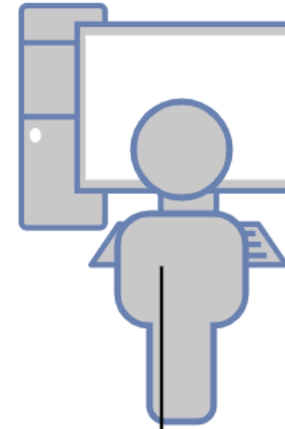
- Aplicații client-server
 - Șablonul Proxy
 - Data transfer object
- Networking și threading în Java
 - Exemplu Mini-Chat
- Networking și threading în C# (curs 6)

Aplicații client-server

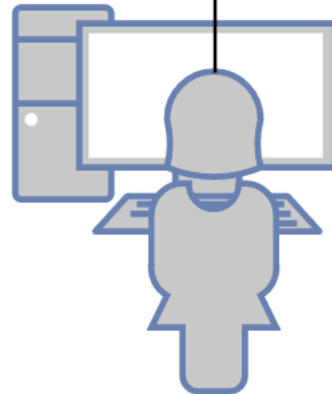
Utilizator 3



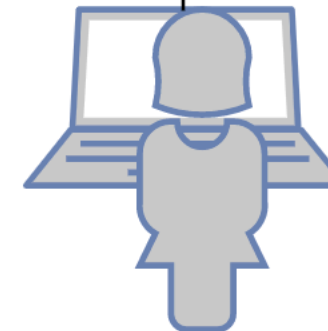
Utilizator 4



Server



Utilizator 1



Utilizator 2

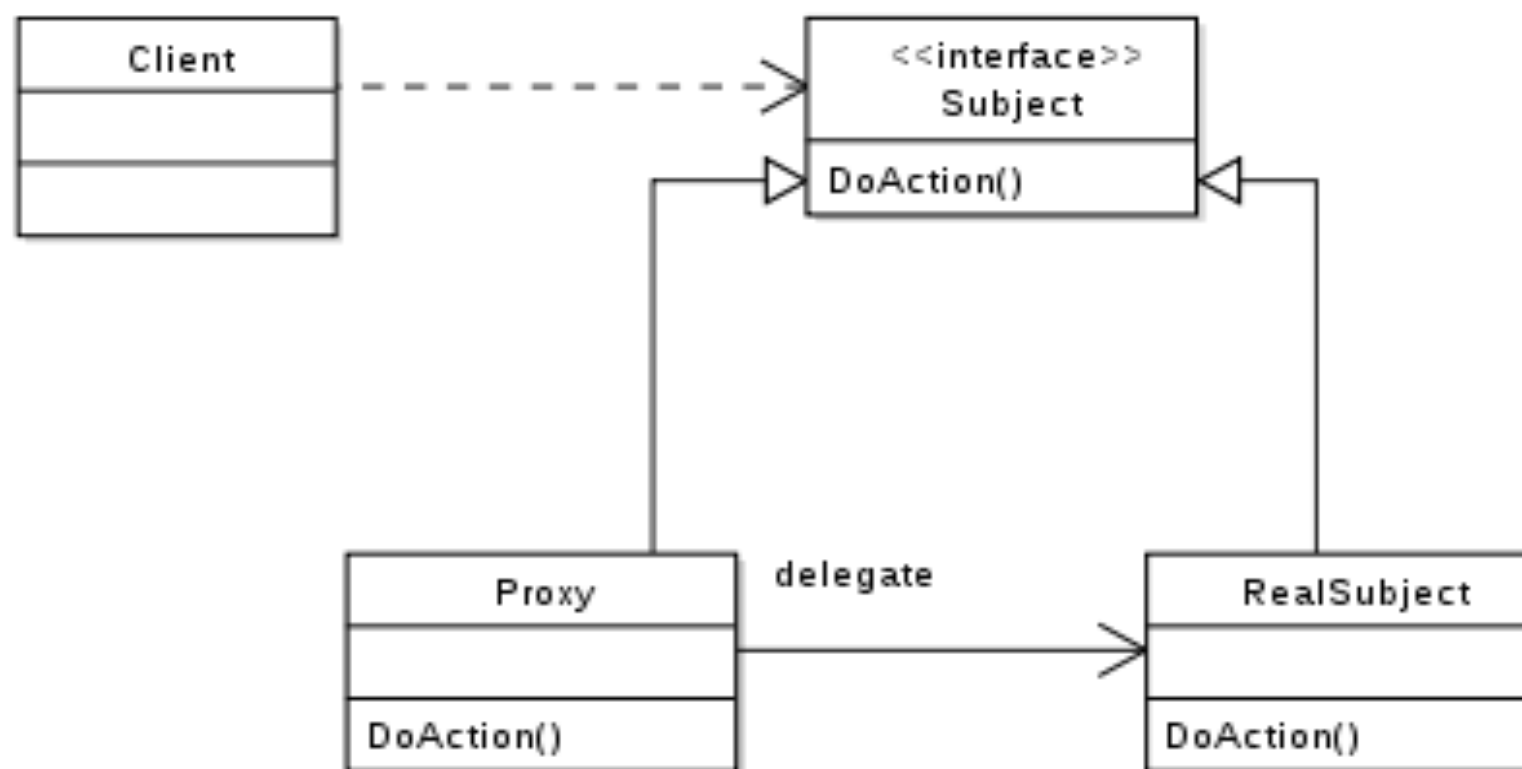
Mini-chat

- Proiectați și implementați o aplicație client-server pentru un mini-chat având următoarele funcționalități:
 - *Login*. După autentificarea cu succes, o nouă fereastră se deschide în care sunt afișați toți prietenii *online* ai utilizatorului și o listă cu mesajele trimise/primate de utilizator. De asemenea, toți prietenii online văd în lista lor că utilizatorul este *online*.
 - *Trimiterea unui mesaj*. Un utilizator poate trimite un mesaj text unui prieten care este online. După trimiterea mesajului, prietenul vede automat mesajul în fereastra lui.
 - *Logout*. Toți prietenii online ai utilizatorului văd în lista lor că utilizatorul nu mai este *online*.

Exemplu Java

Șablonul Proxy

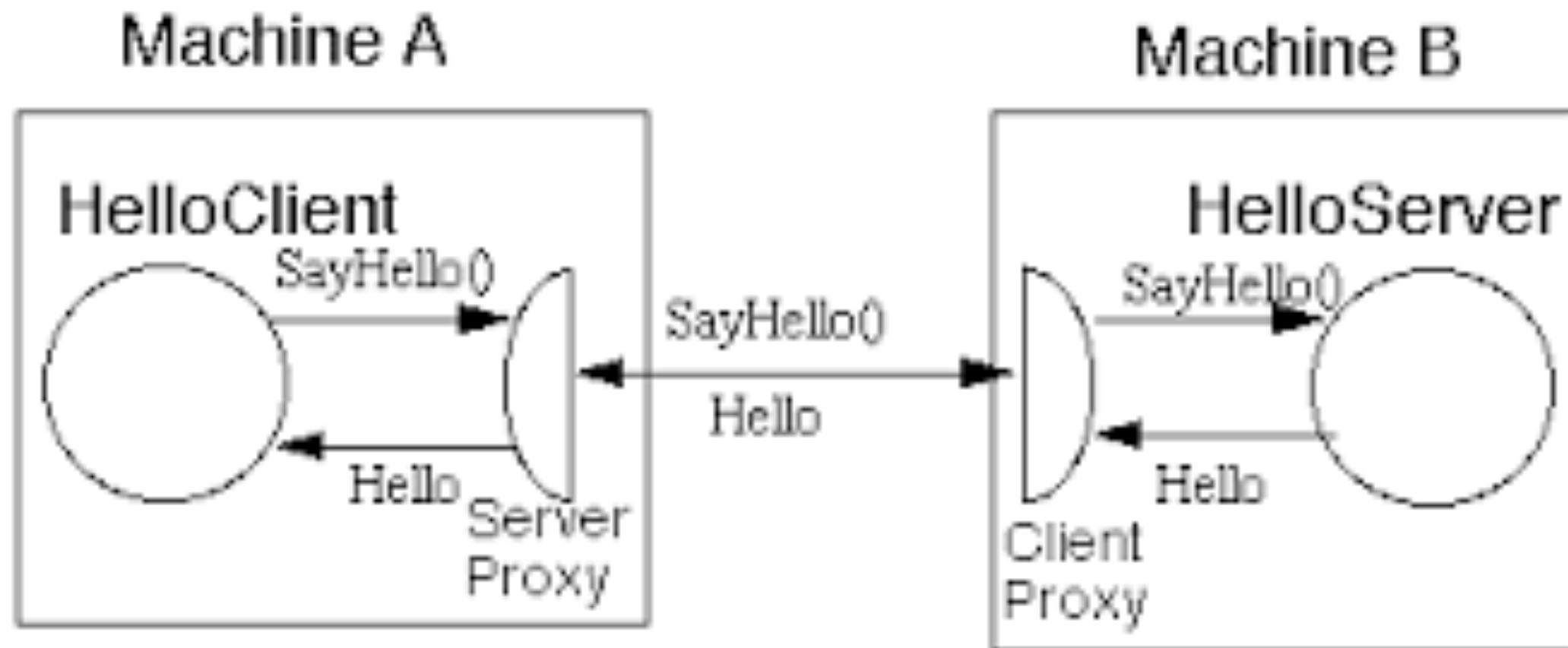
- Asigură pentru un obiect existent, un surogat sau un înlocuitor în scopul controlării accesului la acesta.
- Înlocuitorul poate fi:
 - *Proxy la distanță* (eng. *remote proxy*) - obiect în alt spațiu de adresă,
 - *Proxy virtual* (eng. *virtual proxy*) - un obiect mare din memorie,
 - *Proxy de protecție* - controlează accesul la obiectul original,
 - etc.



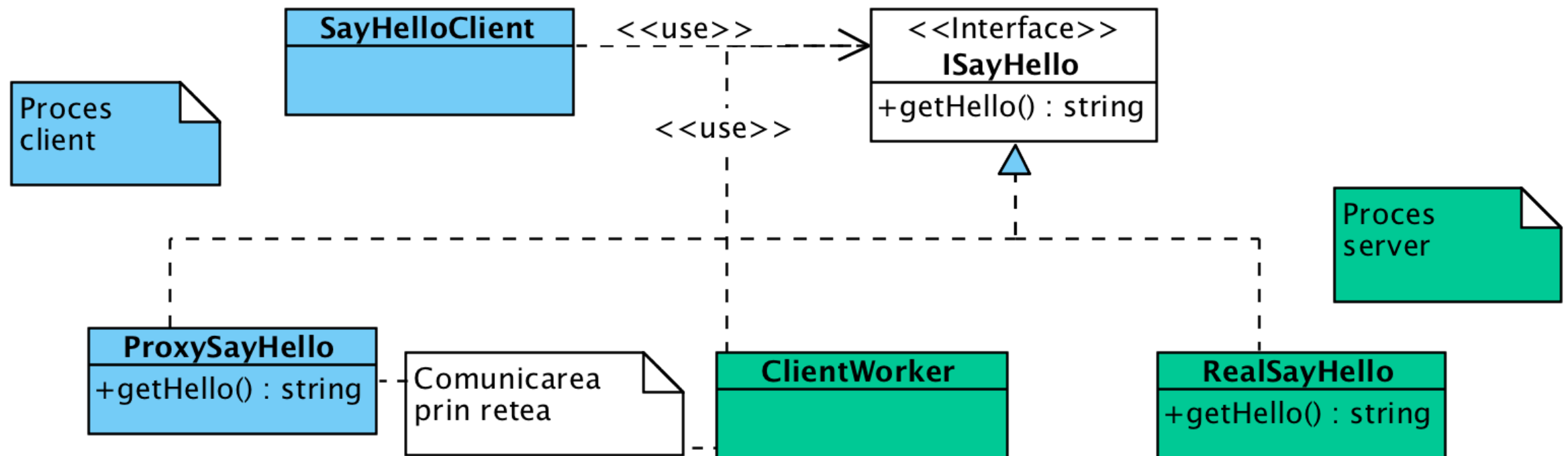
Șablonul *Remote Proxy*

- ***Remote Proxy*** oferă un înlocuitor local pentru un obiect aflat în alt spațiu de adresă/memorie.
- Înlocuitorul este responsabil cu codificarea unei cereri, a parametrilor și trimiterea lor către obiectul real aflat într-un spațiu de adresă diferit.
- *Clientul* cererii crede că comunică cu obiectul real, dar este un proxy între ei.
- Proxy-ul transformă cererile clientului în cereri la distanță, obține rezultatul cererii și îl transmite clientului.

Şablonul Remote Proxy



Șablonul Remote Proxy

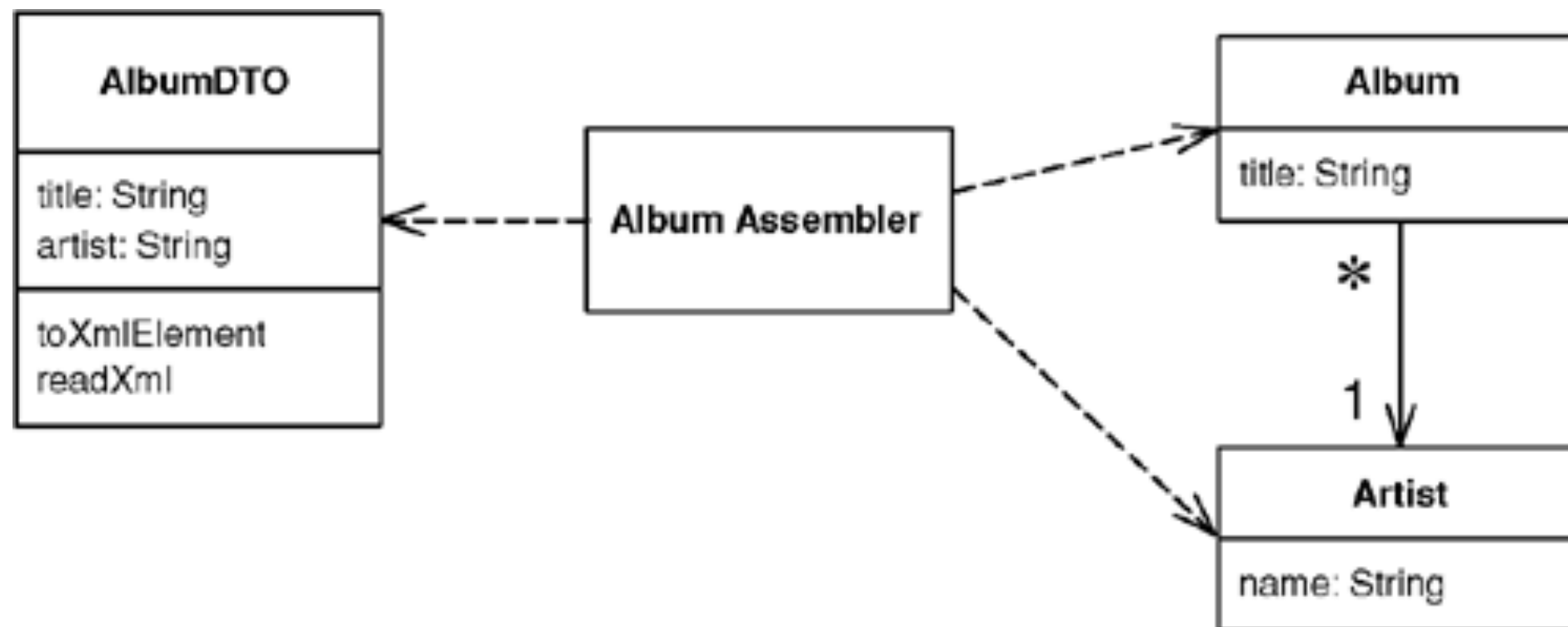


Șablonul *Data Transfer Object* (DTO)

- Un **DTO** este *un obiect care conține informația ce trebuie transmisă între unul sau mai multe procese pentru a reduce numărul de apeluri* (dintre procese).
- **Fiecare apel de metodă remote este costisitor**, de aceea numărul de apeluri ar trebui redus și mai multă informație ar trebui transmisă la un apel.
- O soluție posibilă este de a folosi mai mulți parametri:
 - dificil de programat
 - în unele cazuri nu este posibil (ex., în Java o metodă poate returna o singură valoare).
- **Soluția: crearea unui DTO (*Data Transfer Object*)** care păstrează toată informația necesară unui apel. De obicei obiectul este serializabil (binar, XML, etc.) pentru a putea fi transmis prin rețea.
- Un alt obiect este responsabil cu conversia datelor din model într-un DTO și invers.

Șablonul Data Transfer Object

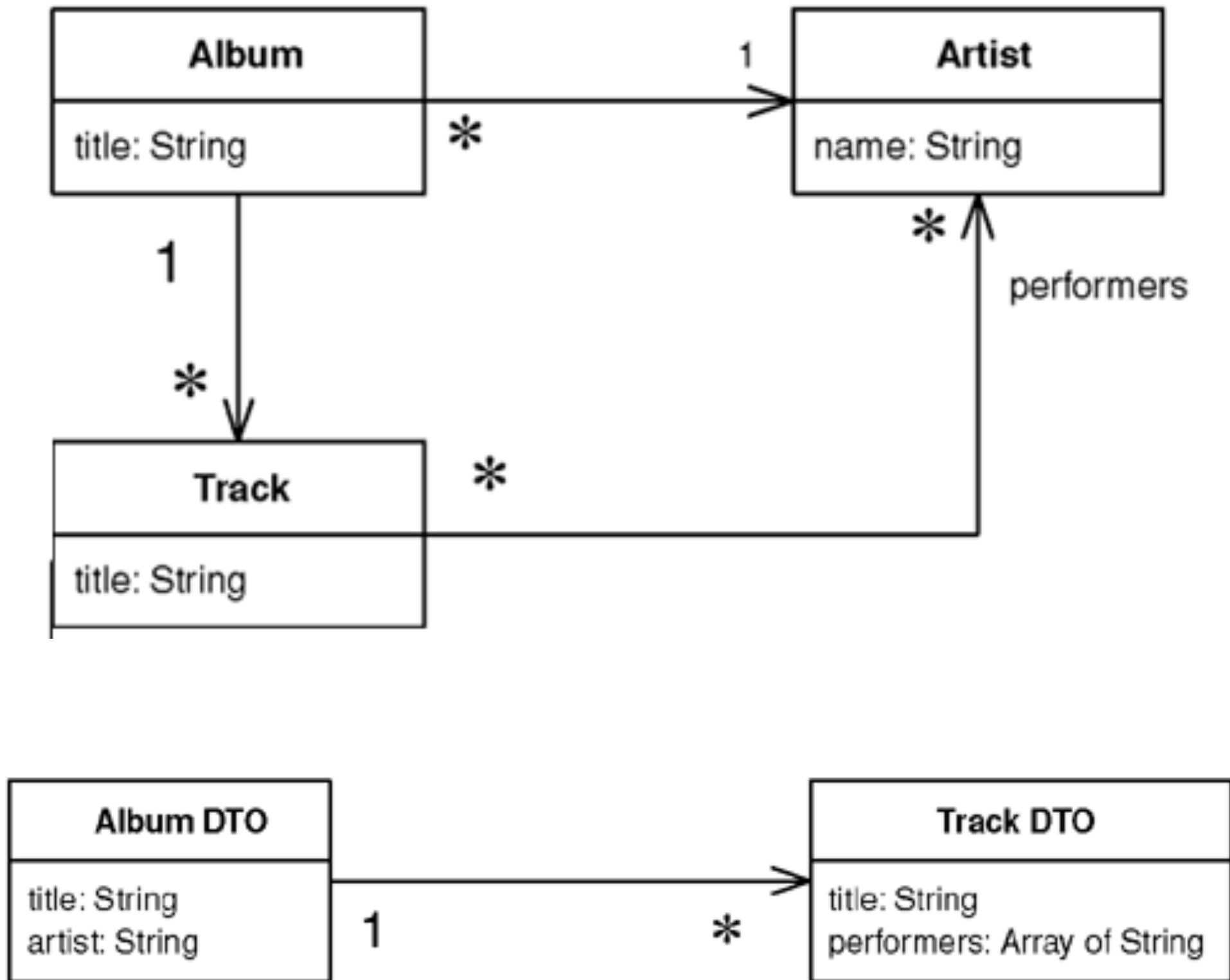
- Un **DTO** conține, de obicei, *multe attribute și metode de tip get/set pentru acestea*.
- Când un obiect remote are nevoie de date, cere DTO-ul corespunzător. **DTO poate să conțină mai multă informație decât este necesară la acel apel**, dar ar trebui să conțină toată informația de care va avea nevoie obiectul remote o perioadă.
- Un **DTO** conține de obicei informație provenind de la mai multe obiecte din model.



Șablonul Data Transfer Object

- Un DTO ar trebui folosit ori de câte ori este necesară transmiterea mai multor date între două procese într-un singur apel de metodă.
- *Alternative:*
 - De a folosi metode de tip set/get cu mai mulți parametri transmiși prin referință.
 - Multe limbaje (ex. Java) permit returnarea unei singure valori.
 - Alternativa poate fi folosită pentru actualizări (metode de tip set), dar nu poate fi folosită pentru a obține date (metode de tip get).
 - Folosirea unei reprezentări sub formă de string.
 - Totul va fi cuplat cu reprezentarea sub formă de string (poate fi costisitoare).

Data Transfer Object - Exemplu



Data Transfer Object - Exemplu

```
class AlbumAssembler{
    public AlbumDTO writeDTO(Album subject) {
        AlbumDTO result = new AlbumDTO();
        result.setTitle(subject.getTitle());
        result.setArtist(subject.getArtist().getName());
        writeTracks(result, subject);
        return result;
    }
    private void writeTracks(AlbumDTO result, Album subject) {
        List<TrackDTO> newTracks = new ArrayList<TrackDTO>();
        for(Track track: subject.getTracks()){
            TrackDTO newDTO = new TrackDTO();
            newDTO.setTitle(track.getTitle());
            writePerformers(newDTO, track);
            newTracks.add(newDTO);
        }
        result.setTracks(newTracks.toArray(new TrackDTO[newTracks.size()]));
    }
}
```

Data Transfer Object - Exemplu

```
private void writePerformers (TrackDTO dto, Track subject) {  
    List<String> result = new ArrayList<String>();  
    for (Artist artist: subject.getPerformers()) {  
        result.add(artist.getName());  
    }  
    dto.setPerformers(result.toArray(new String[result.size()]));  
}
```


Networking în Java

- `java.net` - pachetul conține clase pentru comunicarea TCP/UDP prin rețea.
- TCP: `Socket` și `ServerSocket`.
- UDP: `DatagramPacket`, `DatagramSocket` și `MulticastSocket`.
- Clasa `InetAddress` reprezintă o adresă IP:
 - `Inet4Address`: pentru adrese IPv4 (32 bits).
 - `Inet6Address`: pentru adrese IPv6 (128 bits).

```
InetAddress localhost=InetAddress.getLocalHost();
```

```
InetAddress googAdr=InetAddress.getByName("www.google.com");
```

- `InetSocketAddress` asociere între o adresă IP și un port:

```
InetSocketAddress(InetAddress addr, int port) ;
```

```
InetSocketAddress(String hostname, int port);
```


Networking in Java

- **ServerSocket** reprezintă **clasa** corespunzătoare **serverului care așteaptă conexiuni TCP**.

- Constructori/Metode:

```
public ServerSocket(int port) throws BindException, IOException
```

```
public ServerSocket( ) throws IOException //not bind yet, since Java 1.4
```

```
//binds a server to a port
```

```
public void bind(SocketAddress endpoint) throws IOException
```

```
//blocks and waits for clients
```

```
public Socket accept( ) throws IOException
```

```
//closes the server
```

```
public void close() throws IOException
```

Networking in Java

```
ServerSocket server=null;
try{
    server=new ServerSocket(5555) ;
    while(keepProcessing){
        Socket client=server.accept() ;
        //processing code
    }
}catch(IOException ex){
    //...
}finally{
    if(server!=null){
        try{
            server.close() ;
        }catch(IOException ex){...}
    }
}
```

Networking în Java

- `java.net.Socket` deschide o conexiune TCP din partea clientului.

```
public Socket(String host, int port) throws UnknownHostException,  
                                           IOException
```

```
public Socket(InetAddress host, int port) throws IOException
```

- Metode:

```
public int getPort()
```

```
public InputStream getInputStream() throws IOException
```

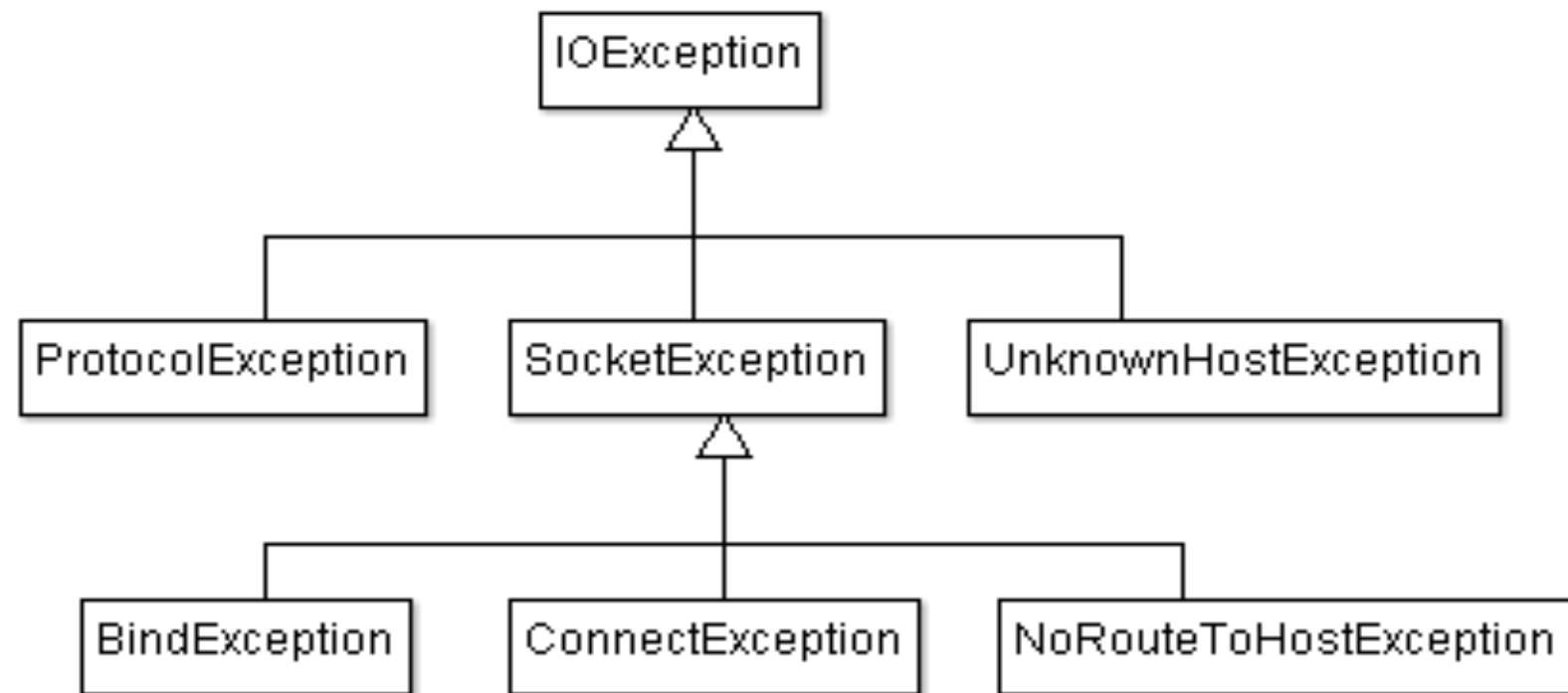
```
public OutputStream getOutputStream() throws IOException
```

```
public void close() throws IOException
```

Networking in Java

```
try (Socket connection=new Socket("172.30.106.5", 5555)) {  
    //processing code  
  
}catch(UnknownHostException e) {  
    //...  
}catch(IOException e) {  
    //...  
}
```

Excepții



Threading în Java

- *Două modalități de definire a unui thread:*
 - Extinderea clasei **Thread** și redefinirea metodei **run**.
 - Implementarea interfeței **Runnable** și definirea metodei **run**.
- Crearea unui thread se face prin intermediul clasei **Thread**

```
public Thread()  
public Thread(Runnable target)
```
- Pornirea execuției unui thread se face prin apelul metodei **start** din clasa **Thread**:

```
public void start()
```

Sincronizarea threadurilor

- Instrucțiunea `synchronized`

```
synchronized(locker_obj) {  
    //code to execute  
}
```

- Sincronizarea unei metode:

```
public synchronized void methodA();
```

- **Yielding**: *un thread renunța la CPU alocat și permite execuția altui thread:*

```
public static void yield( );
```

```
public void run( ) {  
    while (true) {  
        // Time and CPU consuming thread's work...  
        Thread.yield( );  
    }  
}
```


Utilități Java Concurrency

- Java 5 a introdus **utilități pentru concurență** - un framework extensibil care permite **crearea containerelor de thread-uri și cozi sincronizate** (eng. *blocking queues*):
 - **`java.util.concurrent`**: Tipuri utile în programarea concurentă (ex. executors)
 - **`java.util.concurrent.atomic`**: Programare concurentă avansată
 - **`java.util.concurrent.locks`**: Mecanisme de blocare avansate, mai performante decât notify/wait.

Taskuri Java

- Un obiect *task Java* este un obiect a cărui clasă implementează interfața `java.lang.Runnable` (*taskuri runnable*) sau interfața `java.util.concurrent.Callable` (*taskuri callable*).

```
public interface Runnable{  
    void run()  
}
```

```
public interface Callable<V>{  
    V call() throws Exception  
}
```

- Metoda `call()` poate *returna o valoare și poate arunca excepții* (checked).

Execuția taskurilor Java

- **Interfața Executor** - execuția taskurilor runnable:

```
public interface Executor{  
    void execute(Runnable command)  
}
```

- **ScheduledThreadPoolExecutor, ThreadPoolExecutor**

- *Dezavantaje:*

- Se axează doar pe **Runnable**. Metoda **run()** nu returnează nici o valoare. Este dificilă returnarea unei valori ca și rezultat al execuției taskului.
- Nu oferă posibilitatea monitorizării progresului execuției unui task runnable care se execută (se execută încă?, anulat? execuția s-a încheiat?)
- Nu poate executa mai multe taskuri.
- Nu oferă posibilitatea opririi unui executor.

ExecutorService

- `java.util.concurrent.ExecutorService` soluția pentru problemele apărute la interfața `Executor`.
- Este implementat folosind un container de threaduri (eng. *thread pool*).

```
public interface ExecutorService extends Executor {  
  
    void shutdown();  
  
    List<Runnable> shutdownNow();  
  
    <T> Future<T> submit(Callable<T> task);  
  
    <T> Future<T> submit(Runnable task, T result);  
  
    <T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks);  
  
    <T> T invokeAny(Collection<? extends Callable<T>> tasks);  
  
    //alte metode  
}
```

- `ScheduledThreadPoolExecutor`, `ThreadPoolExecutor`

Executorul trebuie oprit după terminarea execuției, altfel aplicația nu își va încheia execuția.

Interfața Future

- Un **obiect** de tip **Future** reprezintă **rezultatul unui calcul asincron**.
- **Rezultatul** este numit *future* pentru că **de obicei nu va fi disponibil decât la un moment în viitor**.
- Are metode pentru: **anularea execuției unui task**, **obținerea rezultatului execuției**, **determinarea dacă un task și-a încheiat execuția**.

```
public interface Future<V>{

    boolean isCancelled();

    boolean isDone();

    boolean cancel(boolean mayInterruptIfRunning)

    V get() throws InterruptedException, ExecutionException;

    //alte metode ...

}
```

Clasa Executors

- Clasa **Executors** conține metode statice care returnează obiecte de tip **ExecutorService**:
 - **newFixedThreadPool(int nThreads) : ExecutorService**
 - **newSingleThreadExecutor() : ExecutorService**
 - **newCachedThreadPool() : ExecutorService**
 - **newWorkStealingPool() : ExecutorService**

Colecții concurente

- Colecții folosite în programarea concurentă.
- Începând cu versiunea 1.5
- Interfața **BlockingQueue**:
 - **Coadă** - conține metode care așteaptă ca coadă să devină nevidă la scoaterea unui element, respectiv așteaptă eliberarea spațiului la adăugarea unui element.
 - Implementările BlockingQueue au fost proiectate și implementate pentru a fi folosite în situații de tip producător-consumator.
 - **ArrayBlockingQueue**, **LinkedBlockingQueue**, **PriorityBlockingQueue**, etc.
- Interfața **BlockingDeque**:
 - Extinde **BlockingQueue** și oferă suport pentru operații de tip FIFO și LIFO.
 - **LinkedBlockingDeque**
- Interfața **ConcurrentMap**:
 - Subinterfață a **java.util.Map**
 - **ConcurrentHashMap**, **ConcurrentSkipListMap**.

Exemplu BlockingQueue

- Producător-Consumator simplu cu BlockingQueue

//ambele threaduri au referință la obiectul *messages*

//inițializarea

```
private BlockingQueue<String> messages=new LinkedBlockingQueue<String>();
```

//Producator

```
try {  
    messages.put(message);  
} catch (InterruptedException e) {  
    e.printStackTrace();  
}
```

//Consumator

```
String message = messages.take();
```


Actualizare GUI

- Interfețele grafice (JavaFX, Swing) folosesc obiecte de tip Component.
- Aceste obiecte ***pot fi modificate (actualizate, șterse, etc) doar de threadul care le-a creat.***
- Nerespectarea acestei reguli are rezultate neașteptate sau aruncă excepții.

```
Platform.runLater(new Runnable() {                //JavaFX
    @Override
    public void run() {
        //codul care modifica informatia de pe interfata grafica
        label.setText("New text ...");
    }
});
```

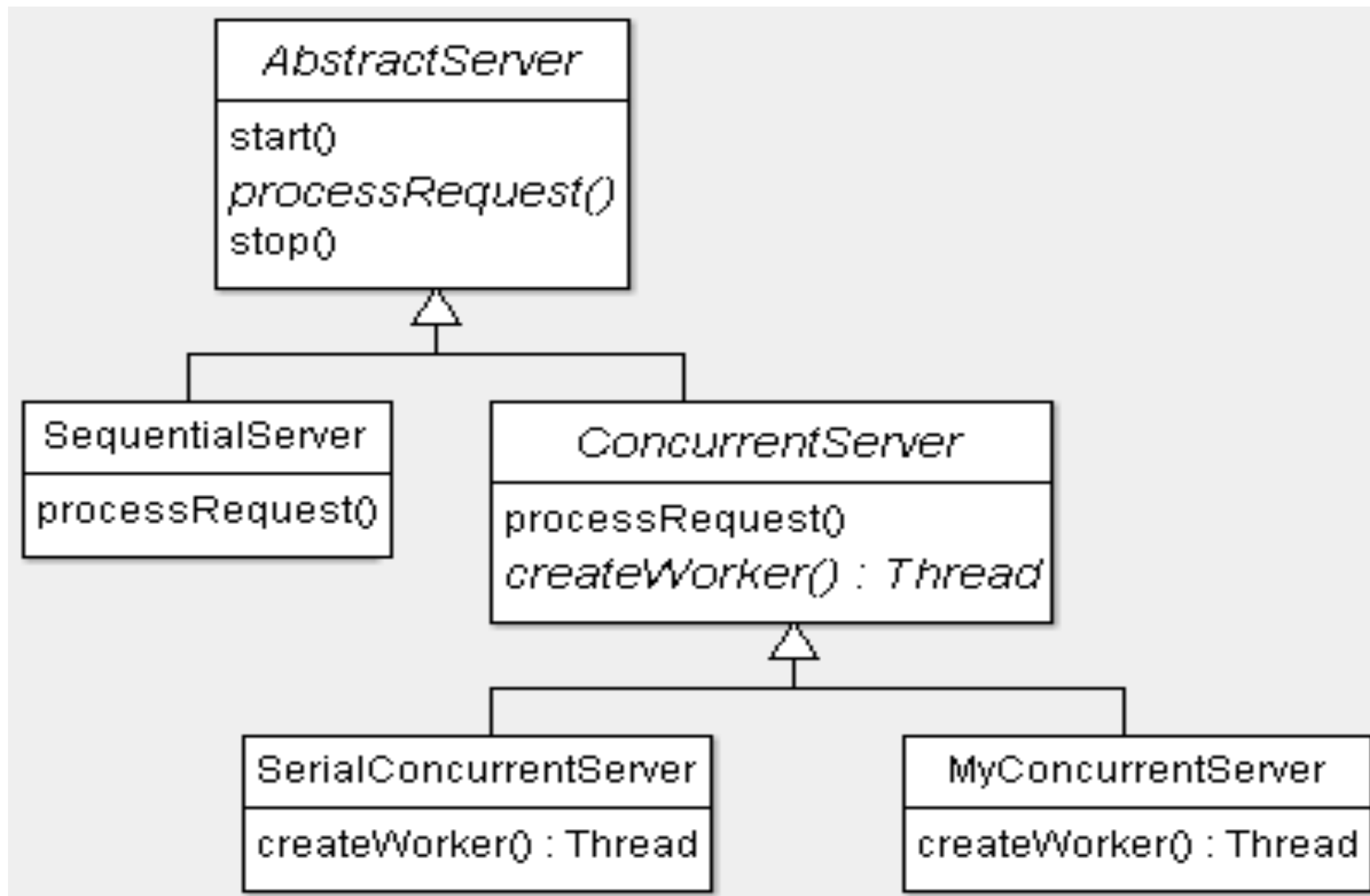
//sau, folosind funcții lambda

```
Platform.runLater(() -> {                        //JavaFX
    //codul care modifica informatia de pe interfata grafica
    label.setText("New text ...");
});
```


Exemplu Java

- O aplicație simplă client/server:
 - Serverul așteaptă conexiuni.
 - Clientul se conectează la server și îi trimite un text.
 - Serverul returnează textul scris cu litere mari, la care adaugă data și ora la care a fost primit textul.

Server Template

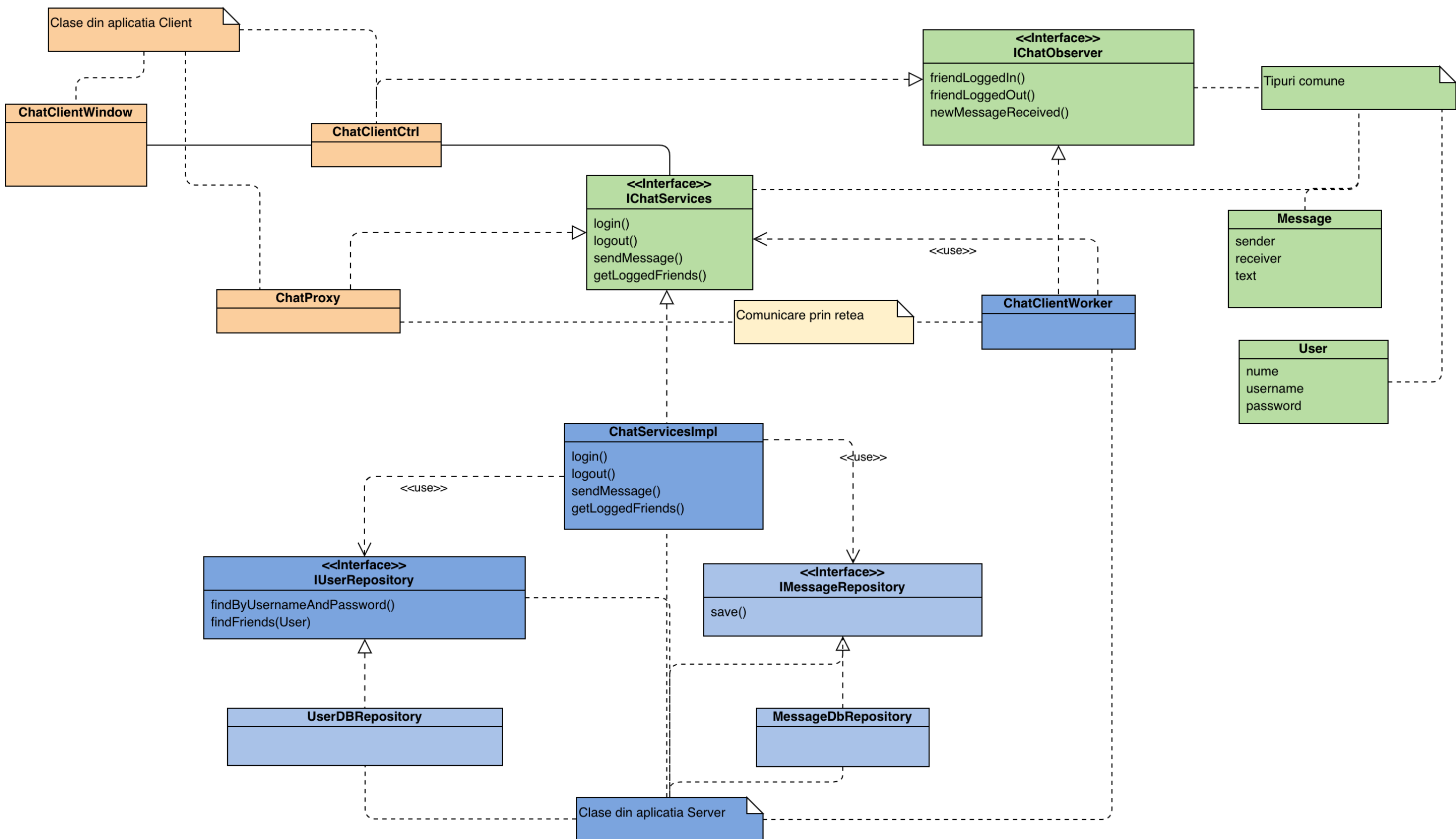


Mini-Chat

- Proiectare (diagrame)
- Implementare Java

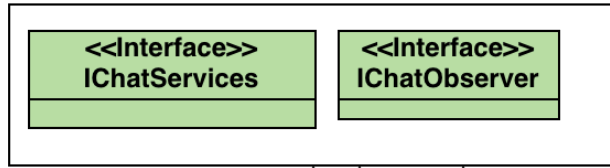
Mini-Chat

- Proiectare (diagrame)

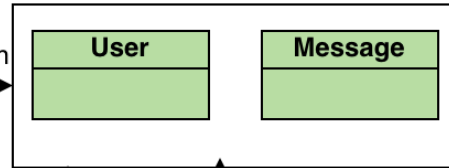


Proiecte

ChatServices



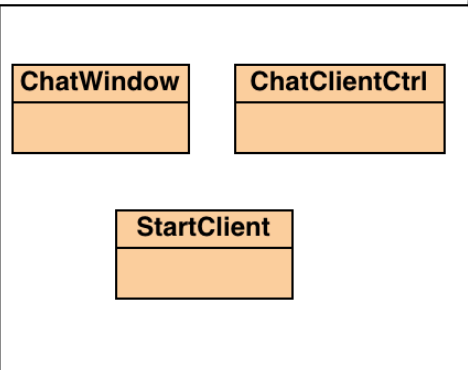
ChatModel



depends on

depends on

ChatClient



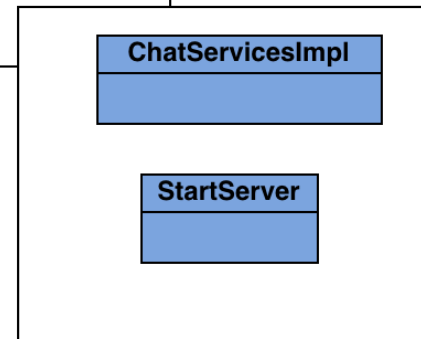
depends on

depends on

depends on

depends on

ChatServer

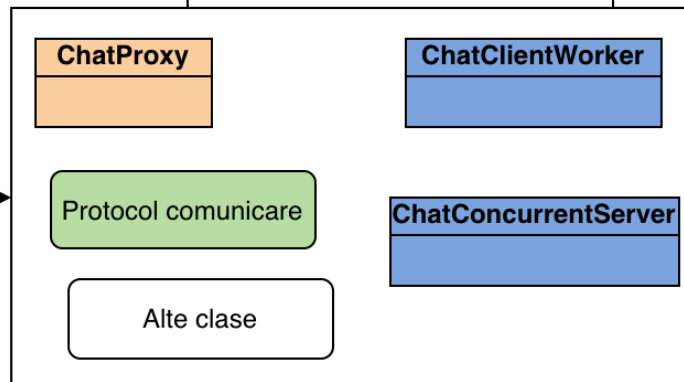


depends on

depends on

depends on

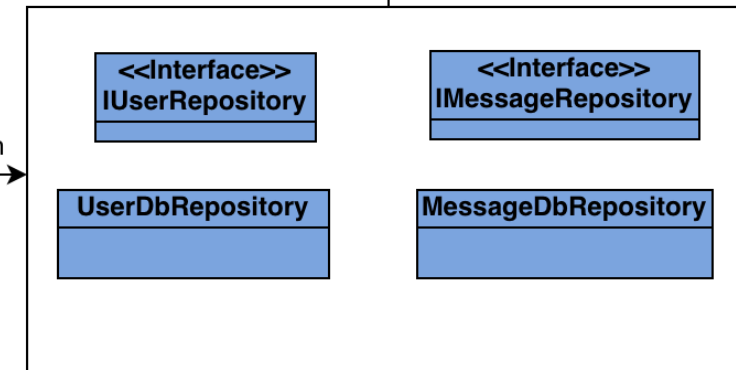
ChatNetworking

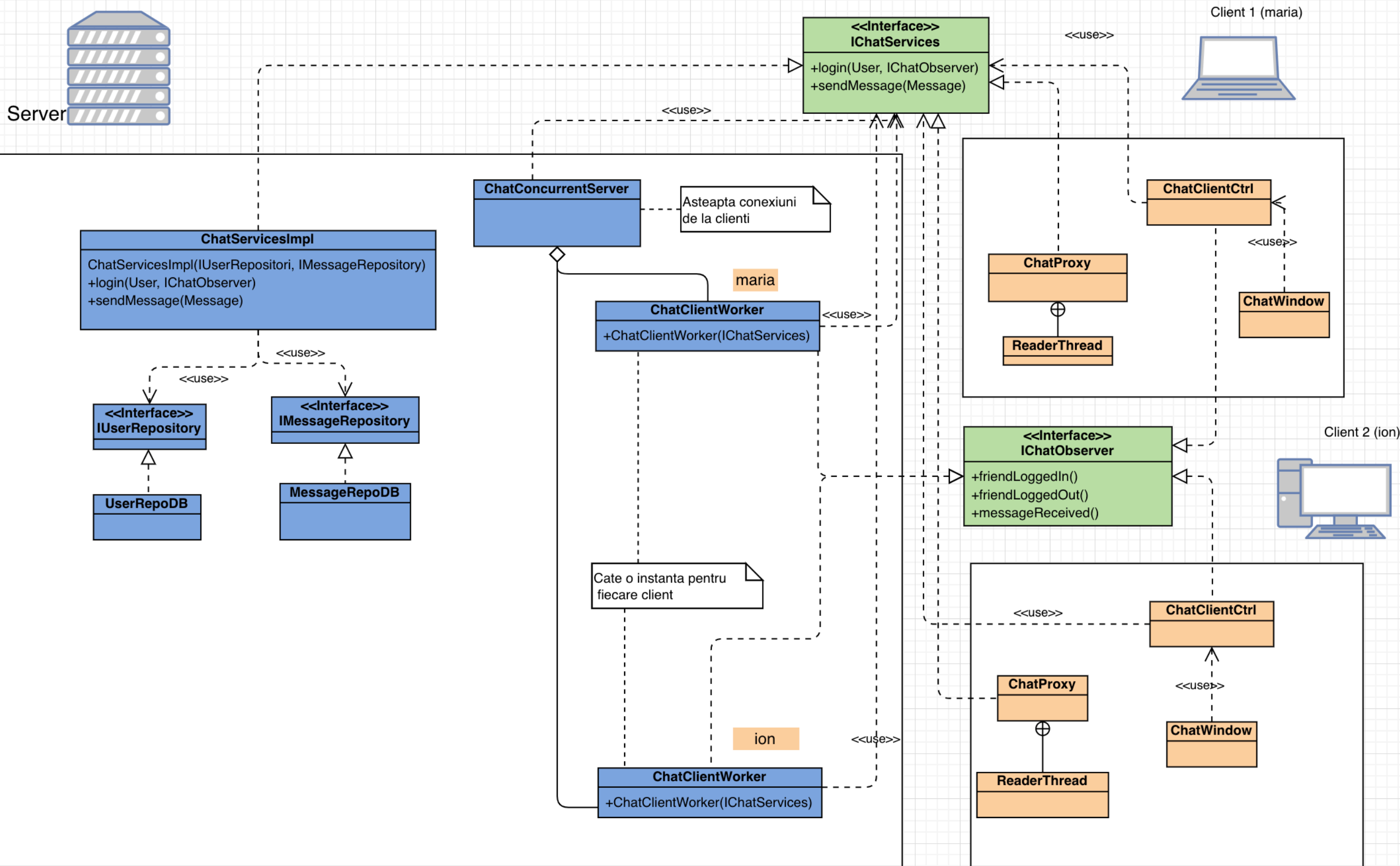


depends on

depends on

ChatPersistence





Medii de proiectare și programare

2024-2025

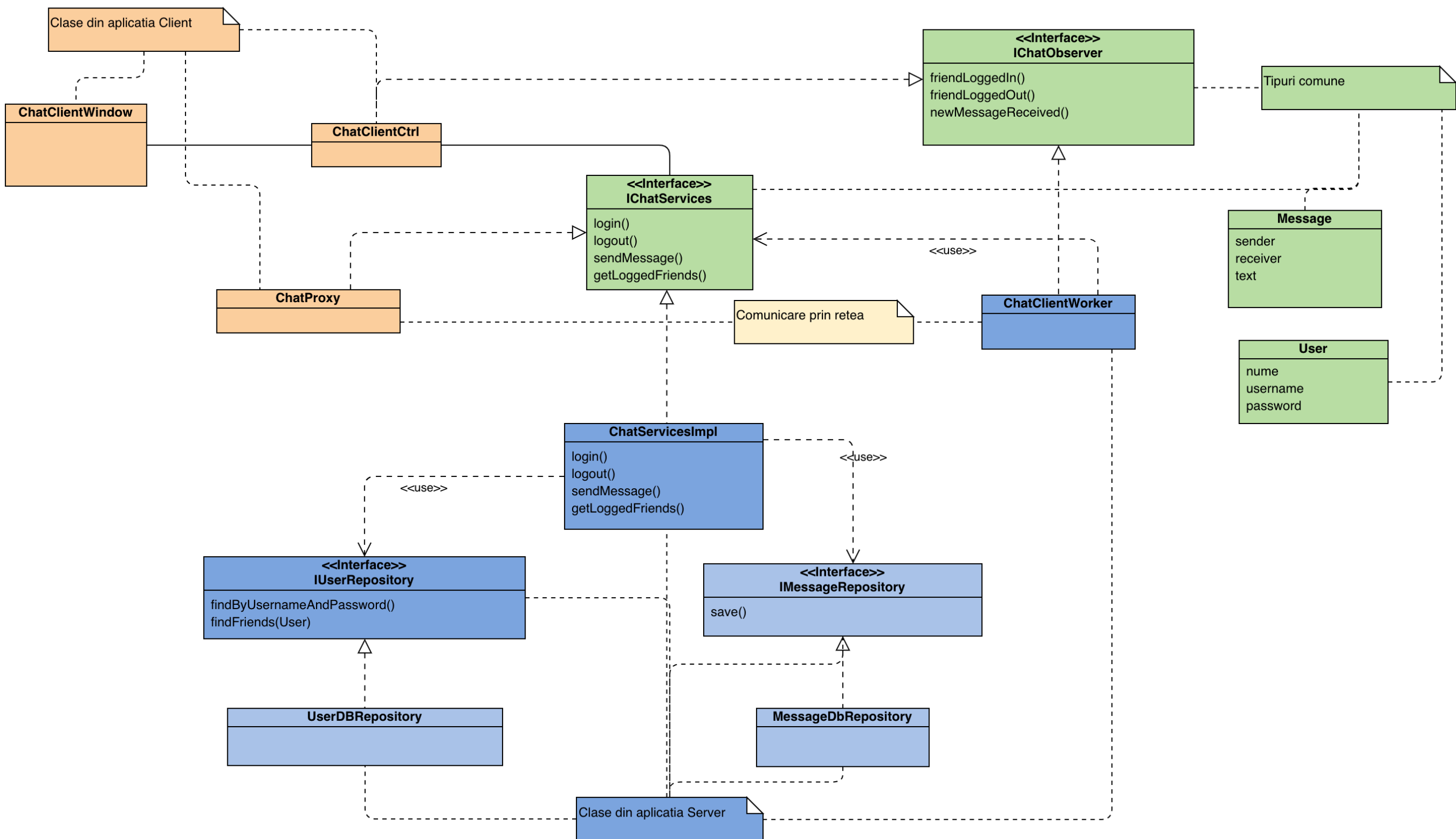
Curs 6

Conținut curs 6

- Exemplu Mini-Chat (networking - Java)
- Networking si threading in C#
- Exemplu Mini-Chat (networking - C#)

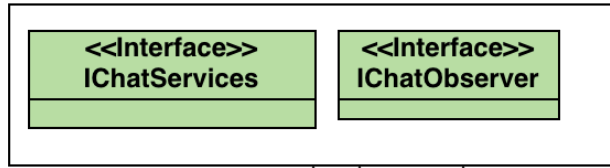
Mini-Chat

- Proiectare (diagrame)
- Implementare Java

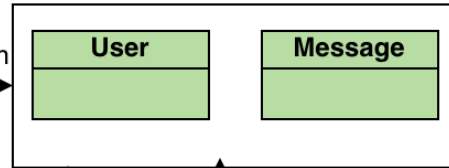


Proiecte

ChatServices



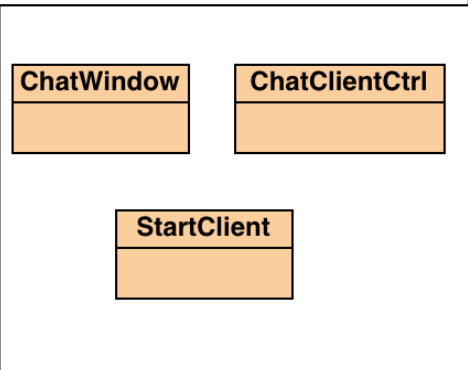
ChatModel



depends on

depends on

ChatClient



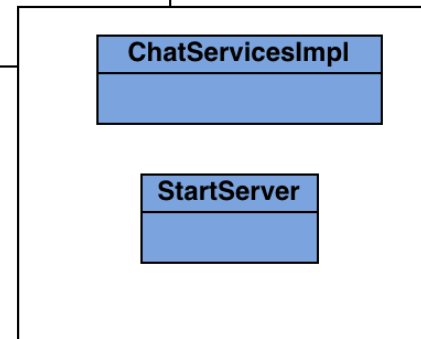
depends on

depends on

depends on

depends on

ChatServer

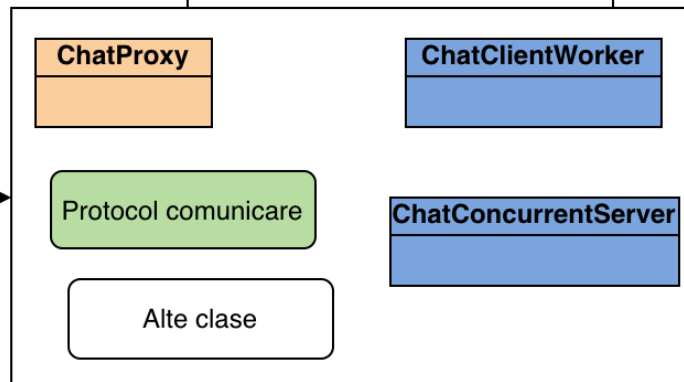


depends on

depends on

depends on

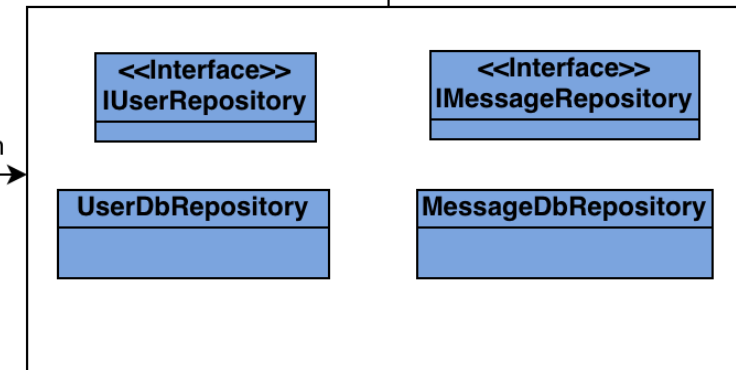
ChatNetworking

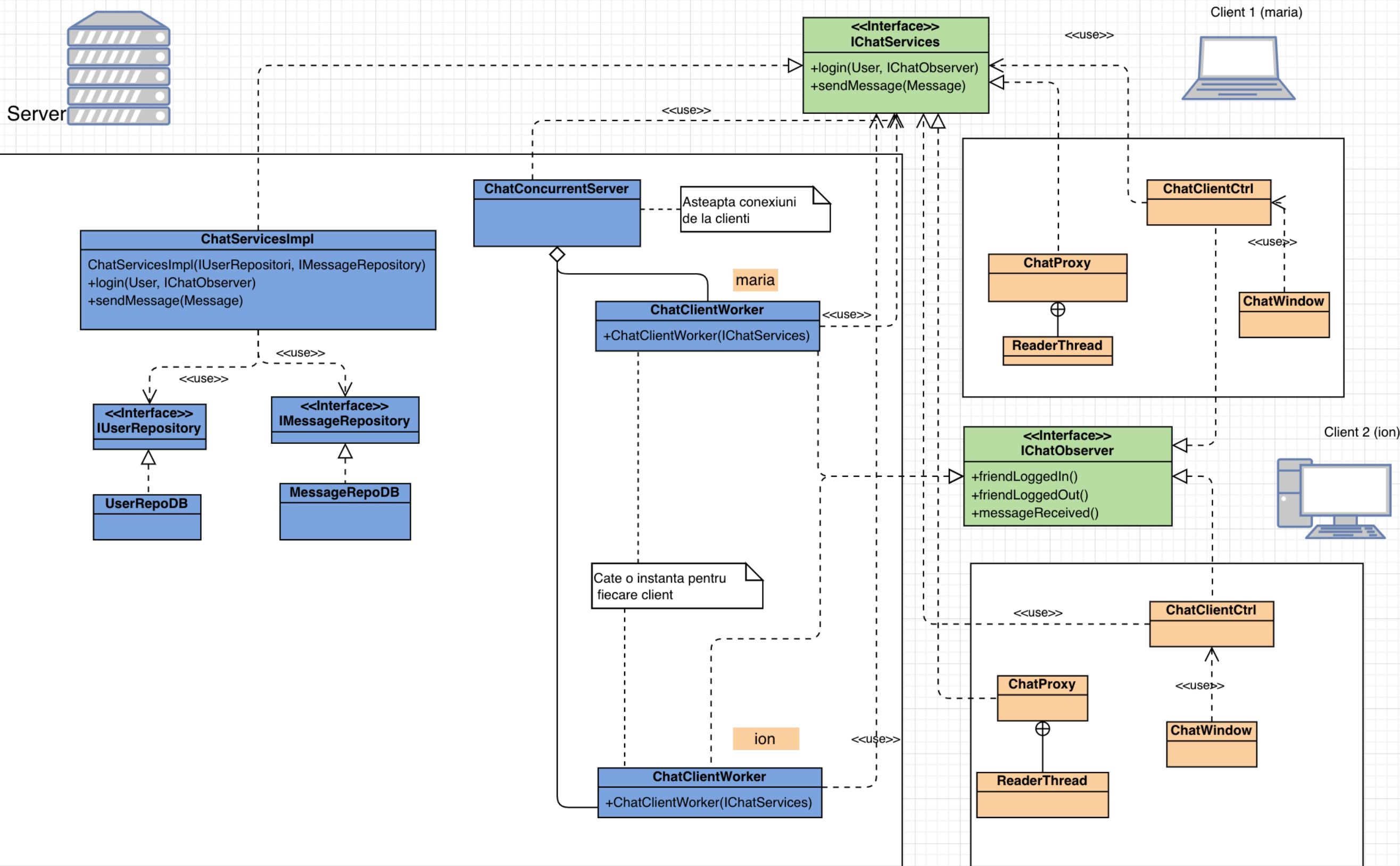


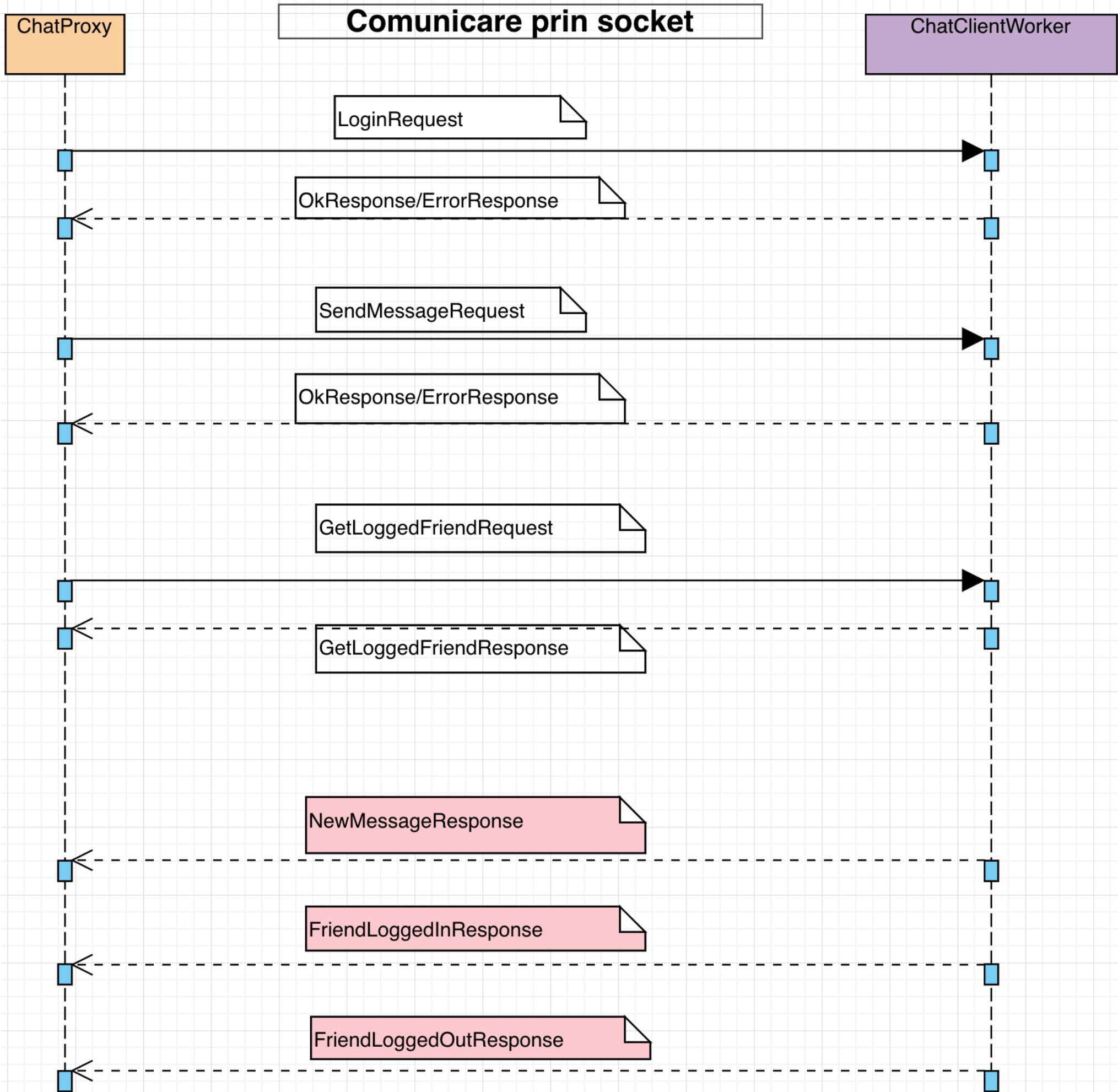
depends on

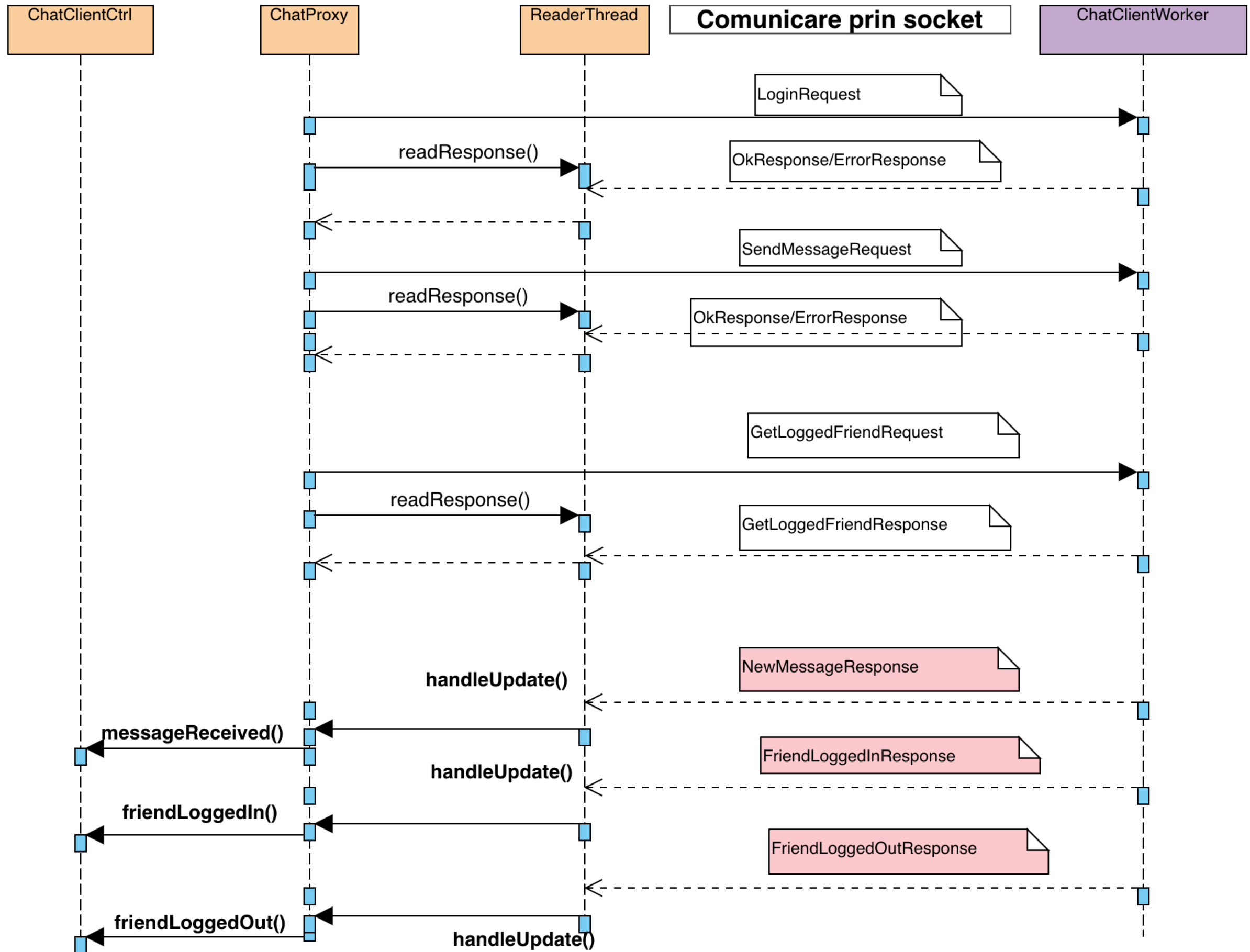
depends on

ChatPersistence

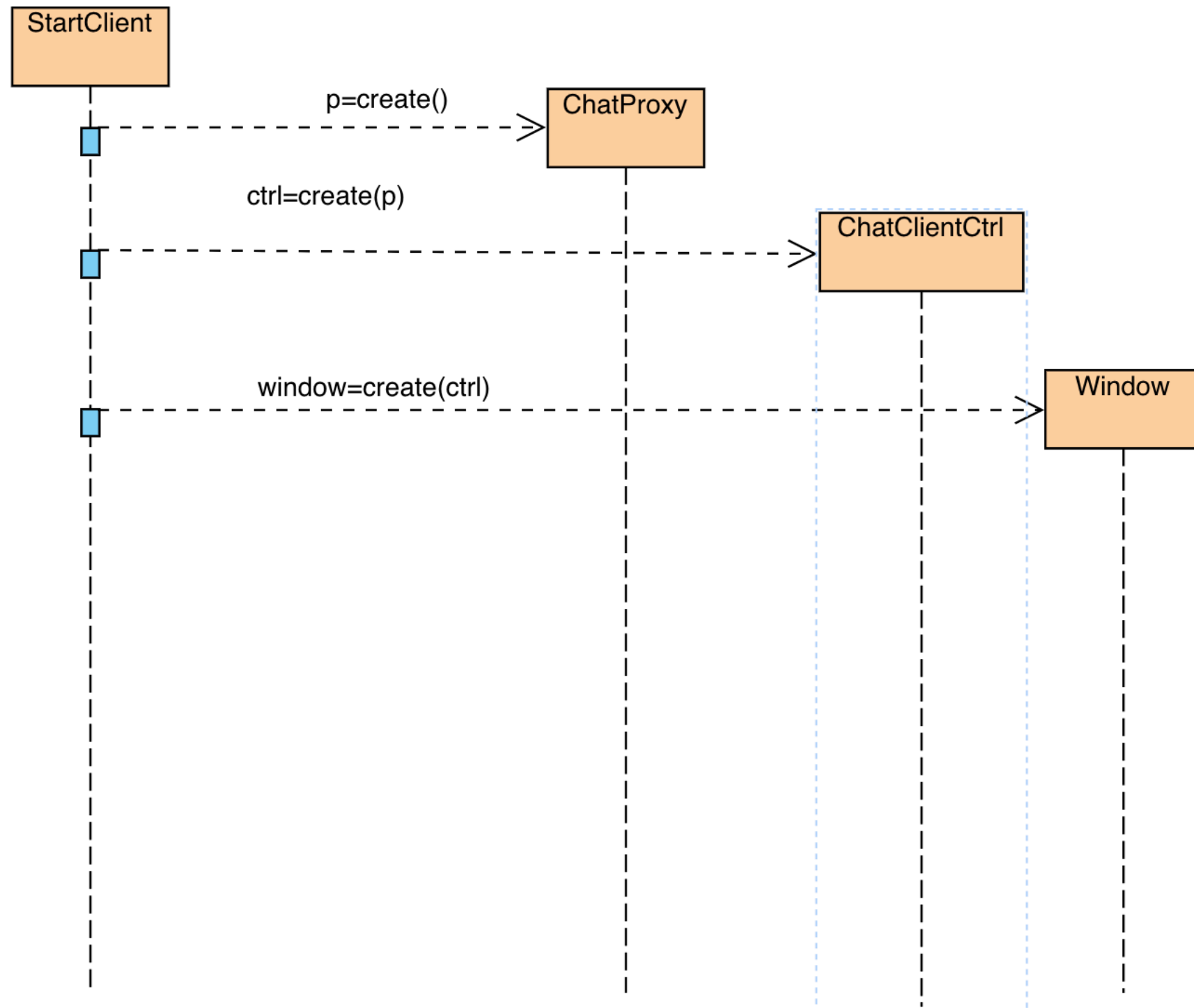




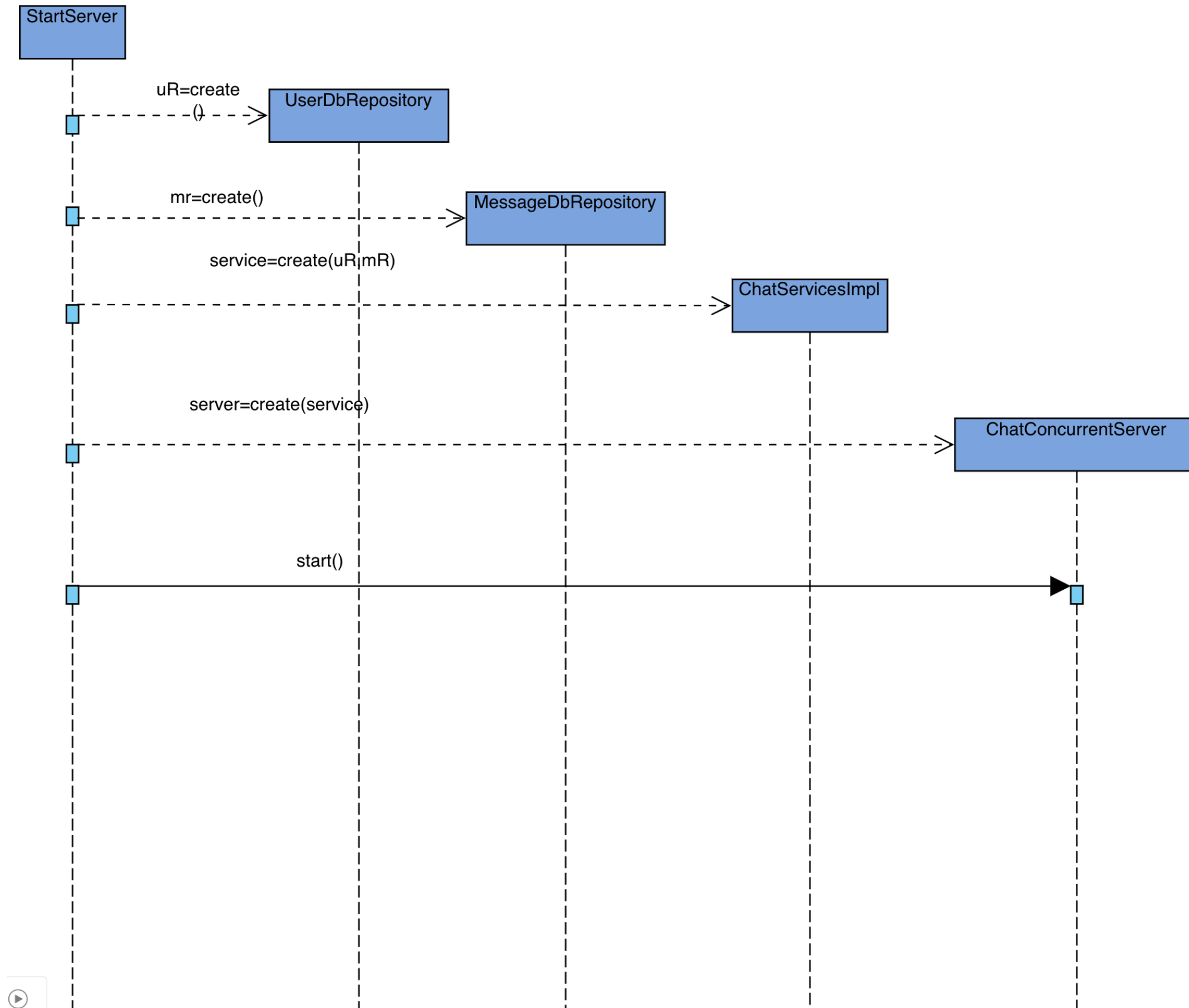




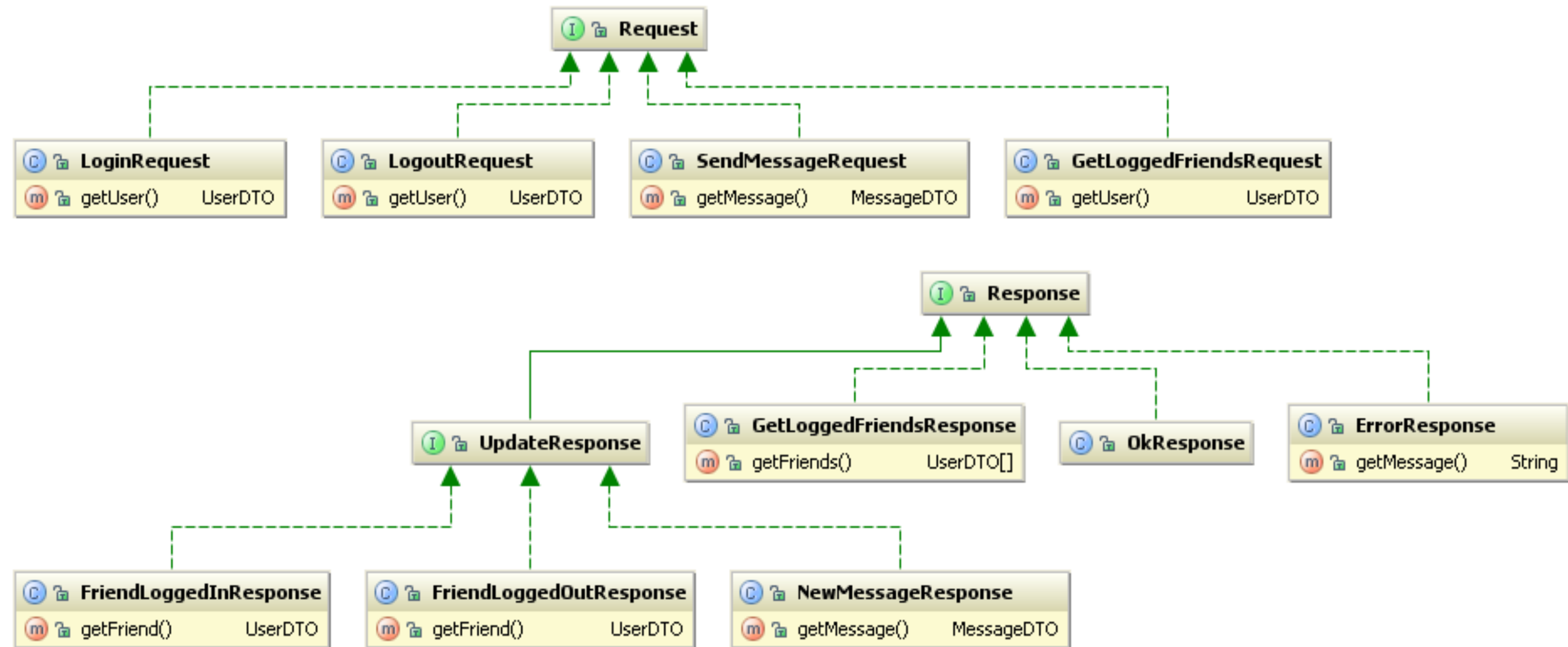
Mini-Chat (Pornire client)



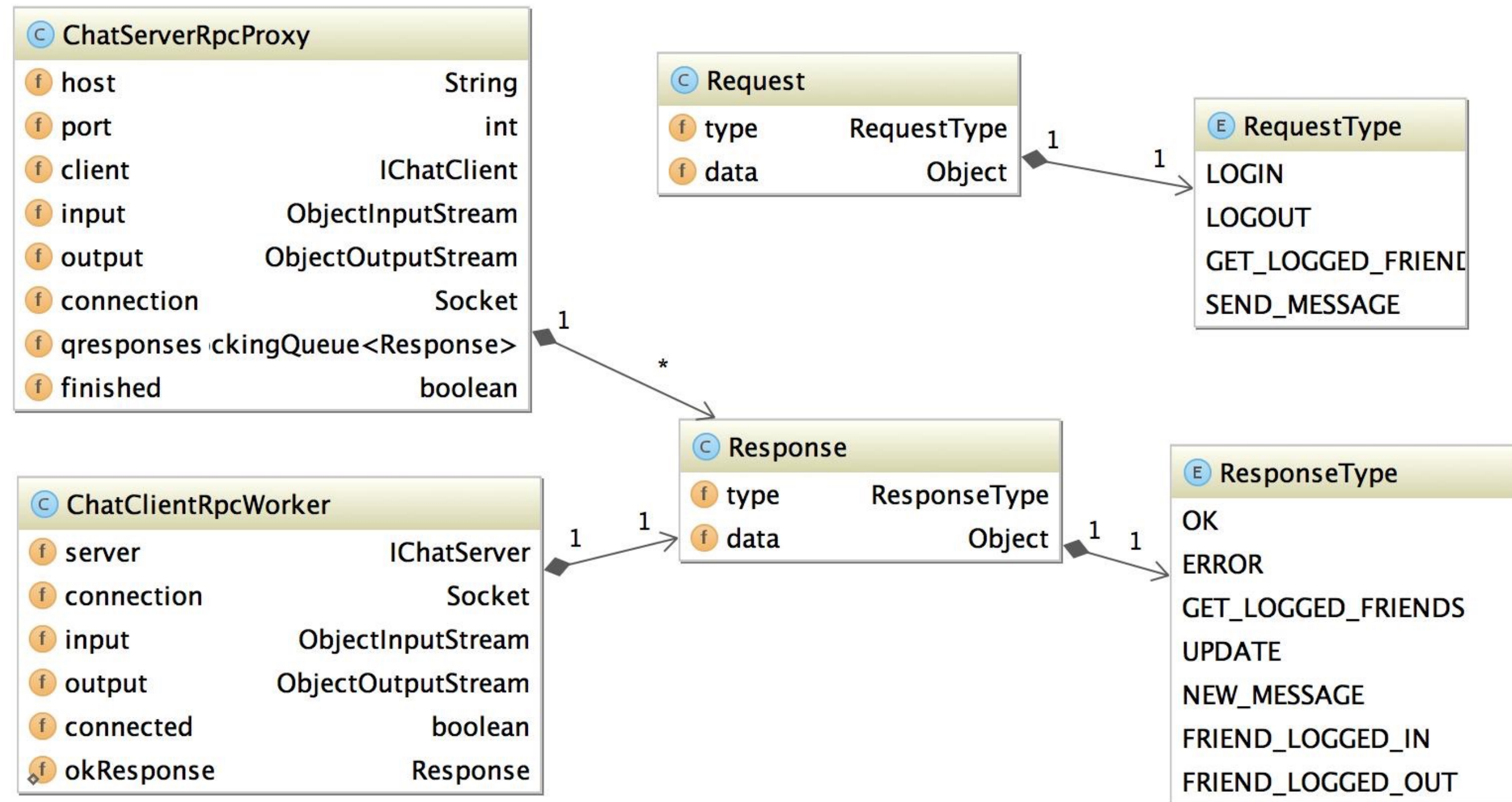
Mini-Chat (Pornire server)



Mini-chat Object Protocol



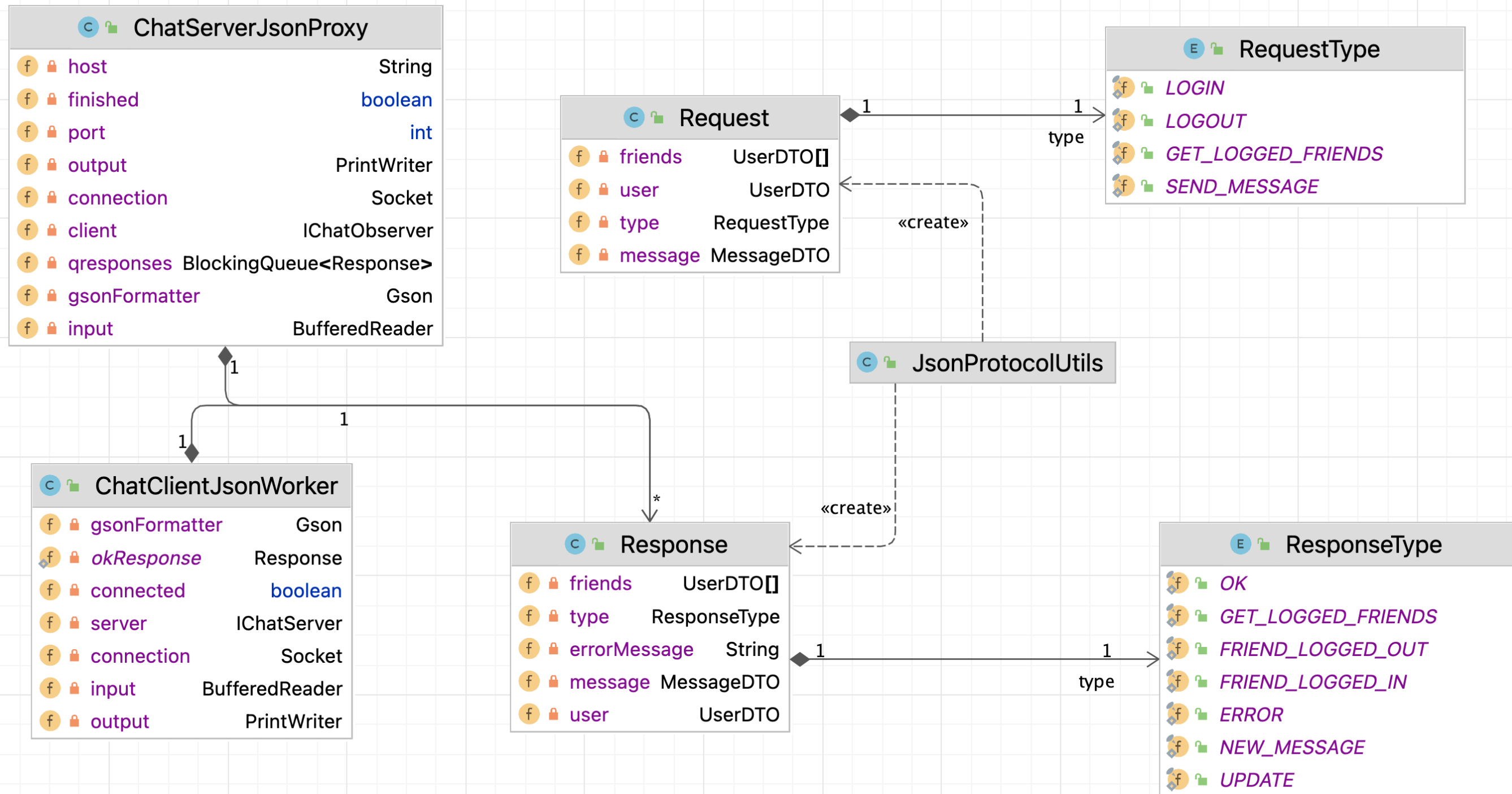
Mini-chat Rpc Protocol



Deserializarea binară - securitate

- Java și C#
- Vulnerabilități:
 - Denial of service (DoS)
 - Controlul accesului
 - Execuție de cod nedorit
- Alternative:
 - Serializare XML, Json, etc. (avantaje/dezavantaje)
- <https://learn.microsoft.com/en-us/dotnet/standard/serialization/binaryformatter-security-guide>
- <https://medium.com/@AlexanderObregon/a-deep-dive-into-java-serialization-e514346ac2b2>

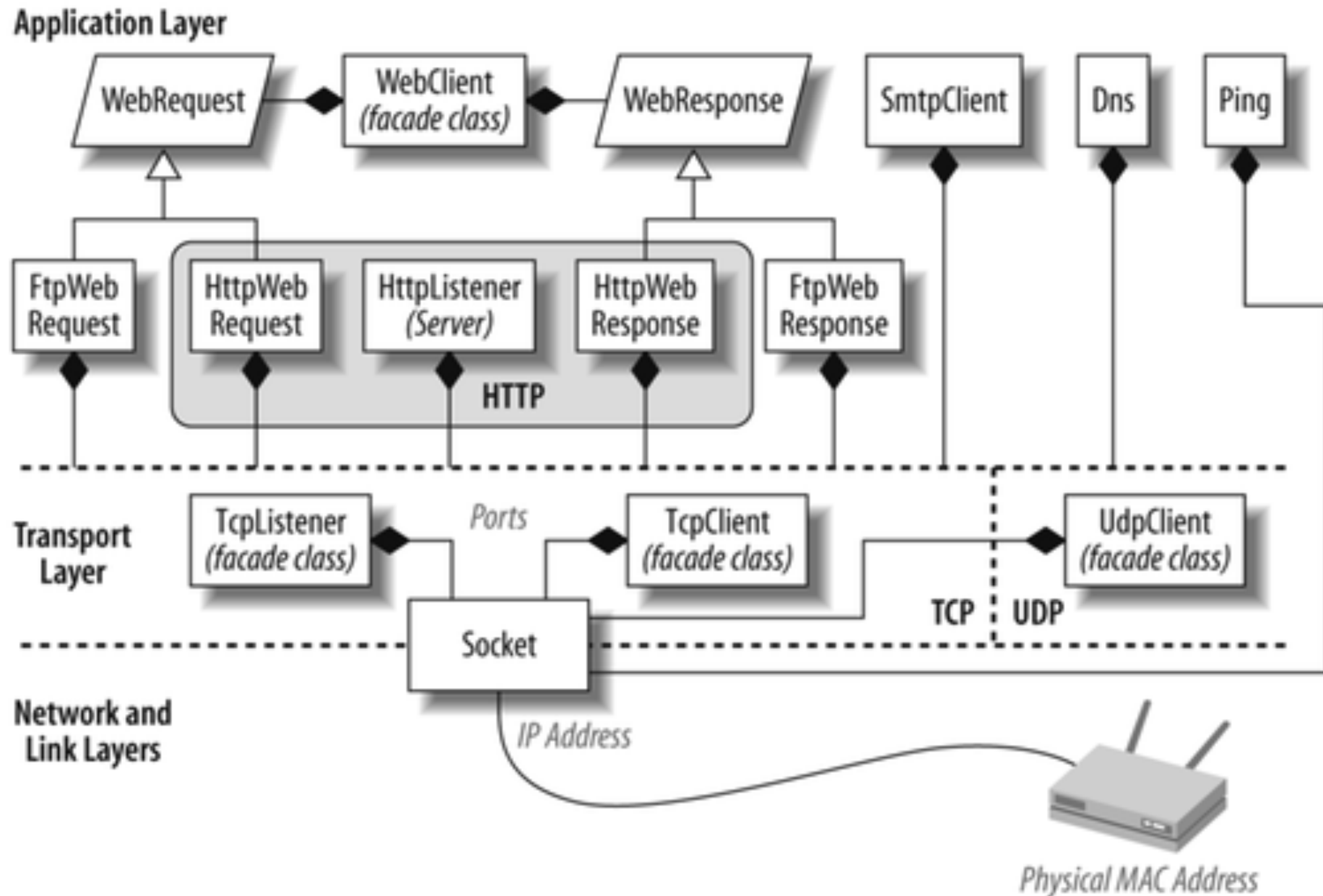
Mini-chat Json Protocol



Networking în C#

- .NET conține clase pentru comunicarea prin rețea folosind protocoale standard cum ar fi HTTP, TCP/IP și FTP.
- Spațiul de nume **System.Net.***:
 - **WebClient** fațade pentru operații simple de download/upload folosind HTTP sau FTP.
 - **WebRequest** și **WebResponse** pentru operații HTTP și FTP complexe.
 - **HttpListener** pentru implementarea unui HTTP server.
 - **SmtpClient** pentru construirea și trimiterea mesajelor folosind SMTP.
 - **TcpClient**, **UdpClient**, **TcpListener** și **Socket** pentru acces direct la nivelul rețea.

Networking in C#



Networking în C#

- Clasa `IPAddress` din `System.Net` reprezintă o adresă IPv4 (32 bits) sau IPv6 (128 bits).

```
IPAddress a1 = new IPAddress (new byte[] { 172, 30, 106, 5 });
```

```
IPAddress a2 = IPAddress.Parse ("172.30.106.5");
```

```
IPAddress a3 = IPAddress.Parse
```

```
(" [3EA0:FFFF:198A:E4A3:4FF2:54fA:41BC:8D31] "); //IPv6
```

- O asociere între o adresă IP și un port este reprezentată folosind clasa `IPEndPoint`:

```
IPAddress a = IPAddress.Parse ("172.30.106.5");
```

```
IPEndPoint ep = new IPEndPoint (a, 55555); // Port 55555
```

```
Console.WriteLine (ep.ToString( )); // 172.30.106.5:55555
```

- Porturile: 1 – 65535.
- Porturile dintre 49152 și 65535 nu sunt rezervate oficial.

System.Net.Sockets Namespace

- Clasele **TcpClient**, **TcpListener** și **UdpClient** încapsulează detaliile creării conexiunilor de tip TCP și UDP.
- **Socket** implementează interfața Berkeley socket.
- **SocketException** excepția aruncată când apare o eroare la comunicarea prin socket.
- **NetworkStream** streamul folosit pentru comunicarea prin rețea.

TcpListener

- TCP server:

```
TcpListener listener = new TcpListener (<ip address>, port);  
listener.Start();  
while (keepProcessingRequests)  
    using (TcpClient c = listener.AcceptTcpClient( ))  
        using (NetworkStream n = c.GetStream( ))    {  
            // Read and write to the network stream...  
        }  
listener.Stop();
```

- **TcpListener** necesită adresa IP la care va aștepta conexiunile clienților (dacă calculatorul are două sau mai multe plăci de rețea).
 - **IPAddress.Any** ascultă pe toate adresele IP locale (sau singura).
- **AcceptTcpClient** blochează execuția până când se conectează un client.

TcpClient

- Client Tcp:

```
using (TcpClient client = new TcpClient (<address>, port))  
    using (NetworkStream n = client.GetStream( ))  
    {  
        // Read and write to the network stream...  
    }
```

- **TcpClient** încearcă crearea conexiunii în momentul creării obiectului folosind adresa IP și portul specificate.
- Constructorul blochează execuția până la stabilirea conexiunii.

NetworkStream

- `NetworkStream` comunicare ***bidirecțională*** pentru transmiterea și recepționarea datelor după stabilirea unei conexiuni.
- Methods:
 - `Read`
 - `Close`
 - `Write`
 - `Seek`
 - `Flush`
- Properties:
 - `CanRead, CanWrite`
 - `Socket`
 - `DataAvailable`
 - `Length`

Threading în C#

- Spatiul de nume `System.Threading` clase și interfețe pentru programarea concurentă:
 - Clasa `Thread`.
 - Delegate: `ThreadStart`, `ParameterizedThreadStart`.
 - Sincronizare: `lock`, `Monitor`, `Mutex`, `Semaphore`, `EventHandles`.
- **Delegates**: reprezintă metoda executată de un thread.

```
public delegate void ThreadStart();
```

```
public delegate void ParameterizedThreadStart(Object obj);
```

- Clasa `Thread`: crearea unui thread, setarea priorității, obținerea informațiilor despre statusul unui thread.

```
public Thread(ThreadStart start);
```

```
public Thread(ParameterizedThreadStart start);
```

Threading in C#

```
class Program {
    static void Main(string[] args) {
        Worker worker=new Worker();
        Thread t1=new Thread(new ParameterizedThreadStart(static_run));
        Thread t2=new Thread(new ThreadStart(worker.run));
        t1.Start("a");
        t2.Start();
    }
    static void static_run(Object data) {
        for(int i=0;i<26;i++) { Console.Write("{0} ",data); }
    }
}
class Worker {
    public void run() {
        for(int i=0;i<26;i++) Console.Write("{0} ",i);
    }
}
//a a a a a a a a a a 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
19 20 21 22 23 24 25 a a a a a a a a a a a a a a a a
```

Sincronizarea threadurilor

- Diferite tipuri:
 - *Blocarea exclusivă*: doar un singur thread poate executa o porțiune de cod la un moment dat.
 - `lock`, `Mutex`, and `SpinLock`.
 - *Blocarea nonexclusivă*: limitarea concurenței.
 - `Semaphore` and `ReaderWriterLock`.
 - *Semnalizarea*: un thread poate bloca execuția până la primirea unei notificări de la unul sau mai multe threaduri.
 - `ManualResetEvent`, `AutoResetEvent`, `CountdownEvent` ȘI `Barrier`.

Medii de proiectare și programare

2024-2025

Curs 7

Conținut curs 7

- Networking C# (continuare)
- Remote Procedure Call
- Aplicații distribuite cross-platform
 - Google Protobuf

Sincronizarea threadurilor

- Diferite tipuri:
 - *Blocarea exclusivă*: doar un singur thread poate executa o porțiune de cod la un moment dat.
 - `lock`, `Mutex`, and `SpinLock`.
 - *Blocarea nonexclusivă*: limitarea concurenței.
 - `Semaphore` and `ReaderWriterLock`.
 - *Semnalizarea*: un thread poate bloca execuția până la primirea unei notificări de la unul sau mai multe threaduri.
 - `ManualResetEvent`, `AutoResetEvent`, `CountdownEvent` ȘI `Barrier`.

Sincronizarea threadurilor –Blocarea

- Instrucțiunea `lock`:

```
lock(locker_obj) {  
    //code to execute  
}
```

- `locker_obj` - tip referință.
- Doar un singur thread poate obține accesul la un moment dat. Dacă mai multe threaduri încearcă să obțină accesul, ele sunt puse într-o coadă și primesc accesul pe baza regulii “primul venit-primul servit”.
- Dacă un alt thread a obținut deja accesul, threadul curent nu își continuă execuția până nu obține accesul.

Sincronizarea threadurilor - Signaling

- *Event wait handles* - construcții simple pentru semnalizare:
 - **EventWaitHandle** - reprezintă un eveniment pentru sincronizarea threadurilor. Unul sau mai multe threaduri blochează execuția folosind un **EventWaitHandle** până când un alt thread apelează metoda **Set** permițând execuția unuia sau mai multor threaduri aflate în așteptare.
 - **AutoResetEvent**, **ManualResetEvent**
 - **CountdownEvent** (Framework 4.0)
- **AutoResetEvent** notifică un thread aflat în așteptare de apariția unui eveniment (doar un singur thread).
 - **Set()** - eliberează un thread aflat în așteptare
 - **WaitOne()** - threadul așteaptă apariția unui eveniment

Observații:

1. Dacă **set** este apelată când nici un thread nu se află în așteptare, handle -ul așteaptă până când un thread apelează metoda **WaitOne**.
2. Apelarea metodei **set** de mai multe ori când nici un thread nu este în așteptare nu va permite mai multor threaduri obținerea accesului când apelează metoda **WaitOne**.

Sincronizarea threadurilor - Signaling

- **ManualResetEvent** (asemănător **AutoResetEvent**) – notifică toate threadurile aflate în așteptare la apariția unui eveniment.
- Crearea unui event wait handle:
 - Constructori:

```
AutoResetEvent waitA=new AutoResetEvent(false);  
AutoResetEvent waitA=new AutoResetEvent(true); //calls Set
```



```
ManualResetEvent waitM=new ManualResetEvent(false);
```
 - Clasa **EventWaitHandle**

```
var auto = new EventWaitHandle (false, EventResetMode.AutoReset);  
var manual = new EventWaitHandle (false, EventResetMode.ManualReset);
```
- Distrugerea unui wait handle:
 - Apelul metodei **close** pentru eliberarea resurselor sistemului de operare.
 - Ștergerea referințelor pentru a permite garbage collector-ului distrugerea obiectului.

Exemplu Signaling

```
class WaitHandleExample
{
    static EventWaitHandle waitHandle = new AutoResetEvent (false);
    static void Main()
    {
        new Thread (Worker).Start();
        Thread.Sleep (1000);    // Pause for a second...
        waitHandle.Set();       // Wake up the Worker.
    }
    static void Worker()
    {
        Console.WriteLine ("Waiting...");
        waitHandle.WaitOne();    // Wait for notification
        Console.WriteLine ("Notified");
    }
}
```

Task-uri C#

- **Limitările threadurilor:**
 - Se pot transmite ușor date unui thread, dar nu se poate obține la fel de ușor rezultatul execuției unui thread.
 - Dacă execuția threadului aruncă o excepție, tratarea excepției și retransmiterea ei este mai dificil de implementat.
 - Nu se poate seta ca un thread să execute altceva când și-a încheiat execuția.
- Un **Task C#** reprezintă o operație concurentă care poate fi (sau nu) executată folosind threaduri.
 - Taskurile pot fi compuse.
 - Pot folosi un container de threaduri pentru a reduce timpul necesar pornirii execuției.
- Tipul **Task** a fost introdus începând cu Framework 4.0 ca făcând parte din biblioteca pentru programare paralelă.
- **System.Threading.Tasks** namespace.

Pornirea execuției unui Task

- Framework 4.5 - Metoda statică **Task.Run** (pornirea execuției unui task folosind threaduri) - parametru de tip **Action** delegate:

```
Task.Run (() => Console.WriteLine ("Ana")) ;
```

- Framework 4.0 - Metoda statică **Task.Factory.StartNew**:

```
Task.Factory.StartNew(() => Console.WriteLine ("Ana")) ;
```

- Implicit, taskurile folosesc threaduri din containere deja create.
- Folosirea metodei **Task.Run** - similară cu execuția explicită folosind threaduri:

```
new Thread (() => Console.WriteLine ("Ana")).Start() ;
```

- **Task.Run** returnează un obiect **Task** care poate fi folosit pentru monitorizarea progresului.
- Nu este necesară apelarea metodei **start**.

Obținerea rezultatului execuției

- **Task<TResult>** subclasă a clasei **Task** care permite returnarea rezultatului execuției.
- **Task<TResult>** poate fi obținut apelând **Task.Run** folosind un delegate de tip **Func<TResult>** (sau o expresie lambda compatibilă).
- Rezultatul poate fi obținut folosind proprietatea **Result**.
- Dacă taskul nu și-a încheiat execuția, apelul proprietății **Result** va bloca execuția threadului curent până la terminarea execuției taskului:

```
Task<int> task = Task.Run(()=>{int x=2; return 2*x; });  
int result = task.Result;    // Blocks if not already finished  
Console.WriteLine (result);    // 4
```

Taskuri și excepții

- Taskurile propagă excepțiile (threadurile nu).
- Dacă un task aruncă o excepție, excepția este rearuncată către codul care apelează metoda `Wait()` a clasei `Task` sau care apelează proprietatea `Result` a clasei `Task<TResult>`:
- Excepția va fi inclusă într-o excepție de tip `AggregateException` de către CLR:

// Start a Task that throws a NullReferenceException:

```
Task task = Task.Run (() => { throw null; });

try{
    task.Wait();
}catch (AggregateException aex) {
    if (aex.InnerException is NullReferenceException)
        Console.WriteLine ("Null!");
    else throw;
}
```

Actualizare GUI

- Interfețele grafice de tip Windows Forms folosesc obiecte de tip Control.
- Aceste obiecte pot fi modificate (actualizate, șterse, etc) doar de threadul care le-a creat.
- Nerespectarea acestei reguli are rezultate neașteptate sau aruncă excepții.
- Dacă se dorește apelarea unui membru (metoda, atribut, proprietate) a obiectului X creat într-un thread Y, cererea trebuie transmisă threadului Y (folosind metoda **Invoke** sau **BeginInvoke** a obiectului X).

Actualizare GUI

- **Invoke** și **BeginInvoke** au un parametru de tip delegate care referă metoda corespunzătoare obiectului de tip Control care se dorește a se executa.
- **Invoke** execuție sincronă: execuția apelantului este blocată până la actualizarea controlului.
- **BeginInvoke** execuție asincronă: execuția apelantului continuă, iar cererea este pusă într-o coadă (corespunzătoare evenimentelor de la tastatură, mouse, etc) și se va executa ulterior.

Exemplu

//1. definirea unei metode pentru actualizarea unui ListBox

```
private void updateListBox(ListBox listBox, IList<String> newData) {  
    listBox.DataSource = null;  
    listBox.DataSource = newData;  
}
```

//2. definirea unui delegate care va fi apelat de GUI Thread

```
public delegate void UpdateListBoxCallback(ListBox list, IList<String>  
data);
```

//3. în celălalt thread se transmite metoda care actualizeaza ListBox:

```
list.Invoke(new UpdateListBoxCallback(this.updateListBox), new Object[]  
{list, data});
```

or

```
list.BeginInvoke(new UpdateListBoxCallback(this.updateListBox), new  
Object[]{list, data});
```

Exemplu C#

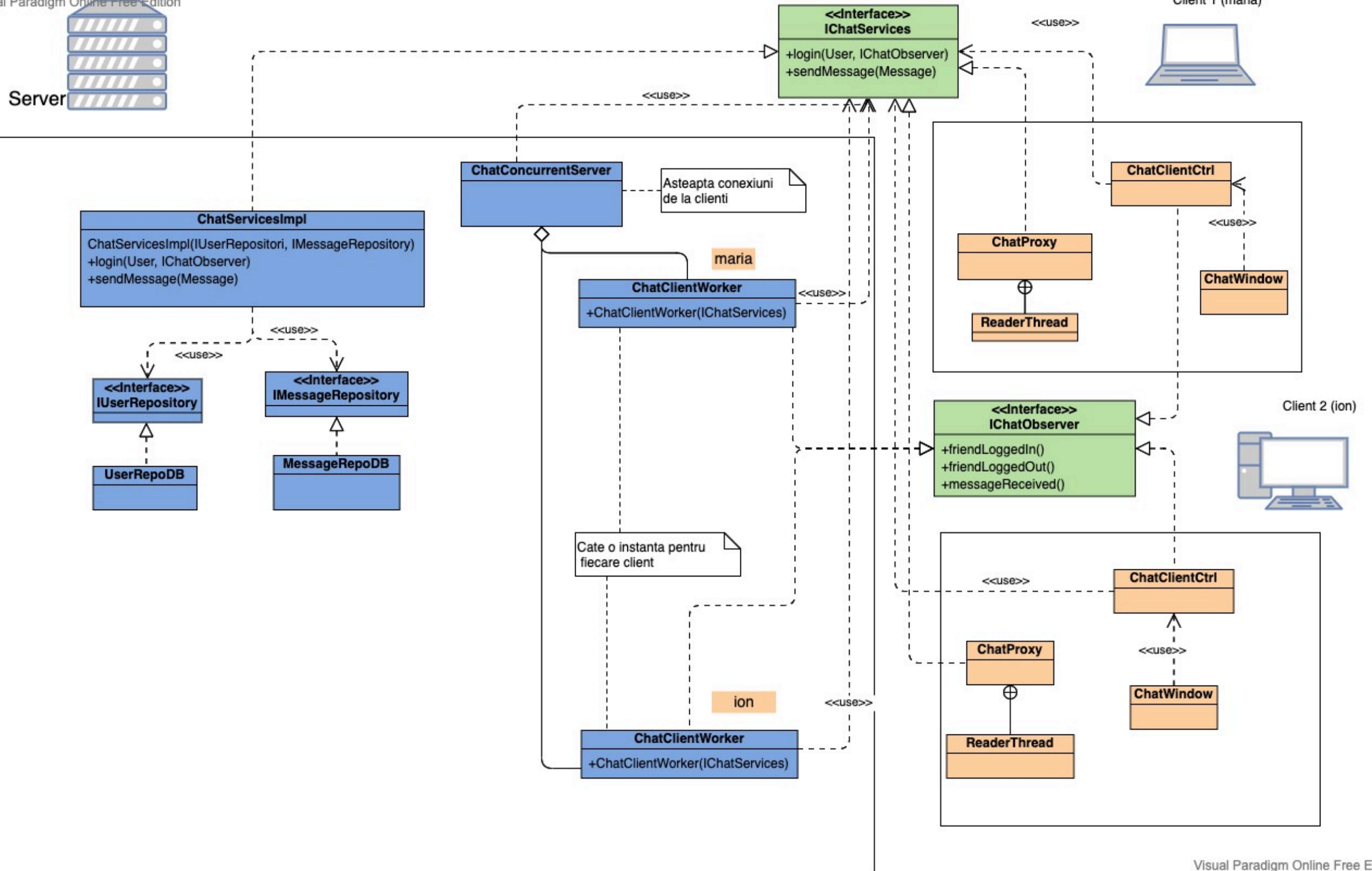
- O aplicație simplă client/server:
 - Serverul așteaptă conexiuni.
 - Clientul se conectează la server și îi trimite un text.
 - Serverul returnează textul scris cu litere mari, la care adaugă data și ora la care a fost primit textul.

Exemplu C#

- Mini-chat C#

Networking

Visual Paradigm Online Free Edition



Visual Paradigm Online Free Edition

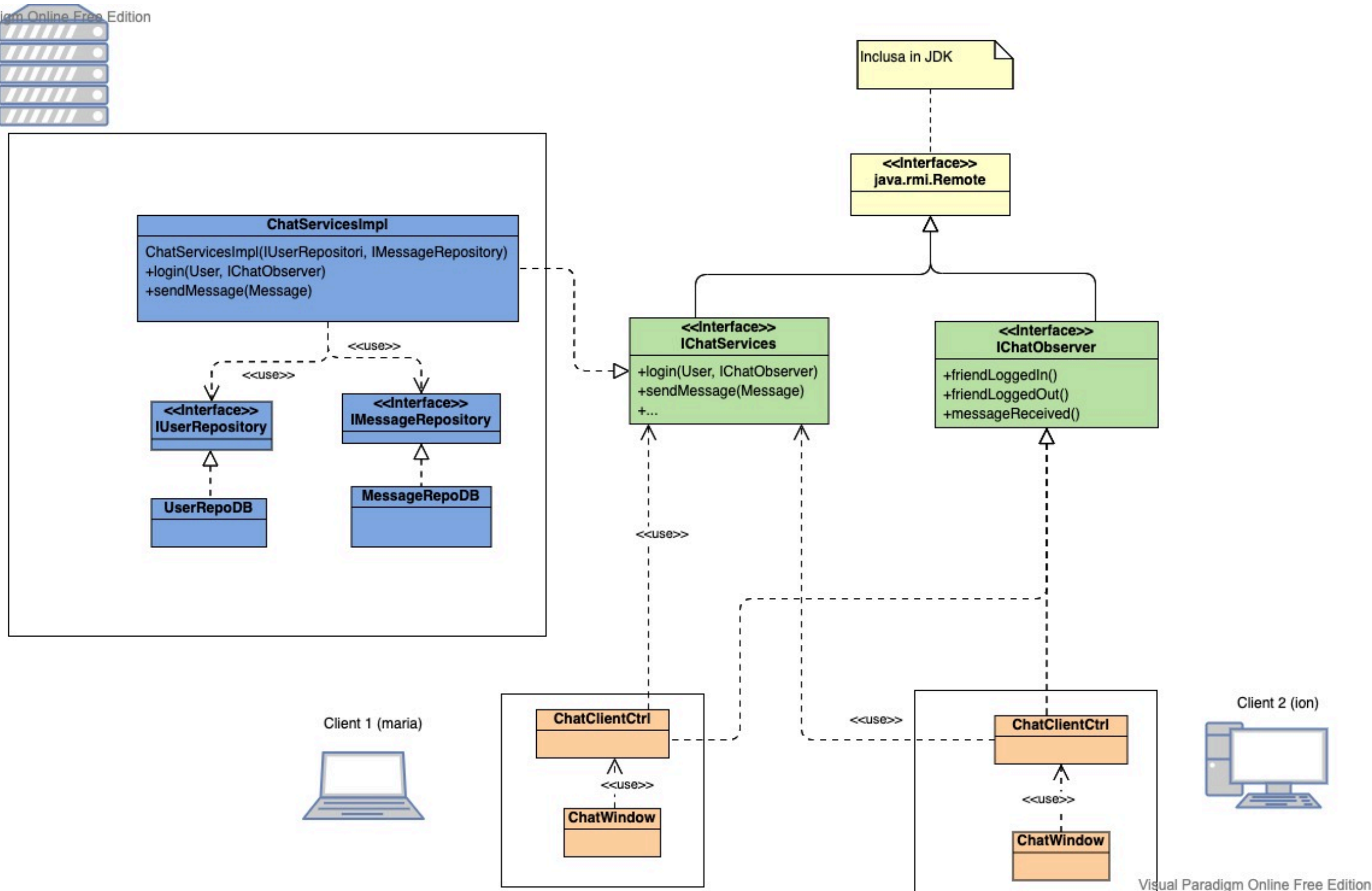
Remote Procedure Call

- **Apelul procedurilor la distanță** (eng. *Remote procedure call - RPC*) este o **tehnologie de comunicare între procese** care permite unei aplicații să inițieze execuția unei subrutine sau a unei proceduri în alt spațiu de adrese, **fără ca programatorul să scrie explicit codul corespunzător interacțiunii dintre procese**.
- Programatorul scrie aproximativ același cod indiferent dacă apelează o rutină locală (din același proces) sau una la distanță (din alt proces, în alt spațiu de adrese).
- Când se folosește paradigma orientată pe obiecte RPC - apeluri la distanță sau apelul metodelor la distanță.
- RPC este adesea folosit pentru implementarea aplicațiilor client-server.
- Un apel la distanță este inițiat de client prin trimiterea unei cereri către un server la distanță pentru execuția unei proceduri cu parametrii dați. Serverul trimite un răspuns clientului, iar aplicația își continuă execuția.
- Cât timp serverul procesează cererea clientului, execuția clientului este blocată (așteaptă până când serverul a terminat procesarea cererii).

Java RMI

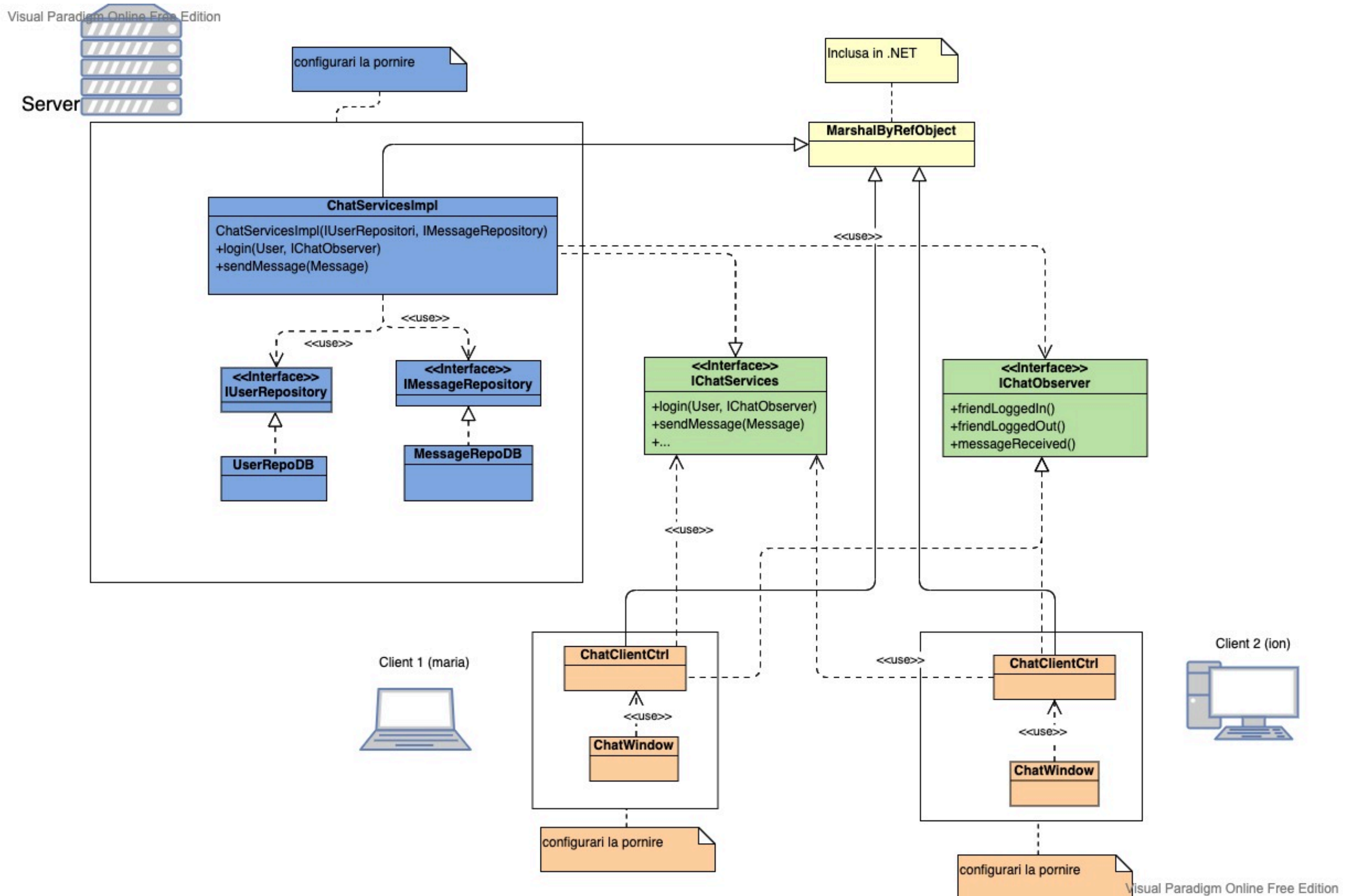
Visual Paradigm Online Free Edition

Server



Visual Paradigm Online Free Edition

NET. Remoting



Remote Procedure Call

Java RMI, NET. Remoting

- **Avantaje**

- Dezvoltarea ușoară a aplicațiilor client-server
- Ascunderea detaliilor de implementare pentru comunicarea client-server.

- **Dezavantaje**

- Aplicațiile se pot dezvolta într-un singur limbaj (Java/C#/etc)
- Dacă apar versiuni noi pentru interfețe (servicii), trebuie modificați toți clienții.

Aplicații distribuite cross-platform

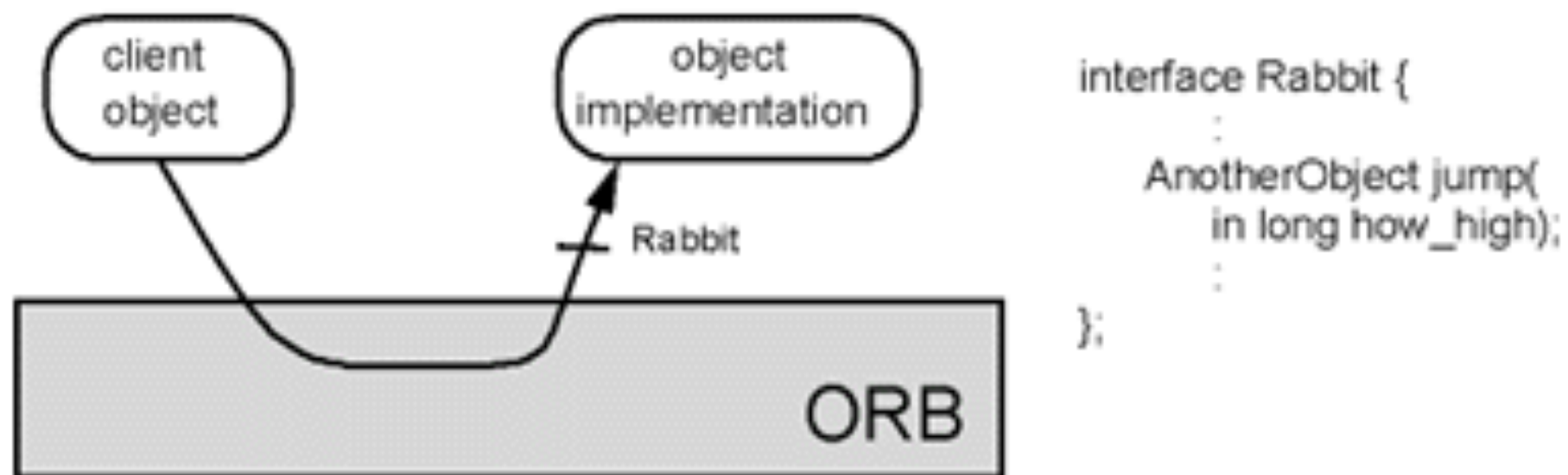
- CORBA
- Protobuf
- gRPC
- Thrift

CORBA

- A fost propusă de OMG (Object Management Group) înainte de anul 2000.
- CORBA (Common Object Request Broker Architecture) este o arhitectură standard pentru sisteme distribuite orientate pe obiect.
- Produsele CORBA oferă un framework pentru dezvoltarea și execuția aplicațiilor distribuite.
- Permite unei colecții de obiecte distribuite **heterogene** să interacționeze.
- CORBA definește arhitectura obiectelor distribuite.
- Paradigma de bază CORBA este de cerere a unor servicii oferite de un obiect distribuit (remote).
- Serviciile oferite de un obiect sunt specificate de interfața acestuia.
- **Interfețele** sunt definite folosind limbajul Interface Definition Language (IDL) definit de OMG.
- Obiectele distribuite (remote) sunt identificate prin referințe, de tipul interfețelor IDL.

Arhitectura CORBA

- Un client păstrează o referință către un obiect distribuit (remote).
- Un proces numit Object Request Broker (ORB), trimite cererea obiectului și returnează rezultatul primit clientului.



Interfețe IDL

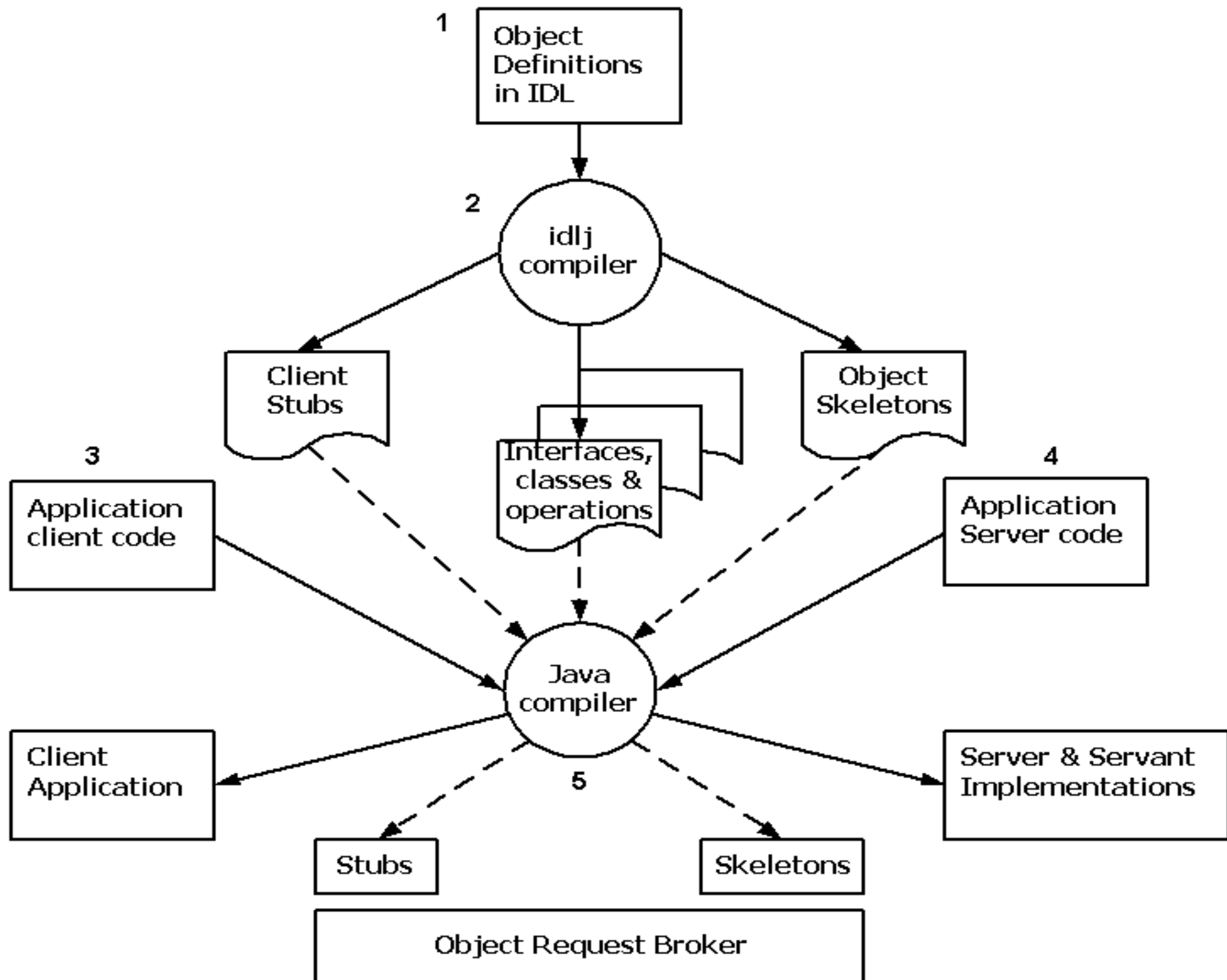
- Limbajul *Interface Definition Language* (IDL) propus de OMG permite specificarea interfețelor obiectelor remote.
- **Interfața** unui obiect remote **indică metodele** ce pot fi apelate la distanță, dar **nu cum sunt implementate aceste metode**.
- În IDL **nu se pot declara starea unui obiect și algoritmi**.
- Implementarea unui obiect CORBA este furnizată într-un limbaj de programare (Java, C++, etc). O interfață specifică contractul dintre codul care folosește obiectul și codul care implementează obiectul. Clienții depind doar de interfață.
- Interfețele IDL nu depind de un anumit limbaj de programare. IDL definește transformări (eng. *language bindings*) pentru diferite limbaje de programare.
- Se permite astfel ca pentru obiectul remote să fie ales cel mai potrivit limbaj de programare, și, de asemenea, permite clienților să aleagă cel mai potrivit limbaj (care poate diferi de cel folosit pentru implementarea obiectului remote).
- Transformări pentru C, C++, Java, Ada, Smalltalk, etc.

Exemplu IDL

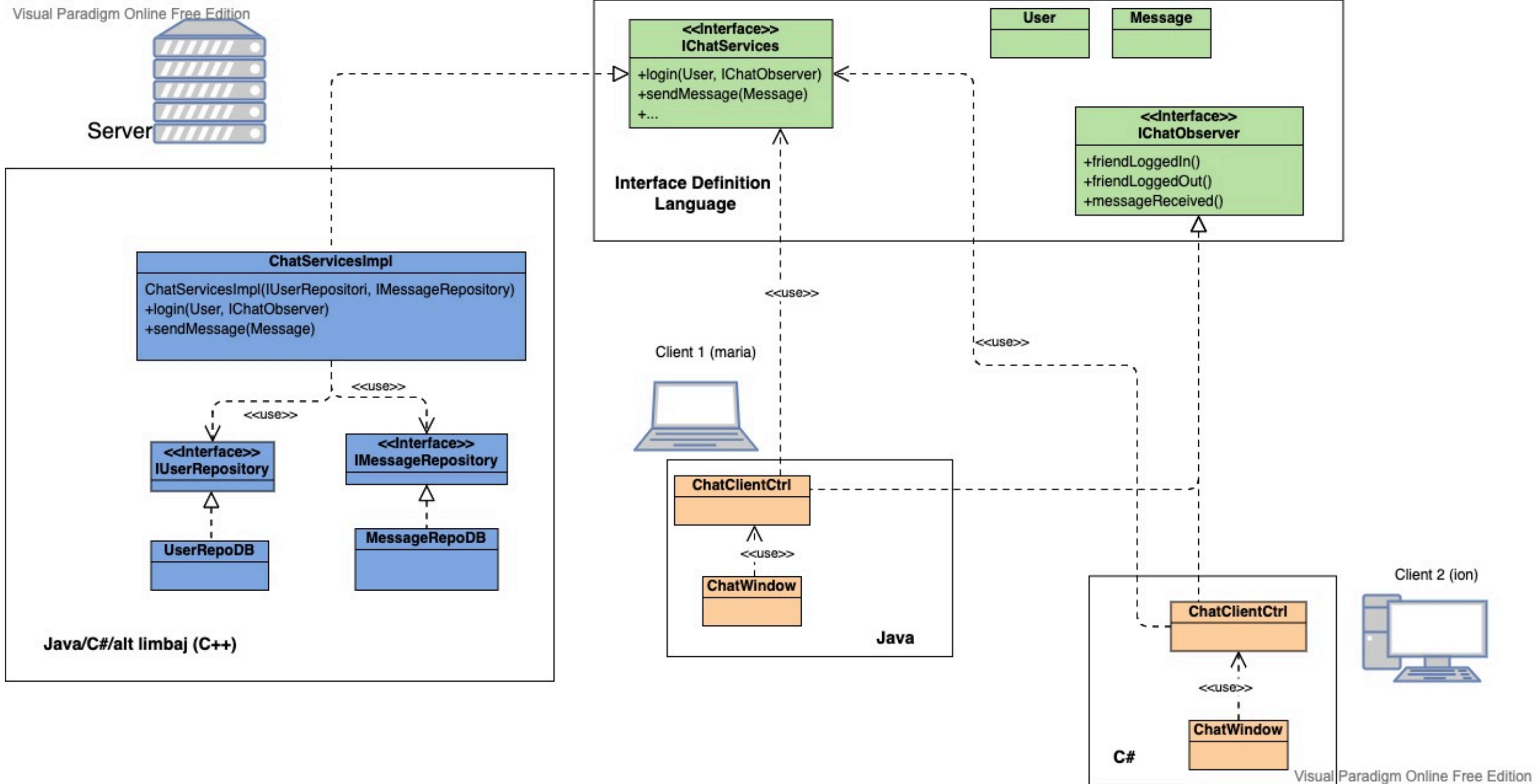
```
module BookShop {
    struct Book {
        double price;
        String title;
        String author
    };
    exception Unknown{};
    interface Library {
        Book getBook(in string title) raises(Unknown);
        readonly attribute string name;

    };
    interface BookFactory {
        Book create_book( in string title, in string author, in double
        price);
    };
};
```

CORBA și Java



CORBA și Java/C#/C++



Protocol Buffers (Protobuf)

- Reprezintă o modalitate independentă de limbaj și platformă de a serializa structuri de date folosite în protocoale de comunicație, stocarea datelor, etc.
- Sunt flexible, eficiente și au un mecanism automat de serializare a datelor structurate (similar cu XML), dar mecanismul este mai rapid, mai simplu și mai ușor de folosit.
- Se definește o singură dată modul de structurare a datelor (folosind un IDL), iar apoi se folosește un compilator special (*protoc*) care generează cod ce permite scrierea/citirea structurilor de date în/din o varietate de streamuri de date și folosind diferite limbaje de programare.
- Este permisă modificarea structurii datelor fără a provoca apariția erorilor în programele dezvoltate folosind vechea structură de date.
- Specificarea structurii datelor se definește folosind tipuri de mesaje protocol buffers și sunt salvate în fișiere **.proto**.
- Fiecare mesaj protocol buffer este o înregistrare logică mică de informații, conținând o serie de perechi cheie-valoare.

Protocol Buffers vs XML

- Protocol buffers au avantaje asupra XML în privința serializării datelor:
 - sunt mai simple
 - sunt de 3 până la de 10 ori mai mici ca și dimensiune
 - sunt de 20 până la de 100 de ori mai rapide
 - sunt mai puțin neclare (ambiguous)
 - clasele generate automat pentru accesarea datelor sunt mai ușor de folosit în limbajul de programare
- Protocol buffers nu sunt totdeauna o soluție mai bună decât XML:
 - Protocol buffers nu sunt potrivite pentru modelarea unui text ce folosește markup (ex. HTML), deoarece nu permite ușor imbricarea structurii datelor cu text.
 - XML este ușor de citit de oameni (eng. *human-readable*) și de editat (eng. *human-editable*);
 - Protocol buffers nu pot fi citite de oameni și editate
 - XML se descrie pe sine.
 - Un protocol buffer are sens doar dacă avem acces și la definiția mesajului (fișierul .proto).

Protocol Buffers vs XML

//XML

<person>

 <name>John Doe</name>

 <email>jdoe@example.com</email>

</person>

//Reprezentarea textuala a unui Protobuf (nu octeții serializați)

person {

 name: "John Doe"

 email: "jdoe@example.com"

}

Protocol Buffers vs XML

//Parsarea unui XML

```
<person>
  <name>John Doe</name>
  <email>jdoe@example.com</email>
</person>
```

//Java

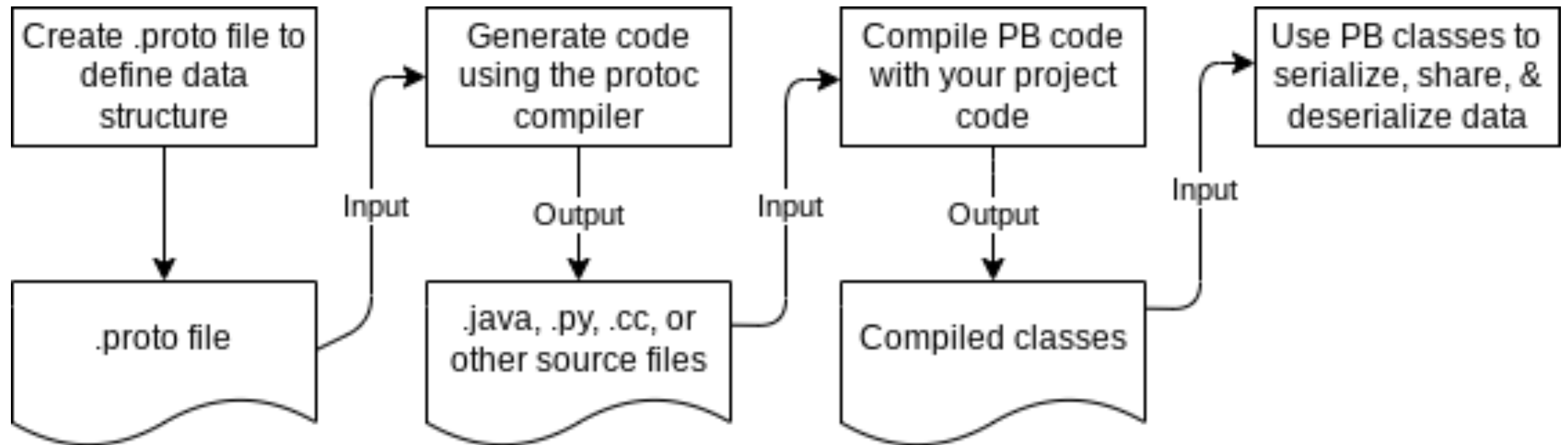
```
person.getElementsByTagName("name").item(0).getNodeValue()
person.getElementsByTagName("email").item(0).getNodeValue();
```

//Parsarea unui mesaj Protobuf

```
person {
  name: "John Doe"
  email: "jdoe@example.com"
}
```

```
System.out.println("Name: " + person.getName());
System.out.println("E-mail: " + person.getEmail());
```


Protocol Buffers Workflow*



*<https://developers.google.com/protocol-buffers/docs/overview>

Protocol Buffers vers. 2 – Exemplu

```
syntax="proto2"
message Person {
    required string name = 1;
    required int32 id = 2;
    optional string email = 3;
    enum PhoneType {
        MOBILE = 0;
        HOME = 1;
        WORK = 2;
    }
    message PhoneNumber {
        required string number = 1;
        optional PhoneType type = 2 [default = HOME];
    }
    repeated PhoneNumber phone = 4;
}
```

Definirea unui nou tip de mesaj - vers. 3

- Tipurile mesajelor sunt salvate într-un fișier `.proto`

```
syntax="proto3";
```

```
message SearchRequest {
```

```
    string query = 1;
```

```
    int32 page_number = 2;
```

```
    int32 result_per_page = 3;
```

```
}
```

- Fiecare câmp are un nume, tip și un tag asociat.
- **Tipul** poate fi:
 - `scalar` (double, float, int32, int64, bool, string, bytes, etc).
 - `tip compus` (enumerare)
 - `tipul unui alt mesaj`

Definirea unei enumerări - vers. 3

- **Enumerare**: un câmp de tip enumerare poate avea ca și valoare doar una dintre constantele specificate.
- Fiecare enumerare trebuie să definească o constantă care are asociată valoarea zero ca primă constantă din enumerare.
- Această valoare va fi folosită ca și valoare implicită.
- Două constante pot avea aceeași valoare (dacă se setează opțiunea allow_alias).

```
enum Corpus {  
    option allow_alias = true;  
    UNIVERSAL = 0;  
    WEB = 1;  
    IMAGES = 2;  
    LOCAL = 3;  
    NEWS = 4;  
    PRODUCTS = 5;  
    VIDEO = 5;  
}  
Corpus corpus = 4;
```

Reguli pentru specificarea câmpurilor - vers. 3

- Un câmp poate fi:
 - *singular*: într-un mesaj bine format acest câmp poate să apară cel mult o dată (zero sau unu).
 - **repeated**: acest câmp poate să apară de ori câte ori (inclusiv zero) într-un mesaj bine format. Ordinea valorilor care se repetă se va păstra.
- Dacă un mesaj nu conține nici o valoare pentru un câmp singular, la parsare câmpul va primi valoarea implicită corespunzătoare tipului său.

```
message Result {  
    string url = 1;  
    string title = 2;  
    repeated string snippets = 3;  
}
```

Atribuirea tagurilor

- Fiecare câmp din definiția unui mesaj are un tag numeric unic asociat.
- Aceste taguri sunt folosite pentru identificarea câmpurilor în formatul binar și nu ar trebui schimbate după începerea folosirii lui.
- Tagurile cu valori între 1 și 15 ocupă un octet (incluzând numărul de identificare și tipul câmpului).
- Tagurile cu valori între 16 - 2047 ocupă 2 octeți.
- Tagurile cu valori între 1 și 15 ar trebui folosite pentru câmpurile folosite cele mai des din mesaj.
- Tagurile pot avea valori între 1 și 536,870,911.
- Numerele între 19000 și 19999 sunt rezervate pentru implementarea Protocol Buffers - compilatorul protoc va genera o eroare dacă se încearcă folosirea lor.

Tipuri multiple de mesaje

- Mai multe tipuri de mesaje pot fi definite într-un singur fișier .proto.

```
message SearchRequest {  
    string query = 1;  
    int32 page_number = 2;  
    int32 result_per_page = 3;  
}
```

```
message SearchResponse {  
    ...  
}
```

Câmpuri rezervate

- Dacă definiția tipului unui mesaj se modifică ulterior prin ștergerea sau comentarea unui câmp, utilizatorii pot refolosi tag-ul asociat câmpului respectiv.
- Pot apărea erori când se folosesc mesaje salvate cu versiuni mai vechi ale aceluiași tip de mesaj.
- Se poate specifica că anumite taguri sau nume de câmpuri sunt rezervate și nu mai pot fi refolosite ulterior, folosind instrucțiunea **reserved**.
- În aceeași instrucțiune **reserved** nu pot fi folosite taguri și nume de câmpuri.

```
message Foo {  
    int32 foo = 2;    // câmpul foo  
  
    reserved 2, 15, 9 to 11;  
    reserved "foo", "bar";  
}
```


Fișiere .proto multiple

- Într-un fișier .proto se pot folosi definițiile din alte fișiere .proto prin importarea lor.
- Importarea definițiilor din alt fișier **.proto** se face cu instrucțiunea `import` la începutul fișierului (după instrucțiunea `syntax`):

```
import "myproject/other_protos.proto";
```

- Compilatorul caută fișierele importate în lista de directoare specificată la linia de comandă ca și parametru al compilatorului folosind opțiunea `-I` sau `--proto_path`.
- Dacă nu a fost specificată această opțiune, compilatorul va căuta în directorul în care a fost rulat compilatorul.

Tipuri Nested

- Se pot defini tipuri de mesaje în interiorul altor tipuri de mesaje:

```
message SearchResponse {  
    message Result {  
        string url = 1;  
        string title = 2;  
        repeated string snippets = 3;  
    }  
    repeated Result result = 1;  
}
```

- Tipul nested se poate folosi din exterior respectând formatul **Parent.Type**:

```
message SomeOtherMessage {  
    SearchResponse.Result result = 1;  
}
```

Tipul Any

- Tipul Any este folosit pentru a defini câmpuri pentru care nu știm/avem fișierul .proto corespunzător.
- Un câmp de tip Any va conține un mesaj necunoscut serializat într-un șir de octeți și un URL folosit ca și identificator global unic care va indica către definiția tipului mesajului.
- Pentru a folosi tipul Any trebuie importat fișierul *google/protobuf/any.proto*.

```
import "google/protobuf/any.proto";
```

```
message ErrorStatus {  
    string message = 1;  
    repeated google.protobuf.Any details = 2;  
}
```

URL implicit asociat unui mesaj definit într-un fișier .proto are formatul

type.googleapis.com/packageName.messageName

Opțiunea oneof

- Este folosită când valoarea unui câmp poate fi cel mult una dintr-o mulțime finită de valori de tipuri diferite. Folosirea opțiunii reduce dimensiunea mesajului.
- Câmpurile Oneof sunt câmpuri normale, exceptând faptul că partajează aceeași zonă de memorie și cel mult un câmp poate avea asociată o valoare la un moment dat.
- Setarea valorii unuia dintre câmpurile marcate *oneof* duce automat la ștergerea valorii celorlalte câmpuri.
- Se poate determina care câmp din mulțimea specificată *oneof* are asociată o valoare (dacă există un astfel de câmp). (Dependent de limbaj)
- Nu se pot folosi câmpuri *repeated* în mulțimea câmpurilor specificate cu *oneof*.

```
message SampleMessage {  
    oneof test_oneof {  
        string name = 4;  
        SubMessage sub_message = 9;  
    }  
}
```

Tipul Map

- Se pot defini dicționare ca și câmp al unui nou tip de mesaj.

```
map<key_type, value_type> map_field = N;
```

- **key_type** poate fi orice tip *integral* (orice tip scalar exceptând tipurile reale și octeți) sau tipul string. **value_type** poate fi orice tip.
- Observații:
 - Câmpurile Map nu pot fi *repeated*.
 - Ordinea serializării valorilor dintr-un dicționar este nespecificată, nu ne putem baza pe o anumită ordine a elementelor din dicționar.
 - Dacă la parsare/deserializare există mai multe chei având aceeași valoare se păstrează ultima valoare întâlnită.

```
map<string, Project> projects = 3;
```

Medii de proiectare și programare

2024-2025

Curs 8

Continut

- Remote Procedure Call
 - Protobuf (cont.)
 - Exemplu Mini-chat
 - gRPC
 - Thrift
- ORM

Pachete

- Opțional se poate specifica pachetul corespunzător tipurilor de mesaje definite într-un fișier .proto pentru a evita coliziuni de nume.

```
package foo.bar;  
message Open { ... }
```

- Numele pachetului poate fi utilizat ulterior la definirea tipului câmpurilor:

```
message Foo {  
    ...  
    foo.bar.Open open = 1;  
    ...  
}
```

- Modul în care specificarea unui pachet afectează codul generat depinde de limbaj:
 - În Java, pachetul este folosit ca și pachet Java, dacă nu se specifică explicit un alt pachet folosind opțiunea `option java_package` în fișierul .proto.
 - În C#, pachetul este folosit ca și spațiu de nume după transformarea la PascalCase, dacă nu se specifică explicit un alt pachet folosind opțiunea `csharp_namespace` în fișierul .proto. (Exemplu, câmpul Open va face parte din spațiul de nume Foo.Bar).

Definirea serviciilor RPC

- Se pot defini servicii RPC într-un fișier `.proto`, iar compilatorul va genera codul corespunzător interfeței și stub-urilor (proxies) în limbajul ales.

```
service SearchService {  
    rpc Search (SearchRequest) returns (SearchResponse);  
}
```

- gRPC - tehnologie open-source cross-platform RPC dezvoltată de Google care folosește protocol buffers și permite generarea codului RPC direct din fișierul `.proto` dacă se folosește un plugin adițional pentru compilatorul protoc.
- Există și alte tehnologii RPC bazate pe Protocol Buffers care pot fi utilizate pentru RPC

Alte opțiuni

- Într-un fișier .proto se pot declara opțiuni individuale care nu modifică semnificația tipurilor de mesaje definite în fișier, dar influențează modul în care este generat codul sursă corespunzător unui anumit limbaj.
- Lista completă a opțiunilor disponibile: [google/protobuf/descriptor.proto](https://github.com/google/protobuf/blob/master/src/descriptor.proto).
- Cele mai folosite opțiuni:
 - **java_package** (nivel fișier): Numele pachetului care va fi folosit la generarea claselor Java. Dacă nu se specifică explicit, numele implicit al pachetului va fi cel specificat folosind opțiunea package.
`option java_package = "com.example.foo";`
 - **java_outer_classname** (nivel fișier): Numele clasei folosite ca și clasă exterioară pentru tipurile mesajelor. Dacă nu se specifică explicit opțiunea **java_outer_classname**, numele clasei se va construi prin transformarea numelui fișierului .proto la camel-case (**foo_bar.proto** va deveni **FooBar.java**).
`option java_outer_classname = "FooBar";`

Alte opțiuni

- `java_multiple_files` (nivel fișier): Determină generarea tipurilor mesajelor, a enumerărilor și a serviciilor la nivel de pachet (nu într-o clasă exterioară) ***în fișiere separate***.

```
option java_multiple_files = true;
```

- Opțiuni C#

```
option (google.protobuf.csharp_file_options).namespace =  
"MyCompany.MyProject";
```

```
option (google.protobuf.csharp_file_options).umbrella_classname =  
"ProjectProtos";
```

Generarea claselor Java/C#

- Generarea claselor Java, C#, Python, C++, etc. se face folosind compilatorul **protoc** asupra fișierului .proto.

```
protoc --proto_path=IMPORT_PATH --java_out=DST_DIR path/to/  
file(s).proto
```

```
protoc --proto_path=IMPORT_PATH --csharp_out=DST_DIR path/to/  
file(s).proto
```

- **IMPORT_PATH** specifică directorul în care compilatorul va căuta fișierele .proto când încearcă să rezolve opțiunile *import*.
 - Dacă nu este specificat, se folosește directorul curent.
 - Se poate folosi opțiunea **-I=IMPORT_PATH** ca și o formă prescurtată a opțiunii **--proto_path**.

Clase generate (Java)

- Pentru fiecare tip de **mesaj *M*** definit într-un fișier `.proto`, compilatorul va genera **clasa *M***.
- Fiecare clasa are un Builder asociat prin intermediul căruia se vor crea instanțe ale clasei respective.
- Clasa *M* și Builder-ul asociat au metode de tip *get* generate automat de compilatorul *protoc*. Clasa Builder are și metode de tip *set* care permit setarea valorilor.
- Clasele corespunzătoare mesajelor generate de compilator sunt ***immutable***.
- După ce un obiect a fost construit, el nu mai poate fi modificat.
- Pentru a construi un mesaj, întâi se construiește builder-ul, se setează valorile câmpurilor folosind builder-ul, iar apoi se apelează metoda `build()` pentru a obține mesajul.
- Fiecare metodă din builder returnează un builder (obiectul *this*) care permite legarea mai multor apeluri de metode `set` într-o singură linie de cod.

Clase generate

- Fiecare mesaj și builder conțin metode care permit verificarea și transformarea mesajului:
 - `toByteArray() : byte[]` serializează mesajul și returnează un șir de octeți conținând mesajul serializat.
 - `parseFrom(byte[] data) : Person` parsează un mesaj dintr-un șir de octeți.
 - `writeTo(OutputStream output)` serializează mesajul și îl scrie într-un `OutputStream`.
 - `writeDelimitedTo(OutputStream output)` serializează mesajul și scrie *dimensiunea* lui și *mesajul* serializat într-un `OutputStream`.
 - `parseFrom(InputStream input) : Person` citește și parsează un mesaj dintr-un `InputStream`.
 - `parseDelimitedFrom(InputStream input) : Person` citește și parsează un mesaj dintr-un `InputStream` (mesajul a fost *scris folosind writeDelimitedTo*).

Exemplu mini-chat (proto2)

```
syntax="proto2";
package chat.protocol;
option java_package = "chat.protocol.protobuf";
option java_outer_classname = "ChatProtobufs";
message User{
    required string id=1;
    optional string passwd=2;
}
message Message{
    required string receiverId=1;
    required string senderId=2;
    required string text=3;
}
message ChatRequest {
    enum Type { Login = 1; Logout = 2; SendMessage = 3; GetLoggedFriends=4 ;}
    // Identifies which request is filled in.
    required Type type = 1;
    // One of the following will be filled in, depending on the type.
    optional User user = 2;
    optional Message message = 3;
}
message ChatResponse{
    enum Type { Ok = 1; Error = 2; GetLoggedFriends=3; FriendLoggedIn = 4; FriendLoggedOut=5; NewMessage=6; }
    // Identifies which request is filled in.
    required Type type = 1;
    // One of the following will be filled in, depending on the type.
    optional string error = 2;
    repeated User friends=3;
    optional User user=4 ;
    optional Message message = 5;
}
```

Exemplu mini-chat (proto3)

```
syntax="proto3";
package chat.protocol;
option java_package = "chat.network.protobufprotocol";
option java_outer_classname = "ChatProtobufs";
message User{
    string id=1;
    string passwd=2;
}
message Message{
    string receiverId=1;
    string senderId=2;
    string text=3;
}
message ChatRequest {
    enum Type {Unkown=0; Login = 1; Logout = 2; SendMessage = 3; GetLoggedFriends=4 ;}
    // Identifies which request is filled in.
    Type type = 1;
    // One of the following will be filled in, depending on the type.
    oneof payload{
        User user = 2;
        Message message = 3;
    }
}
message ChatResponse{
    enum Type { Unknown=0; Ok = 1; Error = 2; GetLoggedFriends=3; FriendLoggedIn = 4; FriendLoggedOut=5; NewMessage=6;}
    // Identifies which request is filled in.
    Type type = 1;
    // One of the following will be filled in, depending on the type.
    string error = 2;
    repeated User friends=3;
    User user=4 ;
    Message message = 5;
}
```


Bibliografie

- Protocol Buffers

<https://developers.google.com/protocol-buffers/>

<https://github.com/google/protobuf>

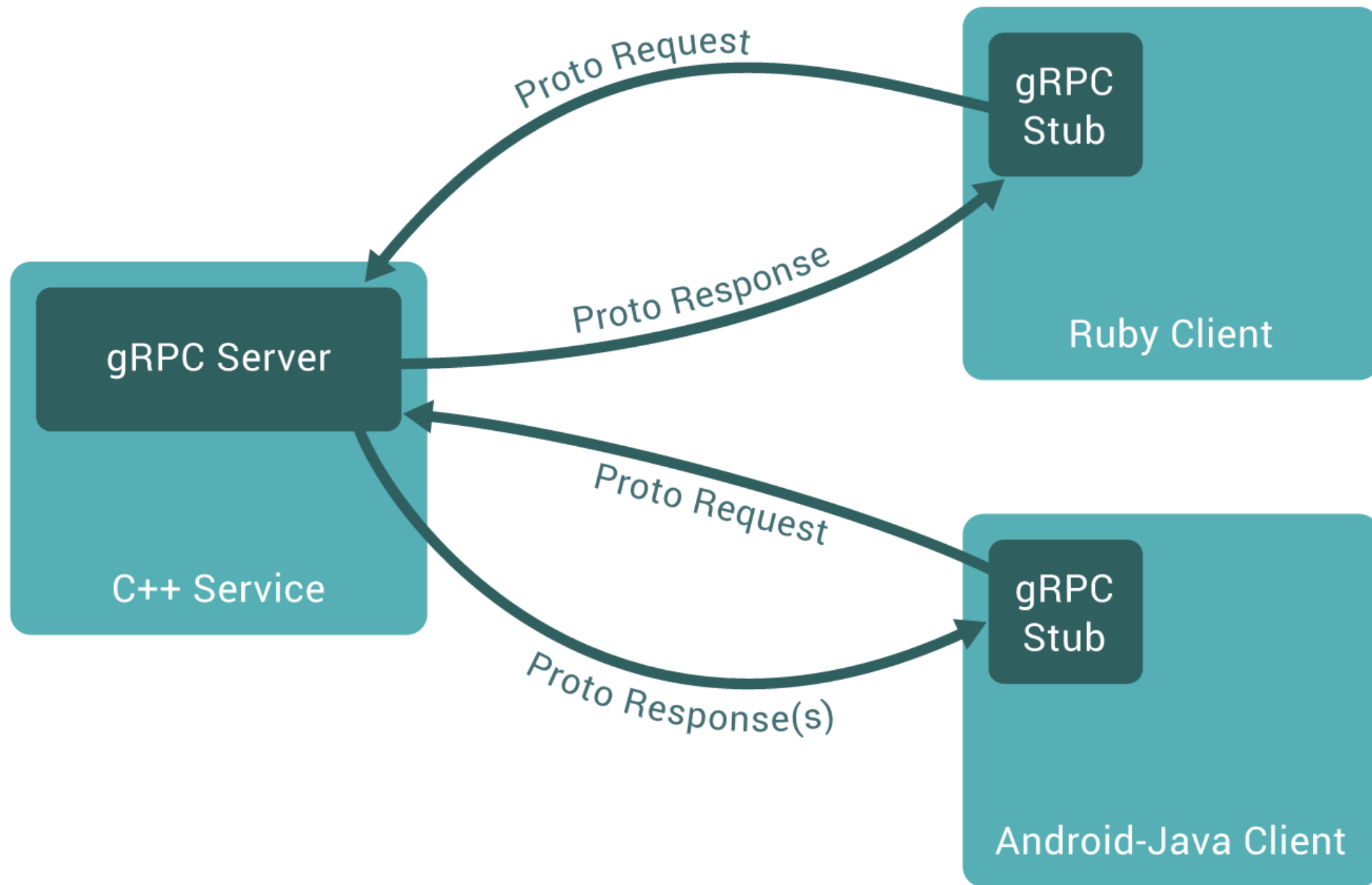
Protocol Buffers

- Exemplu MiniChat

gRPC

- Cu **gRPC** o aplicație client poate apela metode la distanță ca și cum ar fi metode ale unui obiect local.
- Facilitează crearea aplicațiilor distribuite și a serviciilor.
- gRPC folosește ideea definirii unui serviciu (interfețe) care specifică metodele ce pot fi apelate la distanță, împreună cu parametrii și tipul returnat al acestora.
- În aplicația server există un obiect remote care implementează interfața, iar serverul gRPC gestionează cererile clienților.
- Clientul (aplicația client) are un stub (proxy) care oferă aceleași metode ca și obiectul remote.
- Clienții și serverele gRPC pot fi scrise și pot rula în diferite limbaje de programare: Java, C#, C++, Python, Go, etc.

gRPC



gRPC - Definirea unui serviciu

- Implicit, gRPC folosește Protocol Buffers ca și limbaj de definire a tipurilor de mesaje și a serviciilor (IDL).
- Se pot folosi alte IDL-uri.

```
service HelloService {  
    rpc SayHello (HelloRequest) returns (HelloResponse);  
}
```

```
message HelloRequest {  
    string greeting = 1;  
}
```

```
message HelloResponse {  
    string reply = 1;  
}
```

gRPC - Definirea unui serviciu

- gRPC permite specificarea a 4 tipuri de apeluri de metode la distanță:
 - *unare* - clientul trimite o singură cerere la server și primește un singur răspuns de la server (apeluri de funcții normale).

```
rpc SayHello (HelloRequest) returns (HelloResponse) {}
```

- *server streaming* - clientul trimite o cerere la server și primește un *stream* cu ajutorul căruia poate citi o secvență de răspunsuri de la server. Clientul citește răspunsurile până când nu mai sunt mesaje pe stream.

```
rpc LotsOfReplies (HelloRequest) returns (stream HelloResponse) {}
```

- *client streaming* - clientul scrie o secvență de mesaje și le trimite la server folosind streamul furnizat. După ce clientul a terminat scrierea mesajelor, așteaptă ca serverul să le citească și să trimită un singur răspuns.

```
rpc LotsOfGreetings (stream HelloRequest) returns (HelloResponse) {}
```

gRPC - Definirea unui serviciu

- gRPC permite specificarea a 4 tipuri de apeluri de metode la distanță:
 - *Bidirectional streaming* - atât serverul cât și clientul trimit o secvență de mesaje folosind un stream bidirecțional (read-write). Streamurile (read-write) funcționează independent; clienții și serverul pot citi și scrie în ordinea dorită de ei.

Exemple:

-serverul poate aștepta până primește toate mesajele de la client înainte de a trimite răspunsurile;

-serverul poate citi un mesaj, trimite răspunsul, sau altă combinație de read-write.

Ordinea mesajelor din fiecare stream se păstrează.

```
rpc BidiHello(stream HelloRequest) returns (stream HelloResponse) {}
```

gRPC - Definirea unui serviciu

- Pornind de la definiția unui serviciu dintr-un fișier `.proto`, gRPC furnizează **plugin-uri** pentru compilatorul protoc care permit generarea codului corespunzător clienților și serverului.
- Clienții și serverul gRPC folosesc acest API pentru implementarea funcționalității dorite:
 - În aplicația server, serverul implementează metodele declarate de serviciu, și le face disponibile clienților folosind un server gRPC. Frameworkul gRPC decodifică cererile, execută metodele remote și codifică răspunsurile.
 - În aplicația client, clientul are un obiect local (stub, proxy) care implementează aceleași metode ca și serviciul.
- Clientul poate apela aceste metode ca și cum ar fi apeluri locale, folosind parametrii corespunzători.

gRPC - Timeout

- gRPC permite clienților să specifice cât timp sunt dispuși să aștepte ca un apel la distanță să își încheie execuția. Dacă durată specificată a fost depășită, apelul la distanță se va încheia cu eroarea `DEADLINE_EXCEEDED`.
- Pe server se poate verifica dacă un anumit apel la distanță a depășit durată sau cât timp mai are la dispoziție.
- Atât clientul cât și serverul pot anula execuția unui apel la distanță în orice moment. Anularea duce la oprirea imediată a execuției apelului la distanță.
- Anularea nu este o operație de tip “undo”: modificările făcute înainte de anulare nu vor fi anulate și ele.

gRPC - Terminare, canale de comunicare

- Terminarea unui apel - clientul și serverul determină independent succesul execuției unui apel la distanță, iar concluziile lor pot să difere.
 - Exemplu 1: un apel RPC se poate termina cu succes pe server (“I have sent all my responses!”), dar poate eșua pe client (“The responses arrived after my deadline!”).
 - Exemplu 2: serverul poate decide terminarea înainte ca clientul să fi terminat de trimis toate cererile.
- Canal de comunicare gRPC - furnizează o conexiune la un server gRPC specificând adresa și portul și se folosește la instanțierea unui proxy (client stub). Clienții pot specifica parametrii pentru a modifica comportamentul implicit (setarea opțiunii de compresie a mesajelor, etc).
- Un canal are asociată o stare (dacă este conectat, dacă este idle, etc.)

gRPC - Exemplu

- O aplicație simplă client/server:
 - Obiectul remote are o metodă care transformă un text în text cu litere mari și adaugă data și ora la care a fost primit textul.
 - Clientul obține o referință la obiectul remote, apelează metoda și tipărește rezultatul primit.

Bibliografie

- Protocol Buffers

<https://developers.google.com/protocol-buffers/>

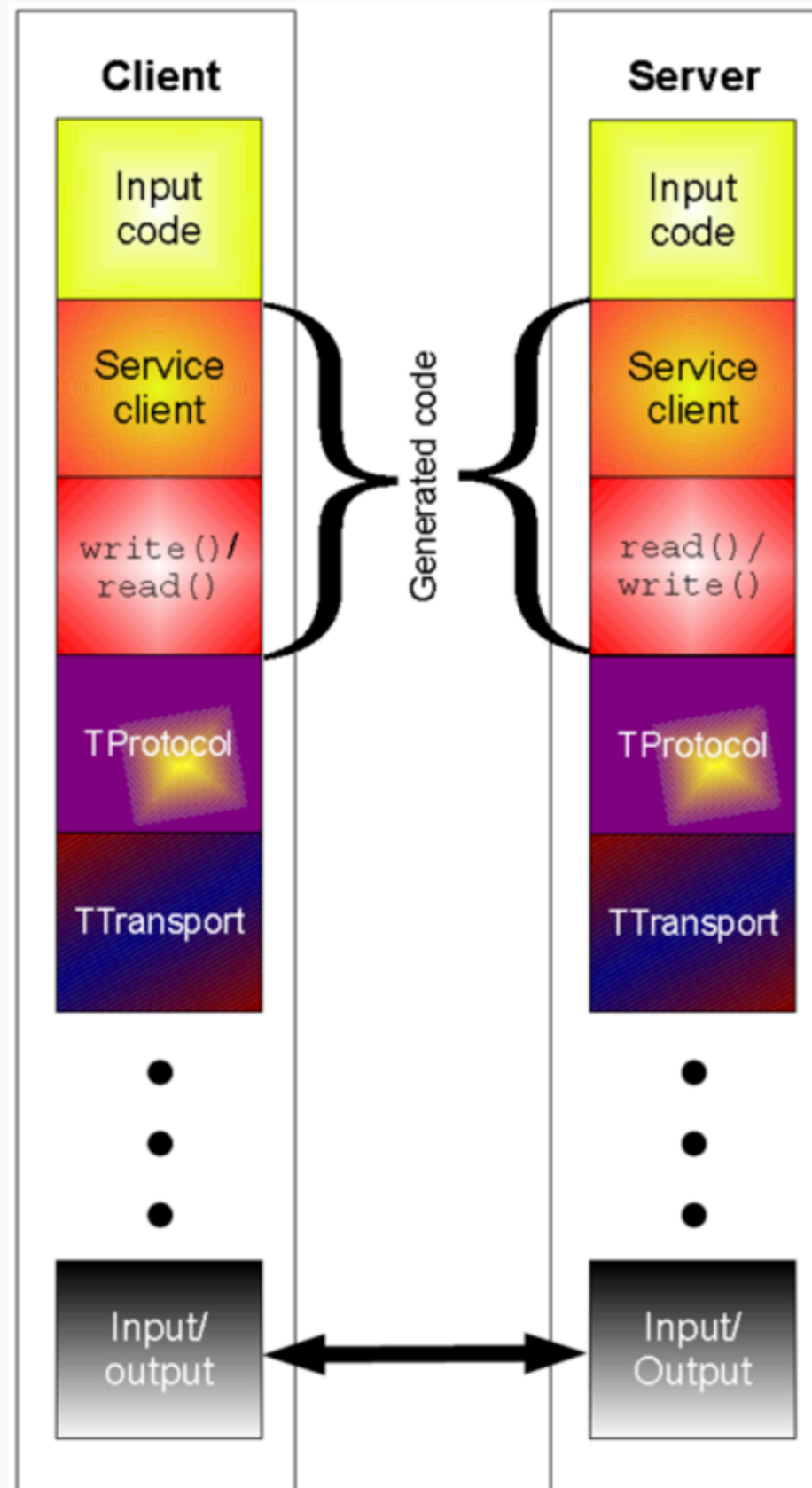
<https://github.com/google/protobuf>

- gRPC

<http://www.grpc.io/>

Apache Thrift

- **Thrift** este o tehnologie cross-platform pentru RPC.
- Thrift oferă separare clară între nivelul transport, serializarea datelor și logica aplicației.
- Thrift a fost dezvoltat inițial de Facebook, acum este un proiect open-source găzduit de Apache.
- Apache Thrift conține un set de unelte de generare de cod care permit utilizatorilor să dezvolte clienți și servere RPC doar prin definirea tipurilor de date necesare și a interfețelor serviciilor într-un fișier IDL **.thrift**.
- Folosind acest fișier ca și date de intrare, se generează automat cod sursă pentru clienți și servere RPC în diferite limbaje de programare care permit aplicațiilor să comunice indiferent de limbajul folosit pentru scrierea clientului/serverului.
- Thrift suportă diferite limbaje de programare: C++, Java, C#, Python, PHP, Ruby.



Thrift Architecture.

Image from wikipedia.org

Apache Thrift

- Componente cheie:

- tipuri de date
- transport
- protocol
- versioning
- processors

- Tipuri de date Thrift

- Sistemul de tipuri de date Thrift nu introduce tipuri speciale dinamice sau obiecte wrapper.
- Programatorul nu trebuie să scrie cod pentru serializarea tipurilor de date sau pentru transportul datelor.
- Conține tipuri de bază, structuri, containere.
- Tipuri de bază:
 - *bool* - valoarea booleană: true sau false
 - *byte* - a signed byte
 - *i16* - întreg cu semn pe 16-biți
 - *i32* - întreg cu semn pe 32-biți
 - *i64* - întreg cu semn pe 64-biți
 - *double* - real pe 64-biți
 - *string*
 - *binary* - șir de octeți folosit pentru reprezentarea blob-urilor.

Tipuri Thrift -Structuri

- O structură Thrift definește un obiect comun tuturor limbajelor ce poate fi transmis prin rețea.
- O structură este echivalentul unei clase dintr-un limbaj orientat obiect.
- Structura conține o mulțime de câmpuri care au un tip definit și un identificator unic asociat.
- Sintaxa asemănătoare structurii C.
- Câmpurile pot fi adnotate cu un identificator numeric unic (valoare întreagă) și o valoare implicită (opțional). Identificatorii trebuie să fie unici în cadrul structurii.
- Dacă identificatorii câmpurilor sunt omiși, li se vor atribui automat valori.

```
struct Example {  
    1:i32 number=10,  
    2:i64 bigNumber,  
    3:double decimals,  
    4:string name="thrifty"  
}
```


Tipuri Thrift -Containere

- Containerele Thrift sunt containere cu tip care se vor mapa la cele mai folosite containere din limbajele de programare corespunzătoare.
- Sunt parametrizate folosind stilul Java Generics/C++ template.
- Trei tipuri de containere disponibile:
 - **list<type>** : o listă de elemente.
 - Se mapează la STL **vector**, Java **ArrayList**, sau orice alt tablou din limbajele scripting. Poate conține duplicate.
 - **set<type>** : O mulțime neordonată de elemente.
 - Se mapează la STL **set**, Java **HashSet**, **set** în Python, sau dicționare native în PHP/Ruby.
 - **map<type1, type2>** : Un dicționar cu chei unice
 - Se mapează la STL **map**, Java **HashMap**, PHP associative array, sau Python/Ruby dictionary.
- Elementele din container pot fi de orice tip valid Thrift, inclusiv alte containere sau structuri.
- În codul generat corespunzător limbajului de programare dorit, fiecare definiție va conține două metode, **read** și **write**, folosite pentru serializarea și transportul obiectelor folosind Thrift TProtocol.

Thrift - Excepții

- Excepțiile Thrift sunt sintactic și funcțional echivalente cu structurile Thrift doar că sunt declarate folosind `exception` în loc de `struct`.
- Clasele generate vor moșteni din clasele de bază corespunzătoare excepțiilor în limbajul de programare folosit, pentru a se putea folosi în mod transparent cu mecanismul de tratare a excepțiilor din limbajul respectiv.

Servicii Thrift

- **Serviciile Thrift** se definesc folosind tipurile Thrift.
- Definirea unui serviciu este echivalentă semantic cu **definirea unei interfețe** într-un limbaj de programare orientat pe obiecte.
- **Compilerul Thrift** va genera stub-uri client și server complet funcționale care implementează interfața.
- **Lista parametrilor și lista excepțiilor** sunt implementate ca și structuri Thrift.

```
service <name> {  
    <returntype> <name>(<arguments>) [throws (<exceptions>)]  
    ...  
}
```

```
service StringCache {  
    void set(1:i32 key, 2:string value),  
    string get(1:i32 key) throws (1:KeyNotFound knf),  
    void delete(1:i32 key)  
}
```

Nivelul transport Thrift

- Nivelul transport este folosit de codul generat automat pentru a ușura transmiterea datelor între clienți și server.
- Thrift se folosește de obicei peste TCP/IP ca și nivel de bază pentru comunicare.
- Codul generat Thrift trebuie să știe doar cum să scrie și cum să citească datele.
- Originea sau destinația datelor sunt irelevante: poate fi socket, memorie partajată sau un fișier pe discul local.
- Interfața Thrift **TTransport** conține metodele:
 - **open** deschiderea transportului
 - **close** închiderea transportului
 - **isOpen** verifică dacă transportul este deschis
 - **read** citește date
 - **write** scrie date
 - **flush** forțează scrierea datelor păstrate în zona tampon.

Nivelul transport Thrift

- Interfață **TServerTransport** folosită pentru crearea și acceptarea obiectelor transport:
 - **open** deschidere
 - **listen** așteaptă conexiuni de la clienți
 - **accept** returnează un nou obiect transport (când s-a conectat un client nou)
 - **close** închidere
- Interfața transport este proiectată pentru implementare ușoară în orice limbaj de programare.
- Se pot defini noi mecanisme de transport:
 - Clasa **TSocket** este implementată în toate limbajele de programare suportate. Oferă o modalitate simplă de comunicare cu un socket TCP/IP.
 - Clasa **TFileTransport** este o abstractizare a unui stream ce reprezintă un fișier de pe discul local. Poate fi folosită pentru a salva cererile clienților într-un fișier de pe disc.

Thrift - Protocol

- Thrift cere respectarea unei anumite structuri a mesajelor când sunt transmise prin rețea, dar **nu știe de protocolul efectiv folosit pentru serializarea/codificarea acestora.**
- Nu contează dacă datele sunt serializate folosind XML, human-readable ASCII (stringuri) sau octeți, dacă mecanismul suportă o mulțime de operații prestabilite care permit citirea și scrierea datelor.
- Interfața Thrift Protocol suportă:
 - *trimiterea mesajelor bidirectional,*
 - *codificarea tipurilor de bază, a structurilor și a containerelor.*
- **Are o implementare care folosește un protocol binar.**
- **Scrie toate datele într-un format binar:**
 - **Tipurile întregi** sunt **convertite într-un format rețea independent de limbaj.**
 - **Stringurile** au **adăugate la început lungimea lor în număr de octeți.**
 - **Mesajele și antetul câmpurilor** sunt **scrise folosind construcții de serializare a datelor întregi.**
 - **Numele câmpurilor nu sunt serializate,** **identificatorii asociați** sunt **suficienți pentru reconstruirea datelor.**

Thrift - Protocol

```
writeMessageBegin(name, type, seq)
writeMessageEnd()
writeStructBegin(name)
writeStructEnd()
writeFieldBegin(name, type, id)
writeFieldEnd()
writeMapBegin(ktype, vtype, size)
writeMapEnd()
...
name, type, seq = readMessageBegin();
readMessageEnd()
name = readStructBegin()
readStructEnd()
name, type, id = readFieldBegin()
    readFieldEnd()
k, v, size = readMapBegin()
readMapEnd()
...
```

Thrift - Versioning

- Thrift este **robust** la schimbări de versiuni și de definiție a datelor.
- **Versioning** este implementat folosind identificatorii asociați câmpurilor:
 - Antetul unui câmp dintr-o structură Thrift este codificat folosind identificatorul unic.
 - Combinația (identificator câmp, tip câmp) este folosită pentru a identifica unic un câmp din structură.
- Limbajul IDL Thrift permite atribuirea automată de identificatori pentru câmpuri, dar se recomandă definirea explicită a acestora.
- Dacă la parsare/decodificare/deserializare se întâlnește un câmp necunoscut acesta este **ignorat** și **distrus**.
- Dacă un câmp care ar trebui să existe nu apare, programatorul poate fi notificat de lipsa acestuia (folosind structura **isset** definită în interiorul fiecărei obiect).
- Obiectul **isset** din interiorul unei structuri Thrift poate fi interogat pentru a determina existența unui anumit câmp. De fiecare dată când se primește o instanță a unei structuri, programatorul ar trebui să verifice existența unui câmp înainte de folosirea lui.

Thrift - Implementare RPC

- Instanțe a clasei `TProcessor` sunt folosite pentru a trata cererile RPC:

```
interface TProcessor {  
    bool process(TProtocol in, TProtocol out) throws TException  
}
```
- Pentru fiecare serviciu dintr-un fișier `.thrift` se generează următoarele:
 - o interfață `ServiceIf` corespunzătoare serviciului
 - clasa `ServiceClient` implementează `ServiceIf`
`TProtocol in`
`TProtocol out`
 - clasa `ServiceProcessor` : `TProcessor`
`ServiceIf handler`
 - clasa `ServiceHandler` implementează `ServiceIf`
 - `TServer(TProcessor processor,`
`TServerTransport transport,`
`TTransportFactory tfactory,`
`TProtocolFactory pfactory)`
`serve()`

Thrift - Implementare RPC

- Serverul încapsulează logica corespunzătoare conexiunii, threadurilor, etc, iar obiectul de tip TProcessor se ocupă de apelul metodelor la distanță.
- Programatorul trebuie să scrie doar codul din fișierul .thrift și implementarea corespunzătoare serviciilor (ServiceHandler)
- Există mai multe implementări posibile pentru TServer:
 - TSimpleServer: server secvențial
 - TThreadedServer: server concurent (se creează câte un thread pentru fiecare conexiune)
 - TThreadPoolServer: server concurent care folosește un container de threaduri.
-

Bibliografie

- Apache Thrift

<http://thrift-tutorial.readthedocs.io/en/latest/index.html>

<https://thrift.apache.org/>

- Exem plu: Text transformer

Object/Relational Mapping (ORM)

- *Object-relational mapping* (ORM, O/RM sau O/R mapping) este o tehnică de programare pentru convertirea informațiilor/tipurilor dintr-un sistem orientat-obiect într-o bază de date relațională.
- Principiul mapării obiect-relație/înregistrare este de a delega altor instrumente managementul persistenței și de a lucra doar cu entitățile din domeniu, nu cu structurile dintr-o bază de date relațională.
- Instrumentele de mapare obiect-relație stabilesc o legătură bidirecțională între o bază de date relațională și obiectele din sistem, pe baza unei configurații și execută interogări SQL la baza de date (interogări construite dinamic).

Terminologie

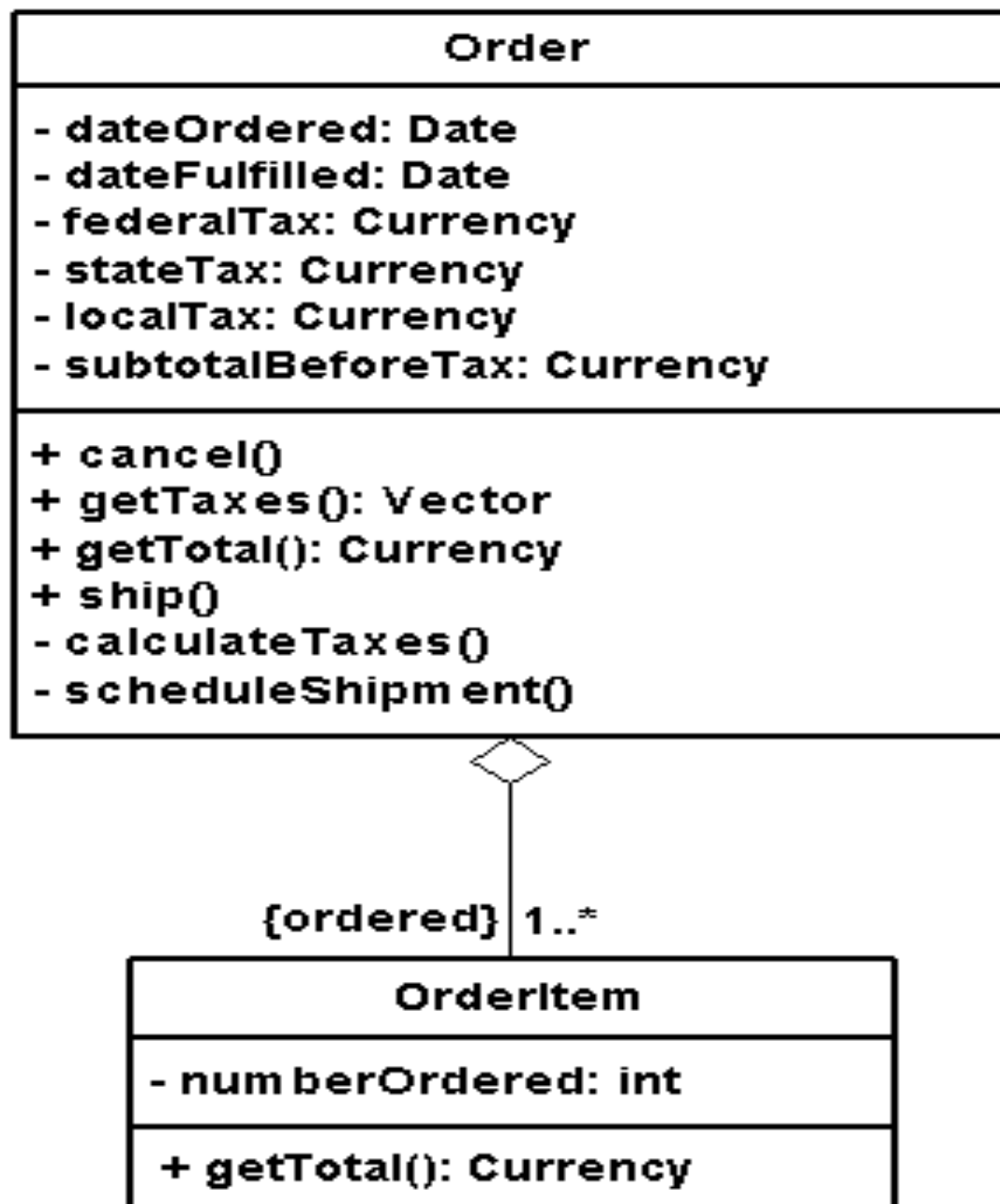
- *Mapare*. Determinarea modului în care obiectele și relațiile dintre ele vor fi păstrate într-un mediu de stocare permanent (ex. bază de date relațională, fișiere XML).
- *Proprietate*. O proprietate care poate avea asociat un atribut `firstName:string` sau o metodă prin care se determină valoarea `getTotal()`.
- *Maparea proprietății*. O mapare care descrie cum va fi stocată valoarea proprietății.
- *Maparea relațiilor*. O mapare care descrie cum vor fi persistate relațiile dintre unul sau mai multe obiecte (asociere, agregare, moștenire).

Mapări simple

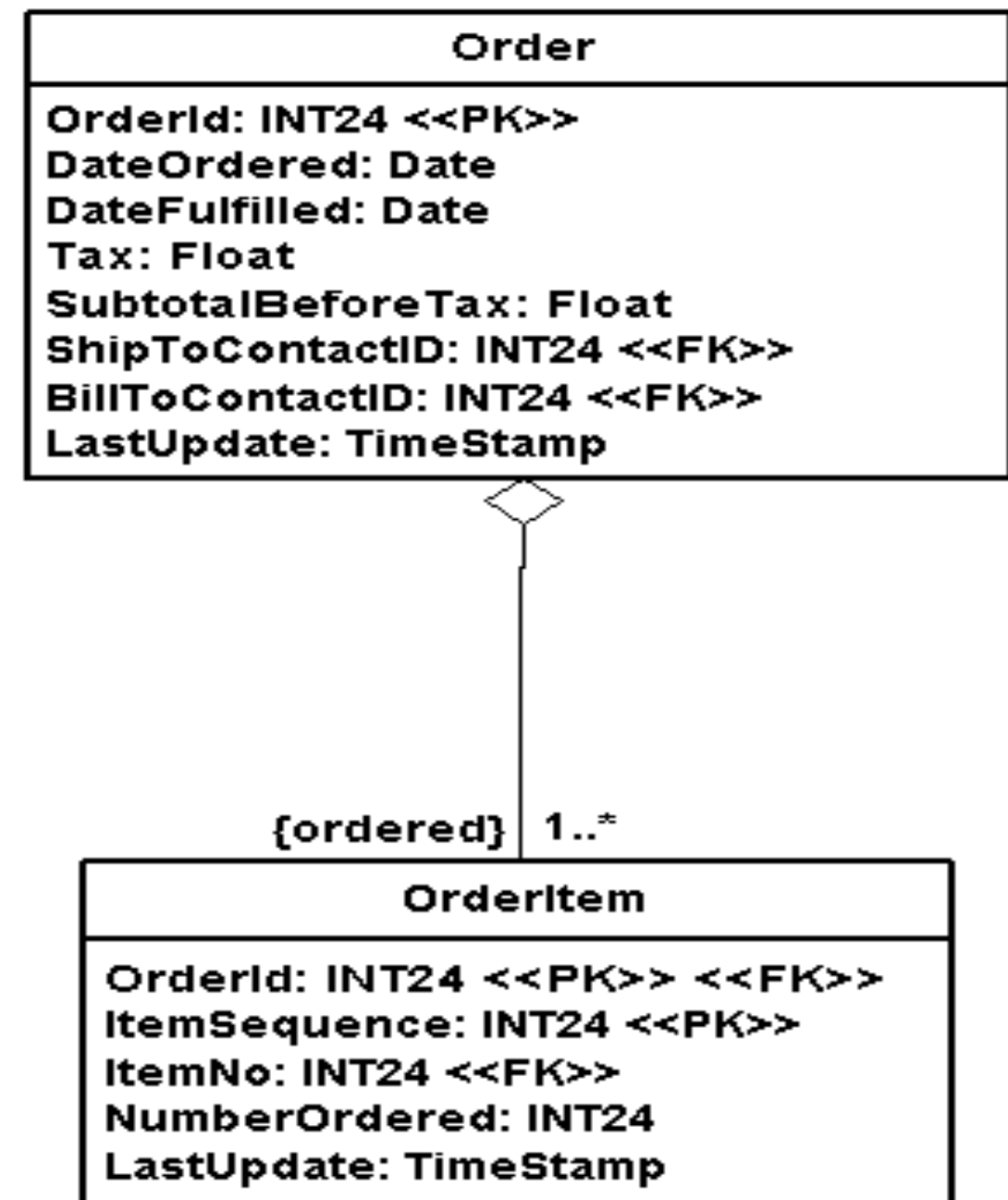
- O clasă este mapată într-o tabelă.
- Exceptând modele foarte simple, maparea unu-la-unu este rară.
- Cea mai simplă mapare este maparea unei proprietăți asociate unui singur atribut la o singură coloană din tabela corespunzătoare.
- Este și mai simplu dacă ambele (atributul și coloana) au aceleași tipuri de bază.
 - ambele sunt stringuri/date,
 - atributul este un string, coloana e de tip char(varchar),
 - atributul este un număr, iar coloana este float.

Mapări simple

<<Class Model>>



<<Physical Data Model>>



Diferențe

- Sunt mai multe attribute pentru **tax** în modelul orientat obiect - doar o singură coloană în schema relațională. Cele trei attribute din clasa **Order** ar trebui însumate și rezultatul păstrat în tabelă.
- În schema relațională apar chei, în modelul orientat obiect nu există chei. Înregistrările din tabela relațională sunt identificate în mod unic de cheia primară, iar relațiile dintre tabele sunt păstrate folosind chei străine.
- Relațiile dintre obiecte sunt păstrate prin referințe, nu prin chei străine. Pentru a putea persista obiectele și relațiilor dintre ele, obiectele trebuie să știe de valoarea cheilor păstrate în baza de date pentru a le putea identifica. Informația adițională este numită “*shadow information*”.
- Sunt folosite diferite tipuri în modelul orientat obiect și în schema relațională:
 - atributul **subTotalBeforeTax** din clasa **Order** este de tip **Currency**
 - coloana **SubTotalBeforeTax** din tabela **Order** este de tip float.
 - Pentru a implementa maparea trebuie să putem converti între cele două reprezentări fără a pierde informații.

Shadow Information

- *Shadow information* sunt orice informații pe care obiectele trebuie să le păstreze (pe lângă informațiile normale) pentru a putea fi persistate.
- Include:
 - *Cheia primară*: în special când cheia primară este o cheie surogat care nu are altă semnificație în domeniu.
 - Informații pentru *controlul concurenței*: timestamps sau incremental counters.
 - Informații pentru *păstrarea versiunii*: *versioning* numbers.
- Exemplu: tabela **Order** are coloana **OrderID** folosită ca și cheie primară și coloana **LastUpdate** folosită pentru controlul concurenței care nu apar în clasa **Order**.

Mapare Metadata

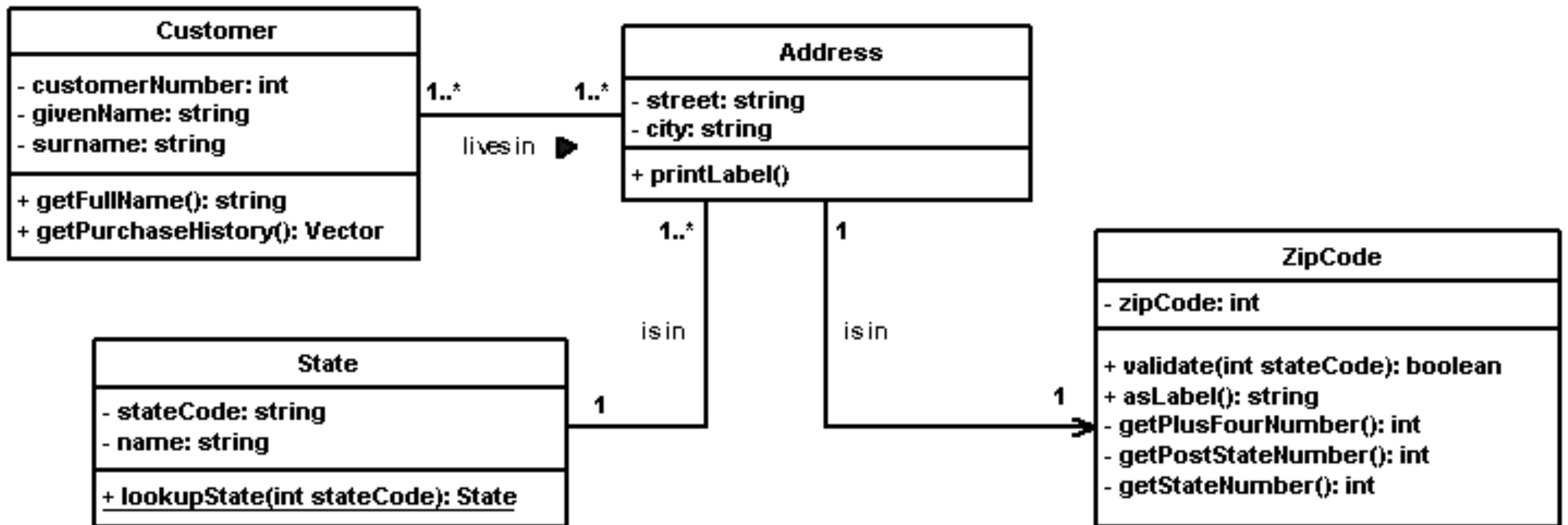
- Metadata păstrează informații despre date. Maparea metadatelor descrie modul în care metadatele reprezentând proprietățile sunt mapate la metadatele corespunzând tabelelor.

Proprietate (OO)	Coloana (BD)
Order.orderID	Order.OrderID
Order.dateOrdered	Order.DateOrdered
Order.dateFulfilled	Order.DateFulfilled
Order.getTotalTax()	Order.Tax
Order.subtotalBeforeTax	Order.SubtotalBeforeTax
Order.shipTo.personID	Order.ShipToContactID
Order.billTo.personID	Order.BillToContactID
Order.lastUpdate	Order.LastUpdate
OrderItem.ordered	OrderItem.OrderID
Order.orderItems.position(orderItem)	OrderItem.ItemSequence
OrderItem.item.number	OrderItem.ItemNo
OrderItem.numberOrdered	OrderItem.NumberOrdered

ORM Impedance Mismatch

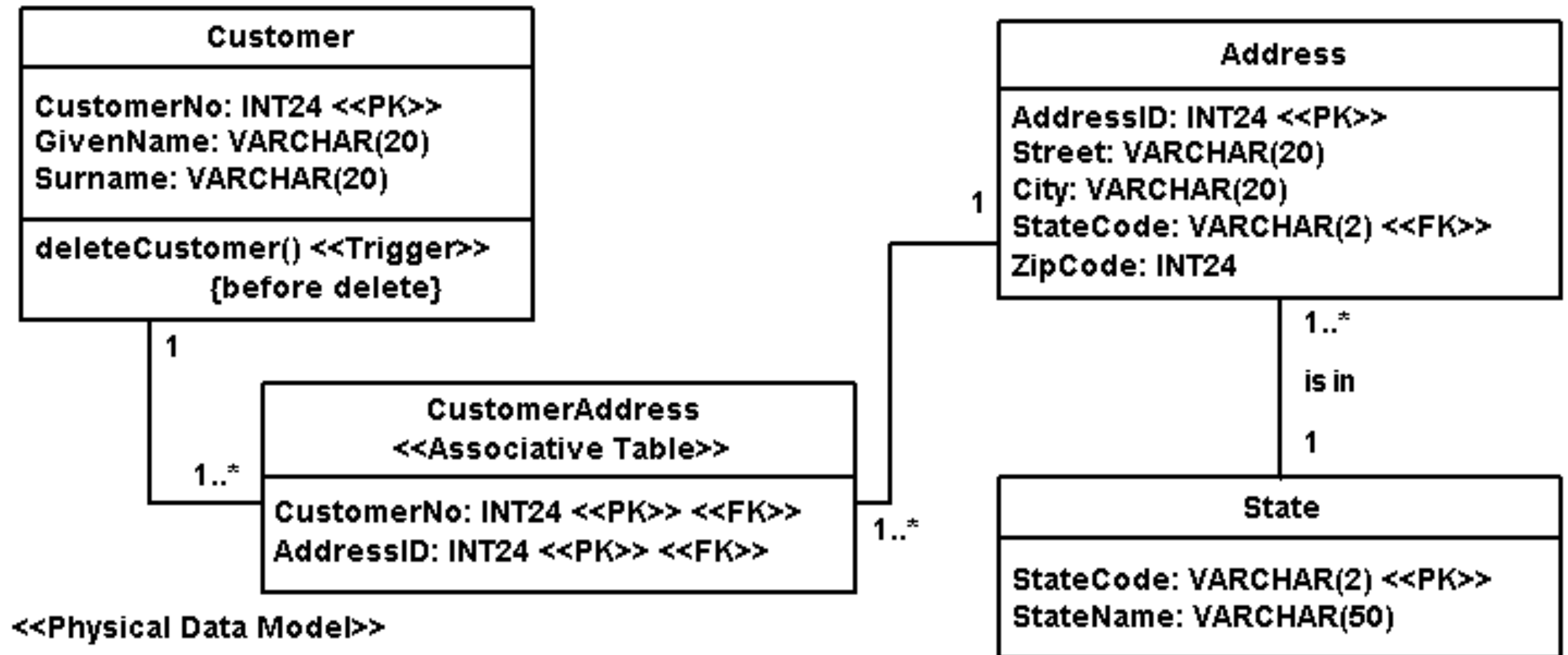
- Paradigma orientată obiect promovează dezvoltarea aplicațiilor folosind obiecte care păstrează date, dar conțin și logica aplicației.
- Bazele de date relaționale stochează datele în tabele și manipulează datele folosind proceduri stocate și interogări SQL.
- Diferențele dintre cele două abordări au fost numite: *object-relational impedance mismatch* sau doar *impedance mismatch*.
- Ex. în paradigma orientată obiect obiectele sunt traversate folosind relațiile dintre ele, în paradigma relațională se folosește operația de join.
- Tipurile de date diferite în limbajele orientate obiect și bazele de date relaționale:
 - Java: string și int - Oracle: varchar și smallint.
 - Java: colecții - Oracle: tabele
 - Java: obiecte - Oracle: blobs

ORM Impedance Mismatch



Copyright 2002-2006 Scott W. Ambler

ORM Impedance Mismatch



Medii de proiectare și programare

2024-2025

Curs 9

Conținut

- Object-Relational Mapping (ORM)
 - Hibernate
 - Entity Framework
- Servicii Web

ORM Impedance Mismatch

- Paradigma orientată obiect promovează dezvoltarea aplicațiilor folosind obiecte care păstrează date, dar conțin și logica aplicației.
- Bazele de date relaționale stochează datele în tabele și manipulează datele folosind proceduri stocate și interogări SQL.
- Diferențele dintre cele două abordări au fost numite: *object-relational impedance mismatch* sau doar *impedance mismatch*.
- Ex. în paradigma orientată obiect obiectele sunt traversate folosind relațiile dintre ele, în paradigma relațională se folosește operația de join.
- Tipurile de date diferite în limbajele orientate obiect și bazele de date relaționale:
 - Java: string și int - Oracle: varchar și smallint.
 - Java: colecții - Oracle: tabele
 - Java: obiecte - Oracle: blobs

Strategii pentru Impedance Mismatch

- Maparea moștenirii
- Maparea relațiilor dintre obiecte
- Maparea proprietăților statice

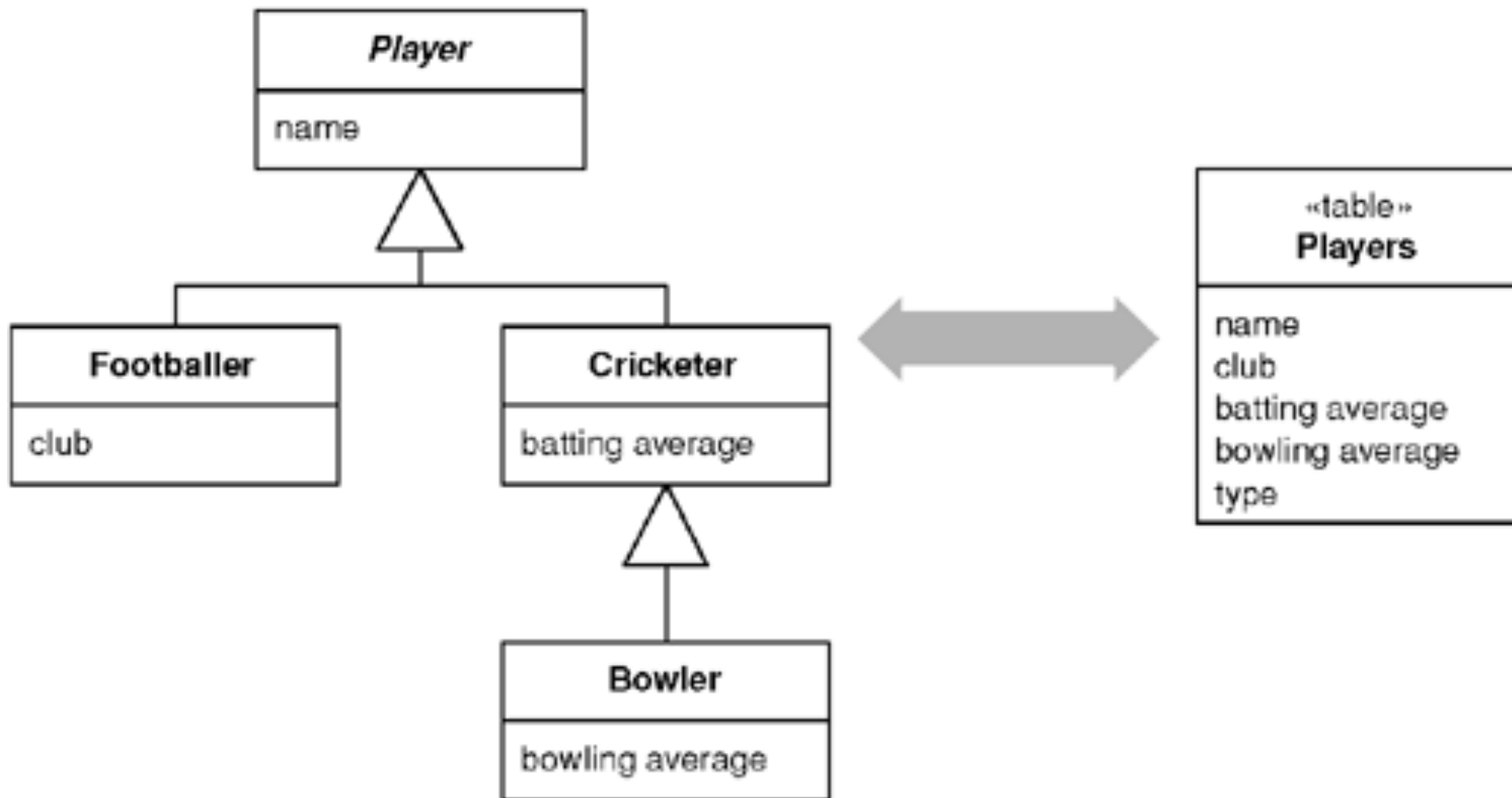
Maparea moștenirii

- Bazele de date relaționale nu suportă moștenirea.
- Programatorul trebuie să mapeze moștenirea dintre entitățile din modelul orientat obiect într-o bază de date relațională.
- Tehnici:
 - Maparea ierarhiei de clase într-o singură tabelă.
 - Maparea fiecărei clase într-o tabelă.
 - Maparea fiecărei clase concrete în tabela ei.
 - Maparea claselor într-o structura de tabele generică.

Moștenirea - O singură tabelă

- Reprezentarea unei ierarhii de clase (moștenire) **ca și o singură tabelă** cu coloane pentru toate attributele din toate clasele din ierarhie.
- Fiecare clasă păstrează informațiile relevante pentru ea într-o înregistrare din tabelă. Coloanele care nu sunt relevante rămân goale.
- Când se încarcă un obiect din tabelă, instrumentul ORM trebuie să știe ce clasă să instanțieze.
- În tabelă se adaugă o coloană care indică ce clasă ar trebui instanțiată (numele clasei sau un cod):
 - Codul trebuie interpretat în codul sursă pentru a putea face maparea cu clasa corespunzătoare.
 - Numele clasei poate fi folosit direct pentru instanțiere (folosind reflecție).

Moștenirea - O singură tabelă



Moștenirea - O singură tabelă

```
Footballer fb=new Footballer("A A", "ABC")
Cricketer cr=new Cricketer("C C", 23);
Bowler bw=new Bowler("B B", 21, 47);
Footballer fb2=new Footballer("D D", "BGD");
Bowler bw2=new Bowler("H H", 12, 23);
```

Players

PK	Name	Club	BattlingAvg	BowlingAvg	Type
1	A A	ABC			Footballer
2	C C		23		Cricketer
3	B B		21	47	Bowler
4	D D	BGD			Footballer
5	H H		12	23	Bowler

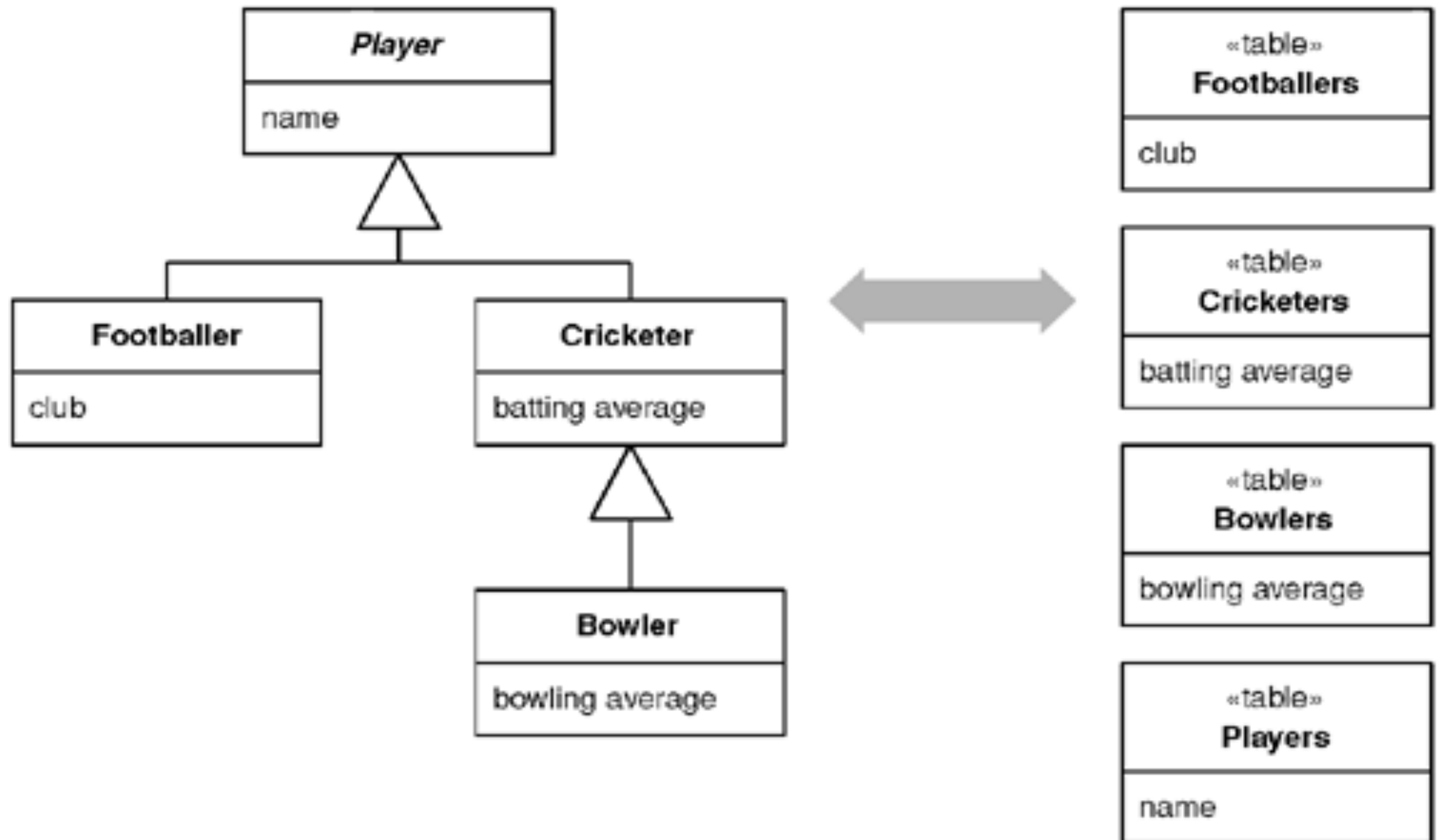
Moștenirea - O singură tabelă

- Avantaje:
 - Există doar o singură tabelă în baza de date.
 - Nu e nevoie de operații join pentru regăsirea informației.
 - Orice refactorizare care mută attributele în ierarhie nu necesită modificarea bazei de date.
- Dezavantaje:
 - Nu toate câmpurile din tabelă sunt relevante (depinde de tipul clasei). Este confuz pentru cei care folosesc tabelele direct (fără instrumentul ORM).
 - Coloanele folosite doar de subclase duc la spațiu nefolosit (multe coloane goale).
 - Tabela poate deveni prea mare, cu mulți indecși și blocări dese ale tablei. Poate afecta performanța.
 - Există un singur spațiu de nume pentru câmpuri, dezvoltatorul trebuie să se asigure că se vor folosi nume diferite în tabelă.
 - Adăugarea numelui clasei (prefix, postfix) poate ajuta. (ex. NumeClasă_NumeProprietate)

Tabelă pentru fiecare clasă

- Fiecare clasă din ierarhie are tabela ei.
- Atributele din clasă se mapează direct la coloanele corespunzătoare din tabelă.
- *Problemă: Cum se leagă înregistrările din tabele?*
 - *Soluția A:* folosirea cheii primare atât în tabela corespunzătoare clasei de bază cât și în clasa derivată. Deoarece clasa de bază are câte o înregistrare pentru fiecare înregistrare din clasele derivate, cheia primară va fi unica între toate tabele.
 - *Soluția B.* Fiecare tabelă să aibă cheia primară proprie, și folosirea cheii străine pentru a păstra legătura cu tabela corespunzătoare clasei de bază.
- *Provocare: încărcarea/regăsirea informațiilor din mai multe tabele în mod eficient.*
 - Operații de join între diferite tabele
 - Operațiile de join între mai mult de 3 sau 4 tabele sunt lente din cauza modului în care bazele de date optimizează operațiile interne.
- *Interogările asupra bazei de date sunt dificile.*

Tabelă pentru fiecare clasă



Tabelă pentru fiecare clasă

```
Footballer fb=new Footballer("A A", "ABC")
Cricketer cr=new Cricketer("C C", 23);
Bowler bw=new Bowler("B B",21, 47);
Footballer fb2=new Footballer("D D", "BGD");
Bowler bw2=new Bowler("H H",12, 23);
```

Players

PK	Name
1	A A
2	C C
3	B B
4	D D
5	H H

Footballers

PK	Club
1	ABC
4	BGD

Cricketers

PK	BattlingAvg
2	23
3	21
5	12

Bowlers

PK	BowlingAvg
3	47
5	23

Tabelă pentru fiecare clasă

- Avantaje:

- Toate coloanele sunt relevante pentru fiecare înregistrare, tabelele sunt mai ușor de înțeles și nu se folosește spațiu în mod ineficient.
- Relația dintre entitățile din modelul orientat obiect și baza de date relațională este ușor de înțeles.

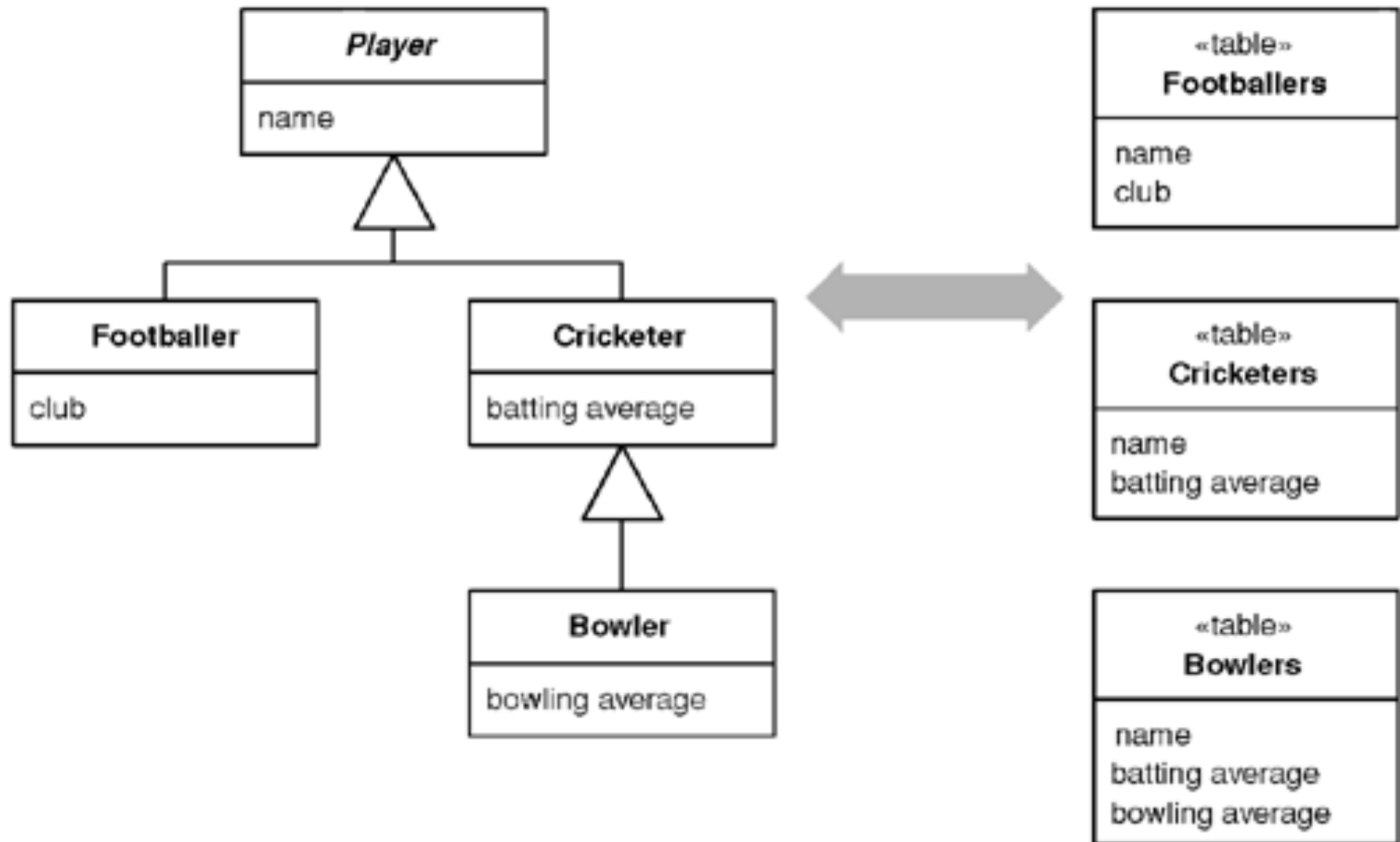
- Dezavantaje:

- Este necesară folosirea mai multor tabele pentru a încărca un obiect din mediu **persistent** (operație join sau mai multe interogări și folosirea memoriei).
- Orice refactorizare (mutarea câmpurilor în ierarhia de clase) cauzează modificarea structurii bazei de date.
- Tabelele corespunzătoare claselor de bază pot cauza probleme de performanță din cauza accesării dese.
- Normalizarea poate duce la înțelegerea dificilă a interogărilor ad-hoc.

Tabelă pentru fiecare clasă concretă

- Fiecare clasă concretă (non-abstract) din ierarhie are tabela ei.
- Fiecare tabelă conține coloane pentru toate proprietățile din ierarhie până la ea. ***Atributele din clasa de bază sunt duplicate în tabelele corespunzătoare subclaselor.***
- Este responsabilitatea programatorului de a se asigura că cheile sunt unice nu doar în tabela corespunzătoare clasei dar și între toate tabelele asociate ierarhiei.

Tabelă pentru fiecare clasă concretă



Tabelă pentru fiecare clasă concretă

```
Footballer fb=new Footballer("A A", "ABC")
Cricketer cr=new Cricketer("C C", 23);
Bowler bw=new Bowler("B B",21, 47);
Footballer fb2=new Footballer("D D", "BGD");
Bowler bw2=new Bowler("H H",12, 23);
```

Footballers

PK	Name	Club
1	A A	ABC
4	D D	BGD

Cricketers

PK	Name	BattlingAvg
2	C C	23

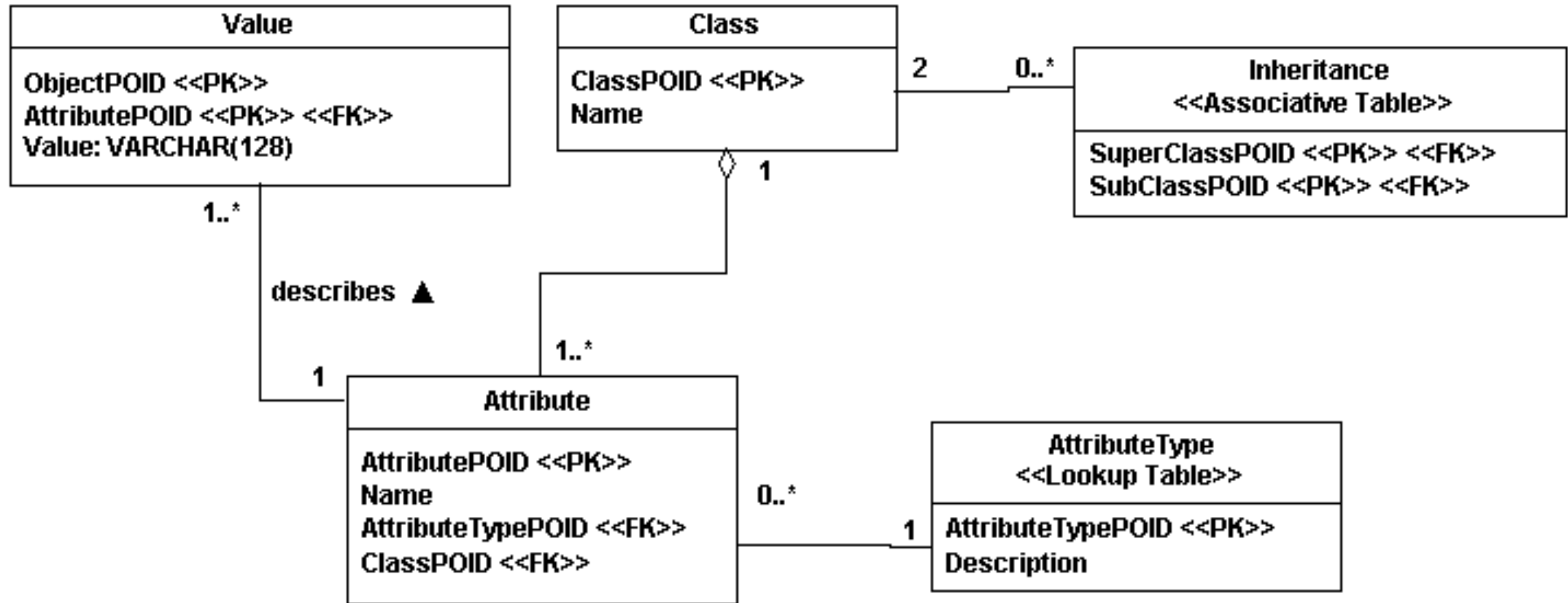
Bowlers

PK	Name	BattlingAvg	BowlingAvg
3	B B	21	47
5	H H	12	23

Tabelă pentru fiecare clasă concretă

- **Avantaje:**
 - Fiecare tabelă păstrează toate informațiile relevante și nu are câmpuri irelevante. Este ușor de înțeles și de alte aplicații care nu folosesc obiecte.
 - Nu este nevoie de operații join pentru citirea datelor.
 - Fiecare tabelă este accesată doar când clasa respectivă este accesată. Performanța este mai bună.
- **Dezavantaje:**
 - Gestiunea dificilă a cheilor primare.
 - Nu pot fi constrânse relațiile către clasele abstracte.
 - Dacă câmpurile din modelul obiectual sunt mutate în ierarhie, trebuie modificate definițiile tabelelor.
 - Dacă se modifică un câmp dintr-o clasă de bază, trebuie modificate toate tabelele corespunzătoare subclaselor, pentru că aceste câmpuri sunt duplicate.
 - O operație de căutare folosind clasa de bază, necesită căutări în toate **tabelele** (accesări multiple ale bazei de date sau o operație de join complicată).

Tabelle generiche



Tabele generice

- Avantaje:
 - Poate fi extinsă pentru a oferi suport pentru o gamă largă de mapări, inclusiv maparea relațiilor.
 - Este flexibilă, permite modificarea ușoară a modului în care sunt păstrate obiectele (trebuie modificate doar metadatele din tabelele *Class*, *Inheritance*, *Attribute* și *AttributeType*).
- Dezavantaje:
 - Este fezabilă doar pentru date de dimensiuni mici, deoarece necesită accesări dese ale bazei de date doar pentru reconstruirea unui singur obiect).
 - Interogările pot fi dificile deoarece necesită accesarea mai multor înregistrări pentru un singur obiect.

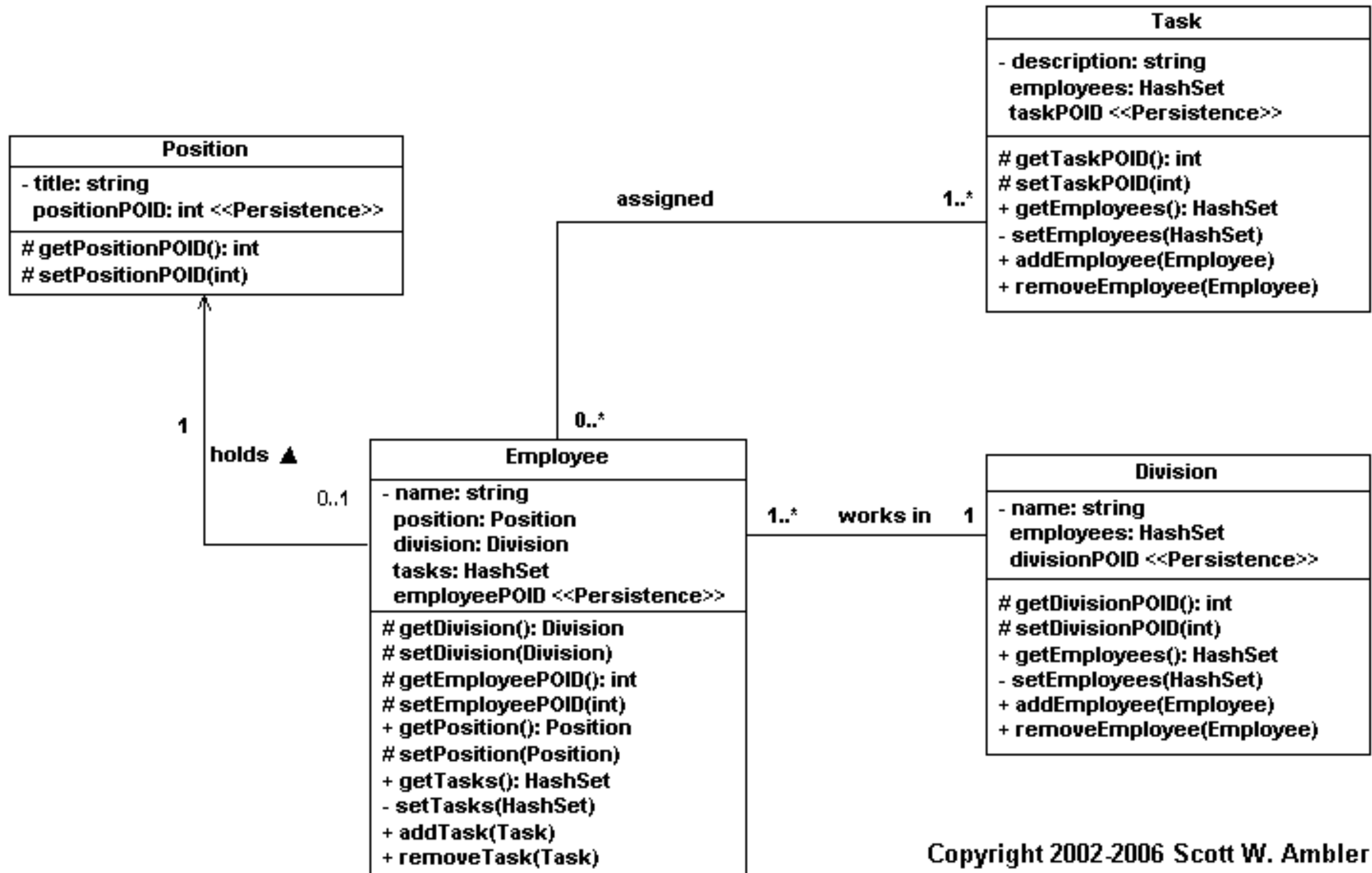
Maparea relațiilor dintre obiecte

- *Problema*: modul în care paradigma orientată obiect și modelul relațional tratează legăturile dintre obiecte/înregistrari, care cauzează două situații.
 1. *Diferența reprezentării*. Obiectele tratează legăturile prin păstrarea referințelor care există în timpul execuției (**referințele sunt temporare**). Bazele de date relaționale tratează legăturile prin păstrarea cheilor în tabele (**cheile sunt permanente**).
 2. *Obiectele pot folosi colecții pentru a gestiona mai multe referințe într-un singur atribut*. Normalizarea obligă ca toate legăturile/valorile să nu fie multiple. Se inversează structura de date dintre obiecte și tabele.
- Exemplu: Un obiect *Order* are o colecție de obiecte de tip *line item* care nu păstrează o referință către obiectul de tip order.
 - ★ Structura tablei este inversă, înregistrările *line item* includ o cheie străină către înregistrarea *order* corespunzătoare (câmpurile dintr-o înregistrare nu pot fi multivaloare).

Tipuri de relații

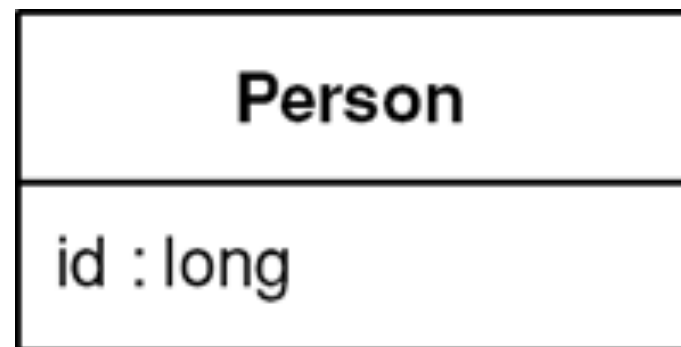
- Două categorii de relații între obiecte importante la mapare:
- **Multiplicitatea**. Include 3 tipuri:
 - Relații *unu-la-unu*. Maximul multiplicității la ambele capete este 1.
 - Relații *unu-la-n (sau n-la-unu)*. Multiplicitatea la unul dintre capete este 1, la celălalt capăt este n.
 - Relații *n-la-n*. Maximul multiplicității la ambele capete este mai mare decât 1.
- **Direcția**. Include 2 tipuri:
 - Relații *unidirecționale*. Un obiect știe de obiectul (obiectele) cu care are o legătură, dar celălalt obiect (celelalte obiecte) nu știe (nu știu) de el.
 - Relații *bidirecționale*. Ambele obiecte aflate într-o relație știu unul de celălalt.

Tipuri de relații



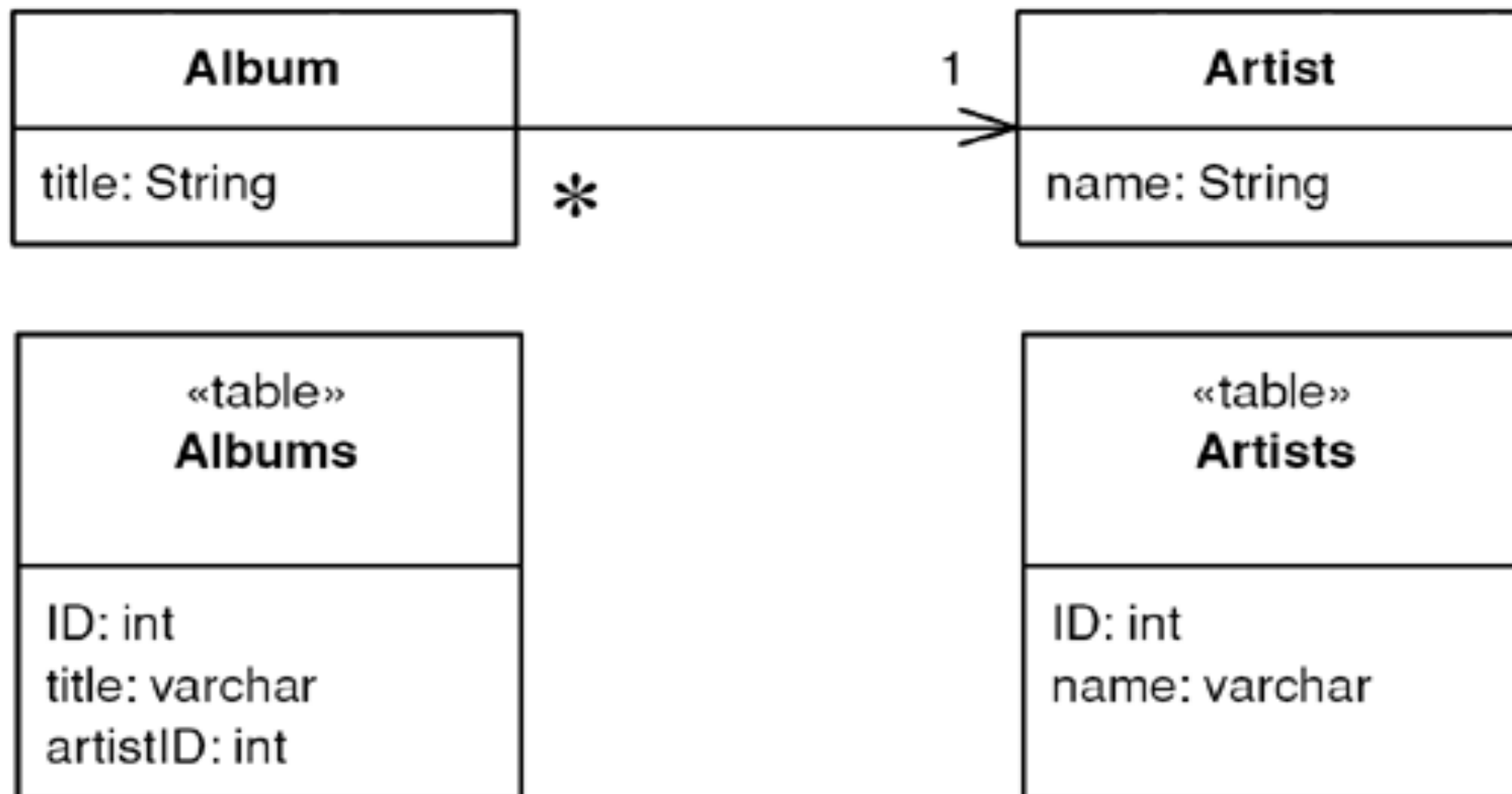
Șablonul *Identity*

- Salvează un identificator (ID) din baza de date într-un obiect pentru a păstra legătura dintre un obiect în memorie și o înregistrare din baza de date.
- Cheia primară dintr-o bază de date relațională este păstrată printre attributele obiectului.
- Șablonul ar trebui folosit când există o mapare între obiectele din memorie și înregistrările din baza de date (cheia primară este diferită de attributele din obiect).



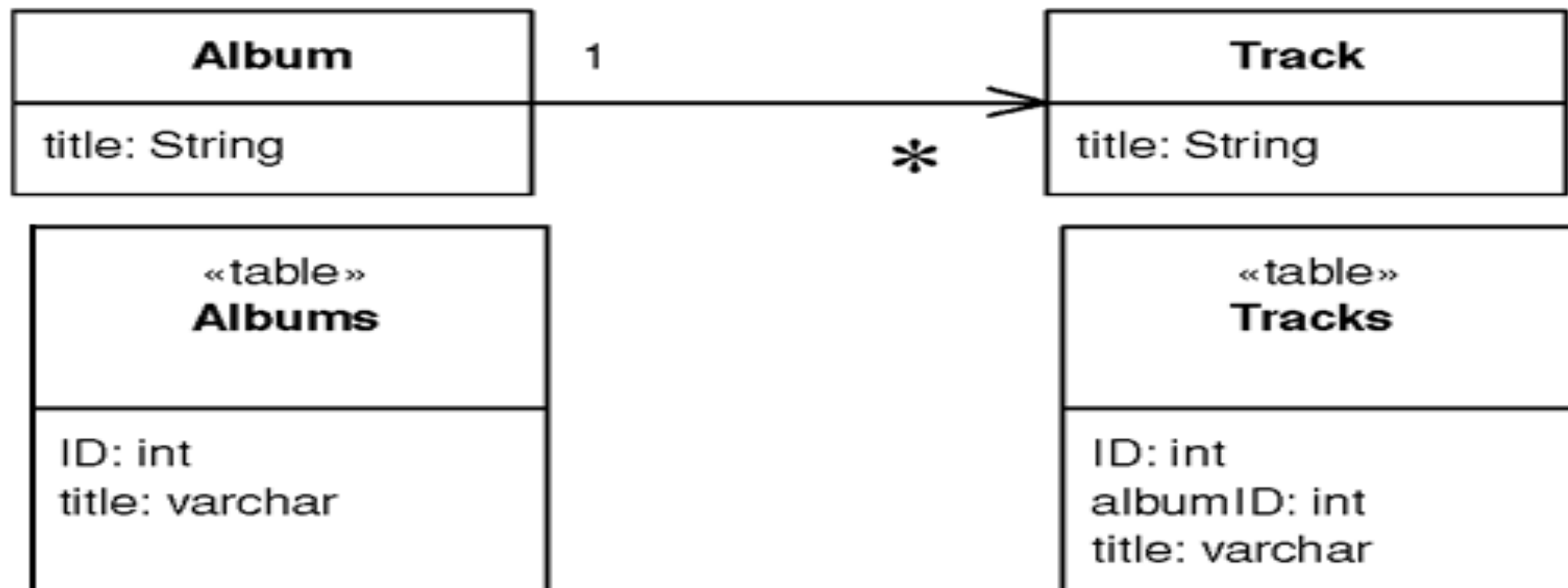
Maparea cheii străine

- Mapează o asociere dintre obiecte ca și cheie străină între tabelele dintr-o bază de date relațională.
- Fiecare obiect conține cheia din tabela corespunzătoare.
- Dacă două obiecte sunt legate printr-o relație de asociere, relația poate fi înlocuită printr-o cheie străină în baza de date.



Maparea cheii străine

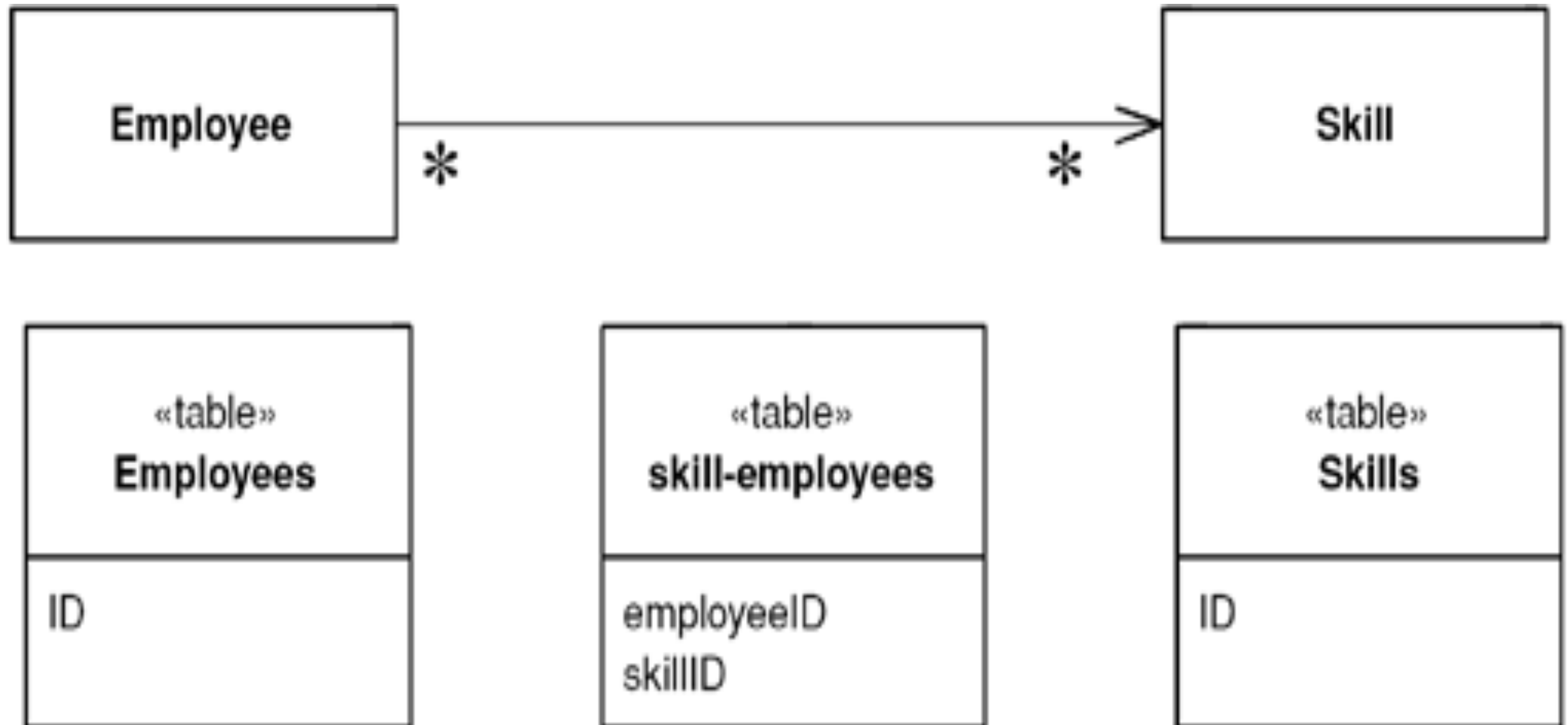
- Maparea unei colecții de obiecte.
- Într-o bază de date relațională nu se poate salva o colecție, trebuie inversată direcția referinței.
- Maparea cheii străine poate fi folosită pentru aproape toate asocierile dintre clase. Nu poate fi folosită pentru asocierea *n-la-n*.



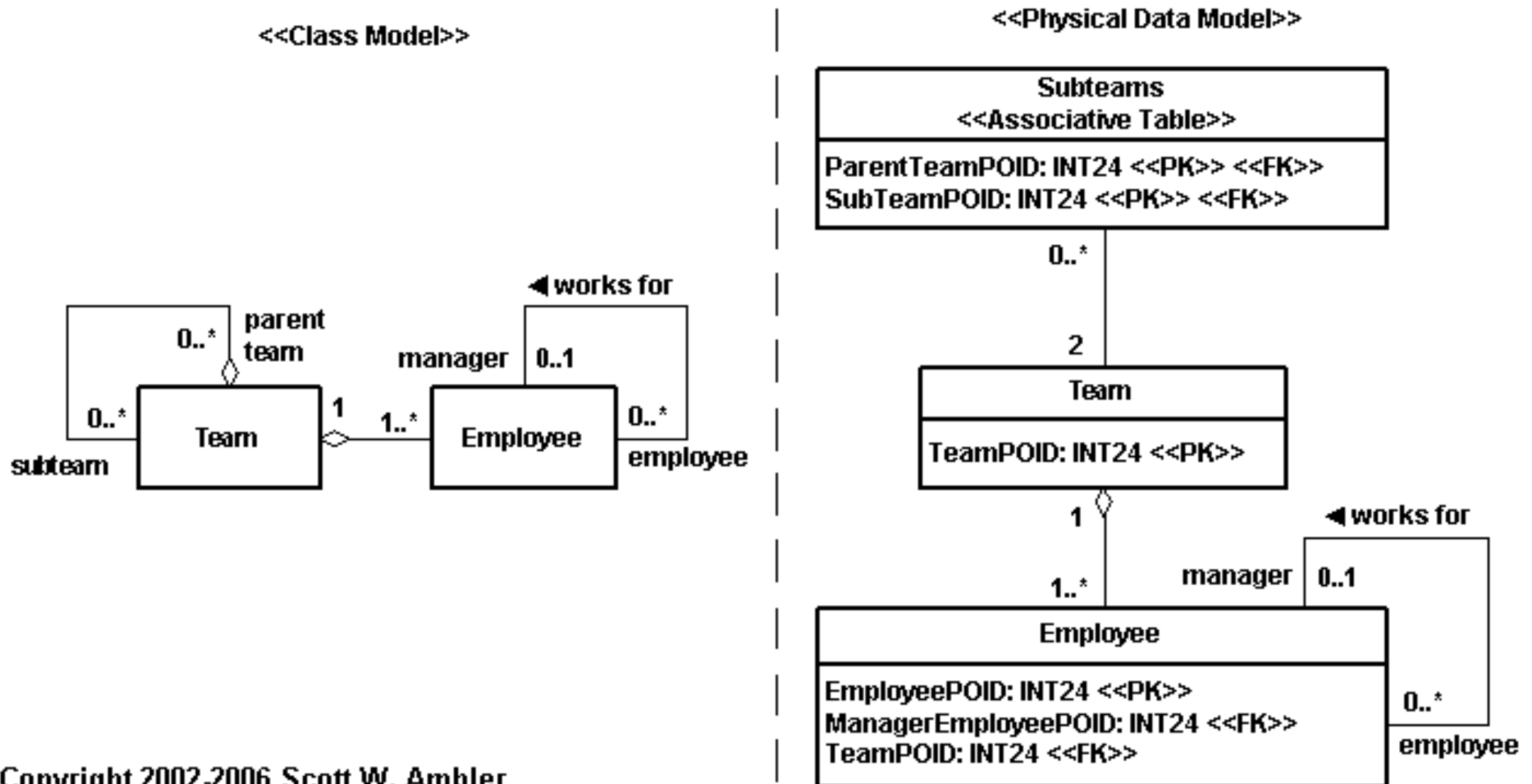
Maparea folosind o tabelă de asociere

- Salvează o asociere ca o tabelă cu chei străine către tabelele legate prin asociere.
- Obiectele pot păstra mulțimi de valori folosind colecții. Bazele de date relaționale nu au această caracteristică și sunt restrictionate la câmpuri cu o singură valoare.
- Ideea este de a crea o tabelă de legătură/asociere pentru a stoca asocierea.
- Tabela are doar două coloane corespunzătoare cheilor străine, conține câte o înregistrare pentru fiecare pereche de obiecte asociate.
- Tabela de legătura nu are echivalentul unui obiect în memorie (nu are ID). Cheia primară este compusă din cheile primare ale celor două tabele asociate.
- Tabela de asociere este folosită cel mai des pentru maparea asocierii n-la-n, dar poate fi folosită și pentru alte tipuri de asocieri (mai dificil, complex).

Tabela de asociere



Maparea asocierilor recursive



Maparea proprietăților statice

TableA

A
<u>StaticAttributeA</u> name

PK	Name	StaticAttributeA
	AA	1
	BB	2
	CC	1

TableA

PK	Name
	AA
	BB
	CC

StaticTableA

StaticAttributeA
1

Maparea proprietăților statice - Strategii (1)

- O tabelă cu o singură înregistrare, o singură coloană pentru fiecare proprietate statică:
 - Pro: Simplu, acces rapid
 - Con: multe tabele mici

A
<u>StaticAttributeA</u> name

B
<u>StaticAttrB1</u>
<u>StaticAttrB2</u>

C
<u>StaticAttrC1</u>

StaticTableA

StaticAttributeA
1

StaticTableB_1

StaticAttrB1
23

StaticTable_B2

StaticAttrB2
ValueA

StaticTableC

StaticAttrC1
23.56

Maparea proprietăților statice - Strategii (2)

- O tabelă cu mai multe coloane, o singură înregistrare pentru fiecare clasă:
 - Pro: Simplu, acces rapid
 - Con: multe tabele mici, dar mai puține decât la strategia precedentă

A
<u>StaticAttributeA</u> name

StaticTableA

StaticAttributeA
1

B
<u>StaticAttrB1</u> <u>StaticAttrB2</u>

StaticTableB

StaticAttrB1	StaticAttrB2
23	ValueA

C
<u>StaticAttrC1</u>

StaticTableC

StaticAttrC1
23.56

Maparea proprietăților statice - Strategii (3)

- O singură tabelă cu mai multe coloane - o singură înregistrare pentru toate clasele:
 - Pro: număr minim de tabele introdus
 - Con: potențiale probleme de concurență dacă mai multe clase trebuie să acceseze datele în același timp.

A
<u>StaticAttributeA</u> name

B
<u>StaticAttrB1</u> <u>StaticAttrB2</u>

C
<u>StaticAttrC1</u>

StaticAttributesTable

A.StaticAttributeA	B.StaticAttrB1	B.StaticAttrB2	C.StaticAttrC1
1	23	ValueA	23.56

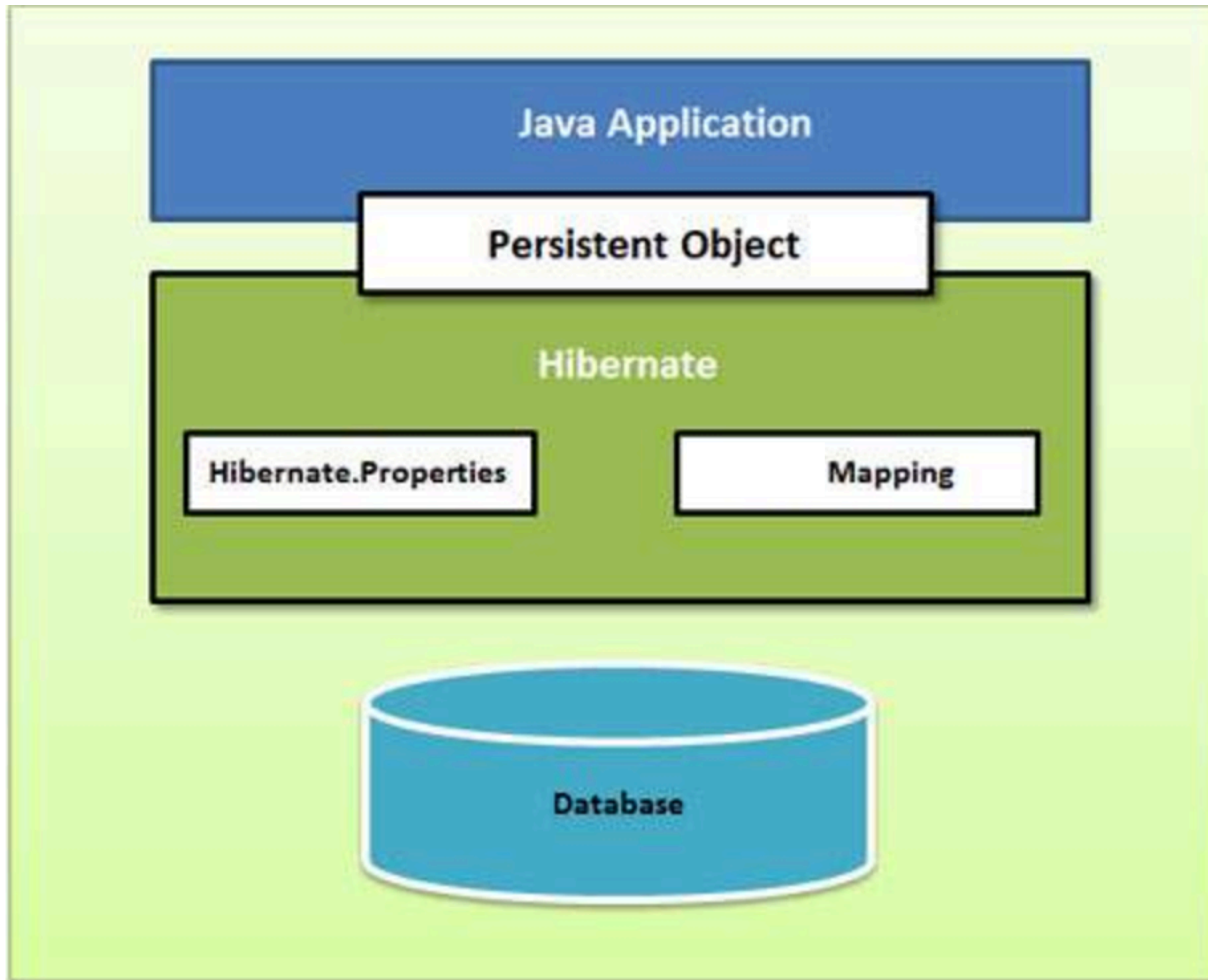
Maparea proprietăților statice

- Strategii (cont):
 - Schemă generică cu mai multe înregistrări pentru toate clasele:
 - Pro: număr minim de tabele introdus. Reduce problemele legate de concurență.
 - Con: Necesitatea convertirii între tipuri de date. Schema este asociată cu numele claselor și a proprietăților statice.

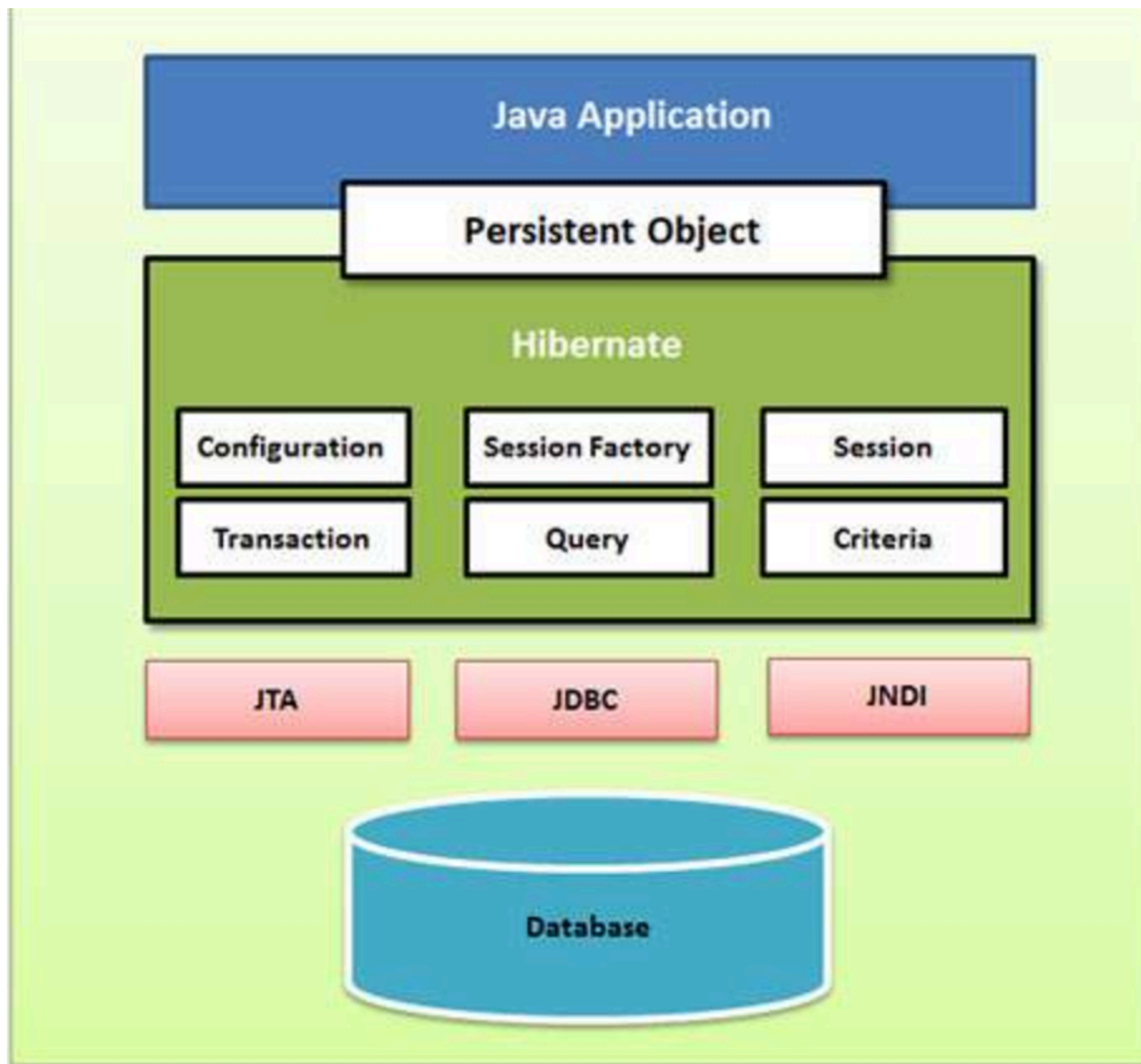
Hibernate

- Instrument open source pentru ORM.
- Aplicațiile care folosesc Hibernate definesc/specifică clasele persistente ce vor fi mapate în tabele într-o bază de date relațională.
- Toate instrucțiunile SQL sunt generate automat în timpul execuției.
- Informațiile despre mapare sunt transmise prin adnotări (vers<6, ca și un document de mapare în format XML).
- Adnotările specifică:
 - cum vor fi mapate proprietățile la coloanele din tabelă (tabele).
 - strategia aleasă de dezvoltator pentru mapare (unde este cazul).

Arhitectura



Arhitectura



Arhitectura

- Interfețele/Clasele definite de Hibernate pot fi clasificate astfel:
 - Interfețe folosite de aplicații pentru a efectua operații CRUD și interogări. Logica aplicației corespunzătoare nivelului de persistență va fi strâns cuplată de acestea: *Session*, *Transaction* și *Query*.
 - Clase apelate de aplicație pentru configurarea instrumentului Hibernate: clasa *Configuration*.
 - Interfețe callback care permit aplicațiilor să reacționeze la evenimente ce apar în timpul execuției instrumentului Hibernate: *Interceptor*, *Lifecycle* și *Validatable*.
 - Interfețe care permit extinderea Hibernate: *UserType*, *CompositeUserType* și *IdentifierGenerator*.
- Hibernate folosește Java API existent: JDBC, Java Transaction API (JTA) și Java Naming and Directory Interface (JNDI).

Interfețe de bază

- Interfața *Session*: este cea mai folosită interfață de către aplicațiile care folosesc Hibernate. O instanță de tip *Session* nu consumă multe resurse și poate fi ușor creată/distrusă.
- Interfața *SessionFactory*. Aplicațiile obțin instanțe de tip *Session* folosind un obiect de tip *SessionFactory*. Păstrează în cache instrucțiunile SQL generate dinamic și alte metadate legate de mapări folosite în timpul execuției.
- Clasa *Configuration*. Un obiect de tip *Configuration* este folosit pentru configurarea și pornirea Hibernate. Aplicațiile pot folosi acest obiect pentru a specifica locația bazei de date și a altor proprietăți specifice Hibernate.
- Interfața *Transaction*. Abstractizează codul corespunzător unei tranzacții de implementarea specifică folosită: o tranzacție JDBC, o tranzacție JTA UserTransaction, etc.
- Interfața *Query*. Permite efectuarea de interogări asupra bazei de date și controlează modul în care este executată interogarea. Interogările pot fi scrise în HQL sau folosind direct limbajul SQL corespunzător bazei de date.

Exemplu

- Etape:
 - Crearea claselor Java corespunzătoare entităților
 - Adăugarea informațiilor pentru mapare (adnotări)
 - Crearea fișierului de configurare Hibernate
 - Implementarea nivelului de persistență folosind funcțiile corespunzătoare
 - Testarea claselor

Exemplu - Entitatea

@Entity

```
public class Book {  
    private Long id;  
    private String title;  
  
    public Book() {} //obligatoriu  
    public Book(String title){ this.title=title;}  
  
    @Id  
    @GeneratedValue(strategy=IDENTITY)  
    public Long getId() { return id;}  
  
    public void setId(Long id) {this.id = id;}  
  
    @NotNull  
    public String getTitle() { return title;}  
  
    public void setTitle(String title) {this.title = title;}  
}
```

Exemplu - Entitatea

```
@Entity
@Table( name = "Messages")
public class Message {
    @Id
    @Column(name = "idMessage")
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "Message_Text")
    private String text;

    public Message() {} //obligatoriu
    public Message(String text) { this.text = text; }

    public Long getId() { return id; }
    private void setId(Long id) { this.id = id; }

    public String getText() { return text; }
    public void setText(String text) { this.text = text; }
}
```

Fișierul de configurare *hibernate.properties*

```
#hibernate.properties -numele implicit căutat de Hibernate
```

```
# URL-ul de conectare la baza de date (JDBC)
```

```
jakarta.persistence.jdbc.url=jdbc:sqlite:/Users/grigo/HibernateExample/hibernate6.db
```

```
# Credentials
```

```
#jakarta.persistence.jdbc.user=
```

```
#jakarta.persistence.jdbc.password=
```

```
jakarta.persistence.schema-generation.database.action=update
```

```
#none/create/etc
```

```
# SQL statement logging
```

```
hibernate.show_sql=true
```

```
hibernate.format_sql=true
```

```
hibernate.highlight_sql=true
```

Exemplu – Salvarea unei entități

```
//INSERT
```

```
void addBook(Book b) {  
    sessionFactory.inTransaction(session -> {  
        session.persist(b);  
        System.out.println("Book added "+b);  
    });  
}
```

```
//Book b=new Book("Test Book ");
```

```
//dupa rulare
```

```
//Book added Book{id='8', title='Test Book '}
```


Exemplu – Modificarea unei entități

```
//FindOne
Book findBook(int id){
    try(Session session=sessionFactory.openSession()){
        return session.find(Book.class, id);
    } //se apelează session.close()
    catch (Exception e){
        System.err.println("Eroare la findBook " +e);
    }
    return null;
}

//update
Book update(int id){
    Book b=findBook(id);
    b.setTitle("New Title");
    sessionFactory.inTransaction(session->{
        session.merge(b);
        //session.flush()
    });
    return b;
}
```


Exemplu – ștergerea unei entități

```
//DELETE  
void delete(Book book) {  
    sessionFactory.inTransaction(session -> {  
        session.remove(book) ;  
    }) ;  
}
```

Exemplu – selectarea unei entități

```
//SELECT
```

```
List<Book> getBooks() {  
    try( Session session= sessionFactory.openSession()) {  
        return session.createQuery("from Book where title like 'Test %'",  
Book.class).setFirstResult(0).  
            setMaxResults(5).list();  
    } //se apelează session.close()  
}
```

```
//Book - numele entității, NU a tabelii
```

```
//title - proprietate a clasei Book, NU numele coloanei din tabela
```

Exemplu – MainBook

```
public class MainBook {
    static SessionFactory sessionFactory;
    public static void main(String[] args) {
        // A SessionFactory is set up once for an application!
        sessionFactory = new Configuration() //cauta fisierul hibernate.properties
            .addAnnotatedClass(Book.class)
            .buildSessionFactory();

        Book b=new Book("Test Book ");
        // persist an entity
        addBook(b);

        System.out.println("Searching book having id 1 "+findBook(1));
        update(6);

        // query data using HQL
        for(Book bb:getBooks())
            System.out.println(bb);

        //delete an entity
        delete(b);

        //close sessionFactory
        sessionFactory.close();
    }
    //... codul metodelor addBook, findBook, update, getBooks, delete
}
```

Configurare Gradle

```
plugins {  
    id 'java'  
    id 'application'  
}  
  
group = 'ro.mpp'  
version = '1.0'  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
  
    implementation 'org.hibernate.orm:hibernate-core:6.4.4.Final'  
  
    // Hibernate Validator  
    implementation 'org.hibernate.validator:hibernate-validator:8.0.0.Final'  
    implementation 'org.glassfish:jakarta.el:4.0.2'  
  
    //pentru lucrul cu diferite baze de date: Sqlite, MariaDb, etc...  
    implementation 'org.hibernate.orm:hibernate-community-dialects:6.4.4.Final'  
  
    testImplementation platform('org.junit:junit-bom:5.9.1')  
    testImplementation 'org.junit.jupiter:junit-jupiter'  
  
    // driverul pentru conectarea la o baza de date Sqlite  
    runtimeOnly 'org.xerial:sqlite-jdbc:3.45.3.0'  
}
```

Interogări ale bazei de date

- Două posibilități:

- Hibernate Query Language

```
session.createQuery("from Category c where c.name like  
    'Laptop%'");
```

- SQL

```
session.createNativeQuery(  
    "select {c.*} from CATEGORY {c} where NAME like 'Laptop%'",  
    "c", Category.class);
```

Obținerea rezultatelor

- Metodele `list()/getResultList()` execută interogarea și returnează rezultatul ca și o listă:

```
List<User> result = session.createQuery("from User",  
    User.class).getResultList();
```

- Un singur obiect ca și rezultat :

```
Bid maxBid =session.createQuery("from Bid b order by  
    b.amount desc", Bid.class)  
    .setMaxResults(1)  
    .uniqueResult();
```

Interogări cu parametri

- Parametrii cu nume

```
String queryString = "from Item item where item.description  
    like :searchString and item.date > :minDate";  
List result = session.createQuery(queryString)  
    .setParameter("searchString", searchS)  
    .setParameter("minDate", minD).list();
```

- Parametrii cu poziție:

```
String queryString = "from Item item where item.description  
    like ?1 and item.date > ?2";  
List result = session.createQuery(queryString)  
    .setParameter(1, searchString)  
    .setParameter(2, minDate)  
    .list();
```


Hibernate Query Language (HQL)

Suportă aproape toate funcțiile și operațiile SQL:

- from clause: `from Cat as cat`
- select clause: `select foo from Foo foo, Bar bar where foo.startDate = bar.date`
- where clause: `from Cat as cat where cat.name='Fritz'`
- aggregate functions:

```
select cat.color, sum(cat.weight), count(cat) from Cat cat
group by cat.color
```

- order by clause
- group by clause
- expressions
- etc.

Maparea relațiilor

Adnotări pentru maparea relațiilor dintre clase:

- **@ManyToMany**
- **@ManyToOne**
- **@OneToOne**
- **@JoinTable** - tabela de asociere
- **@JoinColumn**

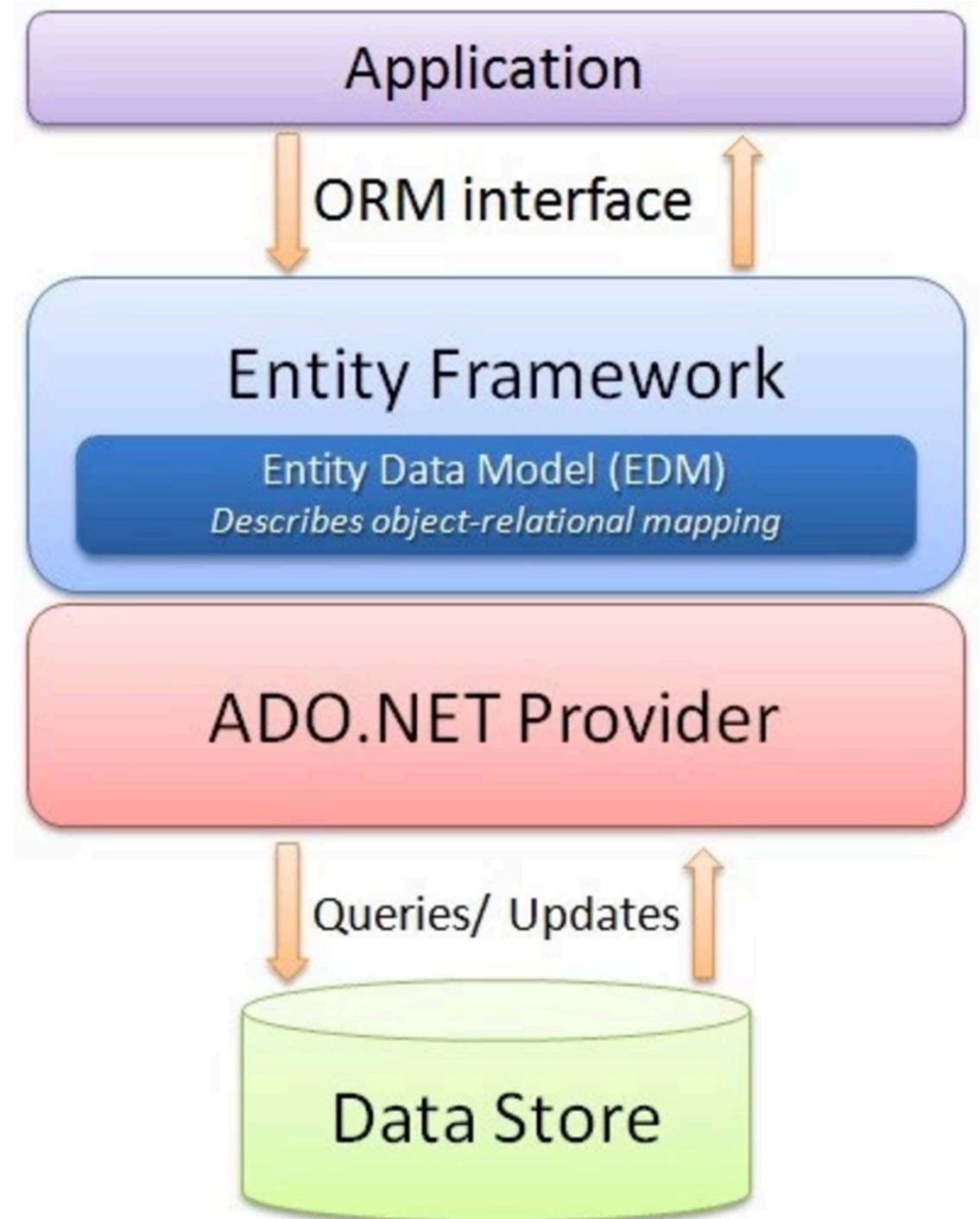
Fetching - **lazy**, **eager**

https://docs.jboss.org/hibernate/orm/6.3/introduction/html_single/Hibernate_Introduction.html

Exemplu Hibernate

.NET ORM

- Instrumente ORM bazate pe LINQ:
 - [LINQ to SQL](#) (L2S)
 - [Entity Framework](#) (EF)
- EF permite o mai bună decuplare a claselor de modelul relațional



.NET L2S

- Decorarea claselor folosind attribute .NET
- Spațiul de nume `System.Data.Linq.Mapping`

```
[Table (Name="Customers")]  
public class Customer  
{  
    [Column(IsPrimaryKey=true)]  
    public int ID {get;set};  
    [Column (Name="FullName")]  
    public string Name {get; set};  
}
```

.NET L2S

```
var context = new DataContext ("database connection string");

Table<Customer> customers = context.GetTable <Customer>();

// numărul de înregistrări din tabelă.
Console.WriteLine (customers.Count());

// Clientul cu Id-ul 2.
Customer cust = customers.Single (c => c.ID == 2);

Customer cust = customers.OrderBy (c => c.Name).First();
cust.Name = "Updated Name";
context.SubmitChanges();
```

.NET EF

- Decorarea claselor folosind attribute .NET
- Referință către [System.Data.Entity.dll](#)

```
[EdmEntityType (NamespaceName = "EFModel", Name = "Customer")]  
public partial class Customer  
{  
    [EdmScalarPropertyAttribute (EntityKeyProperty=true,  
        IsNullable=false)]  
    public int ID { get; set; }  
  
    [EdmScalarProperty (EntityKeyProperty = false, IsNullable = false)]  
        public string Name { get; set; }  
}
```

.NET EF

```
var context = new ObjectContext ("entity connection string");
    context.DefaultContainerName = "EntitiesContainer";
    ObjectSet<Customer> customers =
    context.CreateObjectSet<Customer>();

// numărul de înregistrări din tabelă
    Console.WriteLine (customers.Count());

// Clientul cu ID-ul 2
Customer cust = customers.Single (c => c.ID == 2);

Customer cust = customers.OrderBy (c => c.Name).First();
    cust.Name = "Updated Name";
    context.SaveChanges();
```

II JPA and Hibernate

7. Why JPA and Hibernate matter

Although JDBC does a very good job of exposing a common API that hides the database vendor-specific communication protocol, it suffers from the following shortcomings:

- The API is undoubtedly verbose, even for trivial tasks.
- Batching is not transparent from the data access layer perspective, requiring a specific API than its non-batched statement counterpart.
- Lack of built-in support for explicit locking and optimistic concurrency control.
- For local transactions, the data access is tangled with transaction management semantics.
- Fetching joined relations requires additional processing to transform the `ResultSet` into Domain Models or DTO (Data Transfer Object) graphs.

Although the primary goal of an ORM (Object-Relational Mapping) tool is to automatically translate object state transitions into SQL statements, this chapter aims to demonstrate that Hibernate can address all the aforementioned JDBC shortcomings.

Java Persistence history

The EJB 1.1 release offered a higher-level persistence abstraction through scalable enterprise components, known as Entity Beans. Although the design looked good on paper, in reality, the heavyweight RMI-based implementation proved to be disastrous from a performance perspective. Neither the EJB 2.0 support for *local interfaces* could revive the Entity Beans popularity, and, due to high-complexity and vendor-specific implementation details, most projects chose JDBC instead.

Hibernate was born out of all the frustration of using the Entity Bean developing model. As an open-source project, Hibernate managed to gain a lot of popularity, and so it soon became the *de facto* Java persistence framework.

In response to all the criticism associated with Entity Bean persistence, the Java Community Process advanced a lightweight POJO-based approach, and the JDO specification was born. Although JDO is data source agnostic, being capable of operating with both relational databases as well as NoSQL or even flat files, it never managed to hit mainstream popularity. For this reason, the Java Community Process decided that EJB3 would be based on a new specification, inspired by Hibernate and TopLink, and JPA (Java Persistence API) became the standard Java persistence technology.

The morale of this is that persistence is a very complex topic, and it demands a great deal of knowledge of both the database and the data access usage patterns.

7.1 The impedance mismatch

When a relational database is manipulated through an object-oriented program, the two different data representations start conflicting.

In a relational database, data is stored in tables, and the relational algebra defines how data associations are formed. On the other hand, an object-oriented programming (OOP) language allows objects to have both state and behavior, and bidirectional associations are permitted.

The burden of converging these two distinct approaches has generated much tension, and it has been haunting enterprise systems for a very long time.

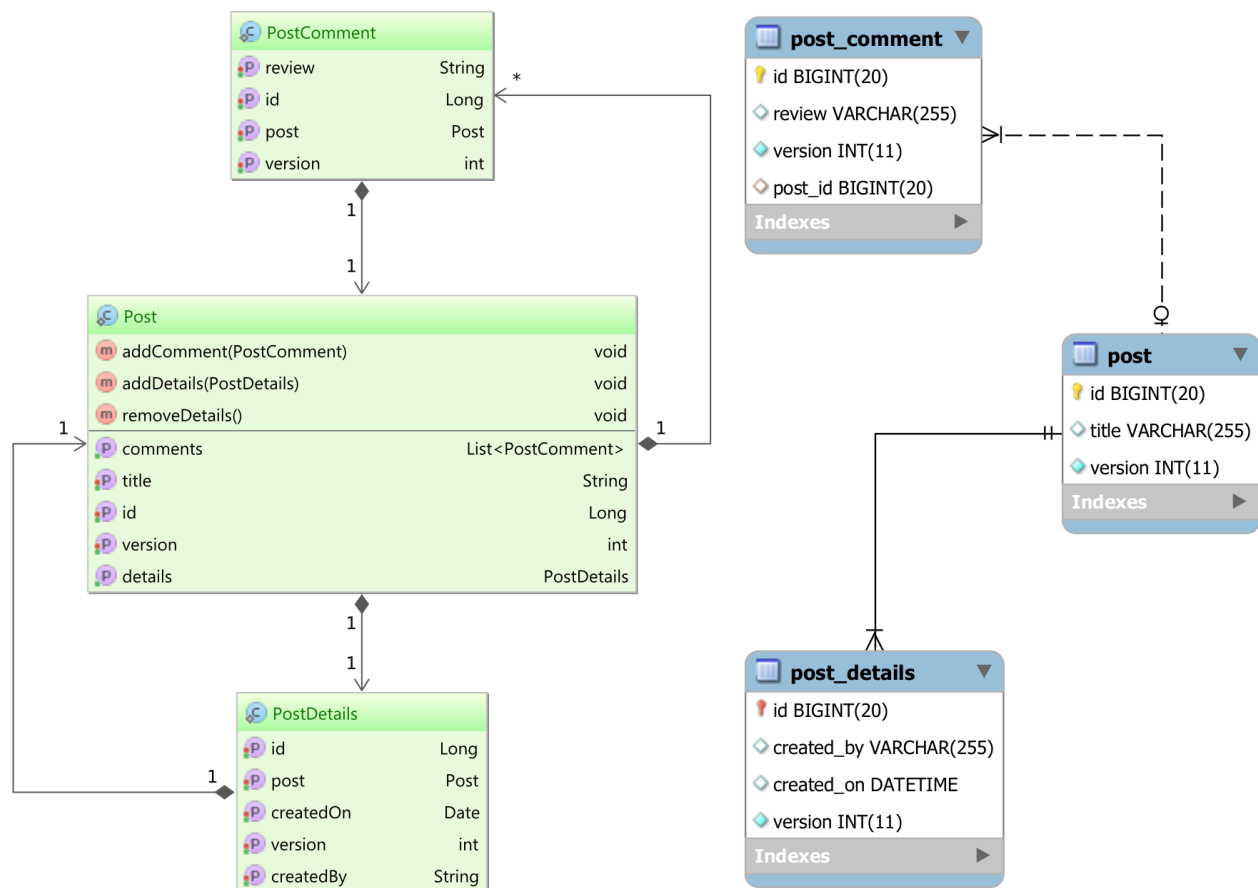


Figure 7.1: Object/Relational Mapping

The above diagram portrays the two different schemas that the data access layer needs to correlate. While the database schema is driven by the SQL standard specification, the Domain Model comes with an object-oriented schema representation as well.

The Domain Model encapsulates the business logic specifications and captures both data structures and the behavior that governs business requirements. OOP facilitates Domain Modeling, and many modern enterprise systems are implemented on top of an object-oriented programming language.

Because the underlying data resides in a relational database, the Domain Model must be adapted to the database schema and the SQL-driven communication protocol. The ORM design pattern helps to bridge these two different data representations and close the technological gap between them. Every database row is associated with a Domain Model object (Entity in JPA terminology), and so the ORM tool can translate the entity state transitions into DML statements.

From an application development point of view, this is very convenient since it is much easier to manipulate Domain Model relationships rather than visualizing the business logic through its underlying SQL statements.

7.2 JPA vs. Hibernate

JPA is only a specification. It describes the interfaces that the client operates with and the standard object-relational mapping metadata (Java annotations or XML descriptors). Beyond the API definition, JPA also explains (although not exhaustively) how these specifications are ought to be implemented by the JPA providers. JPA evolves with the Java EE platform itself (Java EE 6 featuring JPA 2.0 and Java EE 7 introducing JPA 2.1).

Hibernate was already a full-featured Java ORM implementation by the time the JPA specification was released for the first time. Although it implements the JPA specification, Hibernate retains its native API for both backward compatibility and to accommodate non-standard features.

Even if it is best to adhere to the JPA standard, in reality, many JPA providers offer additional features targeting a high-performance data access layer requirements. For this purpose, Hibernate comes with the following non-JPA compliant features:

- extended identifier generators (hi/lo, pooled, pooled-lo)
- transparent prepared statement batching
- customizable CRUD (@SQLInsert, @SQLUpdate, @SQLDelete) statements
- static/dynamic entity/collection filters (e.g. @FilterDef, @Filter, @Where)
- mapping attributes to SQL fragments (e.g. @Formula)
- immutable entities (e.g. @Immutable)
- more flush modes (e.g. FlushMode.MANUAL, FlushMode.ALWAYS)
- querying the second-level cache by the natural key of a given entity
- entity-level cache concurrency strategies
(e.g. Cache(usage = CacheConcurrencyStrategy.READ_WRITE))
- versioned bulk updates through HQL
- exclude fields from optimistic locking check (e.g. @OptimisticLock(excluded = true))
- versionless optimistic locking (e.g. OptimisticLockType.ALL, OptimisticLockType.DIRTY)
- support for skipping (without waiting) pessimistic lock requests
- support for Java 8 Date and Time and stream()
- support for multitenancy



If JPA is the interface, Hibernate is one implementation and implementation details always matter from a performance perspective.

The JPA implementation details leak and ignoring them might hinder application performance or even lead to data inconsistency issues. As an example, the following JPA attributes have a peculiar behavior, which can surprise someone who is familiar with the JPA specification only:

- The `FlushModeType.AUTO`¹ does not trigger a flush for native SQL queries like it does for JPQL or Criteria API.
- The `FetchType.EAGER`² might choose a SQL join or a secondary select whether the entity is fetched directly from the `EntityManager` or through a JPQL (Java Persistence Query Language) or a Criteria API query.

That is why this book is focused on how Hibernate manages to implement both the JPA specification and its non-standard native features (that are relevant from an efficiency perspective).

Portability concerns

Like other non-functional requirements, portability is a feature, and there is still a widespread fear of embracing database-specific or framework-specific features. In reality, it is more common to encounter enterprise applications facing data access performance issues than having to migrate from one technology to the other (be it a relational database or a JPA provider).

The lowest common denominator of many RDBMS is a superset of the SQL-92 standard (although not entirely supported either). SQL-99 supports Common Table Expressions, but MySQL 5.7 does not. Although SQL-2003 defines the `MERGE` operator, PostgreSQL 9.5 favored the `UPSERT` operation instead. By adhering to a SQL-92 syntax, one could achieve a higher degree of database portability, but the price of giving up database-specific features can take a toll on application performance. Portability can be addressed either by subtracting non-common features or through specialization. By offering different implementations, for each supported database system (like the jOOQ framework does), portability can still be achieved.

The same argument is valid for JPA providers too. By layering the application, it is already much easier to swap JPA providers, if there is even a compelling reason for switching one mature JPA implementation with another.

¹<https://docs.oracle.com/javaee/7/api/javax/persistence/FlushModeType.html>

²<https://docs.oracle.com/javaee/7/api/javax/persistence/FetchType.html#EAGER>

7.3 Schema ownership

Because of data representation duality, there has been a rivalry between taking ownership of the underlying schema. Although theoretically, both the database and the Domain Model could drive the schema evolution, for practical reasons, the schema belongs to the database.

An enterprise system might be too large to fit into a single application, so it is not uncommon to split in into multiple subsystems, each one serving a specific goal. As an example, there can be front-end web applications, integration web services, email schedulers, full-text search engines and back-end batch processors that need to load data into the system. All these subsystems need to use the underlying database, whether it is for displaying content to the users or dumping data into the system.

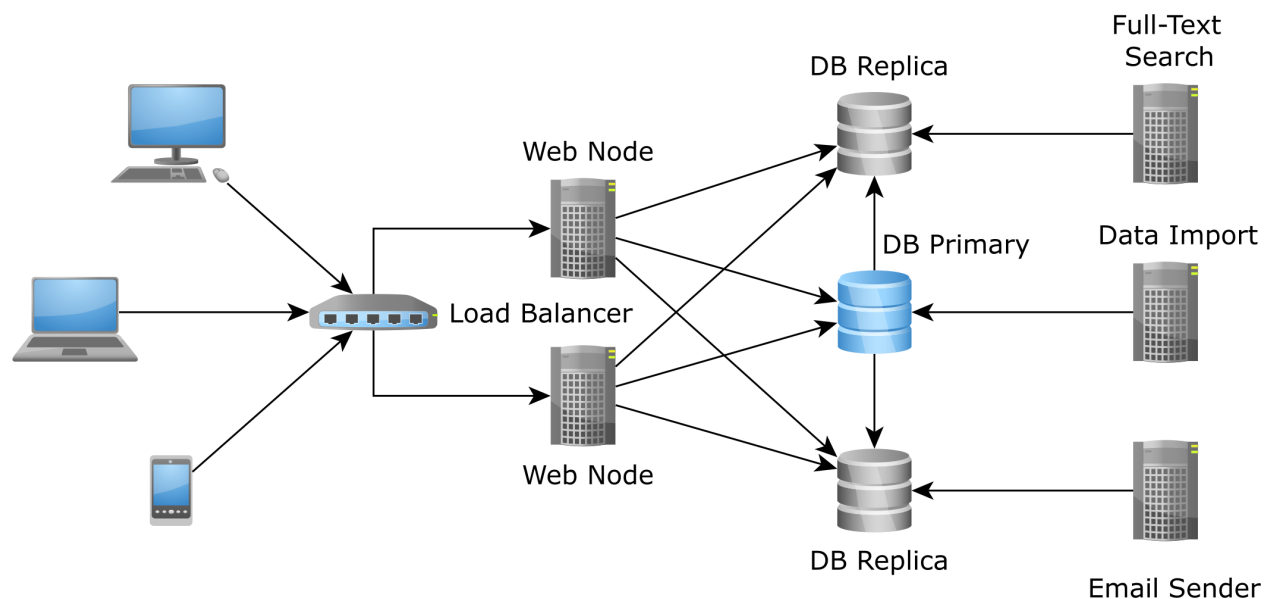


Figure 7.2: Database-centric integration

Although it might not fit any enterprise system, having the database as a central integration point can still be a choice for many reasonable size enterprise systems.

The relational database concurrency models offer strong consistency guarantees, therefore having a significant advantage to application development. If the integration point does not provide transactional semantics, it will be much more difficult to implement a distributed concurrency control mechanism.

Most database systems already offer support for various replication topologies, which can provide more capacity for accommodating an increase in the incoming request traffic. Even if the demand for more data continues to grow, the hardware is always getting better and better (and cheaper too), and database vendors keep on improving their engines to cope with more data.

For these reasons, having the database as an integration point is still a relevant enterprise system design consideration.

The distributed commit log

For very large enterprise systems, where data is split among different providers (relational database systems, caches, Hadoop, Spark), it is no longer possible to rely on the relational database to integrate all disparate subsystems.

In this case, [Apache Kafka](http://kafka.apache.org/)^a offers a fault-tolerant and scalable append-only log structure, which every participating subsystem can read and write concurrently.

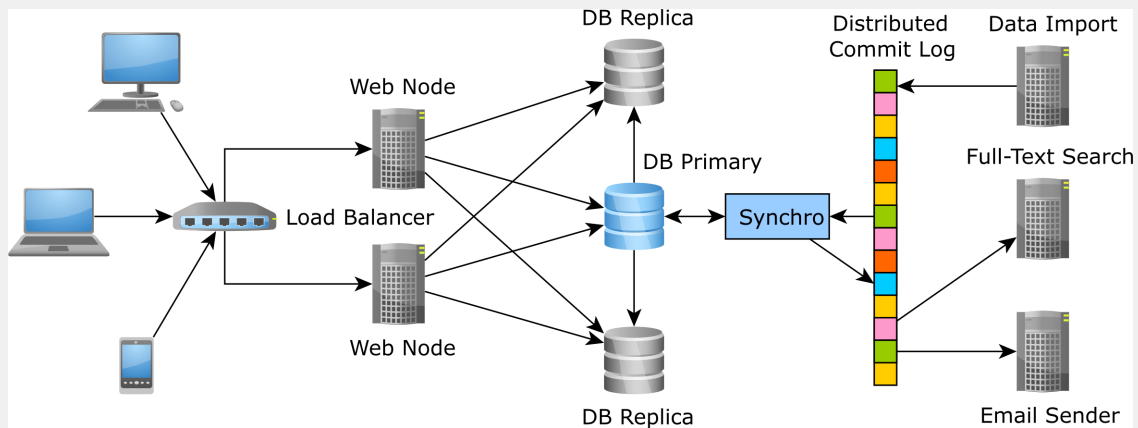


Figure 7.3: Distributed commit log integration

The commit log becomes the integration point, each distributed node individually traversing it and maintaining client-specific pointers in the sequential log structure. This design resembles a database replication mechanism, and so it offers durability (the log is persisted on disk), write performance (append-only logs do not require random access) and read performance (concurrent reads do not require blocking) as well.

^a<http://kafka.apache.org/>

No matter what architecture style is chosen, there is still need to correlate the transient Domain Model with the underlying persistent data.

The data schema evolves along the enterprise system itself, and so the two different schema representations must remain congruent at all times.

Even if the data access framework can auto-generate the database schema, the schema must be migrated incrementally, and all changes need to be traceable in the VCS (Version Control System) as well. Along with table structure, indexes and triggers, the database schema is, therefore, accompanying the Domain Model source code itself. A tool like [Flywaydb](http://flywaydb.org/)³ can automate the database schema migration, and the system can be deployed continuously, whether it is a test or a production environment.

³<http://flywaydb.org/>



The schema ownership goes to the database, and the data access layer must assist the Domain Model to communicate with the underlying data.

7.4 Entity state transitions

JPA shifts the developer mindset from SQL statements to entity state transitions. An entity can be in one of the following states:

Table 7.1: JPA entity states

State	Description
New (Transient)	A newly created entity which is not mapped to any database row is considered to be in the New or Transient state. Once it becomes managed, the Persistence Context issues an insert statement at flush time.
Managed (Persistent)	A Persistent entity is associated with a database row, and it is being managed by the currently running Persistence Context. State changes are detected by the <i>dirty checking</i> mechanism and propagated to the database as update statements at flush time.
Detached	Once the currently running Persistence Context is closed, all the previously managed entities become detached. Successive changes are no longer tracked, and no automatic database synchronization is going to happen.
Removed	A removed entity is only scheduled for deletion, and the actual database delete statement is executed during Persistence Context flushing.

The Persistence Context captures entity state changes, and, during flushing, it translates them into SQL statements. The JPA [EntityManager](http://docs.oracle.com/javaee/7/api/javax/persistence/EntityManager.html#persist-java.lang.Object-)⁴ and the Hibernate [Session](https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/Session.html)⁵ (which includes additional methods for moving an entity from one state to the other) interfaces are gateways towards the underlying Persistence Context, and they define all the entity state transition operations.

⁴<http://docs.oracle.com/javaee/7/api/javax/persistence/EntityManager.html#persist-java.lang.Object->

⁵<https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/Session.html>

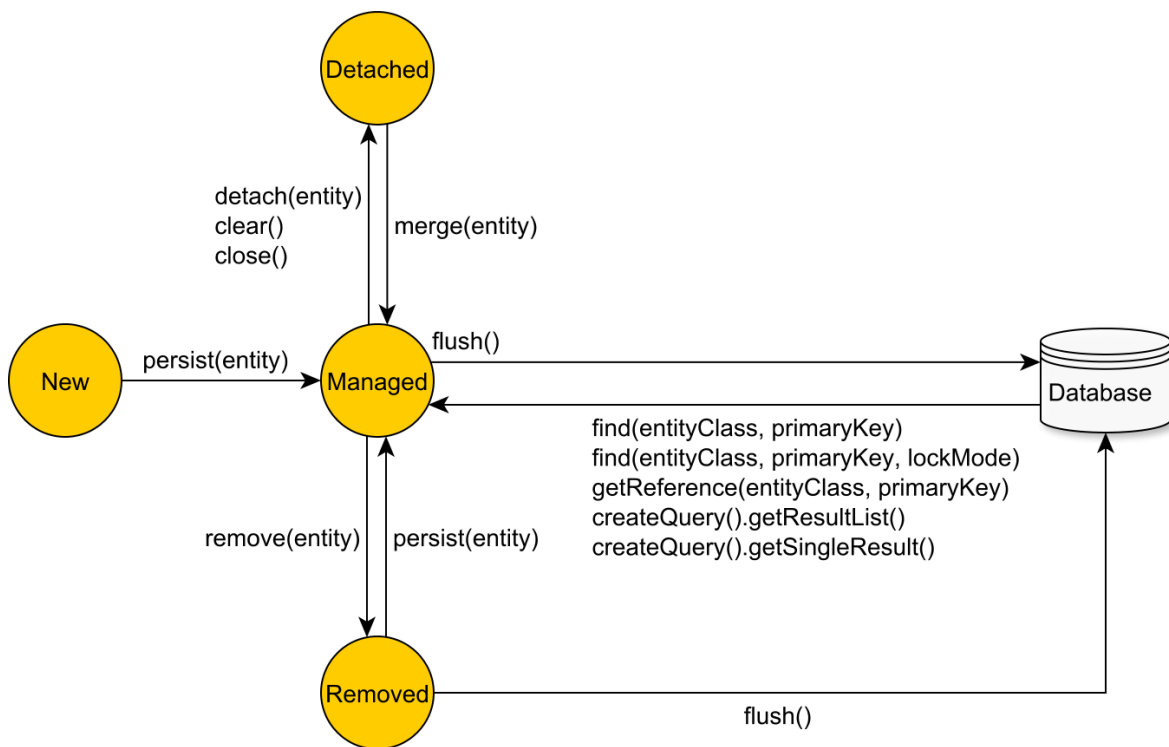


Figure 7.4: JPA entity state transitions

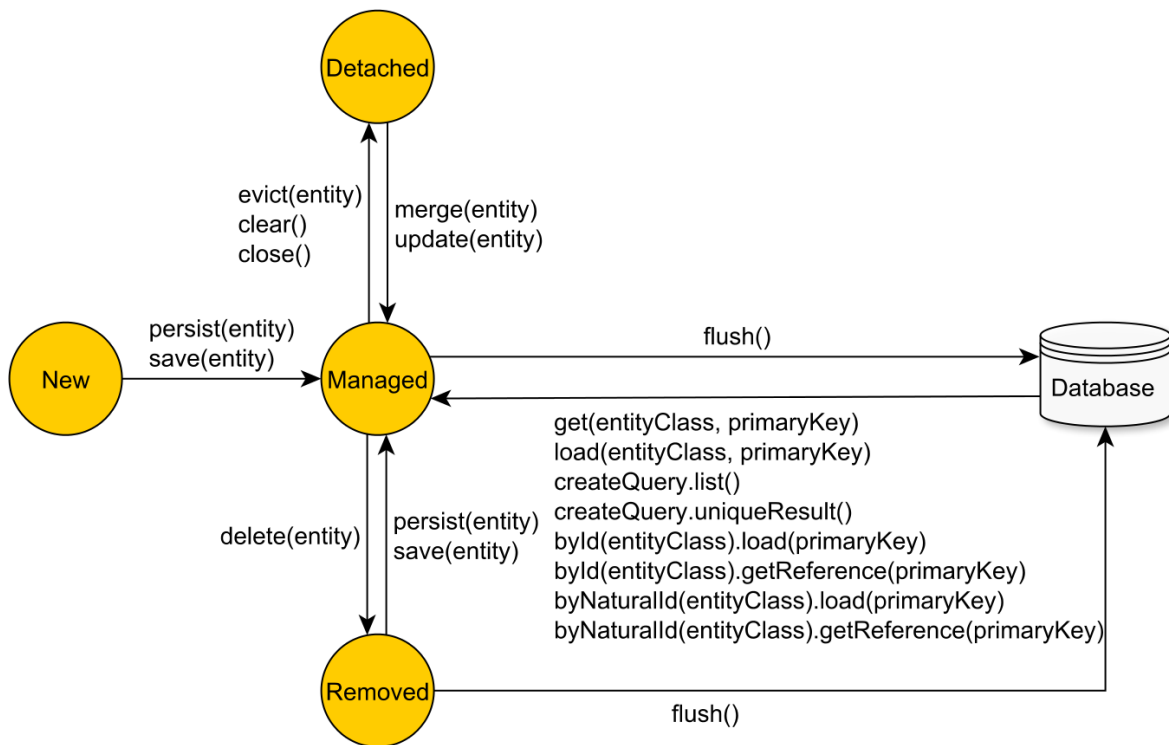


Figure 7.5: Hibernate entity state transitions

7.5 Write-based optimizations

SQL injection prevention

By managing the SQL statement generation, the JPA tool can assist in minimizing the risk of SQL injection attacks. The less the chance of manipulating SQL `String` statements, the safer the application can get. The risk is not completely eliminated because the application developer can still recur to concatenating SQL or JPQL fragments, so rigor is advised.



Hibernate uses `PreparedStatement(s)` exclusively, so not only it protects against SQL injection, but the data access layer can better take advantage of server-side and client-side statement caching as well.

Auto-generated DML statements

The enterprise system database schema evolves with time, and the data access layer must mirror all these modifications as well.

Because the JPA provider auto-generates insert and update statements, the data access layer can easily accommodate database table structure modifications. By updating the entity model schema, Hibernate can automatically adjust the modifying statements accordingly.

This applies to changing database column types as well. If the database schema needs to migrate a postal code from an `INT` database type to a `VARCHAR(6)`, the data access layer will need only to change the associated Domain Model attribute type from an `Integer` to a `String`, and all statements are going to be automatically updated. Hibernate defines a highly customizable JDBC-to-database type mapping system, and the application developer can override a default type association, or even add support for new database types (that are not currently supported by Hibernate).

The entity fetching process is automatically managed by the JPA implementation, which auto-generates the select statements of the associated database tables. This way, JPA can free the application developer from maintaining entity selection queries as well.

Hibernate allows customizing all the CRUD statements, in which case the application developer is responsible for maintaining the associated DML statements.

Although it takes care of the entity selection process, most enterprise systems need to take advantage of the underlying database querying capabilities. For this reason, whenever the database schema changes, all the native SQL queries need to be updated manually (according to their associated business logic requirements).

Write-behind cache

The Persistence Context acts as a transactional write-behind cache, deferring entity state flushing up until the last possible moment.

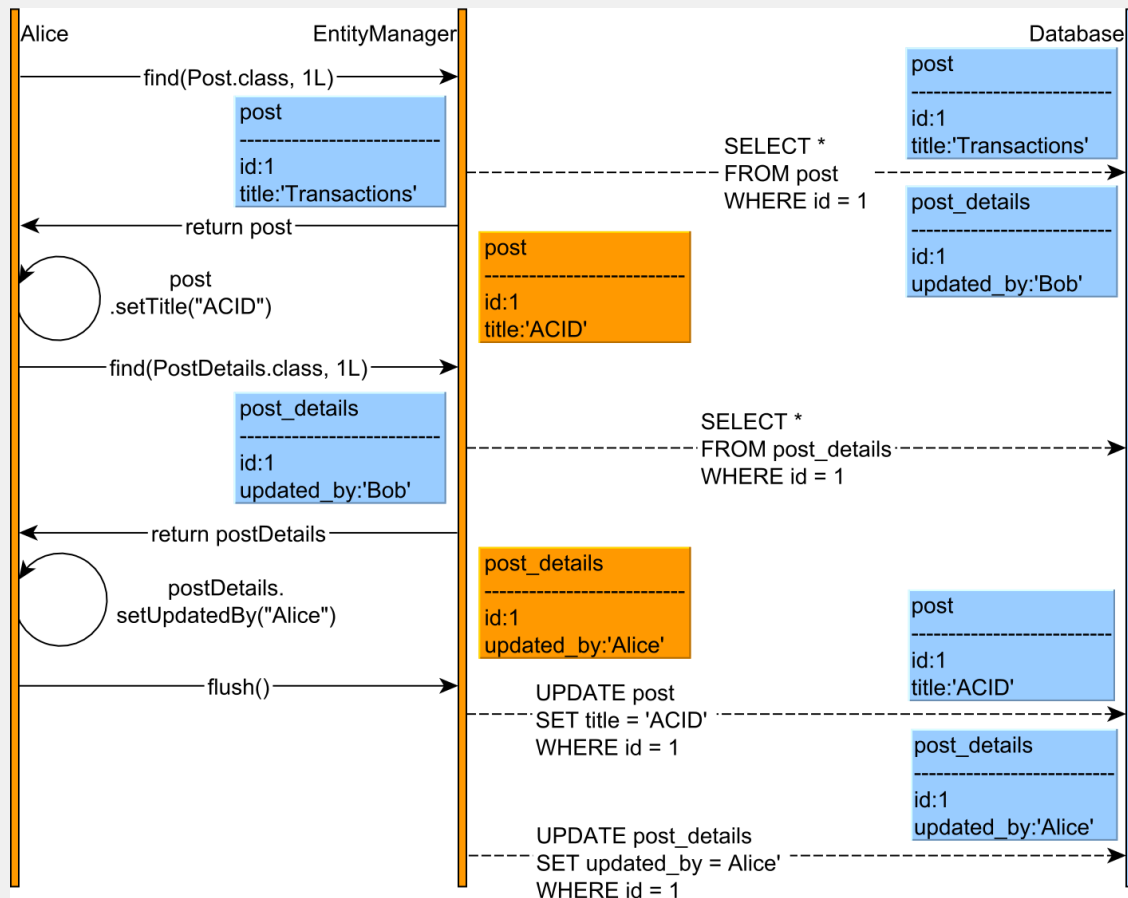


Figure 7.6: Persistence Context

Because every modifying DML statement requires locking (to prevent dirty writes), the write behind cache can reduce the database lock acquisition interval, therefore increasing concurrency.



However, caches introduce consistency challenges, and the Persistence Context requires a flush prior to executing any JPQL or native SQL query, as otherwise it might break the read-your-own-write consistency guarantee.

As detailed in the following chapters, Hibernate does not automatically flush pending changes when a native query is about to be executed, and the application developer must explicitly instruct what database tables are needed to be synchronized.

Transparent statement batching

Since all changes are being flushed at once, Hibernate may benefit from batching JDBC statements. Batch updates can be enabled transparently, even after the data access logic has been implemented. Most often, performance tuning is postponed until the system is already running in production, and switching to batching statements should not require a considerable development effort.



With just one configuration, Hibernate can execute all prepared statements in batches.

Application-level concurrency control

As previously explained, no database isolation level can protect against losing updates when executing a multi-request long conversation. JPA supports both optimistic and pessimistic locking.

The JPA optimistic locking mechanism allows preventing lost updates because it imposes a *happens-before* event ordering. However, in multi-request conversations, optimistic locking requires maintaining old entity snapshots, and JPA makes it possible through *Extended Persistence Contexts* or detached entities.



A Java EE application server can preserve a given Persistence Context across several web requests, therefore providing application-level repeatable reads. However, this strategy is not free since the application developer must make sure the Persistence Context is not bloated with too many entities, which, apart from consuming memory, it can also affect the performance of the Hibernate default dirty checking mechanism.

Even when not using Java EE, the same goal can be achieved using detached entities, which provide fine-grained control over the amount of data needed to be preserved from one web request to the other. JPA allows merging detached entities, which rebecome managed and automatically synchronized with the underlying database system.

JPA also supports a pessimistic locking query abstraction, which comes in handy when using lower-level transaction isolation modes.



Hibernate has a native pessimistic locking API, which brings support for timing out lock acquisition requests or skipping already acquired locks.

7.6 Read-based optimizations

Following the SQL standard, the JDBC `ResultSet` is a tabular representation of the underlying fetched data. The Domain Model being constructed as an entity graph, the data access layer must transform the flat `ResultSet` into a hierarchical structure.

Although the goal of the ORM tool is to reduce the gap between the object-oriented Domain Model and its relational counterpart, it is very important to remember that the source of data is not an in-memory repository, and the fetching behavior influences the overall data access efficiency.



The database cannot be abstracted out of this context, and pretending that entities can be manipulated just like any other plain objects is very detrimental to application performance. When it comes to reading data, the impedance mismatch becomes even more apparent, and, for performance reasons, it is mandatory to keep in mind the SQL statements associated with every fetching operation.

In the following example, the *posts* records are fetched along with all their associated *comments*. Using JDBC, this task can be accomplished using the following code snippet:

```
doInJDBC(connection -> {
    try (PreparedStatement statement = connection.prepareStatement(
        "SELECT * " +
        "FROM post AS p " +
        "JOIN post_comment AS pc ON p.id = pc.post_id " +
        "WHERE " +
        "    p.id BETWEEN ? AND ? + 1"
    )) {
        statement.setInt(1, id);
        statement.setInt(2, id);
        try (ResultSet resultSet = statement.executeQuery()) {
            List<Post> posts = toPosts(resultSet);
            assertEquals(expectedCount, posts.size());
        }
    } catch (SQLException e) {
        throw new DataAccessException(e);
    }
});
```

When joining *many-to-one* or *one-to-one* associations, each `ResultSet` record corresponds to a pair of entities, so both the parent and the child can be resolved in each iteration. For *one-to-many* or *many-to-many* relationships, because of how the SQL join works, the `ResultSet` contains a duplicated parent record for each associated child.

Constructing the hierarchical entity structure requires manual `ResultSet` transformation, and, to resolve duplicates, the parent entity references are stored in a `Map` structure.

```
List<Post> toPosts(ResultSet resultSet) throws SQLException {
    Map<Long, Post> postMap = new LinkedHashMap<>();
    while (resultSet.next()) {
        Long postId = resultSet.getLong(1);
        Post post = postMap.get(postId);
        if (post == null) {
            post = new Post(postId);
            postMap.put(postId, post);
            post.setTitle(resultSet.getString(2));
            post.setVersion(resultSet.getInt(3));
        }
        PostComment comment = new PostComment();
        comment.setId(resultSet.getLong(4));
        comment.setReview(resultSet.getString(5));
        comment.setVersion(resultSet.getInt(6));
        post.addComment(comment);
    }
    return new ArrayList<>(postMap.values());
}
```

The JDBC 4.2 `PreparedStatement` supports only positional parameters, and the first ordinal starts from 1. JPA allows named parameters as well, which are especially useful when a parameter needs to be referenced multiple times, so the previous example can be rewritten as follows:

```
doInJPA(entityManager -> {
    List<Post> posts = entityManager.createQuery(
        "select distinct p " +
        "from Post p " +
        "join fetch p.comments " +
        "where " +
        "    p.id BETWEEN :id AND :id + 1", Post.class)
    .setParameter("id", id)
    .getResultList();
    assertEquals(expectedCount, posts.size());
});
```

In both examples, the object–relation transformation takes place either implicitly or explicitly. In the JDBC use case, the associations must be manually resolved, while JPA does it automatically (based on the entity schema).

The fetching responsibility

Besides mapping database columns to entity attributes, the entity associations can also be represented in terms of object relationships. More, the fetching behavior can be hard-wired to the entity schema itself, which is most often a terrible thing to do.

Fetching multiple one-to-many or many-to-many associations is even more problematic because they might require a Cartesian Product, therefore affecting performance. Controlling the hard-wired schema fetching policy is cumbersome as it prevents overriding an eager retrieval with a lazy loading mechanism.



Each business use case has different data access requirements, and one policy cannot anticipate all possible use cases, so the **fetching strategy should always be set up on a query basis**.

Prefer projections for read-only views

Although it is very convenient to fetch entities along with all their associated relationships, it is better to take into consideration the performance impact as well. As previously explained, fetching too much data is not suitable because it increases the transaction response time.

In reality, not all use cases require loading entities, and not all read operations need to be served by the same fetching mechanism. Sometimes a custom projection (selecting only a few columns from an entity) is much more suitable, and the data access logic can even take advantage of database-specific SQL constructs that might not be supported by the JPA query abstraction.



As a rule of thumb, fetching entities is suitable when the logical transaction requires modifying them, even if that only happens in a successive web request. With this in mind, it is much easier to reason on which fetching mechanism to employ for a given business logic use case.

The second-level cache

If the Persistence Context acts as a transactional write-behind cache, its lifetime will be bound to that of a logical transaction. For this reason, the Persistence Context is also known as the first-level cache, and so it cannot be shared by multiple concurrent transactions.

On the other hand, the second-level cache is associated with an `EntityManagerFactory`, and all Persistence Contexts have access to it. The second-level cache can store entities as well as entity associations (one-to-many and many-to-many relationships) and even entity query results. Because JPA does not make it mandatory, each provider takes a different approach to caching (as opposed to EclipseLink, by default, Hibernate disables the second-level cache).

Most often, caching is a trade-off between consistency and performance. Because the cache becomes another source of truth, inconsistencies might occur, and they can be prevented only when all database modifications happen through a single `EntityManagerFactory` or a synchronized distributed caching solution. In reality, this is not practical since the application might be clustered on multiple nodes (each one with its own `EntityManagerFactory`) and the database might be accessed by multiple applications.



Although the second-level cache can mitigate the entity fetching performance issues, it requires a distributed caching implementation, which might not elude the network-ing penalties anyway.

7.7 Wrap-up

Bridging two highly-specific technologies is always a difficult problem to solve. When the enterprise system is built on top of an object-oriented language, the object-relational impedance mismatch becomes inevitable. The ORM pattern aims to close this gap although it cannot completely abstract it out.

In the end, all the communication flows through JDBC and every execution happens in the database engine itself. A high-performance enterprise application must resonate with the underlying database system, and the ORM tool must not disrupt this relationship.

Just like the problem it tries to solve, Hibernate is a very complex framework with many subtleties that require a thorough knowledge of both database systems, JDBC, and the framework itself. This chapter is only a summary, meant to present JPA and Hibernate into a different perspective that prepares the reader for high-performance object-relational mapping. There is no need to worry if some topics are not entirely clear because the upcoming chapters analyze all these concepts in greater detail.

8. Connection Management and Monitoring

As previously explained in the JDBC Connection Management chapter, for performance reasons, database connections are better off reused. Because JPA providers generate SQL statements on behalf of users, it is very important to monitor this process and acknowledge its outcome. This chapter explains the Hibernate connection provider mechanism and ways to monitor statement execution.

8.1 JPA connection management

Like the whole Java EE suite, the JPA 1.0 specification was very much tied to enterprise application servers. In a Java EE container, all database connections are managed by the application server which provides connection pooling, monitoring, and JTA capabilities.

Once configured, the Java EE `DataSource` can be located through JNDI. In the `persistence.xml` configuration file, the application developer must supply the JNDI name of the associated JTA or `RESOURCE_LOCAL DataSource`. The `transaction-type` attribute must also match the data source transaction capabilities.

```
<persistence-unit name="persistenceUnit" transaction-type="JTA">
  <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
  <jta-data-source>java:global/jdbc/flexypool</jta-data-source>
</persistence-unit>
```

A `RESOURCE_LOCAL` transaction must use a non-jta-data-source `DataSource`.

```
<persistence-unit name="persistenceUnit" transaction-type="RESOURCE_LOCAL">
  <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
  <non-jta-data-source>java:/comp/env/jdbc/hsqldb</non-jta-data-source>
</persistence-unit>
```

While for a Java EE application it is perfectly fine to rely on the application server for providing a full-featured `DataSource` reference, stand-alone applications are usually configured using dependency injection rather than JNDI.

From the JPA implementation perspective, the `DataSource` can be either configured externally or by the JPA provider itself. Most often, configuring an external `DataSource` is still the preferred alternative as it gives more flexibility in decorating the connection providing mechanism (e.g. logging, monitoring).

JPA providers can fetch connections through the underlying JDBC Driver since JPA 2.0 has standardized the database connection configuration properties:

Table 8.1: JPA connection properties

Property	Description
<code>javax.persistence.jdbc.driver</code>	Driver full class name (e.g. <code>org.hsqldb.jdbc.JDBCDriver</code>)
<code>javax.persistence.jdbc.url</code>	Driver Url (e.g. <code>jdbc:hsqldb:mem:test</code>)
<code>javax.persistence.jdbc.user</code>	Database user's name
<code>javax.persistence.jdbc.password</code>	Database user's password

Unfortunately, these properties alone are not sufficient because most enterprise applications need connection pooling and monitoring capabilities anyway. For this reason, JPA connection management is still an implementation-specific topic, and the upcoming sections dive into the connection provider mechanism employed by Hibernate.

8.2 Hibernate connection providers

Hibernate needs to operate both in Java EE and stand-alone environments, and the database connectivity configuration can be done either declaratively or programmatically. To accommodate JDBC Driver connections as well as RESOURCE_LOCAL and JTA DataSource configurations, Hibernate defines its own connection factory abstraction, represented by the `org.hibernate.engine.jdbc.connections.spi.ConnectionProvider` interface:

```
public interface ConnectionProvider extends Service, Wrapped {  
  
    public Connection getConnection() throws SQLException;  
  
    public void closeConnection(Connection connection) throws SQLException;  
  
    public boolean supportsAggressiveRelease();  
}
```

Because the connection provider might influence transaction response time, each provider is analyzed from a high-performance OLTP system perspective.

8.2.1 DriverManagerConnectionProvider

Hibernate picks this provider when being given the aforementioned JPA 2.0 connection properties or the Hibernate-specific configuration counterpart:

- `hibernate.connection.driver_class`
- `hibernate.connection.url`
- `hibernate.connection.username`
- `hibernate.connection.password`.



Although it fetches database connections through the underlying `DriverManager`, this provider tries to avoid the connection acquisition overhead by using a trivial pooling implementation. The Hibernate documentation does not recommend using the `DriverManagerConnectionProvider` in a production setup.

8.2.2 C3P0ConnectionProvider

[C3p0](http://www.mchange.com/projects/c3p0/)¹ is a mature connection pooling solution that has proven itself in many production environments, and, using the underlying JDBC connection properties, Hibernate can replace the built-in connection pool with a `c3p0 DataSource`. To activate this provider, the application developer must supply at least one configuration property starting with the `hibernate.c3p0` prefix:

```
<property name="hibernate.c3p0.max_size" value="5"/>
```

`C3p0` (released in 2001) and [Apache DBCP](https://commons.apache.org/proper/commons-dbcp/)² (released in 2002) are the oldest and the most deployed stand-alone Java connection pooling solutions. Later in 2010, [BoneCP](https://github.com/wwadge/bonecp)³ emerged as a high-performance alternative for `c3p0` and `Apache DBCP`.

Nowadays, the `BoneCP` GitHub page says it is been deprecated in favor of [HikariCP](https://github.com/brettwooldridge/HikariCP)⁴.

¹<http://www.mchange.com/projects/c3p0/>

²<https://commons.apache.org/proper/commons-dbcp/>

³<https://github.com/wwadge/bonecp>

⁴<https://github.com/brettwooldridge/HikariCP>

As of writing, the most attractive Java connection pools are HikariCP, [Vibur DBCP⁵](#), and Apache DBCP2. HikariCP and Vibur DBCP offer built-in Hibernate connection providers.

8.2.3 HikariCPConnectionProvider

Hibernate 5 supports HikariCP (one of the fastest connection pools) via the following dependency:

```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-hikaricp</artifactId>
  <version>${hibernate.version}</version>
</dependency>
```

By specifying the `hibernate.connection.provider_class` property, the application developer can override the default connection provider mechanism:

```
<property name="hibernate.connection.provider_class"
  value="org.hibernate.hikaricp.internal.HikariCPConnectionProvider"/>
```

Unlike `DriverManagerConnectionProvider` or `C3P0ConnectionProvider`, HikariCP requires [specific configuration properties⁶](#):

Table 8.2: HikariCP connection properties

Property	Description
<code>hibernate.hikari.dataSourceClassName</code>	Driver full class name
<code>hibernate.hikari.dataSource.url</code>	Driver Url
<code>hibernate.hikari.dataSource.user</code>	Database user's name
<code>hibernate.hikari.dataSource.password</code>	Database user's password
<code>hibernate.hikari.maximumPoolSize</code>	Maximum pool size

⁵<https://github.com/vibur/vibur-dbcp>

⁶<https://github.com/brettwooldridge/HikariCP>

8.2.4 DataSourceConnectionProvider

This provider is chosen when the JPA configuration file defines a `non-jta-data-source` or a `jta-data-source` element, or when supplying a `hibernate.connection.datasource` configuration property.



Unlike other providers, this one is compatible with JTA transactions, which are mandatory in Java EE.

Spring works with both stand-alone JTA transaction managers (e.g. Bitronix or Atomikos) and Java EE `DataSource(s)`, and, because it offers the best control over the actual `DataSource` configuration, the `DataSourceConnectionProvider` is the preferred choice (even for HikariCP).

8.2.5 Connection release modes

Hibernate defers the database connection acquisition until the current transaction has to execute its first SQL statement (either triggered by a read or a write operation). This optimization allows Hibernate to reduce the physical transaction interval, therefore increasing the chance of getting a connection from the pool.

The connection release strategy is controlled through the `hibernate.connection.release_mode` property which can take the following values:

Table 8.3: Connection release modes

Value	Description
<code>after_transaction</code>	Once acquired, the database connection is released only after the current transaction either commits or rolls back.
<code>after_statement</code>	The connection is released after each statement execution and reacquired prior to running the next statement. Although not required by either JDBC or JTA specifications, this strategy is meant to prevent application servers from mistakenly detecting ⁷ a connection leak between successive EJB (Enterprise Java Beans) calls
<code>auto</code>	This is the default value, and for <code>RESOURCE_LOCAL</code> transactions, it uses the <code>after_transaction</code> mode, while for JTA transactions it falls back to <code>after_statement</code> .

⁷<http://lists.jboss.org/pipermail/hibernate-dev/2006-December/000903.html>

For JTA transactions, the default mode might be too strict since not all Java EE application servers exhibit the same behavior for managing transactional resources. This way, it is important to check if database connections can be closed outside of the EJB component that triggered the connection acquisition event. Spring-based enterprise systems do not use Enterprise Java Beans, and, even when using a stand-alone JTA transaction manager, the `after_transaction` connection release mode might be just fine.

It is somehow intuitive that the `after_statement` mode incurs some performance penalty associated with the frequent acquisition/releasing connection cycles. For this reason, the following test measures the connection acquisition overhead when using Bitronix in a Spring application context. Each transaction executes the same statement (fetching the current timestamp) for a given number of times (represented on the x-axis). The y-axis captures the recorded transaction response times for both `after_statement` and `after_transaction` connection release modes.

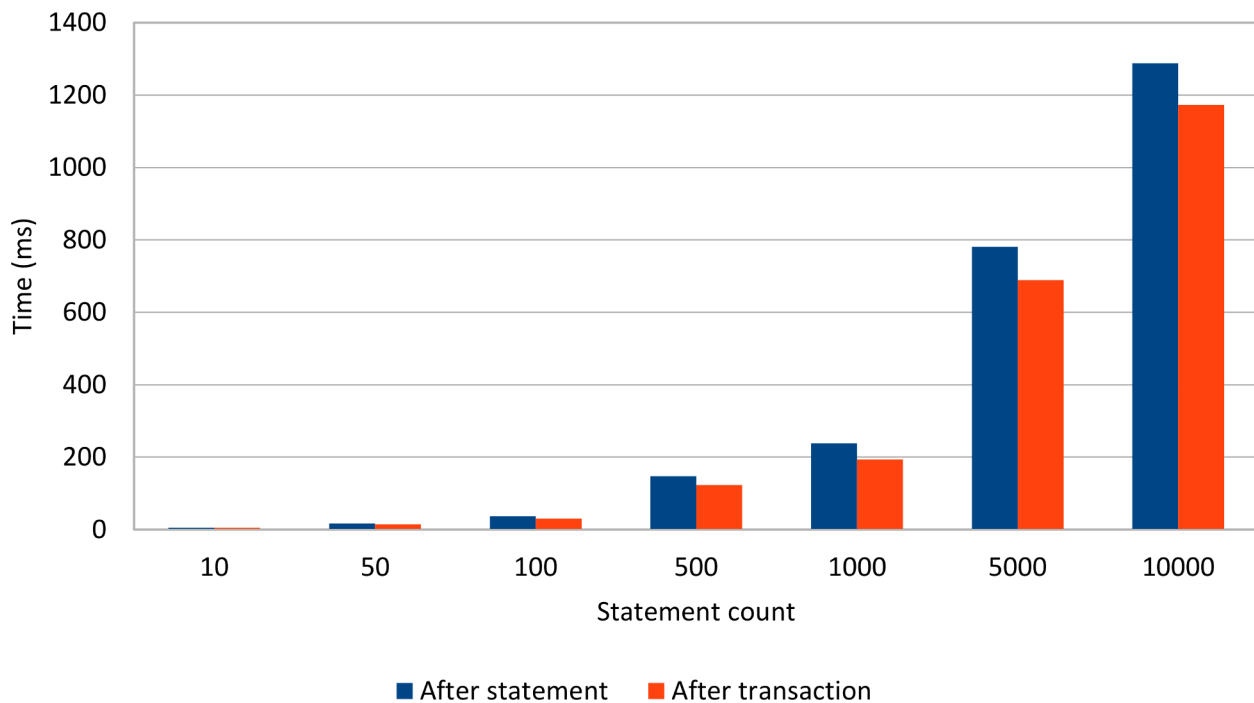


Figure 8.1: Connection release mode

The more statements a transaction executes, the greater the penalty of reacquiring the associated database connection from the underlying connection pool. To better visualize the connection acquisition overhead, the test runs up to 10 000 statements, even if this number is probably too high for the typical OLTP transaction.

Ideally, database transactions should be as short as possible, and the number of statements should not be too high either. This requirement stems from the fact that the number of pooled connections is limited and locks are better released sooner than later.



The `after_transaction` connection release mode is more efficient than the default JTA `after_statement` strategy, and so it should be used if the JTA transaction resource management logic does not interfere with this connection releasing strategy.

8.3 Monitoring connections

As previously concluded, using an externally configured `DataSource` is preferred because the actual `DataSource` can be decorated with connection pooling, monitoring and logging capabilities. Because that is exactly how [FlexyPool⁸](https://github.com/vladmihalcea/flexy-pool) works too, the following diagram captures the `DataSource` proxying mechanism:

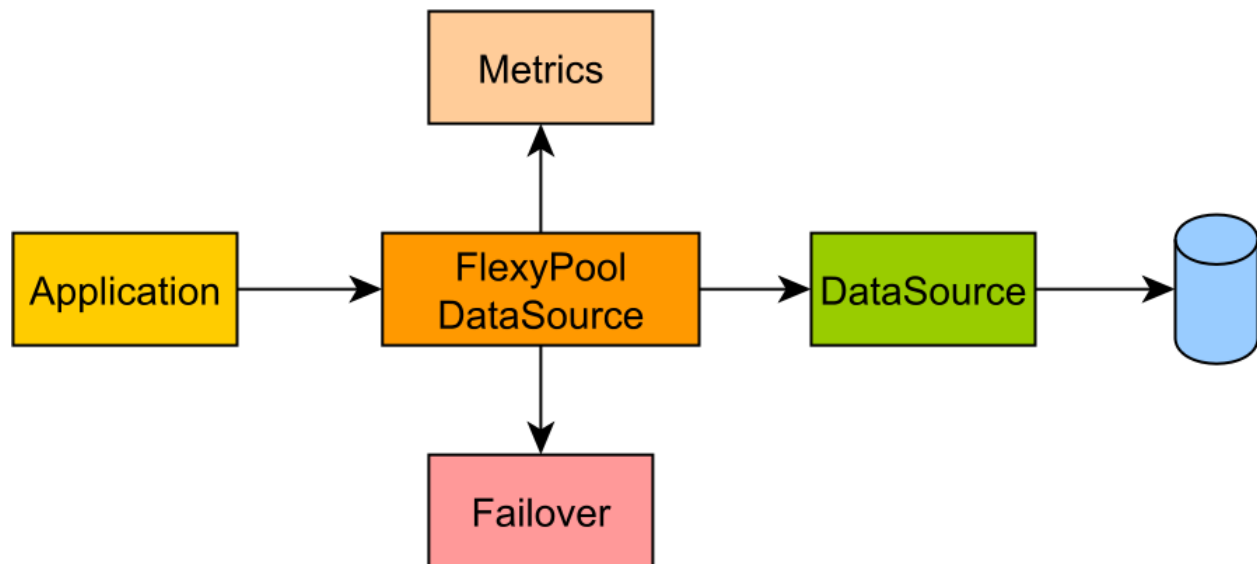


Figure 8.2: FlexyPool architecture

Instead of getting the actual `DataSource` instance, the data access layer gets a proxy reference. The proxy intercepts connection acquisition and releasing requests, and, this way, it can monitor its usage.

When using Spring, setting up FlexyPool is fairly easy because the application has total control over the `DataSource` configuration.

In Java EE, database connections should always be fetched from a managed `DataSource`, and one simple way of integrating FlexyPool is to extend the default `DatasourceConnectionProviderImpl` and substitute the original `DataSource` with the `FlexyPoolDataSource`.

⁸<https://github.com/vladmihalcea/flexy-pool>

For this reason, FlexyPool comes with the following Hibernate connection provider:

```
public class FlexyPoolHibernateConnectionProvider
    extends DataSourceConnectionProviderImpl {

    private transient FlexyPoolDataSource<DataSource> flexyPoolDataSource;

    @Override
    public void configure(Map props) {
        super.configure(props);
        flexyPoolDataSource = new FlexyPoolDataSource<>(getDataSource());
    }

    @Override
    public Connection getConnection() throws SQLException {
        return flexyPoolDataSource.getConnection();
    }

    @Override
    public boolean isUnwrappableAs(Class unwrapType) {
        return super.isUnwrappableAs(unwrapType) ||
            getClass().isAssignableFrom(unwrapType);
    }

    @Override
    public void stop() {
        flexyPoolDataSource.stop();
        super.stop();
    }
}
```

To use the `FlexyPoolHibernateConnectionProvider`, the application must configure the `hibernate.connection.provider_class` property:

```
<property
    name="hibernate.connection.provider_class"
    value="com.vladmihalcea.flexypool.adaptor.FlexyPoolHibernateConnectionProvider"
/>
```

8.3.1 Hibernate statistics

Hibernate has a built-in statistics collector which gathers notifications related to database connections, `Session` transactions and even second-level caching usage. The `StatisticsImplementor` interface defines the contract for intercepting various Hibernate internal events:

StatisticsImplementor	
 <code>openSession()</code>	<code>void</code>
 <code>closeSession()</code>	<code>void</code>
 <code>flush()</code>	<code>void</code>
 <code>connect()</code>	<code>void</code>
 <code>prepareStatement()</code>	<code>void</code>
 <code>closeStatement()</code>	<code>void</code>
 <code>endTransaction(boolean)</code>	<code>void</code>
 <code>loadEntity(String)</code>	<code>void</code>
 <code>fetchEntity(String)</code>	<code>void</code>
 <code>updateEntity(String)</code>	<code>void</code>
 <code>insertEntity(String)</code>	<code>void</code>
 <code>deleteEntity(String)</code>	<code>void</code>
 <code>optimisticFailure(String)</code>	<code>void</code>
 <code>loadCollection(String)</code>	<code>void</code>
 <code>fetchCollection(String)</code>	<code>void</code>
 <code>updateCollection(String)</code>	<code>void</code>
 <code>recreateCollection(String)</code>	<code>void</code>
 <code>removeCollection(String)</code>	<code>void</code>
 <code>secondLevelCachePut(String)</code>	<code>void</code>
 <code>secondLevelCacheHit(String)</code>	<code>void</code>
 <code>secondLevelCacheMiss(String)</code>	<code>void</code>
 <code>naturalIdCachePut(String)</code>	<code>void</code>
 <code>naturalIdCacheHit(String)</code>	<code>void</code>
 <code>naturalIdCacheMiss(String)</code>	<code>void</code>
 <code>naturalIdQueryExecuted(String, long)</code>	<code>void</code>
 <code>queryCachePut(String, String)</code>	<code>void</code>
 <code>queryCacheHit(String, String)</code>	<code>void</code>
 <code>queryCacheMiss(String, String)</code>	<code>void</code>
 <code>queryExecuted(String, int, long)</code>	<code>void</code>
 <code>updateTimestampsCacheHit()</code>	<code>void</code>
 <code>updateTimestampsCacheMiss()</code>	<code>void</code>
 <code>updateTimestampsCachePut()</code>	<code>void</code>

Figure 8.3: Hibernate `StatisticsImplementor` interface

There is a great variety of metrics Hibernate can collect on user's behalf, but, for performance reasons, the statistics mechanism is disabled by default.

To enable the statistics gathering mechanism, the following property must be configured first:

```
<property name="hibernate.generate_statistics" value="true"/>
```

Once statistics are being collected, in order to print them into the current application log, the following logger configuration must be set up:

```
<logger  
  name="org.hibernate.engine.internal.StatisticalLoggingSessionEventListener"  
  level="info" />
```

With these two settings in place, whenever a Hibernate `Session` (Persistence Context) ends, the following report is displayed in the currently running log.

```
37125102 nanoseconds spent acquiring 10000 JDBC connections;  
25521714 nanoseconds spent releasing 10000 JDBC connections;  
95242323 nanoseconds spent preparing 10000 JDBC statements;  
923615040 nanoseconds spent executing 10000 JDBC statements;
```

The default statistics collector just counts the number of times a certain callback method was called, and, if that is not satisfactory, the application developer can supply its own custom `StatisticsImplementor` implementation.

Dropwizard Metrics

In a high-throughput transaction system, the amount of metric data needed to be recorded can be overwhelming, so storing all these values into memory is not practical at all.

To reduce the memory footprint, the [Dropwizard Metrics](https://github.com/dropwizard/metrics)^a project uses various [reservoir sampling](https://en.wikipedia.org/wiki/Reservoir_sampling)^b strategies, which either employ a fixed-size sampler or a time-based sampling window.

Not only it supports a great variety of metrics (e.g. timers, histograms, gauges), but Dropwizard Metrics can use multiple reporting channels as well (e.g. SLF4J, JMX, Ganglia, Graphite).



For all these reasons, it is better to use a mature framework such as Dropwizard Metrics instead of building a custom implementation from scratch.

^a<https://github.com/dropwizard/metrics>

^bhttps://en.wikipedia.org/wiki/Reservoir_sampling

8.3.1.1 Customizing statistics

Although the built-in metrics are rather informative, Hibernate is not limited to the default statistics collector mechanism which can be completely customized.

In the upcoming example, the statistics collector also provides the following metrics:

- the distribution of physical transaction time (the interval between the moment a connection is first acquired and when it gets released)
- a histogram of the number of connections acquisition requests during the lifespan of any given transaction (due to the `after_statement` release mode).

The `StatisticsReport` class provides metric storage and report generation features on top of Dropwizard Metrics:

```
public class StatisticsReport {

    private final Logger LOGGER = LoggerFactory.getLogger(getClass());

    private MetricRegistry metricRegistry = new MetricRegistry();

    private Histogram connectionCountHistogram = metricRegistry.
        histogram("connectionCountHistogram");

    private Timer transactionTimer = metricRegistry.
        timer("transactionTimer");

    private Slf4jReporter logReporter = Slf4jReporter
        .forRegistry(metricRegistry)
        .outputTo(LOGGER)
        .build();

    public void transactionTime(long nanos) {
        transactionTimer.update(nanos, TimeUnit.NANOSECONDS);
    }

    public void connectionsCount(long count) {
        connectionCountHistogram.update(count);
    }

    public void generate() {
        logReporter.report();
    }

}
```

The `StatisticsImplementor` interface defines the contract between the Hibernate internal API and the various custom statistics gathering implementations. For simplicity sake, the following `StatisticsImplementor` interface implementation extends the default `ConcurrentStatisticsImpl` class, as it only needs to override the `connect` and `endTransaction` methods.

```
public class TransactionStatistics extends ConcurrentStatisticsImpl {

    private static final ThreadLocal<AtomicLong> startNanos =
        ThreadLocal.withInitial(AtomicLong::new);

    private static final ThreadLocal<AtomicLong> connectionCounter =
        ThreadLocal.withInitial(AtomicLong::new);

    private StatisticsReport report = new StatisticsReport();

    @Override
    public void connect() {
        connectionCounter.get().incrementAndGet();
        startNanos.get().compareAndSet(0, System.nanoTime());
        super.connect();
    }

    @Override
    public void endTransaction(boolean success) {
        try {
            report.transactionTime(System.nanoTime() - startNanos.get().get());
            report.connectionsCount(connectionCounter.get().get());
            report.generate();
        } finally {
            startNanos.remove();
            connectionCounter.remove();
        }
        super.endTransaction(success);
    }
}
```

The `StatisticsImplementor` is a singleton instance, therefore the `ThreadLocal` counters must be reset after a transaction is ended. When a transaction ends, the report is generated, and both the physical transaction time and the number of connection requests, issued during a particular transaction, are flushed to the log.

Because a Persistence Context can run multiple concurrent transactions, the report is generated at the end of each individual transaction.

To use a custom `StatisticsImplementor` instance, Hibernate requires a `StatisticsFactory` supplied as a configuration property. Taking a `SessionFactoryImplementor` parameter, the `StatisticsImplementor` building process has access to the Hibernate configuration data as well.

```
public class TransactionStatisticsFactory implements StatisticsFactory {
    @Override public StatisticsImplementor buildStatistics(
        SessionFactoryImplementor sessionFactory) {
        return new TransactionStatistics();
    }
}
```

The `hibernate.stats.factory` configuration property must contain the fully qualified name of the `StatisticsFactory` implementation class:

```
<property name="hibernate.stats.factory" value="com.vladmihalcea.book.hpjp.hibernate\
.statistics.TransactionStatisticsFactory" />
```

When running the previous JTA connection release mode example along with this custom statistics collector, the following output is being displayed:

```
type=HISTOGRAM, name=connectionCounterHistogram, count=107,
min=1, max=10000, mean=162.41, stddev=1096.69,
median=1.0, p75=1.0, p95=50.0, p98=1000.0, p99=5000.0, p999=10000.0
```

```
type=TIMER, name=transactionTimer, count=107,
min=0.557524, max=1272.75, mean=27.16, stddev=152.57,
median=0.85, p75=1.24, p95=41.25, p98=283.50, p99=856.19, p999=1272.75,
mean_rate=36.32, rate_unit=events/second, duration_unit=milliseconds
```



For a high-performance data access layer, statistics and metrics becomes mandatory requirements. The Hibernate statistics mechanism is a very powerful tool, allowing the development team to get a better insight into Hibernate inner workings.

8.4 Statement logging

An ORM tool can automatically generate DML statements, and it is the application developer responsibility to validate both their effectiveness as well as their overall performance impact. Deferring the SQL statement validation until the data access layer starts showing performance issues is risky, and it can even impact development cost. For this reason, SQL statement logging becomes relevant from the early stages of application development.



When a business logic is implemented, the Definition of Done should include a review of all the associated data access layer operations. Following this rule can save much hassle when the enterprise system is deployed into production.

Although the JPA 2.1 does not feature a standard configuration property for logging SQL statements, most JPA implementations support this feature through framework-specific setups. For this purpose, Hibernate defines the following configuration properties:

Table 8.4: Connection release modes

Property	Description
<code>hibernate.show_sql</code>	Prints SQL statements to the console
<code>hibernate.format_sql</code>	Formats SQL statements before being logged or printed to the console
<code>hibernate.use_sql_comments</code>	Adds comments to the automatically generated SQL statement

Using the `System` console for logging is bad practice, a logging framework (e.g. Logback or Log4j) being a better alternative for it supports configurable appenders and logging levels.



Because it prints to the console, the `hibernate.show_sql` property should be avoided.

Hibernate logs all SQL statements on a debug level in the `org.hibernate.SQL` logging hierarchy.

To enable statement logging, the following Logback logger must be added to the associated configuration file:

```
<logger name="org.hibernate.SQL" level="debug"/>
```

Because Hibernate uses `PreparedStatement(s)` exclusively, the bind parameter values are not available when the statement gets printed into the log:

```
INSERT INTO post (title, version, id) VALUES (?, ?, ?)
```



Although bind parameters might be logged separately (e.g. `org.hibernate.type.descriptor.sql`), the most straight-forward way of logging SQL statements along with their runtime bind parameter values is to use an external `DataSource` proxy. Because the proxy intercepts all statement executions, the bind parameter values can be introspected and printed as well.

8.4.1 Statement formatting

By default, every SQL statement, no matter how long, is written as a single line of text. To increase readability, Hibernate can transform SQL statements in a human-readable format that spans over multiple log lines. This feature can be activated by setting the following configuration property:

```
<property name="hibernate.format_sql" value="true" />
```

With this setting in place, the previous statement can be formatted as follows:

```
insert
  into
    post
    (title, version, id)
  values
    (?, ?, ?)
```

Although formatting statements can improve readability, this setting is only suitable for the development phase. In a production system, logs are often parsed and aggregated in a centralized system, and the multi-line statement format can impact the log parsing mechanism. Once aggregated, logged queries can be formatted prior to being displayed in the application performance monitoring user interface.



The `hibernate.format_sql` property applies to logged statements only, and it does not propagate to the underlying JDBC `Driver` (SQL statements are still sent as single lines of text).

This way, the statement formatting does not have any effect when statements are logged through an external `DataSource` proxy.

8.4.2 Statement-level comments

Besides formatting, Hibernate can explain the statement generation process by appending SQL-level comments into the statement body. This feature allows the application developer to get a better understanding of the following processes:

- the entity state transition that triggered the currently executing statement
- the reason for choosing a join when fetching a given result set
- the explicit locking mechanism employed by the current statement.

By default, Hibernate does not append any SQL comment in the automatically generated statements, and, to enable this mechanism, the following Hibernate property must be configured:

```
<property name="hibernate.use_sql_comments" value="true" />
```

When persisting a `Post` entity, Hibernate explains the entity state transition associated with this particular statement through the following comment:

```
/* insert com.vladmihalcea.book.hppj.util.providers.BlogEntityProvider$Post */  
INSERT INTO post (title, version, id) VALUES (?, ?, ?)
```



As opposed to SQL statement formatting, SQL comments are generated not only during logging as they propagate to the underlying `Driver` as well.



Although it might be a useful technique for debugging purposes, in a production environment, it is better to leave it disabled, to reduce the database request networking overhead.

8.4.3 Logging parameters

Either the JDBC `Driver` or the `DataSource` must be proxied to intercept statement executions and log them along with the actual parameter values. Besides statement logging, a JDBC proxy can provide other cross-cutting features like long-running query detection or custom statement execution listeners.

8.4.3.1 DataSource-proxy

A lesser-known JDBC logging framework, [datasource-proxy](https://github.com/ttddyy/datasource-proxy)⁹ provides support for custom JDBC statement execution listeners. In Java EE, not all application servers allow configuring an external `DataSource`, as they rely on their own custom implementations that bind the user supplied JDBC `Driver`. Because it can only decorate a `DataSource`, `datasource-proxy` might not be suitable for all Java EE environments.

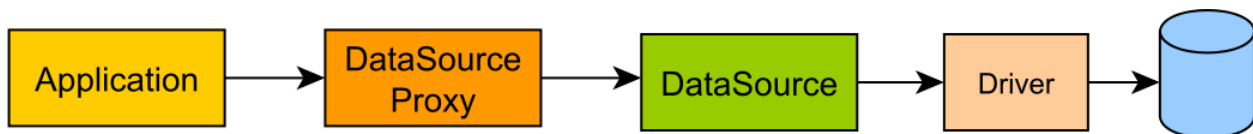


Figure 8.4: DataSource-Proxy Architecture

On the other hand, the programmatic configuration support fits the Java-based configuration approach taken by most modern Spring applications:

```

@Bean
public DataSource dataSource() {
    SLF4JQueryLoggingListener loggingListener = new SLF4JQueryLoggingListener();
    loggingListener.setQueryLogEntryCreator(new InlineQueryLogEntryCreator());
    return ProxyDataSourceBuilder
        .create(actualDataSource())
        .name(DATA_SOURCE_PROXY_NAME)
        .listener(loggingListener)
        .build();
}
  
```

⁹<https://github.com/ttddyy/datasource-proxy>

In the following example, `datasource-proxy` is used to log a batch insert of three `PreparedStatement(s)`. Although normally a batch is printed in a single line of log, the output was split into multiple lines to fit the current page layout.

```
Name:DATA_SOURCE_PROXY, Time:6, Success:True,
Type:Prepared, Batch:True, QuerySize:1, BatchSize:3,
Query:["insert into post (title, version, id) values (?, ?, ?)"],
Params:[(Post no. 0, 0, 0), (Post no. 1, 0, 1), (Post no. 2, 0, 2)]
```

Not only the bind parameter values are now present, but, because they are grouped all together, it is very easy to visualize the batching mechanism too.



With the custom statement listener support, `datasource-proxy` allows building a query count validator to assert the auto-generated statement count and, therefore, prevent N+1 query problems during the development phase.

8.4.3.2 P6Spy

P6Spy¹⁰ was released in 2002, in an era when J2EE application servers were ruling the world of enterprise systems. Because Java EE application servers do not allow programmatic `DataSource` configuration, P6Spy supports a declarative configuration approach (through a `spy.properties` file).

P6Spy offers support for proxying both a JDBC `Driver` (which is suitable for Java EE applications) or a JDBC `DataSource` (supported by some Java EE containers and common practice for Spring enterprise applications).

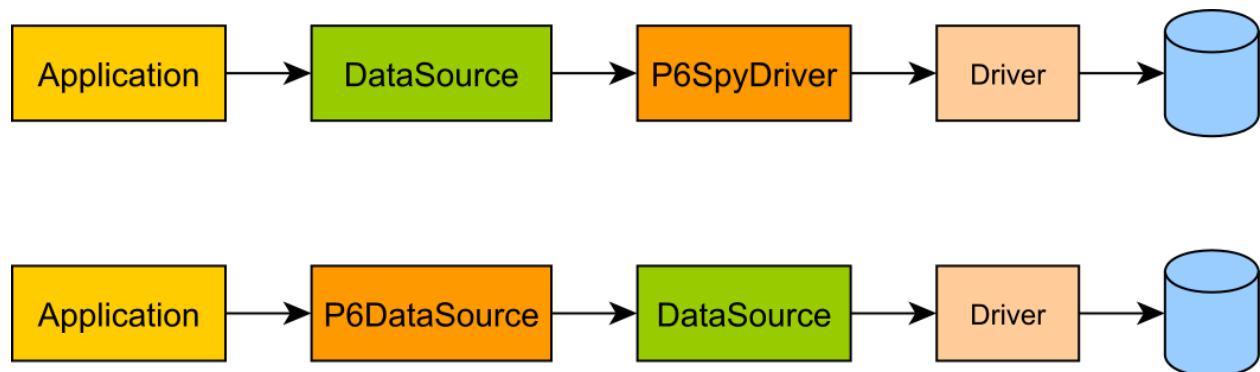


Figure 8.5: P6Spy Architecture

¹⁰<https://github.com/p6spy/p6spy>

Running the previous example gives the following output (formatting was also applied):

```
p6spy - 1448122491807|0|batch|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 0', 0, 0)
p6spy - 1448122491807|0|batch|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 1', 0, 1)
p6spy - 1448122491807|0|batch|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 2', 0, 2)
p6spy - 1448122491812|5|statement|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 2', 0, 2)
```

In the order of their occurrence, the output is built out of the following columns:

Table 8.5: P6Spy output

Field	Description
Timestamp	The statement execution timestamp
Execution time	The statement execution duration (in milliseconds)
Category	The current statement category (e.g. statement, batch)
Connection	The database connection identifier (as assigned by P6Spy)
Original statement	The original statement that was intercepted by P6Spy
Formatted statement	The statement with all parameter placeholders replaced with the actual bind values

The first three lines are associated with adding statements to the batch, while the fourth line is logging the actual batch execution (which also explains the *execution time* column value).



One very useful configuration is the `outagedetection` property, which can detect long-running statements.

9. Mapping Types and Identifiers

JPA addresses the Object/Relational mismatch by associating Java object types to database structures. Assuming there is a `task` database table having four columns (e.g. `id`, `created_by`, `changed_on`, and `status`), the JPA provider must map it to Domain Model consisting of two Java class (e.g. `Task` and `Change`).

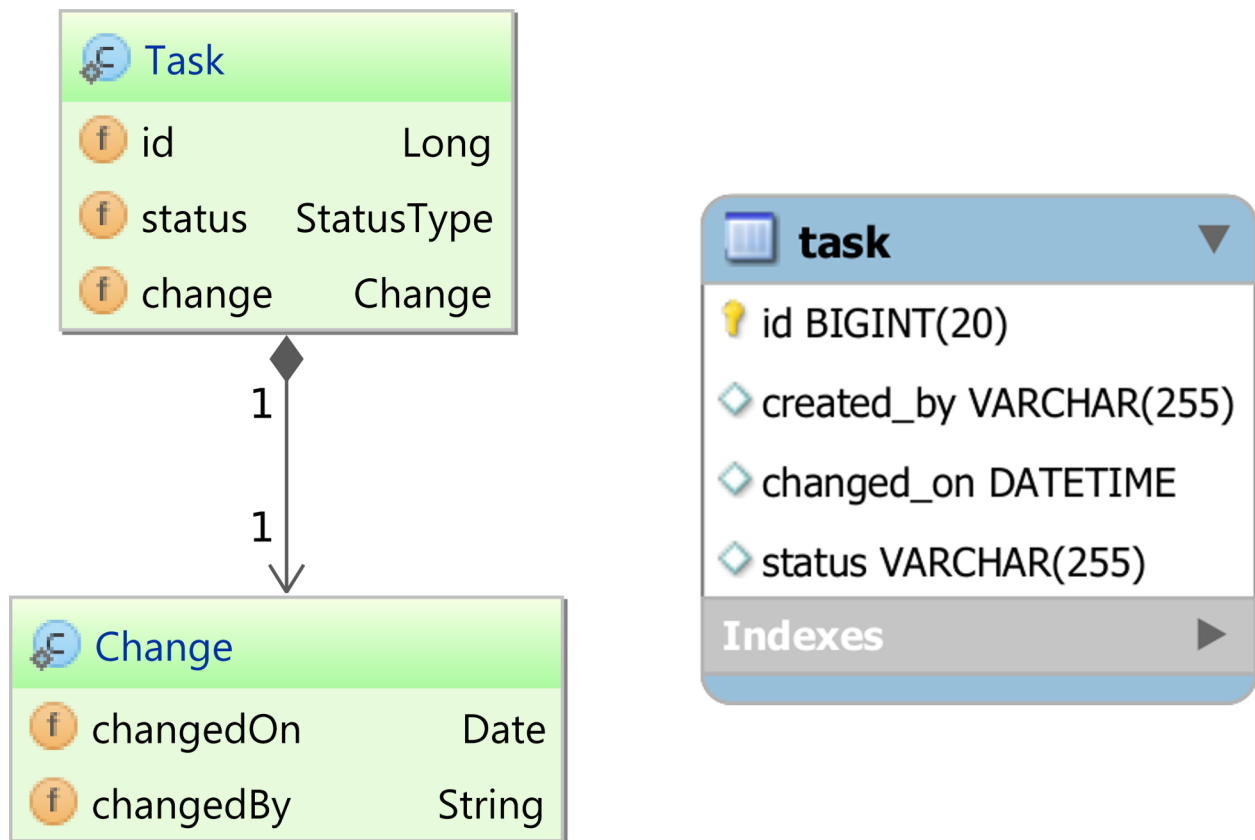


Figure 9.1: Type Mapping

JPA uses three main Object-Relational mapping elements: *type*, *embeddable*, and *entity*. In the previous diagram, the `Task` object is an entity, while the `Change` object is an embeddable type.

Both the entity and the embeddable group multiple Domain Model attributes by relying on Hibernate `Type(s)` to associate database column types with Java value objects (e.g. `String`, `Integer`, `Date`). The major difference between an entity and an embeddable is the presence of an identifier, which is used to associate a database table unique key (usually the primary key) with a Domain Model object attribute.



Although it is common practice to map all database columns, this is not a strict requirement. Sometimes it is more practical to use a root entity and several sub-entities, so each business case fetches just as much info as needed (while still benefiting from entity state management).

Identifiers are mandatory for entity elements, and an embeddable type is forbidden to have an identity of its own. Knowing the database table and the column that uniquely identifies any given row, Hibernate can correlate database rows with Domain Model entities.

An embeddable type groups multiple attributes in a single reusable component.

```
@Embeddable
public class Change {
    @Column(name = "changed_on")
    private Date changedOn;

    @Column(name = "created_by")
    private String changedBy;
}
```



The Domain Model can share state between multiple entities either by using inheritance or composition. Embeddable types can reuse state through composition.

The [composition association](#)¹, defined by UML, is the perfect analogy for the relationship between an entity and an embeddable. When an entity includes an embeddable type, all its attributes become part of the owner entity.

The embeddable object cannot define its own identifier because otherwise, the entity has more than one identities. Lacking an identifier, the embeddable object cannot be managed by a Persistence Context, and its state is controlled by its parent entity.

Because they can have a significant impact on the overall application performance, the rest of the chapter discusses types and identifiers in greater detail.

¹https://en.wikipedia.org/wiki/Object_composition

9.1 Types

For every supported database type, JDBC defines a `java.sql.JDBCType` enumeration. Since it builds on top of JDBC, Hibernate does the mapping between JDBC types and their associated Java counterparts (primitives or Objects).

9.1.1 Primitive types

Table 9.1: Primitive Types

Hibernate type	JDBC type	Java type
<code>BooleanType</code>	<code>BIT</code>	<code>boolean</code> , <code>Boolean</code>
<code>NumericBooleanType</code>	<code>INTEGER</code> (e.g. 0, 1)	<code>boolean</code> , <code>Boolean</code>
<code>TrueFalseType</code>	<code>CHAR</code> (e.g. 'F', 'f', 'T', 't')	<code>boolean</code> , <code>Boolean</code>
<code>YesNoType</code>	<code>CHAR</code> (e.g. 'N', 'n', 'Y', 'y')	<code>boolean</code> , <code>Boolean</code>
<code>ByteType</code>	<code>TINYINT</code>	<code>byte</code> , <code>Byte</code>
<code>ShortType</code>	<code>SMALLINT</code>	<code>short</code> , <code>Short</code>
<code>CharacterType</code>	<code>CHAR</code>	<code>char</code> , <code>Character</code>
<code>CharacterNCharType</code>	<code>NCHAR</code>	<code>char</code> , <code>Character</code>
<code>IntegerType</code>	<code>INTEGER</code>	<code>int</code> , <code>Integer</code>
<code>LongType</code>	<code>BIGINT</code>	<code>long</code> , <code>Long</code>
<code>FloatType</code>	<code>FLOAT</code>	<code>float</code> , <code>Float</code>
<code>DoubleType</code>	<code>DOUBLE</code>	<code>double</code> , <code>Double</code>
<code>CharArrayType</code>	<code>VARCHAR</code>	<code>char[]</code> , <code>Character[]</code>

From one database system to another, the boolean type can be represented either as a `BIT`, `BYTE`, `BOOLEAN` or `CHAR` database type, so defines four `Type(s)` to resolve the boolean primitive type.



Only non-nullable database columns can be mapped to Java primitives (`boolean`, `byte`, `short`, `char`, `int`, `long`, `float`, `double`). For mapping nullable columns, it is better to use the primitive wrappers instead (`Boolean`, `Byte`, `Short`, `Char`, `Integer`, `Long`, `Float`, `Double`).

9.1.2 String types

A Java `String` can consume as much memory as the Java Heap has available. On the other hand, database systems define both limited-size types (`VARCHAR` and `NVARCHAR`) and unlimited ones (`TEXT`, `NTEXT`, `BLOB`, and `NCLOB`).

To accommodate this mapping discrepancy, Hibernate defines the following Type(s):

Table 9.2: String Types

Hibernate type	JDBC type	Java type
StringType	VARCHAR	String
StringNVarcharType	NVARCHAR	String
TextType	LONGVARCHAR	String
NTextType	LONGNVARCHAR	String
MaterializedClobType	CLOB	String
MaterializedNClobType	NCLOB	String

9.1.3 Date and Time types

When it comes to time, there are multiple Java or database representations, which explains the vast number of time-related Hibernate Type(s).

Table 9.3: Date and Time Types

Hibernate type	JDBC type	Java type
DateType	DATE	Date
TimeType	TIME	Time
TimestampType	TIMESTAMP	Timestamp, Date
DbTimestampType	TIMESTAMP	Timestamp, Date
CalendarType	TIMESTAMP	Calendar, GregorianCalendar
CalendarDateType	DATE	Calendar, GregorianCalendar
CalendarTimeType	TIME	Calendar, GregorianCalendar
TimeZoneType	VARCHAR	TimeZone



Handling time is tricky because of various time zones, leap seconds and daylight saving conventions. Storing timestamps in UTC (Coordinated Universal Time) and doing time zone transformations in the data layer is common practice.

9.1.4 Numeric types

Oracle can represent numbers up to 38 digits, therefore only fitting in a `BigInteger` or a `BigDecimal` (`java.lang.Long` and `java.lang.Double` can only store up to 8 bytes).

Table 9.4: Numeric Types

Hibernate type	JDBC type	Java type
<code>BigIntegerType</code>	NUMERIC	<code>BigInteger</code>
<code>BigDecimalType</code>	NUMERIC	<code>BigDecimal</code>

9.1.5 Binary types

For binary types, most database systems offer multiple storage choices (e.g. RAW, VARBINARY, BYTEA, BLOB, CLOB). In Java, the data access layer can use an array of `byte(s)`, a JDBC `Blob` or `Clob`, or even a `Serializable` type, if the Java object was marshaled prior to being saved to the database.

Table 9.5: Binary Types

Hibernate type	JDBC type	Java type
<code>BinaryType</code>	VARBINARY	<code>byte[], Byte[]</code>
<code>BlobType</code>	BLOB	<code>Blob</code>
<code>ClobType</code>	CLOB	<code>Clob</code>
<code>NClobType</code>	NCLOB	<code>Clob</code>
<code>MaterializedBlobType</code>	BLOB	<code>byte[], Byte[]</code>
<code>ImageType</code>	LONGVARBINARY	<code>byte[], Byte[]</code>
<code>SerializableType</code>	VARBINARY	<code>Serializable</code>
<code>SerializableToBlobType</code>	BLOB	<code>Serializable</code>

9.1.6 UUID types

There are various ways of persisting a Java `UUID` (Universally Unique Identifier), and, based on the memory footprint, the most efficient storage types are the database-specific `UUID` column types.

Table 9.6: UUID Types

Hibernate type	JDBC type	Java type
<code>UUIDBinaryType</code>	BINARY	<code>UUID</code>
<code>UUIDCharType</code>	VARCHAR	<code>UUID</code>
<code>PostgresUUIDType</code>	OTHER	<code>UUID</code>



When not natively supported, a `BINARY` type requires fewer bytes than a `VARCHAR`, so the associated index has a smaller memory footprint too.

9.1.7 Other types

Hibernate can also map Java `Enum(s)`, `Class`, `URL`, `Locale` and `Currency` too.

Table 9.7: Other Types

Hibernate type	JDBC type	Java type
EnumType	CHAR, LONGVARCHAR, VARCHAR INTEGER, NUMERIC, SMALLINT, TINYINT, BIGINT, DECIMAL, DOUBLE, FLOAT	Enum
ClassType	VARCHAR	Class
CurrencyType	VARCHAR	Currency
LocaleType	VARCHAR	Locale
UrlType	VARCHAR	URL

9.1.8 Custom types

Not only that it has a very rich set of data types, but PostgreSQL allows adding custom types as well (using the `CREATE DOMAIN`² DDL statement). Choosing the appropriate database type for each Domain Model field can really make a difference in terms of data access performance. Although there is a great variety of built-in `Type(s)`, the application developer is not limited to the off-the-shelf ones only, and new `Type(s)` can be added without too much effort.

In the following example, the business logic requires monitoring access to an enterprise application. For this purpose, the data access layer stores the IP (Internet Protocol) addresses of each logged-in user.

Assuming this internal application uses the IPv4 protocol only, the IP addresses are stored in the Classless Inter-Domain Routing format (e.g. `192.168.123.231/24`). PostgreSQL can store IPv4 addresses either in a `cidr` or `inet` type, or it can use a `VARCHAR(18)` column type.

The `VARCHAR(18)` column requires 18 characters, and, assuming a UTF-8 encoding, each IPv4 address needs at most 18 bytes. The smallest size address (e.g. `0.0.0.0/0`) taking 9 characters, the `VARCHAR(18)` approach requires between 9 and 18 characters for each IPv4 address.

The `inet` type is specially designed for IPv4 and IPv6 network addresses, and it also supports various network address specific operators (e.g. `<`, `>`, `&&`), as well as other address transforming

²<http://www.postgresql.org/docs/9.5/static/sql-createdomain.html>

functions (e.g. `host(inet)`, `netmask(inet)`). As opposed to the `VARCHAR(18)` approach, the `inet` type requires only 7 bytes for each IPv4 address.

For it has a more compact size (the index can better fit into memory) and supporting many specific operators, the `inet` type is a much more attractive choice. Although, by default, Hibernate does not support `inet` types, adding a custom Hibernate `Type` is a straightforward task. The IPv4 address is encapsulated in its own wrapper, which can also define various address manipulation functions too.

```
public class IPv4 implements Serializable {

    private final String address;

    public IPv4(String address) {
        this.address = address;
    }

    public String getAddress() {
        return address;
    }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        return Objects.equals(address, IPv4.class.cast(o).address);
    }

    @Override public int hashCode() {
        return Objects.hash(address);
    }

    public InetAddress toInetAddress() throws UnknownHostException {
        return InetAddress.getByAddress(address);
    }
}
```

When an entity wants to change an `IPv4` field, it must provide a new object instance. An immutable type is much easier to handle since its internal state does not change throughout the currently running Persistence Context.

All custom types must implement the `UserType` interface, and, since the `ImmutableType` takes care of most `UserType` implementation details, the `IPv4Type` can focus on type-specific conversation logic.

```
public class IPv4Type extends ImmutableType<IPv4> {

    public IPv4Type() {
        super(IPv4.class);
    }

    @Override public int[] sqlTypes() { return new int[]{ Types.OTHER}; }

    @Override public IPv4 get(ResultSet rs, String[] names,
        SessionImplementor session, Object owner) throws SQLException {
        String ip = rs.getString(names[0]);
        return (ip != null) ? new IPv4(ip) : null;
    }

    @Override public void set(PreparedStatement st, IPv4 value, int index,
        SessionImplementor session) throws SQLException {
        if (value == null) {
            st.setNull(index, Types.OTHER);
        } else {
            PGobject holder = new PGobject();
            holder.setType("inet");
            holder.setValue(value.getAddress());
            st.setObject(index, holder);
        }
    }
}
```

The `get()` method is used to map the `inet` field to an `IPv4` object instance, while the `set()` is used for transforming the `IPv4` object to the PostgreSQL JDBC driver `inet` equivalent.



`Types.OTHER` is used for mapping database types not supported by JDBC.

```
public abstract class ImmutableType<T> implements UserType {
    private final Class<T> clazz;

    protected ImmutableType(Class<T> clazz) { this.clazz = clazz; }

    @Override public Object nullSafeGet(ResultSet rs, String[] names,
        SessionImplementor session, Object owner) throws SQLException {
        return get(rs, names, session, owner);
    }

    @Override public void nullSafeSet(PreparedStatement st, Object value,
        int index, SessionImplementor session) throws SQLException {
        set(st, clazz.cast(value), index, session);
    }

    @Override public Class<T> returnedClass() { return clazz; }

    @Override public boolean equals(Object x, Object y) {
        return Objects.equals(x, y);
    }

    @Override public int hashCode(Object x) { return x.hashCode(); }

    @Override public Object deepCopy(Object o) { return o; }

    @Override public boolean isMutable() { return false; }

    @Override public Serializable disassemble(Object o) {
        return (Serializable) o;
    }

    @Override public Object assemble(Serializable o, Object owner) { return o; }

    @Override
    public Object replace(Object o, Object target, Object owner) { return o; }

    protected abstract T get(ResultSet rs, String[] names,
        SessionImplementor session, Object owner) throws SQLException;

    protected abstract void set(PreparedStatement st, T value, int index,
        SessionImplementor session) throws SQLException;
}
```

The `@Type` annotation instructs Hibernate to use the `IPv4Type` for mapping the `IPv4` field.

```
@Entity
public class Event {

    @Id @GeneratedValue
    private Long id;

    @Type(type = "com.vladmihalcea.book.hpjp.hibernate.type.IPv4Type")
    @Column(name = "ip", columnDefinition = "inet")
    private IPv4 ip;

    public Event() {}

    public Event(String address) {
        this.ip = new IPv4(address);
    }

    public Long getId() {
        return id;
    }

    public IPv4 getIp() {
        return ip;
    }

    public void setIp(String address) {
        this.ip = new IPv4(address);
    }
}
```

GiST operators

PostgreSQL 9.4 [added GiST operator support^a](http://www.postgresql.org/docs/current/static/gist-builtin-opclasses.html) for `inet` and `cidr` column types. To enable this feature, a GiST index with the `inet_ops` operator class must be created on the associated `inet` columns.

```
CREATE INDEX ON event USING gist (ip inet_ops)
```

^a<http://www.postgresql.org/docs/current/static/gist-builtin-opclasses.html>

Managing Event(s) is easy when Hibernate takes care of the underlying type conversation.

```
final AtomicReference<Event> eventHolder = new AtomicReference<>();

doInJPA(entityManager -> {
    entityManager.persist(new Event());
    Event event = new Event("192.168.0.231");
    entityManager.persist(event);
    eventHolder.set(event);
});

doInJPA(entityManager -> {
    Event event = entityManager.find(Event.class, eventHolder.get().getId());
    event.setIp("192.168.0.123");
});
```

Running the previous example generates the following SQL statements:

```
INSERT INTO event (ip, id) VALUES (NULL(OTHER), 1)
INSERT INTO event (ip, id) VALUES (`192.168.0.231`, 2)

SELECT e0_.id as id1_0_0_, e0_.ip as ip2_0_0_
FROM event e0_
WHERE e0_.id = 2

UPDATE event SET ip=`192.168.0.123` WHERE id = 2
```

One of the best aspects of using database-specific types is getting access to advanced querying capabilities. Because the GiST index allows `inet_ops` operators, the following query can be used to check if an Event was generated for a given subnetwork:

```
Event matchingEvent = (Event) entityManager.createNativeQuery(
    "SELECT e.* " +
    "FROM event e " +
    "WHERE " +
    "    e.ip && CAST(:network AS inet) = TRUE", Event.class)
    .setParameter("network", "192.168.0.1/24")
    .getSingleResult();

assertEquals("192.168.0.123", matchingEvent.getIp().getAddress());
```

9.2 Identifiers

All database tables must have a primary key column, so each row can be uniquely identified (the primary key must be both `UNIQUE` and `NOT NULL`).

Although the SQL standard does not impose primary keys to be immutable, it is [more practical^a](#) to avoid changing them.

^ahttps://asktom.oracle.com/pls/asktom/f?p=100:11:0::::P11_QUESTION_ID:5773459616034

The primary key can have a meaning in the real world, in which case it is a *natural* key, or it can be generated synthetically, in which case it is called a *surrogate* identifier.

For natural keys, unicity is enforced by a real-world unique sequence generator (e.g. National Identification Numbers, Social Security Numbers, Vehicle Identification Numbers). In reality, natural unique numbers might pose problems when the unique constraints do not hold true anymore. For example, a National Identification Number might yield unique numbers, but if the enterprise system must accommodate users coming from multiple countries, it is possible that two different countries assigned the same identifier.

The natural key can be composed of one or multiple columns. Compound natural keys might incur an additional performance penalty because multi-column joins are slower than single-column ones, and multi-column indexes have a bigger memory footprint too.

Natural keys must be sufficiently long to accommodate as many identifiers as the system needs throughout its lifecycle. Because primary keys are often indexed, the longer the key, the more memory an index entry requires. Each joined table includes a foreign key mirroring the parent primary key, and foreign keys are frequently indexed as well.

Index memory impact

Fixed-size non-numerical keys (e.g. `CHAR`, `VARCHAR`) are less efficient than numerical ones (e.g. `INTEGER`, `BIGINT`) both for joining (a simple key performs better than a compound one) or indexing (the more compact the data type, the less memory space is required by an associated index).

A `CHAR(17)` natural key (e.g. Vehicle Identification Number) requires 17 characters (17 bytes when using ASCII characters and a UTF-8 encoding) as opposed to 4 bytes (32 bit `INTEGER`) or 8 bytes (64 bit `BIGINT`).

Surrogate keys are generated independently of the current row data, so table column constraints may evolve with time (changing a user birthday or email address). The surrogate key can be generated by a numerical sequence generator (e.g. a database identity column or a sequence), or it can be constructed by a pseudorandom number generator (e.g. [GUID](#)³ or [UUID](#)⁴). Both the numerical and UUID keys have both pros and cons.

The UUID is defined by the [RFC 4122](#)⁵ standard and it is stored as a 128-bit sequence. The GUID term refers to any globally unique identifier, which might comprise other non-standard implementations. For consistency, this chapter further refers to unique identifiers as UUID.

A UUID takes 128 bits, which is four times more than an `INTEGER` and twice as `BIGINT`. On the other hand, a UUID number has less chance of a conflict in a Multi-Master database replication topology. To avoid such conflicts, many relational database systems increment the identity or sequence numbers in steps, each node getting its own offset. Because UUIDs are not sequential, they induce fragmentation, and that can really affect the performance of clustered indexes.



Requiring less space and being more index-friendly, numerical sequences are preferred over UUID keys.

9.2.1 UUID identifiers

Nevertheless, some enterprise systems use UUID primary keys, so it is worth knowing what Hibernate types work best for this task. The UUID key can either be generated by the application using the `java.util.UUID` class or it can be assigned by the database system.



If the database system does not have a built-in UUID type, a `BINARY(16)` column type is preferred. Although a `CHAR(32)` column could also store the UUID textual representation, the additional space overhead makes it a less favorable pick.

³http://en.wikipedia.org/wiki/Globally_Unique_Identifier

⁴http://en.wikipedia.org/wiki/Universally_Unique_Identifier

⁵<https://www.ietf.org/rfc/rfc4122.txt>

Oracle

There is no UUID type in Oracle, so a `RAW(16)` column must be used instead. The `SYS_GUID()`^a database function can generate a globally unique identifier.

^a<http://docs.oracle.com/database/121/SQLRF/functions202.htm#SQLRF06120>

SQL Server

The `uniqueidentifier`^a data type is used for storing GUID identifiers. The `NEWID()`^b function can generate a UUID compatible with the RFC 4122 standard.

Because by default SQL Server uses clustered indexes for primary keys, to avoid the fragmentation effect, the `NEWSEQUENTIALID()`^c function can assign pseudo-sequential UUID numbers (greater than previously generated ones). This guarantee is kept as long as the Windows server is not restarted.

^a<https://msdn.microsoft.com/en-us/library/ms187942.aspx>

^b<https://msdn.microsoft.com/en-us/library/ms190348.aspx>

^c<https://msdn.microsoft.com/en-us/library/ms189786.aspx>

PostgreSQL

The `UUID type`^a can store RFC 4122 compliant unique identifiers. The database does not offer a built-in UUID generation function, so the identifier must be generated by the data access layer.

^a<http://www.postgresql.org/docs/current5/static/datatype-uuid.html>

MySQL

The UUID must be stored in a `BINARY(16)` column type. The `UUID()`^a functions can generate a 128-bit unique identifier. Because the `UUID()` function might cause problems for statement-based replication, passing the generated identifier as a variable^b is a workaround to this limitation.

^ahttp://dev.mysql.com/doc/refman/5.7/en/miscellaneous-functions.html#function_uuid

^b<http://dev.mysql.com/doc/refman/5.7/en/replication-features-functions.html>

For generating UUID identifiers, Hibernate offers three generators (*assigned*, *uuid*, *uuid2*), which we'll be discussed in greater detail in the following sections.

9.2.1.1 The assigned generator

By simply omitting the `@GeneratedValue`⁶ annotation, Hibernate falls back to the assigned identifier, which allows the data access layer to control the identifier generation process. The following example maps a `java.util.UUID` identifier to a `BINARY(16)` column type:

```
@Entity @Table(name = "post")
public class Post {

    @Id @Column(columnDefinition = "BINARY(16)")
    private UUID id;

    public Post() {}

    public Post(UUID id) {
        this.id = id;
    }
}
```

When persisting a `post`, Hibernate generates the following insert statement:

```
INSERT INTO post (id) VALUES
([86, 10, -104, 26, 60, -115, 79, 78, -118, -45, 64, 94, -64, -40, 66, 100])
```

The `UUIDBinaryType` translates the `java.util.UUID` to an array of `byte(s)` that is stored in the associated `BINARY(16)` column type.



Because the identifier is generated in the data access layer, the database server is freed from this responsibility, and so it can allocate its resources to other data processing tasks.

Hibernate can also generate a UUID identifier on behalf of the application developer, as described in the following two sections.

⁶<https://docs.oracle.com/javaee/7/api/javax/persistence/GeneratedValue.html>

9.2.2 The legacy UUID generator

The [UUID hex generator](#)⁷ is registered under the `uuid` name and generates hexadecimal UUID string representations. Using a 8-8-4-8-4 hex digit layout, the UUID hex generator is not compliant with the RFC 4122 standard, which uses a 8-4-4-4-12 hex digit format. The following code snippet depicts the `UUIDHexGenerator` mapping and the associated insert statement.

```
@Entity @Table(name = "post")
public class Post {

    @Id @Column(columnDefinition = "CHAR(32)")
    @GeneratedValue(generator = "uuid")
    @GenericGenerator(name = "uuid", strategy = "uuid")
    private String id;
}

INSERT INTO post (id) VALUES (402880e451724a820151724a83d00000)
```

9.2.2.1 The newer UUID generator

The newer [UUID generator](#)⁸ is RFC 4122 compliant (variant 2) and is registered under the `uuid2` name (working with `java.lang.UUID`, `byte[]` and `String` Domain Model object types). Compared to the previous use case, the mapping and the test case look as follows:

```
@Entity @Table(name = "post")
public class Post {

    @Id @Column(columnDefinition = "BINARY(16)")
    @GeneratedValue(generator = "uuid2")
    @GenericGenerator(name = "uuid2", strategy = "uuid2")
    private UUID id;
}

INSERT INTO post (id) VALUES
([77, 2, 31, 83, -45, -98, 70, 40, -65, 40, -50, 30, -47, 16, 30, 124])
```

⁷<https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/id/UUIDHexGenerator.html>

⁸<https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/id/UUIDGenerator.html>



Being RFC 4122 compliant and able to operate with `BINARY` column type, the `UUIDGenerator` is preferred over the legacy `UUIDHexGenerator`.

9.2.3 Numerical identifiers

As previously explained, a numerical surrogate key is usually preferred since it takes less space and indexes work better with sequential identifiers. To generate numerical identifiers, most database systems offer either identity (or `auto_increment`) columns or sequence objects.

JPA defines the `GenerationType`⁹ enumeration for all supported identifier generator types:

- `IDENTITY` is for mapping the entity identifier to a database identity column.
- `SEQUENCE` allocates identifiers by calling a given database sequence.
- `TABLE` is for relational databases that do not support sequences (e.g. MySQL 5.7), the *table* generator emulating a database sequence by using a separate table.
- `AUTO` decides the identifier generation strategy based on the current database dialect.



As explained in the JDBC part, database sequences work better with batch updates and allow various application-side optimization techniques as well.

9.2.3.1 Identity generator

The identity column type (included in the `SQL:2003`¹⁰ standard) is supported by `Oracle 12c`¹¹, `SQL Server`¹² and `MySQL (AUTO_INCREMENT)`¹³, and it allows an `INTEGER` or a `BIGINT` column to be auto-incremented on demand.

The incrementation process is very efficient since it uses a lightweight locking mechanism, as opposed to the more heavyweight transactional course-grain locks. The only drawback is that the newly assigned value can only be known after executing the actual insert statement.

⁹<https://docs.oracle.com/javaee/7/api/javax/persistence/GenerationType.html>

¹⁰<http://en.wikipedia.org/wiki/SQL:2003>

¹¹http://docs.oracle.com/database/121/SQLRF/statements_7002.htm#SQLRF55657

¹²<http://msdn.microsoft.com/en-us/library/ms186775.aspx>

¹³<http://dev.mysql.com/doc/refman/5.7/en/example-auto-increment.html>

Batch updates

Because Hibernate separates the id generation from the actual entity insert statement, entities using the identity generator may not participate in JDBC batch updates. Hibernate issues the insert statement during the `persist()` method call, therefore breaking the transactional write-behind caching semantic used for entity state transitions.

Even if some JDBC drivers allow fetching the associated generated keys when executing a batch update, Hibernate still needs an improvement in this regard.

The identity generator can be mapped as follows:

```
@Entity @Table(name = "post")
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
}
```

The following example demonstrates how the transaction write-behind caching model is circumvented by the identity column semantics. Although disabled by default, JDBC batching was enabled to compare results between identity and sequence generators.

```
doInJPA(entityManager -> {
    for (int i = 0; i < batchSize; i++) {
        entityManager.persist(new Post());
    }
    LOGGER.debug("Flush is triggered at commit-time");
});
```

Executing the previous test case generates the following output.

```
INSERT INTO post (id) VALUES (DEFAULT)
INSERT INTO post (id) VALUES (DEFAULT)

DEBUG - Flush is triggered at commit-time
```

Because the associated entity identifier can only be known after the insert statement is executed, Hibernate triggers the entity state transition prior to flushing the currently running Persistence Context.

9.2.3.2 Sequence generator

A sequence is a database object that generates consecutive numbers. Defined by the SQL:2003 standard, database sequences are supported by Oracle, SQL Server 2012 and PostgreSQL, and, compared to identity columns, sequences offer the following advantages:

- The same sequence can be used to populate multiple columns, even across tables.
- Values may be preallocated to improve performance.
- Allowing incremental steps, sequences can benefit from application-level optimization techniques.
- Because the sequence call can be decoupled from the actual insert statement, Hibernate does not disable JDBC batch updates.

To demonstrate the difference between the identity and the sequence identifier generators, the previous example is changed to use a database sequence this time.

```
@Entity @Table(name = "post")
public class Post {

    @Id
    @GeneratedValue(strategy=GenerationType.SEQUENCE)
    private Long id;
}
```

Running the previous test case generates the following output:

```
CALL NEXT VALUE FOR hibernate_sequence
CALL NEXT VALUE FOR hibernate_sequence

DEBUG - Flush is triggered at commit-time

INSERT INTO post (id) VALUES (1, 2)
```

When executing the `persist()` method, Hibernate calls the associated database sequence and fetches an identifier for the newly persisted entity. The actual insert statement is postponed until flush time, which allows Hibernate to take advantage of JDBC batching.

9.2.3.3 Table generator

Because of the mismatch between the identifier generator and the transactional write-behind cache, JPA offers an alternative sequence-like generator that works even when sequences are not natively supported.

A database table is used to hold the latest sequence value, and row-level locking is employed to prevent two concurrent connections from acquiring the same identifier value.

Escaping transactional row-level locking

A database sequence is a non-transactional object because the sequence value allocation happens outside of the transactional context associated with the database connection requesting a new identifier. Database sequences use dedicated locks to prevent concurrent transactions from acquiring the same value, but locks are released as soon as the counter is incremented. This design ensures minimal contention even when the sequence is used concomitantly by multiple concurrent transactions.

Using a database table as a sequence is challenging, as, to prevent two transactions from getting the same sequence value, row-level locking must be used. However, unlike the sequence object locks, the row-level lock is transactional, and, once acquired, it can only be released when the current transaction ends (either committing or rolling back). This would be a terrible scalability issue because a long-running transaction would prevent any other transaction from acquiring a new sequence value.

To cope with this limitation, a separate database transaction is used for fetching a new sequence value. This way, the row-level lock associated with incrementing the sequence counter value can be released as soon as the sequence update transaction ends.

For local transactions, a new transaction means fetching another database connection and committing it after executing the sequence processing logic. This can put additional pressure on the underlying connection pool, especially if there is already a significant contention for database connections.

In a JTA environment, the currently running transaction must be suspended, and the sequence value is fetched in a separate transaction. The JTA transaction manager has to do additional work to accommodate the transaction context switch, and that can also have an impact on the overall application performance.



Without any application-level optimization, the row-level locking approach can become a performance bottleneck if the sequence logic is called way too often.

To continue the previous example, the `post` uses the table generator this time:

```
@Entity @Table(name = "post")
public class Post {

    @Id
    @GeneratedValue(strategy=GenerationType.TABLE)
    private Long id;
}
```

The following output is obtained when inserting a new `post`:

```
SELECT tbl.next_val
FROM hibernate_sequences tbl
WHERE tbl.sequence_name=default
FOR UPDATE

INSERT INTO hibernate_sequences (sequence_name, next_val)
VALUES (default, 1)

UPDATE hibernate_sequences SET next_val=2
WHERE next_val=1 AND sequence_name=default

SELECT tbl.next_val
FROM hibernate_sequences tbl
WHERE tbl.sequence_name=default
FOR UPDATE

UPDATE hibernate_sequences SET next_val=3
WHERE next_val=2 AND sequence_name=default

DEBUG - Flush is triggered at commit-time

INSERT INTO post (id) values (1, 2)
```

The table generator benefits from JDBC batching, but every table sequence update incurs three steps:

- The lock statement is executed to ensure that the same sequence value is not allocated for two concurrent transactions.
- The current value is incremented in the data access layer.
- The new value is saved back to the database and the secondary transaction is committed to release the row-level lock.

Unlike identity columns and sequences, which can increment the sequence in a single request, the table generator entails a significant performance overhead. For this reason, Hibernate comes with a series of optimizers which can improve performance for both sequence and table generators.



Although it is a portable identifier generation strategy, the table generator introduces a serializable execution (the row-level lock), which can hinder scalability. Compared to this application-level sequence generation technique, identity columns and sequences are highly optimized for high concurrency scenarios and should be the preferred choice anyway.

9.2.3.4 Optimizers

As previously mentioned, both the sequence and the table identifier generator have multiple implementations which can improve the performance of the identifier generation process. The sequence and table generators can be split into two categories:

- Legacy implementations (being deprecated since Hibernate 5.0) like `SequenceGenerator`, `SequenceHiLoGenerator` and `MultipleHiLoPerTableGenerator`.
- Newer and more efficient implementations such as `SequenceStyleGenerator` and `TableGenerator`.

These two categories are not compatible, and the application developer must either choose the legacy identifiers or the enhanced ones. Prior to Hibernate 5.0, the legacy identifier generators were provided by default and the application developer could switch to the newer ones by setting the following configuration property:

```
<property name="hibernate.id.new_generator_mappings" value="true"/>
```

Hibernate 5 has decided to drop support for the legacy identifiers and to use the enhanced ones by default.

Among the legacy identifier generators, the `SequenceGenerator` did not offer any optimization, as every new identifier value would require a call to the underlying database sequence. On the other hand, the `SequenceHiLoGenerator` and the `MultipleHiLoPerTableGenerator` offered a *hi/lo* optimization mechanism aimed to reduce the number of calls to a database server. Although these generators are deprecated, the legacy *hi/lo* algorithm is still a valid optimizer even for the newer identifier generators.

9.2.3.4.1 The hi/lo algorithm

The hi/lo algorithm splits the sequences domain into *hi* groups. A *hi* value is assigned synchronously, and every *hi* group is given a maximum number of *lo* entries, that can be assigned offline without worrying about identifier value conflicts.

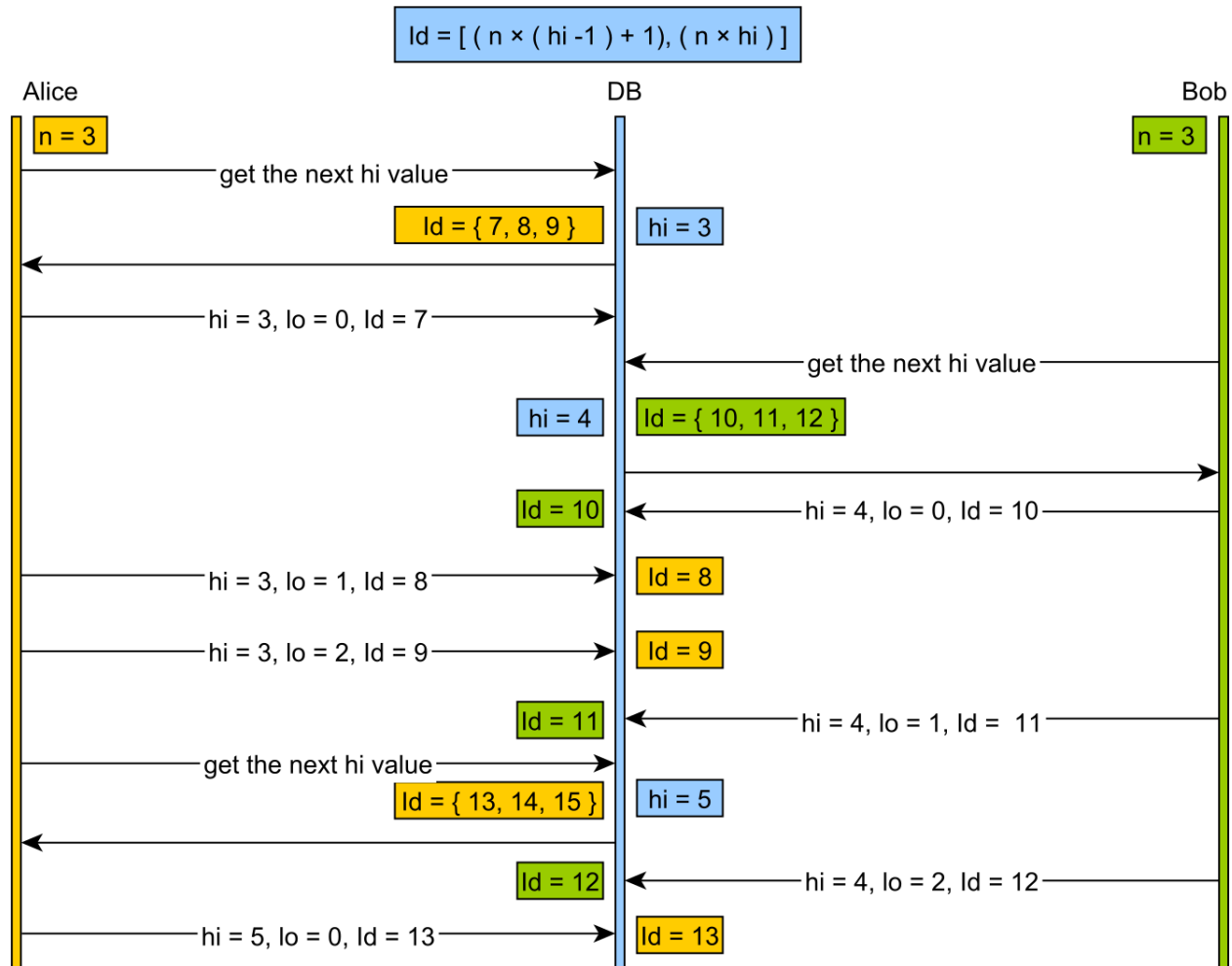


Figure 9.2: The hi/lo algorithm

1. The *hi* token is assigned either by the database sequence or the table generator, so two consecutive calls are guaranteed to see monotonically increasing values.
2. Once a *hi* token is retrieved, the *increment size* (n - the number of **lo** entries) defines the range of identifier values a transaction can safely allocate. The identifiers range is bounded by the following interval: $Id = [n \times (hi - 1) + 1, n \times hi]$ and the allocation is done as follows:
 - the current group values start from $n \times (hi - 1) + 1$
 - the *lo* value is taken from the following interval: $\{ \$\$ \} [0, n - 1] \{ \$\$ \}$
 - by adding the *lo* value to the initial group value, a unique identifier is obtained.
3. When all *lo* values are used, a new *hi* value is fetched, and the cycle continues.

The following example shows how the hi/lo algorithm works in practice. The entity is mapped as follows:

```
@Entity
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "hilo")
    @GenericGenerator(
        name = "hilo",
        strategy = "org.hibernate.id.enhanced.SequenceStyleGenerator",
        parameters = {
            @Parameter(name = "sequence_name", value = "sequence"),
            @Parameter(name = "initial_value", value = "1"),
            @Parameter(name = "increment_size", value = "3"),
            @Parameter(name = "optimizer", value = "hilo")
        }
    )
    private Long id;
}
```

Because the increment size is 3, the following test inserts 4 entities to show the number of database sequence calls.

```
doInJPA(entityManager -> {
    for(int i = 0; i < 4; i++) {
        Post post = new Post();
        entityManager.persist(post);
    }
});
```

Running the previous test generates the following output:

```
CALL NEXT VALUE FOR hilo_sequence
CALL NEXT VALUE FOR hilo_sequence

INSERT INTO post (id) VALUES (1)
INSERT INTO post (id) VALUES (2)
INSERT INTO post (id) VALUES (3)
INSERT INTO post (id) VALUES (4)
```

The first sequence call is for the first three values, while the second one is generated when reaching the fourth entity that needs to be persisted. The more inserts a transaction requires, the better the performance gain from reducing the number of database sequence calls.



Unfortunately, this optimizer has a major limitation. Because the database sequence only assigns group values, all database clients must be aware of this algorithm. If the DBA must insert a row in the table above, he must use the hi/lo algorithm to determine the range of values that she can safely use.

For this reason, Hibernate offers other optimizer algorithms that are interoperable with external clients, unaware of the application-level optimization technique in use.

9.2.3.4.2 The default sequence identifier generator

The JPA identifier generator strategy only specifies the identifier type and not the algorithm used for generating such identifiers.

For the sequence generator, considering the following JPA mapping:

```
@Id
@GeneratedValue(generator = "sequence", strategy=GenerationType.SEQUENCE)
@SequenceGenerator(name = "sequence", allocationSize = 3)
private Long id;
```

Hibernate chooses the `SequenceHiLoGenerator` when the `hibernate.id.new_generator_mappings` configuration property is false. This was the default setting for Hibernate 3 and 4. The legacy `SequenceHiLoGenerator` uses the hi/lo algorithm, and, if the allocation size is greater than one, database interoperability could be compromised (every insert must be done according to the hi/lo algorithm rules).

If the aforementioned configuration property is true (the default setting for Hibernate 5), then the JPA mapping above will use the `SequenceStyleGenerator` instead.

Unlike its previous predecessor, the `SequenceStyleGenerator` uses configurable identifier optimizer strategies, and the application developer can even supply its own optimization implementation.

9.2.3.4.3 The default table identifier generator

Just like with sequences, the JPA table generator mapping can use a legacy or an enhanced generator, depending on the current Hibernate configuration settings:

```
@Id
@GeneratedValue(generator = "table", strategy=GenerationType.TABLE)
@TableGenerator(name = "table", allocationSize = 3)
private Long id;
```

If the `hibernate.id.new_generator_mappings` configuration property is false, then Hibernate will choose the `MultipleHiLoPerTableGenerator`. This generator requires a single table for managing multiple identifiers, and just like `SequenceHiLoGenerator`, it also uses the hi/lo algorithm by default.

When the enhanced identifier generators are activated, Hibernate uses the `TableGenerator` instead, which can also take configurable optimizer strategies.

For both the enhanced sequence and the table identifier generator, Hibernate comes with the following built-in optimizers:

Table 9.8: Hibernate identifier optimizers

Optimizer type	Implementation class	Description
none	NoopOptimizer	Every identifier is fetched using a new roundtrip to the database
hi/lo	HiLoOptimizer	It allocates identifiers by using the legacy hi/lo algorithm
pooled	PooledOptimizer	It is an enhanced version of the hi/lo algorithm which is interoperable with other systems unaware of this identifier generator
pooled-lo	PooledLoOptimizer	It is a variation of the pooled optimizer, the database sequence value representing the lo value instead of the hi one

By default, the `SequenceStyleGenerator` and `TableGenerator` identifier generators uses the `pooled` optimizer. If the `hibernate.id.optimizer.pooled.prefer_lo` configuration property is set to true, Hibernate will use the `pooled-lo` optimizer by default.

Both the `pooled` and the `pooled-lo` encode the database sequence value into the identifier range boundaries, so allocating a new value using the actual database sequence call does not interfere with the identifier generator allocation process.

9.2.3.4.4 The pooled optimizer

The pooled optimizer can be configured as follows:

```
@Entity
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "pooled")
    @GenericGenerator(
        name = "pooled",
        strategy = "org.hibernate.id.enhanced.SequenceStyleGenerator",
        parameters = {
            @Parameter(name = "sequence_name", value = "sequence"),
            @Parameter(name = "initial_value", value = "1"),
            @Parameter(name = "increment_size", value = "3"),
            @Parameter(name = "optimizer", value = "pooled")
        }
    )
    private Long id;
}
```

The increment size gives the range of values allocated by the sequence generator with one database roundtrip. Although it is inefficient to flush the Persistence Context after every persist method call, in this test, the flush outlines when the database sequence was called.

```
doInJPA(entityManager -> {
    for (int i = 0; i < 5; i++) {
        entityManager.persist(new Post());
        entityManager.flush();
    }
    entityManager.unwrap(Session.class).doWork(connection -> {
        try (Statement statement = connection.createStatement()) {
            statement.executeUpdate(
                "INSERT INTO post VALUES NEXT VALUE FOR sequence"
            );
        }
    });
    for (int i = 0; i < 3; i++) {
        entityManager.persist(new Post());
        entityManager.flush();
    }
});
```

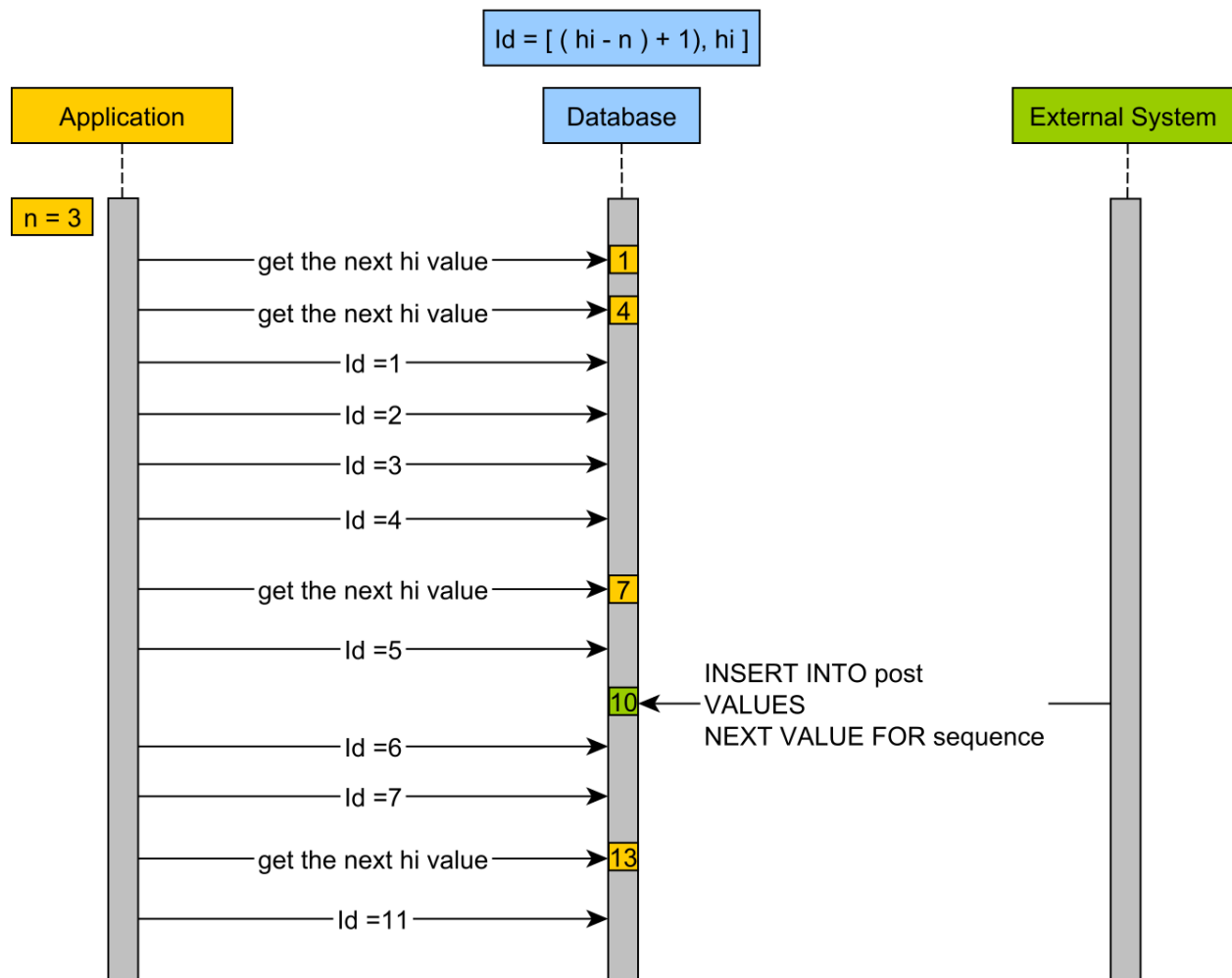


Figure 9.3: The pooled optimizer

If increment size (n) is the number of identifiers within a range, the pooled optimizer will generate identifiers with the following formula: $Id = [(hi - n) + 1, hi]$.

- The first sequence call generates the *lo* value and the second one determines the *hi* value, so the first range of identifiers is {2, 3, 4}.
- When adding the 5th entity, the pooled optimizer calls the sequence again and obtains the next *hi* value, the next identifier range being {5, 6, 7}.
- After inserting the 5th entity, an external system adds a *post* row and assigns the primary key with the value returned by the sequence call.
- The Hibernate application thread resumes and inserts the identifiers 6 and 7.
- The 8th entity requires a new sequence call, and so a new range is allocated {11, 12, 13}.

9.2.3.4.5 The pooled-lo optimizer

By changing the previous mapping to use the `pooled-lo` optimizer, the identifier generation changes as follows:

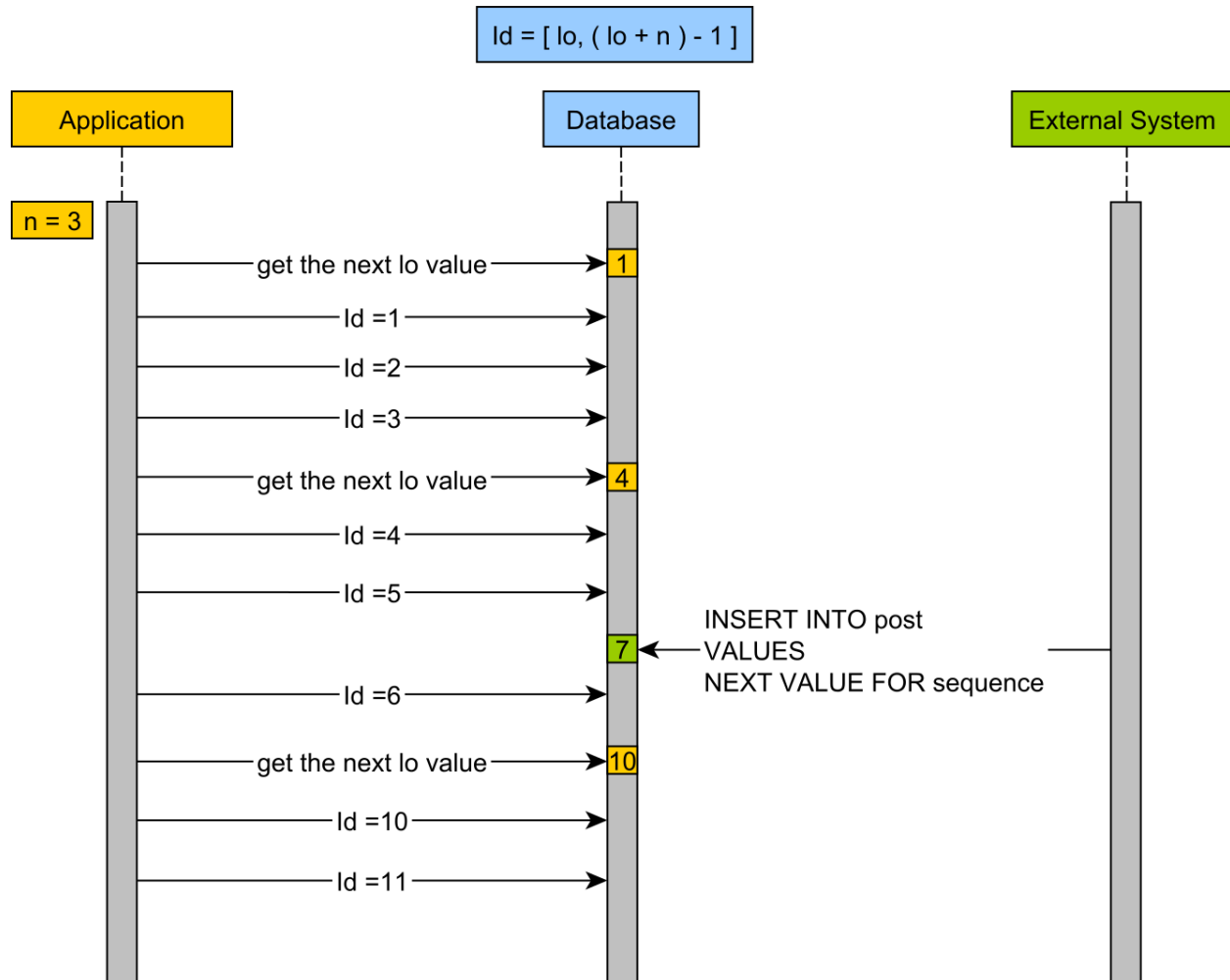


Figure 9.4: The pooled-lo optimizer

If increment size (n) is the number of identifiers within a range, the pooled-lo optimizer will generate identifiers with the following formula: $Id = [lo, (lo + n) - 1]$.

- The first sequence call generates the `lo` value, so the first range of identifiers is {1, 2, 3}.
- When adding the 4th entity, the pooled-lo optimizer calls the sequence and obtains the next `lo` value, the next identifier range being {4, 5, 6}.
- After inserting the 5th entity, an external system adds a `post` row and assigns the primary key with the value returned by the sequence call.
- The Hibernate application thread resumes and inserts the identifier 6.
- The 7th entity requires a new sequence call, and so a new range is allocated {10, 11, 12}.

9.2.3.5 Optimizer gain

To visualize the performance gain of using sequence and table generator optimizers, the following test measures the identifier allocation time when inserting 50 *post* entities and while varying the increment size (1, 5, 10, and 50).

9.2.3.5.1 Sequence generator performance gain

When using a sequence generator with the default pooled optimizer, the following 99th percentile is being recorded:

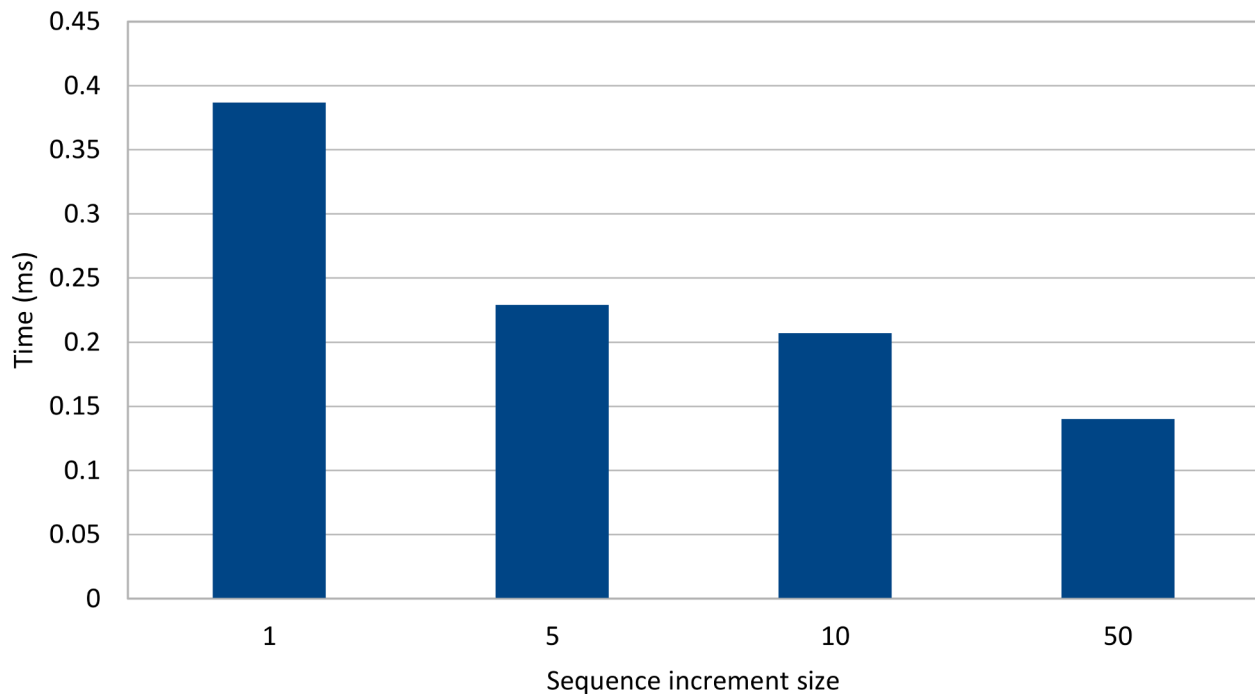


Figure 9.5: Sequence pooled optimizer gain



Database sequences are fast, but, even so, the pooled optimizer manages to reduce the identifier generation time considerably.

For write-intensive applications, the increment size needs to be adjusted according to the number of rows being inserted in one transaction.

9.2.3.5.2 Table generator performance gain

The same test suite is run against a table generator with a pooled optimizer, and the increment size also varies between 1, 5, 10, and 50. Because of the row-level locking and the extra database connection switch overhead, the table generator is less efficient than a database sequence.

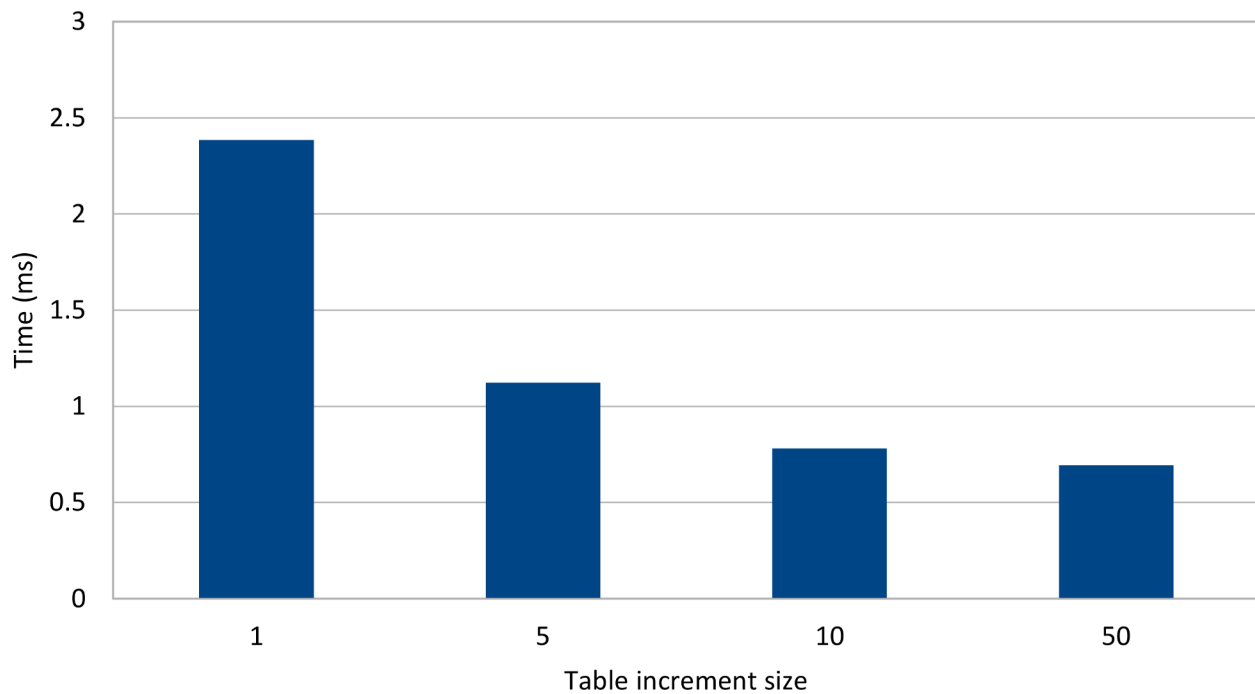


Figure 9.6: Table pooled optimizer gain

Just like with the database sequence, the pooled optimizer managed to reduce the time it took for assigning a new entity identifier.

9.2.3.6 Identifier generator performance

To evaluate the concurrency cost of each identifier generators, the following test measures the time it takes to insert 100 post entities when multiple running threads are involved. JDBC batching is enabled, and the connection pool is adjusted to accommodate the maximum number of database connection required (e.g. 32).



In reality, the application might not be configured with so many database connections, and the table generator connection acquisition cost might be even higher.

The first relational database system under test supports identity columns, so it is worth measuring how the identifier and the table generator compete. Unlike the previous test, this one measures the total time taken for inserting all entities, and not just the identifier allocation time interval.

Each test iteration increases contention by allocating more worker threads that need to execute the same database insert load.

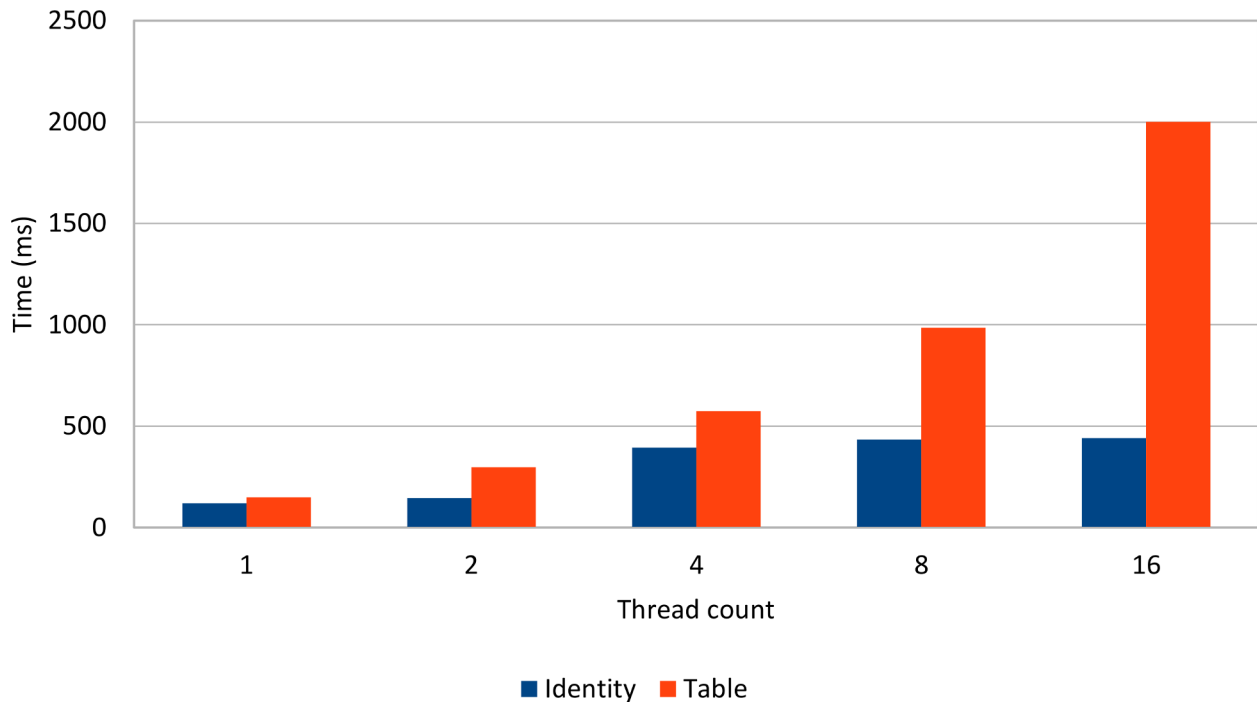


Figure 9.7: Identity vs. Table

Even if it cannot benefit from JDBC batching, the identity generator still manages to outperform the table generator, which uses a pooled optimizer with an increment size of 100.



The more threads are used, the less efficient the table generator becomes. On the other hand, identity columns scale much better with more concurrent transactions.

Even if it does not support JDBC batching, native identity columns are still a valid choice, and, in future, Hibernate might even support batch inserts for those as well.

The gap between the sequence and the table generator is even higher because, just like the table generator, the sequence generator can also take advantage of the pooled optimizer as well as JDBC batch inserts.

Running the same test against a relational database supporting sequences, the following results are being recorded:

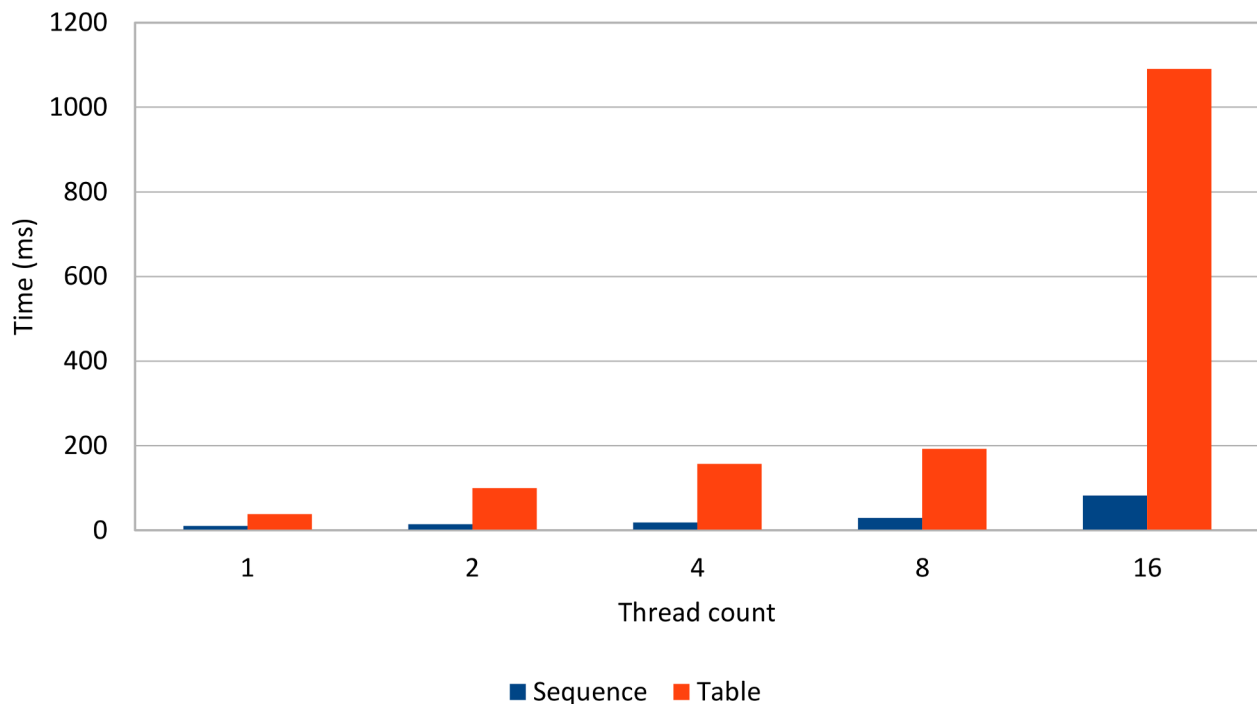


Figure 9.8: Sequence vs. Table

The performance impact of the table generator becomes noticeable in high concurrent environments, where the row-level locking and the database connection switch introduces a serial execution.



Because they use lightweight synchronization mechanisms, database sequences scale better than row-level locking concurrency control mechanisms.

Database sequences are the most efficient Hibernate identifier choice, allowing sequence call optimizers and without compromising JDBC batching.

10. Relationships

In a relational database, associations are formed by correlating rows belonging to different tables. A relationship is established when a child table defines a foreign key referencing the primary key of its parent table. Every database association is built on top of foreign keys, resulting three table relationship types:

- *one-to-many* is the most common relationship, and it associates a row from a parent table to multiple rows in a child table.
- *one-to-one* requires the child table Primary Key to be associated via a Foreign Key with the parent table Primary Key column.
- *many-to-many* requires a link table containing two Foreign Key columns that reference the two different parent tables.

The following diagram depicts all these three table relationships:

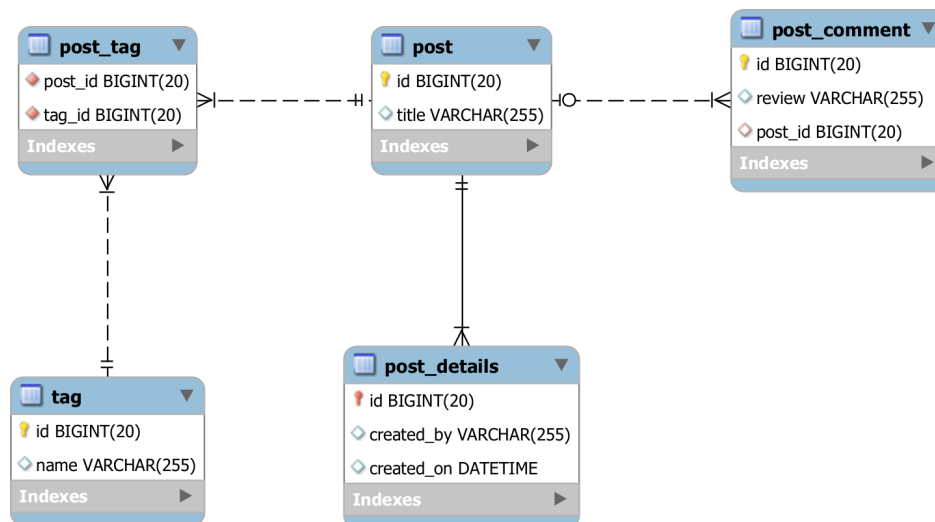


Figure 10.1: Table relationships

The `post` table has a one-to-many relationship with the `post_comment` table because a `post` row might be referenced by multiple comments. The one-to-many relationship is established through the `post_id` column which has a foreign key referencing the `post` table primary key. Because a `post_comment` cannot exist without a `post`, the `post` is the parent-side while the `post_comment` is the child-side.

The `post` table has a one-to-one relationship with the `post_details`. Like the one-to-many association, the one-to-one relationship involves two tables and a foreign key. The foreign key has a uniqueness constraint, so only one child row can reference a parent record.

The `post` and the `tag` are both independent tables and neither one is a child of the other. A `post` can feature several `tag(s)`, while a `tag` can also be associated with multiple `post(s)`. This is a typical many-to-many association, and it requires a junction table to resolve the child-side of these two parent entities. The junction table requires two foreign keys referencing the two parent tables.



The foreign key is, therefore, the most important construct in building a table relationship, and, in a relational database, the child-side controls a table relationship.

In a relational database, the foreign key is associated with the child-side only. For this reason, the parent-side has no knowledge of any associated child relationships, and, from a mapping perspective, table relationships are always unidirectional (the child foreign key references the parent primary key).

10.1 Relationship types

When mapping a JPA entity, besides the underlying table columns, the application developer can map entity relationships either in one direction or in a bidirectional way. This is another impedance mismatch between the object-oriented Domain Model and relational database system because, when using an ORM tool, the parent and the child-side can reference each other.

A relationship is unidirectional if only one entity side maps the table relationship and is bidirectional if the table relationship can be navigated in both directions (either from the entity parent-side or the child-side).

To properly represent both sides of an entity relationship, JPA defines four association mapping constructs:

- `@ManyToOne` represents the child-side (where the foreign key resides) in a database one-to-many table relationship.
- `@OneToMany` is associated with the parent-side of a one-to-many table relationship.
- `@ElementCollection` defines a one-to-many association between an entity and multiple value types (basic or embeddable).
- `@OneToOne` is used for both the child-side and the parent-side in a one-to-one table relationship.
- `@ManyToMany` mirrors a many-to-many table relationship.

Because the entity relationship choice has a considerable impact on the overall application performance, this chapter analyzes the data access operation efficiency of all these JPA associations.

Mapping collections

In a relational database, all table relationships are constructed using foreign keys and navigated through SQL queries. JPA allows mapping both the foreign key side (the child entity has a reference to its parent), as well as the parent side (the parent entity has one or more child entities).

Although `@OneToMany`, `@ManyToMany` OR `@ElementCollection` are convenient from a data access perspective (entity state transitions can be cascaded from parent entities to children), they are definitely not free of cost. The price for reducing data access operations is paid in terms of result set fetching flexibility and performance. A JPA collection, either of entities or value types (basic or embeddables), binds a parent entity to a query that usually fetches all the associated child records. Because of this, the entity mapping becomes sensitive to the number of child entries.

If the children count is relatively small, the performance impact of always retrieving all child entities might be unnoticeable. However, if the number of child records grows too large, fetching the entire children collection may become a performance bottleneck. Unfortunately, the entity mapping is done during the early phases of a project development, and the development team might be unaware of the number of child records a production system exhibits.

Not just the mere size can be problematic, but also the number of attributes of the child entity. Because entities are usually fetched as a whole, the result set is, therefore, proportional to the number of columns the child table contains. Even if a collection is fetched lazily, Hibernate might still require to fully load each entity when the collection is accessed for the first time. Although Hibernate supports extra lazy collection fetching, this is only a workaround and does not address the root problem.

Alternatively, every collection mapping can be replaced by a data access query, which can use a SQL projection that is tailored to the data requirements of each business use case. This way, the query can take business case specific filtering criteria. Although JPA 2.1 does not support dynamic collection filtering, Hibernate offers Persistence Context-bound collection `Filters`.



When handling large data sets, it is good practice to limit the result set size, both for UI (to increase responsiveness) or batch processing tasks (to avoid long running transactions). Just because JPA supports collection mapping, it does not mean they are mandatory for every domain model mapping. Until there is a clear understanding of the number of child records (or if there is even need to fetch child entities entirely), it is better to postpone the collection mapping decision. For high-performance systems, a data access query is often a much more flexible alternative.

10.2 @ManyToOne

The `@ManyToOne` relationship is the most common JPA association, and it maps exactly to the one-to-many table relationship. When using a `@ManyToOne` association, the underlying foreign key is controlled by the child-side, no matter the association is unidirectional or bidirectional.

This section focuses on unidirectional `@ManyToOne` relationships only, the bidirectional case being further discussed with the `@OneToMany` relationship. In the following example, the `Post` entity represents the parent-side, while the `PostComment` is the child-side.

As already mentioned, the JPA entity relationship diagram matches exactly the one-to-many table relationship.

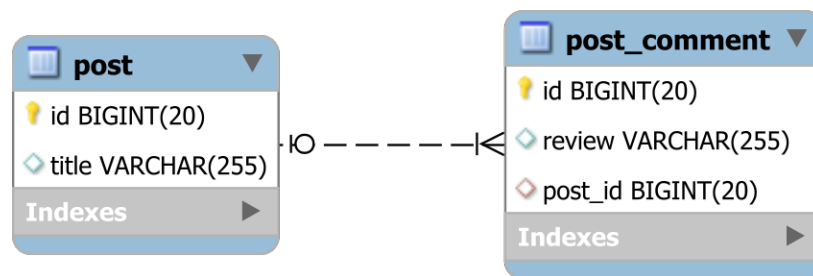


Figure 10.2: The one-to-many table relationship

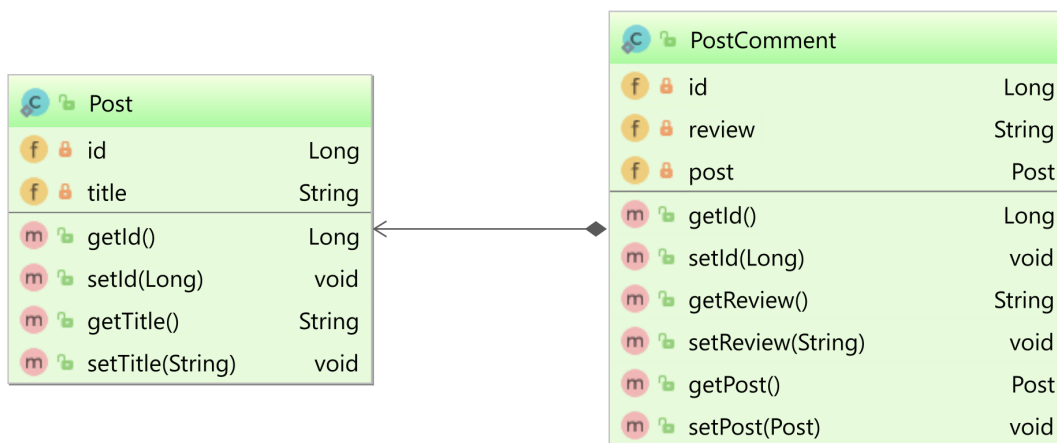


Figure 10.3: `@ManyToOne` relationship

Instead of mapping the `post_id` foreign key column, the `PostComment` uses a `@ManyToOne` relationship to the parent `Post` entity. The `PostComment` can be associated with an existing `Post` object reference, and the `PostComment` can also be fetched along with the `Post` entity.

```
@ManyToOne
@JoinColumn(name = "post_id")
private Post post;
```

Hibernate translates the internal state of the `@ManyToOne` `Post` object reference to the `post_id` foreign key column value.

If the `@ManyToOne` attribute is set to a valid `Post` entity reference:

```
Post post = entityManager.find(Post.class, 1L);
PostComment comment = new PostComment("My review");
comment.setPost(post);
entityManager.persist(comment);
```

Hibernate will generate an insert statement populating the `post_id` column with the identifier of the associated `Post` entity.

```
INSERT INTO post_comment (post_id, review, id) VALUES (1, 'My review', 2)
```

If the `Post` attribute is later set to null:

```
comment.setPost(null);
```

The `post_id` column will also be updated with a `NULL` value:

```
UPDATE post_comment SET post_id = NULL, review = 'My review' WHERE id = 2
```



Because the `@ManyToOne` association controls the foreign key directly, the automatically generated DML statements are very efficient.

Actually, the best-performing JPA associations always rely on the child-side to translate the JPA state to the foreign key column value.

This is one of the most important rules in JPA relationship mapping, and it will be further emphasized for `@OneToOne`, `@OneToMany` and even `@ManyToMany` associations.

10.3 @OneToOne

While the `@ManyToOne` association is the most natural mapping of the one-to-many table relationship, the `@OneToOne` association can also mirror this database relationship, but only when being used as a bidirectional mapping. A unidirectional `@OneToOne` association uses an additional junction table, which no longer fits the one-to-many table relationship semantics.

10.3.1 Bidirectional @OneToMany

The bidirectional `@OneToMany` association has a matching `@ManyToOne` child-side mapping that controls the underlying one-to-many table relationship. The parent-side is mapped as a collection of child entities.

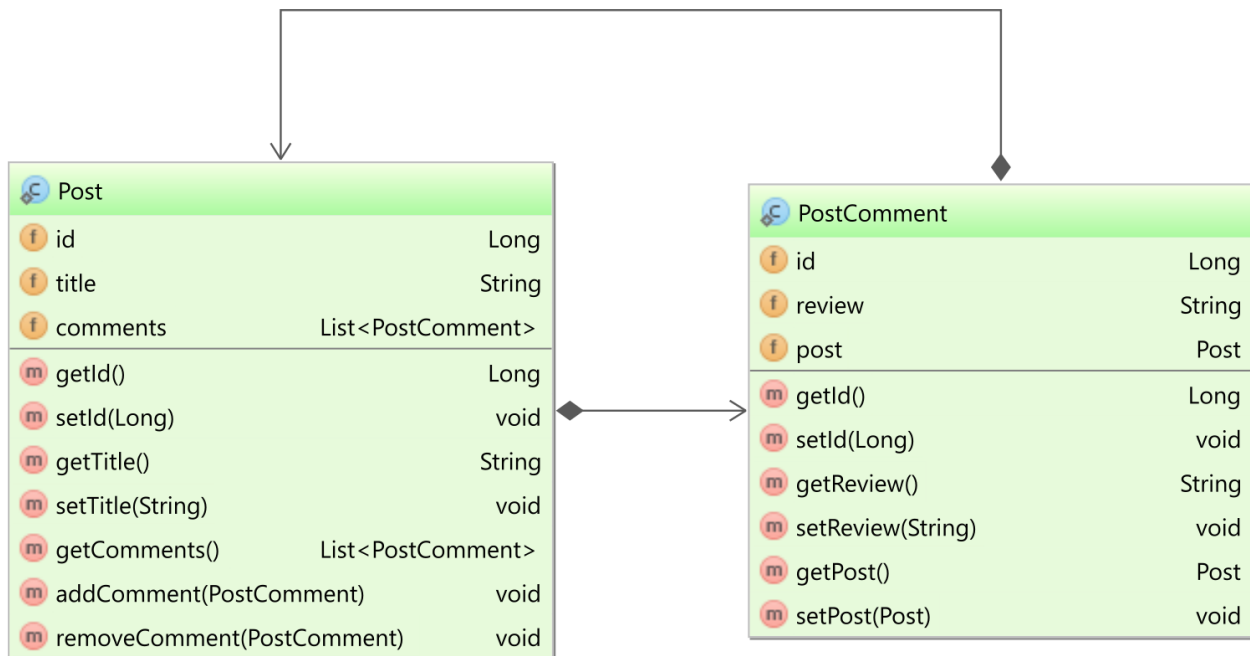


Figure 10.4: Bidirectional `@OneToMany` relationship

In a bidirectional association, only one side can control the underlying table relationship. For the bidirectional `@OneToMany` mapping, it is the child-side `@ManyToOne` association in charge of keeping the foreign key column value in sync with the in-memory Persistence Context. This is the reason why the bidirectional `@OneToMany` relationship must define the `mappedBy` attribute, indicating that it only mirrors the `@ManyToOne` child-side mapping.

```

@OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostComment> comments = new ArrayList<>();
  
```



Even if the child-side is in charge of synchronizing the entity state changes with the database foreign key column value, a bidirectional association must always have both the parent-side and the child-side in sync.

To synchronize both ends, it is practical to provide parent-side helper methods that add/remove child entities.

```
public void addComment(PostComment comment) {
    comments.add(comment);
    comment.setPost(this);
}

public void removeComment(PostComment comment) {
    comments.remove(comment);
    comment.setPost(null);
}
```

One of the major advantages of using a bidirectional association is that entity state transitions can be cascaded from the parent entity to its children. In the following example, when persisting the parent `Post` entity, all the `PostComment` child entities are persisted as well.

```
Post post = new Post("First post");

PostComment comment1 = new PostComment("My first review");
post.addComment(comment1);
PostComment comment2 = new PostComment("My second review");
post.addComment(comment2);

entityManager.persist(post);
```

```
INSERT INTO post (title, id) VALUES ('First post', 1)
INSERT INTO post_comment (post_id, review, id) VALUES (1, 'My first review', 2)
INSERT INTO post_comment (post_id, review, id) VALUES (1, 'My second review', 3)
```

When removing a comment from the parent-side collection:

```
post.removeComment(comment1);
```

The orphan removal attribute instructs Hibernate to generate a delete DML statement on the targeted child entity:

```
DELETE FROM post_comment WHERE id = 2
```

Equality-based entity removal

The helper method for the child entity removal relies on the underlying child object equality for matching the collection entry that needs to be removed.

If the application developer does not choose to override the default `equals` and `hashCode` methods, the `java.lang.Object` identity-based equality is going to be used. The problem with this approach is that the application developer must supply a child entity object reference that is contained in the current child collection.

Sometimes child entities are loaded in one web request and saved in a `HttpSession` or a `Stateful Enterprise Java Bean`. Once the `Persistence Context`, which loaded the child entity is closed, the entity becomes detached. If the child entity is sent for removal into a new web request, the child entity must be reattached or merged into the current `Persistence Context`. This way, if the parent entity is loaded along with its child entities, the removal operation will work properly since the removing child entity is already managed and contained in the children collection.

If the entity has not changed, reattaching this child entity will be redundant and so the `equals` and the `hashCode` methods must be overridden to express equality in terms of a unique business key. In case the child entity has a `@NaturalId` or a unique attribute set, the `equals` and the `hashCode` methods can be implemented on top of that. Assuming the `PostComment` entity has the following two columns whose combination render a unique business key, the equality contract can be implemented as follows:

```
private String createdBy;

@Temporal(TemporalType.TIMESTAMP)
private Date createdOn = new Date();

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    PostComment that = (PostComment) o;
    return Objects.equals(createdBy, that.createdBy) &&
        Objects.equals(createdOn, that.createdOn);
}

@Override
public int hashCode() {
    return Objects.hash(createdBy, createdOn);
}
```

Identifier-based equality

The `java.lang.Object.equalsa` method Javadoc demands the strategy be *reflexive*, *symmetric*, *transitive*, and *consistent*.

While the first three equality properties (*reflexive*, *symmetric*, *transitive*) are easier to achieve, especially with the `java.util.Objectsb` `equals` and `hashCode` utilities, *consistency* requires more diligence.



For a JPA or Hibernate entity, *consistency* means that the equality result is *reflexive*, *symmetric* and *transitive* across all **entity state transitions** (e.g. new/transient, managed, detached, removed).

If the entity has a `@NaturalId` attribute, then ensuring *consistency* is simple since the natural key is assigned even from the transient state, and this attribute never changes afterward. However, not all entities have a natural key to use for equality checks, so another table column must be used instead.

Luckily, most database tables have a primary key, which uniquely identifies each row of a particular table. The only caveat is to ensure *consistency* across all entity state transitions.

A naive implementation would look like this:

```
@Entity
public class Post {

    @Id @GeneratedValue
    private Long id;

    //Getters and setters omitted for brevity

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Post)) return false;
        return Objects.equals(id, ((Post) o).getId());
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

^a<https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html#equals-java.lang.Object->

^b<https://docs.oracle.com/javase/7/docs/api/java/util/Objects.html>

Now, when running the following test case:

```
Set<Post> tuples = new HashSet<>();
tuples.add(entity);
assertTrue(tuples.contains(entity));

doInJPA(entityManager -> {
    entityManager.persist(entity);
    entityManager.flush();
    assertTrue(tuples.contains(entity));
});
```

The final assertion will fail because the entity can no longer be found in the `Set` collection since the new identifier value is associated with a different `Set` bucket than the one where the entity got stored in.

To fix it, the `equals` and `hashCode` methods must be changed as follows:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Post)) return false;
    return id != null && id.equals(((Post) o).getId());
}

@Override
public int hashCode() {
    return 31;
}
```

When the entity identifier is null, equality can only be guaranteed for the same object references. Otherwise, no transient object is equal to any other transient or persisted object. That's why the `equals` method above skips the identifier check if the current object has a null identifier value.

Using a constant `hashCode` value solves the previous bucket-related problem associated with `Set(s)` or `Map(s)` because, this time, only a single bucket is going to be used. Although in general, using a single `Set` bucket is not very efficient for large collection of objects, in this particular use case, this workaround is valid since managed collections should be rather small to be efficient. Otherwise, fetching a `@OneToMany Set` with thousands of entities is orders of magnitude more costly than the one-bucket search penalty.

When using `List(s)`, the constant `hashCode` value is not an issue at all, and, for bidirectional collections, the Hibernate-internal `PersistentList(s)` are more efficient than `PersistentSet(s)`.



The bidirectional `@OneToMany` association generates efficient DML statements because the `@ManyToOne` mapping is in charge of the table relationship. Because it simplifies data access operations as well, the bidirectional `@OneToMany` association is worth considering when the size of the child records is relatively low.

10.3.2 Unidirectional `@OneToMany`

The unidirectional `@OneToMany` association is very tempting because the mapping is simpler than its bidirectional counterpart. Because there is only one side to take into consideration, there is no need for helper methods and the mapping does not feature a `mappedBy` attribute either.

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostComment> comments = new ArrayList<>();
```

Unfortunately, in spite its simplicity, the unidirectional `@OneToMany` association is less efficient than the unidirectional `@ManyToOne` mapping or the bidirectional `@OneToMany` association.

Against any intuition, the unidirectional `@OneToMany` association does not map to a one-to-many table relationship. Because there is no `@ManyToOne` side to control this relationship, Hibernate uses a separate junction table to manage the association between a parent row and its child records.



Figure 10.5: The `@OneToMany` table relationship

The table `post_post_comment` has two foreign key columns, which reference both the parent-side row (the `Post_id` column is a foreign key to the `post` table primary key) and the child-side entity (the `comments_id` references the primary key of the `post_comment` table).

Without going into analyzing the associated data access operations, it is obvious that joining three tables is less efficient than joining just two. Because there are two foreign keys, there need to be two indexes (instead of one), so the index memory footprint increases. However, since this is a regular table mapping for a many-to-many relationship, the extra table and the increased memory footprint are not even the biggest performance issue. The algorithm for managing the collection state is what makes any unidirectional `@OneToMany` association less attractive.

Considering there is a `Post` entity with two `PostComment` child records, obtained by running the following example:

```
Post post = new Post("First post");

post.getComments().add(new PostComment("My first review"));
post.getComments().add(new PostComment("My second review"));
post.getComments().add(new PostComment("My third review"));

entityManager.persist(post);
```

While for a bidirectional `@OneToMany` association there were three child rows being added, the unidirectional association requires three additional inserts for the junction table records.

```
INSERT INTO post (title, id) VALUES ('First post', 1)
INSERT INTO post_comment (review, id) VALUES ('My first review', 2)
INSERT INTO post_comment (review, id) VALUES ('My second review', 3)
INSERT INTO post_comment (review, id) VALUES ('My third review', 4)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 2)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 3)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 4)
```

When removing the first element of the collection:

```
post.getComments().remove(0);
```

Hibernate generates the following DML statements:

```
DELETE FROM post_post_comment WHERE Post_id = 1
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 3)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 4)
DELETE FROM post_comment WHERE id = 2
```

First, all junction table rows associated with the parent entity are deleted, and then the remaining in-memory records are added back again. The problem with this approach is that instead of a single junction table remove operation, the database has way more DML statements to execute.

Another problem is related to indexes. If there is an index on each foreign key column (which is the default for many relational databases), the database engine must delete the associated index entries only to add back the remaining ones. The more elements a collection has, the less efficient a remove operation gets.



The unidirectional `@OneToMany` relationship is less efficient both for reading data (three joins are required instead of two), as for adding (two tables must be written instead of one) or removing (entries are removed and added back again) child entries.

10.3.3 Ordered unidirectional `@OneToMany`

If the collection can store the index of every collection element, the unidirectional `@OneToMany` relationship may benefit for some element removal operations. First, an `@OrderColumn` annotation must be defined along the `@OneToMany` relationship mapping:

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
@OrderColumn(name = "entry")
private List<PostComment> comments = new ArrayList<>();
```

At the database level, the `entry` column is included in the junction table.

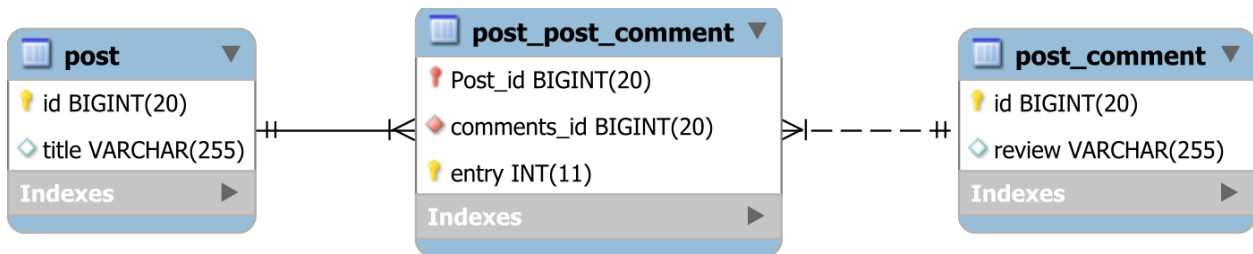


Figure 10.6: The unidirectional `@OneToMany` with an `@OrderColumn`



It is better not to mistake the `@OrderColumn` with the `@OrderBy` JPA annotation. While the former allows the JPA provider to materialize the element index into a dedicated database column so that the collection is sorted using an `ORDER BY` clause, the latter does the sorting at runtime based on the ordering criteria provided by the `@OrderBy` annotation.

Considering there are three `PostComment` entities added for a given `Post` parent entity:

```
post.getComments().add(new PostComment("My first review"));
post.getComments().add(new PostComment("My second review"));
post.getComments().add(new PostComment("My third review"));
```

The index of every collection element is going to be stored in the `entry` column of the junction table:

```
INSERT INTO post_comment (review, id) VALUES ('My first review', 2)
INSERT INTO post_comment (review, id) VALUES ('My second review', 3)
INSERT INTO post_comment (review, id) VALUES ('My third review', 4)
INSERT INTO post_post_comment (Post_id, entry, comments_id) VALUES (1, 0, 2)
INSERT INTO post_post_comment (Post_id, entry, comments_id) VALUES (1, 1, 3)
INSERT INTO post_post_comment (Post_id, entry, comments_id) VALUES (1, 2, 4)
```

When removing elements from the tail of the collection:

```
post.getComments().remove(2);
```

Hibernate only requires a single junction table delete statement:

```
DELETE FROM post_post_comment WHERE Post_id = 1 AND entry = 2
DELETE FROM post_comment WHERE id = 4
```

Unfortunately, this optimization does not hold for entries that are located towards the head of the collection. So, if we remove the first element:

```
post.getComments().remove(0);
```

Hibernate deletes the last entry associated with the parent row from the junction table, and then it updates the remaining entries to preserve the same element ordering as the in-memory collection snapshot:

```
DELETE FROM post_post_comment WHERE Post_id = 1 AND entry = 2
UPDATE post_post_comment SET comments_id = 4 WHERE Post_id = 1 AND entry = 1
UPDATE post_post_comment SET comments_id = 3 WHERE Post_id = 1 AND entry = 0
DELETE FROM post_comment WHERE id = 2
```



If the unidirectional `@OneToMany` collection is used like a stack and elements are always removed from the collection tail, the remove operations will be more efficient when using an `@OrderColumn`. But the closer an element is to the head of the list, the more update statements must be issued, and the additional updates have an associated performance overhead.

10.3.4 @OneToMany with @JoinColumn

JPA 2.0 added support for mapping the `@OneToMany` association with a `@JoinColumn` so that it can map the one-to-many table relationship. With the `@JoinColumn`, the `@OneToMany` association controls the child table foreign key, so there is no need for a junction table.

On the JPA side, the class diagram is identical to the aforementioned unidirectional `@OneToMany` relationship, and the only difference is the JPA mapping which takes the additional `@JoinColumn`:

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
@JoinColumn(name = "post_id")
private List<PostComment> comments = new ArrayList<>();
```

When adding three `PostComment` entities, Hibernate generates the following SQL statements:

```
post.getComments().add(new PostComment("My first review"));
post.getComments().add(new PostComment("My second review"));
post.getComments().add(new PostComment("My third review"));
```

```
INSERT INTO post_comment (review, id) VALUES ('My first review', 2)
INSERT INTO post_comment (review, id) VALUES ('My second review', 3)
INSERT INTO post_comment (review, id) VALUES ('My third review', 4)
UPDATE post_comment SET post_id = 1 WHERE id = 2
UPDATE post_comment SET post_id = 1 WHERE id = 3
UPDATE post_comment SET post_id = 1 WHERE id = 4
```

Besides the regular insert statements, Hibernate issues three update statements for setting the `post_id` column on the newly inserted child records. The update statements are generated by the Hibernate-internal `CollectionRecreateAction` which tries to preserve the element order whenever the collection state changes. In this particular case, the `CollectionRecreateAction` should not be scheduled for execution, however, as of writing (Hibernate 5.2.3), this issue still replicates.

When deleting the last element of the collection:

```
post.getComments().remove(2);
```

Hibernate generates the following SQL statements:

```
UPDATE post_comment SET post_id = null WHERE post_id = 1 AND id = 4
DELETE from post_comment WHERE id = 4
```

Again, there is an additional update statement associated with the child removal operation. When a child entity is removed from the parent-side collection, Hibernate sets the child table foreign key column to null. Afterward, the orphan removal logic kicks in, and it triggers a delete statement against the disassociated child entity.

Unlike the regular `@OneToMany` association, the `@JoinColumn` alternative is consistent in regard to the collection entry position that is being removed. So, when removing the first element of the collection:

```
post.getComments().remove(0);
```

Hibernate still generates an additional update statement:

```
UPDATE post_comment SET post_id = null WHERE post_id = 1 AND id = 2
DELETE from post_comment WHERE id = 2
```

Bidirectional `@OneToMany` with `@JoinColumn` relationship

The `@OneToMany` with `@JoinColumn` association can also be turned into a bidirectional relationship, but it requires instructing the child-side to avoid any insert and update synchronization:

```
@ManyToOne
@JoinColumn(name = "post_id", insertable = false, updatable = false)
private Post post;
```



The redundant update statements are generated for both the unidirectional and the bidirectional association, so the most efficient foreign key mapping is the `@ManyToOne` association.

10.3.5 Unidirectional @OneToMany Set

All the previous examples were using `List(s)`, but Hibernate supports `Set(s)` as well. For the next exercise, the `PostComment` entity uses the following mapping:

```
@Entity(name = "PostComment") @Table(name = "post_comment")
public class PostComment {

    @Id @GeneratedValue
    private Long id;

    private String slug;

    private String review;

    public PostComment() {
        byte[] bytes = new byte[8];
        ByteBuffer.wrap(bytes).putDouble(Math.random());
        slug = Base64.getEncoder().encodeToString(bytes);
    }

    public PostComment(String review) {
        this();
        this.review = review;
    }

    //Getters and setters omitted for brevity

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        PostComment comment = (PostComment) o;
        return Objects.equals(slug, comment.getSlug());
    }

    @Override
    public int hashCode() {
        return Objects.hash(slug);
    }
}
```

This time, the `PostComment` entity uses a `slug` attribute which provide a way to uniquely identify each comment belonging to a given `Post` entity.

Because the `PostComment` references are going to be stored in a `java.util.Set`, it is best to override the `equals` and `hashCode` `Object` methods according to the entity business key. In this particular example, the `PostComment` does not have any meaningful business key, so the `slug` attribute is used for the equality checks.

The parent `Post` entity has a unidirectional `@OneToMany` association that uses a `java.util.Set`:

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
private Set<PostComment> comments = new HashSet<>();
```

When adding three `PostComment` entities:

```
post.getComments().add(new PostComment("My first review"));
post.getComments().add(new PostComment("My second review"));
post.getComments().add(new PostComment("My third review"));
```

Hibernate generates the following SQL statements:

```
INSERT INTO post_comment (review, slug, id)
VALUES ('My second review', 'P+HLCF25scI=', 2)
INSERT INTO post_comment (review, slug, id)
VALUES ('My first review', 'P9y80GLTCyg=', 3)
INSERT INTO post_comment (review, slug, id)
VALUES ('My third review', 'P+fWF+Ck/LY=', 4)

INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 2)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 3)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 4)
```

The remove operation is much more effective this time because the collection element order needs not be enforced anymore.

When removing the `PostComment` child entities:

```
for(PostComment comment: new ArrayList<>(post.getComments())) {
    post.getComments().remove(comment);
}
```

Hibernate generates one statement for removing the junction table entries and three delete statements for the associated `post_comment` records.

```
DELETE FROM post_post_comment WHERE Post_id = 1
```

```
DELETE FROM post_comment WHERE id = 2
```

```
DELETE FROM post_comment WHERE id = 3
```

```
DELETE FROM post_comment WHERE id = 4
```

To avoid using a secondary table, the `@OneToMany` mapping can use the `@JoinColumn` annotation.

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
@JoinColumn(name = "post_id")
private Set<PostComment> comments = new HashSet<>();
```

Upon inserting three `PostComment` entities, Hibernate generates the following statements:

```
INSERT INTO post_comment (review, slug, id)
VALUES ('My third review', 'P8pcnLprqcQ=', 2)
INSERT INTO post_comment (review, slug, id)
VALUES ('My second review', 'P+Gau1+Hhxs=', 3)
INSERT INTO post_comment (review, slug, id)
VALUES ('My first review', 'P+kr9LOQTK0=', 4)
```

```
UPDATE post_comment SET post_id = 1 WHERE id = 2
```

```
UPDATE post_comment SET post_id = 1 WHERE id = 3
```

```
UPDATE post_comment SET post_id = 1 WHERE id = 4
```

When deleting all three `PostComment` entities, the generated statements look like this:

```
UPDATE post_comment SET post_id = null WHERE post_id = 1
```

```
DELETE FROM post_comment WHERE id = 2
```

```
DELETE FROM post_comment WHERE id = 3
```

```
DELETE FROM post_comment WHERE id = 4
```



Although it is an improvement over the unidirectional unordered or ordered `List`, the unidirectional `Set` is still less efficient than the bidirectional `@OneToMany` association.

10.4 @ElementCollection

Although it is not an entity association type, the `@ElementCollection` is very similar to the unidirectional `@OneToMany` relationship. To represent collections of basic types (e.g. `String`, `int`, `BigDecimal`) or embeddable types, the `@ElementCollection` must be used instead. If the previous associations involved multiple entities, this time, there is only a single `Post` entity with a collection of `String` comments.

Value types inherit the persistent state from their parent entities, so their lifecycle is also bound to the owner entity. Any operation against the entity collection is going to be automatically materialized into a DML statement.

From a database perspective, there is one child table holding both the foreign key column and the collection element value.

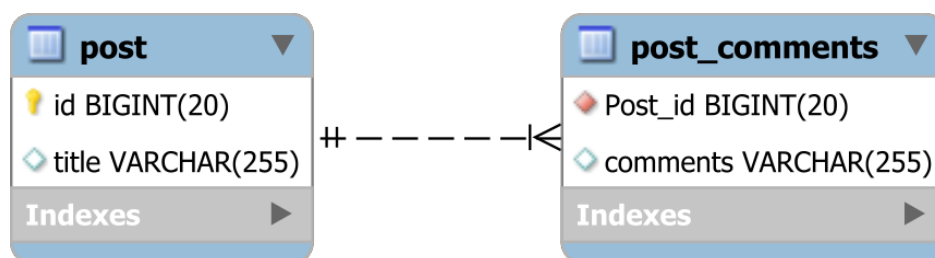


Figure 10.7: The `@ElementCollection` table relationship

10.4.1 @ElementCollection List

Let's consider the use case when a `List` is used with `@ElementCollection`:

Post		
f	id	Long
f	title	String
f	comments	List<String>
m	getId()	Long
m	setId(Long)	void
m	getTitle()	String
m	setTitle(String)	void
m	getComments()	List<String>

Figure 10.8: The `@ElementCollection` relationship

The mapping for the `comments` collection looks as follows:

```
@ElementCollection
private List<String> comments = new ArrayList<>();
```

When it comes to adding or removing child records, the `@ElementCollection` behaves like a unidirectional `@OneToMany` relationship, annotated with `CascadeType.ALL` and `orphanRemoval`.

To persist three comments, the data access layer only has to add them to the parent entity collection:

```
post.getComments().add("My first review");
post.getComments().add("My second review");
post.getComments().add("My third review");
```

Hibernate issues the insert statements during Persistence Context flushing:

```
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My first review')
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My second review')
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My third review')
```

Unfortunately, the remove operation uses the same logic as the unidirectional `@OneToMany` association, so when removing the first collection element:

```
post.getComments().remove(0);
```

Hibernate deletes all the associated child-side records and re-inserts the in-memory ones back into the database table:

```
DELETE FROM Post_comments WHERE Post_id = 1
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My second review')
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My third review')
```



Using an `@ElementCollection` with a `List` is not very efficient because the association is considered to be a bag, in Hibernate terminology.

A bag does not guarantee that elements are uniquely identifiable, hence Hibernate needs to delete and reinsert the elements associated with a given parent entity whenever a change occurs to the `@ElementCollection List`.

10.4.2 @ElementCollection Set

On the other hand, when using a `Set`, the `@ElementCollection` no longer behaves like a bag, and the generated SQL statements are going to be more efficient.

So, let's change the `comments` collection to use a `Set` this time:

```
@ElementCollection
private Set<String> comments = new HashSet<>();
```

After inserting the same 3 comments, as we did when using a `List` with `@ElementCollection`, let's see how the `Set` behaves when it comes to removing elements.

When removing the first element, we added to the `Set`:

```
post.getComments().remove("My first review");
```

Hibernate generates a single SQL delete statement:

```
DELETE FROM Post_comments WHERE Post_id = 1 and comments = 'My first review'
```

When removing the last element, we added to the `Set`:

```
post.getComments().remove("My third review");
```

Again, Hibernate generates a single SQL delete statement:

```
DELETE FROM Post_comments WHERE Post_id = 1 and comments = 'My third review'
```

So, if you want to use an `@ElementCollection` and benefit from its simplicity, it's much better if you are using a `Set`, instead of a `List`.



However, even if using a `Set` can render more efficient SQL statements than when using a `List`, it's still much better if you replace the `@ElementCollection` with either a bidirectional `@OneToMany` association or just a simple `@ManyToOne` association that maps the Foreign Key column.

This way, you can remove elements without having to fetch the collection via the parent entity.

10.5 @OneToOne

From a database perspective, the one-to-one association is based on a foreign key that is constrained to be unique. This way, a parent row can be referenced by at most one child record only.

In JPA, the `@OneToOne` relationship can be either unidirectional or bidirectional.

10.5.1 Unidirectional @OneToOne

In the following example, the `Post` entity represents the parent-side, while the `PostDetails` is the child-side of the one-to-one association.

As already mentioned, the JPA entity relationship diagram matches exactly the one-to-one table relationship.

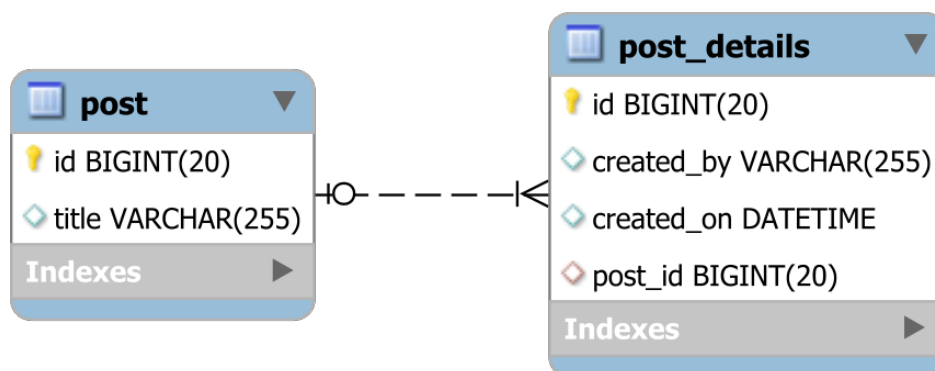


Figure 10.9: The one-to-one table relationship

Even from the Domain Model side, the unidirectional `@OneToOne` relationship is strikingly similar to the unidirectional `@ManyToOne` association.

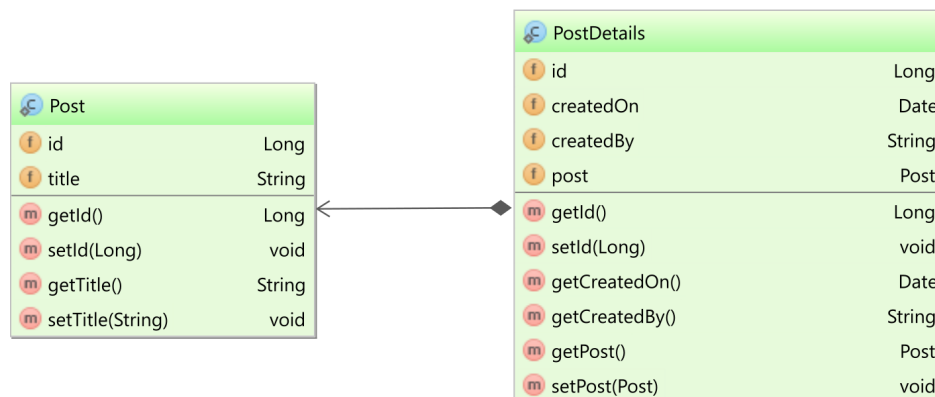


Figure 10.10: The unidirectional `@OneToOne` relationship

The mapping is done through the `@OneToOne` annotation, which, just like the `@ManyToOne` mapping, might also take a `@JoinColumn` as well.

```
@OneToOne
@JoinColumn(name = "post_id")
private Post post;
```

The unidirectional `@OneToOne` association controls the associated foreign key, so, when the `post` attribute is set:

```
Post post = entityManager.find(Post.class, 1L);
PostDetails details = new PostDetails("John Doe");
details.setPost(post);
entityManager.persist(details);
```

Hibernate populate the foreign key column with the associated `post` identifier:

```
INSERT INTO post_details (created_by, created_on, post_id, id)
VALUES ('John Doe', '2016-01-08 11:28:21.317', 1, 2)
```

Even if this is a unidirectional association, the `Post` entity is still the parent-side of this relationship. To fetch the associated `PostDetails`, a JPQL query is needed:

```
PostDetails details = entityManager.createQuery(
    "select pd " +
    "from PostDetails pd " +
    "where pd.post = :post", PostDetails.class)
    .setParameter("post", post)
    .getSingleResult();
```

If the `Post` entity always needs its `PostDetails`, a separate query might not be desirable. To overcome this limitation, it is important to know the `PostDetails` identifier prior to loading the entity.

One workaround would be to use a `@NaturalId`, which might not require a database access if the entity is stored in the second-level cache. Fortunately, there is even a simpler approach which is also portable across JPA providers as well. The JPA 2.0 specification added support for derived identifiers, making possible to link the `PostDetails` identifier to the `post` table primary key.

This way, the `post_details` table primary key can also be a foreign key referencing the `post` table identifier.

The `PostDetails @OneToOne` mapping is changed as follows:

```
@OneToOne
@MapsId
private Post post;
```

This time, the table relationship does not feature any additional foreign key column since the `post_details` table primary key references the `post` table primary key:

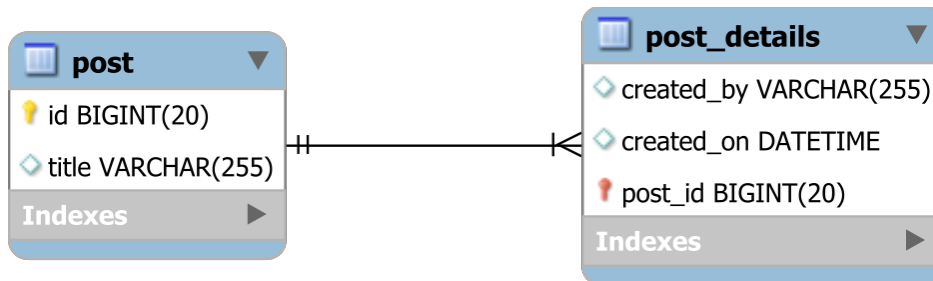


Figure 10.11: The shared key one-to-one

Because `PostDetails` has the same identifier as the parent `Post` entity, it can be fetched without having to write a JPQL query.

```
PostDetails details = entityManager.find(PostDetails.class, post.getId());
```

The shared primary key efficiency

First of all, the shared primary key approach reduces the memory footprint of the child-side table indexes since it requires a single indexed column instead of two. The more records a child table has, the better the improvement gain for reducing the number of indexed columns.

More, the child entity can now be simply retrieved from the second-level cache, therefore preventing a database hit. In the previous example, because the child entity identifier was not known, a query was inevitable. To optimize the previous use case, the *query cache* would be required as well, but the query cache is not without issues either.



Because of the reduced memory footprint and enabling the second-level cache direct retrieval, the JPA 2.0 *derived identifier* is the preferred `@OneToOne` mapping strategy. The shared primary key is not limited to unidirectional associations, being available for bidirectional `@OneToOne` relationships as well.

10.5.2 Bidirectional @OneToOne

A bidirectional `@OneToOne` association allows the parent entity to map the child-side as well:

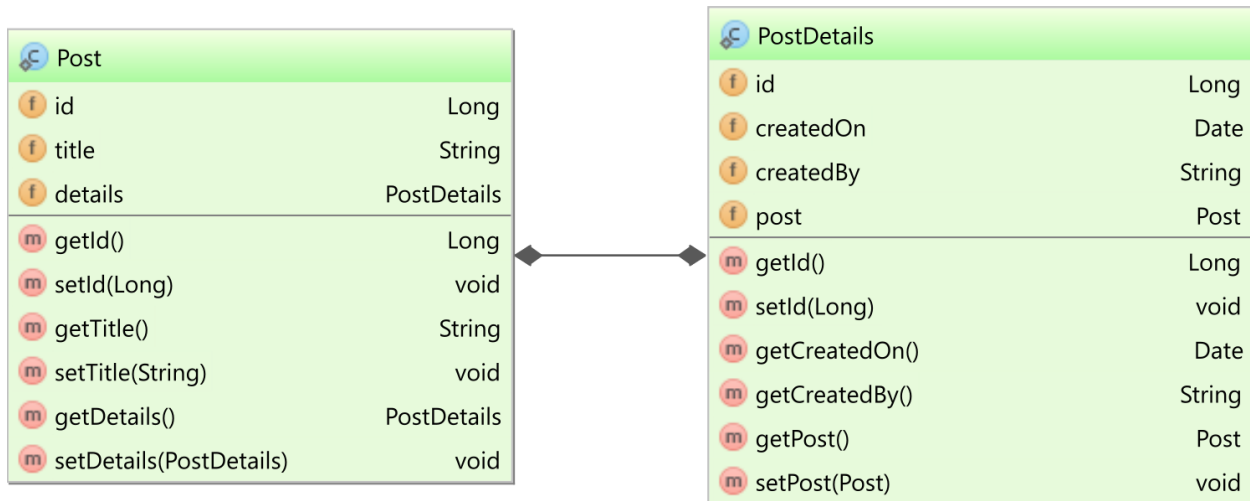


Figure 10.12: The bidirectional `@OneToOne` relationship

The parent-side defines a `mappedBy` attribute because the child-side (which can still share the primary key with its parent) is still in charge of this JPA relationship:

```
@OneToOne(mappedBy = "post", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
private PostDetails details;
```

Because this is a bidirectional relationship, the `Post` entity must ensure that both sides of this relationship are set upon associating a `PostDetails` entity:

```
public void setDetails(PostDetails details) {
    if (details == null) {
        if (this.details != null) this.details.setPost(null);
    }
    else details.setPost(this);
    this.details = details;
}
```

Unlike the parent-side `@OneToMany` relationship where Hibernate can simply assign a proxy even if the child collection is empty, the `@OneToOne` relationship must decide if to assign the child reference to null or to an `Object`, be it the actual entity object type or a runtime `Proxy`.

This is an issue that affects the parent-side `@OneToOne` association, while the child-side, which has an associated foreign key column, knows whether the parent reference should be null or not. For this reason, the parent-side must execute a secondary query to know if there is a mirroring foreign key reference on the child-side.

Even if the association is lazy, when fetching a `Post` entity:

```
Post post = entityManager.find(Post.class, 1L);
```

Hibernate fetches the child entity as well, so, instead of only one query, Hibernate requires two select statements:

```
SELECT p.id AS id1_0_0_, p.title AS title2_0_0_  
FROM   post p  
WHERE  p.id = 1
```

```
SELECT pd.post_id AS post_id3_1_0_, pd.created_by AS created_1_1_0_,  
       pd.created_on AS created_2_1_0_  
FROM   post_details pd  
WHERE  pd.post_id = 1
```

If the application developer only needs parent entities, the additional child-side secondary queries will be executed unnecessarily, and this might affect application performance. The more parent entities are needed to be retrieved, the more obvious the secondary queries performance impact gets.

Limitations

Even if the foreign key is `NOT NULL` and the parent-side is aware about its non-nullability through the optional attribute (e.g. `@OneToOne(mappedBy = "post", fetch = FetchType.LAZY, optional = false)`), Hibernate still generates a secondary select statement.

For every managed entity, the Persistence Context requires both the entity type and the identifier, so the child identifier must be known when loading the parent entity, and the only way to find the associated `post_details` primary key is to execute a secondary query. Because the child identifier is known when using `@MapsId`, in future, [HHH-10771^a](https://hibernate.atlassian.net/browse/HHH-10771) should address the secondary query issue.



Bytecode enhancement is the only viable workaround. However, it only works if the parent side is also annotated with `@LazyToOne(LazyToOneOption.NO_PROXY)`. Because it's simpler and more predictable, the unidirectional `@OneToOne` relationship is often preferred.

^a<https://hibernate.atlassian.net/browse/HHH-10771>

10.6 @ManyToMany

From a database perspective, the `@ManyToMany` annotation mirrors a many-to-many table relationship:



Figure 10.13: The many-to-many table relationship

Like the `@OneToMany` and `@OneToOne` relationships, the `@ManyToMany` association can be either unidirectional or bidirectional.

10.6.1 Unidirectional @ManyToMany List

In our case, if we need to map a unidirectional `@ManyToMany` association between the `Post` and `Tag` entities, it makes sense to have the `Post` entity map the `@ManyToMany` relationship since there is not much need for navigating this association from the `Tag` side (although we can still do it with a JPQL query).

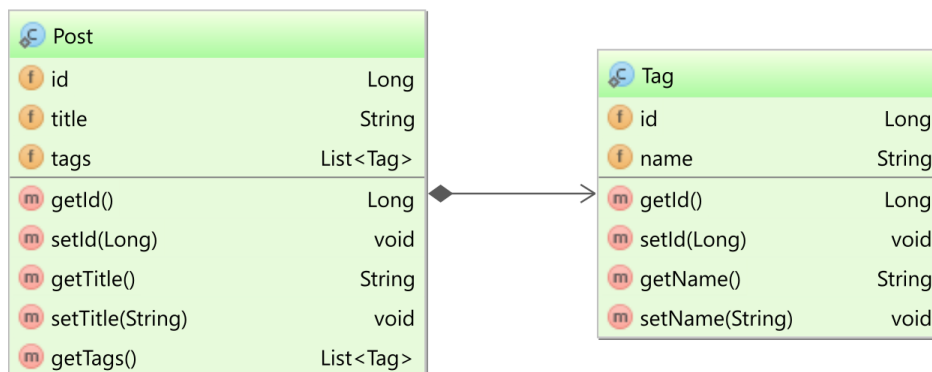


Figure 10.14: The unidirectional `@ManyToMany` relationship

In the `Post` entity, the `tags` collection using a `List`, like this:

```
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(name = "post_tag",
    joinColumns = @JoinColumn(name = "post_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id"))
private List<Tag> tags = new ArrayList<>();
```

When adding several entities:

```
Post post1 = new Post("JPA with Hibernate");
Post post2 = new Post("Native Hibernate");

Tag tag1 = new Tag("Java");
Tag tag2 = new Tag("Hibernate");

post1.getTags().add(tag1);
post1.getTags().add(tag2);
post2.getTags().add(tag1);

entityManager.persist(post1);
entityManager.persist(post2);
```

Hibernate manages to persist both the `Post` and the `Tag` entities along with their junction records.

```
INSERT INTO post (title, id) VALUES ('JPA with Hibernate', 1)
INSERT INTO post (title, id) VALUES ('Native Hibernate', 4)

INSERT INTO tag (name, id) VALUES ('Java', 2)
INSERT INTO tag (name, id) VALUES ('Hibernate', 3)

INSERT INTO post_tag (post_id, tag_id) VALUES (1, 2)
INSERT INTO post_tag (post_id, tag_id) VALUES (1, 3)
INSERT INTO post_tag (post_id, tag_id) VALUES (4, 2)
```

Cascading

For `@ManyToMany` associations, `CascadeType.REMOVE` does not make too much sense when both sides represent independent entities. In this case, removing a `Post` entity should not trigger a `Tag` removal because the `Tag` can be referenced by other posts as well. The same arguments apply to orphan removal since removing an entry from the `tags` collection should only delete the junction record and not the target `Tag` entity.



For both unidirectional and bidirectional associations, it is better to avoid the `CascadeType.REMOVE` mapping. Instead of `CascadeType.ALL`, the cascade attributes should be declared explicitly (e.g. `CascadeType.PERSIST`, `CascadeType.MERGE`).

But just like the unidirectional `@OneToMany` association, problems arise when it comes to removing the junction records from the `List`:

```
post1.getTags().remove(tag1);
```

Hibernate deletes all junction rows associated with the `Post` entity whose `Tag` association is being removed and inserts back the remaining ones:

```
DELETE FROM post_tag WHERE post_id = 1
INSERT INTO post_tag (post_id, tag_id) VALUES (1, 3)
```

10.6.2 Unidirectional @ManyToMany Set

When using a `List`, the `@ManyToMany` association behaves like a bag, and, as explained in the [@ElementCollection](#) section, Hibernate bags are not very efficient.

Therefore, let's change the `tags` collection to use a `Set`:

```
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(name = "post_tag",
    joinColumns = @JoinColumn(name = "post_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id")
)
private Set<Tag> tags = new HashSet<>();
```

Assuming, we added the same 3 `Tag` entities to the `Post` entity, if we remove the first `Tag` from the `tags` collection:

```
post1.getTags().remove(tag1);
```

Hibernate is going to generate a single SQL delete statement, this time:

```
DELETE FROM post_tag WHERE post_id = 1 AND tag_id = 2
```



For a `@ManyToMany` association, it's better to use the `Set` collection type, instead of `List`.

10.6.3 Bidirectional @ManyToMany

The bidirectional @ManyToMany relationship can be navigated from both the `Post` and the `Tag` side.

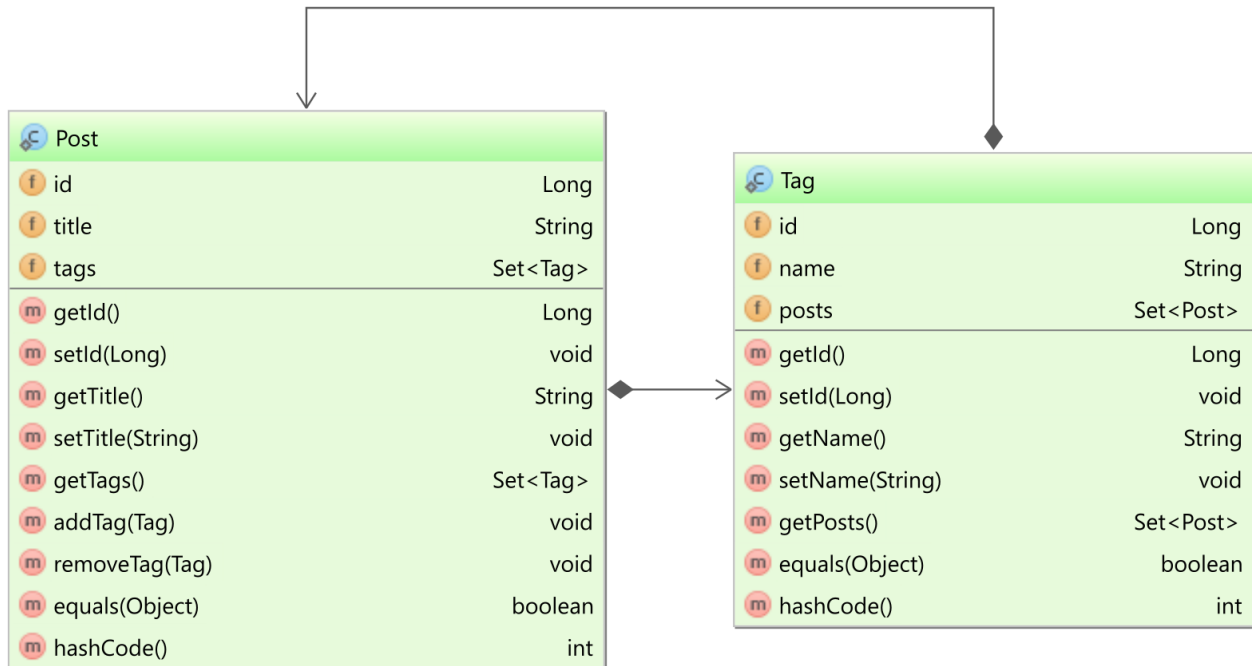


Figure 10.15: The bidirectional @ManyToMany relationship

While in the one-to-many and many-to-one associations the child-side is the one holding the foreign key, for a many-to-many table relationship both ends are parent-sides and the junction table plays the child-side role.

Because the junction table is hidden when using the default @ManyToMany mapping, the application developer must choose an owning and a mappedBy side.

The `Post` entity uses the same mapping as shown in the unidirectional @ManyToMany section:

```

@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(name = "post_tag",
    joinColumns = @JoinColumn(name = "post_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id"))
private Set<Tag> tags = new HashSet<>();
  
```

while the `Tag` entity adds a @ManyToMany association that looks as follows:

```

@ManyToMany(mappedBy = "tags")
private Set<Post> posts = new HashSet<>();
  
```

The `mappedBy` attribute tells Hibernate that the `Post` entity is the owning side of this association. In our case, the `mappedBy` attribute of the `@ManyToMany` annotation in the `Tag` entity points to the `tags` collection of the `Post` entity, meaning that the `Post` entity is going to propagate the association state changes to the underlying junction table.



Hibernate manages each side of a bidirectional `@ManyToMany` relationship just like a unidirectional `@OneToMany` association between the parent-side (e.g. `Post` or the `Tag`) and the hidden child-side (e.g. the `post_tag` table `post_id` or `tag_id` foreign keys).

For this reason, it's much more efficient to use a `Set` collection type, no matter if the `@ManyToMany` is unidirectional or bidirectional.

Like any bidirectional associations, helper methods are needed to keep both parent sides in sync. For a bidirectional `@ManyToMany` association, the helper methods must be added to the entity that is more likely to interact with. In our case, the root entity is the `Post`, so the helper methods are added to the `Post` entity:

```
public void addTag(Tag tag) {
    tags.add(tag);
    tag.getPosts().add(this);
}

public void removeTag(Tag tag) {
    tags.remove(tag);
    tag.getPosts().remove(this);
}
```

Both `Post` and `Tag` entities have unique attributes which can simplify the entity removal operation even when mixing detached and managed entities.

10.6.4 The `@OneToMany` alternative

We can also map a many-to-many table relationship using bidirectional `@OneToMany` associations from the parent entities to the child entity that maps the junction table.

Therefore, the `post_tag` child table is mapped by the `PostTag` entity, and its composite identifier is encapsulated in the `PostTagId` embeddable type as illustrated by the following diagram.

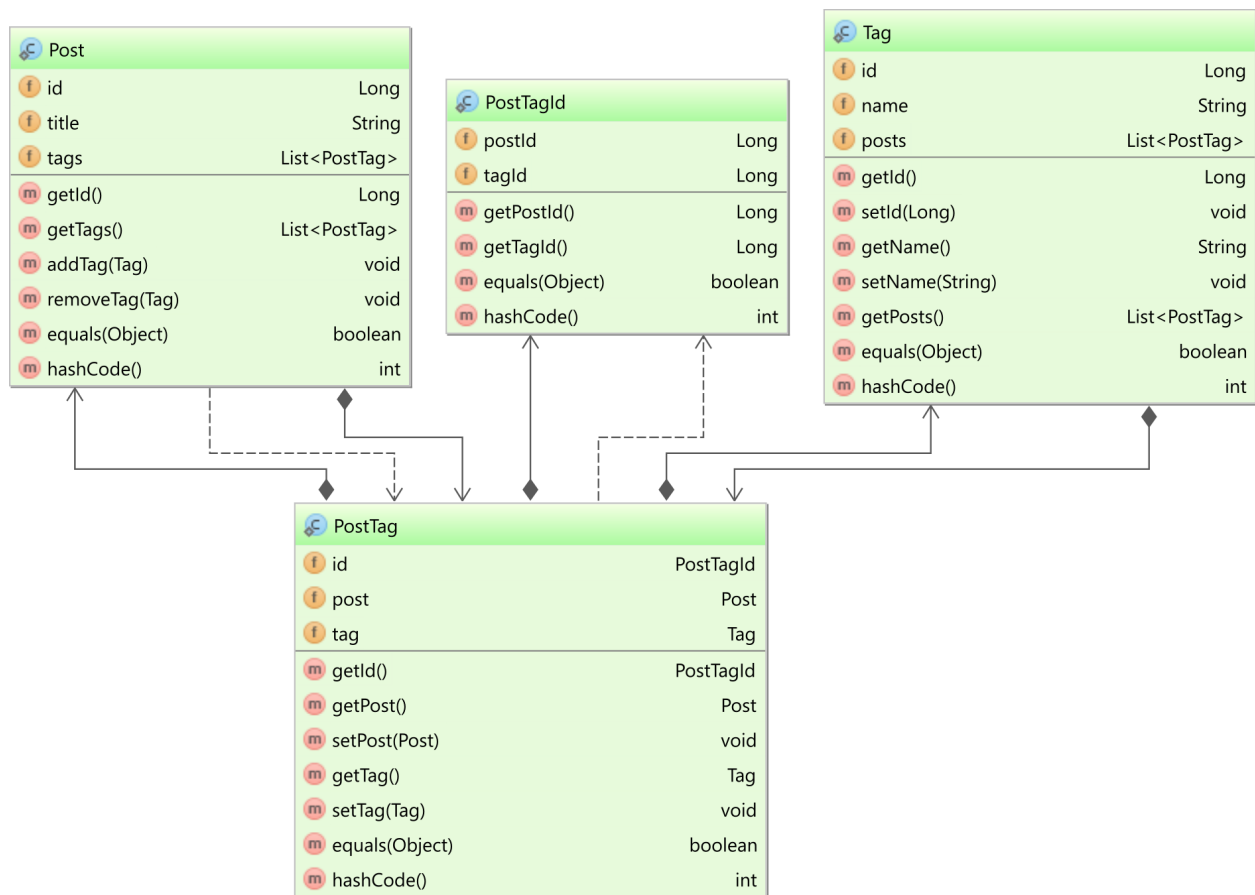


Figure 10.16: The @OneToMany as a many-to-many table relationship

The **PostTagId** maps the **postId** and **tagId** foreign key columns, and the **PostTag** entity uses the **postId** and **tagId** properties of the **PostTagId** identifier to map the **post** and the **tag** @ManyToMany associations.

The **Post** entity maps the bidirectional @OneToMany side of the **post** @ManyToMany association from the **PostTag** child entity:

```
@OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostTag> tags = new ArrayList<>();
```

The **Tag** entity maps the bidirectional @OneToMany side of the **tag** @ManyToMany association defined by the **PostTag** child entity:

```
@OneToMany(mappedBy = "tag", cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostTag> posts = new ArrayList<>();
```

This way, the bidirectional @ManyToMany relationship is transformed in two bidirectional @OneToMany associations.

The PostTag entity has a composed identifier made out of the post_id and tag_id columns.

```
@Embeddable
public class PostTagId implements Serializable {

    @Column(name = "post_id")
    private Long postId;

    @Column(name = "tag_id")
    private Long tagId;

    public PostTagId() {}

    public PostTagId(Long postId, Long tagId) {
        this.postId = postId;
        this.tagId = tagId;
    }

    public Long getPostId() {
        return postId;
    }

    public Long getTagId() {
        return tagId;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        PostTagId that = (PostTagId) o;
        return Objects.equals(postId, that.getPostId()) &&
            Objects.equals(tagId, that.getTagId());
    }

    @Override
    public int hashCode() {
        return Objects.hash(postId, tagId);
    }
}
```

The JPA specification requires the composite identifiers embeddable types to implement the `Serializable` interface, as well as implementing `equals` and `hashCode` based on the associated foreign key column values.

The `PostTag` entity defines two `@ManyToOne` associations, as follows:

```
@Entity
@Table(name = "post_tag")
public class PostTag {

    @EmbeddedId
    private PostTagId id;

    @ManyToOne
    @MapsId("postId")
    private Post post;

    @ManyToOne
    @MapsId("tagId")
    private Tag tag;

    private PostTag() {}

    public PostTag(Post post, Tag tag) {
        this.post = post;
        this.tag = tag;
        this.id = new PostTagId(post.getId(), tag.getId());
    }

    //Getters and setters omitted for brevity

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        PostTag that = (PostTag) o;
        return Objects.equals(post, that.getPost()) &&
            Objects.equals(tag, that.getTag());
    }

    @Override
    public int hashCode() {
        return Objects.hash(post, tag);
    }
}
```

The `@MapsId` annotation instructs Hibernate that the `post_id` and `tag_id` foreign key columns are managed by the `PostTagId` composite identifier, not by the `@ManyToOne` property.

Just like with any bidirectional association, we need to define the helper methods that allow us to keep the `@ManyToOne` and `@OneToMany` sides in sync.

However, as opposed to a standard bidirectional `@OneToMany` association, the `removeTag` helper method in the `Post` entity is much more complex because it needs to locate the `PostTag` associated with the current `Post` entity, as well as the `Tag` object reference in the `tags` collection that needs to be disassociated.

```
public void removeTag(Tag tag) {
    for (Iterator<PostTag> iterator = tags.iterator(); iterator.hasNext(); ) {
        PostTag postTag = iterator.next();
        if (postTag.getPost().equals(this) && postTag.getTag().equals(tag)) {
            iterator.remove();
            postTag.getTag().getPosts().remove(postTag);
            postTag.setPost(null);
            postTag.setTag(null);
            break;
        }
    }
}
```

Notice that, the `removeTag` method relies on the `PostTag` `equals` and `hashCode` methods in order to remove the desired entry.

When rerunning the entity removal example featured in the unidirectional `@ManyToMany` section:

```
post1.removeTag(tag1);
```

Hibernate issues a single delete statement, therefore targeting a single `PostTag` junction record:

```
DELETE FROM post_tag WHERE post_id = 1 AND tag_id = 3
```

Because the `@ManyToOne` side manages the foreign key column changes, the internal collection state of the `@OneToMany` side is not taken into consideration unless we remove an entry.

For this reason, reversing the order of the `tags` elements has absolutely no effect:

```
post1.getTags().sort(
    Comparator.comparing(
        (PostTag postTag) -> postTag.getId().getTagId()
    )
    .reversed()
);
```

To materialize the order of elements, the `@OrderColumn` must be used instead:

```
@OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
@OrderColumn(name = "entry")
private List<PostTag> tags = new ArrayList<>();
```

When using the aforementioned mapping, the `post_tag` junction table features an `entry` column storing the collection element order.

Therefore, if we reverse the element order as we did previously, Hibernate is going to update the `entry` column to maintain the current `PostTag` element order:

```
UPDATE post_tag SET entry = 0 WHERE post_id = 1 AND tag_id = 4
UPDATE post_tag SET entry = 1 WHERE post_id = 1 AND tag_id = 3
```



Mapping the junction entity of a many-to-many table relationship allows us to persist additional column into the junction table (e.g. `created_on`, `created_by`), which cannot be done when using the JPA `@ManyToMany` annotation.

10.7 Hypersistence Optimizer

Knowing which are the most efficient associations is of paramount importance when it comes to designing a high-performance data access layer. But choosing the right mappings is not sufficient. You also have to check the Hibernate configuration properties, as well as the queries that get executed.

For this purpose, I created the [Hypersistence Optimizer](https://vladmihalcea.com/hypersistence-optimizer/)¹ tool. While you could manually investigate the JPA and Hibernate mappings and configurations, having a tool to do this task on your behalf is going to save you a lot of time.

More, Hypersistence Optimizer can validate your JPA and Hibernate usage during testing, and you could even assert the number of issues detected by the tool and trigger a build failure whenever a performance-impacting issue is detected. This way, you can ensure that future data access logic changes won't affect the performance of your application.

Because you purchased this book, you have a **25\$** discount for any Hypersistence Optimizer license. All you have to do is to use the **HO25OFF** coupon code when making the purchase. Or, you can use [this link](https://vladmihalcea.com/hypersistence-optimizer/?coupon_code=HO25OFF)² to quickly navigate to the Hypersistence Optimizer sales page with the discount coupon activated.

10.7.1 Testimonials

“It really pays off when it comes to analyzing complex applications. For architectures that are difficult to manage, it quickly provides clues for further analysis.

This is a huge help both when setting up a new implementation and when optimizing a legacy application.”

– Kevin Peters (Software Engineer - codecentric AG)

“Let's face it, JPA and Hibernate are very powerful but not simple tools. Even experienced developers make mistakes that may result in very heavy database queries, dramatically lowering overall application performance.

Hypersistence Optimizer feels like an additional team member - a JPA expert that is there to help and show how can you optimize your mappings and configurations before you ship them to production.”

– Maciej Walkowiak (Freelance Tech Lead)

¹<https://vladmihalcea.com/hypersistence-optimizer/>

²https://vladmihalcea.com/hypersistence-optimizer/?coupon_code=HO25OFF

11. Inheritance

Java, like any other object-oriented programming language, makes heavy use of inheritance and polymorphism. Inheritance allows defining class hierarchies that offer different implementations of a common interface.

Conceptually, the Domain Model defines both data (e.g. persisted entities) and behavior (business logic). Nevertheless, inheritance is more useful for varying behavior rather than reusing data (composition is much more suitable for sharing structures). Even if the data (persisted entities) and the business logic (transactional services) are decoupled, inheritance can still help varying business logic (e.g. [Visitor pattern](https://en.wikipedia.org/wiki/Visitor_pattern)¹).

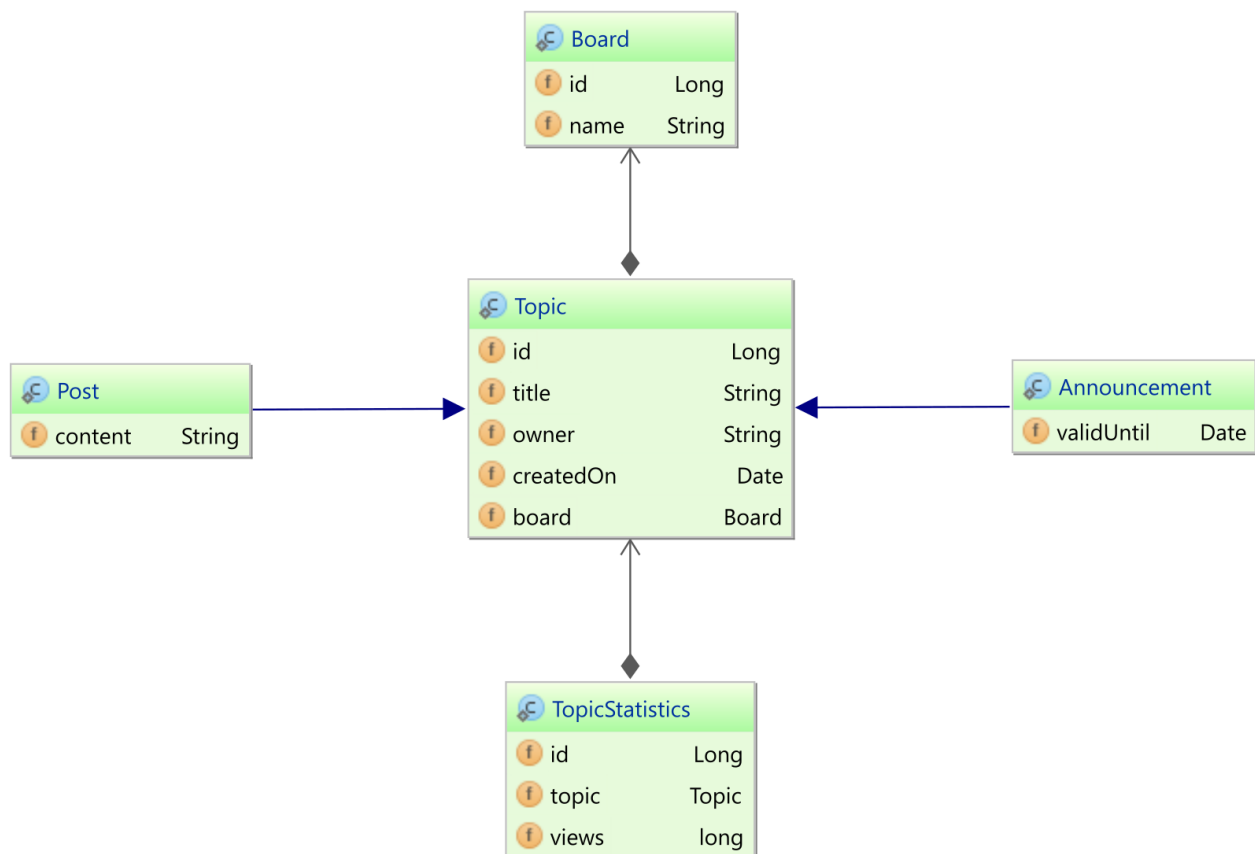


Figure 11.1: Domain Model Inheritance

¹https://en.wikipedia.org/wiki/Visitor_pattern

The root entity of this Domain Model is the `Board` entity because, either directly or indirectly, all the other entities are associated with a `Board`.

```
@Entity @Table(name = "board")
public class Board {

    @Id @GeneratedValue
    private Long id;

    private String name;

    //Getters and setters omitted for brevity
}
```

The end user can submit either a `Post` or an `Announcement` on a particular `Board`. Because the `Post` and the `Announcement` share the same functionality (differing only in data), they both inherit from a `Topic` base class.

The `Topic` class defines a relationship to a `Board` entity, hence the `Post` and the `Announcement` entities can also be associated with a `Board` instance.

```
@Entity @Table(name = "topic")
public class Topic {

    @Id @GeneratedValue
    private Long id;

    private String title;

    private String owner;

    @Temporal(TemporalType.TIMESTAMP)
    private Date createdOn = new Date();

    @ManyToOne(fetch = FetchType.LAZY)
    private Board board;

    //Getters and setters omitted for brevity
}
```

Both the `Post` and the `Announcement` entities extend the `Topic` class and define their own specific attributes.

```
@Entity @Table(name = "post")
public class Post extends Topic {

    private String content;

    //Getters and setters omitted for brevity
}

@Entity @Table(name = "announcement")
public class Announcement extends Topic {

    @Temporal(TemporalType.TIMESTAMP)
    private Date validUntil;

    //Getters and setters omitted for brevity
}
```

The `TopicStatistics` is at the bottom of this Domain Model as it is only needed for monitoring purposes, without being directly associated with the main business logic. Because statistics are needed for both `Post` and `Announcement` entities, the `TopicStatistics` defines a `Topic` entity association.

```
@Entity @Table(name = "topic_statistics")
public class TopicStatistics {

    @Id @GeneratedValue
    private Long id;

    @OneToOne @JoinColumn(name = "id") @MapsId
    private Topic topic;

    private long views;

    //Getters and setters omitted for brevity
}
```



Another approach would be to add this relationship into the `Topic` class, and the `topic` rows would then reference the `topic_statistics` records. For the sake of demonstrating how entity association polymorphism works, the `TopicStatistics` was chosen to be the child-side.

As natural as this Domain Model may be represented in an object-oriented programming language, transposing it to a relational database is anything but straightforward. Although SQL-99 added support for type inheritance, this feature is seldom implemented. Therefore, relational database systems are relying on tuples and relational algebra for representing and manipulating data. For this reason, mapping inheritance in a relational database is one of the most obvious object-relational impedance mismatches.

Without native support from the database system, inheritance can only be emulated through table relationships. In the *Patterns of Enterprise Application Architecture* book, Martin Fowler defines three ways of mapping inheritance into a relational database:

- [Single Table Inheritance](#)², which uses a single database table to represent all classes in a given inheritance hierarchy.
- [Class Table Inheritance](#)³, which maps each class to a table, and the inheritance association is resolved through table joins.
- [Concrete Table Inheritance](#)⁴, where each table defines all fields that are either defined in the subclass or inherited from a superclass.

The JPA specification defines all these three inheritance mapping models through the following strategies:

- `InheritanceType.SINGLE_TABLE`
- `InheritanceType.JOINED`
- `InheritanceType.TABLE_PER_CLASS`.

JPA also covers the case when inheritance is only available in the Domain Model, without being mirrored into the database (e.g. `@MappedSuperclass`).

Whenever the data access layer implements a functionality without support from the underlying database system, care must be taken to ensure that application performance is not compromised. This chapter aims to analyze what trade-offs are required for employing inheritance as well as its impact on application performance.

²<http://martinfowler.com/eaCatalog/singleTableInheritance.html>

³<http://martinfowler.com/eaCatalog/classTableInheritance.html>

⁴<http://martinfowler.com/eaCatalog/concreteTableInheritance.html>

11.1 Single table

The single table inheritance is the default JPA strategy, funneling a whole inheritance Domain Model hierarchy into a single database table.

To employ this strategy, the `Topic` entity class must be mapped with one of the following annotations:

- `@Inheritance` (being the default inheritance model, it is not mandatory to supply the strategy when using single table inheritance).
- `@Inheritance(strategy = InheritanceType.SINGLE_TABLE)`.

The `Post` and the `Announcement` entities do not need any extra mapping (the Java inheritance semantics being sufficient).

Preserving the same layout as depicted in the Domain Model class diagram, the table relationships associated with this inheritance strategy look like this:

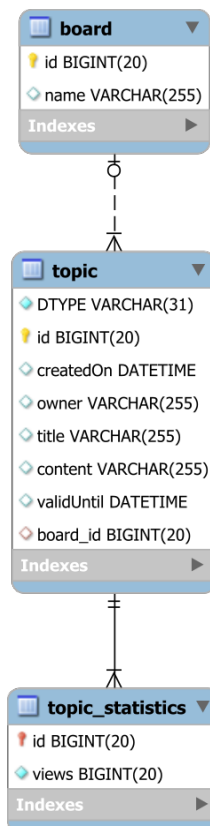


Figure 11.2: Single table

The `topic` table contains columns associated with the `Topic` base class as well as columns related to attributes from `Post` and `Announcement` entities.

In the following example, one `Post` and one `Announcement` entities are going to be persisted along with their associated `@OneToOne TopicStatistics` relations.

```
Post post = new Post();
post.setOwner("John Doe");
post.setTitle("Inheritance");
post.setContent("Best practices");
post.setBoard(board);

entityManager.persist(post);

Announcement announcement = new Announcement();
announcement.setOwner("John Doe");
announcement.setTitle("Release x.y.z.Final");
announcement.setValidUntil(Timestamp.valueOf(LocalDate.now().plusMonths(1)));
announcement.setBoard(board);

entityManager.persist(announcement);

TopicStatistics postStatistics = new TopicStatistics(post);
postStatistics.incrementViews();
entityManager.persist(postStatistics);

TopicStatistics announcementStatistics = new TopicStatistics(announcement);
announcementStatistics.incrementViews();
entityManager.persist(announcementStatistics);
```

Both the `Post` and the `Announcement` entities are saved in the `topic` table whose primary key is shared with the `topic_statistics` table.

```
INSERT INTO topic (board_id, createdOn, owner, title, content, DTYPE, id)
VALUES (1, '2016-01-17 09:22:22.11', 'John Doe', 'Inheritance',
       'Best practices', 'Post', 2)

INSERT INTO topic (board_id, createdOn, owner, title, validUntil, DTYPE, id)
VALUES (1, '2016-01-17 09:22:22.11', 'John Doe', 'Release x.y.z.Final',
       '2016-02-17 09:22:22.114', 'Announcement', 3)

INSERT INTO topic_statistics (views, id) VALUES (1, 2)

INSERT INTO topic_statistics (views, id) VALUES (1, 3)
```

One advantage of using inheritance in the Domain Model is the support for polymorphic queries. When the application developer issues a select query against the `Topic` entity:

```
List<Topic> topics = entityManager.createQuery(
    "select t from Topic t where t.board.id = :boardId", Topic.class)
    .setParameter("boardId", 1L)
    .getResultList();
```

Hibernate goes to the `topic` table, and, after fetching the result set, it maps every row to its associated subclass instance (e.g. `Post` or `Announcement`) by analyzing the discriminator column (e.g. `DTYPE`) value.

```
SELECT t.id AS id2_1_, t.board_id AS board_id8_1_, t.createdOn AS created03_1_,
       t.owner AS owner4_1_, t.title AS title5_1_, t.content AS content6_1_,
       t.validUntil AS validUnt7_1_, t.DTYPE AS DTYPE1_1_
FROM topic t
WHERE t.board_id = 1
```

Domain Model inheritance allows base class entity associations to be automatically resolved upon being retrieved. When loading a `TopicStatistics` along with its `Topic` relation:

```
TopicStatistics statistics = entityManager.createQuery(
    "select s from TopicStatistics s join fetch s.topic t where t.id = :topicId"
    , TopicStatistics.class)
    .setParameter("topicId", topicId)
    .getSingleResult();
```

Hibernate joins the `topic_statistics` and the `topic` tables so that it can create a `TopicStatistics` entity with an actual `Post` or `Announcement` attribute object reference.

```
SELECT
    ts.id AS id1_2_0_, t.id AS id2_1_1_, ts.views AS views2_2_0_,
    t.board_id AS board_id8_1_1_, t.createdOn AS created03_1_1_,
    t.owner AS owner4_1_1_, t.title AS title5_1_1_, t.content AS content6_1_1_,
    t.validUntil AS validUnt7_1_1_, t.DTYPE AS DTYPE1_1_1_
FROM topic_statistics ts
INNER JOIN topic t ON ts.id = t.id
WHERE t.id = 2
```

Even if not practical in this particular example, `@OneToMany` associations are also possible.

The `Board` entity can map a bidirectional `@OneToMany` relationship as follows:

```
@OneToMany(mappedBy = "board")
private List<Topic> topics = new ArrayList<>();
```

Fetching the collection lazily generates a separate select statement, identical to the aforementioned `Topic` entity query. When fetching the collection eagerly, Hibernate requires a single table join.

```
Board board = entityManager.createQuery(
    "select b from Board b join fetch b.topics where b.id = :id", Board.class)
    .setParameter("id", id)
    .getSingleResult();
```

```
SELECT b.id AS id1_0_0_, t.id AS id2_1_1_, b.name AS name2_0_0_,
       t.board_id AS board_id8_1_1_, t.createdOn AS created03_1_1_,
       t.owner AS owner4_1_1_, t.title AS title5_1_1_, t.content AS content6_1_1_,
       t.validUntil AS validUnt7_1_1_, t.DTYPE AS DTYPE1_1_1_,
       t.board_id AS board_id8_1_0_, t.id AS id2_1_0_
FROM   board b
INNER JOIN topic t ON b.id = t.board_id
WHERE  b.id = 1
```

Performance and data integrity considerations

Since only one table is used for storing entities, both reads and writes are fast. Even when using a `@ManyToOne` or a `@OneToOne` base class association, Hibernate needs a single join between parent and child tables. The `@OneToMany` base class entity relationship is also efficient since it either generates a secondary select or a single table join.

Because all subclass attributes are collocated in a single table, `NOT NULL` constraints are not allowed for columns belonging to subclasses. Being automatically inherited by all subclasses, the base class attributes may be non-nullable. From a data integrity perspective, this limitation defeats the purpose of Consistency (guaranteed by the ACID properties).

Nevertheless, the data integrity rules can be enforced through database trigger procedures or `CHECK` constraints (a column non-nullability is accounted based on the class discriminator value). Another approach is to move the check into the data access layer. Bean Validation can validate `@NotNull` attributes at runtime. JPA also defines callback methods (e.g. `@PrePersist`, `@PreUpdate`) as well as entity listeners (e.g. `@EntityListeners`) which can throw an exception when a non-null constraint is violated.

11.1.1 Data integrity constraints

The SQL standard defines the `CHECK` constraint which can be used to apply a row-level verification for each table record that is inserted. Depending on the underlying database, the `CHECK` constraint can be enforced (e.g. Oracle, SQL Server, PostgreSQL, [MySQL 8.0.16⁵](https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html)) or ignored (e.g. [MySQL prior to 8.0.16⁶](https://dev.mysql.com/doc/refman/5.7/en/create-table.html)).

For the aforementioned database tables, the `content` column must never be null if the underlying record is a `Post`, and the `validUntil` column should not be null if the database row represents an `Announcement` entity. Luckily, the default `DTYPE` column specifies the entity type associated with each particular table row.

To ensure the aforementioned data integrity rules, the following `CHECK` constraints needs to be added:

```
ALTER TABLE topic ADD CONSTRAINT post_content_check CHECK (
    CASE WHEN DTYPE = 'Post'
        THEN CASE WHEN content IS NOT NULL THEN 1 ELSE 0 END
        ELSE 1
    END = 1
)

ALTER TABLE topic ADD CONSTRAINT announcement_validUntil_check CHECK (
    CASE WHEN DTYPE = 'Announcement'
        THEN CASE WHEN validUntil IS NOT NULL THEN 1 ELSE 0 END
        ELSE 1
    END = 1
)
```

With these `CHECK` constraints in place, when trying to insert a `Post` entity without a `content`:

```
entityManager.persist(new Post());
```

PostgreSQL generates the following error message:

```
INSERT INTO topic (board_id, createdOn, owner, title, content, DTYPE, id)
VALUES ((NULL(BIGINT), '2016-07-15 13:45:16.705', NULL(VARCHAR), NULL(VARCHAR),
        NULL(VARCHAR), 'Post', 4)

-- SQL Error: 0, SQLState: 23514
-- new row for relation "topic" violates check constraint "post_content_check"
```

⁵<https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html>

⁶<https://dev.mysql.com/doc/refman/5.7/en/create-table.html>

For MySQL versions older than 8.0.16, the same outcome can be achieved with database triggers. For instance, you could add the following `BEFORE INSERT` triggers:

```
CREATE
TRIGGER post_content_check BEFORE INSERT
ON topic
FOR EACH ROW
BEGIN
    IF NEW.DTYPE = 'Post'
    THEN
        IF NEW.content IS NULL
        THEN
            signal sqlstate '45000'
            set message_text = 'Post content cannot be NULL';
        END IF;
    END IF;
END;

CREATE
TRIGGER announcement_validUntil_check BEFORE INSERT
ON topic
FOR EACH ROW
BEGIN
    IF NEW.DTYPE = 'Announcement'
    THEN
        IF NEW.validUntil IS NULL
        THEN
            signal sqlstate '45000'
            set message_text = 'Announcement validUntil cannot be NULL';
        END IF;
    END IF;
END;
```

When running the previous `Post` insert, MySQL generates the following output:

```
INSERT INTO topic (board_id, createdOn, owner, title, content, DTYPE, id)
VALUES ((NULL(BIGINT), '2016-07-15 13:50:51.989', NULL(VARCHAR), NULL(VARCHAR),
        NULL(VARCHAR), 'Post', 4)

-- SQL Error: 1644, SQLState: 45000
-- Post content cannot be NULL
```

You also need to add `BEFORE UPDATE` triggers, so that you can cover modifications for the `post` and `announcement` tables, which could compromise data integrity if the subclass-specific columns were set to `null`.

```
CREATE
TRIGGER post_content_update_check BEFORE UPDATE
ON topic
FOR EACH ROW
BEGIN
    IF NEW.DTYPE = 'Post'
    THEN
        IF NEW.content IS NULL
        THEN
            signal sqlstate '45000'
            set message_text = 'Post content cannot be NULL';
        END IF;
    END IF;
END;

CREATE
TRIGGER announcement_validUntil_update_check BEFORE UPDATE
ON topic
FOR EACH ROW
BEGIN
    IF NEW.DTYPE = 'Announcement'
    THEN
        IF NEW.validUntil IS NULL
        THEN
            signal sqlstate '45000'
            set message_text = 'Announcement validUntil cannot be NULL';
        END IF;
    END IF;
END;
```

Now, if we try to update an `Announcement` entity and set the `validUntil` column to `null`, MySQL will fail with the following error message:

```
UPDATE topic
SET board_id = 1, createdOn = '2016-07-15 13:50:51.989', OWNER = 'John Doe',
    title = 'John Doe, Release x.y.z.Final', validUntil = NULL(TIMESTAMP)
WHERE id = 3

-- SQL Error: 1644, SQLState: 45000
-- Announcement validUntil cannot be NULL
```

Although a little bit verbose, the `CHECK` and `TRIGGER` constraints are very useful to ensure data integrity when using single table inheritance.

11.2 Join table

The join table inheritance resembles the Domain Model class diagram since each class is mapped to an individual table. The subclass tables have a foreign key column referencing the base class table primary key.

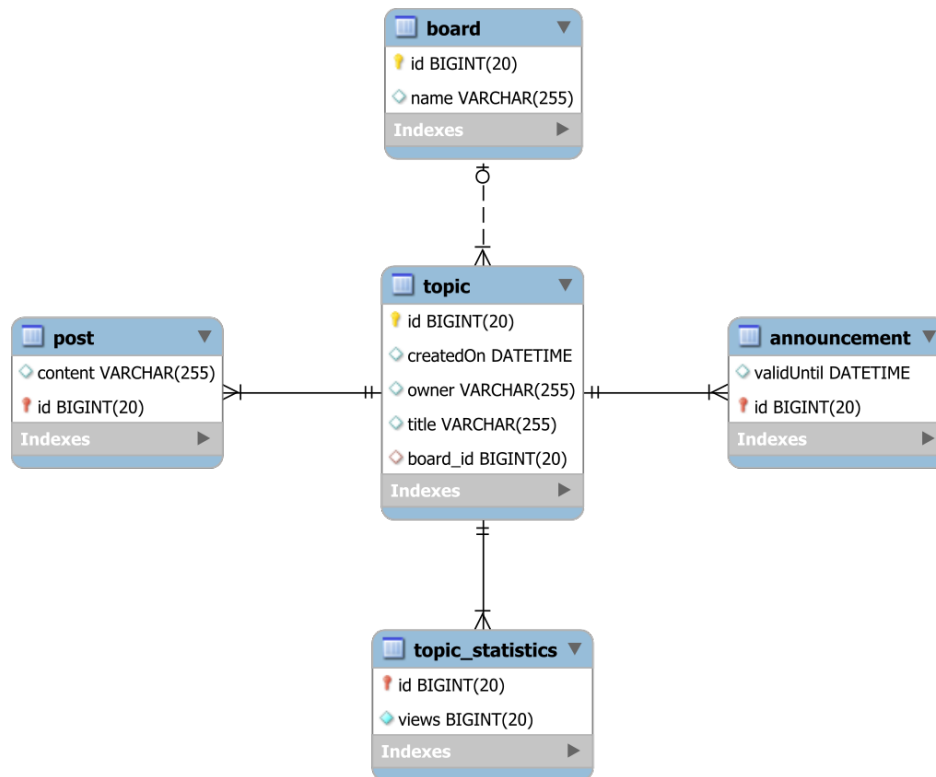


Figure 11.3: Join table

To use this inheritance strategy, the `Topic` entity must be annotated with:

```
@Inheritance(strategy = InheritanceType.JOINED)
```



The `Post` and the `Announcement` entities can use a `@PrimaryKeyJoinColumn` mapping to define the base class foreign key column.

By default, the subclass table primary key column is used as a foreign key as well.

When persisting the same entities defined in the single table section, Hibernate generates the following SQL statements:

```
INSERT INTO topic (board_id, createdOn, owner, title, id)
VALUES (1, '2016-01-17 09:27:10.694', 'John Doe', 'Inheritance', 2)

INSERT INTO post (content, id) VALUES ('Best practices', 2)

INSERT INTO topic (board_id, createdOn, owner, title, id)
VALUES (1, '2016-01-17 09:27:10.694', 'John Doe', 'Release x.y.z.Final', 3)

INSERT INTO announcement (validUntil, id) VALUES ('2016-02-17 09:27:10.698', 3)

INSERT INTO topic_statistics (views, id) VALUES (1, 2)

INSERT INTO topic_statistics (views, id) VALUES (1, 3)
```

The base class information goes into the `topic` table while the subclass content goes in the `post` or the `announcement` tables. When fetching all `Topic` entities associated with a specific `Board`:

```
List<Topic> topics = entityManager.createQuery(
    "select t from Topic t where t.board.id = :boardId", Topic.class)
    .setParameter("boardId", 1L)
    .getResultList();
```

Hibernate must join the base class with each individual subclass table.

```
SELECT
    t.id AS id1_3_,
    t.board_id AS board_id5_3_,
    t.createdOn AS created02_3_,
    t.owner AS owner3_3_,
    t.title AS title4_3_,
    t1_.content AS content1_2_,
    t2_.validUntil AS validUnt1_0_,
    CASE WHEN t1_.id IS NOT NULL THEN 1
         WHEN t2_.id IS NOT NULL THEN 2
         WHEN t.id IS NOT NULL THEN 0
    END AS clazz_
FROM topic t
LEFT OUTER JOIN post t1_ ON t.id = t1_.id
LEFT OUTER JOIN announcement t2_ ON t.id = t2_.id
WHERE t.board_id = 1
```

When loading a `TopicStatistics` entity along with its `Topic` association:

```
TopicStatistics statistics = entityManager.createQuery(
    "select s from TopicStatistics s join fetch s.topic t where t.id = :topicId"
    , TopicStatistics.class)
.setParameter("topicId", topicId)
.getSingleResult();
```

Hibernate must join four tables to construct the desired result set:

```
SELECT
    ts.id AS id1_4_0_,
    t.id AS id1_3_1_,
    ts.views AS views2_4_0_,
    t.board_id AS board_id5_3_1_,
    t.createdOn AS created02_3_1_,
    t.owner AS owner3_3_1_,
    t.title AS title4_3_1_,
    t1_.content AS content1_2_1_,
    t2_.validUntil AS validUnt1_0_1_,
    CASE WHEN t1_.id IS NOT NULL THEN 1
         WHEN t2_.id IS NOT NULL THEN 2
         WHEN t.id IS NOT NULL THEN 0
    END AS clazz_1_
FROM topic_statistics ts
INNER JOIN topic t ON ts.id = t.id
LEFT OUTER JOIN post t1_ ON t.id = t1_.id
LEFT OUTER JOIN announcement t2_ ON t.id = t2_.id
WHERE t.id = 2
```

Considering that the `Board` entity defines a `@OneToMany` `Topic` association:

```
@OneToMany(mappedBy = "board")
private List<Topic> topics = new ArrayList<>();
```

Fetching the collection lazily generates a separate select statement, identical to the previous `Topic` entity query.

When fetching the collection eagerly:

```
Board board = entityManager.createQuery(
    "select b from Board b join fetch b.topics where b.id = :id", Board.class)
    .setParameter("id", id)
    .getSingleResult();
```

Hibernate requires three joins to fetch all topic-related information.

SELECT

```
b.id AS id1_1_0_, t.id AS id1_3_1_, b.name AS name2_1_0_,
t.board_id AS board_id5_3_1_, t.createdOn AS created02_3_1_,
t.owner AS owner3_3_1_, t.title AS title4_3_1_,
t1_.content AS content1_2_1_, t2_.validUntil AS validUnt1_0_1_,
CASE WHEN t1_.id IS NOT NULL THEN 1
      WHEN t2_.id IS NOT NULL THEN 2
      WHEN t.id IS NOT NULL THEN 0
END AS clazz_1_,
t.board_id AS board_id5_3_0__, t.id AS id1_3_0__
FROM board b
INNER JOIN topic t ON b.id = t.board_id
LEFT OUTER JOIN post t1_ ON t.id = t1_.id
LEFT OUTER JOIN announcement t2_ ON t.id = t2_.id
WHERE b.id = 1
```

Performance considerations

Unlike single table inheritance, the joined table strategy allows nullable subclass attribute columns.

When writing data, Hibernate requires two insert statements for each subclass entity, so there's a performance impact compared to single table inheritance. The index memory footprint also increases because instead of a single table primary key, the database must index the base class and all subclasses primary keys.

When reading data, polymorphic queries require joining the base class with all subclass tables, so, if there are n subclasses, Hibernate will need $N+1$ joins. The more joins, the more difficult it is for the database to calculate the most efficient execution plan.

11.3 Table-per-class

The table-per-class inheritance model has a table layout similar to the joined table strategy, but, instead of storing base class columns in the `topic` table, each subclass table also stores columns from the `topic` table. There is no foreign key between the `topic` and the `post` or `announcement` subclass tables, and there is no foreign key in the `topic_statistics` table either.

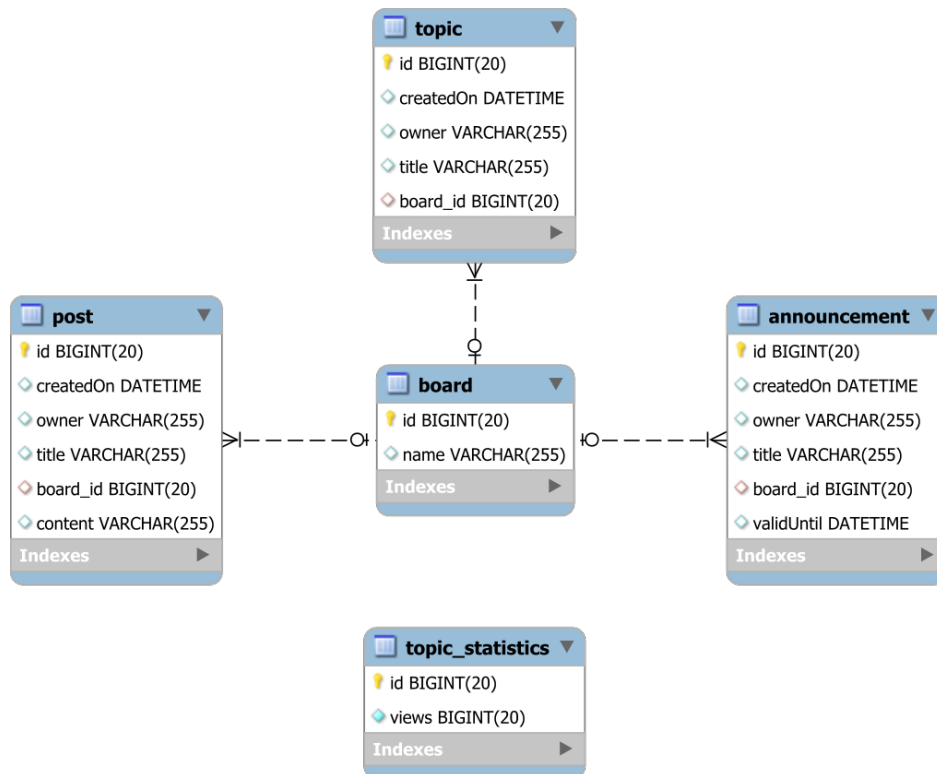


Figure 11.4: Table-per-class

To use this inheritance strategy, the `Topic` must be annotated with `@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)`.

Inserting the `Post` and the `Announcement` entities defined in the single table inheritance section generates the following SQL statements:

```
INSERT INTO post (board_id, createdOn, owner, title, content, id)
VALUES (1, '2016-01-17 09:31:12.018', 'John Doe', 'Inheritance',
       'Best practices', 2)
```

```
INSERT INTO announcement (board_id, createdOn, owner, title, validUntil, id)
VALUES (1, '2016-01-17 09:31:12.018', 'John Doe', 'Release x.y.z.Final',
       '2016-02-17 09:31:12.023', 3)
```

```
INSERT INTO topic_statistics (views, id) VALUES (1, 2)
```

```
INSERT INTO topic_statistics (views, id) VALUES (1, 3)
```

Unlike the joined table inheritance, each persisted subclass entity requires a single insert statement.

When fetching all `Topic` entities associated with a specific `Board`:

```
List<Topic> topics = entityManager.createQuery(
    "select t from Topic t where t.board.id = :boardId", Topic.class)
    .setParameter("boardId", 1L)
    .getResultList();
```

Hibernate uses `UNION ALL` to fetch rows from the base class and every subclass table in this particular inheritance tree.

```
SELECT
    t.id AS id1_3_,
    t.board_id AS board_id5_3_,
    t.createdOn AS created02_3_,
    t.owner AS owner3_3_,
    t.title AS title4_3_,
    t.content AS content1_2_,
    t.validUntil AS validUnt1_0_,
    t.clazz_ AS clazz_
FROM (
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 0 AS clazz_
    FROM topic
    UNION ALL
    SELECT id, createdOn, owner, title, board_id, content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 1 AS clazz_
    FROM post
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content, validUntil, 2 AS clazz_
    FROM announcement ) t
WHERE t.board_id = 1
```

When loading a `TopicStatistics` while also fetching its associated `Topic` relation, Hibernate must use `UNION ALL` for the inheritance tables to construct the desired result set:

```
TopicStatistics statistics = entityManager.createQuery(
    "select s from TopicStatistics s join fetch s.topic t where t.id = :topicId"
    , TopicStatistics.class)
.setParameter("topicId", topicId)
.getSingleResult();
```

SELECT

```
ts.id AS id1_4_0_, t.id AS id1_3_1_, ts.views AS views2_4_0_,
t.board_id AS board_id5_3_1_, t.createdOn AS created02_3_1_,
t.owner AS owner3_3_1_, t.title AS title4_3_1_, t.content AS content1_2_1_,
t.validUntil AS validUnt1_0_1_, t.clazz_ AS clazz_1_
FROM topic_statistics ts
INNER JOIN (
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 0 AS clazz_
    FROM topic
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 1 AS clazz_
    FROM post
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        validUntil, 2 AS clazz_
    FROM announcement
) t ON ts.id = t.id
WHERE t.id = 2
```



The identity generator is not allowed with this strategy because rows belonging to different subclasses would share the same identifier, therefore conflicting in polymorphic `@ManyToOne` or `@OneToOne` associations.

Considering that the `Board` entity defines a `@OneToMany` `Topic` association:

```
@OneToMany(mappedBy = "board")
private List<Topic> topics = new ArrayList<>();
```

Fetching the collection lazily generates a separate select statement, identical to the previous `Topic` entity query.

When fetching the `topics` collection eagerly:

```
Board board = entityManager.createQuery(
    "select b from Board b join fetch b.topics where b.id = :id", Board.class)
    .setParameter("id", id)
    .getSingleResult();
```

Hibernate requires a join with the result of unifying all three topic-related tables.

```
SELECT
    b.id AS id1_1_0_, t1.id AS id1_3_1_, b.name AS name2_1_0_,
    t1.board_id AS board_id5_3_1_, t1.createdOn AS created02_3_1_,
    t1.owner AS owner3_3_1_, t1.title AS title4_3_1_,
    t1.content AS content1_2_1_, t1.validUntil AS validUnt1_0_1_,
    t1.clazz_ AS clazz_1_, t1.board_id AS board_id5_3_0_, t1.id AS id1_3_0_
FROM board b
INNER JOIN (
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 0 AS clazz_
    FROM topic
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 1 AS clazz_
    FROM post
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        validUntil, 2 AS clazz_
    FROM announcement
) t1 ON b.id = t1.board_id
WHERE b.id = 1
```

Performance considerations

While write operations are faster than for the joined table strategy, the read operations are only efficient when querying against the actual subclass entities. Polymorphic queries can have a considerable performance impact because Hibernate must select all subclass tables and use `UNION ALL` to build the whole inheritance tree result set. As a rule of thumb, the more subclass tables, the less efficient the polymorphic queries get.

11.4 Mapped superclass

If the `Topic` class is not required to be a stand-alone entity, it will be more practical to leave inheritance out of the database. This way, the `Topic` can be made abstract and marked with the `@MappedSuperclass` annotation so that JPA can acknowledge the inheritance model on the entity-side only.

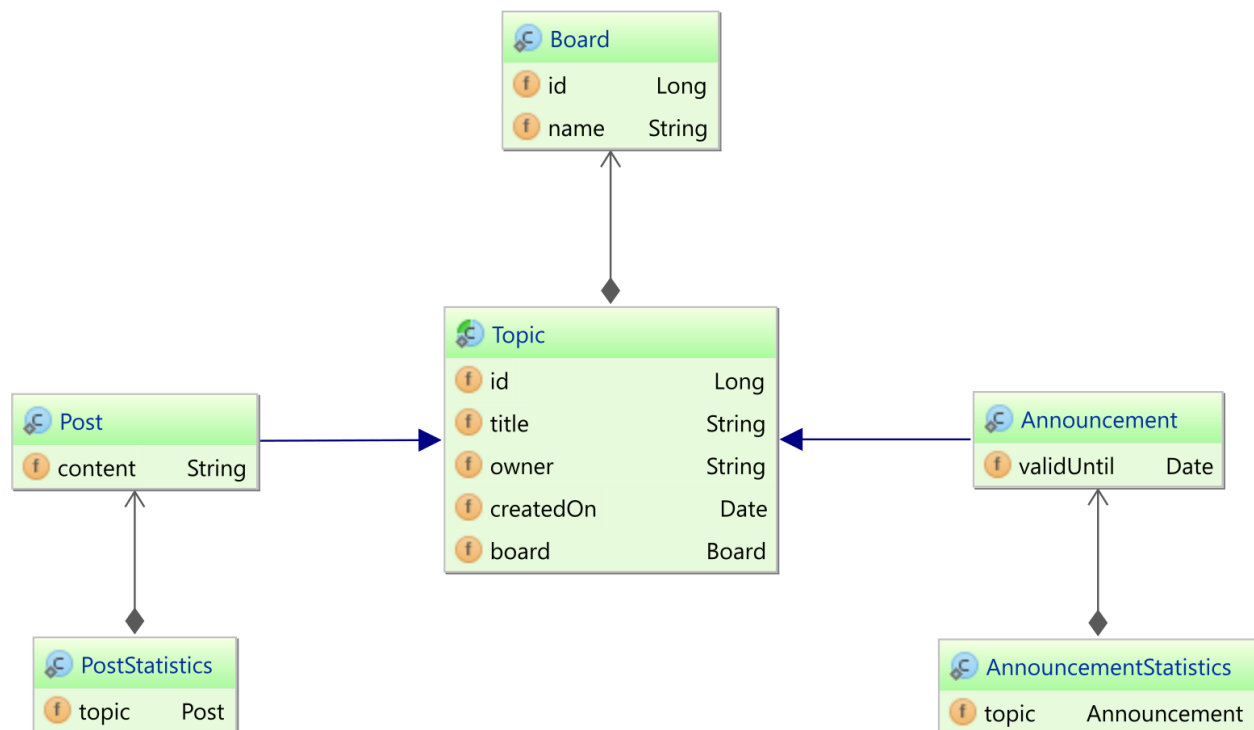


Figure 11.5: Mapped superclass class diagram

Having a single `TopicStatistics` entity with a `@OneToOne` `Topic` association is no longer possible because the `Topic` class is not an entity. This way, each `Topic` subclass must define its own statistics associations (e.g. `PostStatistics`, `AnnouncementStatistics`).

The `PostStatistics` and the `AnnouncementStatistics` entities looks as follows:

```
@Entity @Table(name = "post_statistics")
public class PostStatistics
    extends TopicStatistics<Post> {

    @OneToOne
    @JoinColumn(name = "id")
    @MapsId
    private Post topic;

    private PostStatistics() {}

    public PostStatistics(Post topic) {
        this.topic = topic;
    }

    @Override
    public Post getTopic() {
        return topic;
    }
}

@Entity @Table(name = "announcement_statistics")
public class AnnouncementStatistics
    extends TopicStatistics<Announcement> {

    @OneToOne
    @JoinColumn(name = "id")
    @MapsId
    private Announcement topic;

    private AnnouncementStatistics() {}

    public AnnouncementStatistics(Announcement topic) {
        this.topic = topic;
    }

    @Override
    public Announcement getTopic() {
        return topic;
    }
}
```

To retain the inheritance semantics, the base class attributes are going to be merged with the subclass ones, so the associated subclass entity table contains both. This is similar to the table-per-class inheritance strategy, with the distinction that the base class is not mapped to a database table (hence, it cannot be used in polymorphic queries or associations).

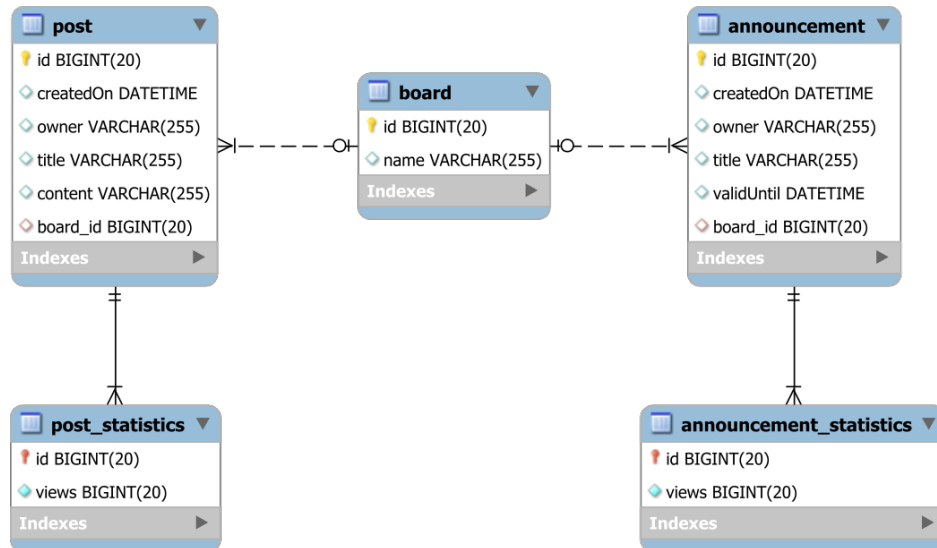


Figure 11.6: Mapped superclass

The `post` and the `announcement` tables feature columns that were inherited from the `Topic` base class. When persisting the same `Post` and `Announcement` entities and saving statistics using the newly defined entity classes:

```
TopicStatistics postStatistics = new PostStatistics(post);
postStatistics.incrementViews();
entityManager.persist(postStatistics);
```

```
TopicStatistics announcementStatistics =
    new AnnouncementStatistics(announcement);
announcementStatistics.incrementViews();
entityManager.persist(announcementStatistics);
```

Hibernate generates the following SQL statements:

```
INSERT INTO post (board_id, createdOn, owner, title, content, id )
VALUES (1, '2016-01-17 09:11:07.525', 'John Doe', 'Inheritance',
       'Best practices', 1)
```

```
INSERT INTO announcement (board_id, createdOn, owner, title, validUntil, id)
VALUES (1, '2016-01-17 09:11:07.525', 'John Doe', 'Release x.y.z.Final',
       '2016-02-17 09:11:07.529', 2)
```

```
INSERT INTO post_statistics (views, id) VALUES (1, 1)
```

```
INSERT INTO announcement_statistics (views, id) VALUES (1, 2)
```

Querying for statistics require specifying the actual `Topic` subclass statistics:

```
PostStatistics statistics = entityManager.createQuery(
    "select s from PostStatistics s join fetch s.topic t where t.id = :postId",
    PostStatistics.class)
.setParameter("postId", postId)
.getSingleResult();
```

This entity query generates a single SQL join statement:

```
SELECT
    s.id AS id1_4_0_,
    p.id AS id1_3_1_,
    s.views AS views2_4_0_,
    p.board_id AS board_id6_3_1_,
    p.createdOn AS created02_3_1_,
    p.owner AS owner3_3_1_,
    p.title AS title4_3_1_,
    p.content AS content5_3_1_
FROM post_statistics s
INNER JOIN post p ON s.id = p.id
WHERE p.id = 1
```

Polymorphic queries against the `Topic` class are not permitted and the `Board` entity cannot define a `@OneToMany Topic` collection either.

Performance considerations

Although polymorphic queries and associations are no longer permitted, the `@MappedSuperclass` yields very efficient read and write operations. Like single and table-per-class inheritance, write operations require a single insert statement and only one table is used for reading data.

Inheritance best practices

All the inheritance mapping models above require trading something in order to accommodate the impedance mismatch between the relational database system and the object-oriented Domain Model.

The default single table inheritance performs the best in terms of reading and writing data, but it forces the application developer to overcome the column nullability limitation. This strategy is useful when the database can provide support for trigger procedures, and the number of subclasses is relatively small.

The join table is worth considering when the number of subclasses is higher, and the data access layer does not require polymorphic queries. When the number of subclass tables is large, polymorphic queries require many joins, and fetching such a result set has a noticeable impact on application performance. This issue can be mitigated by restricting the result set (e.g. pagination), but that only applies to queries and not to `@OneToMany` or `@ManyToMany` associations. On the other hand, polymorphic `@ManyToOne` and `@OneToOne` associations are fine since, in spite of joining multiple tables, the result set can have at most one record only.

Table-per-class is the least effective when it comes to polymorphic queries or associations. If each subclass is stored in a separate database table, the `@MappedSuperclass` Domain Model inheritance might often be a better alternative.



Although a powerful concept, Domain Model inheritance should be used sparingly and only when the benefits supersede trade-offs.

12. Flushing

As explained in the [write-based optimizations](#) section, the Persistence Context acts as a transactional write-behind cache. The `Hibernate Session` is commonly referred to as the first-level cache since every managed entity is stored in a `Map`, and, once an entity is loaded, any successive request serves it from the cache, therefore avoiding a database roundtrip.

However, aside from caching entities, the Persistence Context acts as an entity state transition buffer. Both the `EntityManager` and the `Hibernate Session` expose various entity state management methods:

- The `persist` method takes a transient entity and makes it managed.
- The `merge` method copies the internal state of a detached entity onto a freshly loaded entity instance.
- Hibernate also supports reattaching entity instances (e.g. `update`, `saveOrUpdate` or `lock`) which, unlike `merge`, does not require fetching a new entity reference copy.
- To remove an entity, the `EntityManager` defines the `remove` method, while Hibernate offers a `delete` method.

Like any write-behind cache, the Persistence Context requires flushing in order to synchronize the in-memory persistent state with the underlying database. At flush time, Hibernate can detect if a managed entity has changed since it was loaded and trigger a table row update. This process is called dirty checking, and it greatly simplifies data access layer operations.

So, when using JPA, the application developer can focus on entity state changes, and the Persistence Context takes care of the underlying DML statements. This way, the data access layer logic is expressed using Domain Model state transitions, rather than through insert, update, or delete SQL statements.

This approach is very convenient for several reasons:

- Entity state changes being buffered, the Persistence Context can delay their execution, therefore minimizing the row-level lock interval, associated with every database write operation.
- Being executed at once, the Persistence Context can use JDBC batch updates to avoid executing each statement in a separate database roundtrip.

However, having an intermediate write-behind cache is not without challenges and the Persistence Context can be subject to data inconsistencies. Since efficiency is meaningless if effectiveness is being compromised, this chapter aims to analyze the inner-workings of the flushing mechanism, so the application developer knows how to optimize it without affecting data consistency.

Medii de proiectare și programare

2024-2025

Curs 10

Conținut

- Servicii Web
- REST
- Arhitectura REST
- REST Spring

Servicii Web

- Definiții:

1. Un **serviciu web** este o aplicație/componentă soft disponibilă pe internet și care folosește un sistem standardizat de transmitere a mesajelor în format XML. XML este folosit pentru comunicarea datelor către/de la un serviciu web.
 - Un client apelează un serviciu web trimițând un mesaj în format XML și așteaptă un răspuns în format XML.
 - Comunicarea are loc folosind XML, serviciile web nu sunt dependente de un anumit sistem de operare sau de un anumit limbaj de programare.
2. **Serviciile web** sunt aplicații *self-contained*, modulare și distribuite care pot fi descrise, publicate, localizate și apelate prin rețea pentru a crea produse, procese, etc. Aceste aplicații pot fi locale, distribuite sau pe web. Serviciile web sunt dezvoltate folosind standarde ca TCP/IP, HTTP și XML.
3. **Serviciile web** sunt sisteme de schimbare a mesajelor bazate pe XML care folosesc Internetul pentru interacțiunea dintre aplicații. Aceste sisteme pot fi: programe, obiecte, mesaje, documente, etc.
4. Un **serviciu web** este o colecție de protocoale și standarde folosite pentru transmiterea datelor între aplicații sau sisteme. Aplicații dezvoltate în diferite limbaje de programare și rulând pe diferite platforme (sisteme de operare) pot folosi serviciile web pentru a schimba date în rețea (ex. Internet) într-o manieră similară comunicării între procese pe același calculator. Interoperabilitatea acestora este posibilă datorită folosirii standardelor.

Servicii Web

- Un **serviciu web** este un **serviciu**:
 - Disponibil pe Internet sau într-o rețea privată.
 - Folosește un sistem standardizat de schimbare a mesajelor bazat pe XML.
 - Nu este dependent de un anumit limbaj de programare sau de un anumit sistem de operare.
 - Este self-describing folosind o gramatică XML (DTD, XML Schema).
 - Poate fi descoperit folosind mecanisme simple.
- **Componentele**: structura de baza pentru serviciile web este XML + HTTP. Toate serviciile web standard funcționează folosind componentele:
 - **XML** pentru marcarea informației
 - **SOAP** (Simple Object Access Protocol) pentru transmiterea mesajelor
 - **UDDI** (Universal Description, Discovery and Integration)
 - **WSDL** (Web Services Description Language) pentru a descrie disponibilitatea serviciului

Exemplu WSDL

```
<definitions name = "HelloService"
  targetNamespace = "http://www.examples.com/wsdl/HelloService.wsdl"
  xmlns = "http://schemas.xmlsoap.org/wsdl/"
  xmlns:soap = "http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns = "http://www.examples.com/wsdl/HelloService.wsdl"
  xmlns:xsd = "http://www.w3.org/2001/XMLSchema">

  <message name = "SayHelloRequest">
    <part name = "firstName" type = "xsd:string"/>
  </message>

  <message name = "SayHelloResponse">
    <part name = "greeting" type = "xsd:string"/>
  </message>

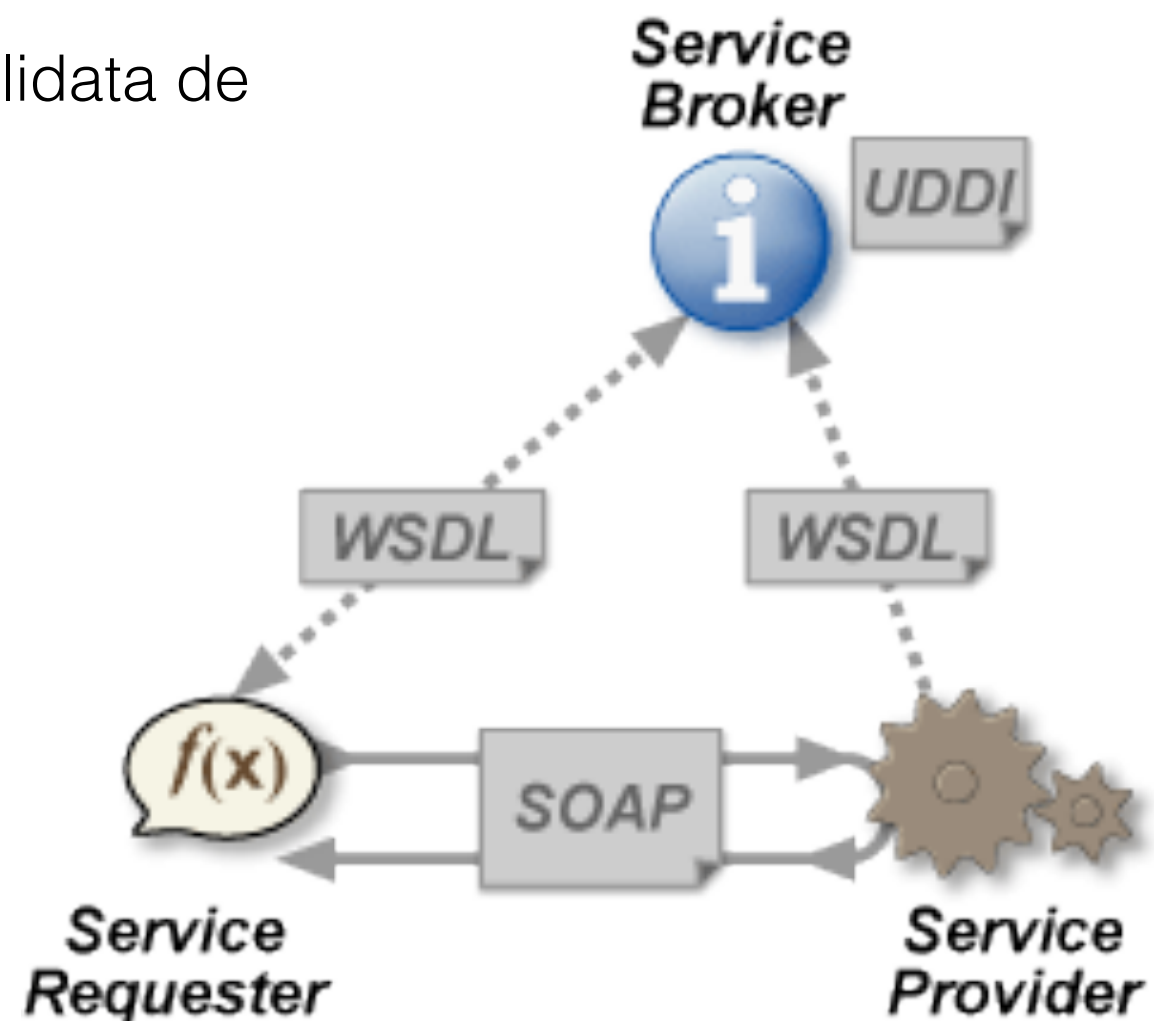
  <portType name = "Hello_PortType">
    <operation name = "sayHello">
      <input message = "tns:SayHelloRequest"/>
      <output message = "tns:SayHelloResponse"/>
    </operation>
  </portType>

  <binding name = "Hello_Binding" type = "tns:Hello_PortType">
    <soap:binding style = "rpc"
      transport = "http://schemas.xmlsoap.org/soap/http"/>
    <operation name = "sayHello">
      <soap:operation soapAction = "sayHello"/>
      <input>
        <soap:body
          encodingStyle = "http://schemas.xmlsoap.org/soap/encoding/"
          namespace = "urn:examples:helloservice"
          use = "encoded"/>
      </input>
      <output>
        <soap:body
          encodingStyle = "http://schemas.xmlsoap.org/soap/encoding/"
          namespace = "urn:examples:helloservice"
          use = "encoded"/>
      </output>
    </operation>
  </binding>

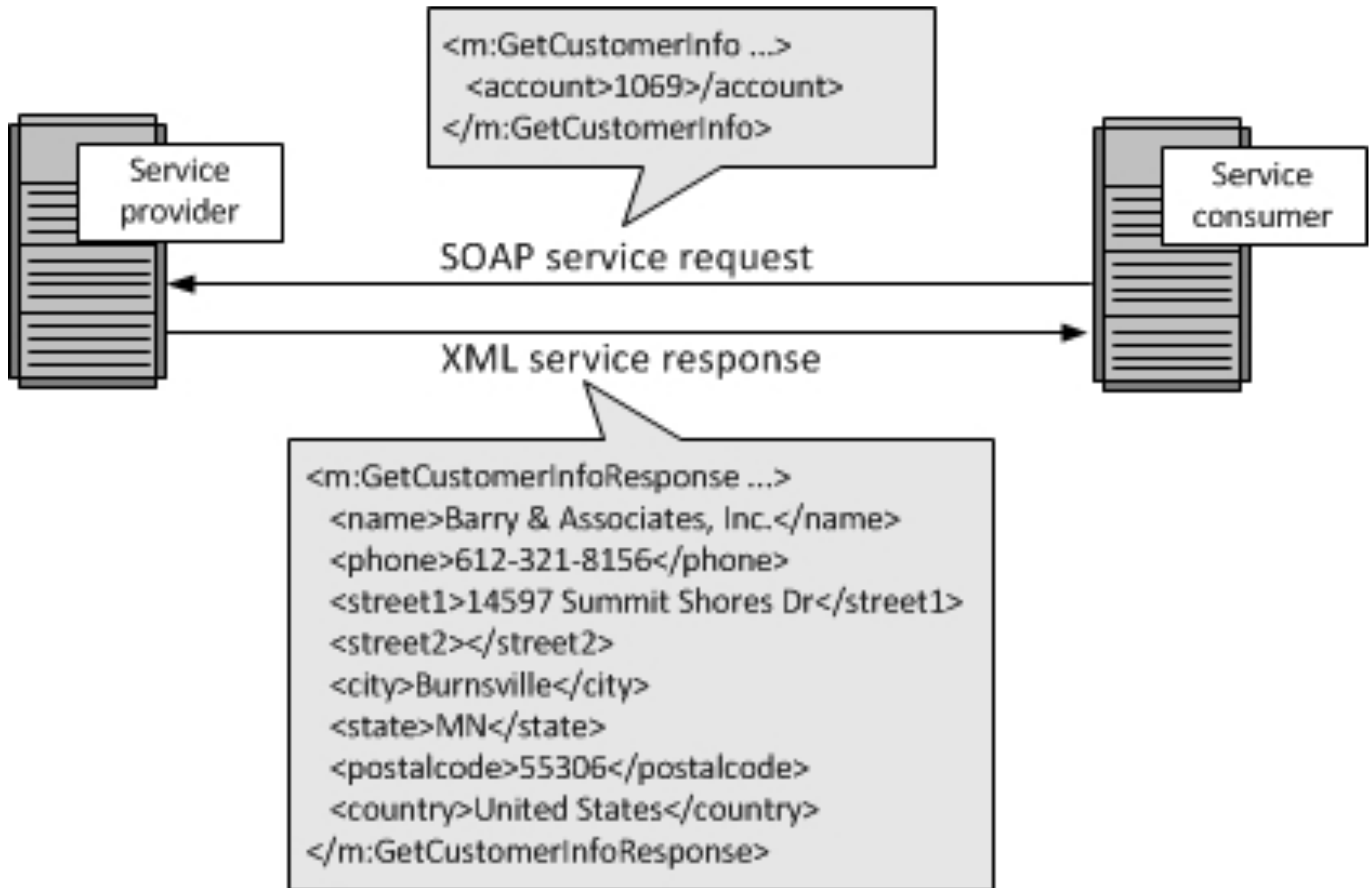
  <service name = "Hello_Service">
    <documentation>WSDL File for HelloService</documentation>
    <port binding = "tns:Hello_Binding" name = "Hello_Port">
      <soap:address
        location = "http://www.examples.com/SayHello/" />
    </port>
  </service>
</definitions>
```

Servicii Web - Arhitectura

- Furnizorul serviciului trimite un fișier WSDL serviciului UDDI.
- Clientul serviciului (consumatorul) contactează UDDI pentru a descoperi cine este furnizorul datelor de care are nevoie, iar apoi contactează furnizorul serviciului folosind protocolul SOAP.
- Furnizorul serviciului validează cererea și trimite datele cerute în format XML folosind protocolul SOAP.
- Structura datelor în format XML ar trebui validată de client folosind un fișier XSD (XML Schema).



Servicii Web - Arhitectura



REST

- REpresentational State Transfer (REST).
- Este un stil arhitectural pentru sisteme distribuite hipermedia.
- A fost propus de Roy T. Fielding în teza sa de doctorat “*Architectural Styles and the Design of Network-based Software Architectures*” (2000).
- Conține 6 constrângeri ce trebuie satisfăcute pentru a fi numit sistem REST veritabil:
 1. Client–server
 2. Fără stare (eng. *stateless*)
 3. Cacheable
 4. Interfață uniformă (eng. *uniform interface*)
 5. Sistem stratificat (eng. *layered system*)
 6. Cod la cerere (eng. *code on demand*) -opțională

REST - Resursa

- Conceptul de bază în REST este *resursa* (eng. *resource*).
- Orice informație care poate fi numită poate fi **resursă**: un document, o imagine, o colecție de alte resurse, un obiect real (ex. persoană, mașină), etc.
- REST folosește *identificatorul resursei* pentru a identifica resursa în momentul interacțiunii între componente.
- Starea unei resurse la un anumit moment de timp este numită **reprezentarea resursei**.
- *Reprezentarea* constă din *date*, *metadata* care descriu datele și *legături hipermedia* care ajută clienții în tranziția la următoarea stare dorită.
- *Formatul datelor dintr-o reprezentare* este numit **tip media** (eng. *media type*).
- Tipul media identifică o specificație care definește modul în care reprezentarea va fi procesată.
- Un API RESTful veritabil seamănă cu hypertext. Fiecare informație ce poate fi obținută are o *adresă explicită* (prin legături și attribute id) sau *implicită* (obținută din definiția tipului media și structura reprezentării).
- *Reprezentările resurselor trebuie să se descrie pe ele* (eng. *self-descriptive*): clientul nu trebuie să știe că resursa este persoană sau mașină. Datele trebuie să poată fi procesate pe baza tipului media asociat resursei.

REST - metode asupra resursei

- Un lucru important asociat cu REST sunt metodele/operațiile ce pot fi aplicate resursei pentru a efectua tranziția dorită.
- Toată informația necesară modificării stării resursei face parte din API-ul cererii pentru acea resursă (inclusiv operațiile/metodele și modul în care vor afecta reprezentarea resursei).
- Este recomandat ca rezultatele interogărilor să fie reprezentate folosind o listă cu legături conținând informații sumare, și nu tablouri conținând toate informațiile corespunzătoare resursei, deoarece interogarea nu este un substitut pentru identificarea resurselor.
- Adesea, metodele/operațiile asupra resursei sunt confundate cu metodele HTTP GET/PUT/POST/DELETE.

REST și HTTP

- REST și HTTP nu reprezintă același lucru.
- Dacă un sistem satisface constrângerile REST, se poate spune ca sistemul este RESTful (modelul de maturitate Richardson, 2009).
- În stilul arhitectural REST, datele și funcționalitățile sunt considerate resurse ce pot fi accesate folosind *Uniform Resource Identifiers* (URIs).
- Resursele sunt prelucrate (adăugate, șterse, modificate, etc) folosind un set simplu, bine definit de operații.
- Clienții și serverele interschimbă (schimbă între ele) reprezentări ale resurselor folosind o interfață standardizată și un protocol standardizat (adesea HTTP).
- Resursele sunt decuplate de reprezentarea lor. Conținutul lor poate fi accesat folosind diferite formate: XML, text, PDF, JSON, HTML, etc.
- Metadata despre resurse este folosită pentru detecția erorilor, controlul caching-ului, negocierea celui mai convenabil format pentru reprezentare, autentificare și controlul accesului.
- Fiecare interacțiune este stateless.
- Aceste principii fac aplicațiile RESTful simple, lightweight și rapide.

Constrângerile arhitecturale REST

- REST este un stil arhitectural ce permite proiectarea unor aplicații slab cuplate (loosely coupled) folosind HTTP.
- Este adesea folosit pentru dezvoltarea serviciilor web.
- REST nu impune nici o regulă despre cum ar trebui implementat la nivelele inferioare, doar impune constrângeri la nivelul de proiectare și lasă implementarea dezvoltatorului.
- REST definește 6 constrângeri arhitecturale a căror satisfacere fac orice serviciu web un API RESTful veritabil:
 1. *Uniform interface*
 2. *Client-server*
 3. *Stateless*
 4. *Cacheable*
 5. *Layered system*
 6. *Code on demand* (opțional)

REST - *Uniform interface*

- Aplicând principiul generalității interfeței componentelor care interacționează, arhitectura întregului sistem este simplificată și interacțiunea între componente este îmbunătățită.
- Pentru a obține o interfață uniformă, se folosesc mai multe constrângeri arhitecturale pentru a ghida execuția componentelor.
- REST este definit de 4 constrângeri ale interfeței:
 - *Identificarea resurselor*
 - *Manipularea resurselor folosind reprezentările*
 - *Mesaje ce se descriu pe sine (self-descriptive)*
 - *Hypermedia ca și motor al stării aplicației (eng. hypermedia as the engine of application state - HATEOAS).*
- Dezvoltatorii trebuie **să decidă interfața** pentru resursele sistemului făcute publice clienților și **să folosească întotdeauna această interfață**.
- O resursă din sistem trebuie să aibă un singur URI, și trebuie să ofere o modalitate de a obține date adiționale sau asociate.
- O resursă nu ar trebui să fie foarte mare și ar trebui să conțină legături către alte informații adiționale (URI relativ) care permit obținerea acelor informații.
- Reprezentările resurselor trebuie să respecte anumite recomandări (convenții de nume, formatul legăturilor, formatul datelor -xml și/sau json).
- Toate resursele ar trebui să fie accesibile folosind o abordare comună (ex. folosind HTTP GET) și, în mod similar, ar trebui să poată fi modificate folosind o abordare consistentă.

REST - *Client-server*

- Aplicațiile client și server trebuie să poată evolua independent fără dependențe între ele.
- Clientul ar trebui să știe doar de URI-ul resursei.
- Prin separarea interfeței clientului de modul de păstrare a resurselor, se îmbunătățește portabilitatea interfeței cu utilizatorul pe platforme diferite și se îmbunătățește scalabilitatea prin simplificarea componentelor de pe server.
- “*Servers and clients may also be replaced and developed independently, as long as the interface between them is not altered.*”

REST - *Stateless*

- Stilul REST a fost inspirat din HTTP.
- Toate interacțiunile client-server trebuie să fie **stateless**.
- Serverul nu va păstra informații despre ultima cerere HTTP de la client.
- Fiecare cerere va fi tratată ca și cum ar fi o cerere nouă (fără sesiune, fără istoric).
- Fiecare cerere trimisă de la client la server trebuie să conțină toate informațiile necesare pentru a înțelege cererea și nu poate folosi informații legate de context păstrate pe server.
- Starea sesiunii este păstrată în întregime la client.
- Dacă o aplicație trebuie să păstreze starea pentru client (clientul se autentifică iar apoi efectuează operații), fiecare cerere de la client trebuie să conțină toate datele necesare pentru îndeplinirea cererii (inclusiv detalii legate de autentificare și autorizare).
- *“No client context shall be stored on the server between requests. Client is responsible for managing the state of application.”*

Constrângeri REST

- *Cacheable*: constrângerea cere ca datele dintr-un răspuns la o cerere să fie **implicit sau explicit marcate ca cacheable sau non-cacheable**. Dacă răspunsul este cacheable, atunci cache-ul de pe client are permisiunea de a refolosi datele ulterior (echivalând cu obținerea lui folosind cereri către server).
- Caching îmbunătățește performanța clientului și îmbunătățește scalabilitatea serverului deoarece numărul de cereri se reduce.
- Caching trebuie aplicat tuturor resurselor ce se declară cacheable. Caching poate fi implementat atât pe server cât și pe client.
- “*Well-managed caching partially or completely eliminates some client–server interactions, further improving scalability and performance.*”
- *Sistem stratificat*:
 - REST permite folosirea unei arhitecturi stratificate a sistemului: API public (URIs) este pe serverul A, datele sunt păstrate pe server B, iar cererile de autentificare se fac pe serverul C.
 - Un client nu știe dacă este conectat direct cu serverul destinație sau cu un intermediar.
- Arhitectura stratificată permite ca sistemul final să fie compus din nivele ierarhice care permit unui nivel comunicarea doar cu nivelul inferior (superior)

Constrângeri REST

- *Code on demand*
- Constrângerea este **opțională**. Adesea, serverele vor trimite reprezentarea statică a resurselor în format XML sau JSON.
- *Serverul poate trimite și cod executabil, pentru a suporta o parte a aplicației* (ex. cod pentru afișarea unei componente grafice)
- REST permite extinderea funcționalității clienților prin descărcarea și executarea codului (sub forma unor applet-uri sau a unor scripturi).
- *Poate simplifica clienții prin reducerea numărului de funcționalități ce trebuie implementate ulterior.*

Recomandări resurse REST

- Se recomandă folosirea consecventă a unei strategii pentru denumirea resurselor.
- API REST folosește URIs pentru accesarea resurselor. URI-urile folosite ar trebui să sugereze clienților modelul resurselor.
- Dacă resursele sunt denumite sugestiv, API-ul corespunzător va fi ușor de folosit și intuitiv. Altfel poate fi dificil de înțeles și utilizat.
- O resursă poate fi *singulară* sau *o colecție de alte resurse*.

Exemplu:

- “**articles**” - colecție de resurse
- “**article**” - singular.
- Identificarea resurselor de tip colecție se poate face folosind substantivul la plural:
Resursa “**articles**” : URI “**/articles**”.
- Identificarea unei resurse singulare se poate face folosind identificatorul:
Resursa “**article**” are URI “**/articles/{articleID}**”.
- O resursă poate conține o colecție de alte resurse (subcolecție).
ex. autorii unui articol pot fi identificați prin:
“**/articles/{articleID}/authors**”.
- Identificarea unei resurse singulare dintr-o subcolecție:
“**/articles/{articleID}/authors/{authorID}**”.

REST Recomandări

- *Folosirea substantivelor pentru reprezentarea resurselor*
- **URI RESTful trebuie să refere resursa, nu să se refere la o acțiune.** Substantivele au proprietăți, și asemănător resursele au attribute.

`http://api.example.com/conference/articles`

~~`http://api.example.com/conference/getArticles`~~ - nu este URI corespunzator REST

`http://api.example.com/conference/articles/{id}/title`

`http://api.example.com/user-management/users`

`http://api.example.com/user-management/users/{id}`

`http://api.example.com/user-management/users/{id}/name`

- *Folosirea consecventă a convențiilor pentru denumirea resurselor și a modului de formare a URI-ului pentru a reduce ambiguitățile și îmbunătățirea înțelegerii și a întreținerii:*

- Folosirea caracterului / pentru a indica relații ierarhice
- A NU se folosi caracterul / la finalul URI-ului

~~`http://api.example.com/device-management/managed-devices/`~~

`http://api.example.com/device-management/managed-devices`

REST Recomandări

- *Folosirea cratimei (-) pentru îmbunătățirea înțelegerii URI*

<http://api.example.com/inventory-management/managed-entities/{id}/install-script-location>

~~<http://api.example.com/inventory-management/managedEntities/{id}/installScriptLocation>~~

- *Folosirea literelor mici în URI*

<http://api.example.org/my-folder/my-doc>

~~<http://api.example.org/My-Folder/my-doc>~~

- *A nu se folosi extensii de fișier:* Nu aduc informații suplimentare clientului și pot induce în eroare dezvoltatorii.
- Pentru a preciza *tipul media corespunzător unei reprezentări* se folosesc antetele *Accept* și *Content-Type*.
- Acestea sunt folosite și pentru a determina modul de procesare a conținutului.

~~<http://api.example.com/device-management/managed-devices.xml>~~

<http://api.example.com/device-management/managed-devices>

REST Recomandări

- *A nu se folosi nume de funcții CRUD (get, add/save, delete, update/modify) în URI-uri*
- URI-urile nu ar trebui folosite pentru a preciza operația/operațiile efectuate.
- URI-urile ar trebui folosite pentru a identifica în mod unic resursele, nu pentru a le manipula.
- Metodele HTTP ar trebui folosite pentru a indica operația CRUD ce trebuie efectuată.

//obținerea tuturor articolelor (echivalentul lui getAll/findAll)

HTTP GET si URI <http://api.example.com/conference/articles>

//Crearea unui nou articol (echivalentul lui save/add)

HTTP POST si URI <http://api.example.com/conference/articles>

//Obținerea articolului cu id-ul dat (echivalentul lui findOne)

HTTP GET si URI <http://api.example.com/conference/articles/{id}>

//Actualizarea articolului cu id-ul dat (echivalentul lui update/modify)

HTTP PUT si URI <http://api.example.com/conference/articles/{id}>

//Stergerea articolului cu id-ul dat (echivalentul lui delete)

HTTP DELETE si URI <http://api.example.com/conference/articles/{id}>

REST Recomandări

- *Folosirea parametrilor interogării HTTP pentru a filtra o colecție.*
 - Sortarea elementelor din colecție
 - Filtrarea elementelor din colecție
 - Limitarea numărului de elemente returnate (paginarea)
 - Nu se creează URI-uri noi, ci se folosesc parametrii de tip query:

`http://api.example.com/conference/articles`

`http://api.example.com/conference/articles?domain=AI`

`http://api.example.com/conference/articles?domain=AI&subdomain=Genetic`

`http://api.example.com/conference/articles?`

`domain=AI&subdomain=Genetic&sort=submission-date`

`//gresit`

`http://api.example.com/conference/getArticlesSortedBy?`

`domain=AI&subdomain=Genetic&sort=submission-date`

REST API - Proiectare

- *Pașii proiectării serviciilor REST*

1. Identificarea modelului obiectual (a resurselor)
2. Crearea modelului pentru URI-uri
3. Determinarea reprezentărilor
4. Atribuirea/asocierea metodelor HTTP
5. Alte acțiuni (Logging, Security, Discovery)

- Identificarea modelului obiectual

- Identificarea obiectelor care vor fi prezentate ca și resurse:
 - User
 - Message,
 - Article,
 - Author,
 - Person
 - etc.

REST API Proiectare

- Crearea modelului URI-urilor
 - Deciderea URI-urilor pentru resursele identificate anterior.
 - Axarea pe relațiile dintre resurse și sub-resurse.
 - Aceste URI-uri asociate resurselor vor fi folosite ca și endpoint-uri pentru serviciile REST.
 - **URI-urile nu conțin verbe sau operații!!!**
 - **URI-urile ar trebui să conțină doar substantive!**

`/articles`

`/articles/{id}`

`/authors`

`/authors/{id}`

`/articles/{id}/authors`

`/articles/{id}/authors/{aid}`

REST API - Proiectare

- *Determinarea reprezentării resurselor*

- Majoritatea reprezentărilor sunt definite în format XML sau JSON.

<code><author></code>	<code>{</code>
<code><name> Pop Ion </name></code>	<code>"name": "Pop Ion"</code>
<code><affiliation> ZY Lab </affiliation></code>	<code>"affiliation": "ZY Lab"</code>
<code></author></code>	<code>}</code>

- *Asocierea metodelor HTTP*

- Obținerea tuturor articolelor sau autorilor

HTTP GET si URI **/articles**

HTTP GET si URI **/authors**

- Paginarea, dacă rezultatul este prea mare.

HTTP GET si URI **/articles?startIndex=0&size=20**

HTTP GET si URI **/authors?startIndex=0&size=20**

- Obținerea tuturor autorilor unui articol (subcolecție)

HTTP GET si URI **/articles/{id}/authors**

REST API -Proiectare

- Obținerea unui singur articol/autor

HTTP GET si URI **/articles/{id}**

HTTP GET si URI **/authors/{id}**

- Obținerea unui singur autor (subcolecție)

HTTP GET si URI **/articles/{id}/authors/{id}**

Reprezentarea subresursei poate fi diferită.

- Crearea unui articol/autor - metoda HTTP POST

HTTP POST si URI **/articles**

HTTP POST si URI **/authors**

IMPORTANT!

- **De obicei, la crearea unei resurse, corpul cererii NU conține informații legate de id, deoarece serverul este responsabil de determinarea acestuia.**
- **Exceptie:** id-ul este informatie relevanta pentru resursa (username, CNP)

REST API - Proiectare

- Actualizarea unui articol/autor - metoda HTTP PUT

HTTP PUT si URI **/articles/{id}**

HTTP PUT si URI **/authors/{id}**

- Ștergerea unui articol/autor - metoda HTTP DELETE

HTTP DELETE si URI **/articles/{id}**

HTTP DELETE si URI **/authors/{id}**

- Un răspuns ar trebui să returneze:

- **202 (Accepted)** dacă resursa a fost pusă într-o coadă și urmează a fi ștearsă (asincron).
- **200 (OK) / 204 (No Content)** dacă resursă a fost ștearsă permanent (sincron).

- Adăugarea unui autor la articol - Metoda HTTP PUT

HTTP PUT si URI **/articles/{id}/authors**

- Ștergerea unui autor de la articol - Metoda HTTP DELETE

HTTP DELETE si URI **/articles/{articleId}/authors/{authorId}**

HTTP Response Status Code

- **2XX -Success** acțiunea cerută de client a fost primită, înțeleasă, acceptată și procesată cu succes.
 - **200 OK**: Răspunsul standard pentru cereri HTTP executate cu succes.
 - **201 Created**: Cererea a fost îndeplinită și a fost creată o nouă resursă.
 - **202 Accepted**: Cererea a fost acceptată pentru procesare, dar procesarea nu a fost încă efectuată.
 - **204 No Content**: Serverul a procesat cererea cu succes, dar procesarea nu a returnat date.
 - **205 Reset Content**: Serverul a procesat cererea cu succes, dar nu a returnat date. Spre deosebire de răspunsul 204, acest răspuns necesită ca clientul să reafișeze vederea/informatia.
- etc.

HTTP Response Status Code

- **4XX** Erori client:
 - **400 Bad Request**: Serverul nu poate sau nu va procesa cererea din cauza unei erori din partea clientului (ex. sintaxa cererii este greșită, conținutul cererii este prea mare, etc.)
 - **401 Unauthorized**: Similar cu 403, dar se folosește când este necesară autentificarea, care încă nu s-a efectuat sau a eșuat.
 - **403 Forbidden**: Cererea este validă, dar serverul refuză procesarea. (Utilizatorul nu are drepturile necesare pentru a accesa resursa respectivă).
 - **404 Not Found**: Resursa cerută nu a fost găsită, dar poate deveni disponibilă în viitor. Sunt permise cereri ulterioare din partea clientului.
 - **405 Method Not Allowed**: Metoda cerută nu este disponibilă pentru resursa cerută. Ex. o cerere GET pentru un formular(eng. *form*) care necesită ca datele să fie transmise folosind POST, sau o cerere PUT pentru o resursă care poate fi doar citită.
 - **406 Not Acceptable**: Reprezentările disponibile pentru resursa cerută nu sunt acceptate de client (folosind antetul *Accept*).
 - etc.
- **5XX** Erori la implementarea serviciilor

HTTP Media Types

- Antetele HTTP **Accept** și **Content-Type** pot fi folosite pentru a descrie reprezentarea folosită pentru schimbarea informației (reprezentarea resurselor).
- Clientul trimite o cerere în care în antetul **Accept** specifică lista tipurilor de reprezentare acceptate (separate prin virgulă).

Exemplu : **Accept: application/json, application/xml**

- Serverul trimite răspunsul folosind ca și reprezentare primul tip mime înțeles din lista precizată (dacă înțelege unul sau mai multe). Tipul folosit este precizat în antetul **Content-Type**.

Exemplu: **Content-Type: application/xml**

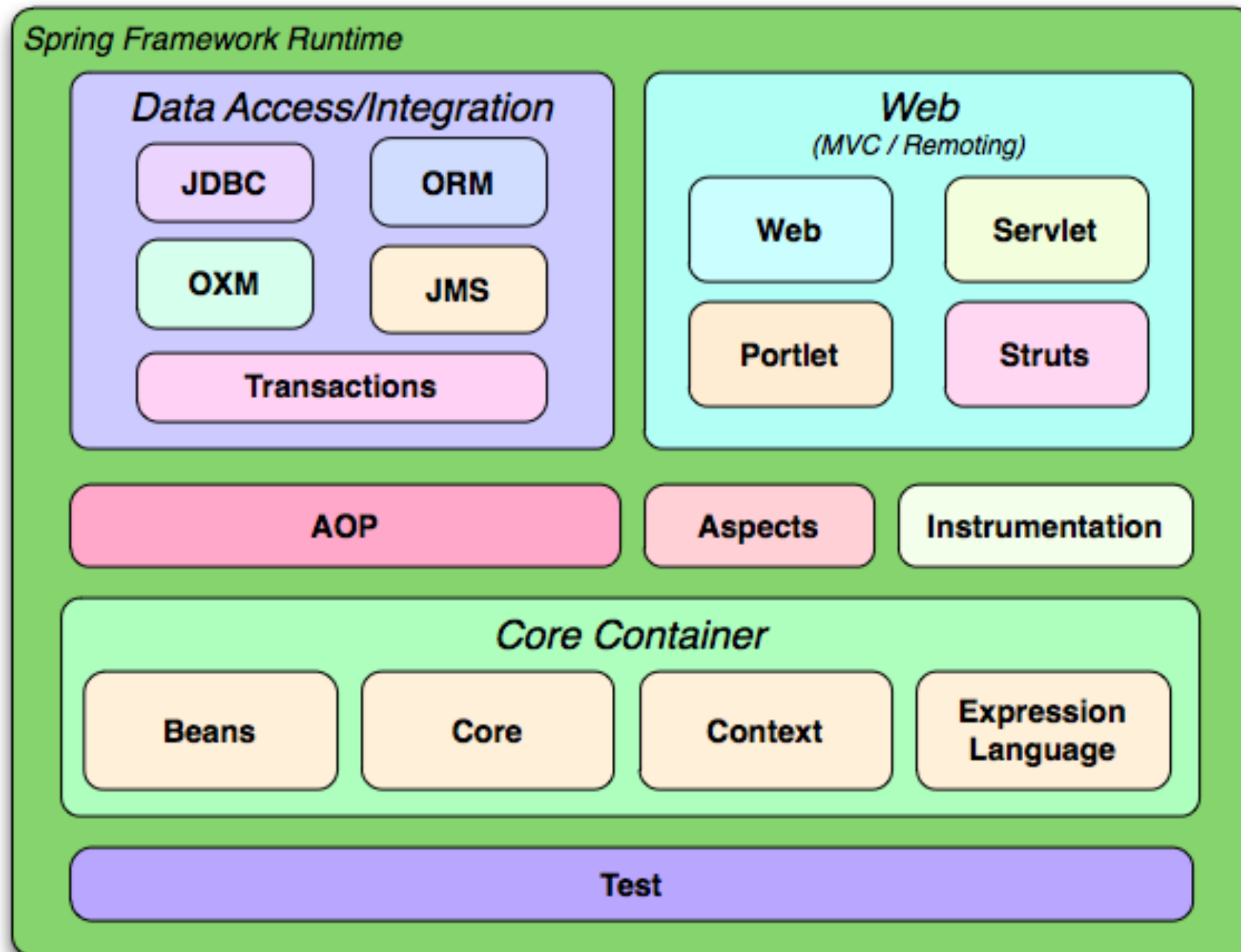
Referințe

- Roy Fielding, PhD Thesis, Chapter 5. Representational State Transfer (REST)

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

- <http://restfulapi.net/>
- <http://www.service-architecture.com/articles/web-services/>
- <https://martinfowler.com/articles/richardsonMaturityModel.html>
- Alte tutoriale

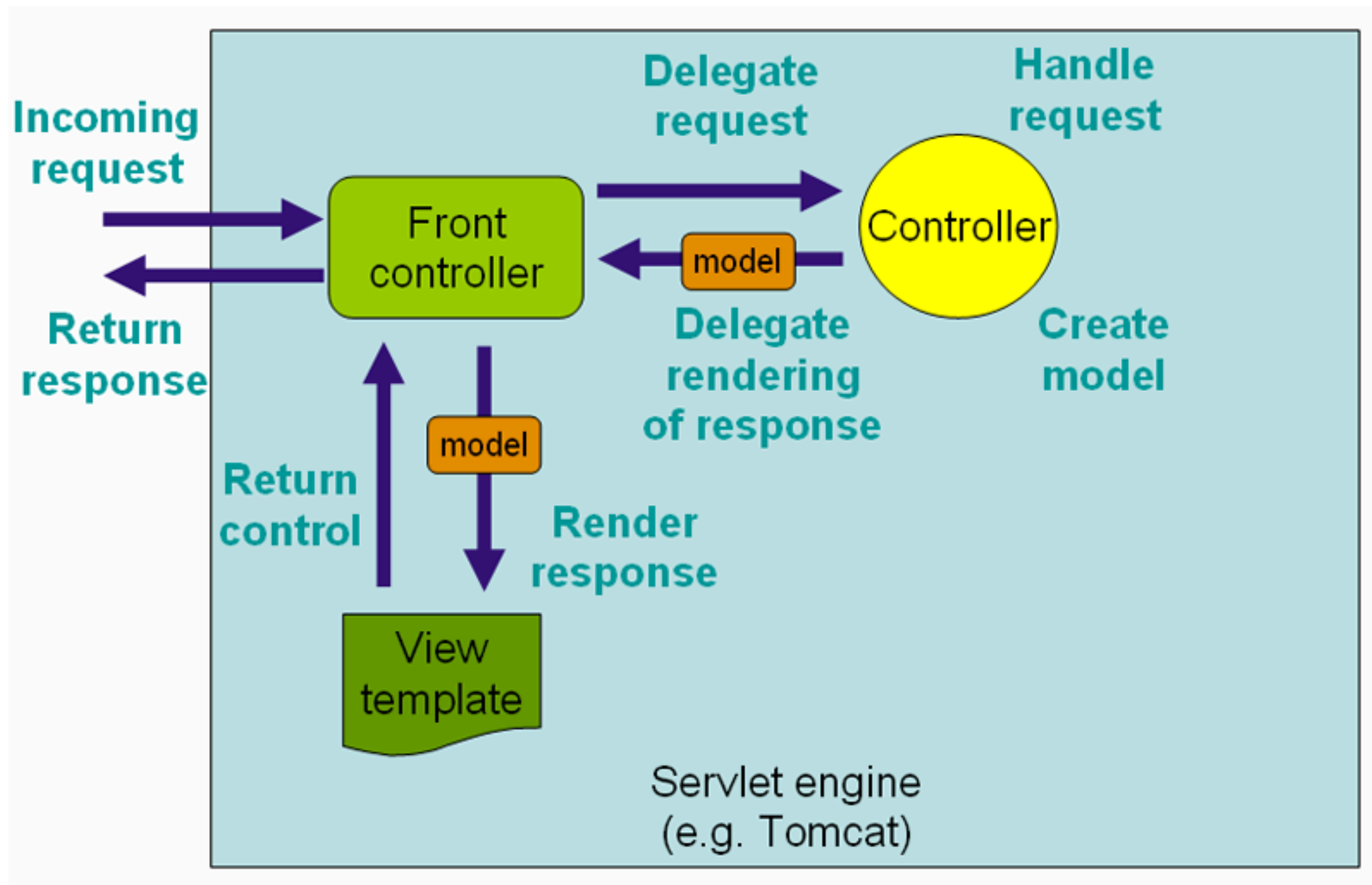
Spring Web MVC



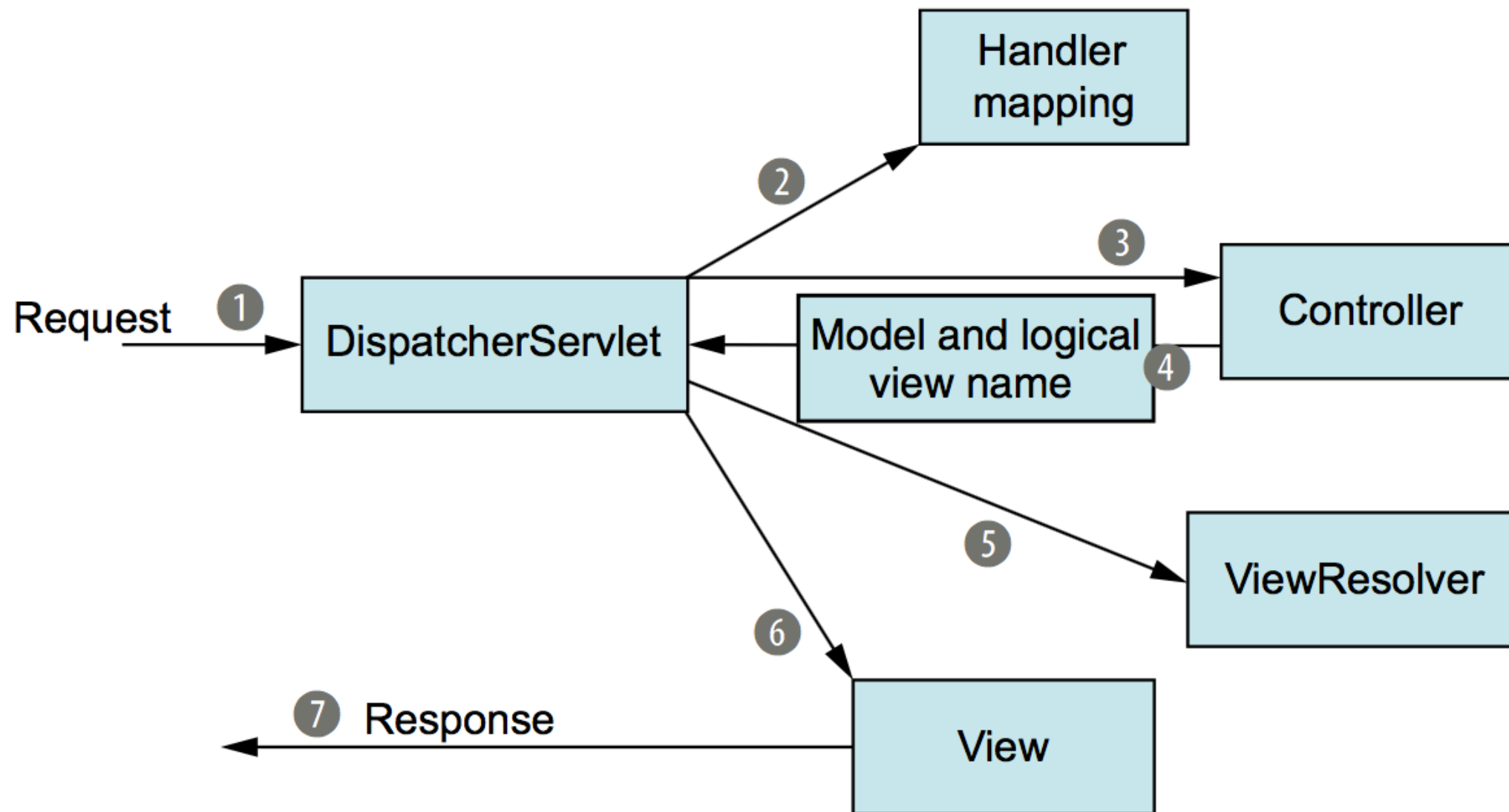
Spring Web MVC

- Frameworkul Spring Web model-view-controller (MVC) este proiectat în jurul unui *DispatcherServlet* care trimite cererile handler-elor, cu mapări configurabile, soluționarea vederilor (eng. *view*), a temei, a datelor și suport pentru încărcarea fișierelor.
- Handler-ul implicit folosește adnotările `@Controller` și `@RequestMapping` ce permit folosirea diferitor metode pentru procesarea cererilor.
- Orice obiect poate fi folosit ca și comandă sau pentru procesarea unei forme.
- Soluționarea vederilor este foarte flexibilă.
- Un controller este responsabil cu pregătirea modelului ce conține datele ce urmează a fi afișate și cu selectarea vederii folosită pentru afișare (folosind un nume pentru vedere).
- Soluționarea numelui asociat unei vederi se poate face prin extensii de fișiere, folosind antetul `Accept`, fișiere de proprietăți, bean-uri, etc.
- Modelul care păstrează datele este o interfață `Map`, care permite decuplarea de tehnologia folosită pentru afișarea acestora.

Spring Web MVC



Spring Web MVC



Spring Web MVC - RestController

- Clase adnotate cu **@RestController**
- Adnotarea **@RestController**: metodele adnotate din controller sunt considerate ca fiind HTTP endpoints și folosesc metadata furnizată de adnotarea **@RequestMapping** (**@GetMapping**, **@PostMapping**, etc) folosită la fiecare metodă/clasă pentru identificarea metodei ce trebuie să proceseze o cerere.
- O metodă va fi folosită pentru procesarea unei cereri HTTP dacă cererea satisface condițiile precizate folosind adnotarea **@RequestMapping/@GetMapping/...** asociată metodei.
- Adnotarea **@RestController** furnizează și valori implicite (informații metadata) pentru toate metodele din clasa adnotată.
- Fiecare metodă poate decide suprascrierea adnotărilor specificate la nivel de clasă, dar anumite informații depind de context.

Spring Web MVC - RestController

```
@RestController
```

```
@RequestMapping("/articles/{articleId}/authors")
```

```
public class AuthorRestController {
```

```
    @RequestMapping(method = RequestMethod.GET) // @GetMapping
```

```
    Collection<Author> readAuthors(@PathVariable String articleId,
```

```
                                   @RequestParam(value="sorted", defaultValue="name")String sortCrit) { ... }
```

```
    @RequestMapping(method = RequestMethod.POST) // @PostMapping
```

```
    ResponseEntity<?> add(@PathVariable String articleId, @RequestBody Author input) { ... }
```

```
    @RequestMapping(value =("/{authorId}", method = RequestMethod.GET)
```

```
    Author readAuthor(@PathVariable String articleId, @PathVariable Long authorId) { ...}
```

```
    private void validateUser(String userId) { .... }
```

```
}
```


Spring Web MVC - RestController

- `{articleId}` și `{authorId}` sunt **variabile de cale** (eng. *path variables*). Sunt asemănătoare caracterelor speciale (eng. *wildcards*).
- Spring MVC va extrage acele părți din URI și le va face disponibile ca și parametri ai metodelor ce procesează cererea. **Parametrii vor avea același nume cu variabila de cale și sunt adnotați cu adnotarea `@PathVariable`.**

Exemplu: O cerere HTTP GET cu URI `/articles/art123/authors/4234`

- Parametrul `@PathVariable String articleId` va avea valoarea `art123`,
- Parametrul `@PathVariable Long authorId` va avea valoare `4234`.
- `@RequestParam` indică **parametrii cererii** (eng. *request query parameters*).

Exemplu: O cerere HTTP GET cu URI `/art123/authors/4234?sorted=affiliation`

- `@RequestParam String sortCrit` va avea valoarea `affiliation`

Tipul parametrilor /returnat

- Datele returnate după procesarea cererii pot avea orice tip.
 - **Tip primitiv/ obiect**: informația va fi convertită la reprezentarea negociată de client și server (json, xml, text, etc).
 - **@RequestBody**: Datele transmise de client vor fi convertite la tipul precizat.
 - **RequestEntity<E>**: Este un wrapper pentru cererea primită de la client. Conține reprezentarea informației și datele adiționale din cererea HTTP (antetele, dimensiunea, etc).
 - **@ResponseBody**: Obiectul returnat va fi convertit la reprezentarea negociată.
 - **ResponseEntity<E>**: Este un wrapper pentru obiectul de tipul E și opțional poate seta tipul de răspuns (HTTP Response Status Code) și alte antete HTTP.

Tipul returnat

```
@RequestMapping(method = RequestMethod.GET)

String greeting(@RequestParam(value="name" defaultValue="") String name ){

    return "Hello "+name+"!";

}
```

```
@RequestMapping(method = RequestMethod.GET)

@ResponseBody String greeting(@RequestParam(value="name" defaultValue="") String name ){

    return "Hello "+name+"!";

}
```

```
@RequestMapping(method = RequestMethod.GET)

ResponseEntity<String> greeting(@RequestParam(value="name" defaultValue="") String name )

{

    return new ResponseEntity<>("Hello "+name+"!");

}
```

```
@RequestMapping(method = RequestMethod.GET)

ResponseEntity<String> greeting(RequestEntity<String> request, @RequestParam(value="name"
defaultValue="") String name ){ ...}
```

Convertirea mesajelor

- Convertirea mesajelor este o modalitate de a transforma datele obținute de controller/de la controller într-o reprezentare ce va fi transmisă clientului.
- Spring conține diferite clase pentru convertirea mesajelor, pentru a putea trata cele mai întâlnite conversii obiect-reprezentare.
 - **BufferedImageHttpMessageConverter**: Obiect **BufferedImage** din/într-o imagine (păstrată ca și un tablou de octeți).
 - **MappingJackson2HttpMessageConverter**: Obiecte simple și HashMaps (String, <? >) în format JSON.
 - **MarshallingHttpMessageConverter** (XML)
 - **StringHttpMessageConverter** (String, text/plain)

Tratarea excepțiilor

```
@RequestMapping(value =("/{id}", method = RequestMethod.GET)

public Article getById(@PathVariable String id){

    return articleRepository.findById(id); //daca id-ul nu exista, returnează null

}
```

- Id-ul nu există, răspunsul va fi 200 (OK), iar conținutul va fi null.

Soluția 1

```
@RequestMapping(value =("/{id}", method = RequestMethod.GET)

public ResponseEntity<?> getById(@PathVariable String id){

    Article article=articleRepository.findById(id);

    if (article==null)
        return new ResponseEntity<String>("Article not found",HttpStatus.NOT_FOUND);

    else

        return new ResponseEntity<Article>(article, HttpStatus.OK);

}
```

Tratarea excepțiilor

Soluția 2

```
@RequestMapping(value =("/{id}", method = RequestMethod.GET)

public ResponseEntity<Article> getById(@PathVariable String id){

    Article article=articleRepository.findById(id);

    if (article==null)

        throw new RepositoryException("Article not found.");

    ...

}

//2.a - in aceeași clasa cu metoda getById

@ExceptionHandler(RepositoryException.class)

@ResponseStatus(HttpStatus.NOT_FOUND)

public String repoException(RepositoryException e) { return e.getMessage(); }

//2.b

@ResponseStatus(HttpStatus.NOT_FOUND)

class RepositoryException extends RuntimeException {

    public RepositoryException(String message) {

        super(message);

    }

}
```

Pornirea aplicatiei

```
package conference;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class StartApplication {

    public static void main(String[] args) {

        SpringApplication.run(StartApplication.class, args);

    }

}

package conference;

@RestController

public class ArticleController{

    @Autowired private ArticleRepository articleRepository;

    // ...

}

@Component

public class ArticleRepositoryJdbc implements ArticleRepository{ ...}
```

Pornirea aplicatiei

- Adnotarea `@SpringBootApplication` specifică containerului Spring următoarele:
 - Adaugă adnotarea `@Configuration` clasei curente (specifică că poate conține definițiile unor bean-uri).
 - Adaugă adnotarea `@EnableAutoConfiguration`: specifică beanului Spring Boot să adauge beanuri în funcție de setările classpath-ului, a altor beanuri sau a altor proprietăți.
 - Adaugă adnotarea `@EnableWebMvc` necesară pentru a preciza ca este o aplicație SpringMVC.
 - Adaugă adnotarea `@ComponentScan` pentru căutarea altor bean-uri pornind de la pachetul “**conference**” (permite identificarea controllerelor REST). (“**conference**” reprezintă pachetul în care este definită clasă adnotată cu `@SpringBootApplication`)
- Metoda `main()` folosește `SpringApplication.run()` din Spring Boot pentru lansarea aplicației.
- **Serverul web (Tomcat) va fi inclus și pornit automat.**

Configurația Gradle

```
plugins {  
    id 'org.springframework.boot' version '3.1.0'  
    id 'io.spring.dependency-management' version '1.0.11.RELEASE'  
    id 'java'  
}  
  
group 'Conference'  
version '1.0'  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    implementation project (':ConferenceModel')  
    implementation project (':ConferencePersistence')  
  
    implementation group: 'com.fasterxml.jackson.core', name: 'jackson-annotations', version: '2.13.1'  
  
    implementation 'org.springframework.boot:spring-boot-starter-actuator'  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    testImplementation('org.springframework.boot:spring-boot-starter-test')  
    testImplementation('com.jayway.jsonpath:json-path')  
}
```


Exemple

- ComputerShop
- Chat

Clienți REST

- API pentru apelarea unui serviciu REST:
 - Java
 - .NET
 - JavaScript
 - etc.
- Browser web
- Aplicații desktop (Extensii ale browserelor web):
 - POSTMAN
 - Advanced REST Client
 - RESTED
 - etc.

REST Client - RestTemplate (Spring)

- RestTemplate definește 36 de metode pentru interacțiunea cu resurse REST.

Method	Description
<code>delete()</code>	Performs an HTTP <code>DELETE</code> request on a resource at a specified URL
<code>exchange()</code>	Executes a specified HTTP method against a URL, returning a <code>ResponseEntity</code> containing an object mapped from the response body
<code>execute()</code>	Executes a specified HTTP method against a URL, returning an object mapped from the response body
<code>getForEntity()</code>	Sends an HTTP <code>GET</code> request, returning a <code>ResponseEntity</code> containing an object mapped from the response body
<code>getForObject()</code>	Sends an HTTP <code>GET</code> request, returning an object mapped from a response body
<code>headForHeaders()</code>	Sends an HTTP <code>HEAD</code> request, returning the HTTP headers for the specified resource URL
<code>optionsForAllow()</code>	Sends an HTTP <code>OPTIONS</code> request, returning the <code>Allow</code> header for the specified URL
<code>postForEntity()</code>	POSTs data to a URL, returning a <code>ResponseEntity</code> containing an object mapped from the response body
<code>postForLocation()</code>	POSTs data to a URL, returning the URL of the newly created resource
<code>postForObject()</code>	POSTs data to a URL, returning an object mapped from the response body
<code>put()</code>	PUTs resource data to the specified URL

Spring REST Client

```
public class StartRestClient {

    public static void main(String[] args) {
        RestTemplate restTemplate=new RestTemplate();

        //Adding an article
        Article article=new Article("AB CB","AI","Genetic");
        try{
            String articleId= restTemplate.postForObject("http://localhost:8080/conference/articles",article, String.class);

            // Updating article ...
            article.setTitle("New title");
            restTemplate.put("http://localhost:8080/conference/articles/"+articleId,
            article);

        }catch(RestClientException ex){
            System.out.println("Exception ... "+ex.getMessage());
        }

    }

}
```

REST Client - RestClient (Spring >=v6.1)

- RestClient - permite apeluri sincrone către Endpoint-uri REST
- Usor de configurat și apelat

```
RestClient defaultClient = RestClient.create(); //configurare default
```

sau

```
//configurari specifice
```

```
RestClient customClient = RestClient.builder()
```

```
    .baseUrl("https://example.com")
```

```
    .defaultHeader("My-Header", "Foo")
```

```
    .requestInterceptor(myCustomInterceptor)
```

```
    .build();
```

```
//apel
```

```
int articleId=123;
```

```
Article article = restClient.get() //post(), delete(), put()
```

```
    .uri("https://conference.com/articles/{id}", articleId)
```

```
    .accept(APPLICATION_JSON) //contentType(APPLICATION_JSON).body(article)
```

```
    .retrieve()
```

```
    .body(Article.class); //toBodilessEntity();
```

.NET REST Client

- Clasa `System.Net.Http.HttpClient`
 - Trebuie adăugat (folosind NuGet) pachetul `System.Net.Http`
 - ***HttpClient ar trebui instanțiat o singură dată într-o aplicație*** și refolosit pe parcursul rulării aplicației.
Crearea unei noi instanțe pentru fiecare cerere, poate genera erori de tip `SocketException` (se atinge numărul maxim de conexiuni permise).

```
class MainClass{
    static HttpClient client = new HttpClient();
    public static void Main(string[] args){
        client.DefaultRequestHeaders.Accept.Clear();
        client.DefaultRequestHeaders.Accept.Add(new MediaTypeWithQualityHeaderValue("application/json"));
        // Get an article ...;
        Article result = await GetArticleAsync("http://localhost:8080/conference/articles/art124");
    }

    static async Task<Article> GetArticleAsync(string path){
        Article article = null;
        HttpResponseMessage response = await client.GetAsync(path);
        if (response.IsSuccessStatusCode)
        {
            var responseString = await response.Content.ReadAsStringAsync();
            article = JsonConvert.DeserializeObject<Article>(responseString);
        }
        return article;
    }
}
```

Referințe

- <https://learn.microsoft.com/en-us/dotnet/fundamentals/networking/http/httpclient>
- Building REST services with Spring:
<https://spring.io/guides/tutorials/rest>
- <http://www.service-architecture.com/articles/web-services/>
- Craig Walls, Spring in Action, 6th Edition, Ed. Manning, 2022
- Alte tutoriale

Medii de proiectare și programare

2024-2025

Curs 11

Conținut

- Servicii REST (cont.)
- Clienți web pentru servicii REST
 - JavaScript:
 - Promise
 - Fetch
 - React

Servicii REST - Exemple

- ComputerShop
- Chat

Clienți REST

- API pentru apelarea unui serviciu REST:
 - Java
 - .NET
 - JavaScript
 - etc.
- Browser web
- Aplicații desktop (Extensii ale browserelor web):
 - POSTMAN
 - Advanced REST Client
 - RESTED
 - etc.

REST Client - RestTemplate (Spring)

- RestTemplate definește 36 de metode pentru interacțiunea cu resurse REST.

Method	Description
<code>delete()</code>	Performs an HTTP DELETE request on a resource at a specified URL
<code>exchange()</code>	Executes a specified HTTP method against a URL, returning a <code>ResponseEntity</code> containing an object mapped from the response body
<code>execute()</code>	Executes a specified HTTP method against a URL, returning an object mapped from the response body
<code>getForEntity()</code>	Sends an HTTP GET request, returning a <code>ResponseEntity</code> containing an object mapped from the response body
<code>getForObject()</code>	Sends an HTTP GET request, returning an object mapped from a response body
<code>headForHeaders()</code>	Sends an HTTP HEAD request, returning the HTTP headers for the specified resource URL
<code>optionsForAllow()</code>	Sends an HTTP OPTIONS request, returning the Allow header for the specified URL
<code>postForEntity()</code>	POSTs data to a URL, returning a <code>ResponseEntity</code> containing an object mapped from the response body
<code>postForLocation()</code>	POSTs data to a URL, returning the URL of the newly created resource
<code>postForObject()</code>	POSTs data to a URL, returning an object mapped from the response body
<code>put()</code>	PUTs resource data to the specified URL

Spring REST Client

```
public class StartRestClient {

    public static void main(String[] args) {
        RestTemplate restTemplate=new RestTemplate();

        //Adding an article
        Article article=new Article("AB CB","AI","Genetic");
        try{
            String articleId= restTemplate.postForObject("http://localhost:8080/conference/articles",article, String.class);

            // Updating article ...
            article.setTitle("New title");
            restTemplate.put("http://localhost:8080/conference/articles/"+articleId,
            article);

        }catch(RestClientException ex){
            System.out.println("Exception ... "+ex.getMessage());
        }

    }
}
```

REST Client - RestClient (Spring >=v6.1)

- **RestClient** - permite apeluri sincrone către Endpoint-uri REST
- Usor de configurat și apelat

```
RestClient defaultClient = RestClient.create(); //configurare default  
sau
```

```
//configurari specifice
```

```
RestClient customClient = RestClient.builder()
```

```
    .baseUrl("https://example.com")
```

```
    .defaultHeader("My-Header", "Foo")
```

```
    .requestInterceptor(myCustomInterceptor)
```

```
    .build();
```

```
//apel
```

```
int articleId=123;
```

```
Article article = restClient.get() //post(), delete(), put()
```

```
    .uri("https://conference.com/articles/{id}", articleId)
```

```
    .accept(APPLICATION_JSON) //contentType(APPLICATION_JSON).body(article)
```

```
    .retrieve()
```

```
    .body(Article.class); //toBodilessEntity();
```

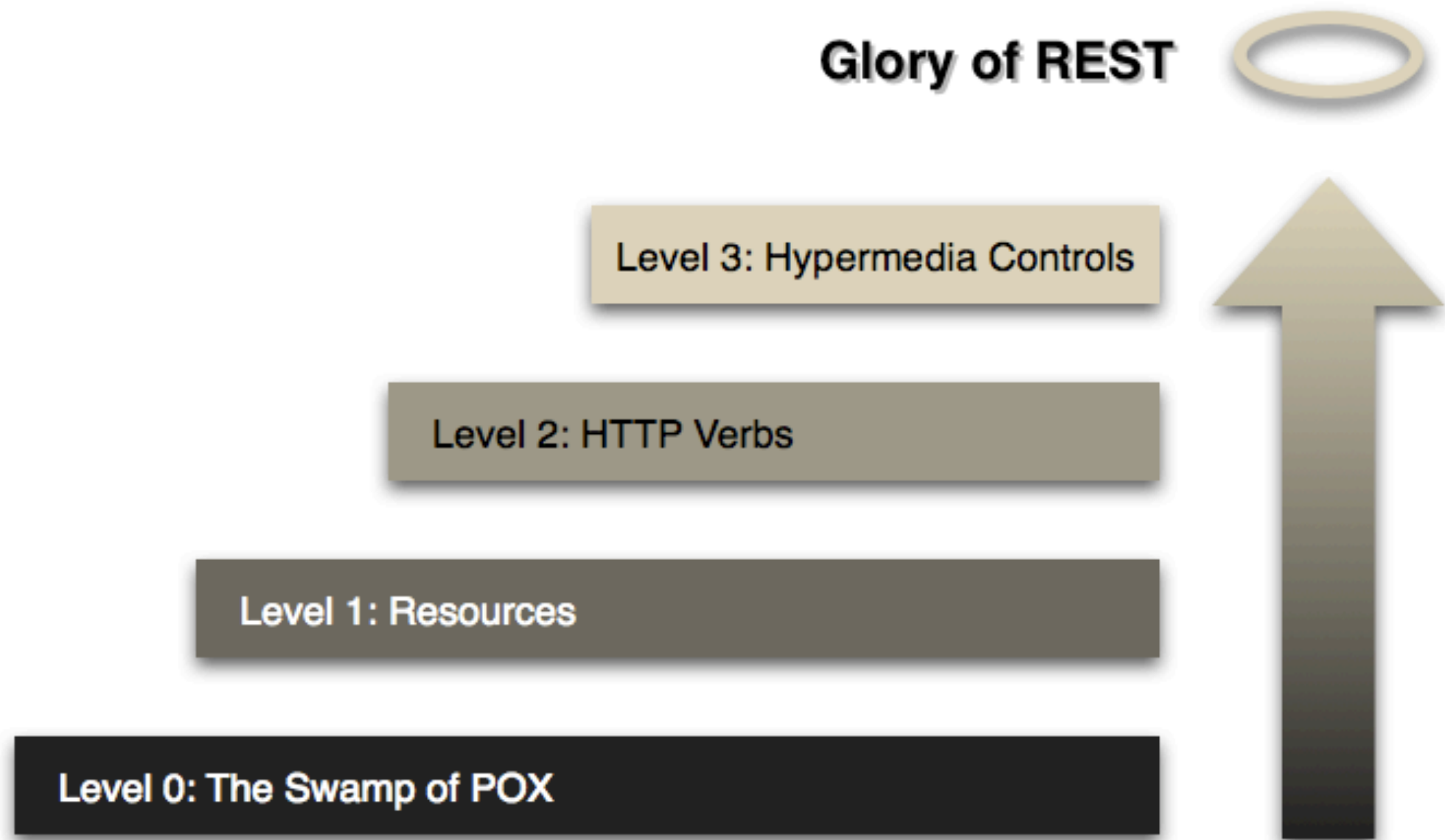
.NET REST Client

- Clasa `System.Net.Http.HttpClient`
 - Trebuie adăugat (folosind NuGet) pachetul `System.Net.Http`
 - ***HttpClient ar trebui instanțiat o singură dată într-o aplicație*** și refolosit pe parcursul rulării aplicației.
Crearea unei noi instanțe pentru fiecare cerere, poate genera erori de tip `SocketException` (se atinge numărul maxim de conexiuni permise).

```
class MainClass{
    static HttpClient client = new HttpClient();
    public static void Main(string[] args){
        client.DefaultRequestHeaders.Accept.Clear();
        client.DefaultRequestHeaders.Accept.Add(new MediaTypeWithQualityHeaderValue("application/json"));
        // Get an article ...;
        Article result = await GetArticleAsync("http://localhost:8080/conference/articles/art124");
    }

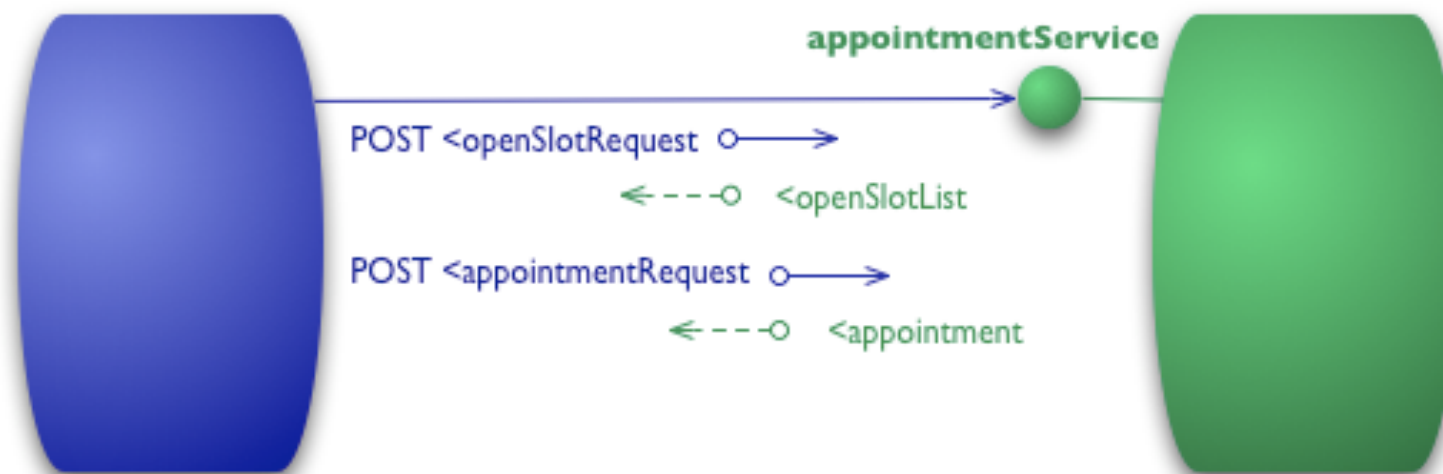
    static async Task<Article> GetArticleAsync(string path){
        Article article = null;
        HttpResponseMessage response = await client.GetAsync(path);
        if (response.IsSuccessStatusCode)
        {
            var responseString = await response.Content.ReadAsStringAsync();
            article = JsonConvert.DeserializeObject<Article>(responseString);
        }
        return article;
    }
}
```


Richardson Maturity Model*



*<https://martinfowler.com/articles/richardsonMaturityModel.html>

Level 0 - The Swamp of POX



One URI, one HTTP method

URI: **POST** /appointmentService HTTP/1.1

Body: <openSlotRequest date = "2010-01-04" doctor = "mjones"/>

Response: 200 OK

...

URI: **POST** /appointmentService HTTP/1.1

Body: <appointmentRequest>

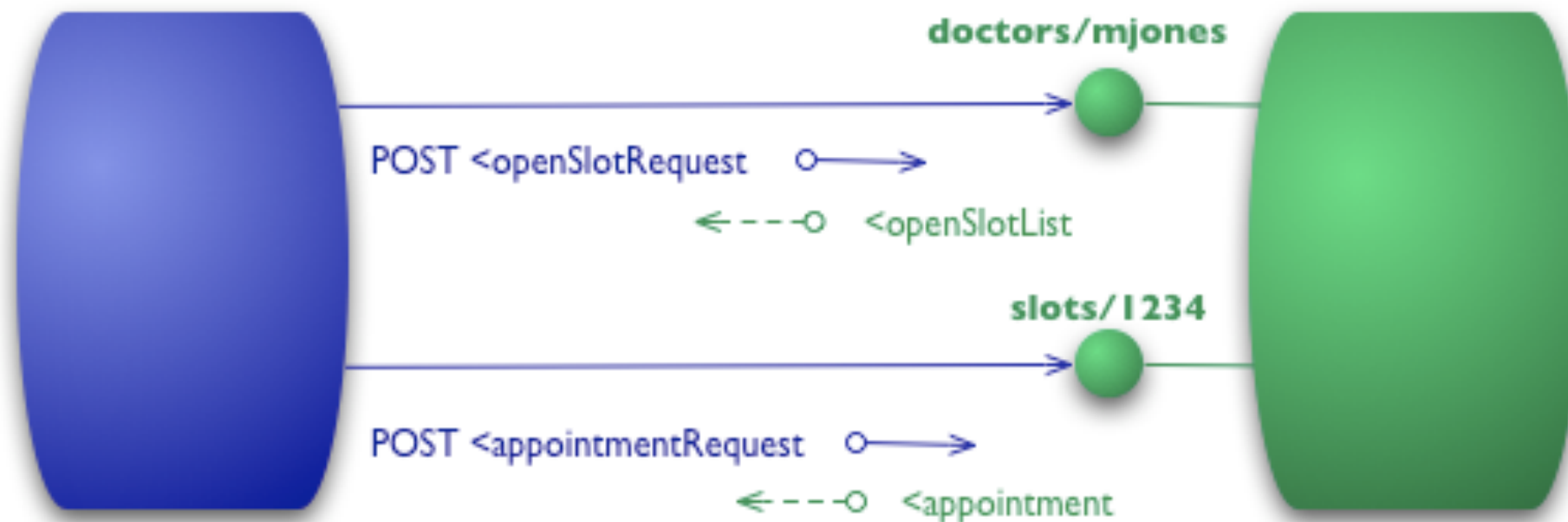
<slot doctor = "mjones" start = "1400" end = "1450"/>

<patient id = "jsmith"/>

</appointmentRequest>

Response: 200 OK ...

Level 1: Resources



Many URI, one HTTP method

URI: **POST** /doctors/mjones HTTP/1.1

Body: `<openSlotRequest date = "2010-01-04"/>`

Body: ~~`<openSlotRequest date = "2010-01-04" doctor = "mjones"/>`~~

Response: 200 OK

`<openSlotList>`

`<slot id = "1234" doctor = "mjones" start = "1400" end = "1450"/>`

`<slot id = "5678" doctor = "mjones" start = "1600" end = "1650"/>`

`</openSlotList>`

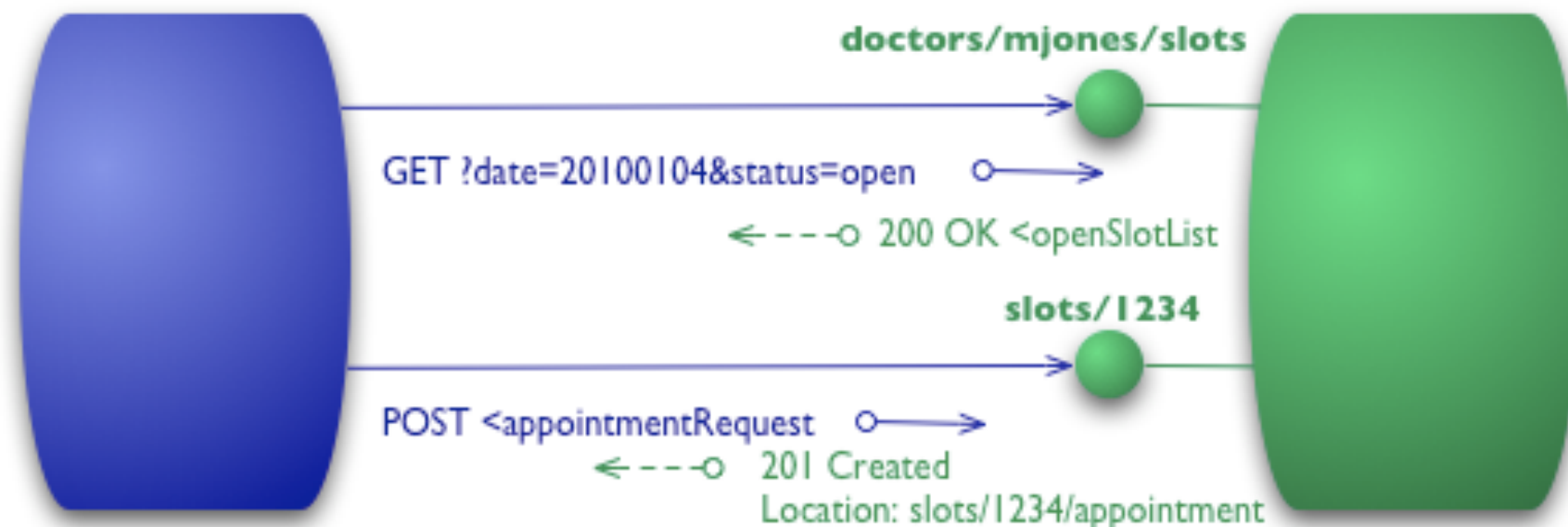
URI: **POST** /slots/1234 HTTP/1.1

Body: `<patient id = "jsmith"/>`

Response: 200 OK

...

Level 2 - HTTP Verbs



Many URIs, each supporting multiple HTTP methods

URI: **GET** /doctors/mjones/slots?date=20100104&status=open HTTP/1.1

Body: empty

Response: 200 OK

`<openSlotList>`

`<slot id = "1234" doctor = "mjones" start = "1400" end = "1450"/>`

`<slot id = "5678" doctor = "mjones" start = "1600" end = "1650"/>`

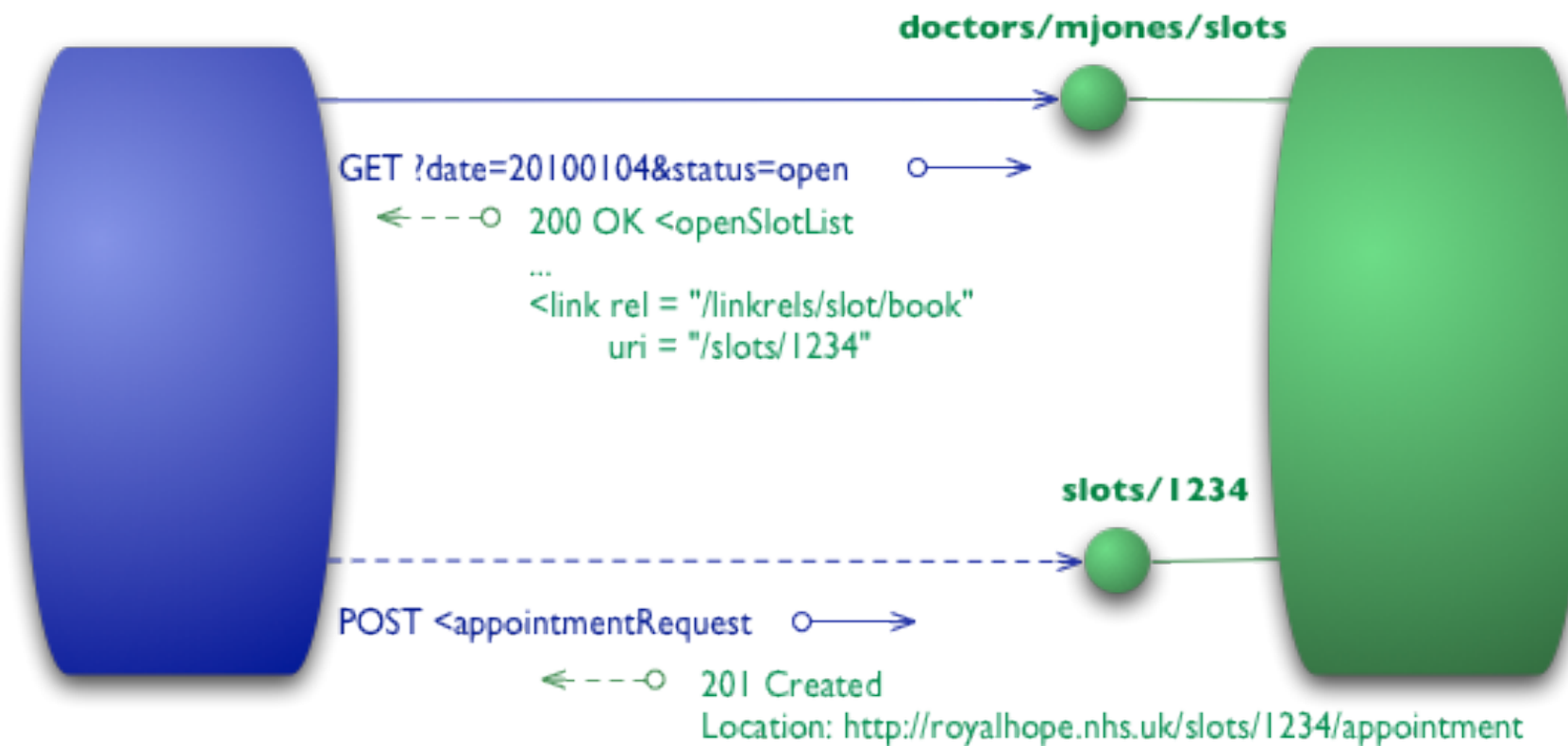
`</openSlotList>`

URI: **POST** /slots/1234 HTTP/1.1

Body: `<patient id = "jsmith"/>`

Body: 201 Created

Level 3 - Hypermedia Controls



Resources describe their own capabilities and interconnections

URI: **GET** /doctors/mjones/slots?date=20100104&status=open HTTP/1.1

```
Body: empty
Response: 200 OK
<openSlotList>
  <slot id = "1234" doctor = "mjones" start = "1400" end = "1450">
    <link rel = "/linkrels/slot/book"
      uri = "/slots/1234"/>
  </slot>
  <slot id = "5678" doctor = "mjones" start = "1600" end = "1650">
    <link rel = "/linkrels/slot/book"
      uri = "/slots/5678"/>
  </slot>
</openSlotList>
```

NU specifică metoda HTTP folosită!!!

Referințe

- <https://learn.microsoft.com/en-us/dotnet/fundamentals/networking/http/httpclient>
- Building REST services with Spring:
<https://spring.io/guides/tutorials/rest>
- <http://www.service-architecture.com/articles/web-services/>
- Craig Walls, Spring in Action, 6th Edition, Ed. Manning, 2022
- Richardson Maturity Model (<https://martinfowler.com/articles/richardsonMaturityModel.html>)
- Leonard Richardson, *The Maturity Heuristic*, <https://www.crummy.com/writing/speaking/2008-QCon/act3.html>
- Alte tutoriale

JavaScript - Evenimente

- JavaScript este **secvențial** (eng. *single threaded*), două părți de cod nu pot fi executate în același timp, trebuie să fie executate una după cealaltă.
- În browsere, JavaScript **împarte un fir de execuție cu alte sarcini**, care diferă de la browser la browser.
- JavaScript se comportă similar cu firul de execuție corespunzător interfețelor grafice: orice modificare a interfeței grafice duce la amânarea următoarelor sarcini corespunzătoare ei.
- În JavaScript **se pot folosi evenimente și funcții callback**:

```
var img1 = document.querySelector('.img-1');
```

```
img1.addEventListener('load', function() {  
    // imaginea a fost încărcată  
});
```

```
img1.addEventListener('error', function() {  
    // a apărut o situație excepțională  
});
```

JavaScript - Evenimente

- Este posibil ca un eveniment să apară înainte de a adăuga un listener. (Se poate folosi proprietatea "*complete*" pentru a trata situația):

```
var img1 = document.querySelector('.img-1');

function loaded() {
    // imaginea a fost încărcată
}

if (img1.complete) {
    loaded();
} else {
    img1.addEventListener('load', loaded);
}

img1.addEventListener('error', function() {
    // a apărut o situație excepțională
});
```

- Această soluție nu tratează cazul apariției unei erori înaintea adăugării listenerului.
- Soluția devine complexă când dorim să tratăm cazul încărcării mai multor imagini.
- Evenimentele sunt utile pentru situații care apar de mai multe ori asupra aceluiași obiect (apăsarea unei taste, etc). În aceste cazuri nu ne interesează ce s-a întâmplat înaintea adăugării listenerului.

JavaScript - Promise

- Pentru tratarea rezultatului unei operații asincrone (succes/eșec), este preferat următorul șablon:

```
img1.callThisIfLoadedOrWhenLoaded(function() {  
    // încărcată  
}).orIfFailedCallThis(function() {  
    // eșec  
});  
  
// și...  
whenAllTheseHaveLoaded([img1, img2]).callThis(function() {  
    // toate imaginile au fost încărcate  
}).orIfSomeFailedCallThis(function() {  
    // încărcarea unei imagini sau a mai multor imagini a eșuat  
});
```

- Conceptul de **promise** a fost introdus pentru astfel de situații.
- Promises sunt *asemănătoare cu listeneri*, exceptând:
 - Rezultatul execuției unui promise poate fi succes/eșec o singură dată. Nu poate fi semnalat succesul/eșecul de mai multe ori. Nu poate fi schimbat rezultatul succes -> eșec și invers.
 - Dacă rezultatul execuției este succes/eșec, dar funcția callback este adăugată ulterior, funcția va fi apelată cu rezultatul corect, chiar dacă rezultatul s-a obținut anterior.
- Este utilă obținerea rezultatului unei operații asincrone (succes/eșec), deoarece este mai important rezultatul obținut decât momentul de timp în care a fost obținut.

JavaScript - Promise

- Un obiect **Promise** reprezintă **eventualul rezultat al unei operații asincrone**. Modalitatea principală de a interacționa cu un promise este de a adăuga funcții callback care vor fi apelate fie cu rezultatul execuției (succes), fie cu motivul eșecului.
- Un **Promise** reprezintă o valoare care poate fi disponibilă *acum* sau *în viitor* sau *niciodată*. Valoarea respectivă e posibil să nu fie cunoscută în momentul creării obiectului.
- Permite adăugarea handlerelor (funcțiile callback) pentru tratarea succesului/eșecului.
- Permite metodelor asincrone să returneze un rezultat asemănător cu metodele sincrone: în loc să returneze rezultatul imediat, metoda asincronă returnează un obiect Promise cu ajutorul căreia poate fi obținut rezultatul execuției.
- Concepte:
 - **Promise**: un obiect cu o metodă *then*, a cărei execuție este conformă cu specificația.
 - **Thenable**: un obiect care definește o metodă *then*.
 - **Valoare**: orice valoare permisă în JavaScript (inclusiv *undefined*, un *thenable* sau un *promise*)
 - **Excepție**: o valoare aruncată folosind *throw*.
 - **Motiv**: o valoare care indică motivul respingerii unui promise.

JavaScript - Promise

```
new Promise( /* executor */ function(resolve, reject) { ... } );
```

- *executor*: O funcție cu doi parametri *resolve* și *reject* (funcții callback).
 - Funcția *executor* este executată imediat de implementarea Promise, transmițând funcțiile *resolve* și *reject* (Executorul este apelat înainte de ieșirea din constructorul corespunzător obiectului promise).
 - Funcțiile *resolve/reject* când sunt apelate acceptă/resping promise-ul.
 - De obicei, executorul inițiază cod asincron, iar după încheierea execuției apelează fie funcția *resolve*(succes) sau *reject* (eșec, eroare).
- Dacă o eroare este aruncată în timpul execuției executorului, promise-ul este respins (*rejected*). Valoarea returnată de executor în acest caz este ignorată.

JavaScript - Stările Promise

- Un obiect *Promise* poate fi în una din următoarele **stări**:
 - **pending**: stare inițială, încă nu a fost îndeplinit sau respins (operația asincronă încă se execută).
 - **fulfilled**: execuția operației asincrone s-a încheiat cu succes (îndeplinit).
 - **rejected**: execuția operației asincrone a eșuat (eroare) (respins).
- Un promise în starea *pending* poate fi îndeplinit cu o anumită valoare, sau poate fi respins pe baza unui motiv (eroarea).
- În oricare dintre cele două situații, unul dintre handlerele asociate folosind metoda *then* este apelat.
- Dacă un promise a fost deja îndeplinit sau respins în momentul atașării unui handler, handler-ul va fi apelat cu rezultatul execuției promise-ului.
- Un promise este **stabilit** (eng. *settled*) dacă a fost fie îndeplinit, fie respins, dar nu este în starea de pending.
- Se mai folosește termenul **rezolvat** (eng. *resolved*) - înseamnă că **obiectul promise este stabil**, sau este într-o înlanțuire de obiecte promise.

JavaScript - Promise

- Crearea unui obiect Promise:

```
var promise = new Promise(function(resolve, reject) {  
    // codul (asincron)  
  
    if (/* totul s-a executat cu succes */) {  
        resolve("Succes!");  
    }  
    else {  
        reject(Error("Eroare"));  
    }  
});
```

- Constructorul primește un singur parametru, un executor. La finalul execuției codului (asincron) se apelează fie funcția *resolve*, fie funcția *reject*.
- Se recomandă respingerea unui promise folosind un obiect de tip **Error** (posibilitatea obținerii stivei de execuție, depanare mai ușoară).

JavaScript - Promises

- Folosirea unui promise:

```
promise.then(function(result) {  
    console.log(result); // "Succes!"  
}, function(err) {  
    console.log(err); // Error: "Eroare"  
});
```

- Funcția *then()* primește doi parametri, un callback pentru execuția cu succes, și un callback pentru eșec. Ambele funcții sunt opționale, se poate adăuga doar un callback (pentru succes sau pentru eșec).
- JavaScript promises pot fi executate și în afara browserelor (ex. folosind NodeJS).
- DOM folosește promises. Funcțiile asincrone din noul DOM APIs folosesc promises (ex. ServiceWorker, Streams, etc.).

Înlănțuirea Promise

- Obiectele promise pot fi înlănțuite, pentru a transforma valorile obținute sau pentru a executa asincron alt cod, unul după altul.

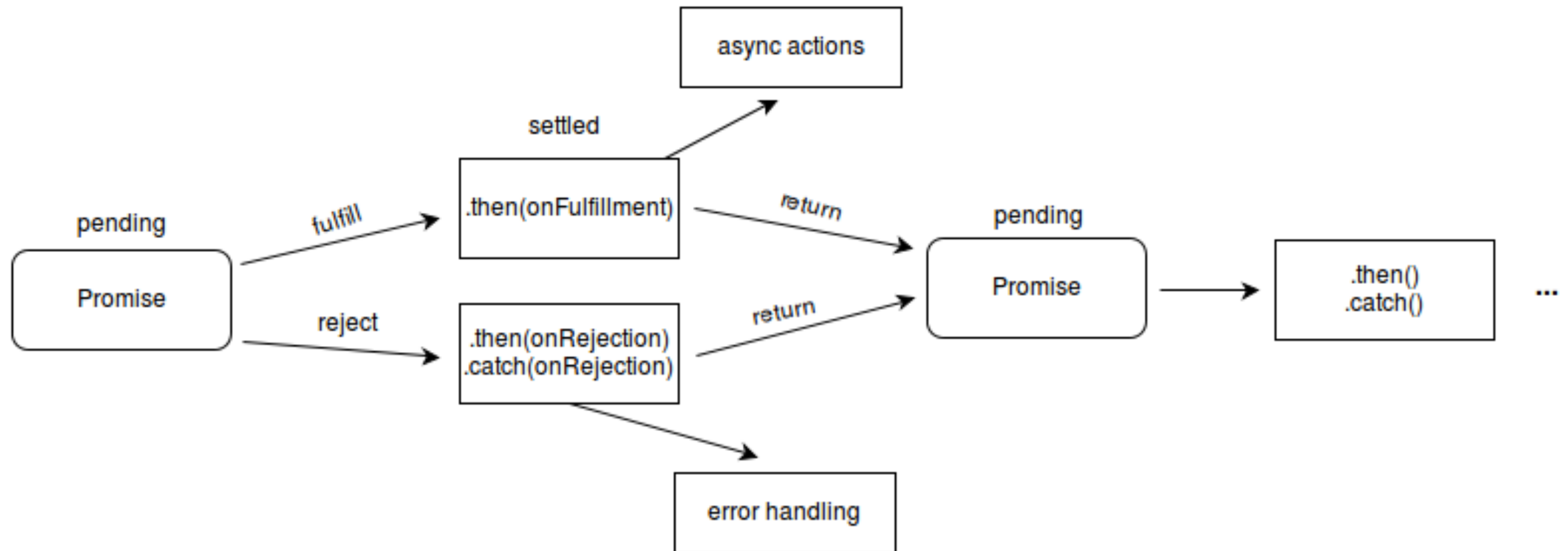
```
var promise = new Promise(function(resolve, reject) {  
    resolve(1);  
});
```

```
promise.then(function(val) {  
    console.log(val); // 1  
    return val + 2;  
}).then(function(val) {  
    console.log(val); // 3  
})
```

```
// ———
```

```
get('url/story.json').then(function(response) {  
    console.log("Success!", response);    //response e in format text  
})
```

Înlănțuire Promises (*chaining*)



JavaScript - Promises chaining

```
get('url/story.json').then(function(response) {  
    return JSON.parse(response);  
}).then(function(response) {  
    console.log("Format JSON!", response);  
})
```

- *JSON.parse()* primește un singur parametru și returnează o valoare transformată, se poate folosi și formatul:

```
get('url/story.json').then(JSON.parse).then(function(response) {  
    console.log("Format JSON:", response);  
})
```

```
function getJSON(url) {  
    return get(url).then(JSON.parse);  
}
```

- Funcția *getJSON()* returnează un alt promise, care conține obiectul JSON obținut după parsarea răspunsului.

Promises - Înlănțuirea acțiunilor asincrone

- Înlănțuirea folosind funcția *then* poate fi folosită și pentru a executa secvențial mai multe operații asincrone (contează ordinea executării lor).
- Dacă metoda *then()* returnează o valoare, următorul apel al funcției *then* primește valoarea respectivă ca și parametru.
- Dacă returnează un alt obiect promise, următorul apel al funcției *then* așteaptă terminarea execuției acestuia, și va fi apelat când obiectul devine *settled* (succes/eșec).

```
getJSON( 'url/story.json' ).then( function( story ) {  
    return getJSON( story.chapterUrls[0] );  
} ).then( function( chapter1 ) {  
    console.log( "Got chapter 1!", chapter1 );  
} )
```

Promises -Tratarea excepțiilor

- Funcția *then()* primește doi parametri: callback succes și callback eșec:

```
get( 'url/story.json' ).then(function(response) {  
    console.log("Success!", response);  
}, function(error) {  
    console.log("Failed!", error);  
})
```

- Se poate folosi și funcția *catch()*:

```
get( 'url/story.json' ).then(function(response) {  
    console.log("Success!", response);  
}).catch(function(error) {  
    console.log("Failed!", error);  
})
```

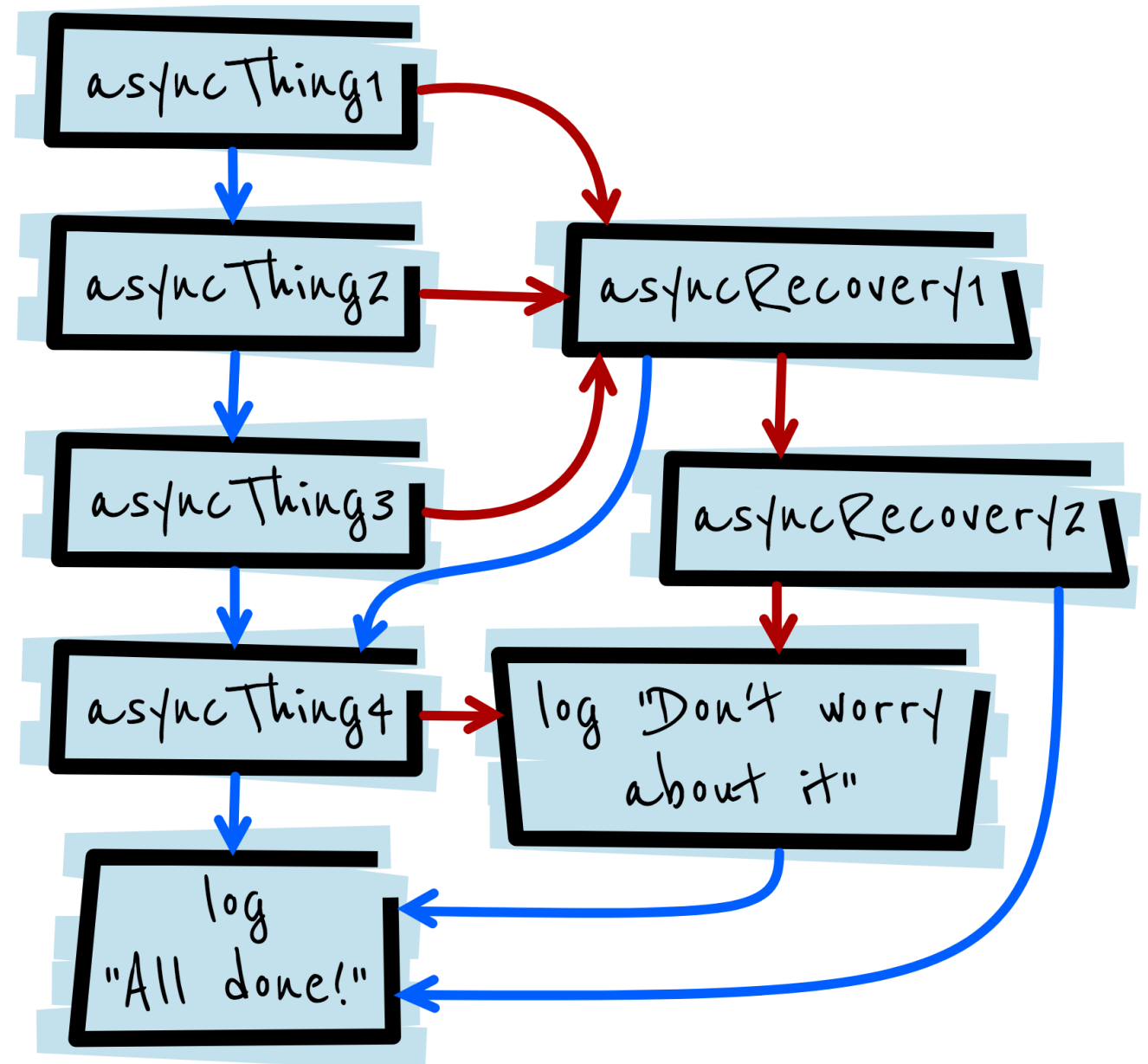
- *Catch()* este sinonim cu *then(undefined, func)*, dar este mai ușor de înțeles.
- Cele două exemple de cod, **nu sunt echivalente** (ultimul este echivalent cu):

```
get( 'url/story.json' ).then(function(response) {  
    console.log("Success!", response);  
}).then(undefined, function(error) {  
    console.log("Failed!", error);  
})
```

Promises - Tratarea excepțiilor

- Dacă un promise este respins (rejected), execuția se mută la următorul apel *then* care conține un callback pentru cazul respingerii, sau până la întâlnirea unui apel *catch*.
- În situația *then(func1, func2)*, se apelează *func1* sau *func2*, dar nu amândouă.
- În situația *then(func1).catch(func2)*, amândouă funcțiile vor fi apelate dacă *func1* este respinsă (fac parte din obiecte promise diferite)

```
asyncThing1().then(function() {  
  return asyncThing2();  
}).then(function() {  
  return asyncThing3();  
}).catch(function(err) {  
  return asyncRecovery1();  
}).then(function() {  
  return asyncThing4();  
}, function(err) {  
  return asyncRecovery2();  
}).catch(function(err) {  
  console.log("Don't worry about it");  
}).then(function() {  
  console.log("All done!");  
})
```



JavaScript excepții și promises

- Respingerea are loc când un obiect promise este respins explicit (prin apelul callbackului *reject*), dar și când o eroare este aruncată în executorul obiectului promise:

```
var jsonPromise = new Promise(function(resolve, reject) {  
  // JSON.parse aruncă eroare dacă stringul nu este în format json  
  // JSON invalid, obiectul promise este respins (rejected) implicit:  
  resolve(JSON.parse("Nu e format JSON"));  
});
```

```
jsonPromise.then(function(data) {  
  // NU va fi apelat niciodată:  
  console.log("Succes!", data);  
}).catch(function(err) {  
  // Va fi apelat:  
  console.log("Esec!", err);  
})
```

- Se recomandă execuția codului asincron în executor, astfel erorile vor fi prinse și vor genera automat respingerea.

JavaScript excepții și promises

- Aceeași situație pentru erorile aruncate în funcțiile callback transmise funcției then():

```
get('/').then(JSON.parse)
  .then(function(data) {
    // Eroare, '/' este o pagina HTML, nu este in format JSON
    // JSON.parse arunca exceptie
    console.log("Succes!", data);
  }).catch(function(err) {
    // Se va executa:
    console.log("Esec!", err);
  })
```

JavaScript Promises

- Funcția *Promise.resolve(val)*, creează un promise care se va îndeplini cu valoarea transmisă ca și parametru.
 - Dacă parametrul este un alt promise, acesta va fi returnat.
 - Exemplu: `Promise.resolve('Ana')` creează un obiect promise, care va fi îndeplinit cu valoarea 'Ana'.
 - Dacă este apelat fără parametri, va fi îndeplinit cu valoarea *"undefined"*.
- Funcția *Promise.reject(val)*, creează un promise care va fi respins cu valoarea transmisă ca și parametru (sau *undefined*).
- Funcția *Promise.all* primește un tablou de obiecte promise și creează un obiect promise care va fi îndeplinit dacă toate sunt îndeplinite. Valoarea este un tablou cu rezultatele obținute la îndeplinirea obiectelor promise, în ordinea acestora din tabloul inițial.

```
Promise.all(arrayOfPromises).then(function(arrayOfResults) {  
    // ...  
})
```

JavaScript Promises

Constructor

```
new Promise(function(  
  resolve, reject) {});
```

resolve(thenable)

Your promise will be fulfilled/rejected with the outcome of **thenable**

resolve(obj)

Your promise is fulfilled with **obj**

reject(obj)

Your promise is rejected with **obj**. For consistency and debugging (e.g., stack traces), **obj** should be an **instanceof Error**. Any errors thrown in the constructor callback will be implicitly passed to **reject()**.

Instance Methods

```
promise.then(  
  onFulfilled,  
  onRejected)
```

onFulfilled is called when/if "promise" resolves. **onRejected** is called when/if "promise" rejects. Both are optional, if either/both are omitted the next **onFulfilled/onRejected** in the chain is called. Both callbacks have a single parameter, the fulfillment value or rejection reason. **then()** returns a new promise equivalent to the value you return from **onFulfilled/onRejected** after being passed through **Promise.resolve**. If an error is thrown in the callback, the returned promise rejects with that error.

```
promise.catch(  
  onRejected)
```

Sugar for **promise.then(undefined, onRejected)**

JavaScript Promises

- Metode statice

Method summaries	
Promise.resolve(promise);	Returns promise (only if promise.constructor == Promise)
Promise.resolve(thenable);	Make a new promise from the thenable. A thenable is promise-like in as far as it has a <code>`then()`</code> method.
Promise.resolve(obj);	Make a promise that fulfills to obj . in this situation.
Promise.reject(obj);	Make a promise that rejects to obj . For consistency and debugging (e.g. stack traces), obj should be an instanceof Error .
Promise.all(array);	Make a promise that fulfills when every item in the array fulfills, and rejects if (and when) any item rejects. Each array item is passed to Promise.resolve , so the array can be a mixture of promise-like objects and other objects. The fulfillment value is an array (in order) of fulfillment values. The rejection value is the first rejection value.
Promise.race(array);	Make a Promise that fulfills as soon as any item fulfills, or rejects as soon as any item rejects, whichever happens first.

JavaScript Promises async/await

- ***async*** definește o funcție asincronă. Rezultatul funcției va fi implicit un Promise.

```
async function name([param[, param[, ... param]]) {  
    statements  
}
```

- Operatorul ***await*** așteaptă rezultatul execuției unui Promise. **Poate fi folosit doar în interiorul unei funcții asincrone.**

```
[value] = await expression;
```

- *expression* poate fi un Promise sau orice valoare
- Dacă *expression* nu este un Promise, ea este convertită folosind `Promise.resolve(expression)`
- *value* va primi rezultatul execuției cu succes a Promise-ului sau valoarea expresiei (dacă expresia nu este un Promise)
- Dacă Promise-ul este respins, expresia *await* aruncă excepție cu valoarea respingerii.

JavaScript Promises async/await

```
function resolveAfter2Seconds(x) {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      resolve(x);  
    }, 2000);  
  });  
}
```

```
async function f1() {  
  var x = await resolveAfter2Seconds(10);  
  console.log(x); // 10  
}
```

```
f1();
```

JavaScript -Fetch

- Funcția `fetch()` permite executarea unor apeluri prin rețea asemănătoare cu `XMLHttpRequest` (XHR).
- Diferența: `Fetch API` folosește obiecte `promise`, care permit scrierea unui cod mai simplu și mai clar, evitând callbackurile complexe și API complex al `XMLHttpRequest`.
- Exemplu `XMLHttpRequest`: cererea unui URL, obținerea unui răspuns și parsarea lui ca și JSON.

```
function reqListener() {  
    var data = JSON.parse(this.responseText);  
    console.log(data);  
}
```

```
function reqError(err) {  
    console.log('Eroare:', err);  
}
```

```
var oReq = new XMLHttpRequest();  
oReq.onload = reqListener;  
oReq.onerror = reqError;  
oReq.open('get', './api/some.json', true);  
oReq.send();
```

JavaScript -Fetch

- Solutia fetch()

```
fetch('./api/slots/1234')  
  .then(  
    function(response) {  
      if (response.status !== 200) {  
        console.log('Eroare. Status Code: ' + response.status);  
        return;  
      }  
    }  
  )
```

```
    // Examinarea textului din răspuns  
    response.json().then(function(data) {  
      console.log(data);  
    });  
  }  
)  
  .catch(function(err) {  
    console.log('Eroare Fetch ', err);  
  });  
});
```

- Rezultatul unui apel *fetch()* este un obiect de tip Stream. Se poate apela metoda *json()*, iar rezultatul va fi un obiect de tip Promise, deoarece citirea streamului se va face asincron.

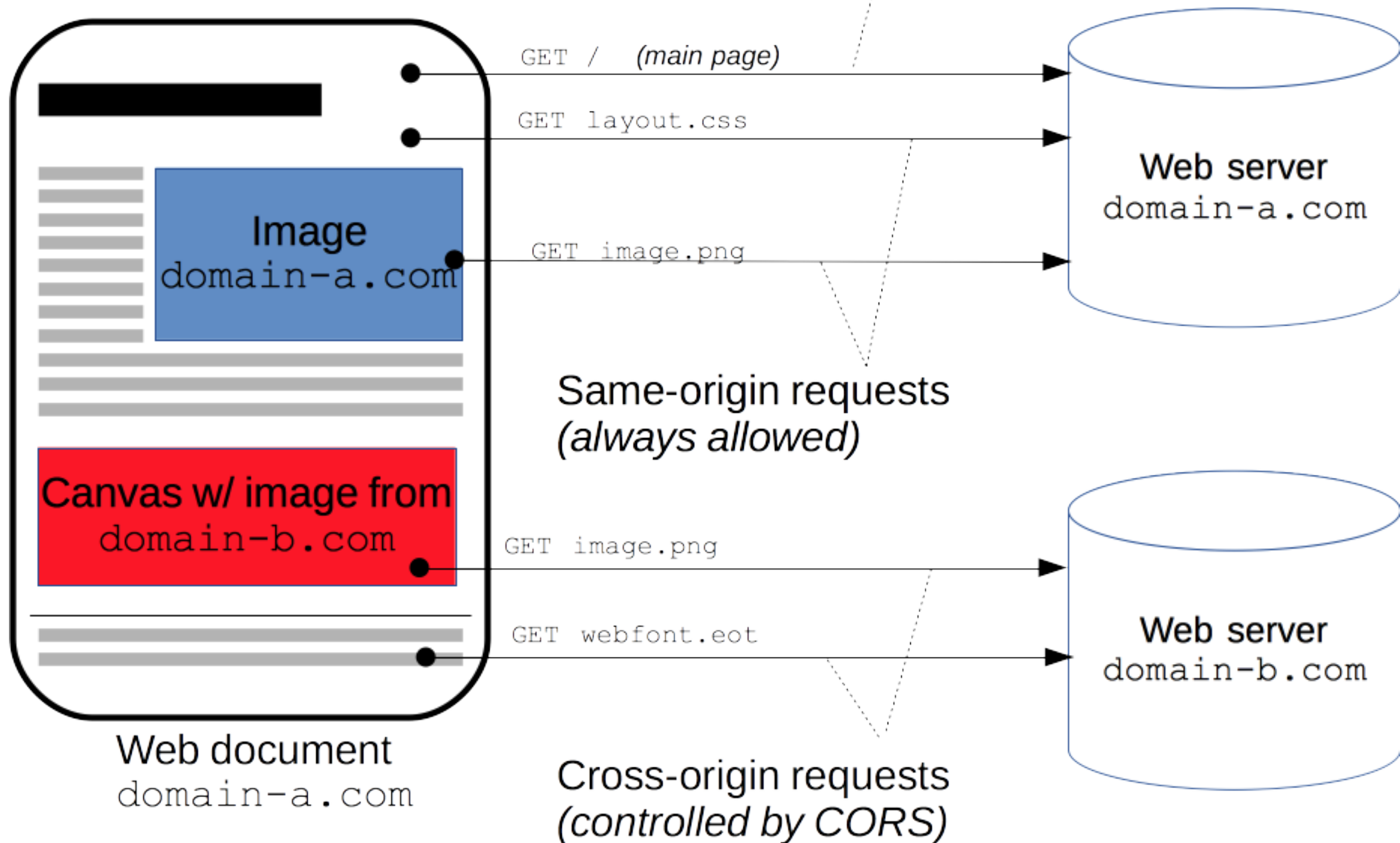
Fetch Response

- Apelul funcției *fetch* returnează un obiect de tip **response**, dacă obiectul de tip *promise* asociat este îndeplinit.
- Proprietăți utile:
 - **Response.status** - status code asociat răspunsului (întreg, implicit 200).
 - **Response.statusText** - textul asociat status code-ului respectiv (string, implicit "OK").
 - **Response.ok** - permite verificarea rapida daca status-ul este între 200-299 (boolean).
- Alte informații care pot fi obținute (ex. antetele):

```
fetch('http://localhost:8080/users').then(function(response) {  
  console.log(response.headers.get('Content-Type'));  
  console.log(response.headers.get('Date'));  
  
  console.log(response.status);  
  console.log(response.statusText);  
  console.log(response.type);  
  console.log(response.url);  
});
```

Cross Origin Resource Sharing (CORS)

Main request: defines origin.



Fetch Response Types

- Când se face o cerere fetch, răspunsul va primi un tip (*response.type*) de tip "*basic*", "*cors*" sau "*opaque*". Aceste tipuri indică de unde provine resursa și poate fi folosit pentru a decide modalitatea de tratare a obiectului de tip response.
 - **basic**: se face o cerere pentru o resursă având aceeași origine ca și cererea. Nu există restricții asupra informațiilor ce pot fi obținute din răspuns.
 - **cors**: se face o cerere pentru o resursă a cărei origine este diferită și care returnează **Cross Origin Resource Sharing (CORS)** în antet. Un răspuns de tip *cors* restricționează antetele care pot fi accesate la `Cache-Control`, `Content-Language`, `Content-Type`, `Expires`, `Last-Modified`, și `Pragma`.
 - **opaque**: se face o cerere pentru o resursă având altă origine și care nu returnează CORS în antet. La acest tip de răspuns nu se pot obține datele returnate sau status-ul răspunsului.
 - Nu se poate verifica dacă cererea s-a efectuat cu succes.

Fetch Response Types

- Se poate defini un mod pentru o cerere fetch, astfel încât doar anumite cereri vor fi apelate:
 - *same-origin*: se execută cu succes doar pentru cereri având aceeași origine, alte cereri vor fi respinse.
 - *cors*: permite cereri având aceeași origine, sau către alte origini care conțin antetele CORs corespunzătoare.
 - *cors-with-forced-preflight*: se verifică anterior că cererea poate fi făcută.
 - *no-cors*: pentru cereri către alte origini care nu au setat antetul CORS, răspunsul va fi de tip opaque.
- Pentru definirea modului se adaugă un parametru cererii fetch:

```
fetch('http://some-site.com/cars/233', {mode: 'cors'})  
  .then(function(response) {  
    return response.text();  
  })  
  .then(function(text) {  
    console.log('Request successful', text);  
  })  
  .catch(function(error) {  
    log('Request failed', error)  
  });
```


Fetch - înlănțuirea promise

- Pași cerere HTTP:
 - verificarea statusului răspunsului
 - parsarea conținutului pentru a obține obiectul JSON.
- Soluție care necesită doar folosirea informației obținute.

```
function status(response) {  
  if (response.status >= 200 && response.status < 300) {  
    return Promise.resolve(response)  
  } else {  
    return Promise.reject(new Error(response.statusText))  
  }  
}
```

```
function json(response) {  
  return response.json()  
}
```

```
fetch('api/users')  
  .then(status)  
  .then(json)  
  .then(function(data) {  
    console.log('Succes - raspuns JSON:', data);  
  }).catch(function(error) {  
    console.log('Cerere esuata', error);  
  });
```

Fetch - Verificarea răspunsului

- Un obiect promise *fetch()* va respinge cu un obiect de tip `TypeError`, când apare o eroare de rețea.
- Un apel al funcției *fetch* ar trebui să verifice execuția cu succes a acesteia, verificând că obiectul de tip promise a fost îndeplinit, iar apoi verificând valoarea proprietății `Response.ok`.
- Exemplu:

```
fetch('api/users').then(function(response) {  
  if(response.ok) {  
    return response.json();  
  }  
  throw new Error('Raspunsul nu a fost ok.');
```

```
}).then(function(data) {  
  //folosirea informatiei  
}).catch(function(error) {  
  console.log('Eroare cu functia fetch: ' + error.message);  
});
```

Fetch - Request

- Funcția *fetch()* poate fi apelată și folosind un parametru de tip Request.
- `Request()` primește aceeași parametrii ca și funcția `fetch()`.

```
var myHeaders = new Headers();
```

```
var myInit = { method: 'GET',  
               headers: myHeaders,  
               mode: 'cors',  
               cache: 'default' };
```

```
var myRequest = new Request('api/users', myInit);
```

```
fetch(myRequest).then(function(response) {  
    return response.json();  
}).then(function(myData) {  
    console.log(myData);  
});
```

- Observație: **Parametrii de tip query trebuie adăugați explicit la cerere (url, antet).**
Nu există metode speciale care permit transmiterea acestora.

Antete Fetch

- *Headers* permite crearea/păstrarea/accesarea antetelor unei cereri/unui răspuns.
- Este un dicționar de perechi (nume-antet, valoare-antet):

```
var content = "Hello World";  
var myHeaders = new Headers();  
myHeaders.append("Content-Type", "text/plain");  
myHeaders.append("Content-Length", content.length.toString());  
myHeaders.append("X-Custom-Header", "ProcessThisImmediately");
```

- Se poate transmite și un tablou conținând valorile antetelor:

```
myHeaders = new Headers({  
  "Content-Type": "text/plain",  
  "Content-Length": content.length.toString(),  
  "X-Custom-Header": "ProcessThisImmediately",  
});  
console.log(myHeaders.has("Content-Type")); // true  
console.log(myHeaders.has("Set-Cookie")); // false  
myHeaders.set("Content-Type", "text/html");  
myHeaders.append("X-Custom-Header", "AnotherValue");  
  
console.log(myHeaders.get("Content-Length")); // 11  
console.log(myHeaders.get("X-Custom-Header")); // ["ProcessThisImmediately", "AnotherValue"]  
  
myHeaders.delete("X-Custom-Header");  
console.log(myHeaders.get("X-Custom-Header")); // [ ]
```

Fetch - Body

- Cererile și răspunsurile pot conține informație/date.
- Informația poate fi de următoarele tipuri:
 - ArrayBuffer
 - ArrayBufferView
 - Blob/File
 - string
 - URLSearchParams
 - FormData
- Sunt definite metode pentru obținerea informației dintr-un răspuns. Toate metodele returnează un obiect de tip promise, care conțin informația.
 - `arrayBuffer()`
 - `blob()`
 - `json()`
 - `text()`
 - `formData()`

Fetch - Feature detection

- Suportul pentru Fetch API poate fi detectat prin verificarea existenței (în fereastră sau scopul workerului) a obiectelor:
 - Headers
 - Request
 - Response
 - funcția fetch().

Exemplu

```
if (self.fetch) {  
    // execuția cererii fetch  
} else {  
    // folosirea XMLHttpRequest?  
}
```

Medii de proiectare și programare

2024-2025

Curs 12

Continut

- Framework web - REACT
- Asynchronous messaging
 - ActiveMQ -JMS, Spring JMS
 - AQMP - RabbitMQ

React

- React este o bibliotecă JavaScript declarativă, flexibilă și eficientă care permite construirea ușoară și rapidă a interfețelor cu utilizatorul.
- Permite dezvoltatorilor să creeze aplicații web mari, care folosesc date ce se pot modifica în timp, fără a reîncărca toată pagina.
- Obiectivele principale: rapiditate, simplitate, scalabilitate.
- React procesează doar interfețe grafice în aplicații.
 - Corespunde View-ului din șablonul Model-View-Controller (MVC) și poate fi folosit împreună cu alte biblioteci sau frameworkuri JavaScript din MVC (ex. Angular).
- Este dezvoltat și întreținut de o comunitate formată din dezvoltatori de la Facebook, Instagram și dezvoltatori individuali.
- Folosit pentru site-uri precum Netflix, Airbnb, Walmart, etc.
- Mai multe versiuni disponibile
 - React 19 with hooks (latest)

React - Elemente

- Elementele React sunt cele mai mici construcții ale unei aplicații React. Un element descrie ceea ce trebuie afișat pe ecran.
- Elementele React sunt obiecte simple și ușor de creat. React DOM se ocupă de actualizarea DOM pentru potrivirea cu elementele React.

```
const element = <h1>Hello, world</h1>;
```

- Este numit un nod DOM, pentru că tot ce este conținut de el va fi gestionat de React DOM.
- Aplicațiile dezvoltate folosind React conțin de obicei un singur nod root.
- Pentru afișarea unui element React într-un nod root:

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <h1>Hello, world</h1>  
  </React.StrictMode>  
);
```

- Actualizarea unui element: **elementele React sunt *immutable***. Un element odată creat, datele sale nu se mai pot modifica.
- Majoritatea aplicațiilor React apelează *render()* o singură dată.
- Biblioteca React actualizează la afișare doar datele modificate. React DOM compară un element și descendenții săi cu datele anterioare și invocă doar actualizările necesare pentru ca DOM să aibă starea dorită.

JSX

```
//nu este nici un string, nici HTML.  
const element = <h1>Hello, world!</h1>;
```

- Sintaxa este numită **JSX**, și este o extensie a sintaxei JavaScript.
- Mai strictă decât HTML (
).
- Este modul recomandat de descriere în React a interfețelor grafice.
- JSX produce elemente React.
- O componenta poate returna doar un singur element JSX.
- Dacă trebuie returnate mai multe se recomandă folosirea <div>...</div> sau <> ... </>
- Se pot include orice expresii JavaScript în JSX, prin includerea lor între acolade('{ ' ' }')

```
const user = {  
  firstName: 'Popescu',  
  lastName: 'Ana'  
};
```

```
function formatName(user) {  
  return user.firstName + ' ' + user.lastName;  
}
```

```
const element = (  
  <h1>  
    Hello, {formatName(user)}!  
  </h1>  
)  
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    element  
  </React.StrictMode>  
)
```

Specificarea atributelor folosind JSX

- Se pot folosi ghilimelele pentru specificarea atributelor de tip string:

```
const element = <div tabIndex="0"></div>;
```

- Se folosesc acoladele pentru includerea unei expresii JavaScript ca și valoarea a unui atribut:

```
const element = <img src={user.avatarUrl}></img>;
```

- **Nu se pun ghilimele când se includ expresii JavaScript** (expresiile vor fi considerate stringuri)

```
const element = </img>; //!!!
```

- Dacă un tag este gol, se închide cu />, asemănător XML:

```
const element = <img src={user.avatarUrl} />;
```

- Tagurile JSX pot conține fii:

```
const element = (  
  <div>  
    <h1>Salut</h1>  
    <h2>Bine ai venit!</h2>  
  </div>  
);
```

Componente React

- React are la bază definirea unor **componente React**.
- O componentă poate păstra mai multe instanțe a altei/altor componente (relația *părinte-copil*).
- Componentele permit împărțirea UI în părți independente și reutilizabile, și dezvoltarea independentă a acestora.
- Definirea unei componente (React 18):

```
function Greeting(props){  
  
    return (<h1>Hello, {props.name}</h1>);  
    // sau return (<>Hello, {props.name}</>);  
  
    //gresit  
    //return (Hello, {props.name});  
}
```

- Rezultatul returnat de funcție specifică modul de afișare a componentei.
- **Numele componentelor încep întotdeauna cu litera mare.**
- Exemplu, `<div />` reprezintă un tag DOM, dar `<Greeting />` reprezintă o componentă și este necesar ca *Greeting* să fie disponibil în scop.
- Datele afișate de o componentă React sunt obținute prin proprietăți (*props*) sau din starea componentei (*state*).

Componente și Props

- Componentele React au două tipuri de date: **starea** și **proprietățile**.
- Când React întâlnește un element reprezentând o componentă definită de dezvoltator, îi transmite atributele JSX ca și un singur obiect (adesea numit “props”), prin parametrii funcției.
- **props** - conține proprietățile definite de componenta care a apelat această componentă.
- **Proprietățile sunt read-only**, o componentă nu trebuie să-și modifice propriile proprietăți:

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <Greeting name="Ana"/> //folosirea componentei  
  </React.StrictMode>  
);
```

```
function Greeting(props) {  
  return (<> {props.name} </>);  
}
```

```
function Greeting({name}) {  
  return (<> {name} </>);  
}
```

```
<button onClick={handleClick}> //fara paranteze când se transmite o funcție!  
  Say hello  
</button>
```

Compunerea componentelor

- Componentele pot referi alte componente.
- Permite folosirea aceluiași nivel de abstractizare a componentelor: un buton, un form, un dialog, etc sunt toate exprimate ca și componente.
- Exemplu -crearea unei componente care afișează componenta *Greeting* de mai multe ori:

```
function App(){  
  return (  
    <div>  
      <Greeting name="Ana" />  
      <Greeting name="Maria" />  
      <Greeting name="Ion" />  
    </div> );  
}
```

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```


React Component -States

- **Starea** (*eng. state*) unei componente este informația pe care componenta se așteaptă să o gestioneze singură.
- Conține date specifice componentei, care se pot modifica în timp.
- **Starea este definită de dezvoltator.** Dacă informația nu este folosită în **return**, atunci nu face parte din starea componentei.

```
import { useState } from 'react';  
function Counter(){  
  const [count, setCount] = useState(0); //count -variabila de stare, 0-valoarea initiala, setCount-functie  
  function handleClick(){  
    setCount(count+1);  
  }  
  return (<button onClick={handleClick}> Apasat de {count} ori</button>);  
}
```

- Variabila `count` va păstra o parte din starea componentei (pot fi mai multe variabile)
- *Variabilele de stare trebuie considerate immutable*, și nu trebuie modificate în mod direct.
- Orice actualizare a unei variabile de stare a componentei (ex. `count`) se face folosind funcția asociată (ex. `setCount`). De fiecare dată când `setCount()` este apelat, React actualizează starea, determină diferențele dintre starea anterioară și noua stare și injectează o mulțime de modificări DOM-ului corespunzător paginii. În acest fel actualizările UI sunt rapide și eficiente.

React Component -States

- Observații legate de starea unei componente:

- **NU se modifică starea direct.**

```
const [count, setCount] = useState(0);
```

```
count=count+1; //!eroare
```

Orice modificare se face prin funcția asociată variabilei (ex. setCount)

- **O componenta poate avea mai multe variabile de stare, fiecare având funcția ei.**

```
function UserForm(props){
```

```
  const [name, setName] = useState('');
```

```
  const [username, setUsername] = useState('');
```

```
  const [passwd, setPasswd] = useState('');
```

```
  //...  
}
```

- Batch processing pentru stare și actualizarea interfeței

```
function handleClick(){
```

```
  setName(newName);
```

```
  setUsername(newUsername)
```

```
  setPassword(newPassword)
```

```
}
```

//interfața grafică se va actualiza o singura dată

React Component -Starea

- Datele sunt transmise de la componenta părinte la descendenți.
 - Nici părinții, nici descendenții nu știu dacă o anumită componentă este *stateful* sau *stateless*.
 - Adesea, starea este numită *stare locală* sau *încapsulată*. Starea nu este disponibilă altor componente, doar componentei care a creat-o.
 - O componentă poate alege să transmită starea sa componentelor fii prin intermediul proprietăților:
- ```
function FormattedDate({date}){
 return (<h2>Data este {date.getDate()}/{date.getMonth()+1}/{date.getFullYear()}</h2>);
}

function Calendar(){
 const [date, setDate]=useState(new Date());
 return (<FormattedDate date={date}/>);
}
```
- Acest mod de a transmite informația este numit flux de date "top-down" sau "unidirectional". Orice stare este totdeauna definită de o anumită componentă, și orice informație sau UI derivat din acea stare poate afecta doar componentele descendente din arbore.
  - În aplicațiile React, faptul că o componentă este *stateful* sau *stateless* este considerat detaliu de implementare care se poate schimba în timp.
  - Componente *stateless* pot conține componente *stateful* și invers.

# Tratarea evenimentelor

- Asemănătoare cu tratarea evenimentelor corespunzătoare elementelor DOM.
- **Diferențe:**
  - Evenimentele React sunt numite folosind camelCase, nu doar litere mici.
  - Cu JSX se poate transmite o funcție ca și un event handler.

HTML:

```
<button onclick="activateLasers()">
 Activate Lasers
</button>
```

React:

```
<button onClick={activateLasers}> //fara paranteze!
 Activate Lasers
</button>
```

- Nu se poate returna *false* pentru a preveni comportamentul implicit din React. Trebuie apelată explicit metoda *preventDefault*.

HTML: pentru a preveni comportamentul implicit de a deschide o nouă pagină:

```

 Click me

```

React:

```
function ActionLink() {
 function handleClick(e) {
 e.preventDefault();
 console.log('The link was clicked.');
```

```
 }

 return (

 Click me

);
}
```

# Tratarea evenimentelor

- O altă modalitate de asociere este folosirea sintaxei arrow function:

```
function LoggingButton() {
 handleClick = () => {
 //...
 }
 //sau handleClick = (e) => { console.log(e)}

 return (
 <button onClick={handleClick}>
 Click me
 </button>
);
}
```

# Input Forms

```
function UserForm(props){

 const [name, setName] = useState('');
 const [username, setUsername] = useState('');
 const [passwd, setPasswd] = useState('');

 function handleSubmit (event){
 let user={id:username,
 name:name,
 passwd:passwd
 }
 //alte operatii
 event.preventDefault();
 }
 return(
 <form onSubmit={handleSubmit}>
 <label>
 Username:
 <input type="text" value={username} onChange={e=>setUsername(e.target.value)} />
 </label>

 <label>
 Name:
 <input type="text" value={name} onChange={e=>setName(e.target.value)} />
 </label>

 <label>
 Passwd:
 <input type="password" value={passwd} onChange={e=>setPasswd(e.target.value)} />
 </label>

 <input type="submit" value="Add user" />
 </form>);
}
```

# Crearea unei aplicații simple React

- **Instalare:** *NodeJs*: <https://nodejs.org/en/>
  - npm: package manager (inclus in nodejs)
- **Creare aplicatie:** `npm create vite@latest my-app -- -- template react`
- Numele directorului cu litere mici!
- Urmati instrucțiunile ...
  - `cd my-app`
- **Pornirea aplicației:**
  - `npm run dev` //va fi disponibilă la <http://localhost:5173/>
  - `npm run build`
- Nu e necesară configurarea folosind babel, webpack/browserify, typescript, etc.

**Important:** NU uitati sa **setați variabila PATH** pentru a include și calea către npm

# Ciclul de viață - Versiuni anterioare

- **Versiuni anterioare** -fiecare componentă avea câteva metode care erau apelate în ciclul său de viață.
- Aceste metode puteau fi redefinite pentru a particulariza componenta.
- Metodele care începeau cu 'will' erau apelate înaintea apariției evenimentului, iar cele cu 'did' erau apelate după apariția evenimentului.
- Metodele apelate când o componentă era creată și adăugată la DOM.
  - *constructor(), componentWillMount(), render(), componentDidMount()*
- *Actualizarea*: cauzată de modificări ale proprietăților sau a stării componente (metodele erau apelate când componenta era re-afișată.)
  - *componentWillReceiveProps(), shouldComponentUpdate(), componentWillUpdate(), render(), componentDidUpdate()*
- *Dezasamblarea*: metoda era apelată când componentă era ștearsă din DOM:
  - *componentWillUnmount()*



# Ciclul de viață - Versiunea curenta

- metoda `useEffects()`

```
import { useState } from 'react';
import { useEffect } from 'react';
```

```
function UserApp(){
 const [users, setUsers] = useState([{"passwd":"m","name":"Marinescu Maria","id":"maria"}]);
```

```
 //...
 useEffect(()=>
 {
 //actualizarea interfeței ...
 GetUsers().then(users=>setUsers(users));
 }, []);
```

Permite executarea unei secvențe de cod după afișarea unei componente și de fiecare data când este actualizata

- `useState`, `useEffect` - **hooks (vers. React 16.8)**

Altele:

- `useContext`
- `useId`
- `useDeferredValue`
- `useTransition`
- `useSyncExternalStore`
- `useInsertionEffect`



# React - arrow functions

```
import React, {useState} from 'react';

const App = () => {
 const greeting = 'React functional component!';
 return <Headline title={greeting} />;
};

const Headline = (props) => {
 const [votes, setVotes] = useState(0);
 const upVote = (event) => {setVotes(votes + 1)};
 return (<div>
 <h1 className="Votes">{props.title}</h1>
 <p>Votes: {votes}</p>
 <p>
 <button onClick={upVote}>Up Vote</button>
 </p>
 </div>);
}
```

# Referințe

- Jake Archibald, *JavaScript Promises: an Introduction*,

<https://developers.google.com/web/fundamentals/getting-started/primers/promises>

- Using Fetch:

[https://developer.mozilla.org/en-US/docs/Web/API/Fetch\\_API/Using\\_Fetch](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch)

- Matt Gaunt, *Introduction to fetch()*,

<https://developers.google.com/web/updates/2015/03/introduction-to-fetch>

- Documentație React, <https://react.dev/>

- Documentatie Vite, <https://vite.dev/guide/>

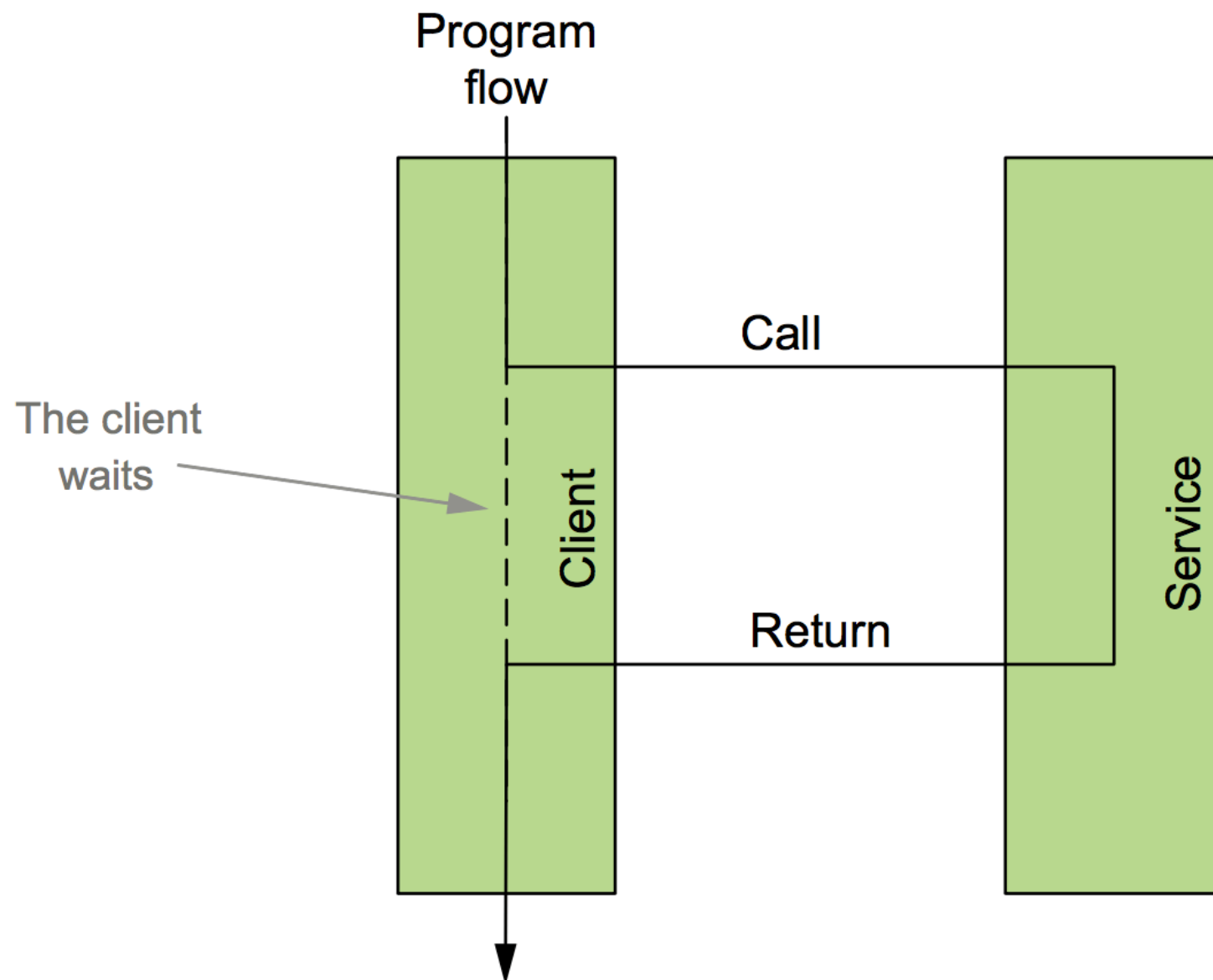
- Versiuni mai vechi React: <https://legacy.reactjs.org/> !!!

# Asynchronous messaging

- *Asynchronous messaging* este o modalitate de a transmite indirect mesaje de la o aplicație la altă aplicație, fără a aștepta un răspuns.
- Este legată de comunicarea dintre aplicații.
- Diferă de alte mecanisme/modalități de comunicare prin modul în care informația este transmisă între sisteme.
- Are câteva avantaje față de comunicarea sincronă (eng. *synchronous messaging*).

# Comunicare sincronă

- Tehnologiile RPC ca gRPC și Thrift folosesc comunicarea sincronă.
- Când un client apelează o metodă a unui obiect remote, clientul trebuie să aștepte terminarea execuției înainte de a trece mai departe.
- Chiar dacă metoda nu returnează nimic, clientul tot trebuie să aștepte terminarea apelului.

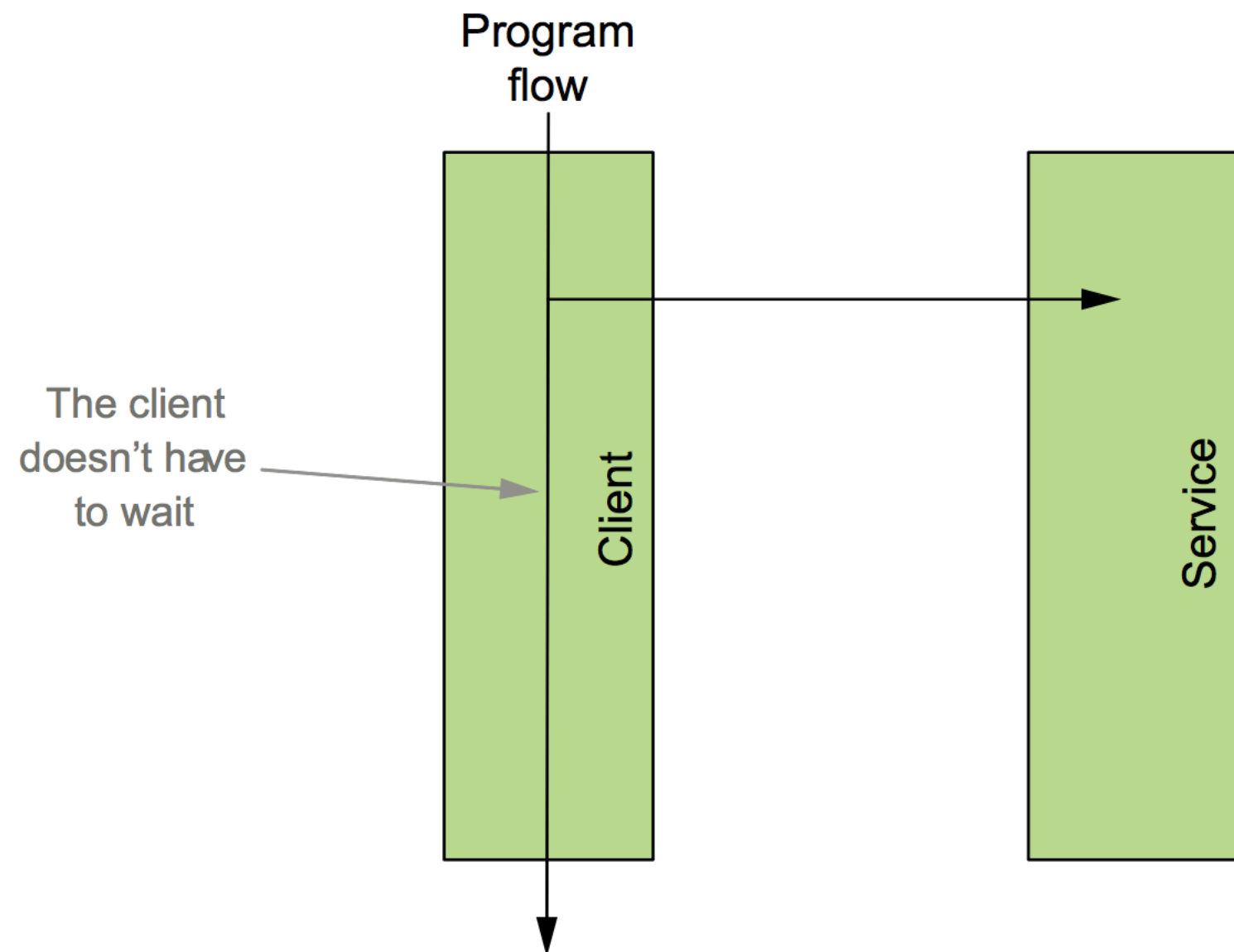


# Limitările comunicării sincrone

- Limitările clientului unui apel la distanță:
  - *Comunicarea sincronă implică așteptare.* Când un client apelează o metodă a unui obiect/serviciu remote, acesta trebuie să aștepte terminarea execuției acesteia înainte de a continua execuția. Dacă clientul face apeluri dese sau dacă apelul se execută mai lent, acest apel ar putea afecta performanța clientului.
  - *Clientul este cuplat cu serviciul prin interfața serviciului.* Dacă interfața serviciului se modifică, toți clienții acestuia trebuie să se modifice corespunzător.
  - *Clientul este dependent de locația serviciului.* Clientul trebuie configurat pentru a specifica locația serviciului (IP și port) pentru a ști unde este localizat serviciul. Dacă locația serviciului se modifică, clientul trebuie reconfigurat pentru a specifica noua locație.
  - *Clientul este dependent de disponibilitatea (eng. availability) serviciului.* Dacă serviciul devine indisponibil, clientul va eșua.
- Aceste limitări trebuie luate în considerare în momentul deciderii folosirii tipului de comunicare de mesaje potrivit aplicației.

# Comunicarea asincronă

- Când **mesajele** sunt **trimise asincron**, **clientul nu trebuie să aștepte terminarea procesării mesajului sau livrarea mesajului**.
- Clientul trimite mesajul și trece la execuția următoarei instrucțiuni, presupunând că serviciul va primi și va prelucra la un moment dat mesajul trimis.
- Acest tip de comunicare are câteva avantaje asupra comunicării sincrone.



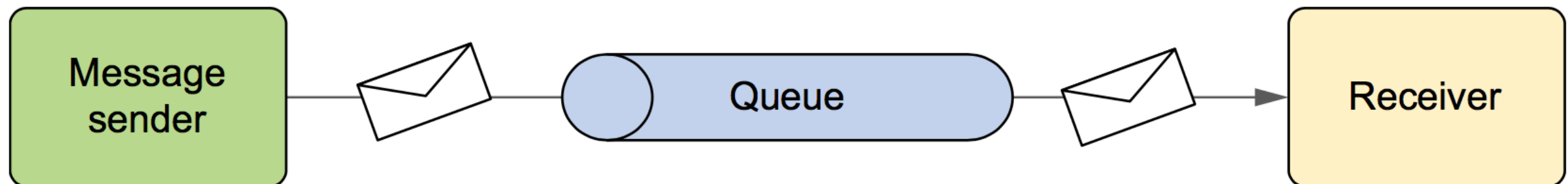
# Asynchronous messaging

- Când o aplicație trimite un mesaj altei aplicații, **nu există o legătură directă** între cele două aplicații.
- Aplicația care trimite mesajul, îl trimite unui serviciu care asigură livrarea acestuia aplicației care îl așteaptă.
- Există doi actori principali: **brokerii de mesaje** și **destinațiile**.
- Când o aplicație trimite un mesaj, îl trimite unui broker de mesaje.
- Brokerul de mesaje asigură livrarea mesajului la destinația specificată, permițând expeditorului continuarea execuției.
- Mesajele trimise asincron au specificată o destinație.
- Destinația specifică doar locația de unde mesajele pot fi preluate, nu și cine le va prelua.
- Tipuri diferite de sisteme de trimitere a mesajelor asincrone pot folosi scheme diferite de rutare a acestora.
- Există două tipuri de bază de destinații: **cozi** și **topic-uri**.
- Fiecare tip de destinație are asociat un model specific de trimitere a mesajelor:
  - **point-to-point** (corespunzător *cozilor*)
  - **publish/subscribe** (corespunzător *topicurilor*).



# Point-To-Point Model

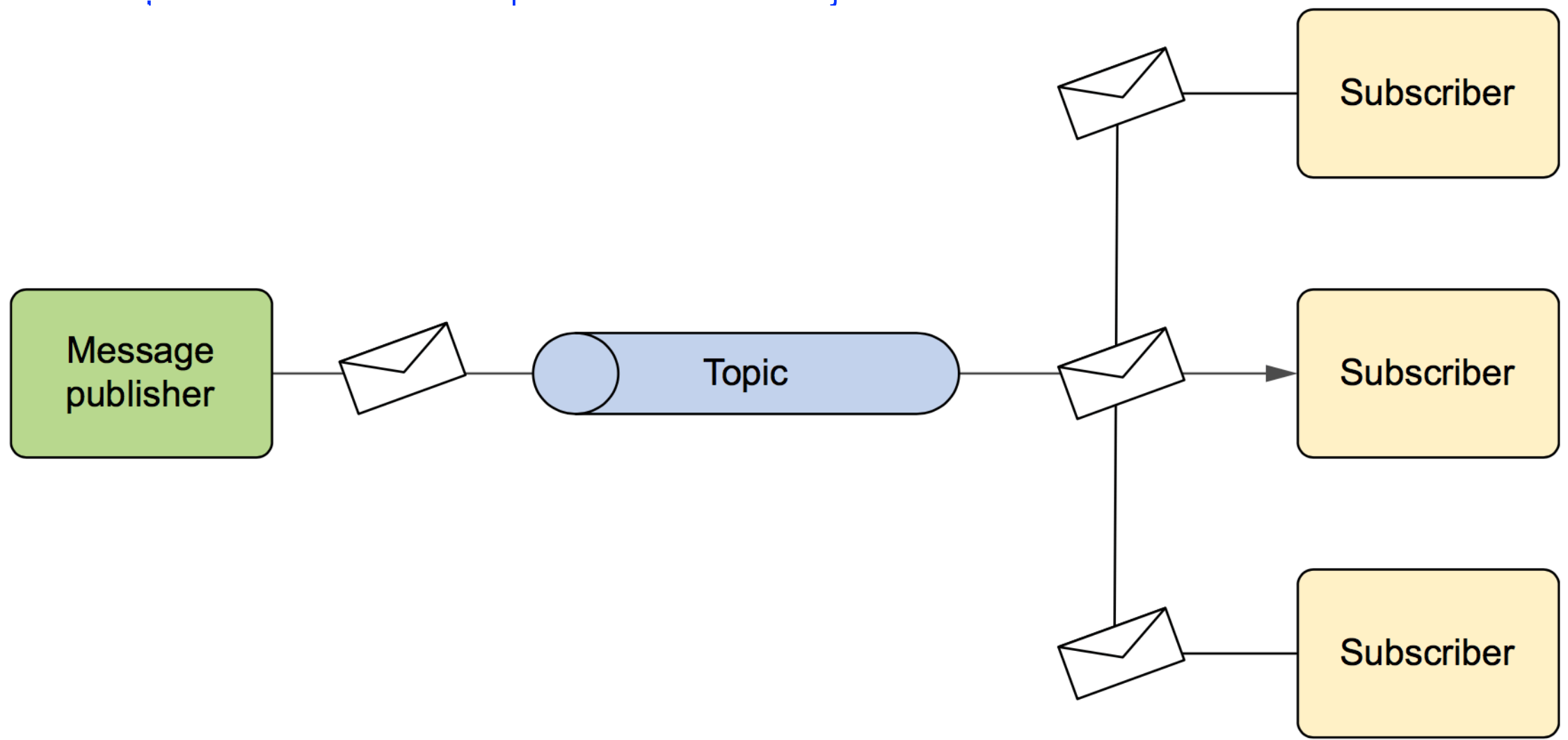
- În **modelul point-to-point**, *fiecare mesaj are exact un expeditor și un destinatar*. Când brokerul de mesaje primește un mesaj, pune mesajul într-o coadă. Când destinatarul cere următorul mesaj, mesajul este scos din coadă și trimis acestuia.
- Deoarece mesajul este șters din coadă în momentul trimiterii, *se garantează livrarea acestuia unui singur destinatar*.
- Chiar dacă fiecare mesaj din coada de mesaje este livrat unui singur destinatar, nu înseamnă că doar un singur destinatar scoate mesaje din coadă.
- Este posibil ca mai mulți destinatari să proceseze mesaje din coadă, dar fiecare va primi propriul mesaj.
- **Dacă mai mulți destinatari așteaptă procesarea unui mesaj din coadă, nu se știe exact ce mesaj va procesa fiecare.** Acest lucru permite aplicațiilor să mărească capacitatea de procesare a mesajelor prin adăugarea unui nou destinatar.





# Publish-Subscribe Model

- In **modelul publish/subscribe**, mesajele sunt trimise unui *topic*. Asemănător cozilor, mai mulți destinatari pot aștepta livrarea unor mesaje de la un topic. Spre deosebire de cozi, **când un mesaj este trimis unui topic toți destinatarii care așteaptă vor primi o copie a mesajului respectiv**.
- Expeditorul (publisher) nu știe care sunt destinatarii (subscribers). El știe doar că mesajul va fi trimis unui topic, nu știe cine așteaptă mesaje la topicul respectiv.
- Expeditorul nu știe nici cum vor fi procesate mesajele.



# Avantajele comunicării asincrone

- ***Nu există timpi de așteptare***

- Când un mesaj este trimis asincron, **clientul nu trebuie să aștepte livrarea sau procesarea mesajului**. Clientul trimite mesajul brokerului de mesaje, cu încrederea că mesajul va ajunge la destinația specificată.
- Deoarece nu trebuie să aștepte, clientul poate executa alte sarcini, îmbunătățind performanța acestuia.

- ***Orientată pe mesaje și decuplată***

- Spre deosebire de comunicarea RPC (orientată pe apelul unei metode), **mesajele trimise asincron sunt centrate pe date**. Clientul nu este dependent de o anumită semnătură a metodei. **Fiecare abonat la o coadă sau un topic care poate procesa datele trimise de client, poate procesa mesajul**. **Clientul nu trebuie să știe particularitățile specifice serviciului**.

- ***Independența locației***

- Serviciile RPC sincrone sunt localizate de obicei prin adresa lor (IP și port), clienții fiind afectați de orice modificare a locației. Dacă adresa IP sau portul se modifică, clientul trebuie modificat corespunzător, altfel clientul nu va mai putea accesa serviciul.

# Avantajele comunicării asincrone

- **Independența locației (cont.)**

- In contrast, clienții sistemelor bazate pe mesaje nu știu ce serviciu va procesa mesajele lor sau locația acestora. Clientul știe doar de coada sau topicul prin intermediul căreia/căruia mesajul va fi transmis. Nu contează locația serviciului, atâta timp cât acesta poate procesa mesajele din coadă sau topic.
- Servicii multiple se pot abona la un singur topic, fiecare primind o copie a aceluiași mesaj, dar fiecare poate procesa în mod diferit mesajul. Fiecare serviciu va folosi în mod independent aceeași informație primită de la topic.
- În modelul point-to-point, se pot crea clustere de servicii. Dacă clientul nu este dependent de locația serviciului, și singura cerință legată de serviciu este că acesta trebuie să poată accesa brokerul de mesaje, mai multe servicii pot fi configurate să primească mesaje de la aceeași coadă. Dacă la un moment dat un serviciu devine suprasolicitat, se pot porni noi instanțe ale aceluiași serviciu care se abonează la aceeași coadă.

# Avantajele comunicării asincrone

- **Livrare garantată**

- Pentru ca un client să poată comunica cu un serviciu sincron, serviciul trebuie să fie disponibil la adresa și portul specificate. Dacă serviciul devine indisponibil (temporar sau pe o perioadă nedefinită), clientul nu își poate continua execuția.
- Dacă mesajele sunt trimise asincron, clientul are garanția că mesajele sale vor fi livrate. Chiar dacă un serviciu nu este disponibil în momentul trimiterii mesajului, mesajul va fi păstrat de brokerul de mesaje până când serviciul va deveni din nou disponibil.
- Disponibilitatea brokerului de mesaje

# Brokeri de Mesaje

- **Apache ActiveMQ** (<http://activemq.apache.org/>)
  - Open source
  - Suportă clienți din limbaje și protocoale diferite: Java, C, C++, C#, Ruby, Perl, Python, PHP.
  - Oferă suport pentru frameworkul Spring, ActiveMQ putând fi ușor integrat și configurat în aplicații bazate pe Spring.
  - Suportă concepte avansate: grupuri de mesaje, destinații virtuale, destinații compuse
- **RabbitMQ** (<https://www.rabbitmq.com/>)
  - Open source
  - Simplu și ușor de integrat și folosit (și in cloud).
  - Suportă mai multe protocoale de comunicare bazate pe mesaje.
  - Poate fi rulat pe diferite sisteme de operare și în cloud. Oferă instrumente de dezvoltare pentru diferite limbaje de programare (Java, .NET, PHP, Ruby, Python, etc).
- **Apache Kafka, Amazon MQ**, etc

# ActiveMQ

- Instalarea:
  - Arhiva <http://activemq.apache.org/>
- Pornirea:
  - MacOS: *activemq start*
- Verificarea pornirii corecte:
  - <http://127.0.0.1:8161/admin>
  - Cont: user *admin*; pass *admin*
- Oprirea
  - MacOS: *activemq stop*

# Java Message Service (JMS)

- Un standard Java care definește o interfață comună pentru lucrul cu brokeri de mesaje.
- Înainte de apariția JMS, fiecare broker de mesaje avea propriul API făcând codul aplicațiilor dependent de broker și greu de reutilizat pentru brokeri diferiți.
- Folosind JMS, toți brokerii de mesaje pot fi accesați folosind aceeași interfață comună (asemănător JDBC).



# Trimiterea unui mesaj

```
ConnectionFactory cf = new ActiveMQConnectionFactory("tcp://localhost:61616");
Connection conn = null;
Session session = null;
try {
 conn = cf.createConnection(); conn.start();
 session = conn.createSession(false, Session.AUTO_ACKNOWLEDGE);
 Destination destination = new ActiveMQQueue("test.queue");
 MessageProducer producer = session.createProducer(destination);
 TextMessage message = session.createTextMessage();
 message.setText("Hello world!");
 producer.send(message); //trimiterea mesajului
} catch (JMSException e) {
 // handle exception ...
} finally {
 try {
 if (session != null) { session.close(); }
 if (conn != null) { conn.close(); }
 } catch (JMSException ex) { //... tratarea exceptiilor
 }
}
```



# Citirea unui mesaj

```
ConnectionFactory cf = new ActiveMQConnectionFactory("tcp://localhost:61616");
Connection conn = null;
Session session = null;
try {
 conn = cf.createConnection();
 conn.start();
 session = conn.createSession(false, Session.AUTO_ACKNOWLEDGE);
 Destination destination = new ActiveMQQueue("test.queue");
 MessageConsumer consumer = session.createConsumer(destination);
 Message message = consumer.receive();
 TextMessage textMessage = (TextMessage) message;
 System.out.println("Primit mesaj: " + textMessage.getText());
} catch (JMSEException e) {
 // handle exception ...
} finally {
 try {
 if (session != null) { session.close(); }
 if (conn != null) { conn.close(); }
 } catch (JMSEException ex) { //... tratarea exceptiilor
 }
}
```

# Spring JMS

- Crearea unui *ConnectionFactory*

- **ActiveMQConnectionFactory** este clasa JMS connection factory oferită de ActiveMQ

```
<bean id="connectionFactory"
 class="org.apache.activemq.spring.ActiveMQConnectionFactory" />
```

Implicit, **ActiveMQConnectionFactory** presupune că brokerul de mesaje așteaptă la locația **localhost** pe portul **61616**.

```
<bean id="connectionFactory"
 class="org.apache.activemq.ActiveMQConnectionFactory">
 <property name="brokerURL" value="tcp://localhost:61616"/>
</bean>
```

Schimbarea locației implicite se poate face folosind proprietatea **brokerURL** a brokerului de mesaje.

# Spring JMS

- Declararea unei destinații pentru mesaje folosind ActiveMQ
  - Pentru trimiterea și livrarea mesajelor este necesară definirea unei destinații.
  - În funcție de tipul aplicației, destinația poate fi o *coadă* sau un *topic*.
  - Independent de tipul de destinație folosit, destinația trebuie specificată folosind clasa specifică corespunzătoare brokerului de mesaje folosit.

- Declararea unei cozi

```
<bean id="queue" class="org.apache.activemq.command.ActiveMQQueue">
 <constructor-arg value="test.queue"/>
</bean>
```

- Declararea unui topic

```
<bean id="topic" class="org.apache.activemq.command.ActiveMQTopic">
 <constructor-arg value="test.topic"/>
</bean>
```

- Constructorul primește ca și parametru numele cozii de mesaje/topicului.
- Numele va fi folosit de brokerul de mesaje pentru trimiterea/livrarea mesajelor.
- `test.queue/test.topic`.

# Template-uri Spring JMS

- Clasa **JmsTemplate** este soluția oferită de Spring pentru a evita duplicarea codului JMS.
- **JmsTemplate** are ca și responsabilități: crearea conexiunii, obținerea unei sesiuni și trimiterea și livrarea mesajelor.
- Programatorii se ocupă doar de construirea mesajelor ce trebuie trimise, respectiv de procesarea mesajelor primite.
- **JmsTemplate** tratează toate excepțiile de tip **JMSException** (pachetul **javax.jms**) ce pot să apară și le convertește în propriile excepții de tip **JMSException** (pachetul **org.springframework.jms**).
- Dacă o excepție **JMSException** este aruncată pe perioada lucrului cu **JmsTemplate**, obiectul de tip **JmsTemplate** va prinde excepția și o va rearunca sub forma unei excepții specifice Spring (subclasă a excepției proprii **JmsException**).
- **JMSException** (din pachetul **javax.jms**) are o mulțime de subclase care descriu tipul de excepție apărut, dar toate sunt de tip checked.
- **JmsTemplate** prinde excepțiile checked și le rearuncă folosind excepții de tip runtime.

# Medii de proiectare și programare

2024-2025

Curs 13

# Continut

- Asynchronous Messaging (cont.)
- WebSockets

# Template-uri Spring JMS

- Clasa **JmsTemplate** este soluția oferită de Spring pentru a evita duplicarea codului JMS.
- **JmsTemplate** are ca și responsabilități: crearea conexiunii, obținerea unei sesiuni și trimiterea și livrarea mesajelor.
- Programatorii se ocupă doar de construirea mesajelor ce trebuie trimise, respectiv de procesarea mesajelor primite.
- **JmsTemplate** tratează toate excepțiile de tip **JMSException** (pachetul **javax.jms**) ce pot să apară și le convertește în propriile excepții de tip **JMSException** (pachetul **org.springframework.jms**).
- Dacă o excepție **JMSException** este aruncată pe perioada lucrului cu **JmsTemplate**, obiectul de tip **JmsTemplate** va prinde excepția și o va rearunca sub forma unei excepții specifice Spring (subclasă a excepției proprii **JmsException**).
- **JMSException** (din pachetul **javax.jms**) are o mulțime de subclase care descriu tipul de excepție apărut, dar toate sunt de tip checked.
- **JmsTemplate** prinde excepțiile checked și le rearuncă folosind excepții de tip runtime.



# Template-uri Spring JMS

- Pentru a putea folosi obiecte de **JmsTemplate**, acestea trebuie declarate la configurare:

```
<bean id="jmsTemplate"
 class="org.springframework.jms.core.JmsTemplate">

 <constructor-arg ref="connectionFactory" />

</bean>
```

- Se pot seta și alte proprietăți.
- Clasa **JmsTemplate** implementează interfața **JmsOperations**
- Interfața **JmsOperations** specifică operațiile



# Spring JMS - trimiterea unui mesaj

```
public interface AlertService {
 void sendAlert(Alert alert);
}

public class AlertServiceImpl implements AlertService {
 private JmsOperations jmsOperations; //interfata implementata de JmsTemplate

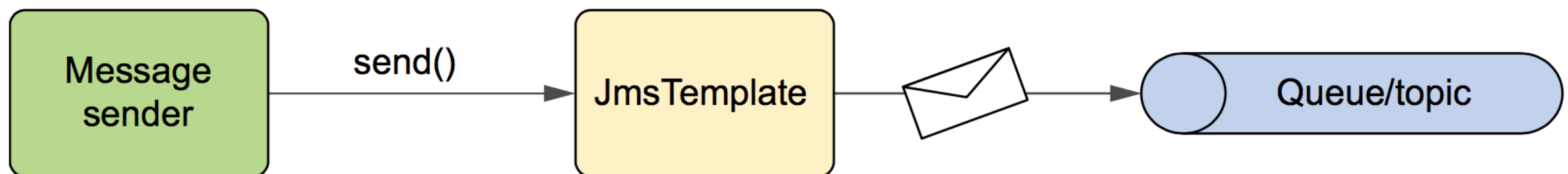
 public AlertServiceImpl(JmsOperations jmsOperatons) {
 this.jmsOperations = jmsOperations;
 }

 public void sendAlert(Alert alert) {
 //trimiterea unui mesaj
 jmsOperations.send("alert.queue", //Specificarea destinatiei
 new MessageCreator() { //Crearea mesajului
 public Message createMessage(Session session)
 throws JMSException {
 return session.createObjectMessage(alert);
 }
 });
 }
}

} //AlertServiceImpl
```

# Spring JMS - trimiterea unui mesaj

- Primul parametru al operației **send()** (interfața **JmsOperations**) specifică numele destinației (coadă sau topic).
- La apelul metodei **send()**, obiectul **JmsTemplate** se ocupă cu obținerea unei conexiuni și a unei sesiuni JMS și cu trimiterea mesajului din partea expeditorului.
- Mesajul este construit folosind un obiect de tip **MessageCreator**
- Metoda **createMessage()** (din clasa **MessageCreator**), cere sesiunii crearea unui mesaj trimițând ca și parametru obiectul **Alert** care păstrează datele conținute de mesaj transmis.
- Obiectul **JmsTemplate** tratează toate complexitățile trimiterii unui mesaj.



# Spring JMS - trimiterea unui mesaj

- Specificarea unei destinații implicite:
  - De obicei, în cadrul unei aplicații mesajele se vor trimite la aceeași destinație.
  - Se poate specifica o destinație implicită în momentul instanțierii unui obiect de tip **JmsTemplate**.
  - Nu mai este necesară specificarea destinației în momentul trimiterii unui mesaj.

```
<bean id="jmsTemplate"
 class="org.springframework.jms.core.JmsTemplate">
 <constructor-arg ref="connectionFactory"/>
 <property name="defaultDestination" ref="queue"/>
</bean>
```

```
//Nu mai e necesara specificarea destinatiei
```

```
jmsOperations.send(
 new MessageCreator() {
 ...
 }
);
```

# Spring JMS - trimiterea unui mesaj

- Convertirea mesajelor la trimitere
  - **JmsTemplate** oferă **convertAndSend()**. Spre deosebire de **send()**, metoda **convertAndSend()** nu primește ca și parametru un obiect de tip **MessageCreator**.
  - Metoda **convertAndSend()** folosește un obiect de tip message converter predefinit pentru a crea mesaje.

```
public void sendAlert(Alert alert) {
 jmsOperations.convertAndSend(alert);
}
```

- **JmsTemplate** folosește o implementare a interfeței **MessageConverter** pentru a converti obiectele în mesaje.
- **MessageConverter** este o interfață definită de Spring având metodele:

```
public interface MessageConverter {
 Message toMessage(Object object, Session session)
 throws JMSException, MessageConversionException;
 Object fromMessage(Message message)
 throws JMSException, MessageConversionException;
}
```

# MessageConverters

- **Implicit JmsTemplate** folosește un **SimpleMessageConverter** pentru trimiterea mesajelor folosind **convertAndSend()**.
- Se poate specifica alt tip de convertor la configurarea obiectului de tip JmsTemplate.

```
<bean id="converter"
 class="org.springframework.jms.support.converter.MappingJackson2MessageConverter">
 <property name="targetType" value="TEXT"/>
 <property name="typeIdPropertyName" value="_alert"/>
</bean>
```

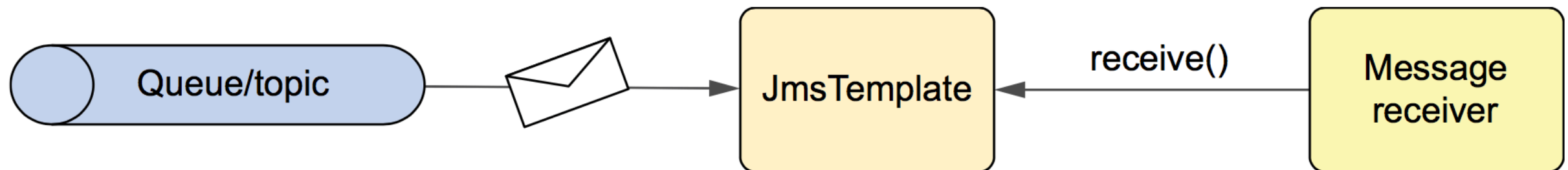
```
<bean id="jmsTemplate" class="org.springframework.jms.core.JmsTemplate">
 <constructor-arg ref="connectionFactory"/>
 <property name="defaultDestination" ref="topic"/>
 <property name="messageConverter" ref="converter"/>
</bean>
```

- **MappingJackson2MessageConverter** (JSON),
- **MarshallingMessageConverter** (JAXB XML format),
- **SimpleMessageConverter** (Strings to/from **TextMessage**, byte arrays to/ from **BytesMessage**, Maps to/from **MapMessage**, și Serializable objects to/from **ObjectMessage**)

# Primirea/Procesarea mesajelor

- Apelul metodei `receive()` din `JmsOperations`.
- La apelul metodei `receive()`, se încearcă obținerea unui mesaj de la brokerul de mesaje. **Dacă nici un mesaj nu este disponibil se așteaptă până când un mesaj este trimis la destinație.**

```
public Alert receiveAlert() {
 try {
 ObjectMessage receivedMessage =(ObjectMessage) jmsOperations.receive() ;
 return (Alert) receivedMessage.getObject() ;
 } catch (JMSException jmsException) {
 throw JmsUtils.convertJmsAccessException(jmsException) ;
 }
}
```



# Primirea/Procesarea mesajelor

- **MessageConverters** pot converti obiecte în mesaje la apelul metodei **convertAndSend()**.
- Pot fi folosiți și la procesarea mesajelor, prin apelul metodei **receiveAndConvert()** (**JmsTemplate**):

```
public Alert retrieveAlert() {
 return (Alert) jmsOperations.receiveAndConvert();
}
```
- Un **dezavantaj** al procesării mesajelor folosind **metodele receive() și receiveAndConvert()** este că ambele metode **sunt sincrone**.
- **Apelantul metodelor trebuie să aștepte până când un mesaj devine disponibil** (sau o condiție de timeout devine adevărată)
- **Message driven beans (EJB)** - pentru procesarea asincronă a mesajelor.



# Exemple

- Exemplu simplu ActiveMQ, producător/consumator Java, producător/consumator C#
- Implementarea notificării în aplicația MiniChat folosind un broker de mesaje.

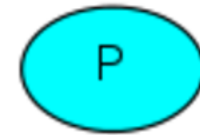


# Advanced Message Queuing Protocol (AMQP)

- Protocol avansat pentru trimiterea mesajelor.
- În JMS există 3 tipuri de participanți:
  - *expeditorul mesajului* (producătorul),
  - *destinatarul* (destinatarii) mesajului (consumatorul/consumatorii),
  - *destinația* (coadă sau topic) folosită pentru livrarea mesajelor între expeditori și destinatari.
- În JMS, destinația decuplează expeditorul de destinatar, dar amândoi sunt cuplați de destinație.
- Destinația are dublă responsabilitate:
  - *primirea mesajelor*
  - determinarea modului în care mesajele vor fi distribuite destinatarilor: coadă/ topic.

# RabbitMQ - Terminologie

**Producător** (eng. *producer*): un program care trimite mesaje.



**Coadă** (eng. *queue*): o zonă tampon mare de mesaje, limitată doar de configurația mașinii pe care rulează RabbitMQ și dimensiunea HDD. Mai mulți producători pot trimite mesaje aceleiași cozi. Mai mulți consumatori pot citi/primi mesaje de la aceeași coadă.

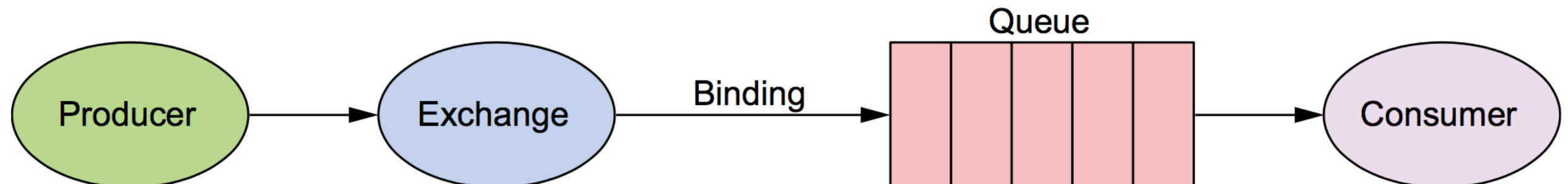


**Consumator** (eng. *consumer*): un program care așteaptă să primească mesaje.



# Advanced Message Queuing Protocol (AMQP)

- Producătorii AMQP nu trimit mesajele direct unei cozi.
- AMQP introduce un nou nivel între producător și cozile care păstrează mesajele: **exchange**.
- Un producător de mesaje trimite mesajul unui exchange.
- Exchange-ul (care are asociate una sau mai multe cozi), **rutează mesajele** cozilor corespunzătoare.
- Consumatorii iau mesajele din cozi și le procesează. Algoritmul de rutare folosit are un impact minor asupra modului de dezvoltare a producătorilor/consumatorilor.
- Producătorii trimit mesajele unui exchange cu o **cheie de rutare** (eng. *routing key*), consumatorii iau mesajele din coada (adesea mesajele nu mai conțin cheia de rutare).
- RabbitMQ este un broker de mesaje bazat pe AMQP.

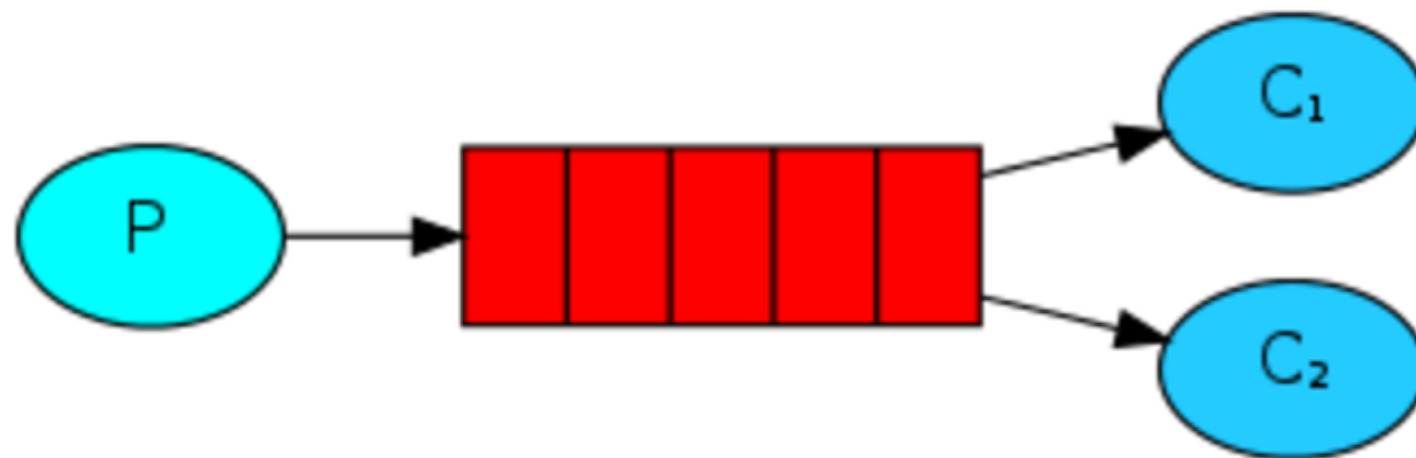


# RabbitMQ - Modele asemănătoare JMS

- Model simplu:

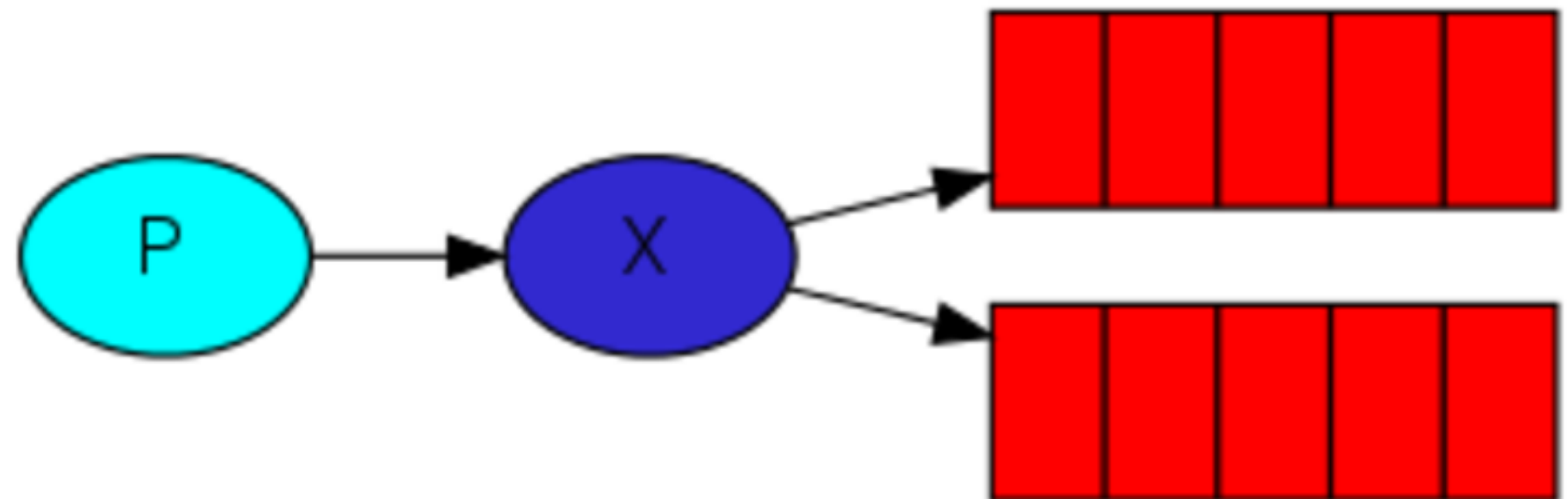


- Cozi de lucru:



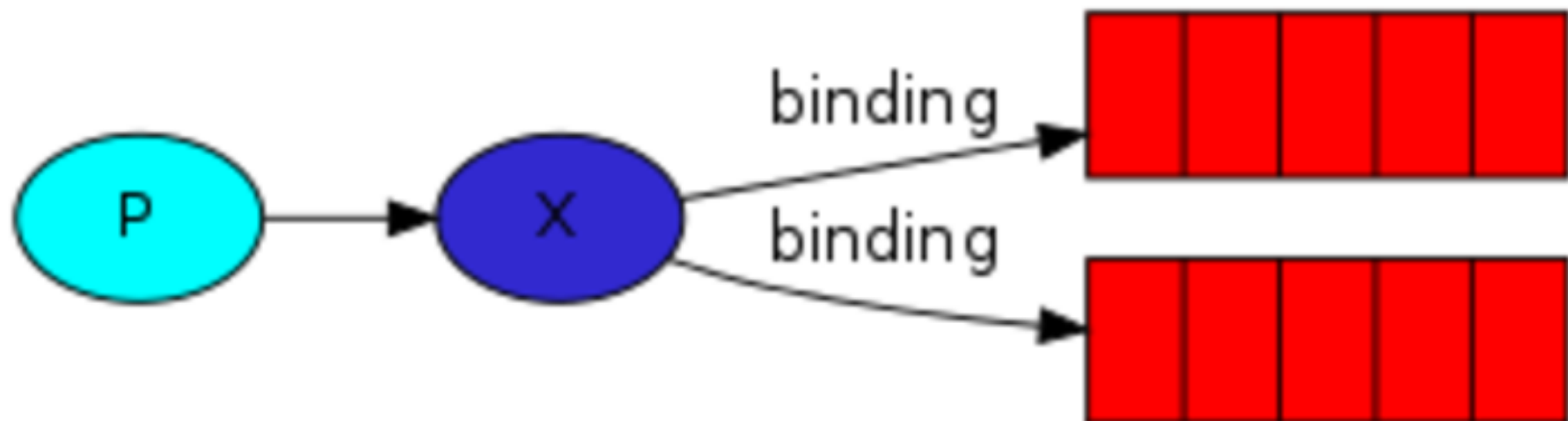
# RabbitMQ - Model de transmitere a mesajelor

- Ideea de bază a modelului RabbitMQ este că producătorul NU trimite mesajul direct unei cozi.
- Producătorul nu știe nici dacă mesajul chiar va fi distribuit unei cozi.
- Producătorul poate doar să trimită mesaje unui *exchange*.
- Un *exchange* primește mesajele de la producători și le trimite spre cozile corespunzătoare.
- Un *exchange* trebuie să știe exact ce să facă cu un mesaj în momentul primirii:
  - Trebuie adăugat la o anumită coadă?
  - Trebuie adăugat la mai multe cozi?
  - Trebuie șters (eng. *discarded*)?
- Regulile folosite pentru luarea deciziei sunt definite de **tipul de exchange**:
  - *direct*,
  - *topic*,
  - *headers*
  - *fanout*.



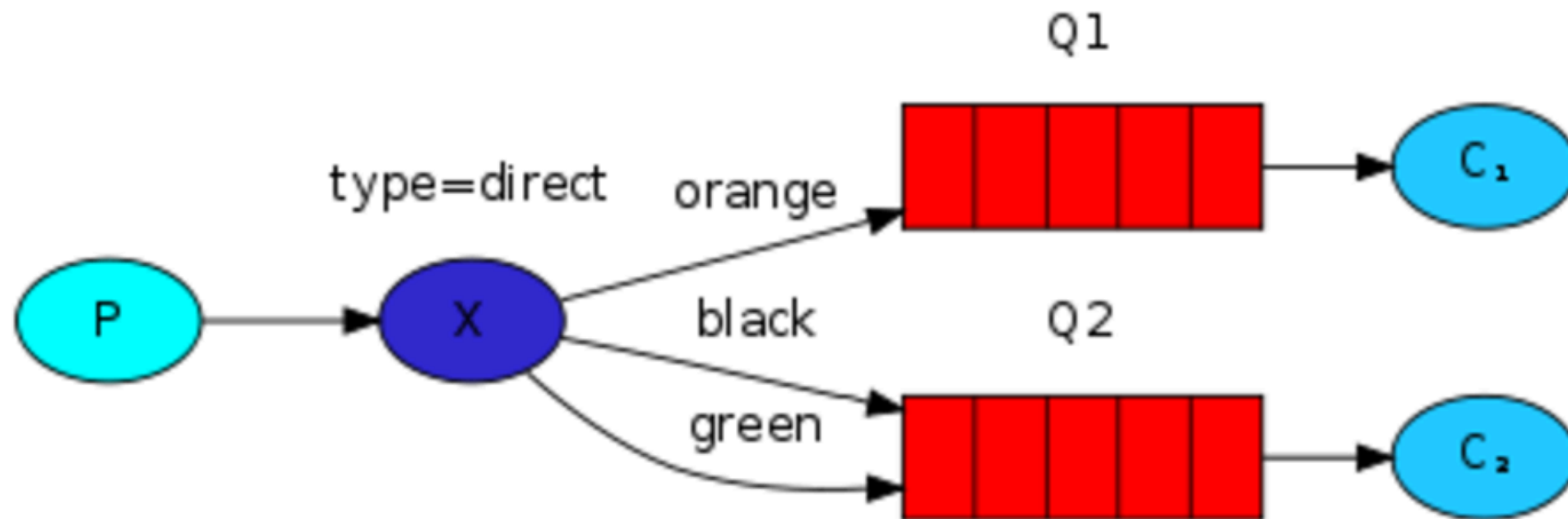
# RabbitMQ - Fanout exchange

- **Fanout exchange** - distribuie mesajele tuturor cozilor cunoscute.
- **Asocieri** (eng. *bindings*): Relația dintre un exchange și o coadă este numită **asociere** (binding). Coada este interesată să primească mesaje de la exchange.
- Asocierile pot avea un extra parametru, numit **cheie de asociere** (eng. *binding key*)
- Semnificația cheii de asociere depinde de tipul de exchange.
- Fanout exchanges ignoră această valoare.



# RabbitMQ - Direct exchange

- **Direct exchange**: - mesajul va fi distribuit cozilor care au asociate ca și chei exact cheia de rutare a mesajului.
- Exemplu:
  - direct exchange X cu două cozi asociate (Q1, Q2). Prima coadă are cheia de asociere *orange*, a doua coadă are asociate cheile: *black* și *green*.
  - Un mesaj cu cheia de rutare *orange* va fi distribuit cozii Q1.
  - Mesajele cu cheile de rutare *black* sau *green* vor fi distribuite cozii Q2.
  - Mesajele cu alte chei de rutare (ex. *white*, *red*, etc.) se vor pierde.





# RabbitMQ - Topic exchange

- Mesajele trimise unui **topic exchange** nu pot avea o cheie de rutare arbitrară.
- Cheia de rutare trebuie să fie o listă de cuvinte separate prin punct. Cuvintele sunt de obicei caracteristici ale mesajelor.
- Exemple valide: "stock.usd.nyse", "nyse.vmw", "quick.orange.rabbit".
- In *cheia de rutare* pot fi oricâte cuvinte, dar lungimea nu trebuie să depășească 255 bytes.
- Cheia folosită pentru specificarea asocierilor are aceeași formă.
- Un mesaj având o anumită cheie de rutare va fi trimis tuturor cozilor asociate cu o cheie de asociere care se potrivește.
- Cazuri speciale pentru cheile folosite la specificarea asocierilor:
  - Caracterul \* (star) poate înlocui un singur cuvânt.
  - Caracterul # (hash) poate înlocui zero sau mai multe cuvinte.
- Topic exchange se poate comporta ca și celelalte tipuri de exchange:
  - Când o coadă are asociată cheia "#" (hash) - va primi toate mesajele indiferent de cheia de rutare folosită (*fanout exchange*).
  - Dacă nu se folosesc caracterele speciale "\*" și "#" la asocierile cozilor, topic exchange se va comporta ca și un *direct exchange*.



# RabbitMQ -Exemplu (1)

- Trimiterea unor mesaje care descriu animale.
- Cheia de rutare este formată din 3 cuvinte: "<viteză>.<culoare>.<animal>".
  - S-au creat 3 asocieri (eng. *bindings*): coada **Q1** este asociată cu cheia de asociere *"\*.orange.\*"*, iar coada **Q2** cu cheile *"\*.\*.rabbit"* și *"lazy.#"*.
- Sumar asocieri:
  - **Q1** este interesată de toate animalele având culoarea *orange*.
  - **Q2** este interesată de toți iepurii și de animale - speciile leneșe.



# RabbitMQ - Exemplu (2)

Cheia de rutare - Mesaj	Coadă
quick.orange.rabbit	Q1, Q2
lazy.orange.elephant	Q1, Q2
quick.orange.fox	Q1
lazy.brown.fox	Q2
lazy.pink.rabbit	Q2 (o dată)
quick.brown.fox	Nici una
orange	Nici una
quick.orange.male.rabbit	Nici una
lazy.orange.male.rabbit	Q2

# RabbitMQ

- **Headers Exchange:**

- Este proiectat pentru rutarea folosind mai multe atribute, care sunt păstrate în antetul (header-ul) mesajului, și nu ca o cheie de rutare.
- Cheia de rutare este ignorată. Atributele folosite pentru rutare sunt luate din antet.
- Un mesaj este o potrivire, dacă valoarea din antet este egală cu valoarea precizată la asociere.
- Este posibil să se facă asocierea dintre o coadă și un exchange folosind mai multe antete.
- În acest caz, broker-ul are nevoie de informații adiționale:
  - *Ar trebui să ia în considerare mesajele care conțin cel puțin un antet sau toate antetele?*
  - *Se poate configura opțiunea folosind argumentul x-match de la binding:*
    - *any* - este de ajuns ca un singur antet să apară
    - *all* - toate antetele trebuie să apară.

# Referințe

- RabbitMQ:

<https://www.rabbitmq.com/getstarted.html>

- Craig Walls, Spring in Action, Fourth Edition, Ed. Manning, 2015

- ActiveMQ:

<http://activemq.apache.org/hello-world.html>

WebSockets

# HTTP

## Limitările HTTP:

- *Stateless*:
  - Browserul deschide o conexiune pe portul 80.
  - Trimite o cerere HTTP serverului web.
  - Aplicația server decide ce va face cu cererea, procesează datele, generează răspuns HTML și îl trimite serverului web.
  - Serverul web adaugă antetele HTTP corespunzătoare răspunsului, îl trimite browserului și închide conexiunea.
- Site-urile web trebuie să păstreze informațiile despre utilizatori (ex. cookies).
  - Informația este transmisă între client și serverul web la fiecare cerere.
  - Informațiile adiționale măresc dimensiunea pachetelor transmise și pot face aplicațiile web vulnerabile (securitatea datelor).
- Comunicarea este întotdeauna inițiată de client și fiecare pereche cerere/răspuns este independentă de celelalte cereri/răspunsuri.

# HTTP

- Aplicații *real-time*: burse de valori, vânzarea biletelor, trafic, citirea aparatelor medicale
- Metode:
  - *Polling*, browserul trimite regulat cereri HTTP și primește răspunsul imediat. Este o soluție bună dacă se cunosc intervalele la care mesajele devin disponibile, deoarece se pot sincroniza cererile clienților cu momentele când informațiile sunt disponibile pe server.
  - *Long-polling*, browserul trimite o cerere la server și serverul păstrează cererea deschisă pentru o perioadă prestabilită de timp. Dacă în acea perioadă se primește o notificare, serverul trimite clientului un răspuns care conține și notificarea. Dacă nu se primește nici o notificare, serverul trimite un răspuns pentru a încheia cererea clientului.
  - *Streaming*, browserul trimite o cerere completă dar serverul trimite și păstrează un răspuns incomplet ce este actualizat continuu și păstrat deschis pe termen nelimitat (sau o perioadă prestabilită de timp). Răspunsul este actualizat de fiecare dată când trebuie trimisă o notificare, dar serverul nu semnalează terminarea răspunsului, păstrând astfel conexiunea deschisă pentru mesaje ulterioare. Pot să apară probleme din cauza păstrării într-un buffer a răspunsurilor.



# WebSockets

- WebSocket-urile sunt o **conexiune persistentă, bi-direcțională, full-duplex** de la un browser web la un server.
- După ce conexiunea a fost stabilită, ea rămâne deschisă până când clientul sau serverul decide să o închidă.
- Cât timp conexiunea este deschisă, clientul și serverul își pot trimite mesaje unul altuia în orice moment de timp.
- Programarea web devine astfel bazată pe eveniment, și nu doar inițiată de utilizator. Este *stateful*.
- O singură aplicație server care rulează știe de toate conexiunile, permițând comunicarea cu orice număr de conexiuni deschise în orice moment de timp.



# Protocolul WebSocket

- În 2011, IETF a standardizat protocolul WebSocket sub denumirea de RFC 6455.
- De atunci, majoritatea browserelor Web implementează cod API pe partea de client care suportă protocolul WebSocket.
- S-au dezvoltat biblioteci în Java, .NET, Ruby, Objective C, JavaScript, etc. care implementează protocolul WebSocket.
- Browsere care suportă WebSockets: nativ în Chrome, Firefox, Opera și Safari (inclusiv Safari pentru dispozitive mobile), Internet Explorer.
- Orice browser care nu suportă WebSockets poate folosi soluția *polyfill* *Flash*.

# Protocolul WebSocket - Handshake

Pentru stabilirea unei conexiuni WebSocket, **clientul trimite o cerere HTTP pentru un *handshake* WebSocket:**



```
GET /echo HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
Origin: http://example.com
```

# Protocolul WebSocket - Handshake

- Dacă serverul acceptă cererea de actualizare a protocolului, va returna un răspuns **HTTP 101 Switching Protocols**:



**HTTP/1.1 101 Switching Protocols**

**Upgrade: websocket**

**Connection: Upgrade**

**Sec-WebSocket-Accept: HSmrc0sMlYUkAGmm5OPpG2HaGWk=**

**Sec-WebSocket-Protocol: echo**

- După ce serverul returnează răspunsul 101, **protocolul la nivel de aplicație se va schimba din HTTP în WebSockets**, care va folosi conexiunea TCP stabilită anterior. Din acest moment, **ambii participanți pot trimite/primi mesaje în orice moment de timp**.



# WebSocket - URI

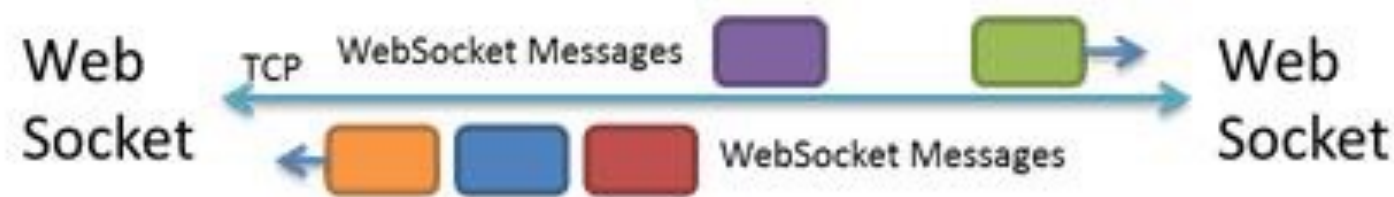
- Protocolul WebSocket definește două formate URI noi, similare cu formatele HTTP:
- `"ws:" "://" host [ ":" port ] path [ "?" query ]` modelată după formatul "http:"
  - Portul implicit este 80.
  - Este folosit pentru conexiuni nesecurizate (necriptate).
- `"wss:" "://" host [ ":" port ] path [ "?" query ]` modelată după formatul "https:"
  - Portul implicit este 443.
  - Este folosit pentru conexiuni securizate peste Transport Layer Security (TLS).

`ws://example.com`

`ws://example.com:8080/echo`

# Mesaje WebSocket

- După o înțelegere (handshake) cu succes, aplicația și serverul web pot interschimba mesaje WebSocket.
- Un mesaj este compus dintr-o secvență de unul sau mai multe fragmente de mesaj/ date numite "*frames*". Fiecare frame conține informații cum ar fi:
  - Dimensiunea/lungimea framelui.
  - Primul frame din mesaj va conține tipul mesajului (text sau binar).
  - Un flag (*FIN*) care indică dacă acesta este ultimul frame din mesaj.



## WebSocket Frames

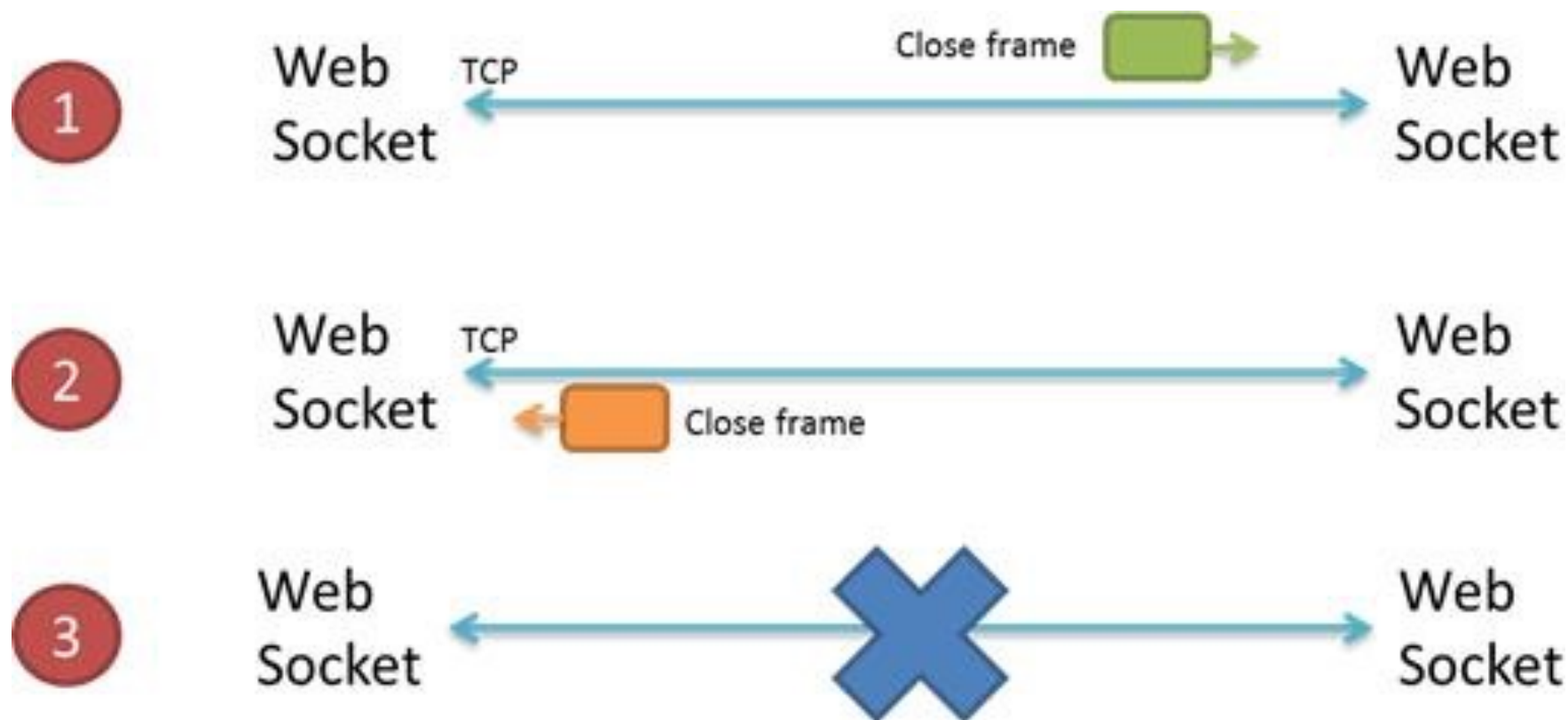
I'm the first binary fragment frame and I'm THIS big

I'm the next fragment frame and I'm THIS big

I'm the final fragment frame and I'm THIS big

# Închiderea unui WebSocket

- Fiecare participant la conexiune (clientul sau serverul) poate iniția o cerere de terminare a înțelegerii (*handshake*).
- Un frame special – un frame *close* – este trimis celuilalt participant. Frame-ul *close* poate să conțină opțional un motiv pentru terminare și un status code.
- Protocolul definește un set de valori ce pot fi folosite pentru status code.
- Cel care trimite frame-ul *close*, nu trebuie să mai trimită alte mesaje după trimiterea acestuia.
- Când celălalt participant primește un frame *close*, el va răspunde cu propriul frame *close*. Poate să trimită mesajele netrimise anterior (din buffer) înainte de a trimite frameul *close*.

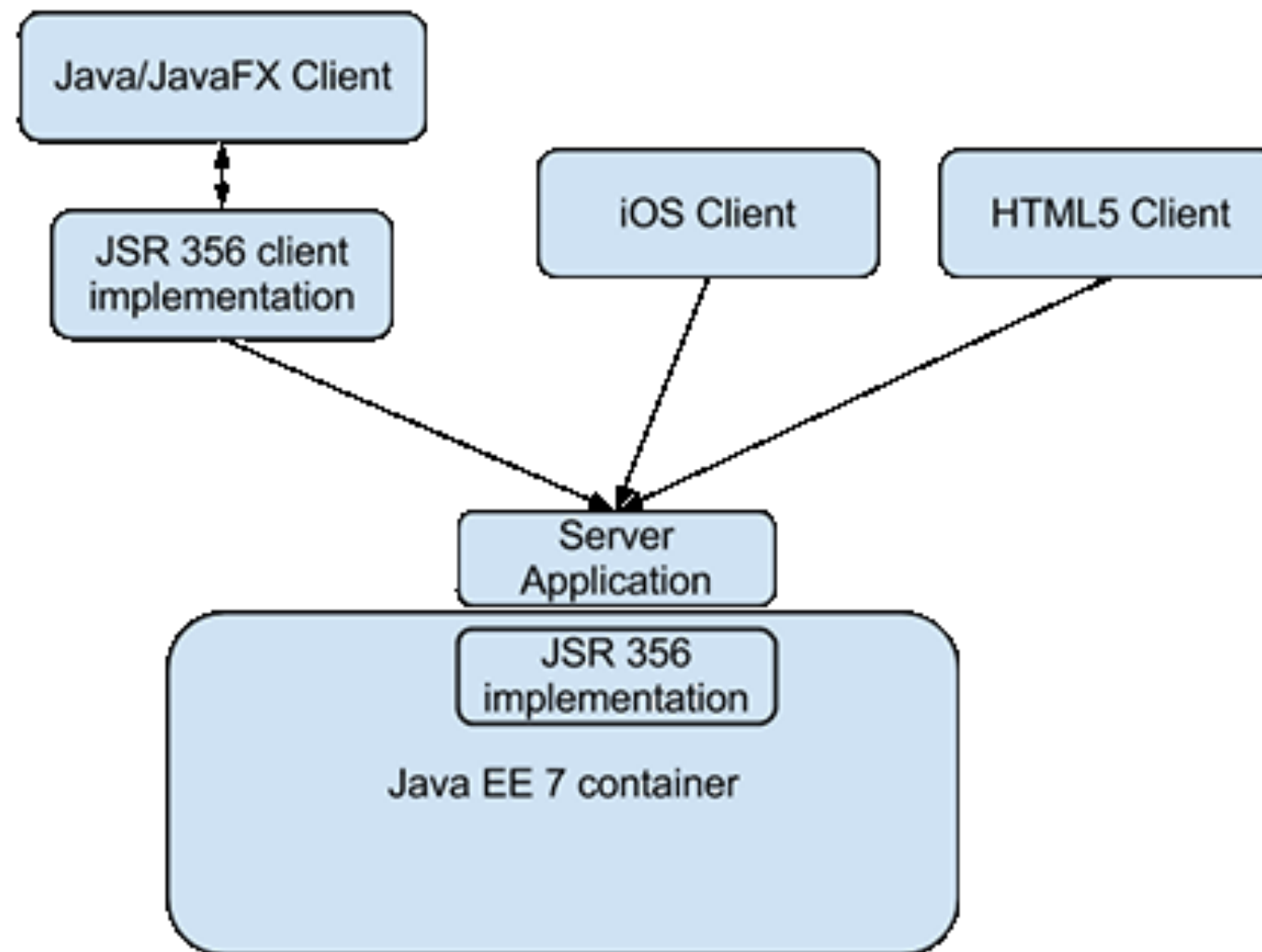


# WebSockets API - Java

- **JSR 356**, Java API for WebSocket, [specifică API-ul care poate fi folosit de dezvoltatorii Java când vor să integreze/folosească WebSocket-uri în aplicațiile lor, atât pe partea de server cât și pe partea de client Java.](#)
- Fiecare implementare a protocolului WebSocket care pretinde să respecte JSR 356 trebuie să implementeze acest API.
- Dezvoltatorii pot dezvolta aplicațiile lor bazate pe WebSocket independent de implementarea WebSocket.
- JSR 356 face parte din standardul Java EE 7.
- Toate serverele de aplicații care respectă Java EE 7 vor avea/conține o implementare a protocolului WebSocket care respectă standardul JSR 356.



# WebSockets API - Java



- Un client Java poate folosi o bibliotecă client care respectă JSR 356 pentru tratarea problemelor (eng. *issues*) specifice protocolului WebSocket.
- Alți clienți (iOS, HTML5) pot folosi alte implementări (care nu sunt bazate pe Java) dar care respectă specificația RFC 6455 pentru a comunica cu aplicația server.



# WebSockets API - Java

- JSR 356 folosește adnotări și injectare.
- Sunt suportate **două modele de programare**:
  - *Bazate pe adnotări*. Se folosesc clase POJO adnotate, cu ajutorul cărora dezvoltatorii pot trata evenimentele ce apar în ciclul de viață al unui WebSocket.
  - *Bazate pe interfețe*. Dezvoltatorii pot trata evenimentele prin implementarea interfeței *Endpoint* care conține metodele corespunzătoare evenimentelor ce pot să apară.
- **Evenimentele din ciclul de viață**:
  - Un client inițiază conexiunea trimițând o cerere de tip HTTP handshake.
  - Serverul răspunde cu un handshake.
  - Conexiunea este stabilită. Din acest moment **conexiunea este simetrică, bi-direcțională**.
  - Ambii participanți (clientul și serverul) **pot să trimită și să primească mesaje**.
  - Unul dintre participanți **închide conexiunea**.
- Majoritatea evenimentelor din ciclul de viață pot fi mapate la metode Java, indiferent de abordarea folosită (adnotări sau interfețe).

# Java WebSockets API - Anotări

- Un endpoint care acceptă cereri WebSocket (serverul) poate fi orice POJO (Plain Old Java Object) annotat cu `@ServerEndpoint`.
- Această anotare specifică containerului (web) că clasa annotată ar trebui considerată un endpoint WebSocket.
- Elementul anotării specifică calea către endpoint-ul WebSocket.

```
@ServerEndpoint("/hello")
```

```
public class MyEndpoint { }
```

```
@ServerEndpoint("/hello/{userid}")
```

```
public class MyEndpoint { }
```

- valoarea variabilei `{userid}` poate fi obținută în metodele corespunzătoare ciclului de viață folosind anotarea `@PathParam`.

# Java WebSockets API - Adnotări

- Un endpoint care inițiază o conexiune WebSocket (clientul) poate fi un POJO adnotat cu `@ClientEndpoint`.
- `ClientEndpoint` nu accepta variabile de cale (ex. {userid}) deoarece nu așteaptă cereri.

`@ClientEndpoint`

```
public class MyClientEndpoint {}
```

- Inițierea unei conexiuni WebSocket, folosind abordarea bazată pe adnotări:

```
javax.websocket.WebSocketContainer container =
javax.websocket.ContainerProvider.getWebSocketContainer();
```

```
container.connectToServer(MyClientEndpoint.class,
new URI("ws://localhost:8080/hello"));
```

- Clasele adnotate cu `@ServerEndpoint` sau `@ClientEndpoint` sunt numite endpoint-uri adnotate.

# Java WebSockets API - Adnotări

- După ce a fost stabilită o conexiune WebSocket se creează un obiect de tip `Session` și metoda adnotată cu `@OnOpen` va fi apelată.
- Metoda poate avea diferiți parametri:
  - Un parametru de tip `javax.websocket.Session` specificând sesiunea (obiectul Session) creată.
  - Un obiect de tip `EndpointConfig` care conține informații despre configurația endpoint-ului.
  - Zero sau mai mulți parametri de tip String adnotați cu `@PathParam`, care referă variabilele de cale din calea endpoint-ului(server).

## `@OnOpen`

```
public void myOnOpen (Session session) {
 System.out.println ("WebSocket opened: "+session.getId());
}
```

# Java WebSockets API - Adnotări

- O instanță de tip **Session** este validă atâta timp cât conexiunea WebSocket nu este închisă.
- Clasa **Session** conține metode care permit dezvoltatorilor să obțină mai multe informații despre conexiune.
- Conține metoda **getUserProperties()** care returnează un dicționar **Map<String, Object>** ce păstrează date specifice aplicației.
- Permite dezvoltatorilor să păstreze informații specifice în obiectul de tip **Session**, informații care ar trebui păstrate între diferite apeluri ale metodei.

# Java WebSockets API - Adnotări

- Când unul dintre **participanți primește un mesaj**, este apelată metoda adnotată cu **@OnMessage**.
- Metoda poate avea **parametrii**:
  - Un parametru de tip **javax.websocket.Session**.
  - Zero sau mai mulți parametri de tip string adnotați cu **@PathParam**, care referă valorile variabilelor de cale (server).
  - Mesajul care poate fi de tip text, binar sau pong.
- Pentru fiecare tip de mesaj este permisă câte o metodă adnotată cu **@OnMessage**. Parametrii permiși pentru specificarea conținutului depind de tipul mesajului primit.

## **@OnMessage**

```
public void myOnMessage (String txt) {
 System.out.println ("WebSocket received message: "+txt);
}
```

# Java WebSockets API - Adnotări

- Dacă metoda adnotată cu `@OnMessage` returnează ceva, valoarea returnată va fi transmisă celuilalt participant la conexiune.

`@OnMessage`

```
public String myOnMessage (String txt) {
 return txt.toUpperCase();
}
```

- Metodă alternativă de a trimite mesaje folosind o conexiune WebSocket:

```
RemoteEndpoint.Basic other = session.getBasicRemote();
other.sendText ("Hello, world");
```

- Metoda `getBasicRemote()` din clasa `Session` returnează o reprezentare a celuilalt participant la conexiunea WebSocket, un `RemoteEndpoint`. Acea instanță `RemoteEndpoint` poate fi folosită pentru trimiterea diferitor tipuri de mesaje (text, binar, pong).
- Obiectul de tip `Session` poate fi obținut/păstrat în metodele corespunzătoare ciclului de viață. (ex. metoda adnotată cu `@OnOpen`)



# Java WebSockets API - Adnotări

- Metoda `@OnClose` este apelată când se închide conexiunea WebSocket.
- Poate avea parametrii:
  - Un obiect de tip `javax.websocket.Session`. Acest parametru nu mai poate fi folosit după ce conexiunea WebSocket este închisă (imediat după terminarea execuției metodei).
  - Un obiect de tip `javax.websocket.CloseReason` care descrie motivul închiderii conexiunii WebSocket (ex. închidere normală, eroare de protocol, serviciu supraîncărcat, etc.).
  - Zero sau mai mulți parametri de tip String adnotați cu `@PathParam`, referind variabilele de cale din URL asociat endpoint-ului (server).

## `@OnClose`

```
public void myOnClose (CloseReason reason) {
 System.out.println ("Closing a WebSocket due to
 "+reason.getReasonPhrase());
}
```

- În cazul apariției unei erori, va fi apelată metoda adnotată cu `@OnError`.



# Java WebSockets API - Mesaje

- Orice obiect Java poate fi transmis sau primit ca și mesaj WebSocket.
- Trei tipuri diferite de mesaje:
  - *Text*
  - *Binare*
  - *Pong* (specifice conexiunii WebSocket).
- Pentru mesaje de tip text sunt permisi următorii parametrii:
  - **String** care reprezintă tot mesajul
  - Un tip primitiv Java sau clasa asociată care reprezintă mesajul convertit la acel tip
  - **String** și o valoare booleană care reprezintă o parte din mesaj (mesajul poate avea mai multe părți)
  - **Reader** pentru a primi întregul mesaj ca și un stream (blochează execuția până la terminarea citirii de pe stream)
  - orice obiect Java pentru care participantul la conexiune are un *text decoder* (**Decoder.Text** sau **Decoder.TextStream**).

# Java WebSockets API - Mesaje

- Parametrii permisi pentru **mesajele binare**:
  - **byte[]** sau **ByteBuffer** pentru a primi tot mesajul
  - perechea (**byte[]**, **boolean**) sau (**ByteBuffer**, **boolean**) pentru a primi mesajul în părți.
  - **InputStream** pentru a citi tot mesajul dintr-un stream (*blocking*).
  - orice obiect pentru care participantul are un *binary decoder* (**Decoder.Binary** sau **Decoder.BinaryStream**).
- Parametrii permisi pentru **mesaje pong**:
  - **PongMessage** pentru tratarea mesajelor de tip pong.
- Orice obiect Java poate fi convertit într-un mesaj de tip text sau binar folosind un convertor (eng. *encoder*).
- Mesajul în format text/binar va fi transmis celuilalt participant, unde va fi reconvertit (eng. *decoded*) într-un obiect Java.
- Pentru transmiterea mesajelor WebSocket se folosește adesea formatul XML sau JSON. În acest caz convertirea/reconvertirea se reduce la transformarea unui obiect Java într-un XML/JSON și invers.

# Java WebSockets API - Mesaje

- Un convertor este o clasă care implementează interfața `javax.websocket.Encoder`, iar un reconvertor este o clasă care implementează interfața `javax.websocket.Decoder`.
- Participanții la conexiunea WebSocket trebuie să știe posibili convertori/reconvertori.
- Lista convertorilor/reconvertorilor poate fi transmisă ca și elemente ale adnotărilor `@ClientEndpoint` și `@ServerEndpoint`.

```
@ServerEndpoint(value="/endpoint", encoders = MessageEncoder.class,
decoders= MessageDecoder.class)
public class MyEndpoint {
 ...
}
```

# Java WebSockets API - Messages

```
class MessageEncoder implements Encoder.Text<MyJavaObject> {
 @Override
 public String encode(MyJavaObject obj) throws EncodingException {
 ...
 }
}
```

```
class MessageDecoder implements Decoder.Text<MyJavaObject> {
 @Override
 public MyJavaObject decode (String src) throws DecodeException {
 ...
 }

 @Override
 public boolean willDecode (String src) {
 // return true if we want to decode this String into
 // a MyJavaObject instance
 }
}
```

# Java WebSockets API - Messages

- Interfață **Encoder** are următoarele subinterfețe:
  - **Encoder.Text** - convertirea obiectelor Java în mesaje de tip text
  - **Encoder.TextStream** - adăugarea obiectelor Java la un *character stream*
  - **Encoder.Binary** - convertirea obiectelor Java în mesaje binare
  - **Encoder.BinaryStream** - adăugarea obiectelor Java la un *binary stream*
- Interfață **Decoder** are 4 subinterfețe:
  - **Decoder.Text** - convertirea mesajelor text într-un obiect Java
  - **Decoder.TextStream** - citirea unui obiect Java dintr-un *character stream*
  - **Decoder.Binary** - convertirea unei mesaj binar într-un obiect Java
  - **Decoder.BinaryStream** - citirea unui obiect Java dintr-un *binary stream*

# Abordarea bazată pe interfețe

- Clasa `javax.websocket.Endpoint`:

- se redefinesc metodele `onOpen`, `onClose`, și `onError`:

```
public class MyOwnEndpoint extends javax.websocket.Endpoint {
 public void onOpen(Session session, EndpointConfig config) {...}
 public void onClose(Session session, CloseReason closeReason) {...}
 public void onError (Session session, Throwable throwable) {...}
}
```

- Pentru interceptarea mesajelor trebuie înregistrat un obiect de tip `javax.websocket.MessageHandler` în definirea metodei `onOpen`:

```
public void onOpen (Session session, EndpointConfig config) {
 session.addMessageHandler (new MessageHandler() {...});
}
```

# Abordarea bazată pe interfețe

- Interfața **MessageHandler** are două subinterfețe:
  - **MessageHandler.Partial** - apariția unui mesaj parțial.
  - **MessageHandler.Whole** - apariția unui mesaj complet.

```
public void onOpen (Session session, EndpointConfig config) {
 final RemoteEndpoint.Basic remote = session.getBasicRemote();
 session.addMessageHandler (new MessageHandler.Whole<String>() {
 public void onMessage(String text) {
 try {
 remote.sendString(text.toUpperCase());
 } catch (IOException ioe) {
 // handle send failure here
 }
 }
 });
}
```



# Spring WebSocket API

- Interfața **WebSocketHandler** pentru procesarea mesajelor și gestiunea evenimentelor din ciclul de viață
  - **TextWebSocketHandler** - procesarea mesajelor text
  - **BinaryWebSocketHandler** - procesarea mesajelor binare
- Metode:
  - **afterConnectionEstablished(WebSocketSession session)**
  - **handleMessage(WebSocketSession session, WebSocketMessage<?> message)**
  - **supportsPartialMessages()**
  - **afterConnectionClosed(WebSocketSession session, CloseStatus closeStatus)**
  - **handleBinaryMessage(WebSocketSession session, BinaryMessage message)**
  - **handleTextMessage(WebSocketSession session, TextMessage message)**
  - **handlePongMessage(WebSocketSession session, PongMessage message)**



# Spring WebSocket API

```
public class ChatWebsocketHandler extends TextWebSocketHandler {
 private static List<WebSocketSession> sessions = new CopyOnWriteArrayList<>();
 @Override
 public void handleTextMessage(WebSocketSession session, TextMessage message) {
 System.out.println("[Websocket] New TextMessage: " + message.getPayload());
 }
 @Override
 public void afterConnectionEstablished(WebSocketSession session) {
 sessions.add(session);
 }
 @Override
 public void afterConnectionClosed(WebSocketSession session, CloseStatus status) {
 sessions.remove(session);
 }
 public void sendUsersAll(User[] users) {
 ObjectMapper objectMapper = new ObjectMapper();
 try{
 String result=objectMapper.writeValueAsString(users);
 for(WebSocketSession websocketSession : sessions) {
 try {
 websocketSession.sendMessage(new TextMessage(result));
 }catch (IOException e){
 System.err.println("error sending message to websocket"+e);
 }
 }
 } catch(JsonProcessingException e){
 System.err.println("error writing object"+e);
 }
 }
}
```

# Spring WebSocket API

- Interfața **WebSocketConfigurer** pentru configurarea WebSocket

**@Configuration**

**@EnableWebSocket**

```
public class WebSocketConfiguration implements WebSocketConfigurer {
```

```
 @Bean
```

```
 public ChatWebsocketHandler createChatWebsocketHandler() {
```

```
 return new ChatWebsocketHandler();
```

```
 }
```

```
public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
```

```
 registry.addHandler(createChatWebsocketHandler(), "/chatws");
```

```
 //.setAllowedOrigins("*");
```

```
}
```

```
}
```

# STOMP + WebSocket

- STOMP (Simple Text Oriented Messaging Protocol) - protocol bazat pe frame-uri pentru trimiterea mesajelor

**SEND**

**destination:** /queue/trade

**content-type:** application/json

**content-length:** 44

**{"action": "BUY", "ticker": "MMM", "shares", 44}^@**

- Se pot folosi brokeri de mesaje (ActiveMQ, RabbitMQ)
- Interfața **WebSocketMessageBrokerConfigurer** pentru configurare
  - **registerStompEndpoints(...)**
  - **configureMessageBroker(...)**

<https://stomp.github.io/>

<https://docs.spring.io/spring-framework/reference/web/websocket/stomp.html>

# WebSockets

- Exemple:
  - Broadcast
  - Whiteboard
  - Chat users
- Referințe
  - RFC 6455: <https://tools.ietf.org/html/rfc6455>
  - Johan Vos, *JSR 356, Java API for WebSocket*, <http://www.oracle.com/technetwork/articles/java/jsr356-1937161.html>
  - Brian Raymor, *WebSockets: Stable and Ready for Developers*, <https://msdn.microsoft.com/en-us/aa969243.aspx>
  - Spring WebSockets: <https://docs.spring.io/spring-boot/reference/messaging/websockets.html>

# Medii de proiectare și programare

2024-2025

Curs 14

# Conținut

- Autentificare REST
  - JWT
- Examen

# Autentificare REST

- *Autentificarea* (eng. *Authentication*): verificarea credențialelor trimise de client (utilizator, parolă)
- *Autorizarea* (eng. *Authorization*): verificarea dreptului de a accesa anumite resurse (deja autentificat)
- *Antetul HTTP Authorization*
- Constrângerea REST *stateless*: la fiecare cerere HTTP trebuie transmise informațiile necesare
- Abordări:
  - HTTP Authentication (RFC 2617)
    - Basic Authentication
    - Digest Authentication
  - OAuth 2.0



# Autentificare REST

- **HTTP Basic Authentication**

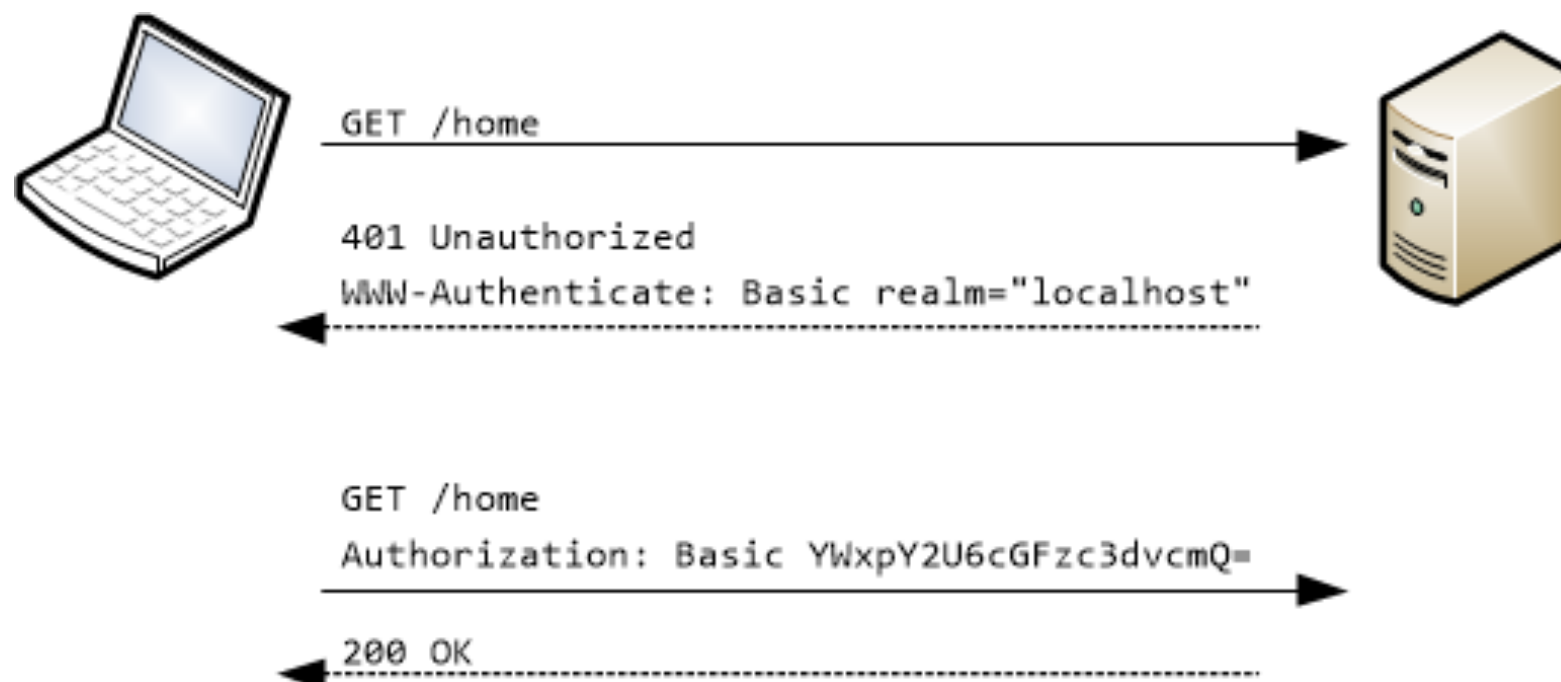
- Folosirea antetului HTTP Authorization în care se transmit username-ul și parola codificate în base64 la fiecare cerere:

**GET / HTTP/1.1**

**Host: example.org**

**Authorization: Basic Zm9vOmJhcg==**

- Credențialele sunt codificate, dar **nu sunt criptate!**
- Informațiile pot fi obținute ușor.
- Se recomandă folosirea doar cu SSL/TLS





# Autentificare REST

- **HTTP Digest Authentication**

- Folosirea antetului HTTP Authorization în care se transmit username-ul, parola și alte informații codificate.
- HMAC (hash based message authentication)
- Se transmite o versiune criptată a parolei
- Se codifică și alte informații despre resursa dorită

Exemplu: utilizator “ion”, parola “ionpass”,

**GET /users/ion/account**

```
digest = base64encode(hmac("sha256", "ionpass",
 "GET+/users/ion/account"))
```

**GET /users/ion/account HTTP/1.1**

**Host: example.org**

**Authorization: hmac ion:[digest]**

- Parola nu poate fi criptată pe server, pentru a putea reconstrui digest
- Se recomandă folosirea unui “secret” cunoscut de client și server

# Autentificare REST

- **HTTP Digest Authentication**

- **Avantaj:** un hacker nu poate modifica cererea (ar trebui să modifice valoarea *digest* și nu cunoaște “secret”-ul)
- **Dezavantaj:** un hacker poate executa cererea de oricâte ori
  - **Soluție:** se transmit mai multe informații în *digest*:
    - Data curentă
    - **Nonce** (*N*umber used *once*). La cereri subsecvente valoarea nonce este diferită.

```
digest = base64encode(hmac("sha256", "secret",
 "GET+/users/ion/account+30may201912:59:24+123456"))
```

```
GET /users/ion/account HTTP/1.1
```

```
Host: example.org
```

```
Authorization: hmac ion:123456:[digest]
```

```
Date: 30 may 2019 12:59:24
```

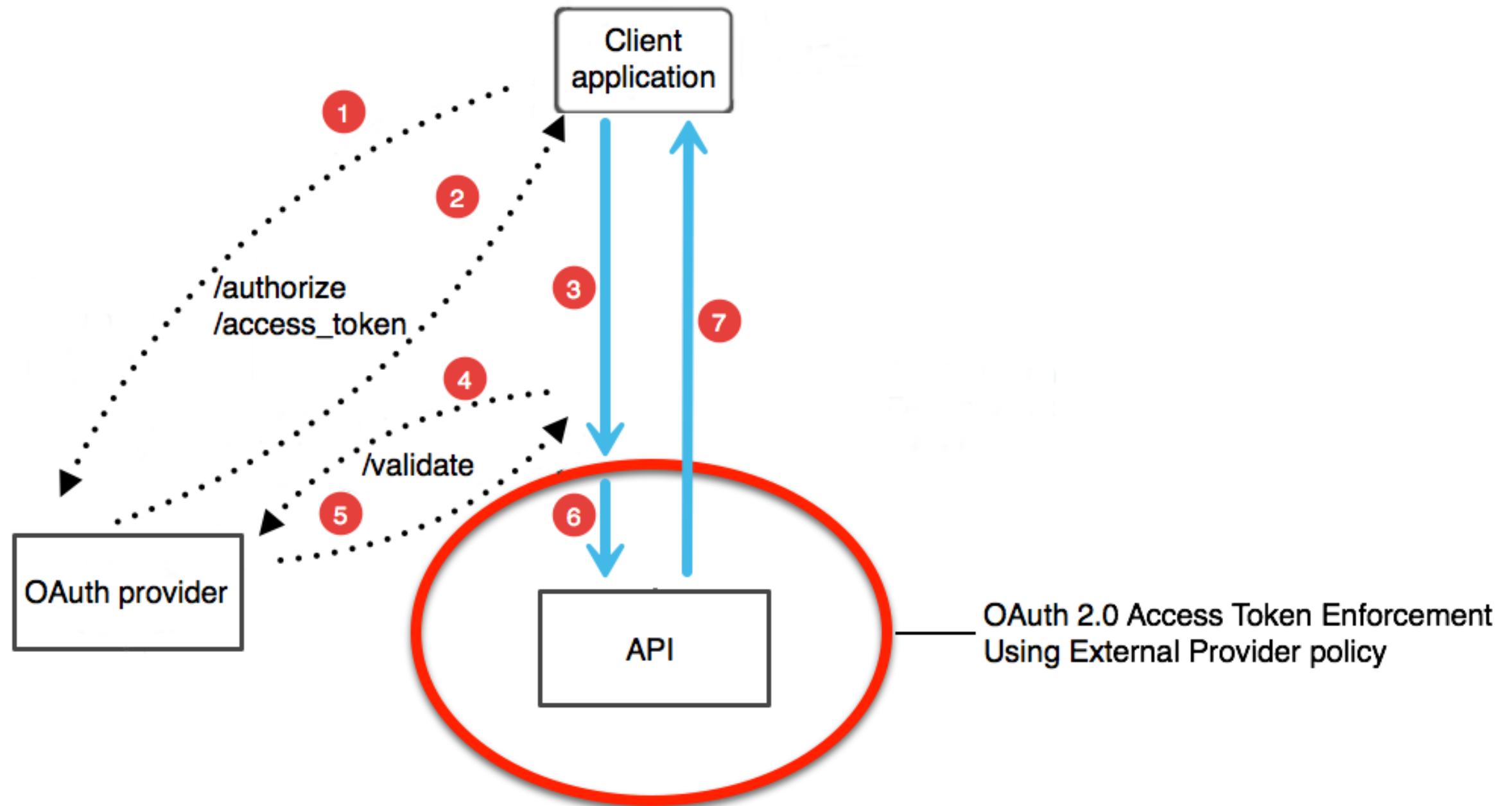
# Autentificare REST

- **OAuth**

- OAuth 1.0 decembrie 2007
  - Sigur, dar dificil de implementat (criptografie, interoperabilitate)
- OAuth 2.0 octombrie 2012
  - Mai ușor de implementat
  - Compromisuri la nivel de securitate
  - Cel mai folosit
- **Flux:**
  - Aplicația client se înregistrează la un **furnizor**
  - Furnizorul transmite clientului un “secret” unic
  - Clientul include “secret”-ul la fiecare cerere
  - Dacă cererea nu este formată corect, are date lipsă sau nu conține secretul corect, ea va fi respinsă.

# Autenticare REST - OAuth

## OAuth Dance



# JSON Web Token (JWT)

- **JSON Web Token** este un standard (RFC 7519) care definește un mod *compact* și *independent* de a transmite informații între participanți ca și un object JSON.
  - *Compact*: are dimensiuni reduse.
    - poate fi transmis prin URL, parametrii POST, antet HTTP
    - este transmis rapid
  - *Independent*: un token JWT conține toată informația necesară despre o entitate pentru a evita interogări multiple ale bazei de date.
    - cine primește tokenul nu trebuie să facă alte cereri la server pentru a-l valida.
- Informația conținută într-un token JWT poate fi verificată și se poate avea încredere în ea pentru că este **semnată digital**.
- JWT poate fi semnat folosind:
  - un *secret* (folosind HMAC)
  - cheie publică/privată folosind RSA, ECDSA, etc

# JWT

- Aplicabilitate JWT
  - Autentificare/Autorizare
  - Schimb de informații între participanți (client/server)
    - Securizat
- **Structura unui token JWT**
  - 3 părți separate prin '.': **aaa**.**bbb**.**ccc**
  - Header (**aaa**)
  - Payload (**bbb**)
  - Semnătura digitală (**ccc**)

# Header JWT

- Conține informații despre tipul de token și algoritmi de criptare folosiți pentru conținut.
- Are **două părți**:
  - tipul tokenului (JWT)
  - algoritmul de criptare (ex. HMAC SHA256, RSA)

```
{
 "alg": "HS256",
 "typ": "JWT"
}
```



# Payload JWT

- Conține informații despre entitate ce pot fi verificate (identitatea utilizatorului, permisiunile, etc), eng. *claims*
- 7 *claims* rezervate și recomandate (dar nu obligatorii):
  - *iss* (issuer), *sub* (subject) -utilizatorul, *aud* (audience) - pentru cine a fost creat, *exp* (expiration time), *nbf* (not before time), *iat* (issued at time), *jti* (JWT ID) -identificator unic, poate fi folosit o singură dată

```
{
 "sub": "1234567890",
 "name": "John Doe",
 "admin": true
}
```

- Poate conține și alte informații adiționale (eng. *custom claims*). Poate fi folosit orice nume, care nu este rezervat prin specificație.



# Semnătura digitală JWT

- Este folosită pentru a verifica expeditorul și pentru a verifica dacă mesajul a fost modificat pe parcurs.
- Se codifică Base64 header-ul și payload-ul și se criptează folosind algoritmul specificat în header (împreună cu secretul/cheie publică-privată)

**HMACSHA256 (**

**base64UrlEncode(header) + "." +**

**base64UrlEncode(payload) ,**

**secret)**

- **Înainte de folosirea unui token JWT trebuie verificată semnătura lui.**

# JWT.io

ALGORITHM

HS256



## Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ikp1hcm1uZXNjdSBWYXNpbGUiLCJpYXQiOiE1MTYyMzkwMjJ9.J3H8MqVValMLWo1ycEOUBNax3glXeURR2XwU5Z9F9Zc
```

## Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{ "alg": "HS256", "typ": "JWT"}
```

PAYLOAD: DATA

```
{ "sub": "1234567890", "name": "Marinescu Vasile", "iat": 1516239022}
```

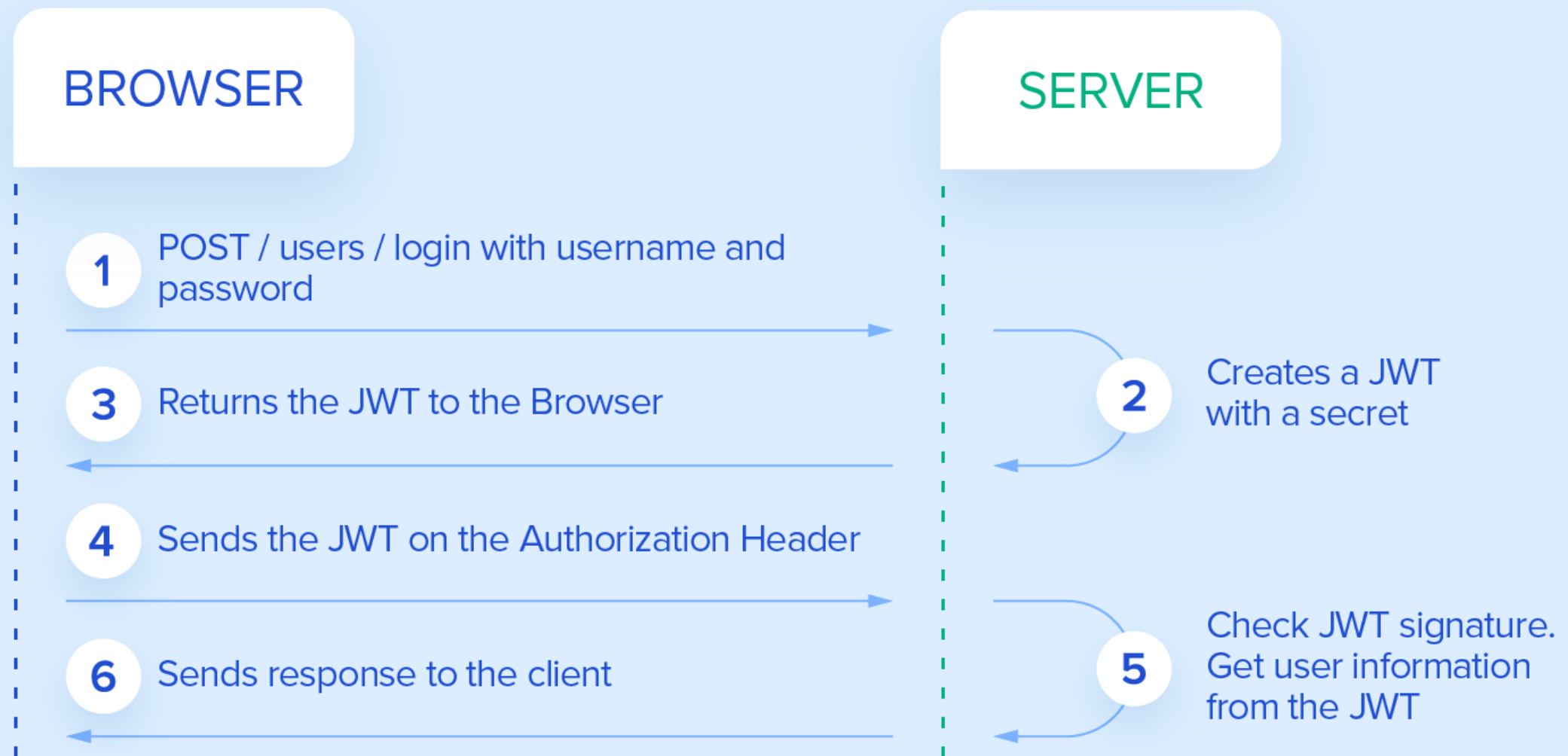
VERIFY SIGNATURE

```
HMACSHA256(
 base64UrlEncode(header) + "." +
 base64UrlEncode(payload),
 secretul meu
) ☒ secret base64 encoded
```

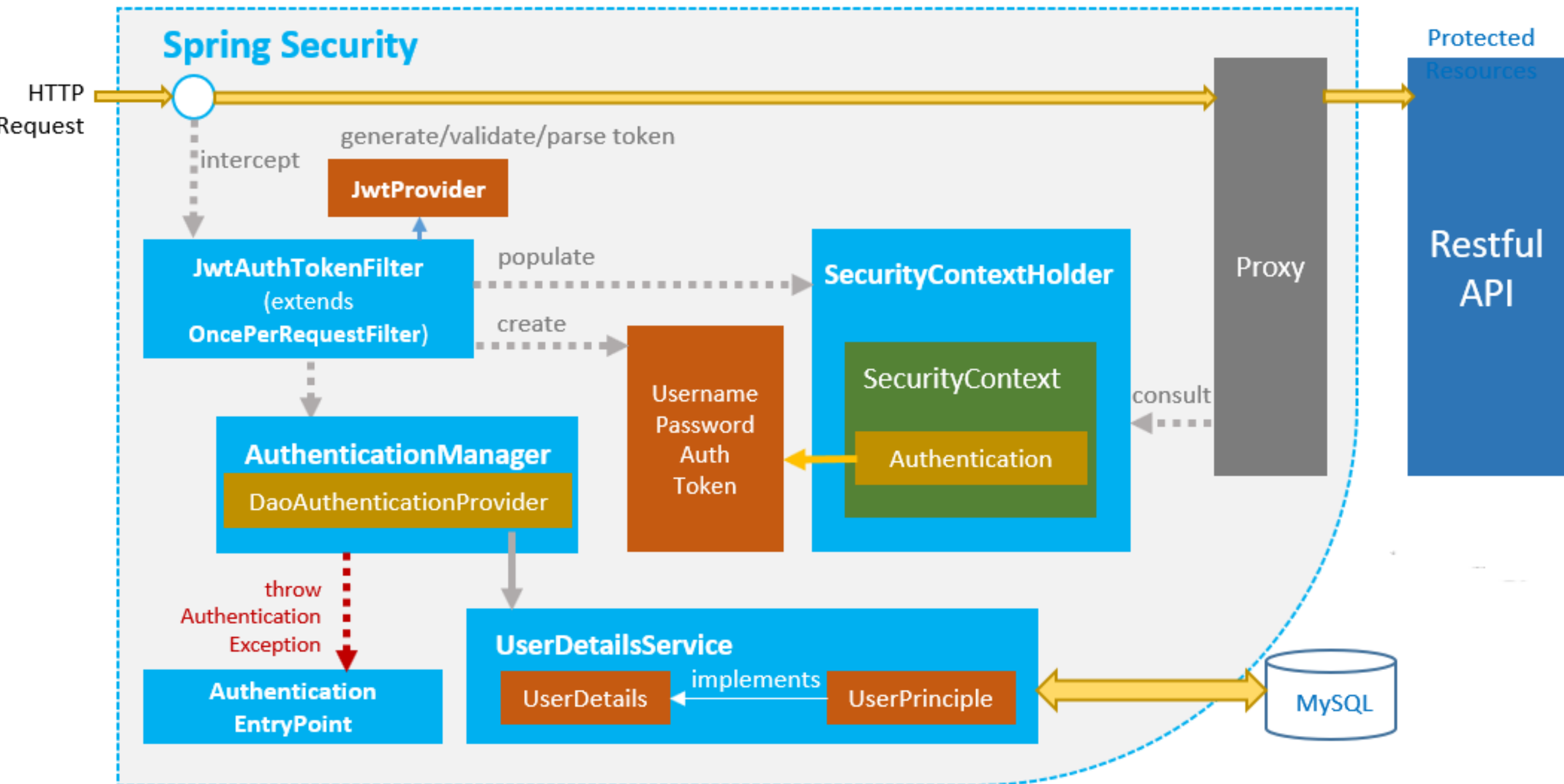
✔ Signature Verified

SHARE JWT

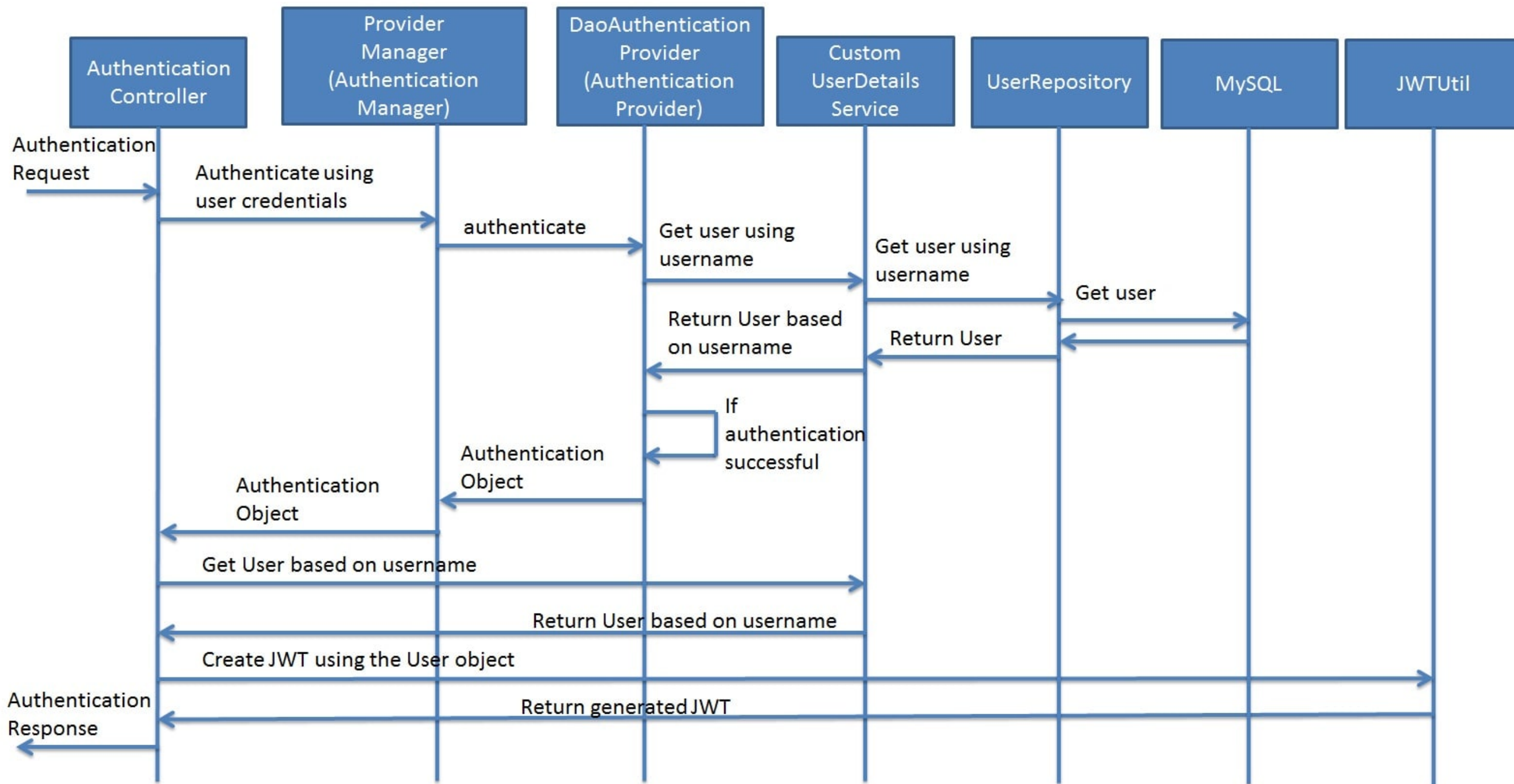
# JWT



# Spring Security +JWT



# Spring Security +JWT



\*<https://www.cloudtechtwitter.com/2023/09/demystifying-jwt-based-authentication.html>

# Spring Security +JWT

Exemplu:

API public:

**POST** `api/login`

**GET** `api/messages/test`

API necesită autentificare

**GET** `api/messages?to={username}`

Testare folosind client REST (Postman)

Client web

# Referințe

- RFC 7235 - Access Authentication Framework  
<https://tools.ietf.org/html/rfc7235#section-2>
- RFC 2617 - HTTP Authentication: Basic and Digest Access Authentication  
<https://tools.ietf.org/html/rfc2617>
- RFC 6749 - OAuth2 standard  
<https://tools.ietf.org/html/rfc6749>
- Guy Levin, *RESTful API Authentication Basics*,  
<https://blog.restcase.com/restful-api-authentication-basics/>
- Tutoriale web:
  - <https://www.bezkoder.com/spring-boot-jwt-authentication/>
  - <https://www.javaguides.net/2024/05/spring-boot-react-jwt-authentication.html>
  - ...



# Examen

- **Calcul media finală:**
  - **10%** Teme lab (4 teme) - TL
  - **30%** Media lab - Lab (Lab  $\geq 4.5$  - examen in sesiunea normală)
  - **60%** Nota examen, (NE  $\geq 4.5$ )

**Media finală =  $0.1 * TL + 0.3 * Lab + 0.6 * NE$**

**Promovat: Media finală  $\geq 4.5$ !**



# Examen - Structura

- 2 întrebări teoretice - 15 min - 2p
- Dezvoltarea unei aplicații client server - 2 h - 8p
  - Puteți reutiliza codul sursă (teme laborator, alte exemple, etc).
  - Puteți folosi resurse de pe Internet (pentru erori, alte probleme).
  - **NU AVEȚI VOIE SĂ DISCUȚAȚI CU ALTE PERSOANE** (viu grai, chat, telefon, email, social media, etc)
  - **NU AVEȚI VOIE SĂ FOLOSITI INSTRUMENTE AI** (ChatGPT, Gemini, Copilot, etc)
  - Denumirile interfețelor, claselor, atributelor, etc. trebuie modificate conform enunțului problemei!
  - Contează arhitectura și proiectarea (definire interfețe și folosirea lor)!
  - Jurnalizare, fișiere de configurare, conectare la baza de date, ORM
  - 2 servicii REST (respectarea regulilor de formare a URL-ului, a informațiilor ce trebuie returnate)
  - 1 client web pentru 1 serviciu REST
  - Observer distribuit (notificare)
  - Pentru a fi evaluată, soluția dezvoltată trebuie să compileze și să ruleze cel puțin o cerință!!!
  - Se punctează doar funcționalitățile care se execută corect!
  - **NU orice soluție va fi evaluată cu punctaj maxim** (nu respectă conceptele, principiile și recomandările prezentate la curs)!

Exemplu problemă